Database Management Systems

UE23CS351A -Introduction

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgment: Dr. Nagasundari S (Dept of CSE), Teaching Assistants

Course Objectives:

Course Objectives:

- To introduce fundamental concepts of database systems, relational algebra, and basic SQL querying.
- To develop the ability to use advanced SQL techniques and understand the strategies for efficient query processing and optimization.
- To equip students with normalization principles, functional dependencies, and transaction management to ensure data integrity.
- To understand the modern data management systems including NoSQL - graph and vector databases.

https://medium.com/@ilyessedka/why-the-creator-of-pandas-says-data-scientists-must-learn-sql-840243480d81

Database Management Systems

Course Outline

Unit 1: Introduction to Database Management

Purpose, View of data, Database design- Introduction to databases, Database application architecture, E-R Model, reducing ER to a relational schema. Relational operations (Algebra), Unary Operations - Unity, Binary, Aggregate Functions, Grouping, SQL overview, Data definition, Structure of SQL queries, constraints and keys.

14 Hours

Unit 2: Relational Model and Database Design

Additional Basic Operations, Set Operations, Null Values, Aggregate Functions, Nested Subqueries, Database Modification, Join expressions, Views, Triggers, Functions, and Procedures, CTE, windows functions and full text search functions.

14 Hours

Unit 3: Advanced Design Concepts and Implementation

Introduction to strategies of Query processing and Query optimization, Functional Dependencies, Inference Rules, Normal Forms Based on Primary Keys (1NF, 2NF, and 3NF), General Definitions of Second and Third Normal Forms Boyce-Codd Normal Form, Higher Normal Forms.

14 Hours

Unit 4: Advanced Databases

Database transactions, Concurrency control, Locking, Introduction to NoSQL databases, Key-value database, Graph databases (Neo4j), Introduction to Vector databases, Vector search with Relational and NOSQL databases, Database Application Connectivity.

14 Hours

Lab/Hands-on sessions

- Draw an ER diagram for a given problem statement and Conversion of an ER diagram into Relational schema
- DDL and DML
- SQL – Joins: inner, outer; Sub queries: correlated and uncorrelated
- SQL – Creating Functions and Procedures
- SQL – Creating Triggers, Transaction isolation
- NoSQL database queries
- Vector database
- Designing Database application

Tools/ Languages: MySQL Workbench, Python, Redis, Neo4J, ERwin or any draw tool for ER modeling

Course Outcomes

At the end of this course, the student will be able to:

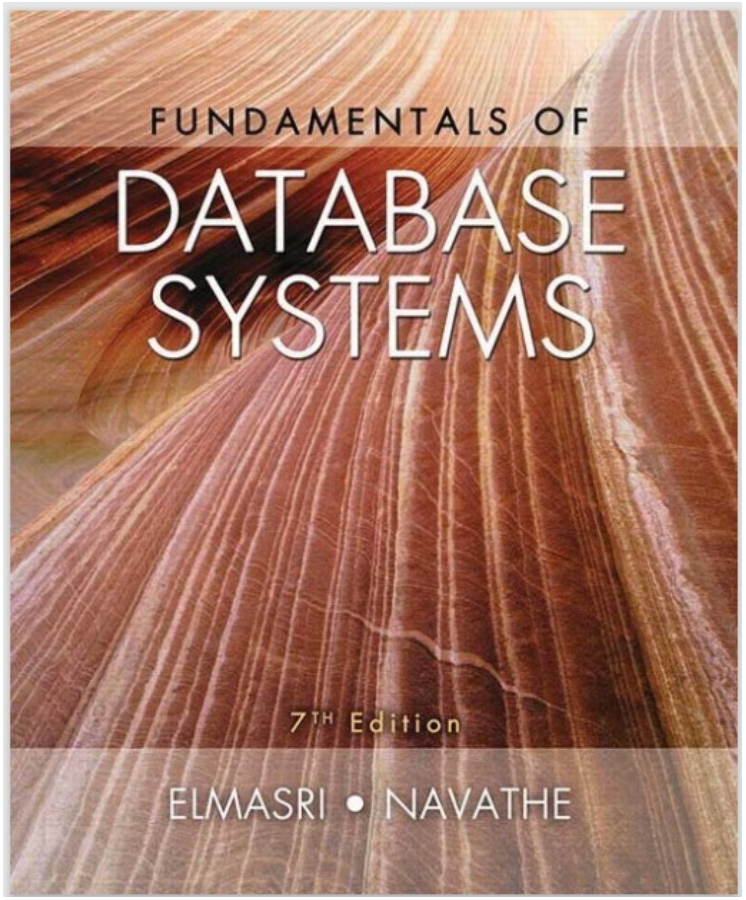
- Demonstrate **an ability to explain the structure of a database system**, ER diagrams, relational algebra, and write basic SQL queries involving constraints and keys
- **Construct and optimize SQL queries** including nested subqueries, joins, views, triggers, and stored procedures
- **Apply normalization techniques** using functional dependencies and design schemas, and ensure the implementation of transaction and concurrency control mechanisms.
- **Create applications of graph and vector databases**.

Database Management Systems

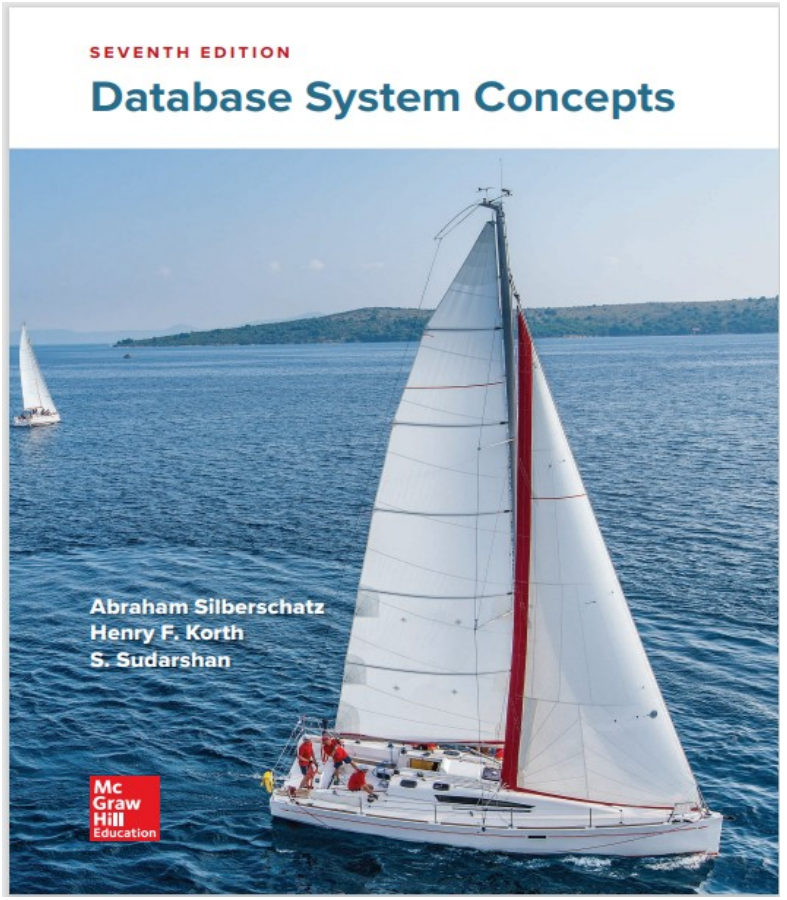
Textbooks

PES UNIVERSITY
CELEBRATING 50 YEARS

Textbook 1 (TB1)



Textbook 2 (TB2)



Database Management Systems

Textbooks

Text Book(s):

1. “Fundamentals of Database Systems”, Ramez Elmasri, Shamkant B Navathe, Pearson, 7th Edition, 2017.
2. “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition.
3. Essential Concepts of Vector Databases: Mastering Vector Databases for Advanced Data Management by Paulo Dichone, 1st edition, Packt publisher

Reference Book(s):

1. Database Management Systems, R Ramakrishnan, J Gehrke, 3rd Edition, McGraw Hill, 2002.
2. Data on the Web: From Relations to Semistructured Data and XML, S Abiteboul, P Buneman, D Suciu, Morgan Kauffman, 1999.

Database Management Systems

Course Evaluation Policy

Theory Components	Conduction Marks	Scaled Marks
ISA-01	40	20
ISA-02	40	20
Assignment (Experiential Learning -Jackfruit)	10	10
ESA	100	50
Total	150	100

Lab Components	Conduction Marks	Scaled Marks
Weekly Labs (Experiential Learning - Banana)	8 labs * 10 Marks =80 Marks	10
Miniproject (Experiential Learning - Orange)	Review-01 -> 5 Marks Review-02 -> 5 Marks Review-03 -> 10 Marks Review-04 -> 10 Marks Total 40 Marks	10
Total (FSA Component)	100	20

Database Management Systems

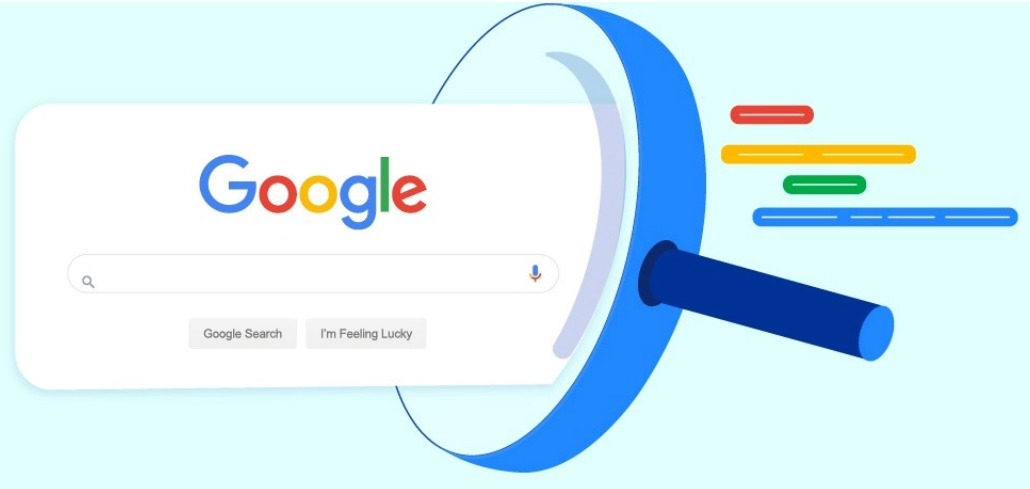
Introduction

- Have you ever wondered on *how a large e-commerce website like Amazon is able to retrieve the products that you would want to purchase?*
- Nowadays social media (eg: Instagram) one of the most active platforms where people post their day to day activities etc. *how and where is all these information stored?* By typing a person's name we can get to see their entire profile, posts that person has posted how is this possible?
- In a bank, multiple transactions take place everyday. Even if 2 transactions are happening simultaneously from account, still the money in the accounts is properly deducted and credited *how does this happen?*

Database Management Systems

Introduction

- Suppose you visit your doctor for a general checkup. Now by typing in your name or your ID, all your medical history details would be visible to the doctor. *How does this happen?*
- When you search for a specific topic on Google, ***how does the search engine quickly retrieve relevant web pages from millions of websites and present them to you?***

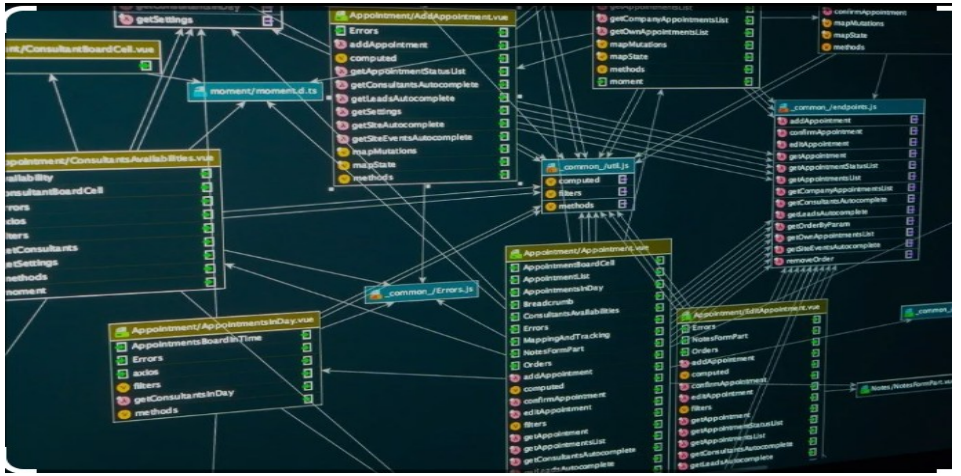


All the above questions have a single answer and that is **DATABASE**

Before going ahead and understanding what database exactly and how is designed and used for all the other application, **let us try answering a few more questions:**

- What do you mean by **Data**?
- How is it different from **Information**?

In current world, there is a lot of data that is available, how are all these stored or organized so that we can get the required information?



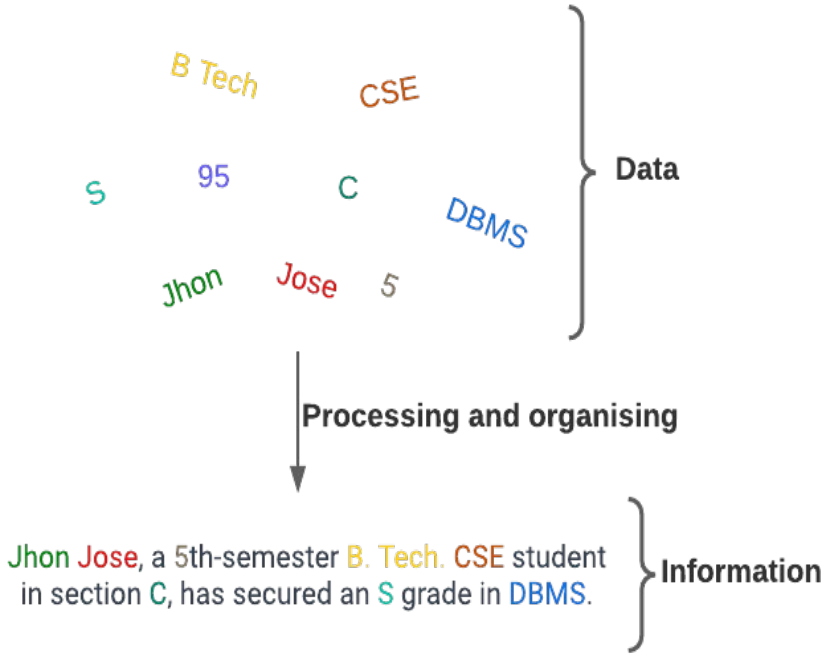
What Do you mean by Data & Information?

Data

- Data represents the **raw material**, like individual puzzle pieces, that serves as the foundation for knowledge.
- Data is a raw and unorganized fact that is required to be processed to make it meaningful.

Information

- When data is processed, **organized, structured, or presented in a given context to make it useful is called information**.
- Data is a collection of facts, while information puts those facts into context
- It is organized and is utilized by humans in some significant way to make decisions and draw some conclusions



Data doesn't depend on information, Information depends on data

What is a Database?

Database

- A database is a **collection of related data** representing some aspect of the real world, also called the **mini-world** or the **universe of discourse (UoD)**.
- It is a **logically coherent collection**, meaning it is not just a random assortment of data but is **organized** with **inherent meaning and structure.**
- Databases are designed, built, and populated for a **specific purpose**, catering to the needs of applications or systems that interact with them.
- They can **vary in size and complexity**, ranging from small databases used by individual applications to large-scale enterprise databases handling vast amounts of data.

Could you think of what do these individual terms mean w.r.t student information Database?

What is a Database

Student Information Database for a university

Collection of Related Data: The database contains various pieces of information related to students, courses, and grades. It includes tables for students, courses, and enrollments, all connected through relationships.

Miniworld (Universe of Discourse): The miniworld in this context is the university's academic environment. The database captures essential aspects of the real-world entities like students and courses within the university.

Logically Coherent Collection: The database is not a random assortment of data but organized logically. For instance, student information is stored in the "Students" table with fields like student ID, name, and contact information, creating a coherent representation of students' details.

Designed for a Specific Purpose: The database is designed to store and manage student information, course details, and enrollments efficiently. It serves the purpose of providing a centralized system for university administrators to access and update student-related data.

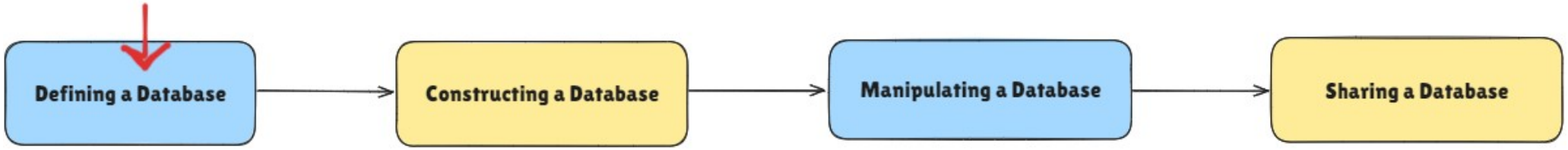
Any Size and Complexity: The database's size and complexity can vary based on the university's size and the number of students and courses it manages. Larger universities might have more extensive databases with additional tables for faculty, academic departments, and research projects.

Database Management System

- A modern database system is a **complex software system** whose task is to manage a large, complex collection of data.
- DBMS contains information about a particular enterprise
  - *Collection of interrelated data*
  - *Set of programs to access the data*
  - *An environment that is both convenient and efficient to use*
- Database systems are used to manage collections of data that are:
  - Highly valuable
  - Relatively large
  - Accessed by multiple users and applications, often at the same time.

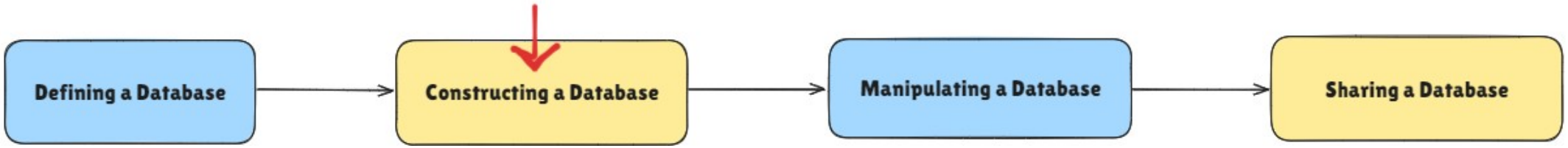
Let us break these statements down further and understand them in the context of the student information database

Database Management System



- A general-purpose software system that facilitates the **processes of defining**, constructing, manipulating, and sharing databases among various users and applications
  - **Defining a database involves specifying the data types, structures, and constraints of the data to be stored in the database. By defining the structure and data types, the DBMS ensures the consistency and integrity of the stored data.**
    - The "Students" table may include fields such as "Student ID" (numeric), "Name" (text), "Email" (text), "Date of Birth" (date), and "Major" (text).
    - The "Courses" table may have fields like "Course ID" (numeric), "Course Name" (text), "Credits" (numeric), and "Instructor" (text).

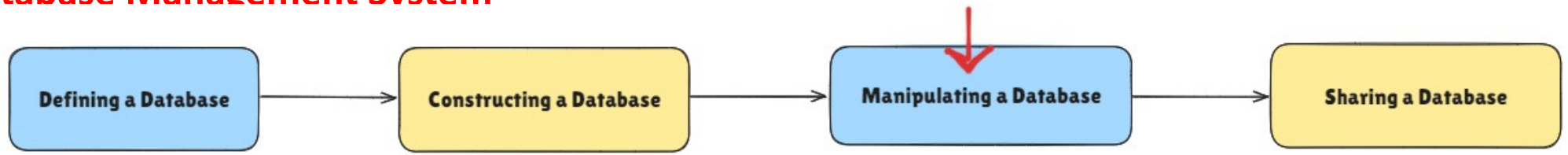
Database Management System



- A general-purpose software system that facilitates the processes of defining, **constructing**, manipulating, and sharing databases among various users and applications
  - **Constructing the database** *is the process of storing the data on some storage medium that is controlled by the DBMS. It organizes the data in a way that allows for efficient retrieval and manipulation. The DBMS ensures data is securely stored and can be accessed by authorized users or applications.*
    - The DBMS may store the Student Information Database on a server with appropriate data storage capacity and performance capabilities.

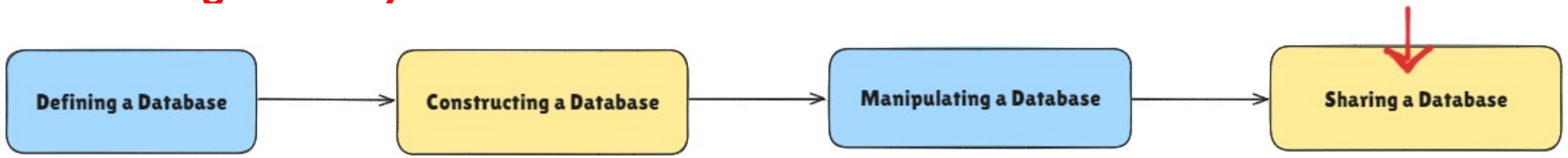
Database Management System

Database Management System



- A general-purpose software system that facilitates the processes of defining, constructing, **manipulating**, and sharing databases among various users and applications
  - **Manipulating a database** *includes functions such as querying the database to retrieve specific data, updating the database to reflect changes in the mini world, and generating reports from the data. It ensures that any* ***changes made to the database follow predefined constraints and maintain data consistency****.*
    - A university administrator may use SQL queries to retrieve specific information, such as "**Retrieve all students enrolled in Computer Science courses.**" The DBMS processes the query, retrieves the relevant data from the database, and presents it to the user.
    - Allows **authorized users to update the database**, such as adding new students or updating grades.

Database Management System



- A general-purpose software system that facilitates the processes of defining, constructing, manipulating, and **sharing** databases among various users and applications
  - **Sharing a database** *allows multiple users and programs to access the database simultaneously, provided they have the necessary permissions*
    - Faculty members, administrative staff, and students can access relevant information concurrently, as long as they have the necessary permissions.
    - For example, faculty members can access student details for their courses, while administrators can update and maintain the overall database.

Questions

Some questions to revise

1) How do you define data, and how does it differ from information in the context of database management systems?
2) List out the data types you would take to define a database for an online bookstore.
3) Explain the role of a DBMS in managing highly valuable and relatively large collections of data. Why is this important for organizations?
4) What are some of the common operations performed on a database? How do these operations interact with the data stored in the database?

Database Management Systems

Why study databases

Databases help us to:

- Store **large amounts of data** (file structure, disk management)
- Understand the data (data models)
- Keeps data **secure** (security, recovery)
- Find required data and **use/manipulate it** (query languages, **concurrency control, and data analysis tools)**
- Get accurate information (as databases have **built-in constraints** and checks help in this)
- Maintain **Data integrity** (ensures data is accurate and consistent)

Data Integrity Example: Ensuring a student's ID is unique (Entity Integrity), all enrollments refer to valid students (Referential Integrity), ages are valid numbers (Domain Integrity), and business rules like discount prices are logically correct (User-Defined Integrity).

Database Applications

1) Enterprises

- **Customer Relationship Management (CRM)**: Databases store customer interactions, preferences, and sales data.
- **Supply Chain Management**: Databases track inventory levels, supplier information, and logistics.
- **Human Resources**: Manage employee data, payroll, and benefits using databases.
- **Financial Systems**: Handle accounting, budgeting, and financial reporting through database solutions.
- **Example**: ***Salesforce CRM*** *is widely used by enterprises to manage customer relationships and sales data within a database.*

2) Manufacturing

- **Production Tracking**: Databases monitor production schedules, inventory levels, and quality control.
- **Supply Chain Management**: Manage supplier information, procurement, and logistics using databases.
- **Product Lifecycle Management**: Track product design, development, and updates in a database.
- **Example**: **Toyota** *uses databases to manage their Just-In-Time (JIT) manufacturing processes, ensuring efficient production and inventory management.*

Database Applications

3) Banking and Finance

- **Account Management**: Databases store customer accounts, transactions, and balance information.
- **Loan Processing**: Manage loan applications, approvals, and repayments through a database system.
- **Fraud Detection**: Monitor transactions for unusual activity and potential fraud using database systems.
- **Example**: ***JPMorgan Chase*** *uses advanced database systems to manage customer accounts, process transactions, and detect fraud.*

Database Applications

7) Web-based Services

- **User Data Management**: Databases store user profiles, preferences, and activity logs.
- **Content Management**: Manage website content, including articles, images, and videos, using a database system.
- **Analytics**: Track user interactions, traffic patterns, and conversion rates through databases.
- **Example**: ***Netflix*** *uses a sophisticated database system to manage user preferences, content recommendations, and streaming data.*

Database Applications

8) Document Databases

- **Data Storage**: Document databases store semi-structured data in formats like JSON, BSON, or XML, allowing for flexible schema design and easy data retrieval.
- **Content Management**: Manage collections of documents such as articles, patents, research papers, and other unstructured content.
- **Scalability**: Document databases are designed to handle large volumes of data and can scale horizontally across multiple servers.
- **Use Cases**: Often used for content management systems, catalogues, and data lakes where data doesn't fit neatly into relational models.
- **Example**: ***MongoDB*** *is a popular document database used by companies like* ***The New York Times*** *to manage and deliver vast amounts of news articles and media content.*

9) Navigation Systems

- **Location Data**: Databases store geographic information including locations of places of interest, routes, and geographic features.
- **Real-Time Updates**: Manage real-time traffic data, route optimization, and point-of-interest information.
- **User Interaction**: Provide features like route planning, navigation guidance, and local searches.
- **Integration**: Often integrated with other systems such as weather services, local businesses, and emergency services to enhance user experience.
- **Example**: ***Google Maps*** *uses a complex database system to manage and update location data, provide navigation services, and offer real-time traffic updates to millions of users worldwide.*

How are databases used?

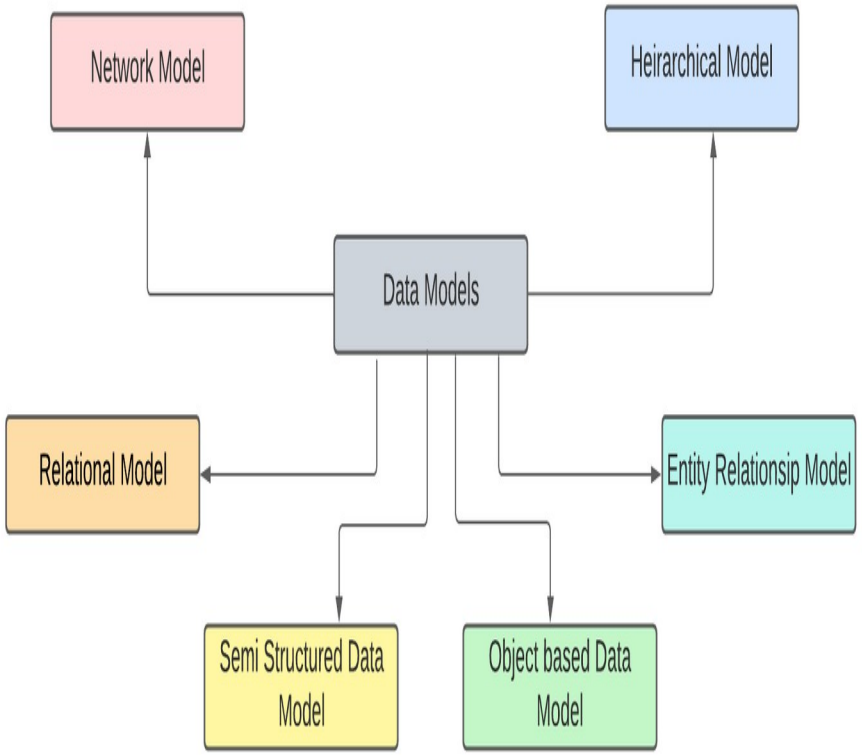
There are two modes in which databases are used today

- **Online Transaction Processing (OLTP)**:
  - Used by a large number of users for small data retrieval and updates
  - common in most database applications like banking, universities, and airlines.

- **Data Analytics/Online Analytical Processing(OLAP)**:
  - Involves *processing data to draw conclusions and create predictive models for business decisions.*
  - **Examples** include loan approval, targeted advertisements, and manufacturing decisions.
  - Data mining combines AI and statistical techniques for efficient analysis of large databases.

View of Data

- **A database system is a collection of interrelated data and a set of programs that allow users to access and modify these data.**
- A major purpose of a database system is to provide users **with an abstract view** of the data.
  - Data models
    - A collection of conceptual tools for describing data, data relationships, data semantics and constraints.
  - Data abstraction
    - Hide the complexity of data structures to represent data in the database from users through several levels of data abstraction.
- **The structure of a database is defined by its data model**, which includes tools to describe data, relationships, semantics, and consistency constraints.



Questions

Some questions to be explored??

1) List and describe three real-world applications of databases in different industries. How do these applications benefit from using databases?
2) What are the two modes in which databases are used today?
3) Discuss the various data models. How does the relational model differ from the semi-structured and object-based models?
4) Explain the significance of document databases in storing and managing semi-structured data. How does MongoDB exemplify the use of document databases in real-world applications?

How are databases used?

Some online resources -

- https://dev.mysql.com/doc/
- https://www.oracle.com/in/database/what-is-database/
- https://docs.oracle.com/cd/E17952_01/index.html
- https://downloads.mysql.com/docs/mysql-tutorial-excerpt-8.0-en.a4.pdf

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Purpose of Database System

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgment: Dr. Nagasundari S (Dept of CSE), Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 1: Introduction to Database Management and SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G.

Department of Computer Science and Engineering

File-based System:

- One way to keep information on a computer is to store it in **permanent files.**
- A company *system has a number of application programs; each of them is designed to manipulate data files*.
- These *application programs have been written at the request of the users in the organization.*
- New applications are added to the system as the **need arises.**

Challenges of File-processing Systems

Let us take an example of a University Organization system. We need to store information about the following entities:

- **Instructors, Students, Departments, Courses offered.**
- One of the ways to keep this information is to store it in **operating file systems**
- To allow the users to manipulate the information, the system would have a **number of application programs that manipulate** the files including:
  - Addition of new students, instructors, and courses
  - Register students for different courses
  - Assign grades to students, compute GPA, and generate transcripts.

Challenges of File-processing Systems

Scenario: The university decides to **add a new major.**

What specific steps must be taken to integrate a new major into the university's existing system?

1. **Create a New Department:**
   - Set up a new department for the major.
   - Create new permanent files for department information.
2. **Develop New Application Programs:**
   - Write programs to manage rules for the new major.
   - Ensure programs handle course registration, grading, and transcripts for the new major.

This typical **file-processing system** is supported by a conventional operating system. The system stores permanent records in various files, and it needs different application programs to extract records from and add records to, the appropriate files.

Challenges of File-processing Systems

What is the need for a Database Management System when we can do the university organization using a simple file-processing system?

Consider the following situations:

- Double Major Students :

  What if there is a student who has enrolled for a double major (Physics and Computer Science) how many times do we store all his details?

- Updating information :

  What happens if a student changes their address or phone number? Is it guaranteed that the update reflects in all relevant files across departments?

Challenges of File-processing Systems

- Efficient Data Retrieval:

  The university transport department wants to find out all students who live within a particular postal code area. *Can he quickly retrieve the data or does he have to build another application for it?*

- Data Format Consistency:

  What if files that store these data are written **in different formats** is it easy to access the data in this form?

- Adding New Constraints:

  Suppose a new constraint is to be added, can it be done efficiently without having to make extensive changes to existing application programs?

Challenges of File-processing Systems and the Purpose of Database Systems

There are a lot of drawbacks to the file-processing system :

- Data Redundancy and Inconsistency
- Difficulty in accessing the data
- Data isolation
- Integrity problems
- Atomicity problems
- Concurrent access anomalies
- Security problems

PES UNIVERSITY
CELEBRATING 50 YEARS

1. Data Redundancy & Inconsistency

Consider the question of **the student enrolled for a double major. What would happen in this situation in case of file system?**

Details of the students would be duplicated for each department - **Data Redundancy**

This would lead to the following problems:

- Takes up more storage space
- May also lead to **Data Inconsistency** in case of changes

Solution to this problem:

- Store information in centralized location thus Minimizes redundancy
- Changes can be made at a single spot, thus ensuring consistency
- This can be achieved through a database system

2. Difficulty in accessing the data

Consider that the university wants **to find all the students living in a specific postal code area**. How will we execute this in traditional file system?

- Either we need to get a list of all students and manually sift through it
- Or create another special program for this task

Both the above-mentioned are time-consuming and inconvenient

Thus we can see that Conventional file-processing systems are not designed for quick and efficient data retrieval, especially when new types of queries arise.

The solution to this problem:

- **Central information** store for fast retrieval
- Use a powerful query language for answering new queries
- Efficient ways to search and filter data for speeding up data retrieval
- This can be achieved through a database system

3. Data Isolation

Consider a concurrency situation,
where identifying when and how changes made by one operation become visible to other concurrent users.

This is a challenge because New applications has to retrieve the appropriate data, which might be stored in various files.

A file-based system must manage, or prevent, concurrency by the application programs.

When an application opens a file, it will lock the file. This means that no one else has access to the file at the same time.

The solution to this problem:

- Ability to allow multiple transactions to access same data without interfering with each other
- Consistent view of the data

4. Integrity problems

Scenario: Each department has an **account for research funds, and the account balance must always be above 0.**

How can we implement this?

Each application program is created for maintaining departmental accounts but adding new constraints becomes challenging as it requires editing all the existing application programs.

The solution to this problem:

- Specify integrity constraints at the database level to specify rules directly in the schema
- System should enforce constraints automatically
- To add a new constraint, updating the schema alone will be sufficient, and the system will handle the rest automatically.
- This would be easily solved by using DBMS which provides all these facilities

5. Atomicity problems:

In a computer system failure can happen unexpectedly

Consider a banking system, a sum of 5000 rupees is getting transferred from account A to B.

What happens internally?

- An amount of 5000 Rs. gets deducted from account A
- The amount (5000 Rs.) gets credited to account B

Now **either both the steps should be performed or none should be executed.**

In a file system, The program might successfully deduct 5000 from account A but fail to credit it to account B.

This results in an **inconsistent** database state, with money missing from one account and not added to the other.

The solution to this problem:

- Atomicity should be ensured
- Automatic rollback in case of failure
- DBMS would ensure this and makes it reliable and consistent even if failure occurs

6. Concurrent Access Anomalies

Consider a bank transaction example. There is a bank account (Account A) with a balance of $10,000.

- Two bank clerks attempt to deduct money from Account A at almost the same time - one deducts $500 and the other $100.
- Since they work concurrently, they both read the initial balance of $10,000, subtract their respective amounts, and write back the results.
- Depending on which one writes the new balance last, the account could end up with either $9,500 or $9,900 instead of the correct value of $9,400.

The solution to this problem:

- Transaction and Locking to be implemented
- Transactions ensure atomicity
- Locking prevents conflicting changes.
- The DBMS would provide these and maintains data consistency and integrity

7. Security problems

Imagine a university using a file-based system. Over time, they've added various application programs without a central security system.

As a result, payroll personnel might accidentally see and even change academic records, compromising data privacy and confidentiality because everyone has the same permissions.

The solution to this problem:

- Create a provision for administrator to define access controls and permissions for each user or group.
- These permissions should enforced at the database level, regardless of the application programs used.
- DBMS supports various methods for introducing security like User Authentication, Data Encryption, Auditing and Logging, Centralized security management

Characteristics of Database Systems

- **Self-describing nature of a database system**
  - The database system contains not only the database itself but also a complete definition or description of the database structure and constraints (Metadata).
- **Insulation between programs and data, and data abstraction**
  - The structure of data files is stored in the DBMS catalog separately from the access programs (program-data independence)
- **Support multiple views of the data**
  - DBMSs support different types of users, each of whom may require a different perspective or view of the database. A view may be a subset of the database or it may contain virtual data that is derived from the database files but is not explicitly stored.
- **Sharing of data and multi user transaction processing**
  - A multi user DBMS, allows multiple users to access the database at the same time. It ensures that data changes are effected in a controlled manner so that the result of the changes are correct

Advantages of Database Systems

- Controlling redundancy in data storage.
- Sharing of data among multiple users.
- Restricting unauthorized access to data
- Providing persistent storage for program Objects
  - E.g., Object-oriented DBMSs make program objects persistent
- Providing Storage Structures and search techniques for efficient Query Processing
- Providing backup and recovery
- Providing multiple interfaces to different classes of users.

Advantages of Database Systems

- Representing complex relationships among data.
- Enforcing integrity constraints on the database.
- Drawing inferences and actions from the stored data using deductive and active rules and triggers
- Potential for enforcing standards
- Reduced application development time
- Flexibility to change data structures
- Availability of current information
- Economies of scale

Identify the Problem: Case Study Applications

1. <u>Scenario 1:</u> Banking Transactions:

At a bank, two customers attempt to withdraw money from the same account simultaneously. Due to a lack of concurrency control, the system fails to manage the simultaneous transactions correctly, resulting in the account being overdrawn.

Problem Type: Concurrent Access Anomalies

Scenario 2: Customer Records:

A company stores customer information in multiple spreadsheets, with each department maintaining its own copy. Occasionally, customer addresses and contact details differ between spreadsheets, leading to confusion and errors in communication.

Problem Type: Data Redundancy and Inconsistency

1

Name	Address	Phone
John Doe	123 Elm Street	555-9067
Grace	12 Oak Street	123-8907
Judy	45 York Lane	237-1123

2

Name	Address	Phone
Grace	12 Oak Street	123-8907
John Doe	129 Blueberry Lane	555-9067
Judy	45 York Lane	237-1123

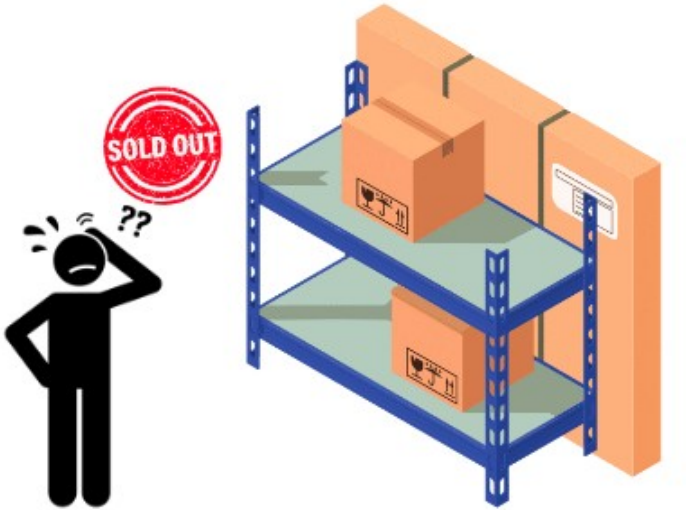
3

Name	Address	Phone
John Doe	123 Elm Street	555-9067
Judy	45 York Lane	237-1123
Grace	12 Oak Street	123-8907

<u>Scenario 3:</u> Inventory and Orders:

A small business uses separate systems to manage inventory and orders. The inventory system is not updated in real-time with sales data, leading to situations where the business oversells products, unaware that the stock has been depleted.

Problem Type: Data Isolation

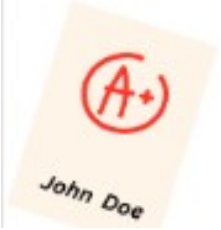


<u>Scenario 4:</u> Student Grades:

A university records student grades manually, and sometimes errors are made in entering the data, such as typing grades incorrectly. There is no system in place to check for or correct these errors automatically.

But, if grade is entered as ‘x’ which is not a valid grade then constraints can be added to handle.

Problem Type: Integrity Problems



Name	Grade
John Doe	X
Judy	A
Grace	C

THANK YOU

Mahitha G
Department of Computer Science and Engineering
mahithag@pes.edu

Database Management Systems

Database Design

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgments: Dr. Nagasundari S (Dept of CSE), Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 1: Introduction to Database Management and SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G.

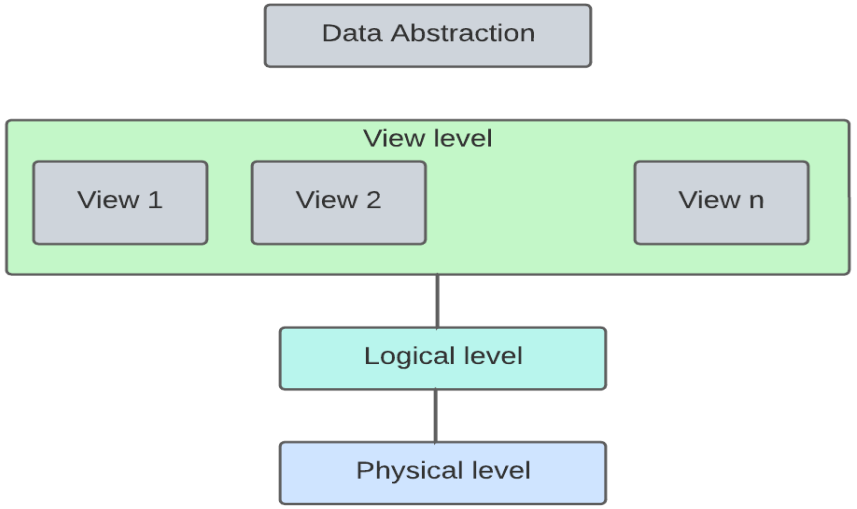
Department of Computer Science and Engineering

Database Management Systems

Data Abstraction

- DBMS plays an important role in many areas. One of the main requirements is that it **must retrieve the data efficiently.**
- This need for efficiency has led the database developers to use complex data structures to represent data in a database
- Many database-system users are not computer trained and do not know how to write complex queries for getting information they are looking for.
- Hence *developer hide all these complexity from the users through several layers of data abstraction, in order to simplify the users' interaction with the system*

- The different levels of data abstraction:
  - Physical Level
  - Logical level
  - View level



Data Abstraction

Let us now have a look at what happens at each level:

Physical Level :

- It is the **lowest level of data abstraction**
- It **describes how the data is actually stored in the computer**

Logical Level :

- This **describes what kind of data is stored in the database** and **how different pieces of information are related to each other**
- The logical level thus describes **the entire database in terms of a small number of relatively simple structures.**
- This level provides a more organized and simpler view of the data

Data Abstraction

Let us now have a look at what happens at each level:

View Level :

- This is the **highest level of abstraction that individual users interact with and that only displays a part of the entire database**

- Even though the logical level uses simpler structures**, complexity remains because of the variety of information stored in a large database.** Many users of the database system do not need all this information; ***instead, they need to access only a part of the database.***
- At this level, different users might see different parts of the database, depending on their needs and permissions

Data Abstraction

Now we can notice that:

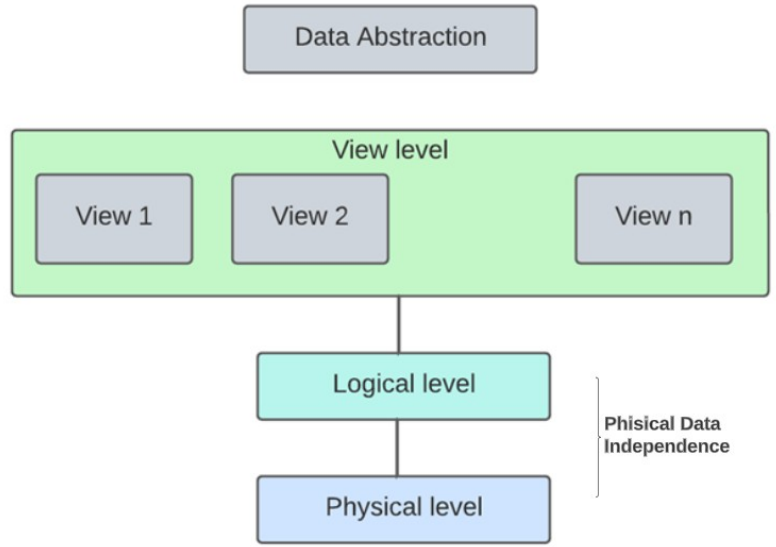
- Although the implementation of simple structures at the logical level may involve complex physical-level structures, **the user at the logical level does not need to be aware of this.**
- **Similarly the user who views the database for various purposes need not be aware of the Logical level implementation**, for them it would be like an interface and need not worry about the logical view

PHYSICAL DATA INDEPENDENCE:

- It refers **to the characteristic of being able to modify the physical schema without any alterations to the conceptual or logical schema**, done for optimization purposes.

Example :

- the **Conceptual structure of the database would not be affected by any change in the storage size of the database system server**. Changing from sequential to random access files is one such example.
- These alterations or modifications to the physical structure may include:
  - Utilizing new storage devices.
  - Modifying data structures used for storage.
  - Altering indexes or using alternative file organization techniques etc.

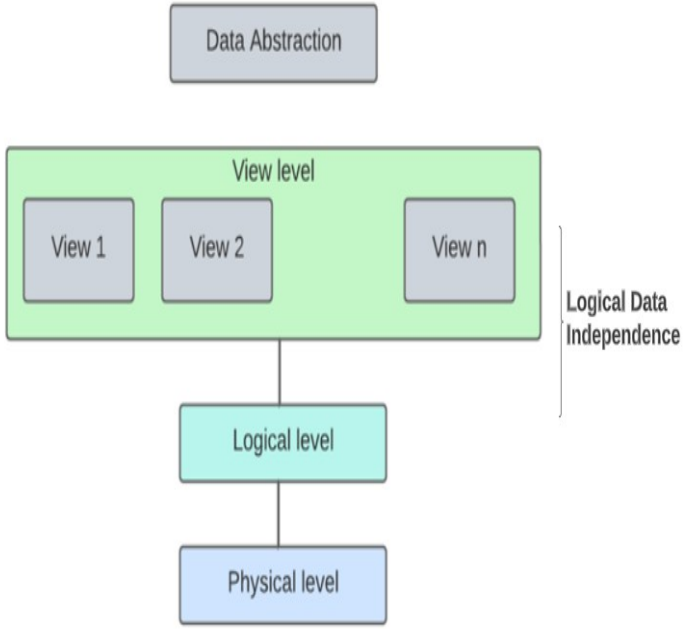


LOGICAL DATA INDEPENDENCE

- It refers **characteristic of being able to modify the logical schema without affecting the external schema or application program.**
- The *user view of the data would not be affected by any changes to the conceptual view of the data.*

Example:

- These changes may include the **insertion or deletion of attributes**,
- altering table structures entities or relationships to the logical schema, etc.



Database Management Systems

Data Abstraction

Now let us understand this with the help of an **example of a University database**

- Let the university have the given Record types/ tables as shown

Now for the given database,

- Physical level:
  - Specify how student records, course information, and department data are **stored on the hard drive or in memory.**
- Logical Level:
  - It would **define entities** like "Department," "Student," and "Course" and their attributes (e.g., department name, student ID, course code).
  - It **also outlines the relationships** between these entities, such as "Students belong to Departments" and "Courses are taken by Students."

Department

Dept_ID	Dept_Name	Building

Course

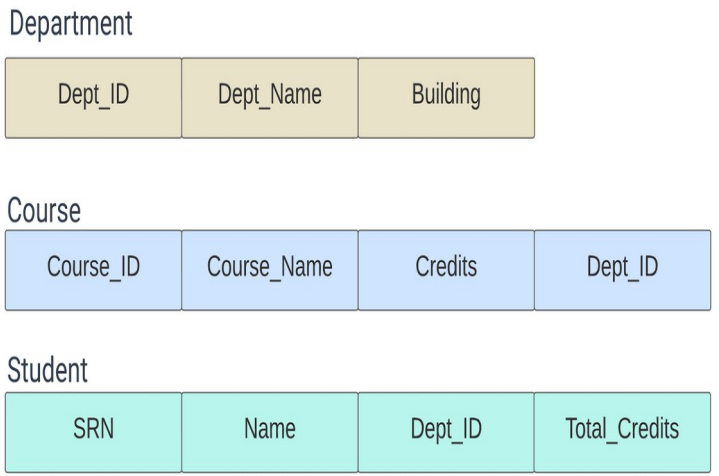
Course_ID	Course_Name	Credits	Dept_ID

Student

SRN	Name	Dept_ID	Total_Credits

Data Abstraction

- View Level:
  - *A faculty member may have access to a view showing only the courses they are teaching* and the students enrolled in those courses.
  - *A student, on the other hand, may have a view displaying only the courses they are registered* for and their respective grades.
  - These views provide a simplified and customized perspective of the database for different users.



Database Management Systems

Data Abstraction

Let us consider the situation where *the University decides to migrate its database from one storage system to another or optimize the storage layout to improve performance.* What would be the consequence of this?

Without Physical data independence:

- Minor changes to the storage system or hardware upgrades could *potentially break existing queries*, *applications, and interfaces*.
- As a result, any update to the storage system would require extensive modifications to the application layer, leading to higher maintenance costs and longer development cycles.

Department

Dept_ID	Dept_Name	Building

Course

Course_ID	Course_Name	Credits	Dept_ID

Student

SRN	Name	Dept_ID	Total_Credits

Data Abstraction

Let us consider the situation *where the University decides to migrate its database from one storage system to another or optimize the storage layout to improve performance.* What would be the consequence of this?

With physical data independence:

- the **database administrators can make these changes without altering the logical schema,** which means users' queries and applications will continue to work seamlessly.
- **It allows for easier maintenance, upgrades, and performance enhancements without disrupting the application layer.**

Department

Dept_ID	Dept_Name	Building

Course

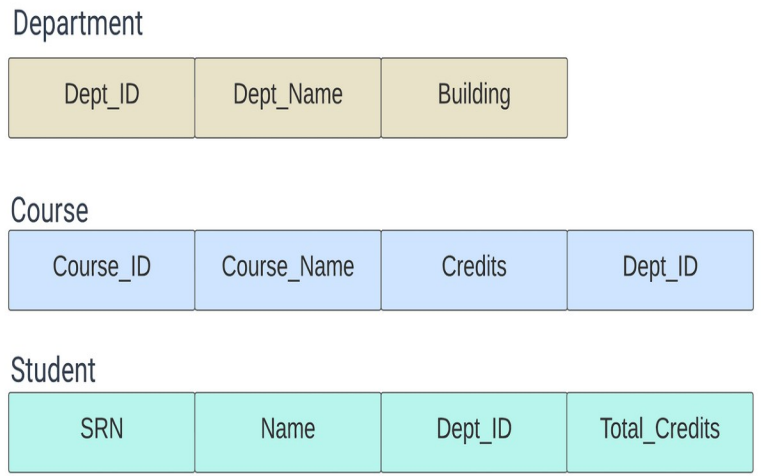
Course_ID	Course_Name	Credits	Dept_ID

Student

SRN	Name	Dept_ID	Total_Credits

Data Abstraction

Let's say **the University decides to reorganize its department structure and add new attributes to the "Department" entity. Then what would be the effect?**



Without Logical data independence:

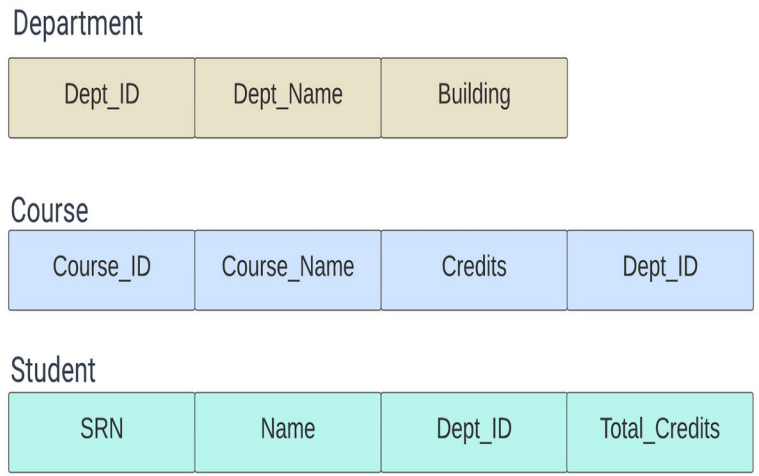
- Any *alteration to the logical schema, such as adding or removing attributes, or changing relationships between entities, would result in changes propagating throughout the applications that rely on the database.*
- This would force **developers to modify all affected application** code to accommodate the changes in the schema.
- Consequently, *even minor changes to the data model could lead to widespread disruption and downtime for the applications*, increasing the risk of introducing errors and inconsistencies.

Data Abstraction

Let's **say the University decides to reorganize its department structure and add new attributes to the "Department" entity. Then what would be the effect?**

With Logical data independence:

- The **database designers can make these modifications without impacting the existing application programs** that access the database.
- This flexibility allows for **easier adaptability to changes** in the University's requirements and business rules without causing application disruptions.

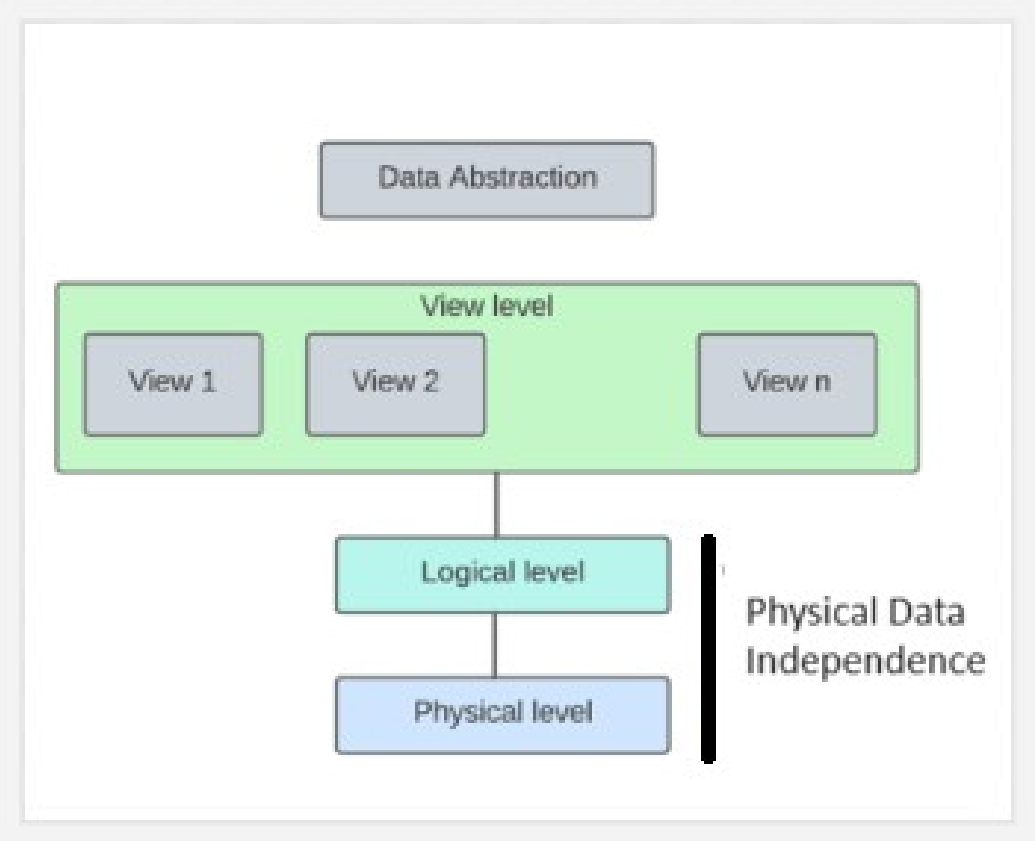


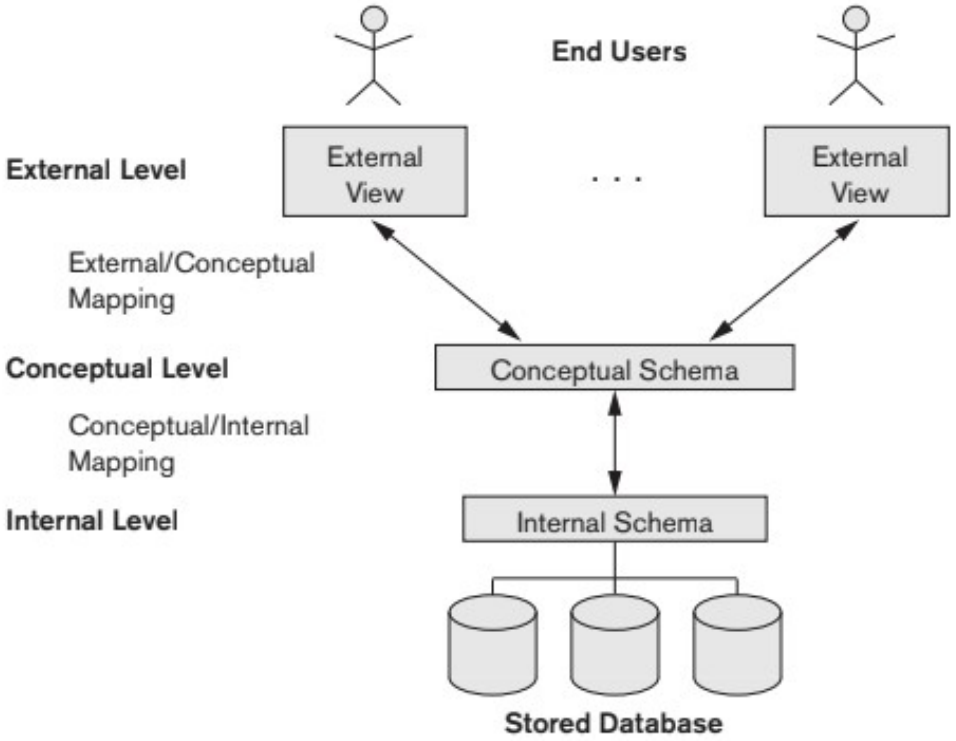
Instances and Schema

- Databases change over time as information is inserted and deleted or modified.
- **The collection of information stored in the database at a particular moment is called an instance of the database.**
- **The overall design of the database is called the database schema.**
- Let us try to understand what that exactly means with an **analogy to a program written in any programming language**
  - Database Schema => variable declarations along with the associated type definitions
  - Instances => Value of the variable at a particular point in time in the program

Database Management Systems

Three Schema Architecture





Database State

- The actual data in a database may change quite frequently.
- The data in the database at a particular moment in time is called a **database state** or **snapshot**.
- It is also called the *current set* of **occurrences** or **instances** in the database.
- In a given database state, each schema construct has its own current set of instances.
- Many database states can be constructed to correspond to a particular database schema.

Database State V/S Schema

- The distinction between database schema and database state is very important.
- When we **define** a new database, we specify its database schema only to the DBMS.
- At this point, the corresponding *database state is the empty state with no data.*
- We get the initial state of the database when the database is first **populated** or **loaded** with the initial data.
- From then on, every time an update operation is applied to the database, we get another database state. **At any point in time, the database has a *current state*.**

Database Management Systems

Database State V/S Schema

A simplified COMPANY relational database schema

EMPLOYEE F.K.

Ename	Ssn	Bdate	Address	Dnumber

P.K.

DEPARTMENT F.K.

Dname	Dnumber	Dmgr_ssn

P.K.

Sample database state for the relational database schema

EMPLOYEE

Ename	Ssn	Bdate	Address	Dnumber
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4
Wallace, Jennifer S.	987654321	1941-06-20	291Berry, Bellaire, TX	4
Narayan, Ramesh K.	666884444	1962-09-15	975 Fire Oak, Humble, TX	5
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1

DEPARTMENT

Dname	Dnumber	Dmgr_ssn
Research	5	333445555
Administration	4	987654321
Headquarters	1	888665555

Why Database Design ?

Scenario:

Imagine you are tasked with designing a database for a new online bookstore. The bookstore needs to manage a large catalog of books, customer information, orders, and reviews. Customers should be able to search for books, place orders, leave reviews, and track their order history.

Question

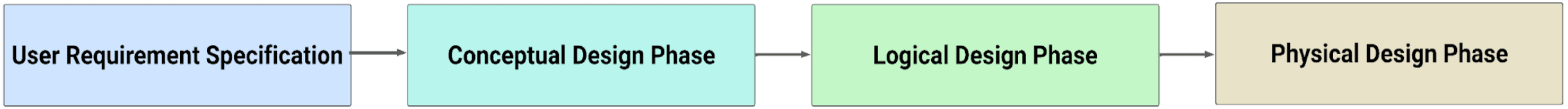
What factors would you need to consider while designing the database system to ensure it meets all the needs of the online bookstore?

(Hint :
- *Types of data Eg: books, customers, orders, reviews?*
- *How will you ensure data integrity and security?)*

Database Design

- **Database design mainly involves the design of the database schema.**
- The design of a complete database application environment that meets the needs of the enterprise being modeled requires attention to a broader set of issues.
- **A high-level data model gives designers a conceptual framework** to specify user data requirements and structure the database accordingly.

The various steps involved in the **design process** are :



Let us now try and understand what happens at each step:

1. USER REQUIREMENT SPECIFICATION
   - Interact *with domain experts and users to fully understand the data needs and functional requirements of the database.*
   - Identify the operations or transactions that users will perform on the data.
   - Document and review the user requirements to ensure clarity and accuracy.
2. CONCEPTUAL DESIGN PHASE
   - Choose a *high-level data model (e.g., entity-relationship model) to conceptualize* the data requirements and relationships.
   - Develop a conceptual schema that provides a detailed overview of the enterprise's data structure.
   - Review the schema to ensure all data requirements are met and remove any redundant features.

3. LOGICAL DESIGN PHASE

- *Map the high-level conceptual schema to the implementation data model of the database system* that will be used (e.g., relational model).
- Use normalization algorithms to generate a set of tables that properly group attributes and reduce data redundancy.
- Confirm that the logical design meets the functional requirements specified in the user requirements phase.

4. PHYSICAL DESIGN PHASE

- Specify the physical features of the database, including **file organization and internal storage structures**.
- Consider performance optimization and **choose appropriate indexing** techniques to enhance data retrieval speed.
- Review and refine the physical design to ensure it aligns with the requirements and constraints of the database system and hardware.

Database Management Systems

Typical DBMS Component Modules

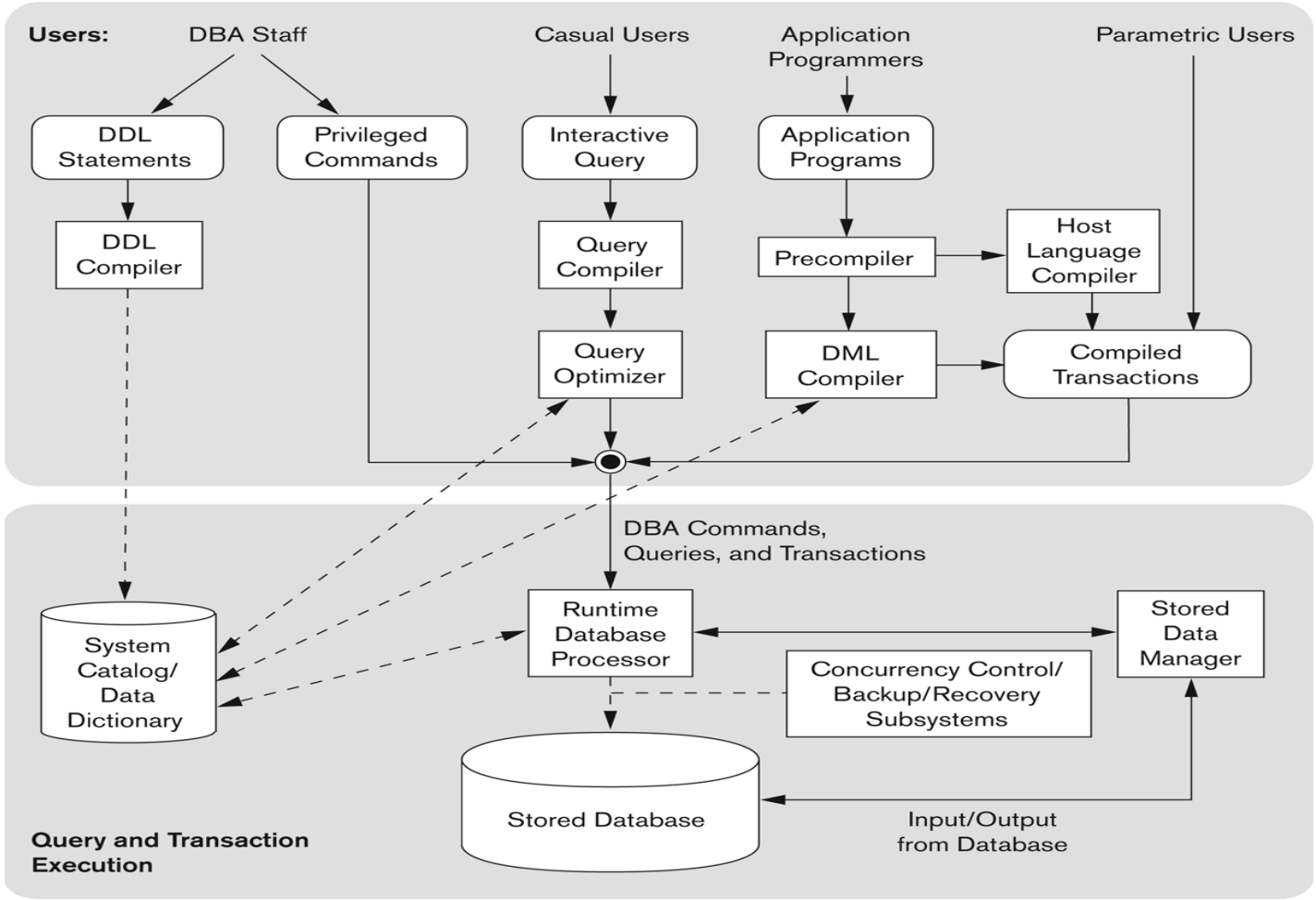


Figure 2.3
Component modules of a DBMS and their interactions.

Typical DBMS Component Modules

- SDM - controls access to DBMS info that is stored on disk (dotted lines)
- DDL Compiler - processes schema definitions specified in the DDL and stores metadata in catalog
- Query Compiler – handles high level queries - parses and compiles a query and calls to runtime processor for execution
- Pre Compiler – extracts DML commands from application program
- Run time db processor – executes privileged commands/queries/canned transactions.

Database System Utilities

- Loading utility – *Load existing data files into a database. Includes data conversion tools from old format to new db format*
- Backup utility - Back up of the database periodically. (Incremental/Total)
- Database storage reorganization - Reorganizing database file structures and create new access paths to improve performance.
- Performance monitoring utilities. – monitors db usage and provides statistics to DBA

PES UNIVERSITY
CELEBRATING 50 YEARS

Scenario:

Imagine you are responsible for the database system of a popular online multiplayer game. The game has thousands of players online simultaneously, from different parts of the world. Each player needs real-time updates for their game stats, friends list, and in-game purchases.

Question:
How would you design the database system to ensure efficient, secure, and real-time data access for all players?

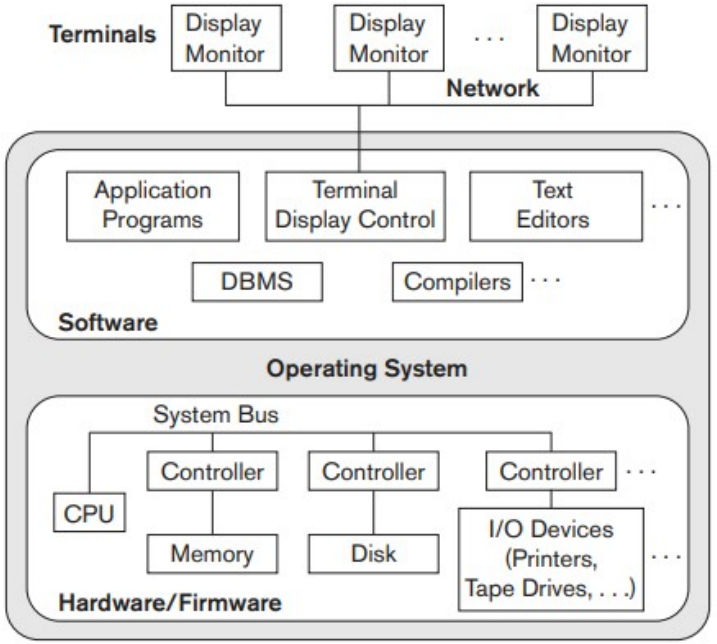
- **Centralized databases (Centralized architecture)**

Characteristics:

- **Single Location:** All data is stored and managed on a central server or database.
- **Shared Memory:** Suitable for systems with multiple CPUs that access a common shared memory space.
- **Resource Allocation:** Resources like CPU, memory, and storage are centralized, making it easier to manage and maintain.

Use Cases:

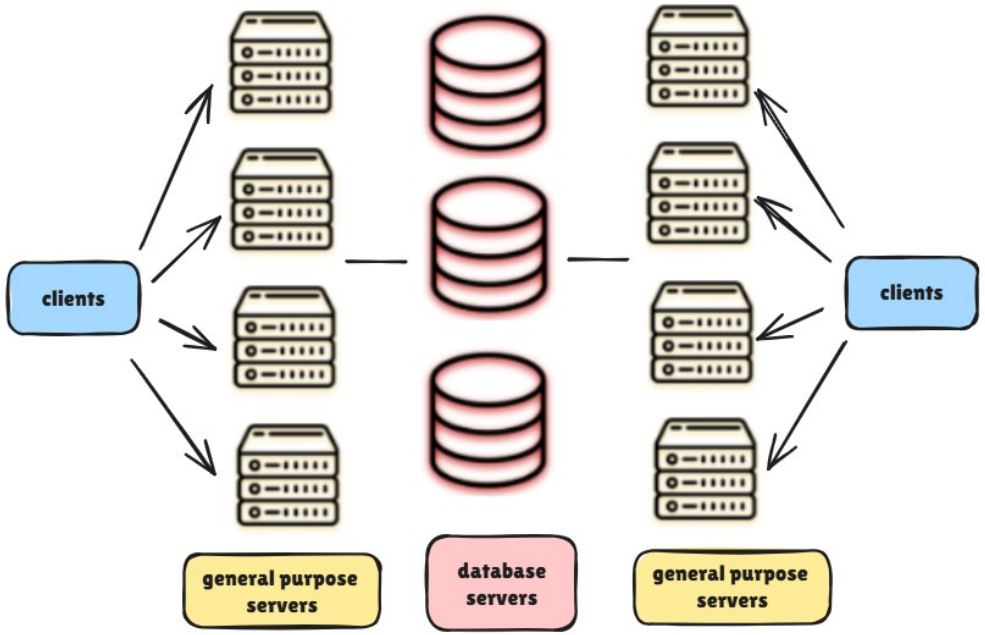
- **Small to Medium-Sized Applications:** Suitable for applications with moderate data needs and user loads.
- **Legacy Systems**: Often used in older systems where scaling is not as critical or feasible.



Centralized/Shared Memory DB

Client Server Architecture:

- Client-server architecture is a model where a client (the user or application) interacts with a server (the database system) to access and manage data.
- The client is the user-facing component that interacts with the server to request data or perform operations. (web browser, mobile apps, desktop applications)
- The server is the backend component that manages the database, processes requests from clients, and handles data storage and retrieval. (MySQL, Oracle, Postgres)



Database Architecture

Parallel databases:

- **Designed to run on a cluster of multiple machines**
- Enables better scalability and higher processing capabilities.
- Many cores shared memory and Shared Disk

Parallel databases are ideal for **high-performance applications** requiring **extensive data processing**, such as large-scale data warehousing, real-time analytics, and **high-throughput** transactional systems.

Example: How do you think NASA manages and processes the vast amounts of satellite data it collects?
Answer: NASA uses parallel databases and high-performance computing (**HPC**) systems. NASA can efficiently handle and analyze massive datasets from satellite missions, enabling quick and insightful analysis for complex tasks like climate modeling and environmental monitoring.

NASA's HPC Cluster

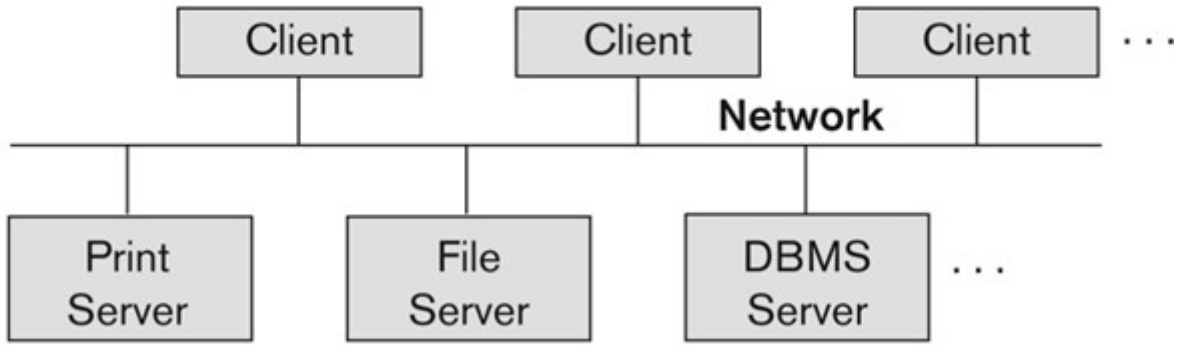
Distributed Database:

A distributed database is a database system in which **data is stored across multiple physical locations**. These locations can be spread across various machines within a single data center or across different geographic locations.

Advantages:

- **Scalability:** Can handle large amounts of data and high traffic by distributing the load across multiple nodes.
- **Fault Tolerance:** Provides high availability and fault tolerance through data replication and distribution.
- **Performance:** Can improve performance by distributing queries and data access across multiple servers.

Two-Tier Client Server Architectures



In a two-tier database architecture, there are two main components: the client (**user interface**) and the server (**database**).

- **Client:** This is the user interface where users interact with the application. It can be a desktop application or a web-based interface.
- **Server:** This is where the database is stored and managed. The server handles data storage, retrieval, and management tasks.
- Client programs send query and transaction requests via ODBC or JDBC API, the server processes these requests and sends the results back to the client, which then processes and displays the results.

Two-Tier Client Server Architectures

Clients

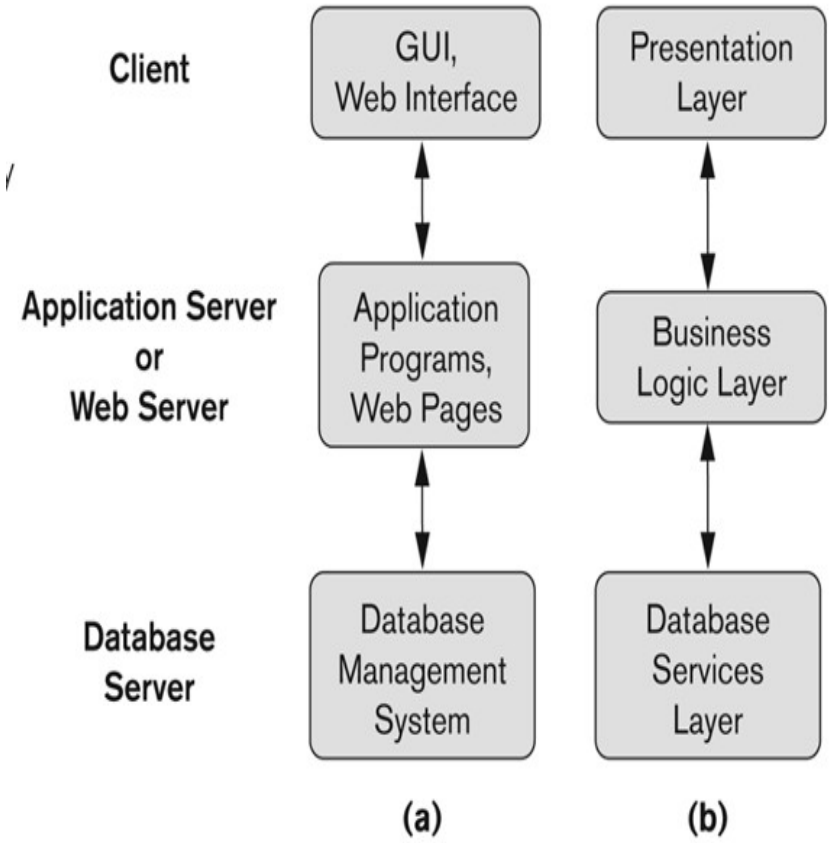
- Provide **appropriate interfaces** through a client software module to access and utilize the various server resources.
- Clients may be diskless machines or PCs or Workstations with disks with only the client software installed.
- Connected to the servers via some form of a network.
- (LAN: local area network, wireless network, etc.)

Servers

- Provides database query and transaction services to the clients
- Relational DBMS servers are often called **SQL servers, query servers, or transaction servers**
- Applications running on clients utilize an Application Program Interface (**API**) to access server databases via standard interface such as:
- ODBC: Open Database Connectivity standard
- JDBC: for Java programming access
- Client and server must install appropriate client module and server module software for ODBC or JDBC

Three-Tier Client Server Architectures

- Common for Web applications
- Intermediate Layer called Application Server or Web Server:
  - Stores the web connectivity software and the business logic part of the application used to access the corresponding data from the database server
  - Acts like a conduit for sending partially processed data between the database server and the client.
- Three-tier Architecture Can Enhance Security:
  - Database server only accessible via middle tier
  - Clients cannot directly access database server



Questions

Scenarios for Architecture Selection:

1. **Weather Forecasting System:** A weather forecasting system collects data from various sensors and satellites, processes the data to generate forecasts, and distributes the information to users through a web interface and mobile application.

2. **E-Learning Platform:** An e-learning platform offers online courses to students around the world. The platform needs a database to manage course content, track student progress, and support interactive features such as quizzes and discussion forums.

3. **Social Media Platform:** A social media platform with millions of users worldwide needs a database system that can handle high read and write loads, store large amounts of user-generated content, and provide fast access to data.

Scenarios for Architecture Selection: Solutions -

- **Weather Forecasting System: Parallel database architecture**.

This architecture can handle large volumes of data from sensors and satellites, process it efficiently, and distribute forecasts quickly through web and mobile interfaces.

- **E-Learning Platform: 3-tier architecture**.

This architecture supports the management of course content, tracking of student progress, and interactive features like quizzes and discussion forums, ensuring scalability and efficient data access.

- **Social Media Platform: Distributed database architecture**.

This architecture supports high read and write loads and large amounts of user-generated content, providing fast access to data for users worldwide.

Database Users

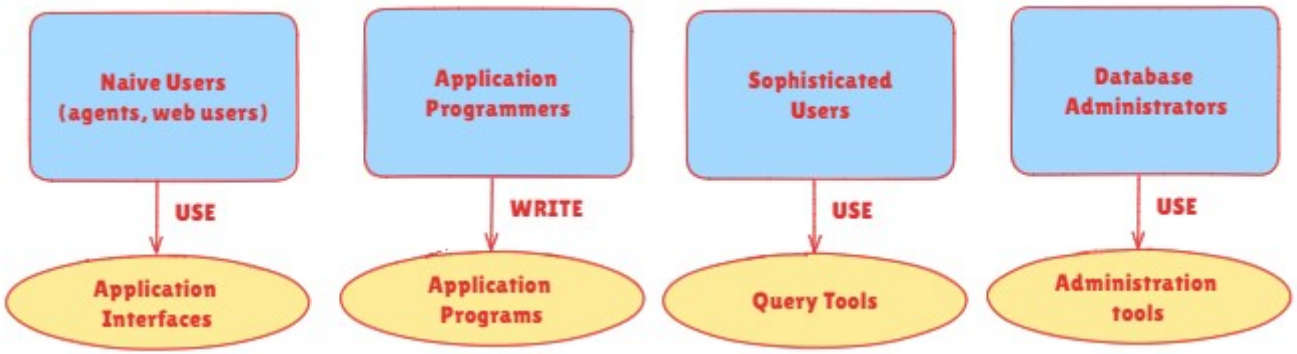
People who work with a database can be categorized as:

1. Database Users
2. Database Administrators.

Database Users

There are four different types of database-system users, differentiated by the way they expect to interact with the system. Different types of user interfaces have been designed for different types of users.

1. Naïve Users
2. Application Programmers
3. Sophisticated Users
4. Database Administrators



Database Users

Naïve Users:

- Naive users also known as **Parametric End users**, don't have any knowledge of databases but still frequently use the database applications to get the desired results. With the help of the interface provided by the DBMS applications.

- **For Example:** Have you ever tried to book a ticket for your favorite movie? It's straightforward and easy.
- But, do you have any idea about the complex database operations happening behind the scenes to make this process smooth and efficient?
- In database terminology, you are a **naive user** or a **parametric end user**. This means you interact with the database through a predefined interface without needing to know anything about how the database works.

Database Users

Application programmers :

- Application Programmers also known as Back-End Developers, are computer professional users who are responsible **for developing the application programs.**
- **Example:** For an e-commerce site, application programmers connect the app to the database, write queries to display products, manage user registrations and orders, handle database errors, and secure user data.

Sophisticated Users:

- Sophisticated users are individuals who possess a deep understanding of database management systems (DBMS) and are proficient in using Data Definition Language (DDL) and Data Manipulation Language (DML) commands.
- **Example:** Data engineers and developers often access databases directly using SQL queries to fetch, delete, update, or insert data, making them sophisticated users.

Database Administrator

One of the main reasons for using DBMSs is to have central control of both the data and the programs that access those data.

Database Administrator is an individual or a team of users who defines the database schema and takes charge of controlling various levels of the database within the organization.

Functions of a Database Administrator (DBA):

1. **Schema definition:** DBA creates the original database schema using data definition statements (DDL).
2. **Storage structure and access-method definition:** DBA specifies parameters for data physical organization and index creation.
3. **Schema and physical-organization modification:** DBA makes changes to reflect organization's needs or enhance performance.
4. **Granting authorization for data access:** DBA regulates user access by granting different types of authorization.
5. **Routine maintenance:** DBA performs periodic backups, ensures disk space availability, and monitors database performance.

Q & A

Questions:

1. "Application programmers are responsible for defining storage structures and access methods in the database.“ – True/False

2. "A _____ user interacts with a system using predefined interfaces and does not need to write code.“

3. "The role of a _____ includes creating the original database schema and managing user access.“

4. "What type of user would most likely use a query language to retrieve specific data from a database?“

Q & A

Questions:

1. "Application programmers are responsible for defining storage structures and access methods in the database.“ – **False (Database Administrators handle this)**

2. "A **Naïve** user interacts with a system using predefined interfaces and does not need to write code.“

3. "The role of **Database Administrator** a includes creating the original database schema and managing user access.“

4. "What type of user would most likely use a query language to retrieve specific data from a database?“ - **Sophisticated User**

Q & A

Some more questions for revision:

1. What are the four main phases of database design? Briefly explain what happens in each phase.
2. What is the role of the **storage manager** in a DBMS? List any three components it manages.
3. What are the three main steps in **query processing**, and why is query optimization important?
4. Define a transaction in DBMS. What are the key properties a transaction must maintain?

Q & A

5. Define the three levels of data abstraction in a DBMS and explain with an example from a university database.
6. What is the difference between *physical* and *logical* data independence? Give one example for each.
7. Differentiate between *database schema* and *database instance* with an analogy from programming.
8. What is a view schema? How does it help with security and user customization?

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Database Design

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgments: Dr. Nagasundari S (Dept of CSE), Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 1: Introduction to Database Management and SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G.

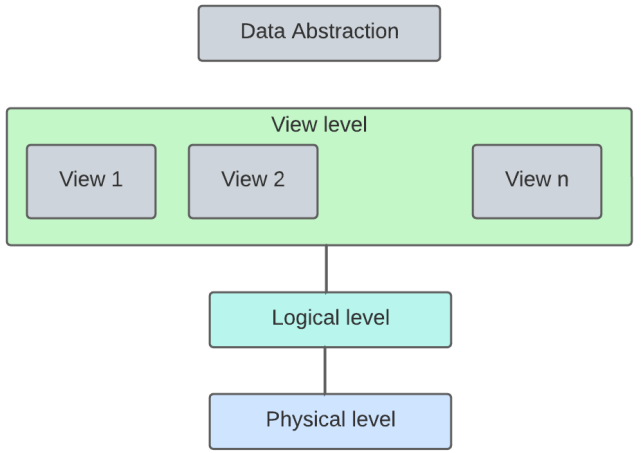
Department of Computer Science and Engineering

Database Management Systems

Data Abstraction

- DBMS plays an important role in many areas. One of the main requirements is that it must retrieve the data efficiently.
- This need for efficiency has led the database developers to use complex data structures to represent data in a database
- Many database-system users are not computer trained and do not know how to write complex queries for getting information they are looking for.
- Hence developer hide all these complexity from the users through several layers of data abstraction, in order to simplify the users' interaction with the system

- The different levels of data abstraction:
  - Physical Level
  - Logical level
  - View level



Data Abstraction

Let us now have a look at what happens at each level:

Physical Level :

- It is the lowest level of data abstraction
- It describes how the data is actually stored in the computer

Logical Level :

- This describes what kind of data is stored in the database and how different pieces of information are related to each other
- The logical level thus describes the entire database in terms of a small number of relatively simple structures.
- This level provides a more organized and simpler view of the data

Data Abstraction

Let us now have a look at what happens at each level:

View Level :

- This is the highest level of abstraction that individual users interact with and that only displays a part of the entire database
- Even though the logical level uses simpler structures, complexity remains because of the variety of information stored in a large database. Many users of the database system do not need all this information; instead, they need to access only a part of the database.
- At this level, different users might see different parts of the database, depending on their needs and permissions

Data Abstraction

Now we can notice that:

- Although the implementation of simple structures at the logical level may involve complex physical-level structures, the user at the logical level does not need to be aware of this.
- Similarly the user who views the database for various purposes need not be aware of the Logical level implementation, for them it would be like an interface and need not worry about the logical view

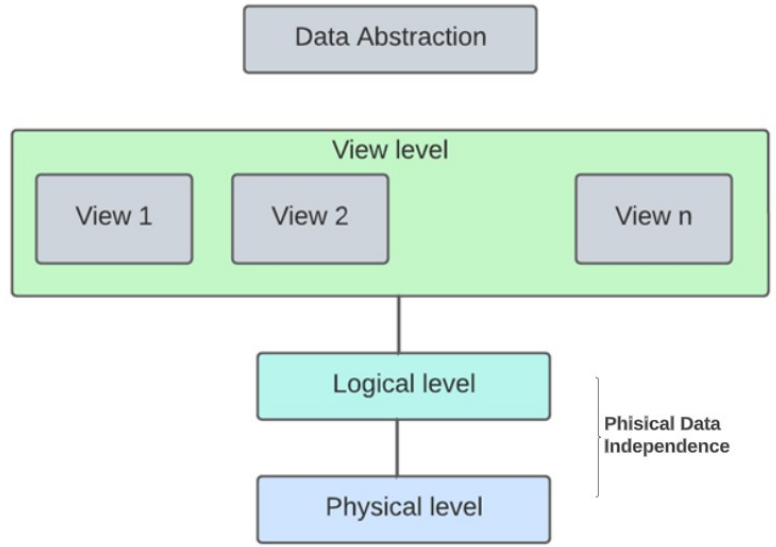
PES UNIVERSITY
CELEBRATING 50 YEARS

PHYSICAL DATA INDEPENDENCE:

- It refers to the characteristic of being able to modify the physical schema without any alterations to the conceptual or logical schema, done for optimization purposes.

Example :

- the Conceptual structure of the database would not be affected by any change in the storage size of the database system server. Changing from sequential to random access files is one such example.
- These alterations or modifications to the physical structure may include:
  - Utilizing new storage devices.
  - Modifying data structures used for storage.
  - Altering indexes or using alternative file organization techniques etc.

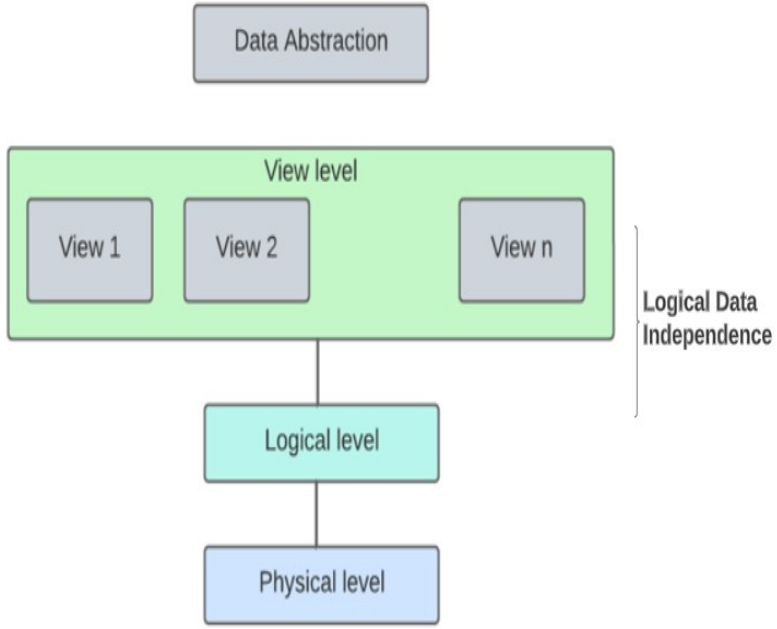


LOGICAL DATA INDEPENDENCE

- It refers characteristic of being able to modify the logical schema without affecting the external schema or application program. The user view of the data would not be affected by any changes to the conceptual view of the data.

Example:

- These changes may include the insertion or deletion of attributes,
- altering table structures entities or relationships to the logical schema, etc.



Database Management Systems

Data Abstraction

Now let us understand this with the help of an example of a University database

- Let the university have the given Record types/ tables as shown

Now for the given database,

- Physical level:
  - Specify how student records, course information, and department data are stored on the hard drive or in memory.
- Logical Level:
  - It would define entities like "Department," "Student," and "Course" and their attributes (e.g., department name, student ID, course code).
  - It also outlines the relationships between these entities, such as "Students belong to Departments" and "Courses are taken by Students.“

Department

Dept_ID	Dept_Name	Building

Course

Course_ID	Course_Name	Credits	Dept_ID

Student

SRN	Name	Dept_ID	Total_Credits

Data Abstraction

Now let us understand this with the help of an example of a University database

- Let the university have the given Record types/ tables as shown

Now for the given database,

- View Level:
  - A faculty member may have access to a view showing only the courses they are teaching and the students enrolled in those courses.
  - A student, on the other hand, may have a view displaying only the courses they are registered for and their respective grades.
  - These views provide a simplified and customized perspective of the database for different users.

Similarly , how could physical and logical data independence be of an advantage in the case of the university database?

Department

Dept_ID	Dept_Name	Building

Course

Course_ID	Course_Name	Credits	Dept_ID

Student

SRN	Name	Dept_ID	Total_Credits

Data Abstraction

Let us consider the situation where the University decides to migrate its database from one storage system to another or optimize the storage layout to improve performance. What would be the consequence of this?

Department

Dept_ID	Dept_Name	Building

Course

Course_ID	Course_Name	Credits	Dept_ID

Student

SRN	Name	Dept_ID	Total_Credits

Without Physical data independence:

- Minor changes to the storage system or hardware upgrades could potentially break existing queries, applications, and interfaces.
- As a result, any update to the storage system would require extensive modifications to the application layer, leading to higher maintenance costs and longer development cycles.

Data Abstraction

Let us consider the situation where the University decides to migrate its database from one storage system to another or optimize the storage layout to improve performance. What would be the consequence of this?

With physical data independence:

- the database administrators can make these changes without altering the logical schema, which means users' queries and applications will continue to work seamlessly.
- It allows for easier maintenance, upgrades, and performance enhancements without disrupting the application layer.

Department

Dept_ID	Dept_Name	Building

Course

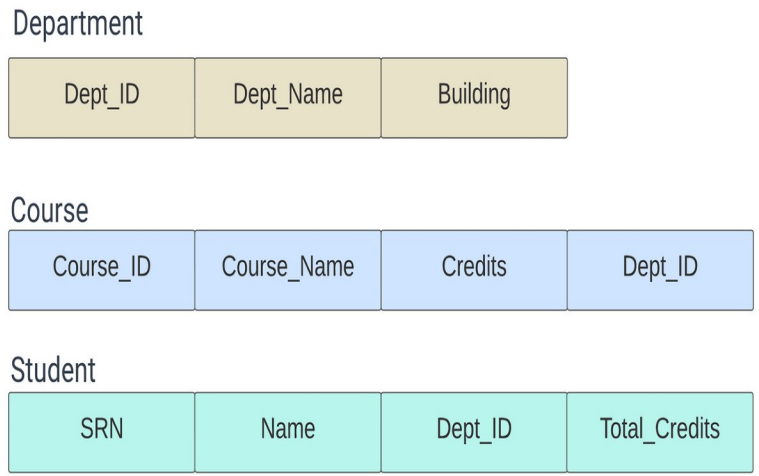
Course_ID	Course_Name	Credits	Dept_ID

Student

SRN	Name	Dept_ID	Total_Credits

Data Abstraction

Let's say the University decides to reorganize its department structure and add new attributes to the "Department" entity. Then what would be the effect?

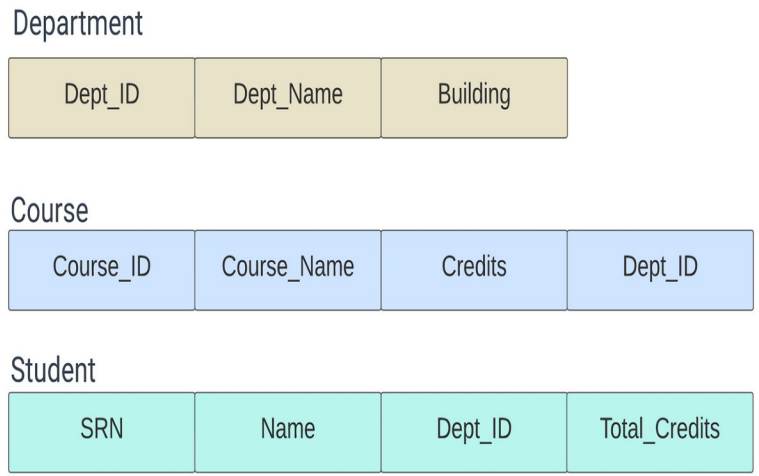


Without Logical data independence:

- Any alteration to the logical schema, such as adding or removing attributes, or changing relationships between entities, would result in changes propagating throughout the applications that rely on the database.
- This would force developers to modify all affected application code to accommodate the changes in the schema.
- Consequently, even minor changes to the data model could lead to widespread disruption and downtime for the applications, increasing the risk of introducing errors and inconsistencies.

Data Abstraction

Let's say the University decides to reorganize its department structure and add new attributes to the "Department" entity. Then what would be the effect?



With Logical data independence:

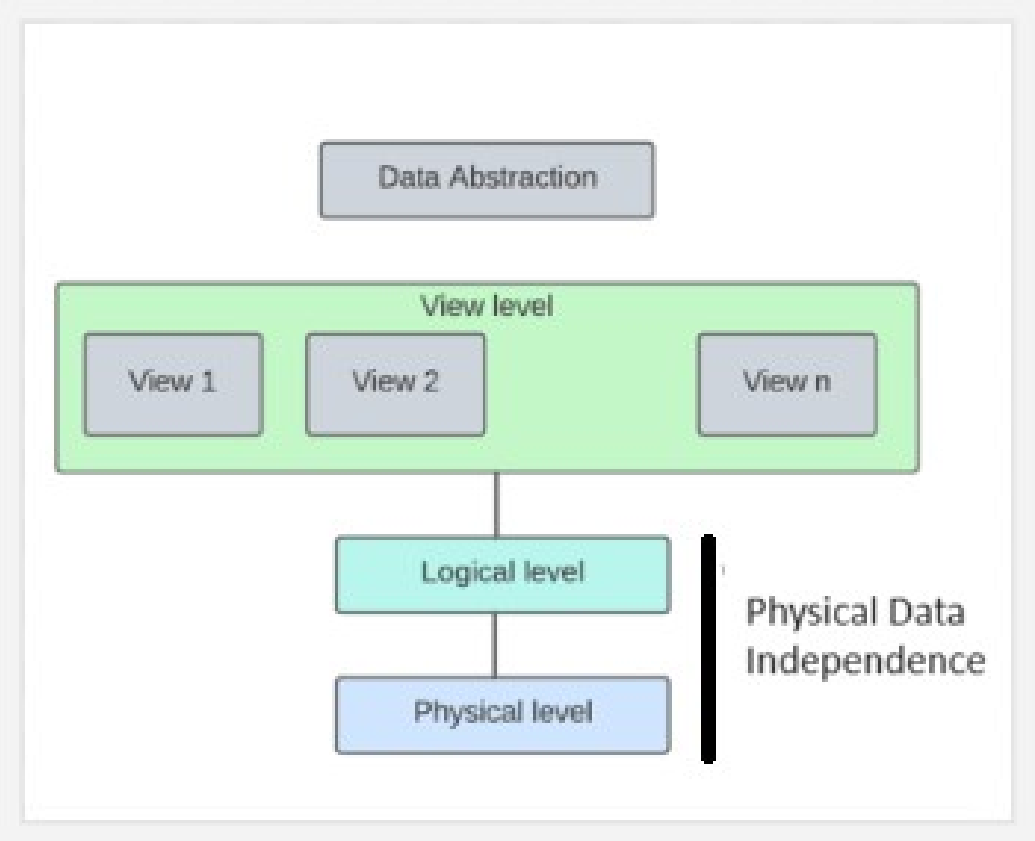
- The database designers can make these modifications without impacting the existing application programs that access the database.
- This flexibility allows for easier adaptability to changes in the University's requirements and business rules without causing application disruptions.

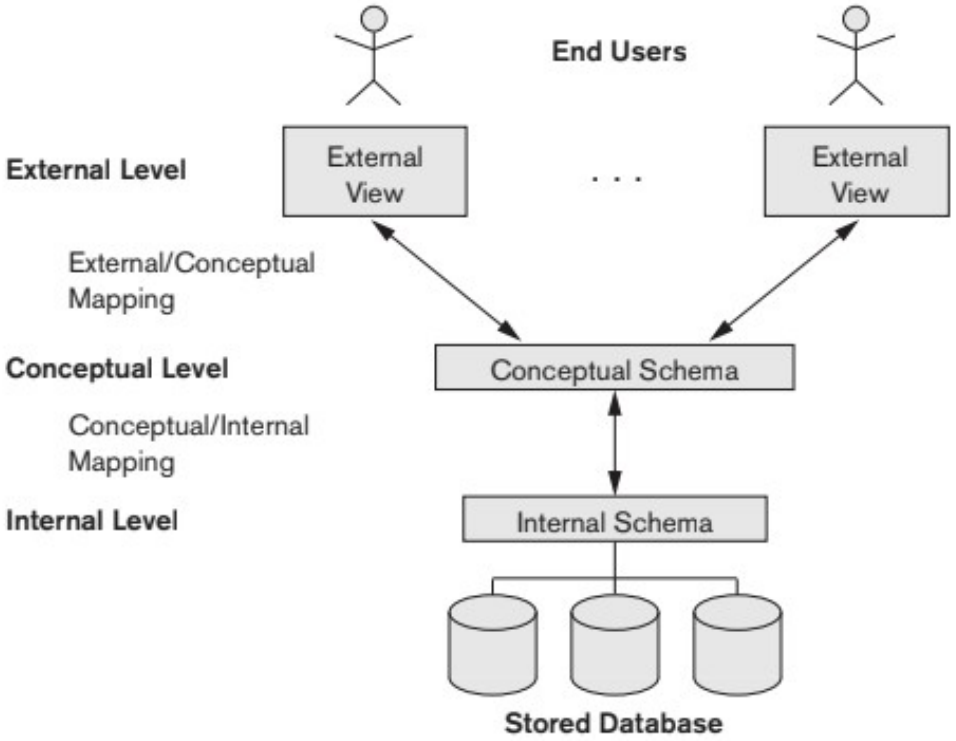
Instances and Schema

- Databases change over time as information is inserted and deleted or modified.
- **The collection of information stored in the database at a particular moment is called an instance of the database.**
- **The overall design of the database is called the database schema.**
- Let us try to understand what that exactly means with an analogy to a program written in any programming language
  - Database Schema => variable declarations along with the associated type definitions
  - Instances => Value of the variable at a particular point in time in the program

Database Management Systems

Three Schema Architecture





Database State

- The actual data in a database may change quite frequently.
- The data in the database at a particular moment in time is called a **database state** or **snapshot**.
- It is also called the *current set* of **occurrences** or **instances** in the database.
- In a given database state, each schema construct has its own current set of instances.
- Many database states can be constructed to correspond to a particular database schema.

Database State V/S Schema

- The distinction between database schema and database state is very important.
- When we **define** a new database, we specify its database schema only to the DBMS.
- At this point, the corresponding database state is the empty state with no data.
- We get the initial state of the database when the database is first **populated** or **loaded** with the initial data.
- From then on, every time an update operation is applied to the database, we get another database state. At any point in time, the database has a *current state*.

Database Management Systems

Database State V/S Schema

A simplified COMPANY relational database schema

EMPLOYEE F.K.

Ename	Ssn	Bdate	Address	Dnumber

P.K.

DEPARTMENT F.K.

Dname	Dnumber	Dmgr_ssn

P.K.

Sample database state for the relational database schema

EMPLOYEE

Ename	Ssn	Bdate	Address	Dnumber
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4
Wallace, Jennifer S.	987654321	1941-06-20	291Berry, Bellaire, TX	4
Narayan, Ramesh K.	666884444	1962-09-15	975 Fire Oak, Humble, TX	5
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1

DEPARTMENT

Dname	Dnumber	Dmgr_ssn
Research	5	333445555
Administration	4	987654321
Headquarters	1	888665555

Why Database Design ?

Scenario:

Imagine you are tasked with designing a database for a new online bookstore. The bookstore needs to manage a large catalog of books, customer information, orders, and reviews. Customers should be able to search for books, place orders, leave reviews, and track their order history.

Question

What factors would you need to consider while designing the database system to ensure it meets all the needs of the online bookstore?

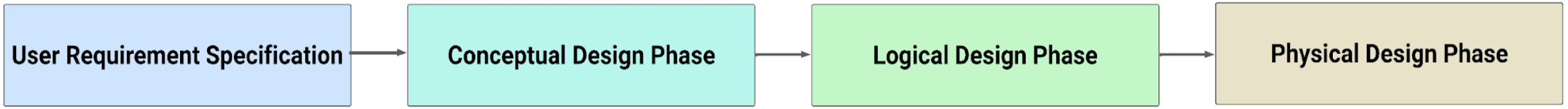
(Hint :

- *Types of data Eg: books, customers, orders, reviews?*
- *How will you ensure data integrity and security?)*

Database Design

- Database design mainly involves the design of the database schema.
- The design of a complete database application environment that meets the needs of the enterprise being modeled requires attention to a broader set of issues.
- A high-level data model gives designers a conceptual framework to specify user data requirements and structure the database accordingly.

The various steps involved in the design process are :



Let us now try and understand what happens at each step:

1. USER REQUIREMENT SPECIFICATION

   • Interact with domain experts and users to fully understand the data needs and functional requirements of the database.

   • Identify the operations or transactions that users will perform on the data.

   • Document and review the user requirements to ensure clarity and accuracy.

2. CONCEPTUAL DESIGN PHASE

   • Choose a high-level data model (e.g., entity-relationship model) to conceptualize the data requirements and relationships.

   • Develop a conceptual schema that provides a detailed overview of the enterprise's data structure.

   • Review the schema to ensure all data requirements are met and remove any redundant features.

3. LOGICAL DESIGN PHASE

- Map the high-level conceptual schema to the implementation data model of the database system that will be used (e.g., relational model).
- Use normalization algorithms to generate a set of tables that properly group attributes and reduce data redundancy.
- Confirm that the logical design meets the functional requirements specified in the user requirements phase.

4. PHYSICAL DESIGN PHASE

- Specify the physical features of the database, including file organization and internal storage structures.
- Consider performance optimization and **choose appropriate indexing** techniques to enhance data retrieval speed.
- Review and refine the physical design to ensure it aligns with the requirements and constraints of the database system and hardware.

Database Management Systems

Typical DBMS Component Modules

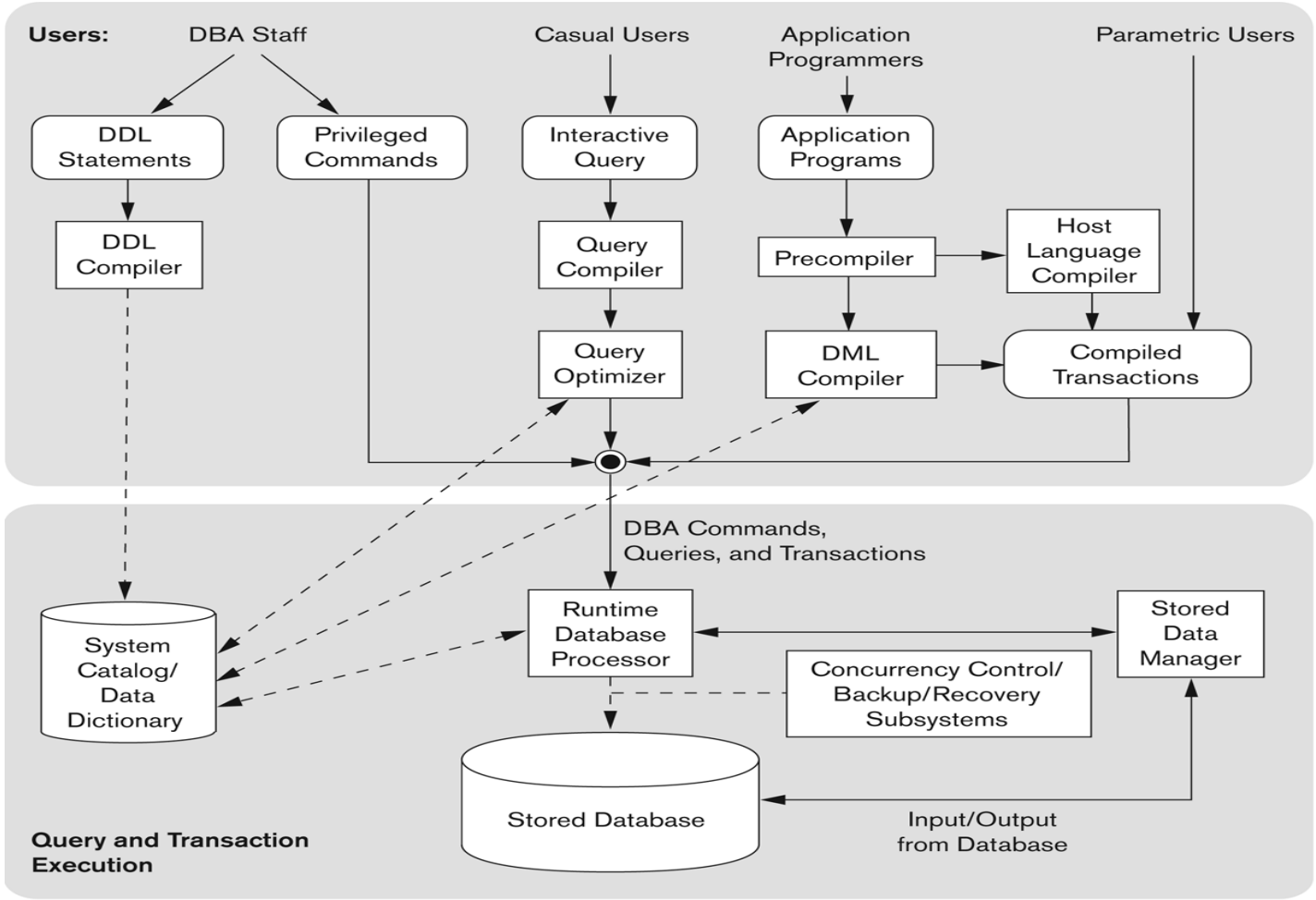


Figure 2.3
Component modules of a DBMS and their interactions.

Typical DBMS Component Modules

- SDM - controls access to DBMS info that is stored on disk (dotted lines)
- DDL Compiler - processes schema definitions specified in the DDL and stores metadata in catalog
- Query Compiler – handles high level queries - parses and compiles a query and calls to runtime processor for execution
- Pre Compiler – extracts DML commands from application program
- Run time db processor – executes privileged commands/queries/canned transactions.

Database System Utilities

- Loading utility – Load existing data files into a database. Includes data conversion tools from old format to new db format
- Backup utility - Back up of the database periodically. (Incremental/Total)
- Database storage reorganization - Reorganizing database file structures and create new access paths to improve performance.
- Performance monitoring utilities. – monitors db usage and provides statistics to DBA

PES UNIVERSITY
CELEBRATING 50 YEARS

Scenario:

Imagine you are responsible for the database system of a popular online multiplayer game. The game has thousands of players online simultaneously, from different parts of the world. Each player needs real-time updates for their game stats, friends list, and in-game purchases.

Question:
How would you design the database system to ensure efficient, secure, and real-time data access for all players?

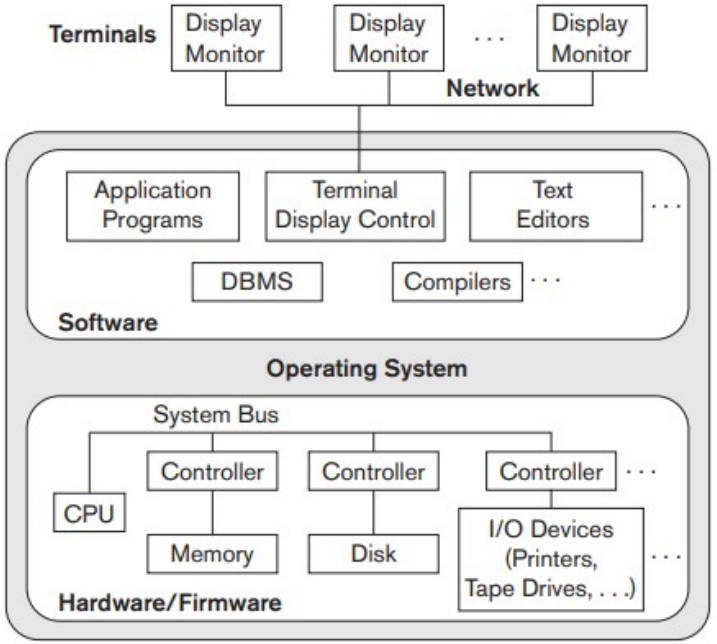
- **Centralized databases (Centralized architecture)**

Characteristics:

- **Single Location:** All data is stored and managed on a central server or database.
- **Shared Memory:** Suitable for systems with multiple CPUs that access a common shared memory space.
- **Resource Allocation:** Resources like CPU, memory, and storage are centralized, making it easier to manage and maintain.

Use Cases:

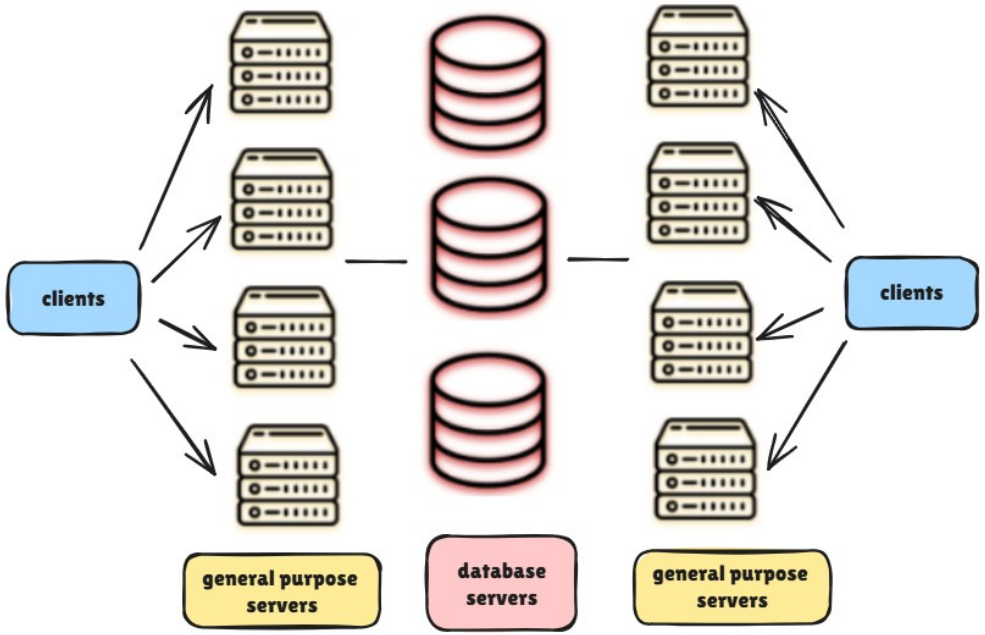
- **Small to Medium-Sized Applications:** Suitable for applications with moderate data needs and user loads.
- **Legacy Systems**: Often used in older systems where scaling is not as critical or feasible.



Centralized/Shared Memory DB

Client Server Architecture:

- Client-server architecture is a model where a client (the user or application) interacts with a server (the database system) to access and manage data.
- The client is the user-facing component that interacts with the server to request data or perform operations. (web browser, mobile apps, desktop applications)
- The server is the backend component that manages the database, processes requests from clients, and handles data storage and retrieval. (MySQL, Oracle, Postgres)



Database Architecture

Parallel databases:

- Designed to run on a cluster of multiple machines
- Enables better scalability and higher processing capabilities.
- Many cores shared memory and Shared Disk

Parallel databases are ideal for **high-performance applications** requiring **extensive data processing**, such as large-scale data warehousing, real-time analytics, and **high-throughput** transactional systems.

Example: How do you think NASA manages and processes the vast amounts of satellite data it collects?
Answer: NASA uses parallel databases and high-performance computing (**HPC**) systems. NASA can efficiently handle and analyze massive datasets from satellite missions, enabling quick and insightful analysis for complex tasks like climate modeling and environmental monitoring.

NASA's HPC Cluster

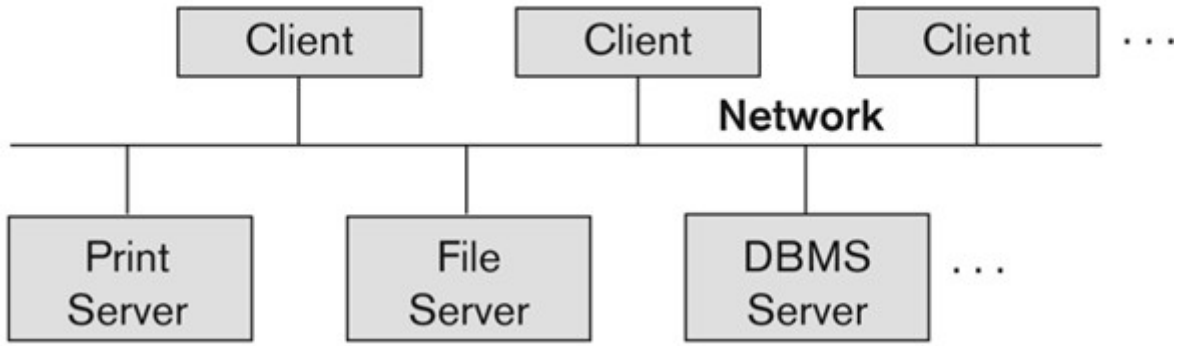
Distributed Database:

A distributed database is a database system in which data is stored across multiple physical locations. These locations can be spread across various machines within a single data center or across different geographic locations.

Advantages:

- **Scalability:** Can handle large amounts of data and high traffic by distributing the load across multiple nodes.
- **Fault Tolerance:** Provides high availability and fault tolerance through data replication and distribution.
- **Performance:** Can improve performance by distributing queries and data access across multiple servers.

Two-Tier Client Server Architectures



In a two-tier database architecture, there are two main components: the client (**user interface**) and the server (**database**).

- **Client:** This is the user interface where users interact with the application. It can be a desktop application or a web-based interface.
- **Server:** This is where the database is stored and managed. The server handles data storage, retrieval, and management tasks.
- Client programs send query and transaction requests via ODBC or JDBC API, the server processes these requests and sends the results back to the client, which then processes and displays the results.

Two-Tier Client Server Architectures

Clients

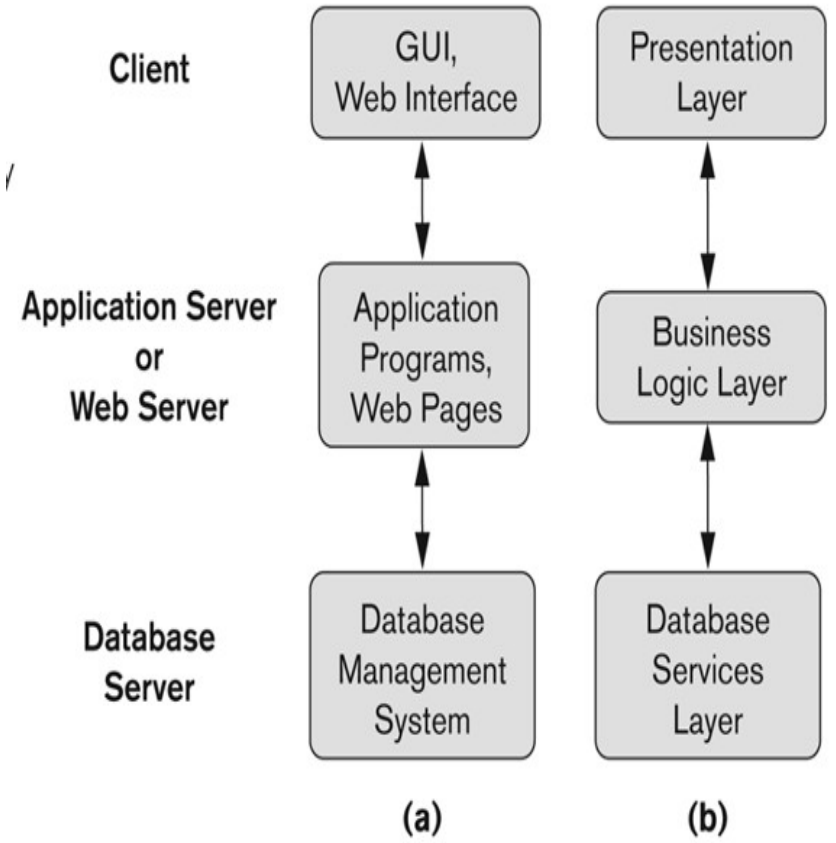
- Provide appropriate interfaces through a client software module to access and utilize the various server resources.
- Clients may be diskless machines or PCs or Workstations with disks with only the client software installed.
- Connected to the servers via some form of a network.
- (LAN: local area network, wireless network, etc.)

Servers

- Provides database query and transaction services to the clients
- Relational DBMS servers are often called SQL servers, query servers, or transaction servers
- Applications running on clients utilize an Application Program Interface (**API**) to access server databases via standard interface such as:
- ODBC: Open Database Connectivity standard
- JDBC: for Java programming access
- Client and server must install appropriate client module and server module software for ODBC or JDBC

Three-Tier Client Server Architectures

- Common for Web applications
- Intermediate Layer called Application Server or Web Server:
  - Stores the web connectivity software and the business logic part of the application used to access the corresponding data from the database server
  - Acts like a conduit for sending partially processed data between the database server and the client.
- Three-tier Architecture Can Enhance Security:
  - Database server only accessible via middle tier
  - Clients cannot directly access database server



Questions

Scenarios for Architecture Selection:

1. **Weather Forecasting System:** A weather forecasting system collects data from various sensors and satellites, processes the data to generate forecasts, and distributes the information to users through a web interface and mobile application.

2. **E-Learning Platform:** An e-learning platform offers online courses to students around the world. The platform needs a database to manage course content, track student progress, and support interactive features such as quizzes and discussion forums.

3. **Social Media Platform:** A social media platform with millions of users worldwide needs a database system that can handle high read and write loads, store large amounts of user-generated content, and provide fast access to data.

Scenarios for Architecture Selection: Solutions -

- **Weather Forecasting System: Parallel database architecture**.

This architecture can handle large volumes of data from sensors and satellites, process it efficiently, and distribute forecasts quickly through web and mobile interfaces.

- **E-Learning Platform: 3-tier architecture**.

This architecture supports the management of course content, tracking of student progress, and interactive features like quizzes and discussion forums, ensuring scalability and efficient data access.

- **Social Media Platform: Distributed database architecture**.

This architecture supports high read and write loads and large amounts of user-generated content, providing fast access to data for users worldwide.

Database Users

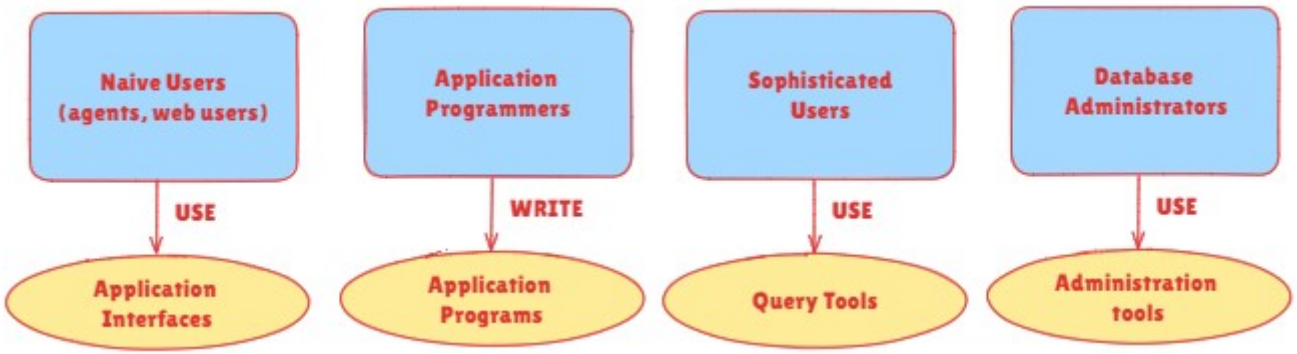
People who work with a database can be categorized as:

1. Database Users
2. Database Administrators.

Database Users

There are four different types of database-system users, differentiated by the way they expect to interact with the system. Different types of user interfaces have been designed for different types of users.

1. Naïve Users
2. Application Programmers
3. Sophisticated Users
4. Database Administrators



Database Users

Naïve Users:

- Naive users also known as **Parametric End users**, don't have any knowledge of databases but still frequently use the database applications to get the desired results. With the help of the interface provided by the DBMS applications.

- **For Example:** Have you ever tried to book a ticket for your favorite movie? It's straightforward and easy.
- But, do you have any idea about the complex database operations happening behind the scenes to make this process smooth and efficient?
- In database terminology, you are a **naive user** or a **parametric end user**. This means you interact with the database through a predefined interface without needing to know anything about how the database works.

Database Users

Application programmers :

- Application Programmers also known as Back-End Developers, are computer professional users who are responsible **for developing the application programs.**
- **Example:** For an e-commerce site, application programmers connect the app to the database, write queries to display products, manage user registrations and orders, handle database errors, and secure user data.

Sophisticated Users:

- Sophisticated users are individuals who possess a deep understanding of database management systems (DBMS) and are proficient in using Data Definition Language (DDL) and Data Manipulation Language (DML) commands.
- **Example:** Data engineers and developers often access databases directly using SQL queries to fetch, delete, update, or insert data, making them sophisticated users.

Database Administrator

One of the main reasons for using DBMSs is to have central control of both the data and the programs that access those data.

Database Administrator is an individual or a team of users who defines the database schema and takes charge of controlling various levels of the database within the organization.

Functions of a Database Administrator (DBA):

1. **Schema definition:** DBA creates the original database schema using data definition statements (DDL).
2. **Storage structure and access-method definition:** DBA specifies parameters for data physical organization and index creation.
3. **Schema and physical-organization modification:** DBA makes changes to reflect organization's needs or enhance performance.
4. **Granting authorization for data access:** DBA regulates user access by granting different types of authorization.
5. **Routine maintenance:** DBA performs periodic backups, ensures disk space availability, and monitors database performance.

Q & A

Questions:

1. "Application programmers are responsible for defining storage structures and access methods in the database.“ – True/False

2. "A _____ user interacts with a system using predefined interfaces and does not need to write code.“

3. "The role of a _____ includes creating the original database schema and managing user access.“

4. "What type of user would most likely use a query language to retrieve specific data from a database?“

Q & A

Questions:

1. "Application programmers are responsible for defining storage structures and access methods in the database.“ – **False (Database Administrators handle this)**

2. "A **Naïve** user interacts with a system using predefined interfaces and does not need to write code.“

3. "The role of **Database Administrator** a includes creating the original database schema and managing user access.“

4. "What type of user would most likely use a query language to retrieve specific data from a database?“ - **Sophisticated User**

Q & A

Some more questions for revision:

1. What are the four main phases of database design? Briefly explain what happens in each phase.
2. What is the role of the **storage manager** in a DBMS? List any three components it manages.
3. What are the three main steps in **query processing**, and why is query optimization important?
4. Define a transaction in DBMS. What are the key properties a transaction must maintain?

Q & A

5. Define the three levels of data abstraction in a DBMS and explain with an example from a university database.
6. What is the difference between *physical* and *logical* data independence? Give one example for each.
7. Differentiate between *database schema* and *database instance* with an analogy from programming.
8. What is a view schema? How does it help with security and user customization?

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Entity-Relationship Model

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgments: Dr. Nagasundari S (Dept of CSE), Teaching Assistants

PES UNIVERSITY

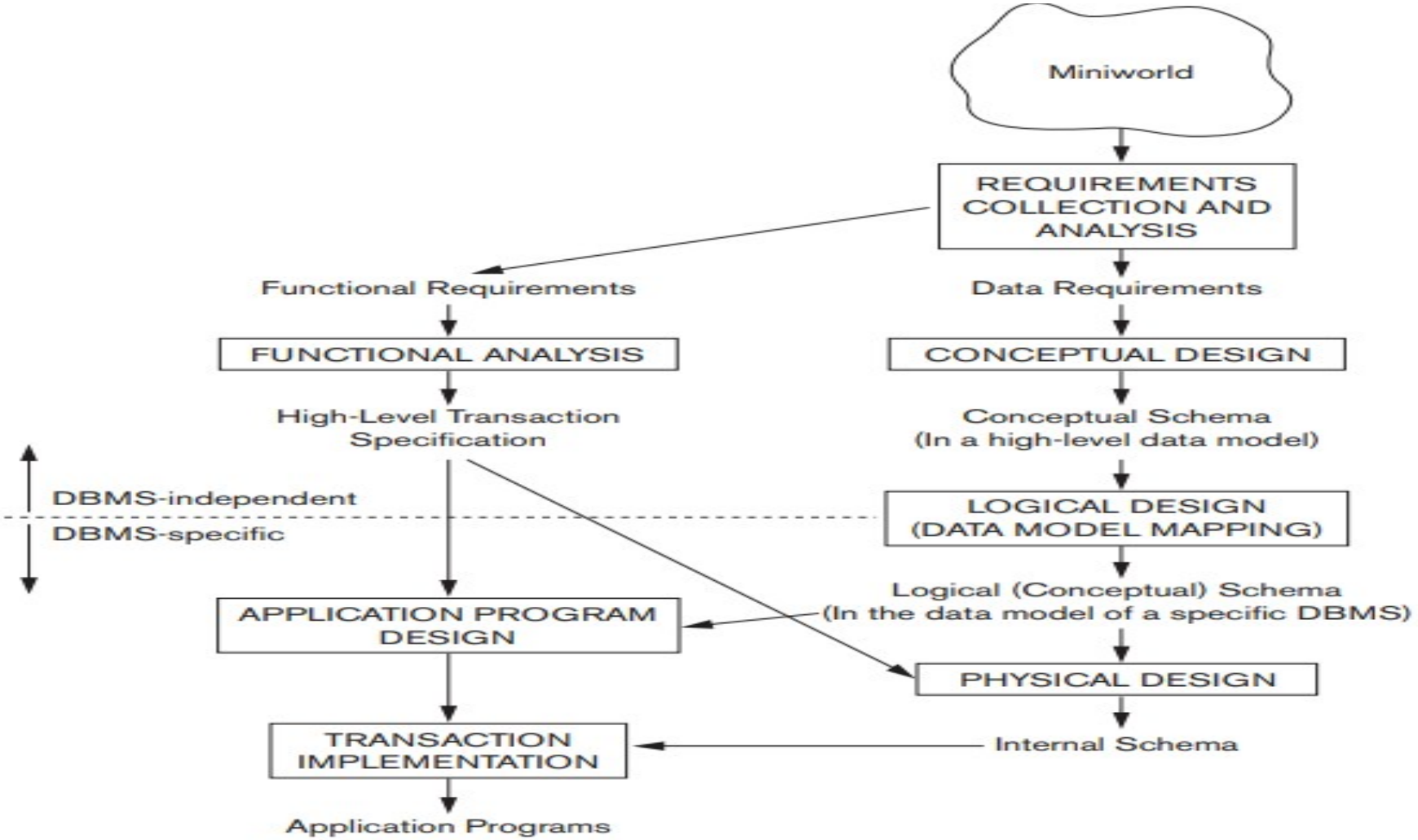
Database Management Systems

Unit 1: Introduction to Database Management and SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G.

Department of Computer Science and Engineering


Miniworld
REQUIREMENTS COLLECTION AND ANALYSIS
Functional Requirements
Data Requirements
FUNCTIONAL ANALYSIS
CONCEPTUAL DESIGN
High-Level Transaction Specification
Conceptual Schema (In a high-level data model)
DBMS-independent
DBMS-specific
LOGICAL DESIGN (DATA MODEL MAPPING)
Logical (Conceptual) Schema (In the data model of a specific DBMS)
APPLICATION PROGRAM DESIGN
PHYSICAL DESIGN
TRANSACTION IMPLEMENTATION
Internal Schema
Application Programs

E-R model

The Entity-Relationship (E-R) model is a data modeling technique designed to assist in database design.

It enables the creation of an enterprise schema that captures the logical structure of a database.

Purpose and Function: The E-R model aids in representing real-world entities, their meanings, and interactions within a conceptual schema for database design.

E-R model

- E-R diagrams provide a graphical representation of the database's logical structure. They use symbols to depict entities, relationships, and attributes, making complex relationships easier to understand.
- E-R diagrams are valued for their simplicity and clarity, making them a widely used tool in database design.
- By mapping out the entities, attributes, and relationships before implementation, E-R **Diagrams help in identifying potential issues, redundancies, or inconsistencies in the database design**. This leads to more efficient and optimized database structures, reducing the need for costly modifications later.
- E-R Diagrams serve as a comprehensive documentation tool that records the structure of the database.

Entity

- **What is an Entity?**
  - An entity is a distinct "thing" or "object" in the real world.
  - It's different from other objects and can be a person, course, or more.

- **Properties of an Entity:**
  - An entity has specific attributes or properties.
  - Some properties must hold unique values to tell entities apart.
- **Example - Student Entity:**
  - In a university, each student is an entity.
  - A unique “Studnt_ID" property distinguishes individuals.

Entity

- **Unique Identification of entity:**
  - Unique property values identify each entity.
  - For instance, a student’s “student_id" value like PES001 makes them distinct.
  - Courses can also be entities in a university setting.
  - A "course_id" attribute uniquely identifies each course.
- **Concrete and Abstract Entities:**
  - Entities can be concrete (like people or books) or abstract (like courses or reservations).
  - Both types serve as distinct objects in the system.

Entity Sets

- **Entity Set (or Entity Collection):**
  - An entity set is a group of entities of the same kind sharing common attributes or properties.
  - For instance, "instructor" represents all university instructors, and "student" represents all university students.

E1 is an entity having Entity Type Student and the set of all students is called Entity Set.

- **Extension vs. Entity Set:**
  - The real collection of entities is the "extension" of an entity set.
  - Similar to the difference between a relation and a relation instance.
- **Non-Disjoint Entity Sets:**
  - Entity sets can overlap these sorts of entity sets are called non-disjoint sets.
  - Example: "person" entity set includes instructors, students, both, or neither.

Entity Sets

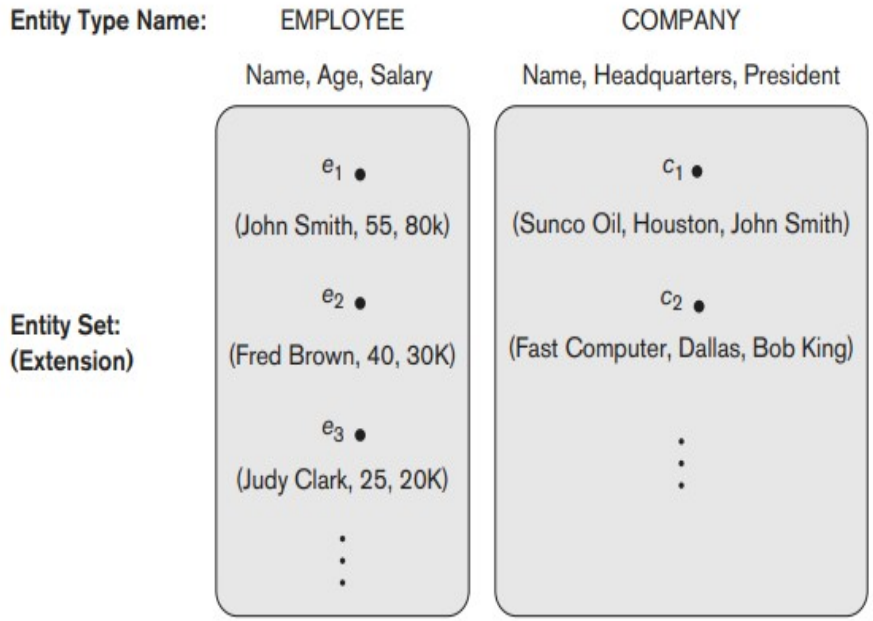
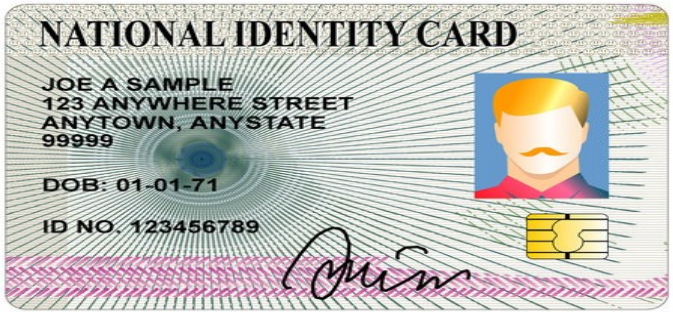


Figure 3.6
Two entity types, EMPLOYEE and COMPANY, and some member entities of each.

Attributes

- An entity's attributes provide distinct values for identification.
  - For instructors, the ID attribute stands out as a unique identifier.
- It prevents confusion when multiple instructors share the same name.
- How do we decide on the unique identifier for the case of Instructors?
  - Government-issued IDs were historically used, but they raise security and privacy concerns.
  - **Enterprises opt to establish their own identifiers to ensure security and privacy**.



Attributes

- Entities are represented by specific attributes.
- Attributes are qualities or characteristics of each entity.
- They **signify the type of information stored for each entity**.
- Every entity possesses its own set of values for attributes.
- Example: Instructors have unique values for attributes like ID, name, dept name, and salary.

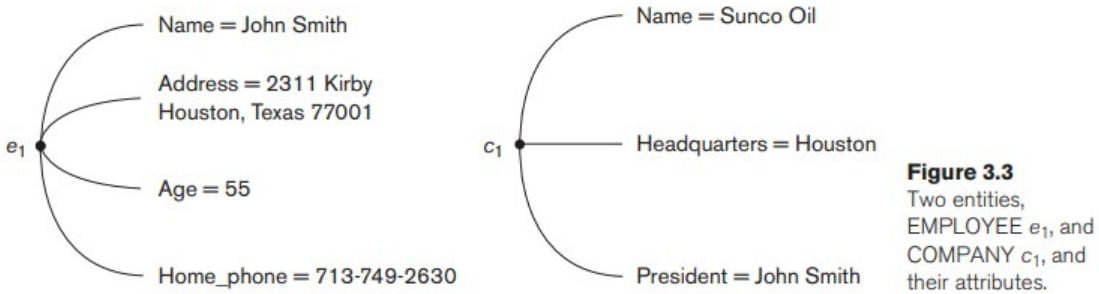


Figure 3.3
Two entities, EMPLOYEE e_1, and COMPANY c_1, and their attributes.

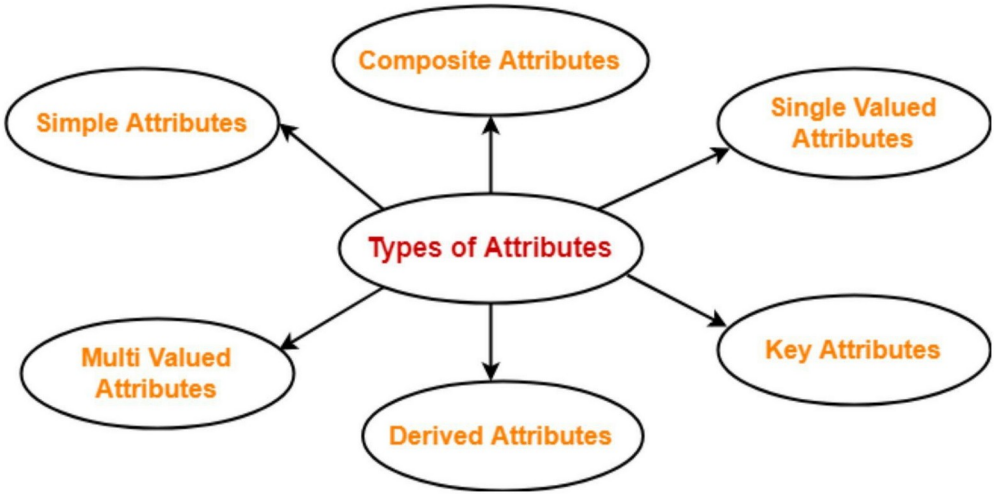
Value Set (Domain)

- For each attribute there are a set of permitted values that can be taken which is called DOMAIN or VALUE SET of that attribute.
  - If the range of ages allowed for employees is between 16 and 70, we can specify the value set of the Age attribute of EMPLOYEE to be the set of integer numbers between 16 and 70.
  - We can specify the value set for the Name attribute to be the set of strings of alphabetic characters separated by blank characters, and so on.

- Mathematically, an attribute A of entity set E whose value set is V can be defined as a function from E to the power set6 P(V) of V:
  $A : E \rightarrow P(V)$

Types of Attributes

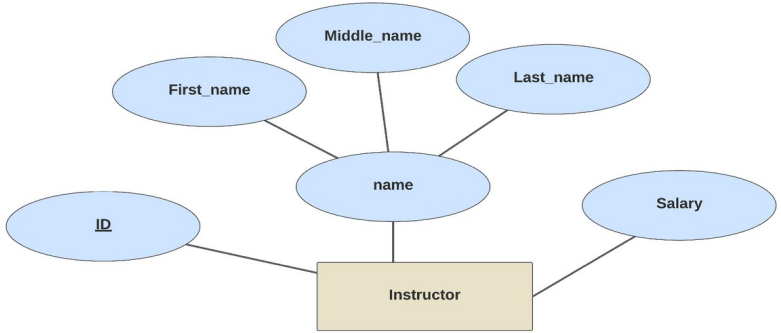
- An attribute used in E-R model, can be characterized as:
  - **Simple / Composite attribute**
  - **Single-valued / Multivalued attributes**
  - **Derived attributes**



Simple and Composite Attribute

- Formally saying the attributes that cannot be further subdivided into components is a **simple attribute**
  - Ex: The ID of the student or Instructor, Roll number of the student, phone number, email id, etc.

- On the contrary those attributes that can be split into components are called a **composite attribute**
  - Ex: The address can be further split into house number, street number, city, state, country, and pin code,
  - The name can also be split into first name middle name, and last name.



Composite Attributes

- Let us consider the university database. Like any other organization, the management would keep a record of all the addresses of the student, instructors and the staff

- Now address would contain the entire street, city, state, and postal_code information.
- Let us say the address of the student is :
  - "123 Main Street, Yeshwanthpur, Bangalore, 560063"

- Now this is how all the information is stored in one attribute.
  - Let us say we would want to group students through the postal code for allotting the busses. How do we do it?
  - Currently the postal code is not an attribute alone or stored specifically as an attribute so we cannot get the postal code alone as it is a part of the address

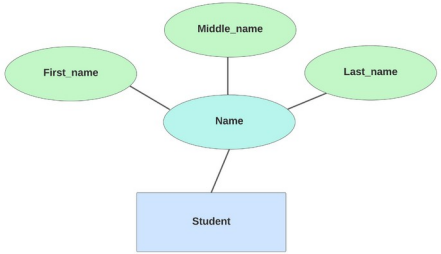
- Thus just having the address stored as one attribute is not always appropriate, especially when we would want to retrieve students on a part of the address.

Composite Attributes

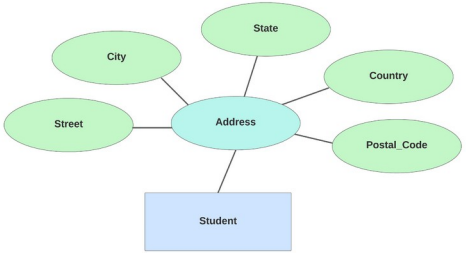
- For solving this issue, we try to store these types of attributes by dividing the attribute into subparts.
- These attributes which can be divided into subparts are called '**Composite attributes**'.
- Using composite attributes in a design schema is a good choice if a user will wish to refer to an entire attribute on some occasions, and to only a component of the attribute on other occasions.

Representation of the Composite attribute

- Let us consider the examples discussed above, and how are they represented it in the E-R diagram
  - Name could be subdivided into First Name, Middle Name, Last Name

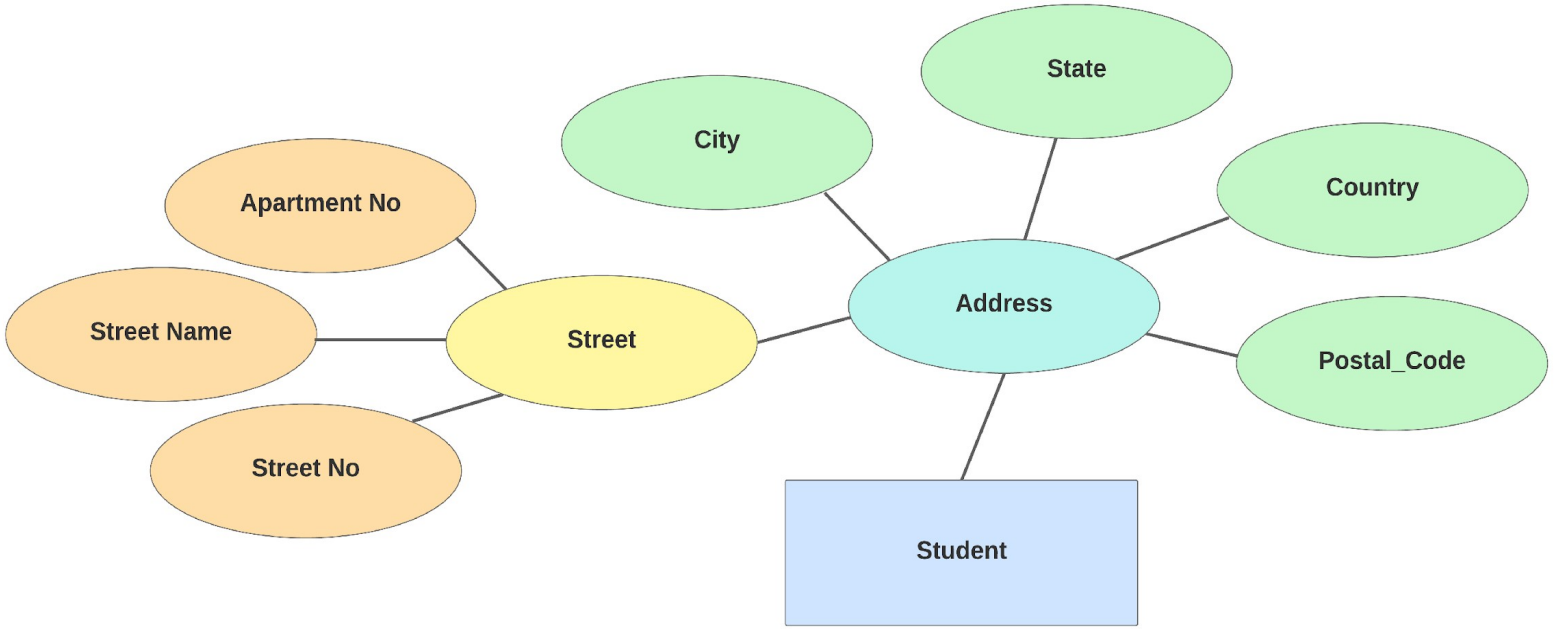


  - The address can be further split into street, city, state, country, and pin code



Database Management Systems

Representation of the Composite attribute



Multivalued Attributes

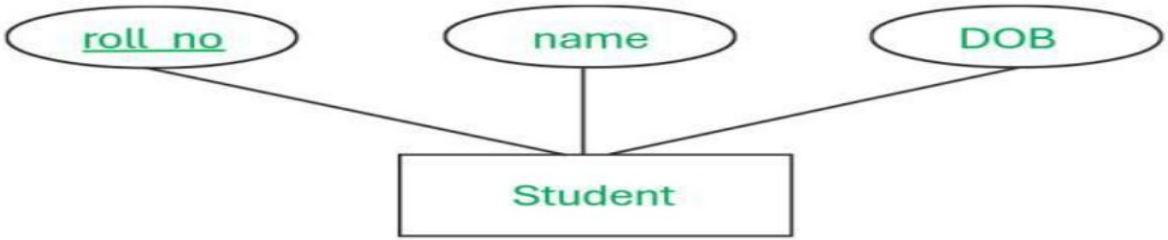
- Suppose the university has decided to add the phone number to the instructor relation.
- An instructor may have Zero, One, or Several phone numbers, and different instructors may have different number of phone numbers.
- Same is the case with the instructor's mail_id.
- Till now all the attributes we have studied were having only a single value and not multiple values. So how do we deal with these type of situation ?

- These types of attributes which can have multiple values are called as **multivalued Attributes**

- Examples for the same are :
  - Instructor Phone numbers
  - Instructors mail_id

Single-valued and Multivalued Attributes

- The attribute which takes up only a single value for each entity instance is a **single-valued attribute**.
  - Example: The name of the instructor, age of the student, etc.



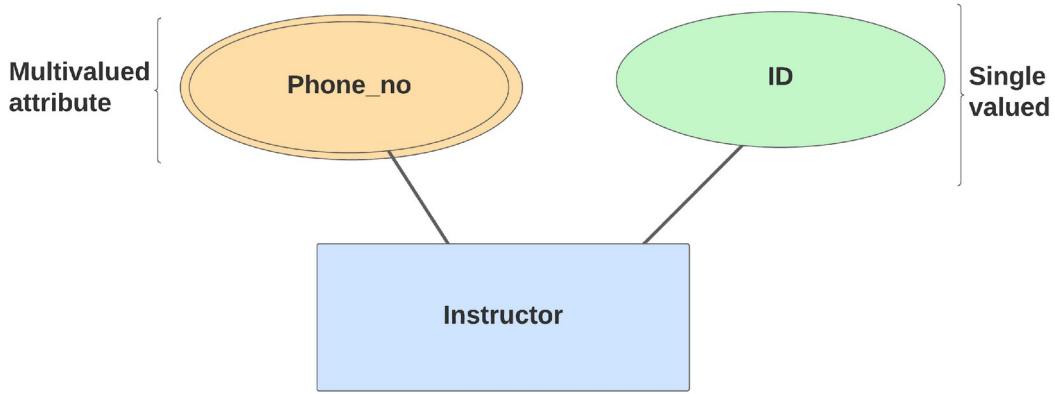
Single-Valued

- The attribute which takes up more than a single value for each entity instance is a **multi-valued attribute**.
  - Example: Phone number of the instructor, Mail id, etc.

Representation of Multivalued Attributes

- The multivalued attribute is represented in the E-R diagram as shown below:
- The multivalued attribute is represented by a double ellipse.

- Considering the instructor's Phone_no as a multivalued attribute and ID as an single-valued attribute:



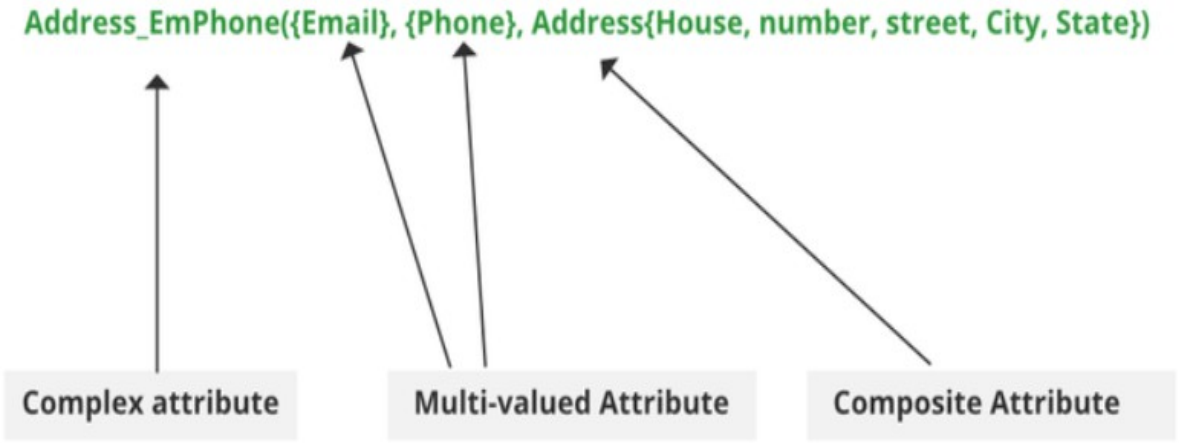
Complex Attributes

- Those attributes, which can be formed by the nesting of composite and multi-valued attributes, are called "**Complex Attributes**".
- These attributes are rarely used in DBMS.

- Let us take an example of the same.
  - Let us consider that the student relation contains an attribute called **PreviousDegrees** that stores all the previous education qualifications of the students
  - PreviousDegrees of a STUDENT is a composite multi-valued attribute denoted by **{PreviousDegrees (College, Year, Degree, Field)}**
  - Multiple PreviousDegrees values can exist. Each has four subcomponent attributes: College, Year, Degree, Field
  - Therefore it is a complex attribute

Question

Given ,an attribute Address_EmPhone consisting of email, phone and Address. Can you identify which are the different attribute types that make up this attrib



Complex attributes

Derived Attributes

- Consider the following schema for the Instructor relation
  - **Instructor** (ID, Name, Department, Date_of_Birth, Age)

- Can you identify any problem in the above schema?
- If you have noticed, we know that **the age can be calculated from the Date of birth**, i.e. we can take the system's current date and the Date of birth of the instructor and we can find the age.

- So there is no need to store this information in the actual database and can be calculated.

Stored and Derived Attributes

- So as shown above, the value of the age attribute can be derived from the date of Birth.

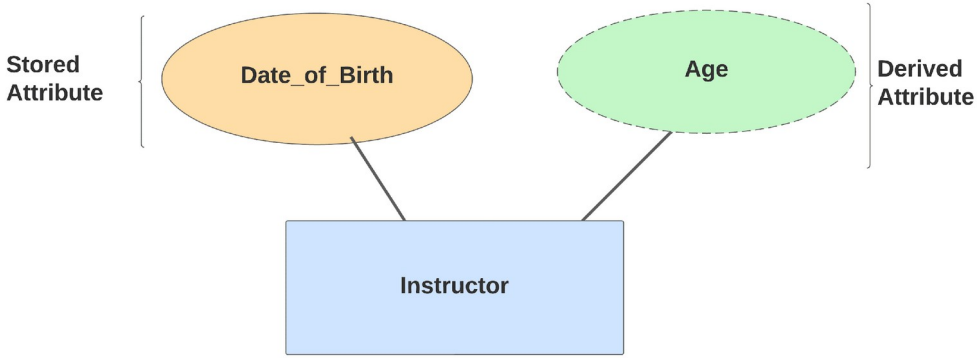
- Therefore the Age attribute is called a Derived Attribute
- And the Date_of_Birth attribute is called as a stored attribute.

- The **stored attribute** are those attributes that are actually stored in the database.
  - Example: Date_of_Birth is a stored attribute

- An attribute that can be derived from other attributes is **derived attributes**.
  - Example: the Age attribute of the instructor is a derived attribute

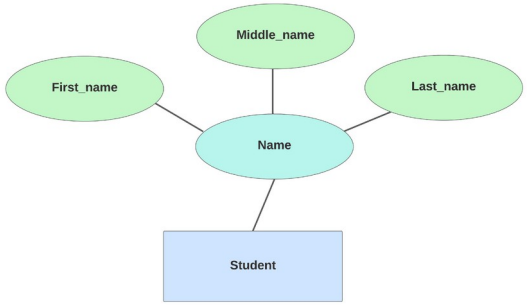
Representation of Derived Attribute

- Considering the above example of the instructor relation, Date_of_Birth is a stored attribute and Age is an derived attribute
- In the below E-R Diagram, Age (derived attribute) is represented by a dashed oval.
- The E-R representation would be:



Null Values

- Let us consider the diagram representation shown:
- What if the student doesn’t have a middle name?



- In such situations the value of the middle name would be set to NULL

Null Values

- An attribute takes **a null value when an entity does not have a value for it.**
- So what does it mean if an attribute is NULL?

- The Null value may indicate "**not applicable**"—that is, the value does not exist for the entity.
- Null can also designate that an attribute value is **unknown**. **An unknown value may be either:**
  - **missing (the value does exist, but we do not have that information)**
  - **not known (we do not know whether or not the value actually exists).**

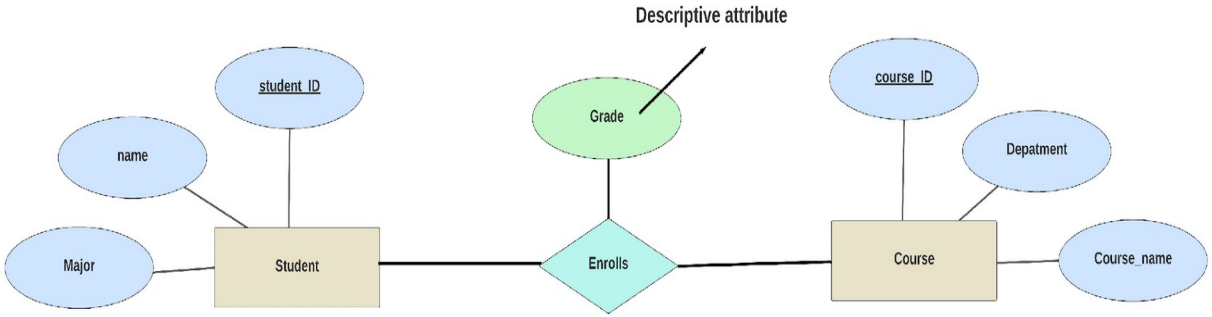
- A null value for the apartment number attribute could mean:
  - Address does not include an apartment number (not applicable),
  - That an apartment number exists but we do not know what it is (missing),
  - We do not know whether or not an apartment number is part of the instructor's address (unknown).

Descriptive Attributes

- Let us now consider the scenario **that every student enrolls for certain courses.**
- Now as we know along with *what each student enrolls for which course, the university must also keep track of the grades that the student would score for that particular course.*
- Now **where do we place this grade attribute**? In the Student entity or the Courses entity?
- Actually if we observe this attribute is associated with the relation 'enroll' i.e. only if the student enrolls for the particular course, he can get a grade. Such type of attributes are called **Descriptive attributes**
- So it wouldn't be correct to place it in any of the entities
- Then **where do we place them?**

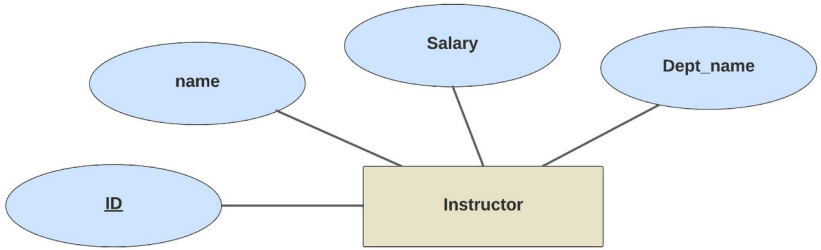
Descriptive Attributes

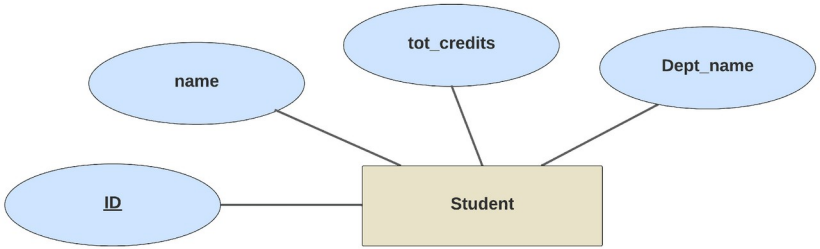
- A relationship may also have attributes called **descriptive attributes.**
- An attribute of a relationship set is represented in an E-R diagram by an **oval connected to the relationship's diamond**
- The below diagram shows the discussed example in the form of an ER diagram



Redundant attributes

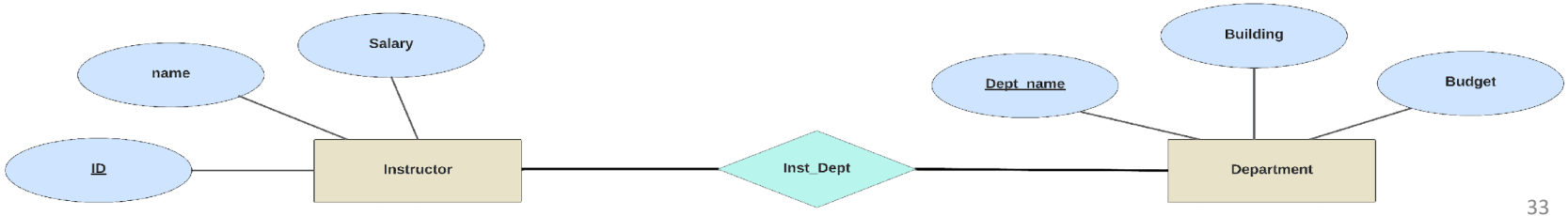
- In the university organization, consider the following entity sets:
  - Student
  - Instructor
- Attributes are selected to capture values in the database;
  - Instructors: (ID, name, dept_name, salary)
  - Students: (ID, name, dept_name, tot cred)
- **Additional attributes like phone number, office number are optional based on the designer's understanding.**





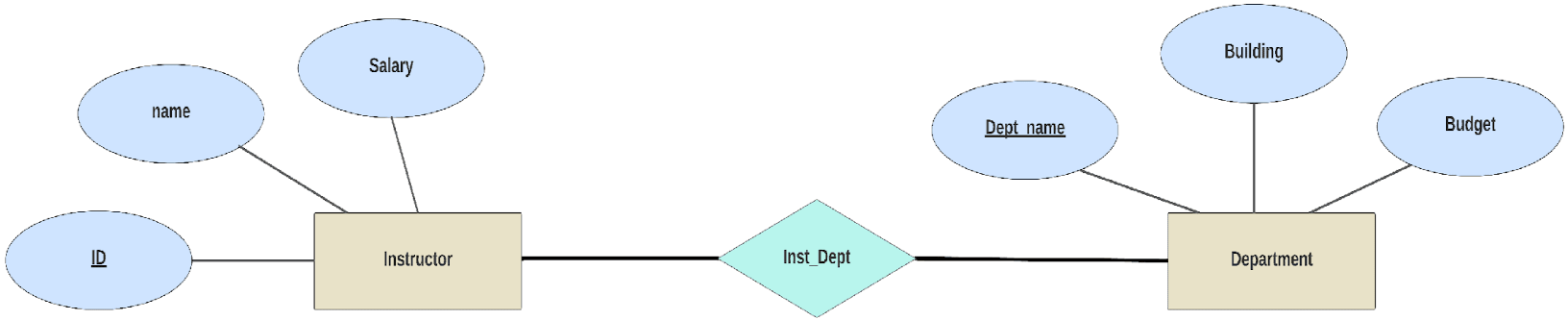
Redundant attributes

- Consider the entity department in the university database:
  - Department (dept_name, building, budget)
- Relationship sets connect entities together, like instructor and department.
- **Redundancy can occur due to relationship sets, leading to attribute removal for clarity.**
- The dept_name attribute, redundant in both instructor and department, is removed.
- Instead we represent this using a relationship set (inst_dept) between instructor and department.



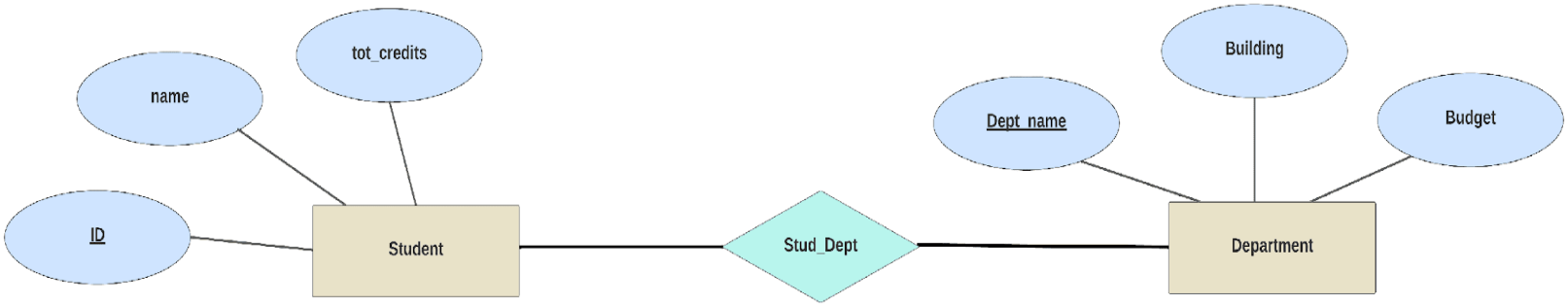
Redundant attributes

- *Redundant dept name attribute in instructor is removed since it's a primary key in the department table.*
- In a relational schema, dept_name may be reintroduced in the instructor's relation if each instructor has one department.
- If instructors have multiple departments, a separate relation (inst_dept) records these relationships.
- Treating instructor-department connection as a relationship prevents assumptions of single department association.



Redundant attributes

- Similarly, **student entity set is related to the department entity set through a relationship set (student dept).**
- **No need for dept name attribute in student entity set.**



Redundant attributes

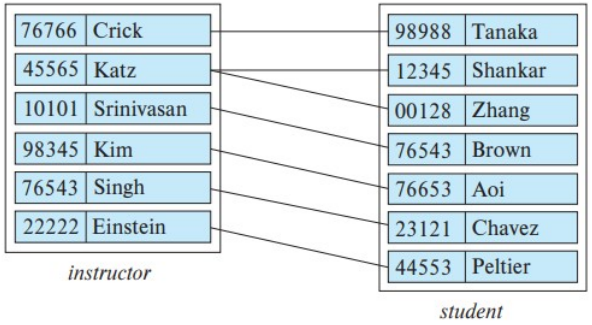
- Consider Course offerings (sections) and associated time slots.
- Each time slot has a unique time slot id and weekly meetings (day, start time, end time). Weekly meeting times are modeled as a multivalued composite attribute.
- Entity sets: section and time slot.
  - Section: (course id, sec id, semester, year, building, room number, time slot id)
  - Time slot: (time slot id, {(day, start time, end time)}).
- Entities related through relationship set sec time slot.
- Issue of redundancy with time slot id in both entity sets.
- **Time slot id is the primary key for the time slot entity.**
- Therefore, **Remove the time_slot_id attribute from the section entity.**
- **A good entity relationship model would not contain any redundant attributes**
- Redundancy removal enhances data clarity and avoids duplication

Multivalued composite attribute

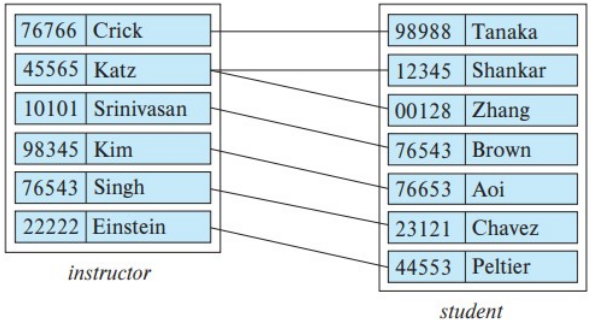
Relationship and Relationship sets

- **What is a Relationship:**
  - A relationship signifies connections between entities.
  - Example: "**advisor**" links instructor Katz to student Shankar.
- **Relationship Set:**
  - **A relationship set contains relationships of the same type.**
  - Example: "advisor" relationship set links students and their advisors.

- **Entity Sets and Relationships:**
  - Consider "instructor" and "student" entity sets.
  - "advisor" relationship set signifies student-advisor connections.

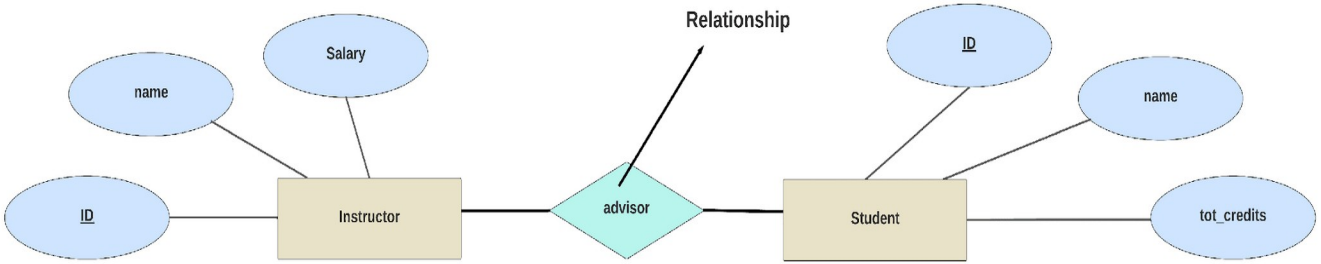


- **Relationship Instance:**
  - **A relationship instance in an E-R schema represents an association between the named entities in the real-world** enterprise that is being modeled.
  - As an illustration, the individual instructor entity Katz, who has instructor ID 45565, and the student entity Shankar, who has student ID 12345, participate in a relationship instance of advisor.
  - This relationship instance represents that in the university, the instructor Katz is advising student Shankar



Relationship and Relationship sets

- **Representation in E-R Diagram:**
  - In **an E-R diagram, relationships are depicted by diamonds.**
  - Lines connect the diamond to relevant entity sets (rectangles).
- For example let us try and represent the advisor relation



Relationship sets

- A relationship type R among n entity types E1, E2, . . . , En defines a set of associations—or a relationship set—among entities from these entity types.
- *Similar to the case of entity types and entity sets, a relationship type and its corresponding relationship set are customarily referred to by the same name, R.*
- Mathematically, the relationship set R is a set of relationship instances ri, where each ri associates n individual entities (e1, e2, . . . , en), and each entity ej in ri is a member of entity set Ej , $1 \leq j \leq n$.
- Hence, a relationship set is a mathematical relation on E1, E2, . . . , En; alternatively, it can be defined as a subset of the Cartesian product of the entity sets $E1 \times E2 \times \ldots \times En$.
- Each of the entity types E1, E2, . . . , En is said to participate in the relationship type R; similarly, each of the individual entities e1, e2, . . . , en is said to participate in the relationship instance ri = (e1, e2, . . . , en).

Recursive Relationships

- **The function that an entity plays in a relationship is called that entity's role.**
- Since entity sets participating in a relationship set are generally distinct, roles are implicit and are not usually specified.
- However, *they are useful when the meaning of a relationship needs clarification. Such is the case when the entity sets of a relationship set are not distinct; that is, the same entity set participates in a relationship set more than once, in different roles*.
- In this type of relationship set, sometimes called a **recursive relationship set**, explicit role names are necessary to specify how an entity participates in a relationship instance.

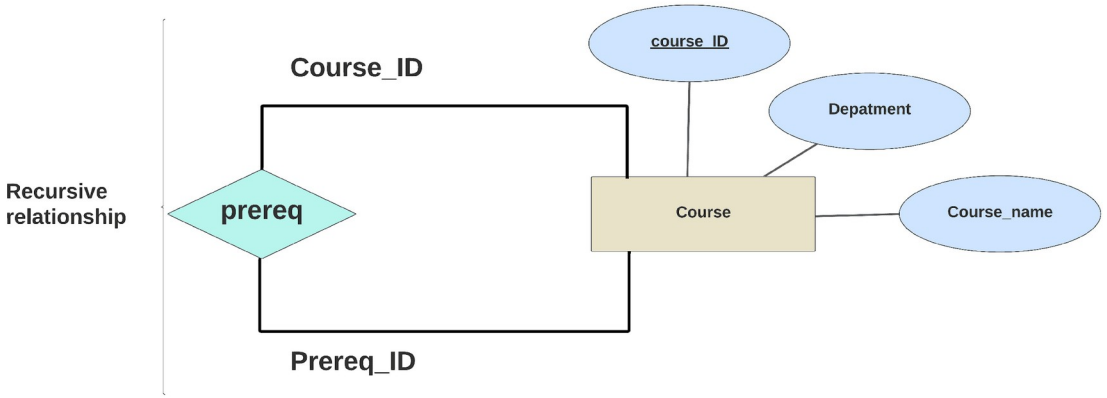
Recursive Relationships

- Let us try and understand it with the help of an example
- consider the entity-set course that records information about all the courses offered in the university. Now some courses that are offered by the university have some prerequisite courses which are needed to be completed
- **To depict the situation where one course (C2) is a prerequisite for another course (C1) we have a relationship set prereq that is modeled by ordered pairs of course entities.**

- So *how do we indicate this in the ER diagram?*

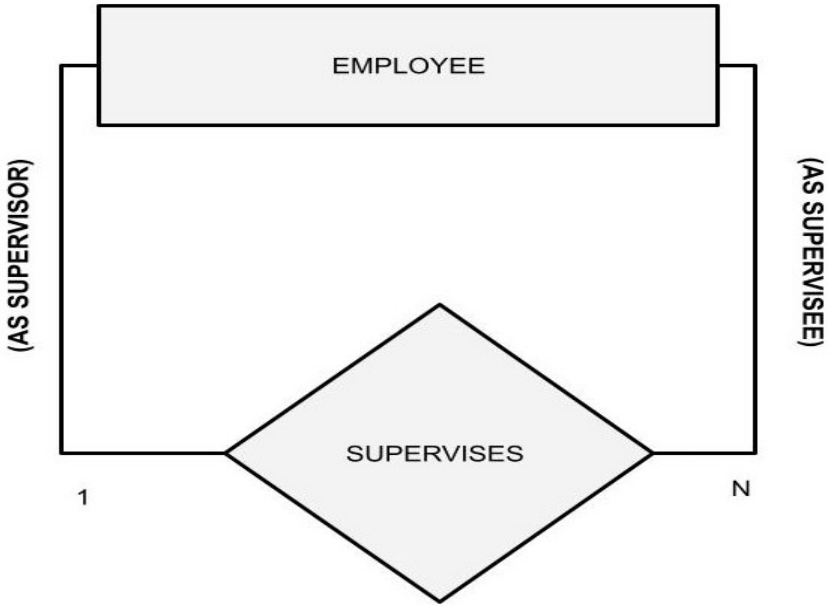
Recursive Relationships

- We indicate roles in E-R diagrams by labeling the lines that connect diamonds to rectangles.
- The diagram *below shows the role indicators course id and prereq id between the course entity set and the prereq relationship set.*



Recursive Relationship - Employee - Supervisor Example

Can you think of another example of a recursive-relationship?



Relationship Set and it's degree

- The relationship sets advisor is an example of a **binary relationship set**—that is, one that involves two entity sets.
- Most of the relationship sets in a database system are binary.
- Occasionally, however, **relationship sets could even involve more than two entity sets. The number of entity sets that participate in a relationship set is the degree of the relationship set.**
  - A binary relationship set is of degree 2;
  - A ternary relationship set is of degree 3.

Relationship Set Example

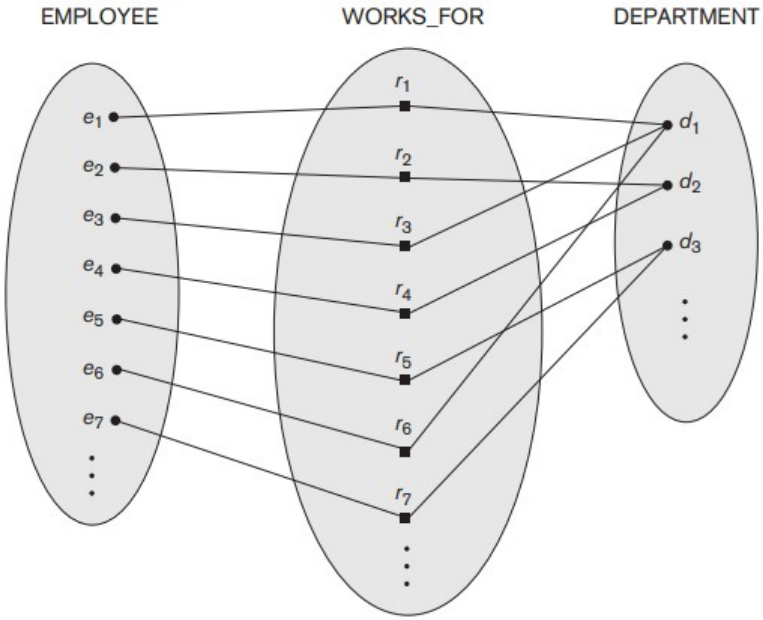


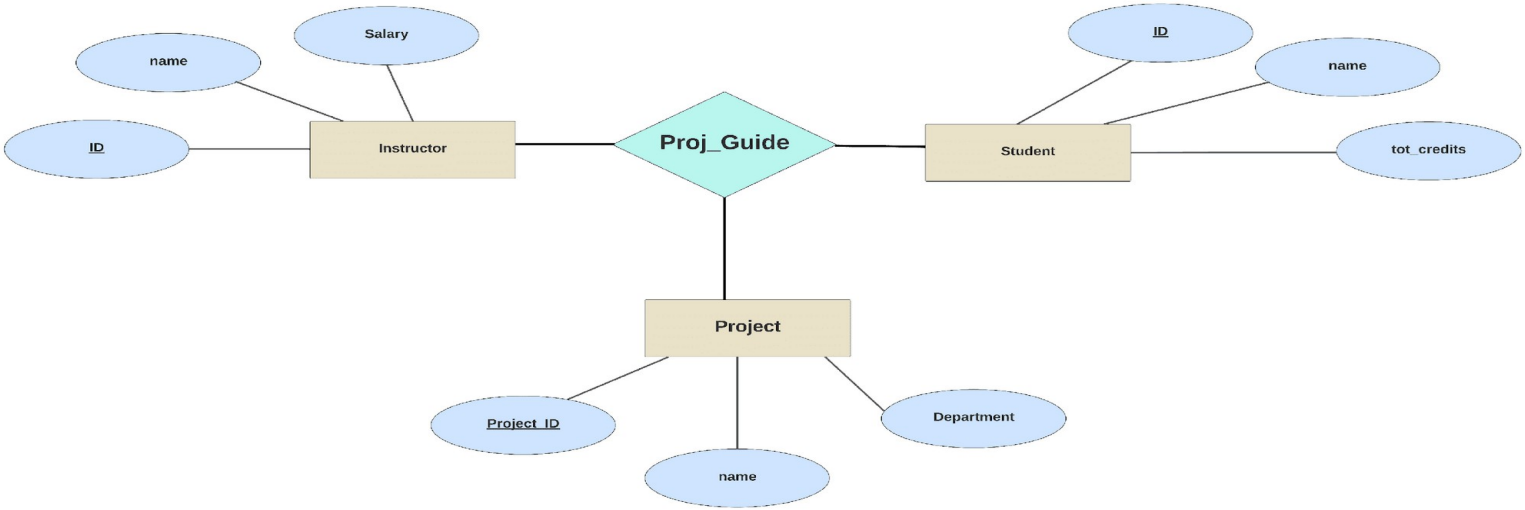
Figure 3.9
Some instances in the WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT.

Ternary Relationships

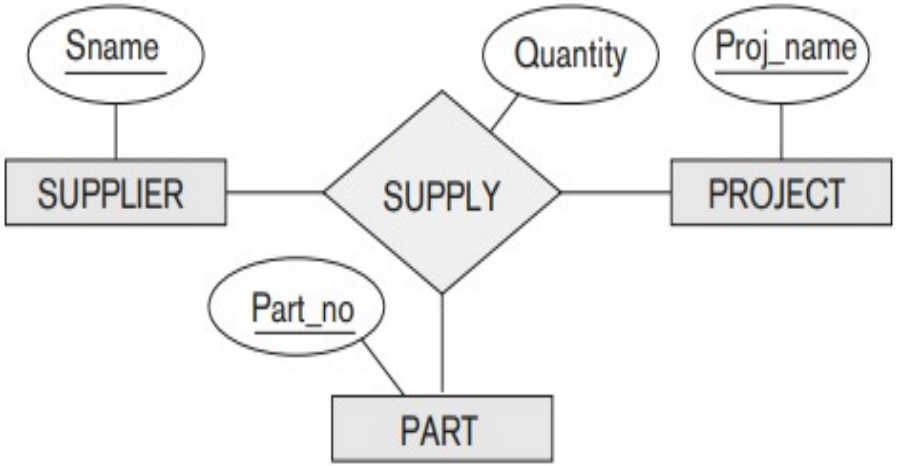
- Let us consider an example, *suppose that we have an entity set project that represents all the research projects carried out in the university.*
- Consider the **entity sets instructor, student, and project**.
- *Each project can have multiple associated students and multiple associated instructors. Furthermore, each student working on a project must have an associated instructor who guides the student on the project.*
- Suppose we were to represent the information of which instructor is guiding which student for which project
- How do we represent this or model this in an ER diagram?

Ternary Relationship Example

- To represent the above situation, we would have to relate the three entity sets through a ternary relationship set proj_guide, This ternary relation relates entity sets instructor, student, and project.
- *An instance of projguide indicates that a particular student is guided by a particular instructor on a particular project*.
- Note that a student could have different instructors as guides for different projects, which cannot be captured by a binary relationship between students and instructors.

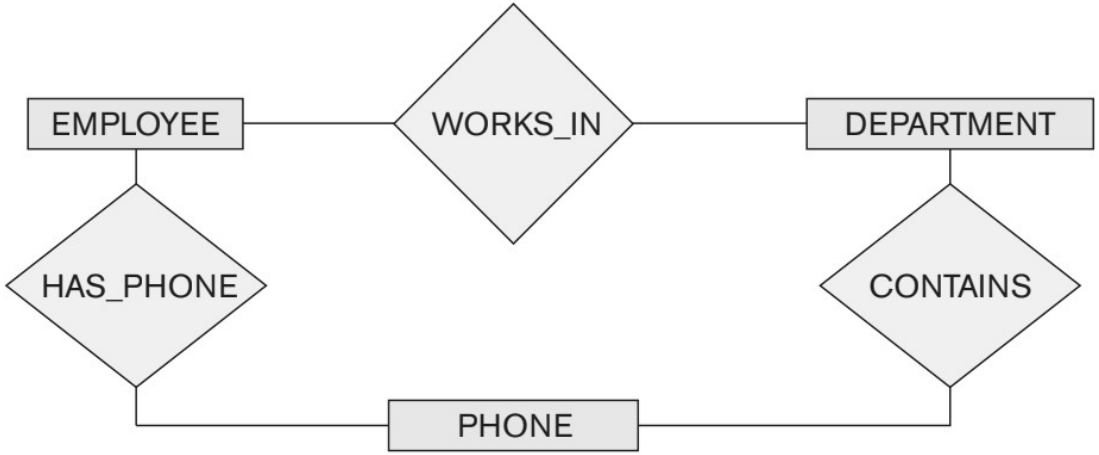


Ternary Relationship Example



Redundant relationship - Question

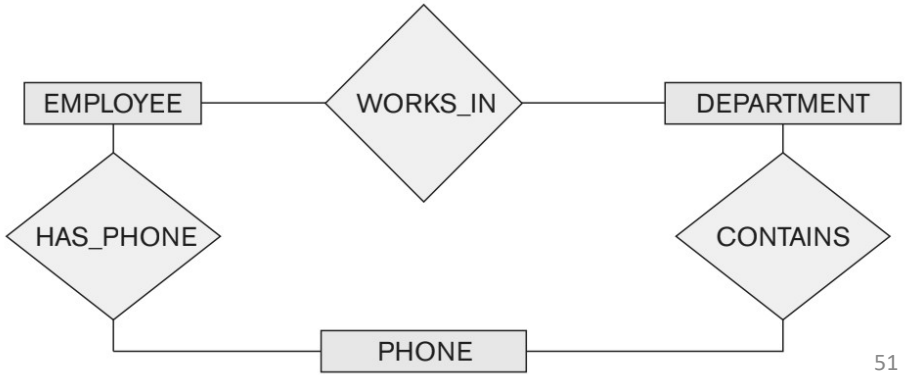
Assume that an employee may work in up to two departments or may not be assigned to any department. Assume that each department must have one and may have up to three phone numbers. Under what conditions would the relationship HAS_PHONE be redundant in this example?



Redundant relationship - Question

Assume that an employee may work in up to two departments or may not be assigned to any department. Assume that each department must have one and may have up to three phone numbers. Under what conditions would the relationship HAS_PHONE be redundant in this example?

- *The HAS_PHONE relationship is redundant **if phone numbers are exclusively associated with departments, not individual employees.***
- *If employees only use department phone numbers, their association with these numbers can be inferred through the WORKS_IN relationship, eliminating the need for a separate HAS_PHONE relationship.*



Example - 1

- Let us consider the Instructor relation of the university database.
- Every instructor would have a unique ID, along with this the instructor would also have the name, address, phone number, Date of Birth, age, and department details to be there in the database
- Given instructor could be a part of only one department. However, each instructor could have multiple phone numbers.
- The address is subdivided into street, city, state, and zip. The street would be again divided into street_no, street_name, and apt_number.
- The name attribute would be subdivided into first name, middle name, and last name.

- **Could you draw an E-R diagram for containing the entity Instructor along with all the attributes?**

State
City
Zip
Address
hierarchy
Apartment No
Last_name
Street
Street Name
compound attribute
Middle_name
Name
Instructor
Street No
First_name
Age
Derived attribute
ID
Date_of_Birth
Multivalued Attribute
Phone_no
Department

Example - 2

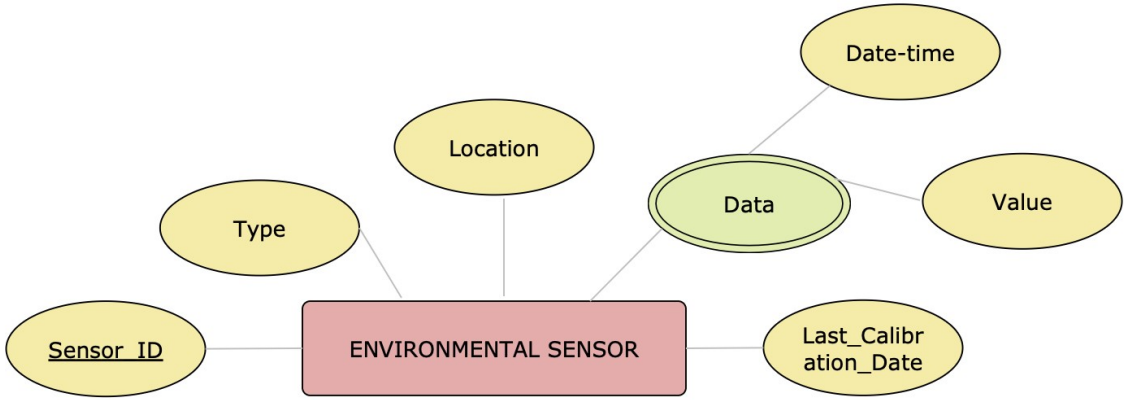
You are designing a database for a **Smart City Infra-Management System**. One of the key entities is the **Environmental Sensor**, which is deployed across the city to collect various environmental data like air quality, noise levels, etc.

Model the **Environmental Sensor** entity in an ER diagram with the following details:

- Each sensor must be uniquely **ID**entified.
- Each sensor has a **Type** (e.g., temperature, humidity, pollution).
- Each sensor is installed at a specific **Location**.
- It collects multiple **Data** readings,containing:
  - **Date-Time** of reading
  - **Value** of the reading
- Each sensor also has a **Last Calibration Date**.

Example - 2: ER Diagram

Solution:



Database Management Systems

Q & A

1) Which of the following is an example of a recursive relationship?

A) Course prerequisites

B) Instructor-student relationship

C) Student-course enrollment

D) Student-advisor relationship

2. What does a binary relationship set involve?

A) One entity set

B) Two entity sets

C) Three entity sets

D) More than three entity sets

Q & A

3) **In an E-R diagram, how is a relationship between two entity sets depicted?**

A) By a diamond

B) By a rectangle

C) By a line

D) By an oval

4) **Which of the following is true about the relationship set ‘prereq’ in a university database?**

A) It connects students to courses

B) It connects instructors to students

C) It connects courses to other courses

D) It connects projects to instructors

Database Management Systems

Q & A

5) **What does a ternary relationship set involve?**

A) Two entity sets

B) Three entity sets

C) One entity set

D) No entity sets

6) **In an E-R diagram, how is a primary key represented?**

A) Underlined attribute in the oval

B) Underlined attribute in the rectangle

C) Underlined attribute in the diamond

D) Underlined relationship set

Database Management Systems

Q & A

7) **What does an extension of an entity set refer to?**

A) The actual collection of entities

B) The unique identifier for each entity

C) The general set of attributes

D) The relationships between entity sets

8) **What does an E-R diagram's diamond represent?**

A) Entity Set

B) Attribute

C) Relationship Set

D) Primary Key

Q & A

ANSWERS:

1. A
2. B
3. A
4. C
5. B
6. B
7. A
8. C

THANK YOU

Mahitha G
Department of Computer Science and Engineering
mahithag@pes.edu

Database Management Systems

Entity-Relationship Model

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgments: Dr. Nagasundari S (Dept of CSE), Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 1: Introduction to Database Management and SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G.

Department of Computer Science and Engineering

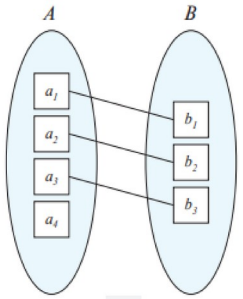
Cardinality Ratio

- Mapping cardinalities, or cardinality ratios, express the number of entities to which another entity can be associated via a relationship set.
- Mapping cardinalities are most useful in describing binary relationship sets, although they can contribute to the description of relationship sets that involve more than two entity sets.

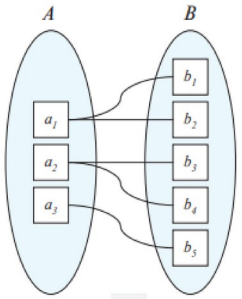
- A binary relationship set R between entity sets A and B, the possible mapping cardinality are:
  - One-to-one
  - One-to-many
  - Many-to-one
  - Many-to-many

Cardinality Ratio

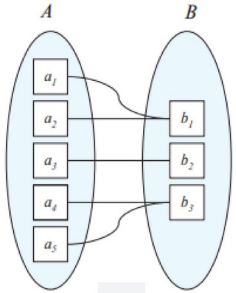
- **One-to-one**
  - An entity in A is associated with at most one entity in B, and an entity in B is associated with at most one entity in A.
- **One-to-many**
  - An entity in A is associated with any number (zero or more) of entities in B. An entity in B, however, can be associated with at most one entity in A.
- **Many-to-one**
  - An entity in A is associated with at most one entity in B. An entity in B, however, can be associated with any number (zero or more) of entities in A.
- **Many-to-many**
  - An entity in A is associated with any number (zero or more) of entities in B, and an entity in B is associated with any number (zero or more) of entities in A.



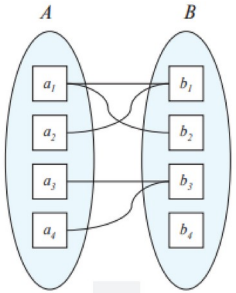
One-to-one



One-to-many



Many-to-one



Many-to-many

Cardinality Ratio

- Let us now consider the advisor relationship set.
- A student can be advised by several instructors - this relationship set can be classified as many-to-many.
- In contrast, if a particular university imposes a constraint that a student can be advised by only one instructor, an instructor can advise several students, then the relationship set from instructor to student must be one-to-many.

one-to-many

many-to-many

Cardinality Ratio

In the E-R diagram notation, we indicate cardinality constraints on a relationship by drawing:

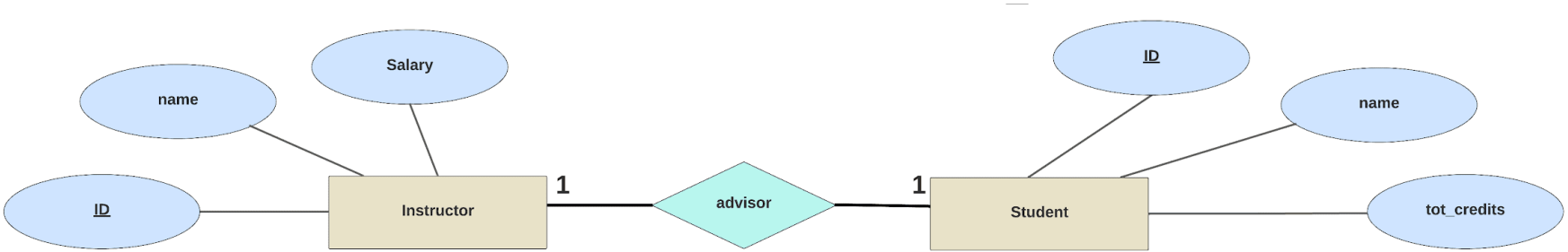
- An **undirected line (—)**

between the relationship set and the entity set in question.

Let us try and understand how to use the above mapping technique with the help of university database.

One-to-one (1:1)

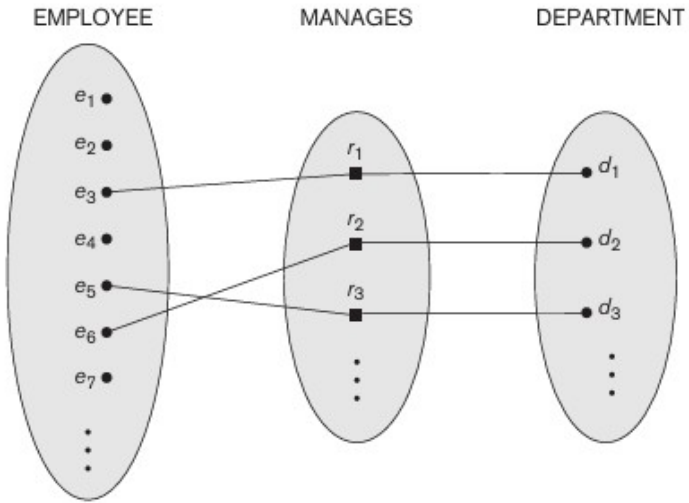
- We draw a line from the relationship set to both entity sets.
- For example, the lines to instructor and student indicate that an instructor may advise at most one student, and a student may have at most one advisor.



One-to-one (1:1)

- An example of a 1:1 binary relationship is MANAGES as shown in the figure,which relates a department entity to the employee who manages that department.

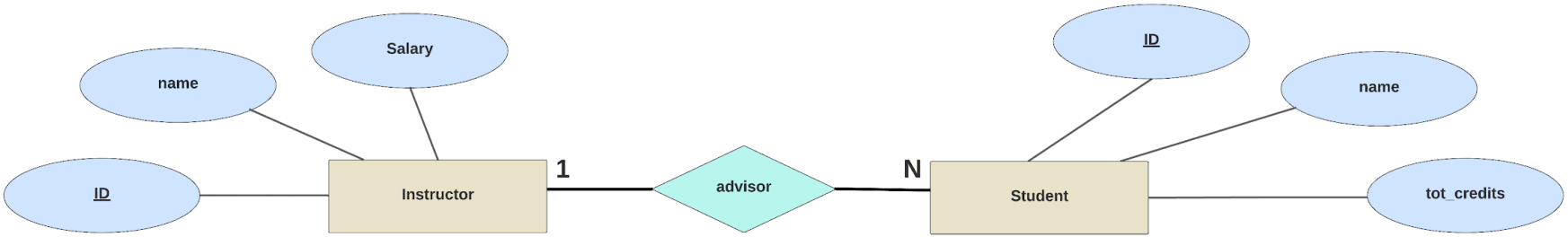
- This represents that at any point in time an employee can manage at most one department and a department can have at most one manager.



Cardinality Ratio

One-to-many (1:N)

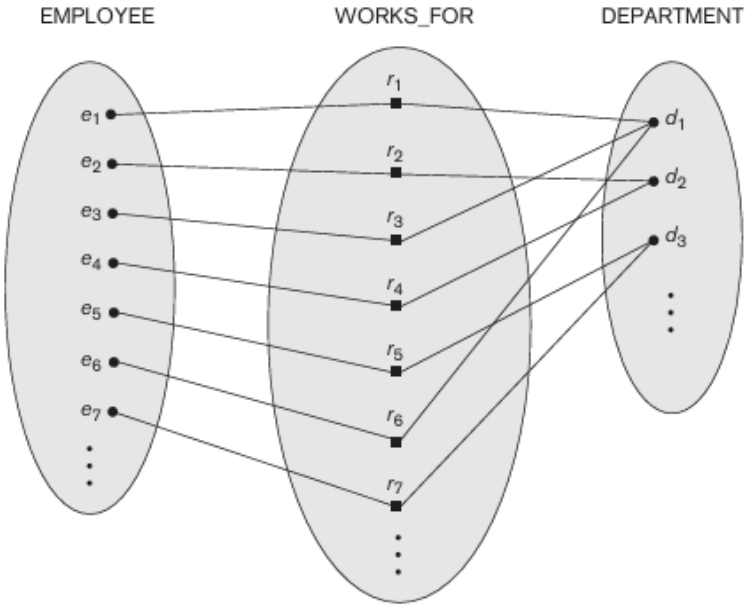
- This indicates that an instructor may advise many students, but a student may have at most one advisor.



Cardinality Ratio

One-to-many (1:N)

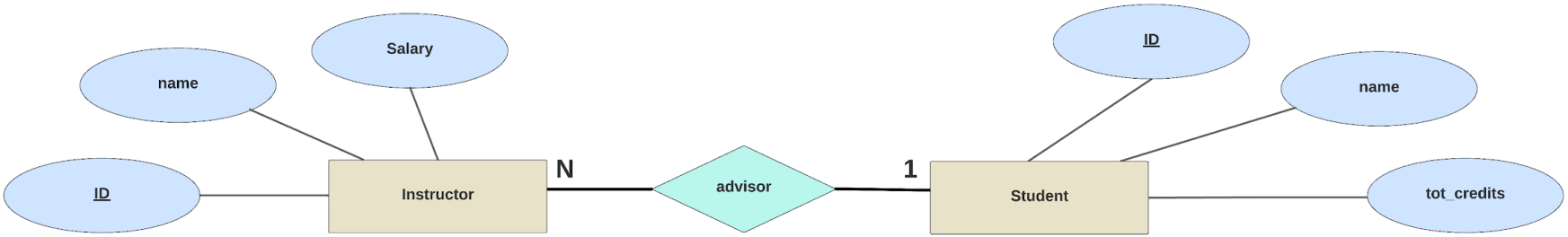
- For example, In the WORKS_FOR binary relationship type, DEPARTMENT:EMPLOYEE is of cardinality ratio 1:N.
- Meaning that each department can be related to any number of employees (N) but an employee can be related to (work for) at most one department.
- This means that for this particular relationship type WORKS_FOR, a particular department entity can be related to any number of employees.



Cardinality Ratio

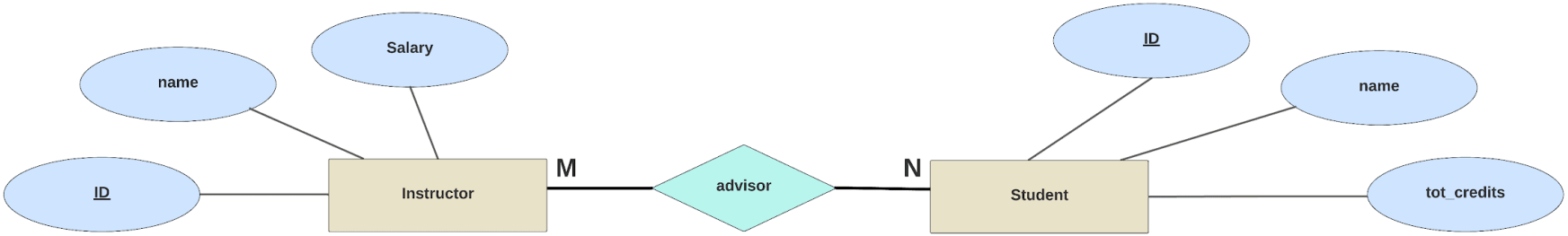
Many-to-one (N:1)

- The converse of the before type
- This indicates that an instructor may advise at most one student, but a student may have many advisors.



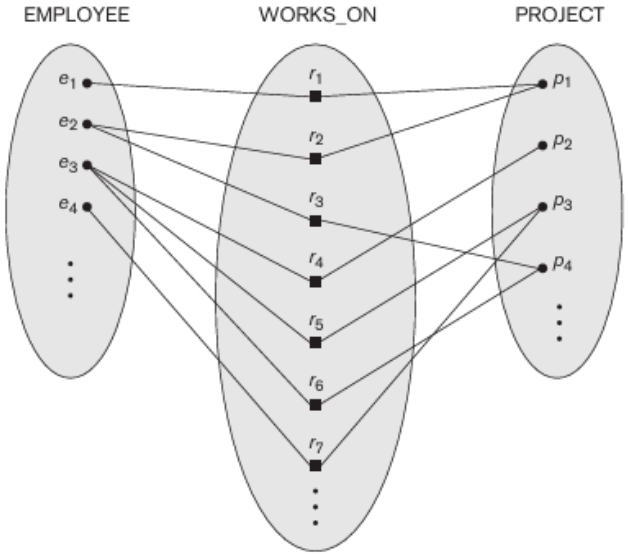
Many-to-many (M:N)

- We draw an undirected line from the relationship set to both entity sets.
- Thus, there are undirected lines from the relationship set advisor to both entity sets instructor and student.
- This indicates that an instructor may advise many students and a student may have many advisors.



Many-to-many (M:N)

- The relationship type WORKS_ON is of cardinality ratio M:N
- The rule is that an employee can work on several projects and a project can have several employees which is represented by the diagram on the right.



Participation Constraints and Existence Dependencies

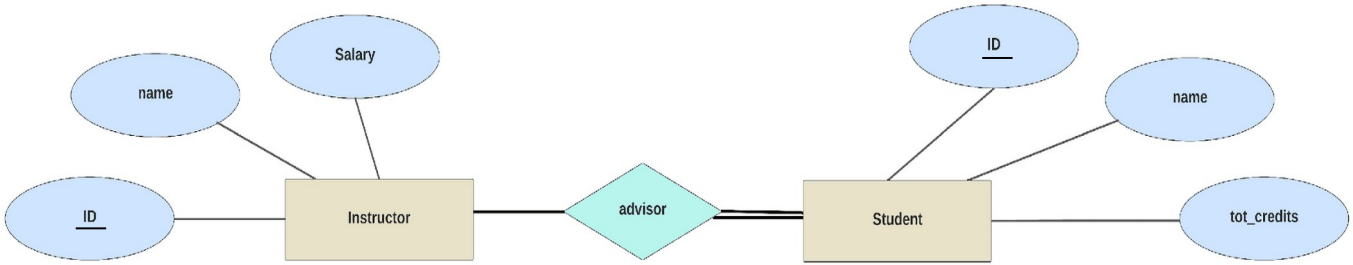
- There are two types of participation :
  - **Total participation**
  - **Partial participation**

- The participation of an ***entity set E in a relationship set R is said to be total if every entity in E must participate in at least one relationship in R.***
  - Considering the university database, a university may require every student to have at least one advisor Therefore, the participation of student in the relationship set advisor is total.

- *If it is possible that some entities in E do not participate in relationships in R, the participation of entity set E in relationship R is said to be* ***partial****.*
  - An instructor need not advise any students therefore, The participation of instructor in the advisor relationship set is therefore partial

Participation Constraints and Existence Dependencies

- We indicate the **total participation of an entity in a relationship set using double lines.**
- The example of the advisor relationship set described in the previous slide would look as shown
- *where the double line indicates that a student must have an advisor.*

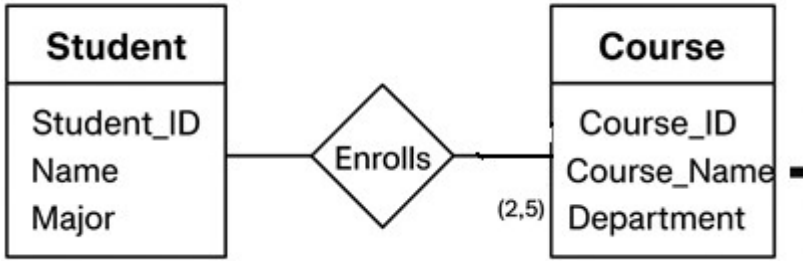


Cardinality ratio

- Let us consider the entities:
  - Student
    - ➢ Attributes: Student_ID, Name, Major
  - Course
    - ➢ Attributes: Course_ID, Course_Name, Department

- The institution has made a rule that given every student has to be enrolled for a minimum of 2 courses and a maximum of 5 courses

- How would we represent this in the ER diagram?



Min-Max notation

- E-R diagrams also **provide a way to indicate more complex constraints** on the number of times each entity participates in relationships in a relationship set.
- A line may have an associated minimum and maximum cardinality, shown in the form **(l,h)**
  - **l** is the **minimum cardinality**
  - **h** is the **maximum cardinality**.

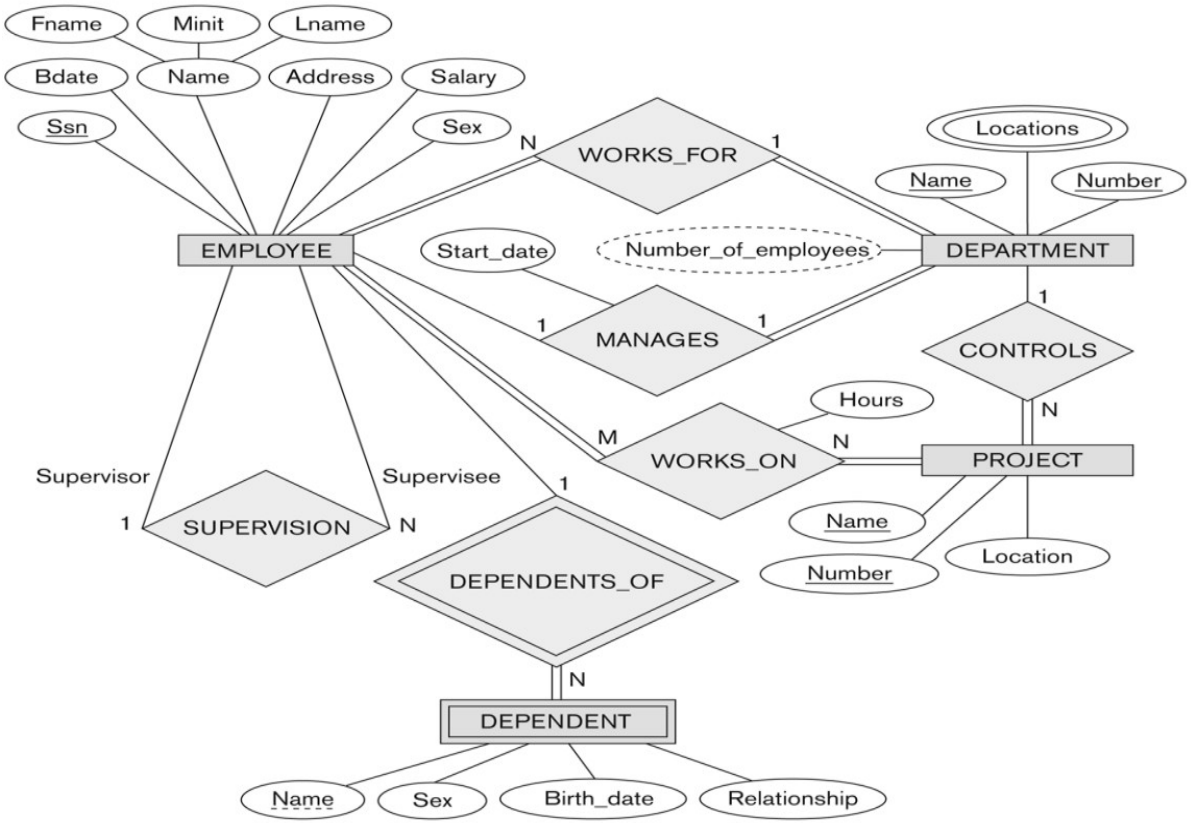
- A minimum value = 1 ➔total participation of the entity set in the relationship set;
- A maximum value = 1 ➔ that the entity participates in at most one relationship,
- A maximum value = **N** indicates **no limit**.

Min-Max notation and Chen notation

Min-Max [Entity 1]	Chen-Notation	Min-Max [Entity 2]
(0,1)	1:1	(0,1)
(0,N)	1:n	(0,1)
(0,N)	1:n + total part.	(1,1)
(0,N)	n:m	(0,N)
(1,1)	total part. + 1:1	(0,1)
(1,N)	total part. + 1:n	(0,1)
(1,1)	total part. + 1:1 + total part.	(1,1)
(1,N)	total part. + 1:n + total part.	(1,1)
(1,N)	total part. + n:m	(0,N)
(1,N)	total part. + n:m + total part.	(1,N)

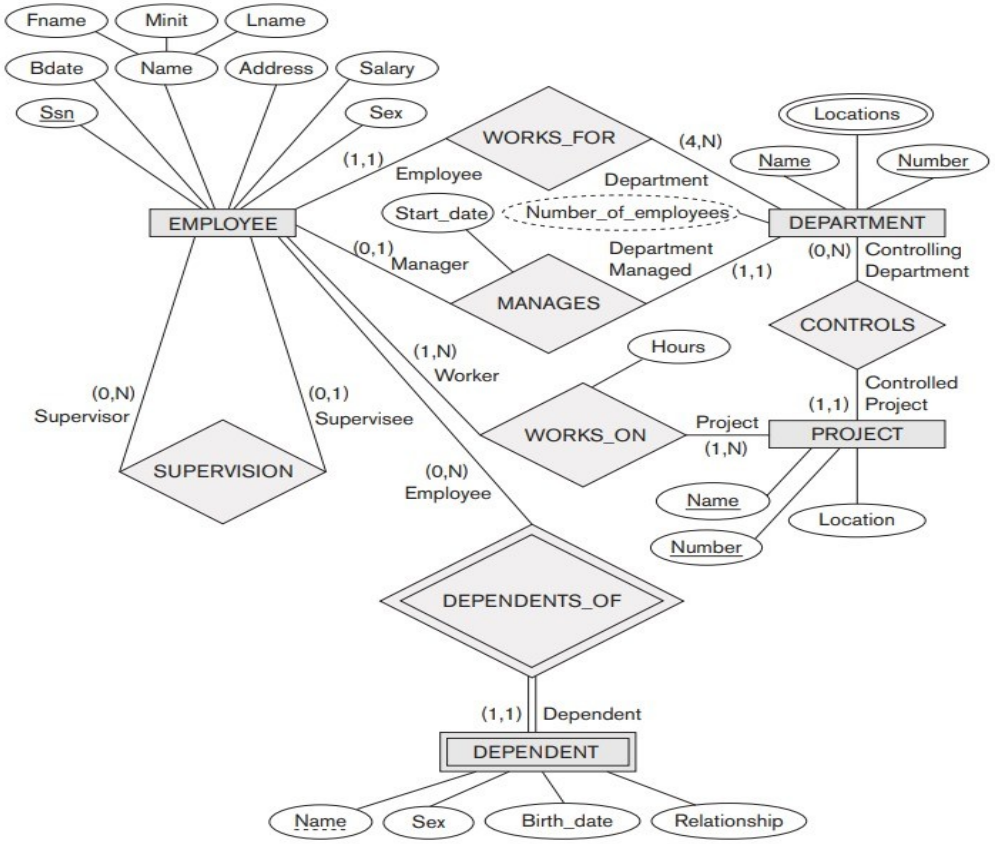
Database Management Systems

Example of Chen Notation



Database Management Systems

(Min,Max) Notation



Primary Key In Entity Sets

- Individual entities are **conceptually distinct**.
- From a database perspective, differences among entities are expressed through attributes.
- Attribute values must uniquely identify each entity.
- **No two entities in an entity set can share the same values for all attributes.**
- Key for an entity is a set of attributes that distinguish entities.
- Concepts of superkey, candidate key, and primary key apply to entity sets.

Primary Key In Relationship Sets

Keys also help uniquely identify relationships. Thus differentiates relationships from each other.

- So we will now look at how can we identify/distinguish relationships
- Let R be a relationship set involving entity sets E1, E2, …, En.
- Let **primary-key(Ei)** denote the set of attributes that forms the primary key for entity set Ei .
- Assume for now that the attribute names of all primary keys are unique.
- The composition of the primary key for a relationship set depends on the set of attributes associated with the relationship set R.
  - Case 1: Relationship R has **no attributes**
  - Case 2: If relation R has **{a1, a2, …, am} attributes** associated with it

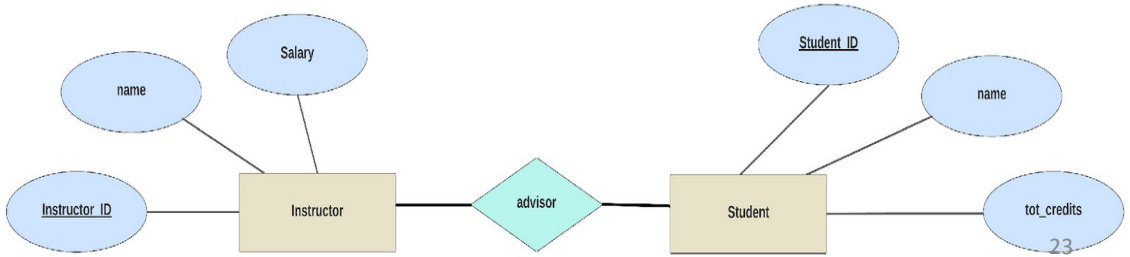
Primary Key In Relationship Sets

Case 1:

- If the relationship set R has **no attributes** associated with it,
  then the set of attributes:

  primary-key(E1) ∪ primary-key(E2) ∪ ⋯ ∪ primary-key(En)

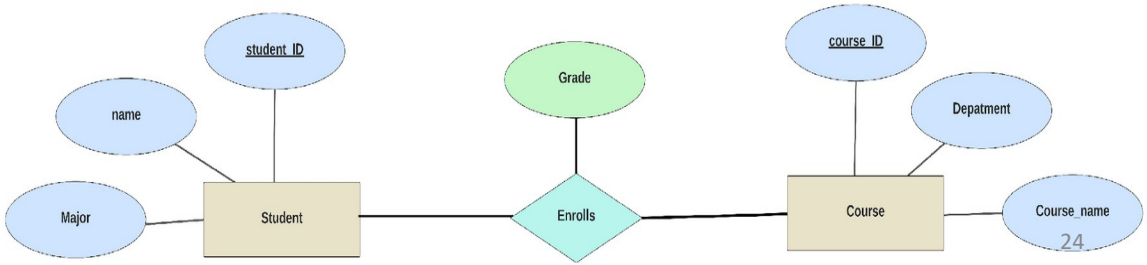
  describes an individual relationship in set R.
- For example consider the relationship of advisor between Instructor and Student as shown
- The set of attributes that would describe the relationship would be
  - {primary-key(Instructor) ∪ primary-key(Student)}
  - {Instructor_ID, Student_ID}



Primary Key In Relationship Sets

Case 2:

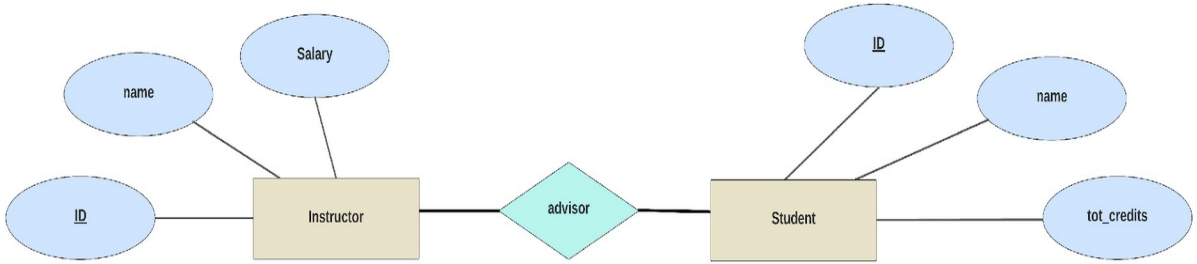
- If the relationship set R has **attributes {a1, a2, …, am}** associated with it,
  then the set of attributes
  primary-key(E1) ∪ primary-key(E2) ∪ ⋯ ∪ primary-key(En) ∪ {a1, a2, …, am}
  describes an individual relationship in set R.
- For example let us consider the student enrolls for a course and for each course the student would have an associated grade
- The set of attributes that would describe the relationship would be
  - {primary-key(Student) ∪ primary-key(Course) ∪ {Grade}}
  - {Student_ID, Course_ID, (



Primary Key In Relationship Sets

- If the attribute names of primary keys are not unique across entity sets, the attributes **are renamed** to distinguish them;
  - Then the name of the entity set combined with the name of the attribute would form a unique name.

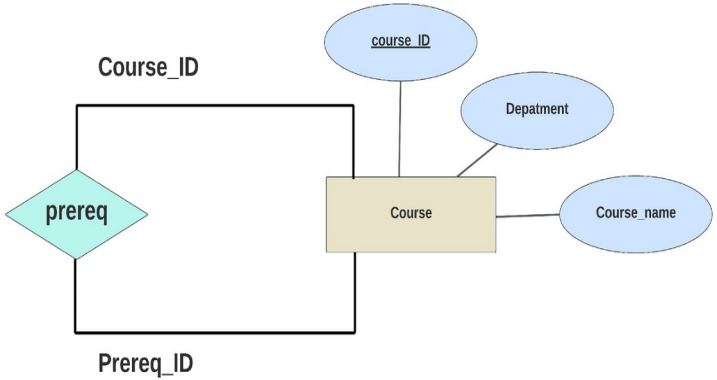
- Considering the below diagram, the attribute name of primary key of both entities are the same (ID)
- So in this case the set of attributes that would describe the relationship would be
  - {(Entity name + primary-key(Instructor)) U (Entity name + primary-key(Student))}
  - {Instructor_ID, Student_ID}



Primary Key In Relationship Sets

- If an entity set participates more than once in a relationship set (as in the prereq relationship in the university database), the role name is used instead of the name of the entity set, to form a unique attribute name.

- For example, in the prereq relationship, the Course entity participates in the relation more than once.
- Therefore we use the role name as shown below
- {Course_ID, Prereq_ID}



Primary Key In Relationship Sets

- Recall that a relationship set is a set of relationship instances, and each instance is uniquely identified by the entities that participate in it.

- Thus, in both of the cases, {Case 1 or Case 2}
  the set of attributes
  primary-key(E1) ∪ primary-key(E2) ∪ ⋯ ∪ primary-key(En)
  forms a superkey for the relationship set.

Primary Key In Relationship Sets

- The **choice of the primary key** for a binary relationship set depends on the **mapping cardinality** of the relationship set.

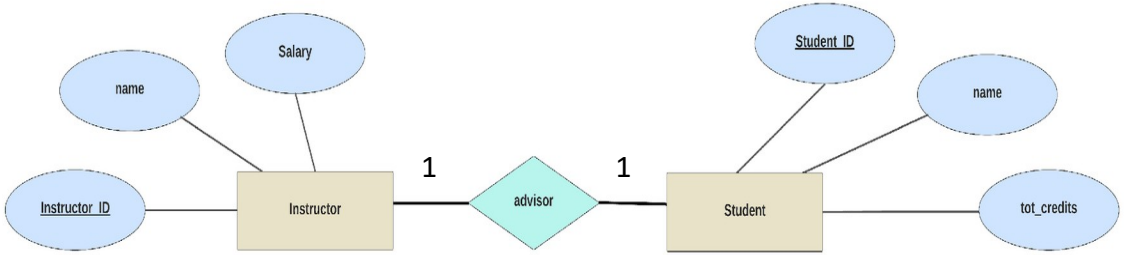
- As we have seen there are 4 different types:
  - One-to-one
  - One-to-many
  - Many-to-one
  - Many-to-many

- Let us find out how do we choose the primary key for these binary relationships

Primary Key In Relationship Sets

One-to-One

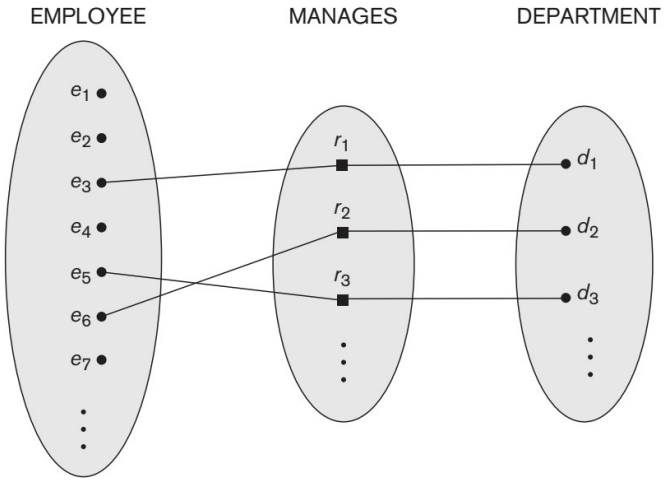
- For this type of relation, the primary key of either one of the participating entity sets forms a minimal superkey, and either one can be chosen as the primary key of the relationship set.
- Let us consider the situation:
- If an instructor can advise only one student, and each student can be advised by only one instructor, that is, if the advisor relationship is one-to-one
  - then the primary key of either student or instructor can be chosen as the primary key for the advisor.



Primary Key In Relationship Sets

One-to-One

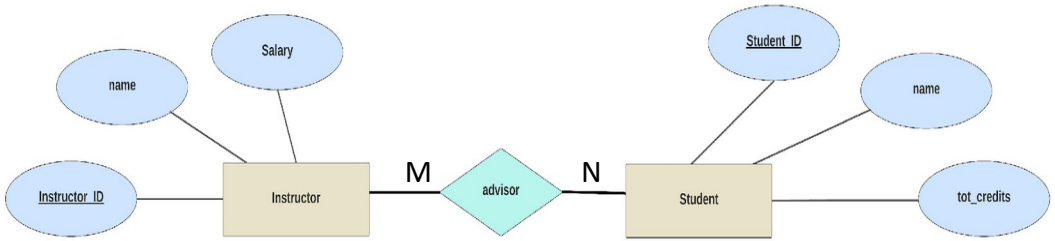
- Let us consider that at any point in time , an employee can manage at most one department and a department can have at most one manager.
- Hence the primary key of either employee or department can be chosen as the primary key for the manages relationship.



Primary Key In Relationship Sets

Many-to-Many

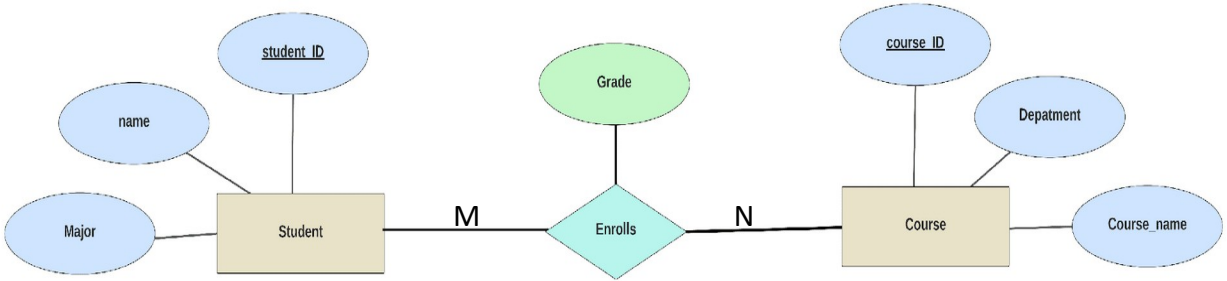
- For these relationships, the preceding union of the primary keys is a minimal superkey and is chosen as the primary key.
- As an example we can consider the entity sets instructor and student, and the relationship set advisor. the relationship set is many-to-many.
- The primary key for the advisor relation :
  - {primary-key(Instructor) ∪ primary-key(Student)}
  - {Instructor_ID, Student_ID}



Primary Key In Relationship Sets

Many-to-Many

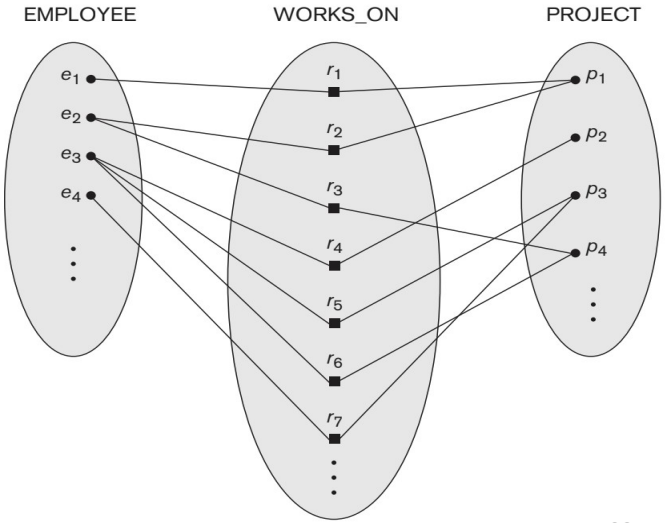
- For these relationships, the preceding union of the primary keys is a minimal superkey and is chosen as the primary key.
- As another example we can consider the entity sets student and courses, and the relationship set enrolls. the relationship set is many-to-many.
- The primary key for the advisor relation :
  - {primary-key(Student) U primary-key(Course)}
  - {Student_ID, Course_ID}
  - The advisory relation : {Student_ID, Course_ID, Grade}
  - Note that grade is not a part of the primary key



Primary Key In Relationship Sets

Many-to-Many

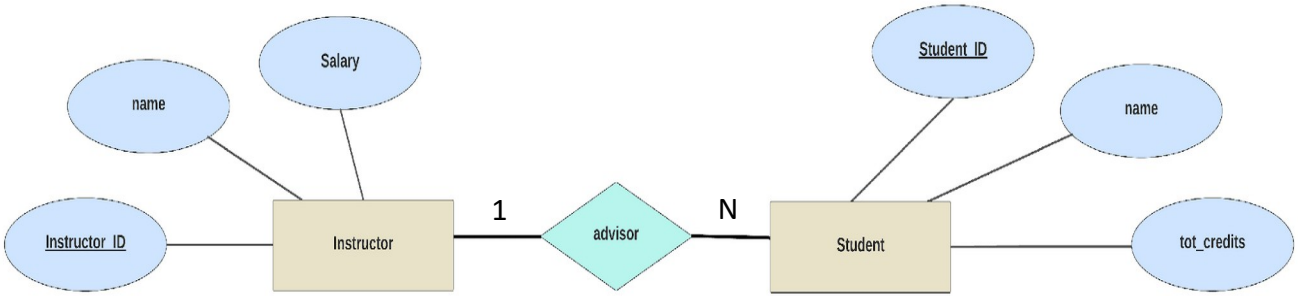
- For these relationships, the preceding union of the primary keys is a minimal superkey and is chosen as the primary key.
- As an example we can consider the entity sets Employee and Project, and the relationship works_on.
- The primary key for the works_on relation :
  - {primary-key(Employee) U primary-key(Project)}



Primary Key In Relationship Sets

One-to-Many or Many-to-One

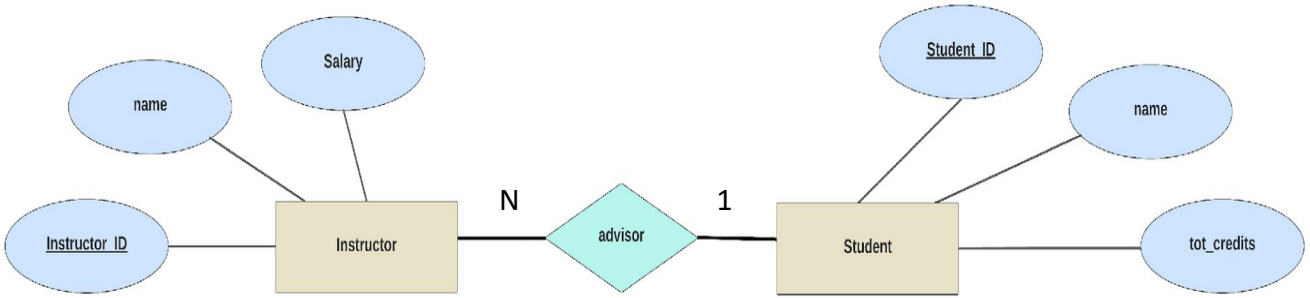
- the primary key of the "**many**" side is a **minimal superkey** and is used as the primary key.
- For example, if the advisor relationship is **many**-to-one from **student** to instructor, that is, each student can have at most one advisor
  - then the primary key of advisor is simply the **primary key of student**.



Primary Key In Relationship Sets

One-to-Many or Many-to-One

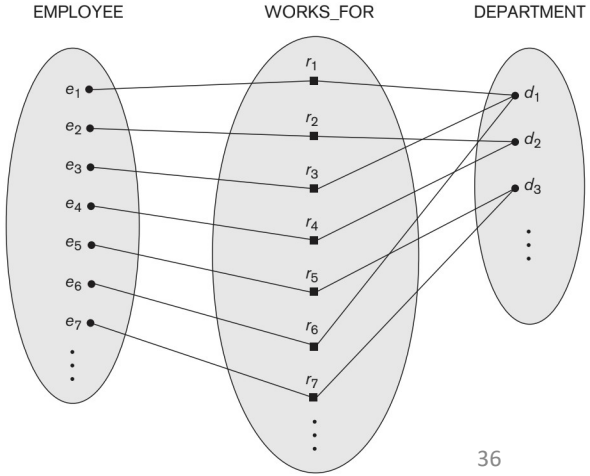
- the primary key of the "**many**" side is a **minimal superkey** and is used as the primary key.
- On the contrary, if an instructor can advise only one student, that is, if the advisor relationship is **many**-to-one from **instructor** to student
  - then the primary key of advisor is simply the **primary key of instructor**.



Primary Key In Relationship Sets

One-to-Many or Many-to-One

- the primary key of the "**many**" side is a **minimal superkey** and is used as the primary key.
- Let us consider the entity sets Employee and Department and the relationship works_for.
- In the figure, the employees e1, e3, and e6 work for department d1; the employees e2 and e4 work for department d2; and the employees e5 and e7 work for department d3.
  - then the primary key of works_for is simply the **primary key of employee**.



Weak Entity Sets

- A weak entity set is one whose existence is dependent on another entity set, called it's **identifying entity** set;
- Instead of associating a primary key with a weak entity, we use the primary key of the identifying entity, along with extra attributes, called **discriminator attributes** to uniquely identify a weak entity.
- An entity set that is not a weak entity set is termed a **strong entity** set.
- Every weak entity must be associated with an identifying entity; that is, the weak entity set is said to be existence dependent on the identifying entity set.
- The identifying entity set is said to own the weak entity set that it identifies. The relationship associating the weak entity set with the identifying entity set is called the **identifying relationship**.

Weak Entity Sets

- In the university database, let us consider the following schemas for the entities course and section:
- Course (Course_ID, title, dept_name, credits)
  - Primary key {Course_ID}
- Section (Course_ID, sec_ID, semester, year, building, room_number, time_slot_id)
  - Primary key {Course_ID, sec_ID, semester, year}

- Now let us consider that every course should have a section where the course would be taking place.
- How do we relate both these entities for showing the above relationship?

Weak Entity Sets

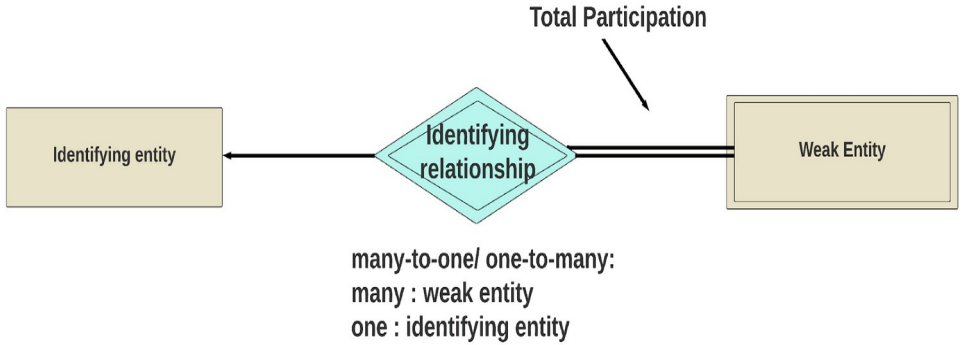
- For the above question, we would connect sections to courses using a relationship called "sec_course."
- But we should note that the "sec course" information is kind of repetitive because each section already has a "course id" telling us which course it belongs to.

- To solve this problem there are 2 possibilities:
  - Possibility 1: Remove the "sec_course" relationship
    - ➢ Then the connection between sections and courses isn't as clear. That is it becomes implicit in an attribute, which is not desirable
  - Possibility 2: Remove the "course_id" from the section and keep only the other details like section number, year, and semester.
    - ➢ If we do this, sections might share the same details (section number, year, and semester) for different courses, making them hard to tell apart.

- To deal with this problem, we treat the relationship sec_course as a special relationship that provides extra information, in this case, the course id, required to identify section entities uniquely

Weak Entity Sets

- The identifying relationship is many-to-one from the weak entity set to the identifying entity set, and the participation of the weak entity set in the relationship is total.



- The identifying relationship set should not have any descriptive attributes, since any such attributes can instead be associated with the weak entity set.
- The ER representation of the weak entity is as shown

Primary Key for Weak Entity Sets

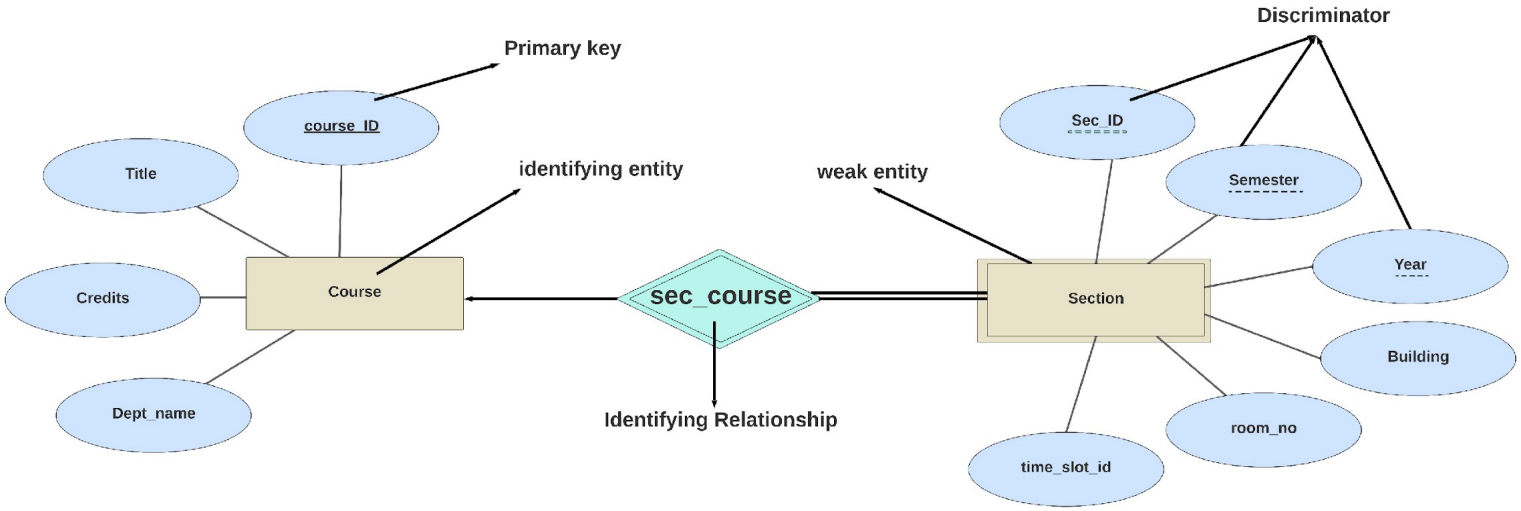
- For the situation considered for sec_course
  - Weak entity: Section
  - Identifying entity: Course,
  - Identifying relationship: Sec_course.
  - The primary key of the section is formed by the primary key of the identifying entity set (that is, course), plus the discriminator of the weak entity set (that is, section).
  - Primary key is {**course id**, **sec id, year, semester**}.

Primary-key(Identifying entity: Course)

Discriminator(weak entity: Section)

Weak Entity Sets

- Note that the ER diagram for the same would be as follows:
  - a weak entity set is depicted via a double rectangle
  - discriminator is underlined with a dashed line.
  - The relationship set connecting the weak entity set to the identifying strong entity set is depicted by a double diamond.



Weak Entity Sets

- Note that we could have chosen to make sec_id globally unique across all courses offered in the university, in which case the section entity set would have had a primary key.
- However, conceptually, a section is still dependent on a course for its existence, which is made explicit by making it a weak entity set.

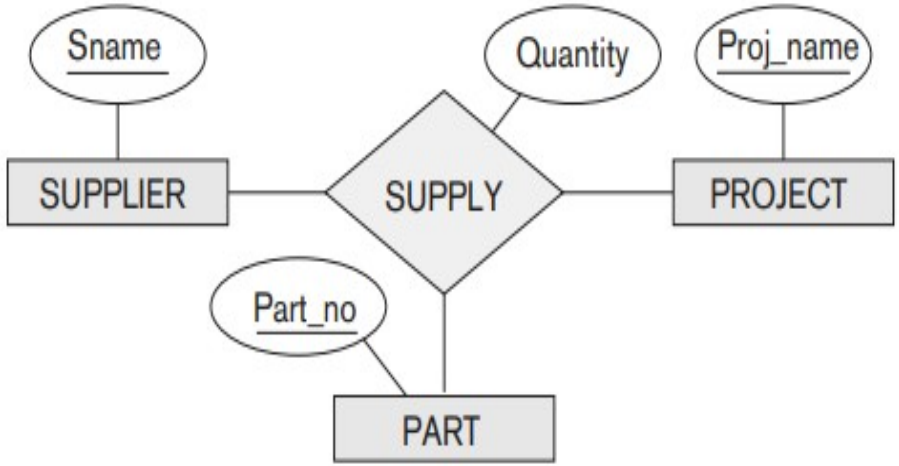
- **In general, a weak entity set must have total participation in its identifying relationship set, and the relationship is many-to-one toward the identifying entity set.**

Weak Entity Sets

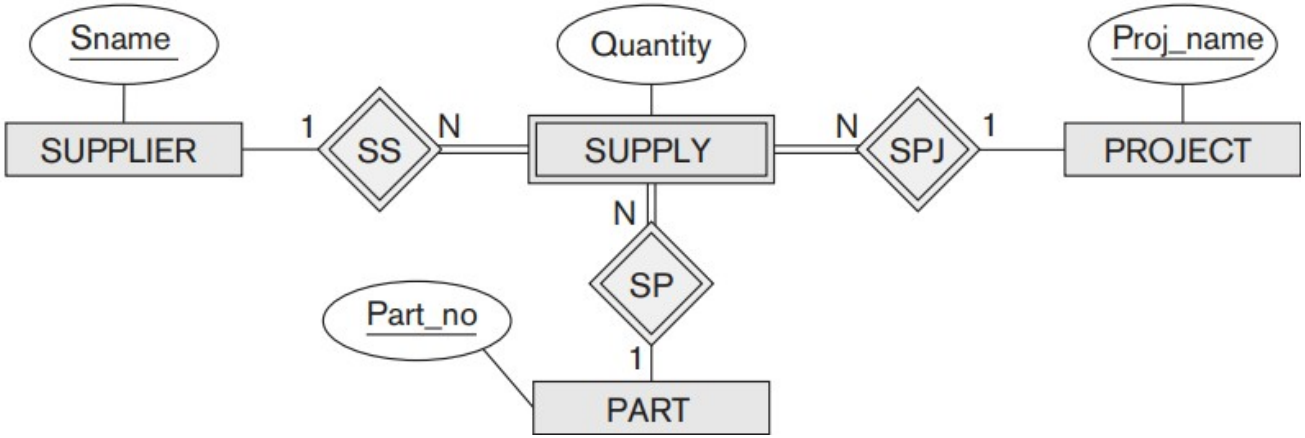
- **A weak entity set can participate in relationships other than the identifying relationship.**
  - For instance, the section entity could participate in a relationship with the time slot entity set, identifying the time when a particular class section meets.

- It is also possible to have a weak entity set with more than one identifying entity set. A particular weak entity would then be identified by a combination of entities, one from each identifying entity set.
- The primary key of the weak entity set would consist of the union of the primary keys of the identifying entity sets, plus the discriminator of the weak entity set.

Weak Entity Sets in a Ternary Relationship



Weak Entity Sets in a Ternary Relationship



SUPPLY represented as a weak entity type

Example

In the previous slide set, you have already modeled the **ENVIRONMENTAL SENSOR** entity.

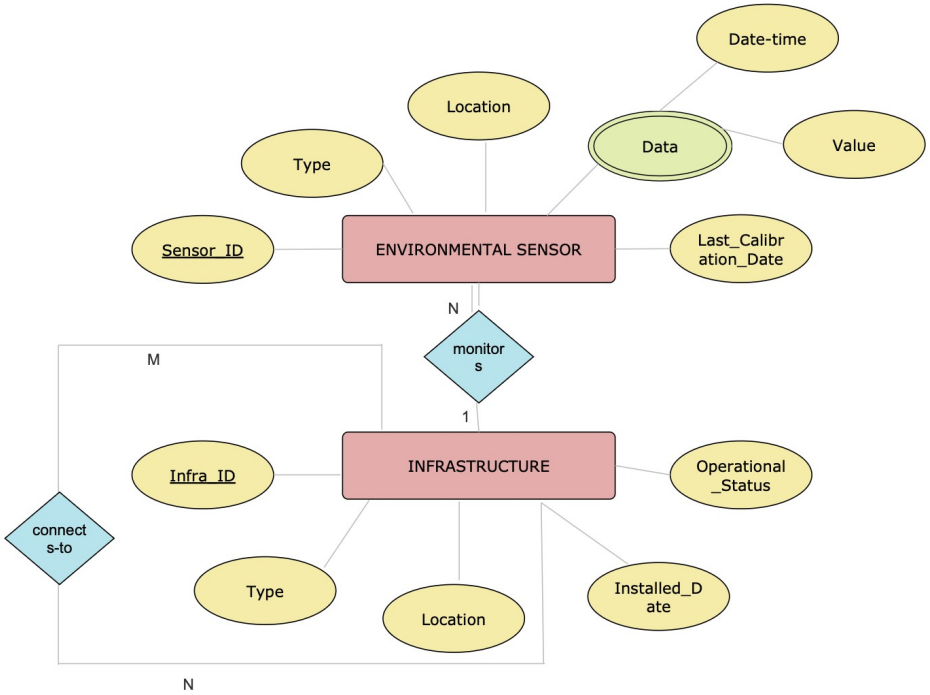
Now, you are provided with additional data related to **urban infrastructure** components. Each **Infrastructure** entity has the following attributes: Infrastructure_ID,Type, Location,Installed_Date and Operational_Status

Based on this:

1. **Draw an ER diagram** that shows the relationship between Environmental Sensor and Infrastructure.
   - Specify the **cardinality and participation (total/partial)** on both sides.
   - Consider that each sensor is installed **on exactly one infrastructure element**, but each infrastructure **may host zero or more sensors**.
2. Identify and model any **recursive relationship** (self-relationship) that may exist within the Infrastructure entity.(Think of how one road may lead to another road).

Database Management Systems

Example



Example

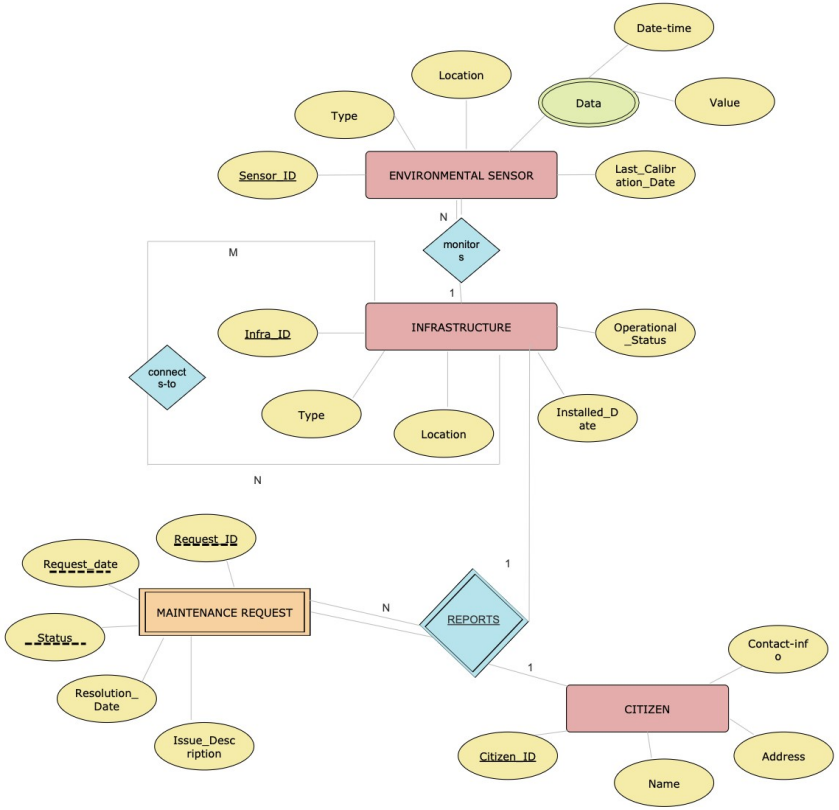
In the same smart urban system, residents often report issues they observe in public infrastructure—such as broken streetlights, potholes, or malfunctioning traffic signals—via a digital platform. Each report contains details about the affected infrastructure, the citizen who raised it, and specifics like the description, date, and status of the issue.

Note that a single citizen may report multiple issues across different infrastructures, and an infrastructure element may receive requests from various citizens. However, a maintenance request cannot exist independently—it must always be associated with both a citizen and an infrastructure element.

Task:

- Using this context**, extend your existing ER model to include this maintenance request process.**
- Be precise in capturing how requests are tracked and related to both citizens and infrastructure.
- Reflect on how the uniqueness of each request might be determined.

Example



In this model, the **Maintenance Request** entity is correctly represented as a **weak entity**, as it depends on both the **Citizen** and the **Infrastructure** entities for its identification and existence.

The attributes associated with the entities are intentionally kept generic to provide flexibility and can be further refined or expanded based on the specific requirements of the implementation context.

With this foundation, you are now well-prepared to **design and interpret Entity-Relationship (ER) models** for real-world systems, ensuring clarity, scalability, and logical consistency in your data modeling practices.

1)Which of the following statements is true about weak entity sets?

A. They can exist without being related to a strong entity
B. They always have a multivalued attribute
C. They have a total participation in the identifying relationship
D. They have their own primary key

2)In an ER diagram, a weak entity set is represented by:

A. A rectangle with a double border
B. A diamond with a dashed border
C. An ellipse with a dashed border
D. A rectangle with a dashed border

3) In an ER diagram, the identifying relationship of a weak entity set is represented by:

A. A dashed line diamond
B. A double diamond
C. A triangle
D. A rectangle with a double border

4) **Which of the following must a weak entity set have?**

A. A discriminator
B. A multivalued attribute
C. A recursive relationship
D. A derived attribute

5)**Role names are used instead of entity set names in primary key formation when:**

A. The relationship has attributes associated with it

B. An entity set participates more than once in the same relationship set

C. The relationship is of many-to-many cardinality

D. The primary key attributes have identical names

6)In a 1:1 binary relationship set, which statement about primary key selection is INCORRECT?

A. The primary key of either participating entity set can be chosen

B. Both entity primary keys together form a minimal superkey

C. Either entity's primary key alone forms a minimal superkey

D. The choice between entity primary keys is arbitrary from a uniqueness perspective

Primary Key In Relationship Sets

7)**If a relationship set R has attributes {a1, a2, ..., am}, these attributes:**

A. Must be included in the primary key of R

B. Are never included in the primary key of R

C. Are included in the super key but not necessarily the primary key of R

D. Replace the entity primary keys in forming the relationship primary key

8)**Which of the following best describes why a weak entity set cannot exist independently?**

A. It lacks a relationship with any other entity

B. It depends on the foreign key of another entity set

C. It does not have a primary key of its own

D. It always participates in recursive relationships

9) **In the ER model, a double line connecting an entity to a relationship signifies:**

A. The relationship is one-to-one

B. Partial participation

C. Total participation of the entity in that relationship

D. Recursive relationship

10)A university policy mandates every student must enroll in at least one course, but instructors may or may not teach any course. What participation constraints apply?

A. Total for both Student and Instructor

B. Partial for both

C. Total for Student, Partial for Instructor

D. Partial for Student, Total for Instructor

11)In min-max notation, if the line from Student to Enrolls has (2, 5), what does it represent?

A. A student can enroll in at most 2 and at least 5 courses

B. Each student must enroll in exactly 5 courses

C. A student can enroll in **minimum 2** and **maximum 5** courses

D. The enrollment is optional

Answers:

1. C
2. A
3. B
4. A
5. B
6. B
7. C
8. C
9. C
10. C
11. C

THANK YOU

Mahitha G
Department of Computer Science and Engineering
mahithag@pes.edu

Database Management Systems

Converting E-R Diagram to Relational Schema

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgments: Dr. Nagasundari S (Dept of CSE), Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 1: Introduction to Database Management and SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G.

Department of Computer Science and Engineering

Reducing ER diagrams to Relational Schema

- Both the E-R model and the relational database model are abstract and logical representations of real-world organizations.
- Due to shared design principles, an E-R design can be transformed into a relational design.
- For each entity set and for each relationship set in the database design, there is a unique relation schema to which we assign the name of the corresponding entity set or relationship set.

Database Management Systems

Reducing ER diagrams to Relational Schema

- Steps to convert ER diagram into a relational schema
  - Mapping of strong entity sets
  - Mapping of weak entity sets
  - Mapping of strong entity sets with complex attributes
  - Compound attributes
  - Derived attributes
  - Multivalued attributes
  - Mapping of relational sets
    - 1:1 or one-to-one
    - 1:N or one-to-many or many-to-one
    - M:N or many-to-many
    - N-ary relations

Representation of Strong Entity Sets

- Let **E** be a strong entity set with only **simple descriptive attributes** a1, a2, …, an.
- We represent this entity with a **schema** called **E** with n distinct attributes.
- Each tuple in a relation on this schema corresponds to one entity of the entity set E.
- For schemas derived from strong entity sets, the primary key of the entity set serves as the primary key of the resulting schema.
- From the university DB:
  - Consider the entity set “student”.
  - Attributes: ID, name, tot_cred.
- We represent this entity set by a schema called student with three attributes:
- **Student** (ID, name, tot cred)

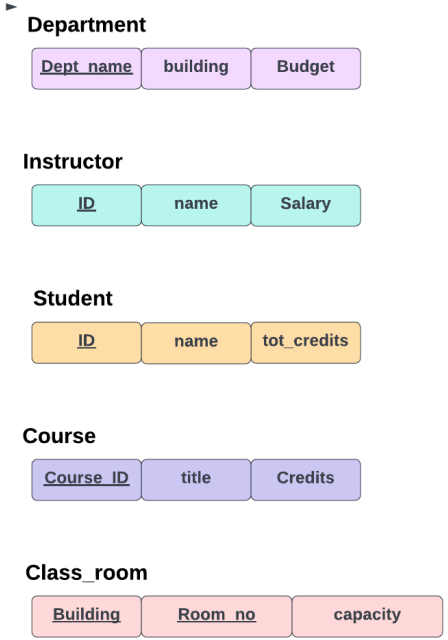
Student

ID	name	tot_credits

Database Management Systems

Representation of Strong Entity Sets

- Similarly converting all the strong entities with simple attributes in the university DB, we get the following schemas:
  - **Classroom**(<u>building, room number</u>, capacity)
  - **Department**(<u>dept name</u>, building, budget)
  - **Course**(<u>course id</u>, title, credits)
  - **Instructor**(<u>ID</u>, name, salary)
  - **Student**(<u>ID</u>, name, tot cred)



Representation of Weak Entity Sets

- Let **A** be a **weak entity set** with attributes {a1, a2, …, am}.
- Let B be the strong entity set on which A depends. Let the **primary key** of **B** consist of attributes {b1, b2, …, bn}.
- We represent the entity set A by a relation schema called A with one attribute for each member of the set:

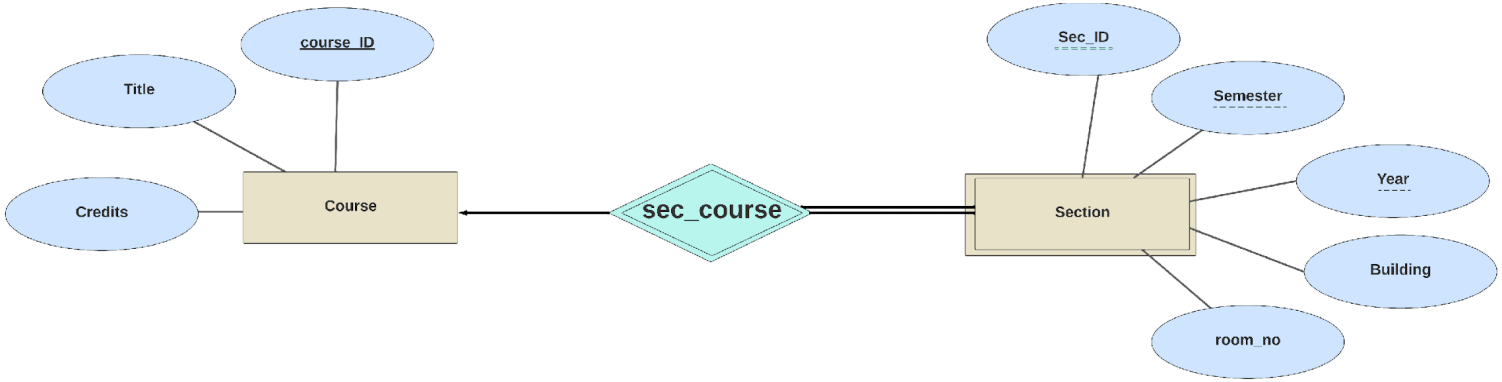
$$\{a1, a2, \ldots, am\} \cup \{b1, b2, \ldots, bn\}$$

- The primary key consists of the primary key of Identifying/strong entity set union with the discriminator of weak entity
- Foreign key constraint is also added to the relational schema of the weak entity (to the primary-key of the identifying entity set)

Database Management Systems

Representation of Weak Entity Sets

- Consider the section entity of the university database as shown

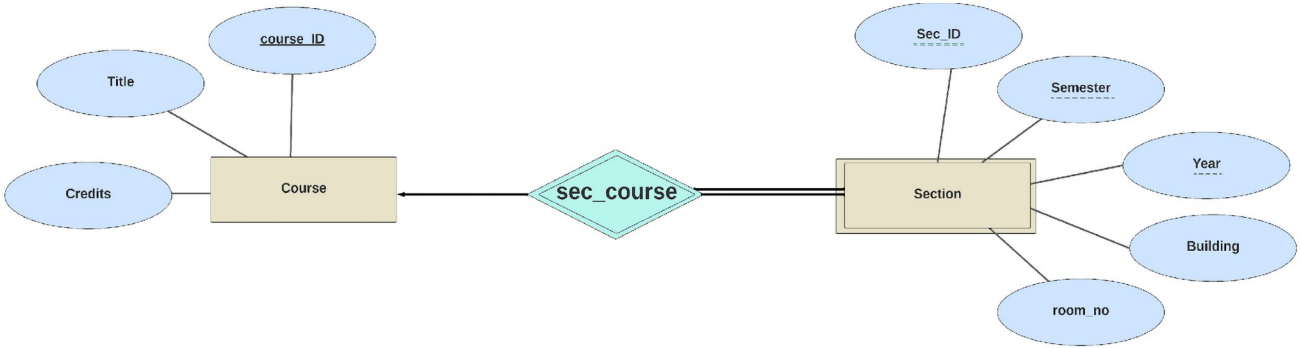


- Relational schema:
  - **Section**(course_id, sec_id, semester, year, building, room_no)

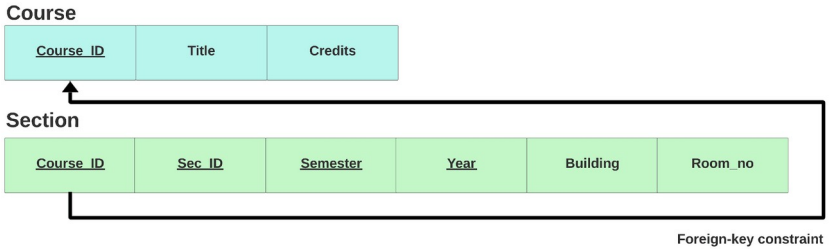
Database Management Systems

Representation of Weak Entity Sets

- Consider the section entity of the university database as shown



- Relational schema with foreign-key constraint:

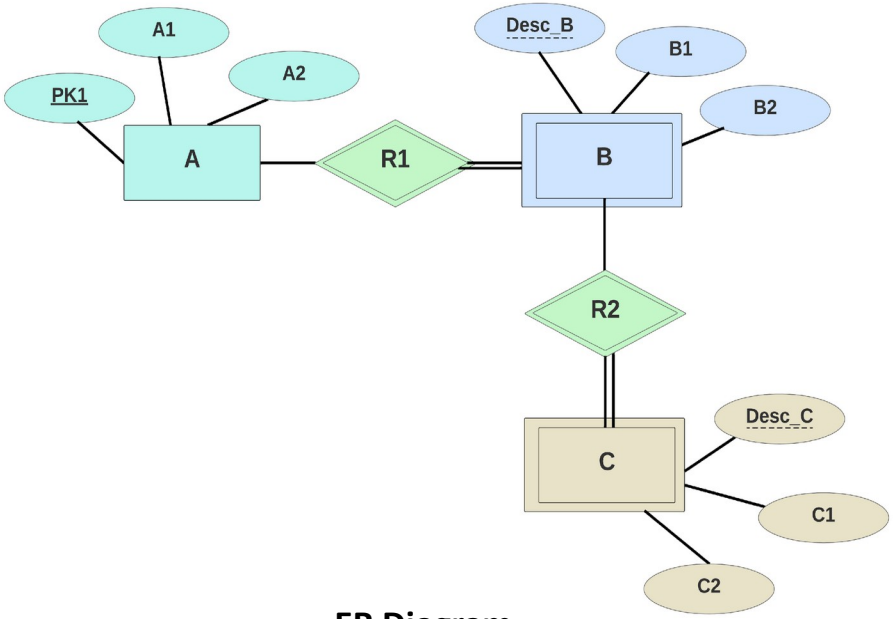


Representation of Weak Entity Sets

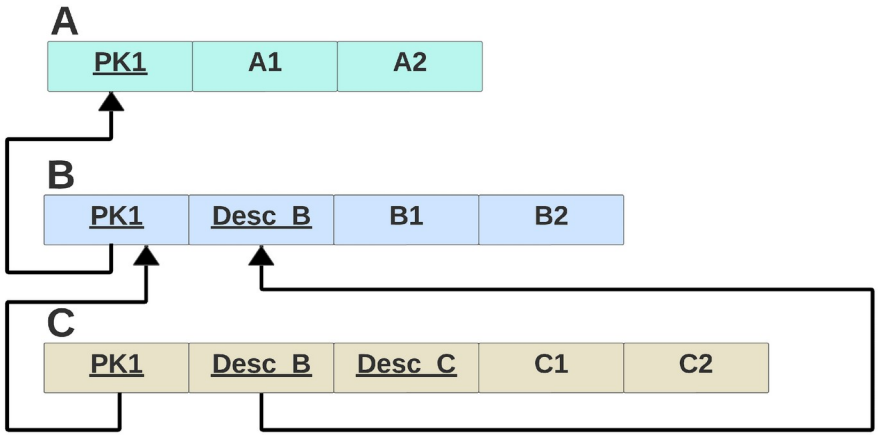
- For schemas derived from a weak entity set, the combination of the primary key of the strong entity set and the discriminator of the weak entity set serves as the primary key of the schema.
- In addition to creating a primary key, we also create a foreign-key constraint on the relation A, specifying that the attributes b1, b2, …, bn reference the primary key of the relation B.
- The foreign-key constraint ensures that for each tuple representing a weak entity, there is a corresponding tuple representing the corresponding strong entity

Representation of Weak Entity Sets

- In general, for the given ER diagram, the relational schema is as given:



ER Diagram

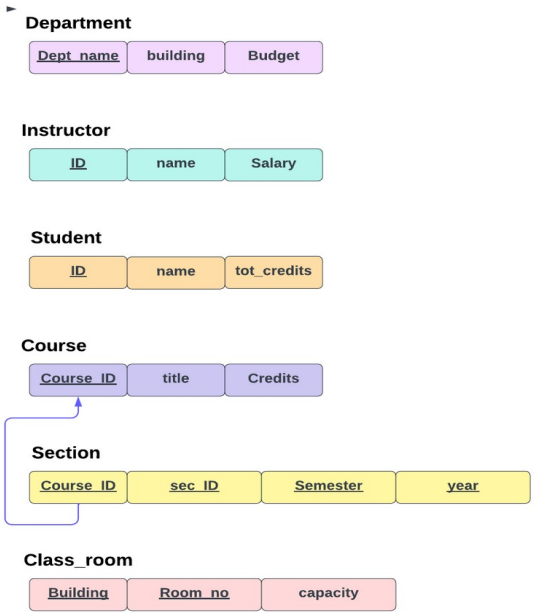


Relational Schema

Database Management Systems

Representation of Weak Entity Sets

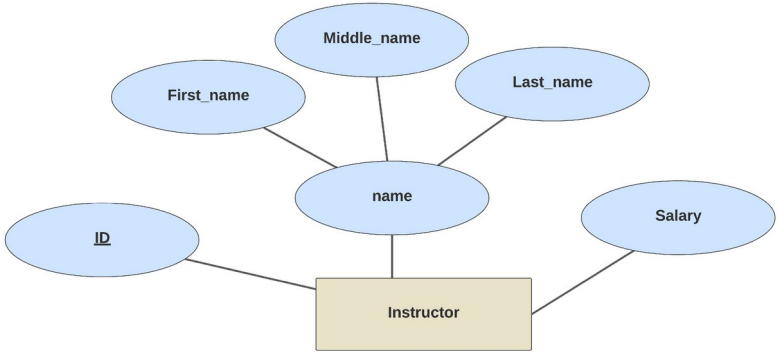
- After incorporating the weak entities the university database would look like:



Representation of Strong Entity Sets with Complex Attributes

COMPOSITE ATTRIBUTES

- Composite attributes are handled by creating a separate attribute for each of the component attributes (Do not create a separate attribute for the composite attribute itself).
- Consider the version of the Instructor entity set as shown.
- For the composite attribute “name”, the schema generated for the instructor contains the attribute’s first name, middle initial, and last name;
- The schema:



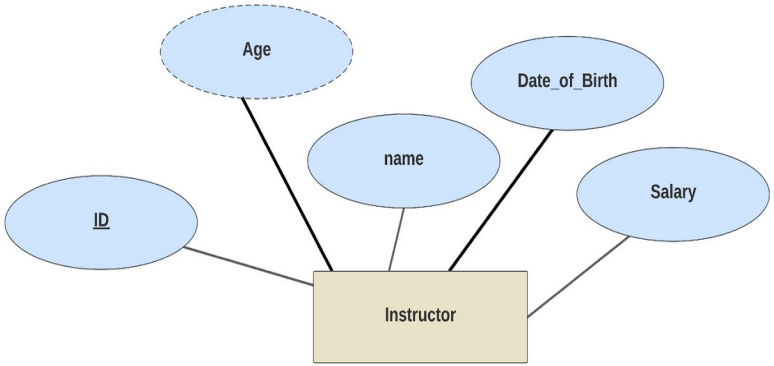
Instructor(ID, First_name, Middle_name, Last_name, Salary)

Representation of Strong Entity Sets with Complex Attributes

DERIVED ATTRIBUTES

- Derived attributes are not explicitly represented in the relational data model.
- They can be represented as stored procedures, functions, or methods in other data models
- Consider the version of the Instructor entity set as shown.
- The derived attribute Age would be eliminated in the relational schema
- The schema:

Instructor(ID, name, Date_of_Birth , Salary)



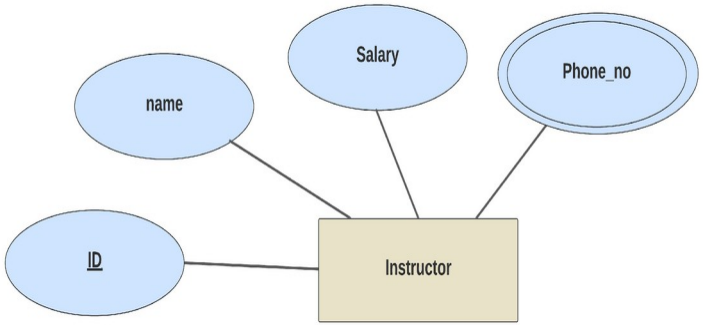
Representation of Strong Entity Sets with Complex Attributes

MULTIVALUED ATTRIBUTES

- For a multivalued attribute M, create a relation schema R with an attribute A that corresponds to M and attributes corresponding to the primary key of the entity set or relationship set of which M is an attribute.
- Consider the Instructor entity set as shown where:
  - Primary key: ID
  - Multivalued attribute: Phone_no
- primary key of the new relation schema **R** consists of all attributes of the schema.
- the relational schema:

  Instructor(<u>ID</u>, name, Salary)

  R : Instructor_phone(<u>ID, Phone_no</u>)

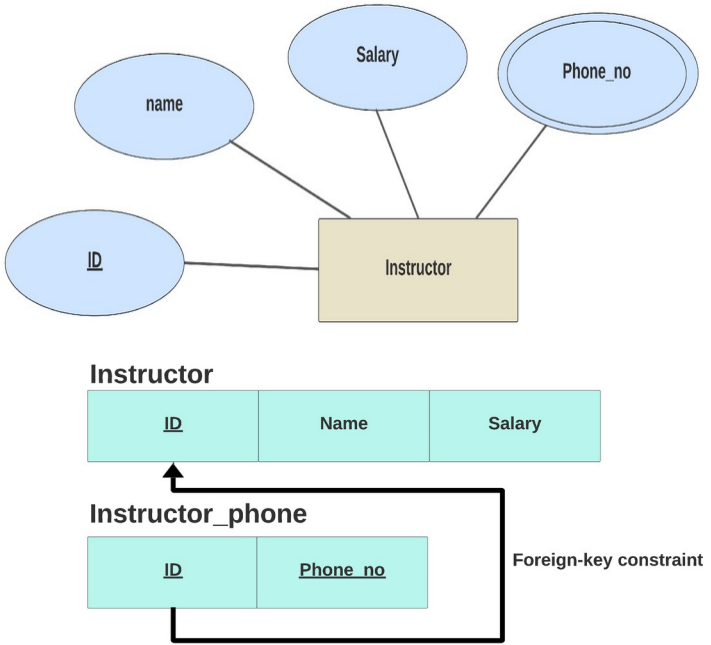


Representation of Strong Entity Sets with Complex Attributes

MULTIVALUED ATTRIBUTES

- A **foreign-key constraint** is to be added to the relation schema created from the multivalued attribute.
- In that newly created schema, the attribute generated from the primary key of the entity set must reference the relation generated from the entity set.
- In the above example, the foreign-key constraint on the instructor phone relation would be that attribute ID references the instructor relation
- Schema:

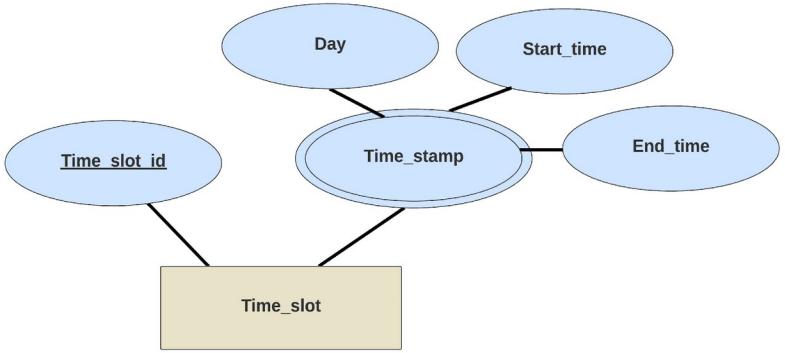
Instructor(ID, name, Salary)
R : Instructor_phone(ID, Phone_no)



Representation of Strong Entity Sets with Complex Attributes

MULTIVALUED ATTRIBUTES

- In the case: entity set consists of only two attributes
  - A single primary-key attribute B
  - A single multivalued attribute M
- The relation schema for the entity set would contain only one attribute, namely, the primary-key attribute B.
- In such cases we drop this relation, while retaining the relation schema with the attribute B and attribute A that corresponds to M.
- In the time_slot entity set shown,
  - B: Time_slot_id
  - M: Time_stamp



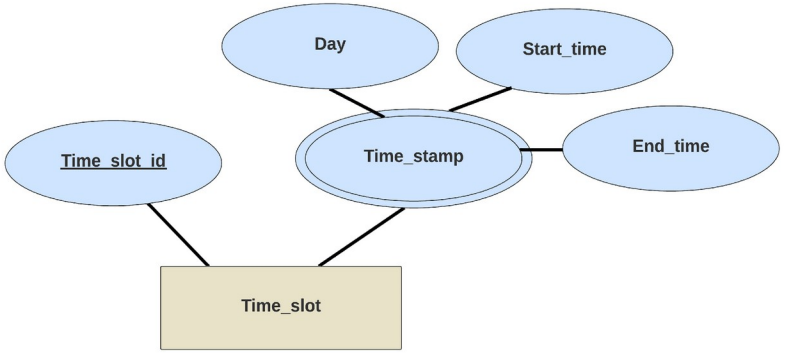
Schema:

Time_slot(Time_slot_id, Day, Start_time, End_time)

Representation of Strong Entity Sets with Complex Attributes

MULTIVALUED ATTRIBUTES

- In the schema shown, the End_time is not included as a part of the primary key
- This is because there can not be two meetings of a class that start at the same time on the same day of the week but end at different times;
- Due to this constraint, the end_time has been omitted from the primary key of the time slot schema.



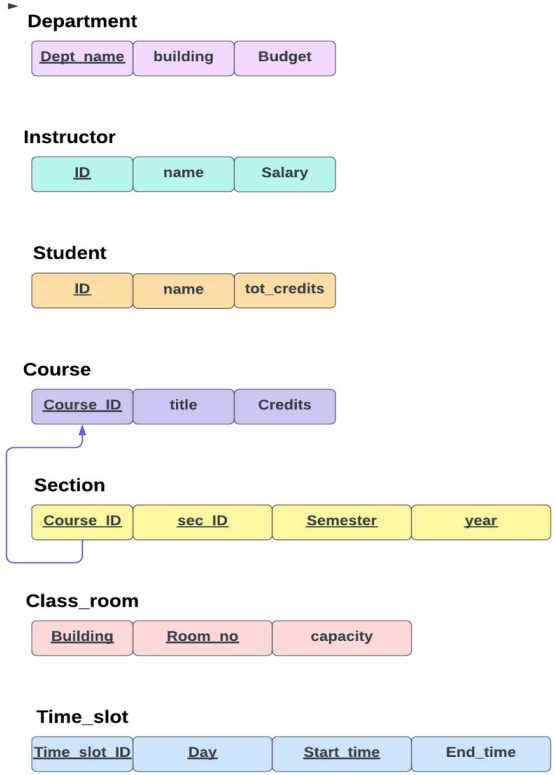
Schema:

Time_slot(Time_slot_id, Day, Start_time, End_time)

Database Management Systems

Representation of Strong Entity Sets with Complex Attributes

- After incorporating the complex attributes the university database would look like



Representation of Relationship Sets

Mapping of Binary 1:1 Relation Types

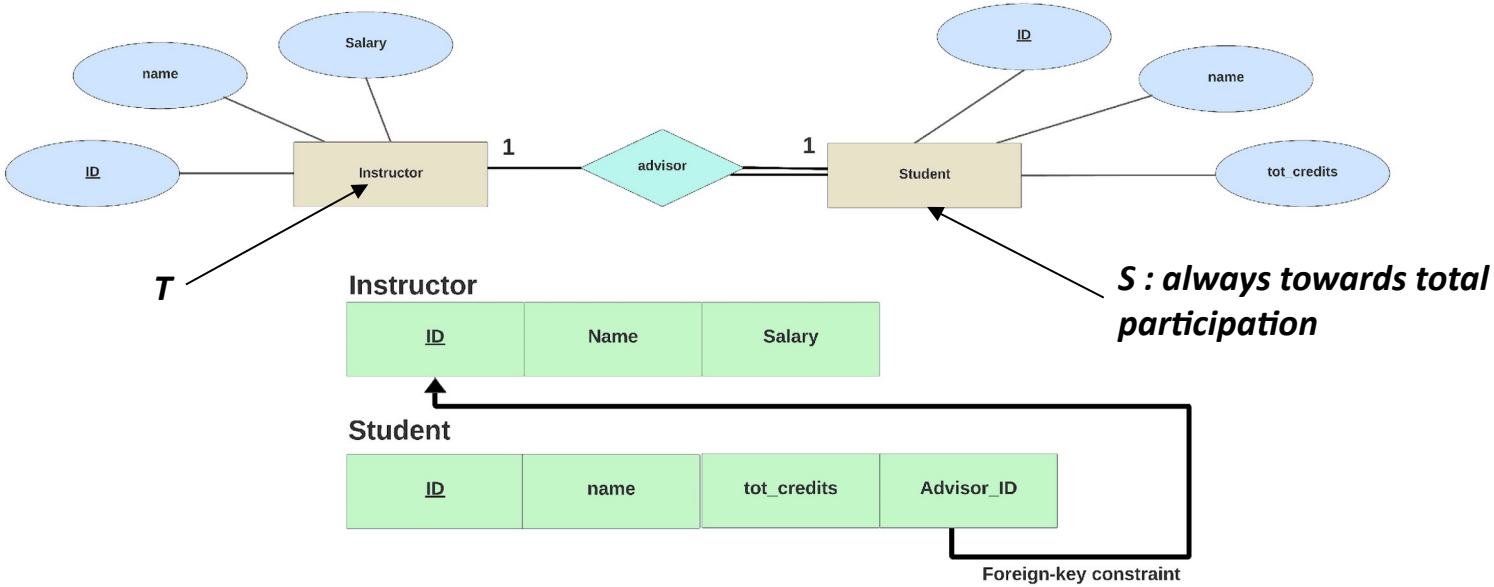
- For each binary 1:1 relationship type R in the ER schema, identify the relations S and T that correspond to the entity types participating in R.
- There are three possible approaches:
  - **Foreign Key (2 relations) approach:**
  - **Merged relation (1 relation) option:**
  - **Cross-reference or relationship relation (3 relations) option:**

Representation of Relationship Sets

- There are three possible approaches:
  - **Foreign Key (2 relations) approach:**
    - Choose one of the relations-say S and include a foreign key in S the primary key of T.
    - It is better to choose an entity type with total participation in R in the role of S. And add all the descriptive attributes of the relation
  - **Merged relation (1 relation) option:**
    - This alternative, we merge the two entity types and the relationship into a single relation.
    - This may be appropriate when both participations are total. And add all the descriptive attributes of the relation
  - **Cross-reference or relationship relation (3 relations) option:**
    - The third alternative is to set up a third relation R for the purpose of cross-referencing the primary keys of the two relations S and T representing the entity types. And add all the descriptive attributes of the relation

Representation of Relationship Sets

- Consider for this situation, the advisor relation to be one-to-one or 1:1 (Foreign key approach)



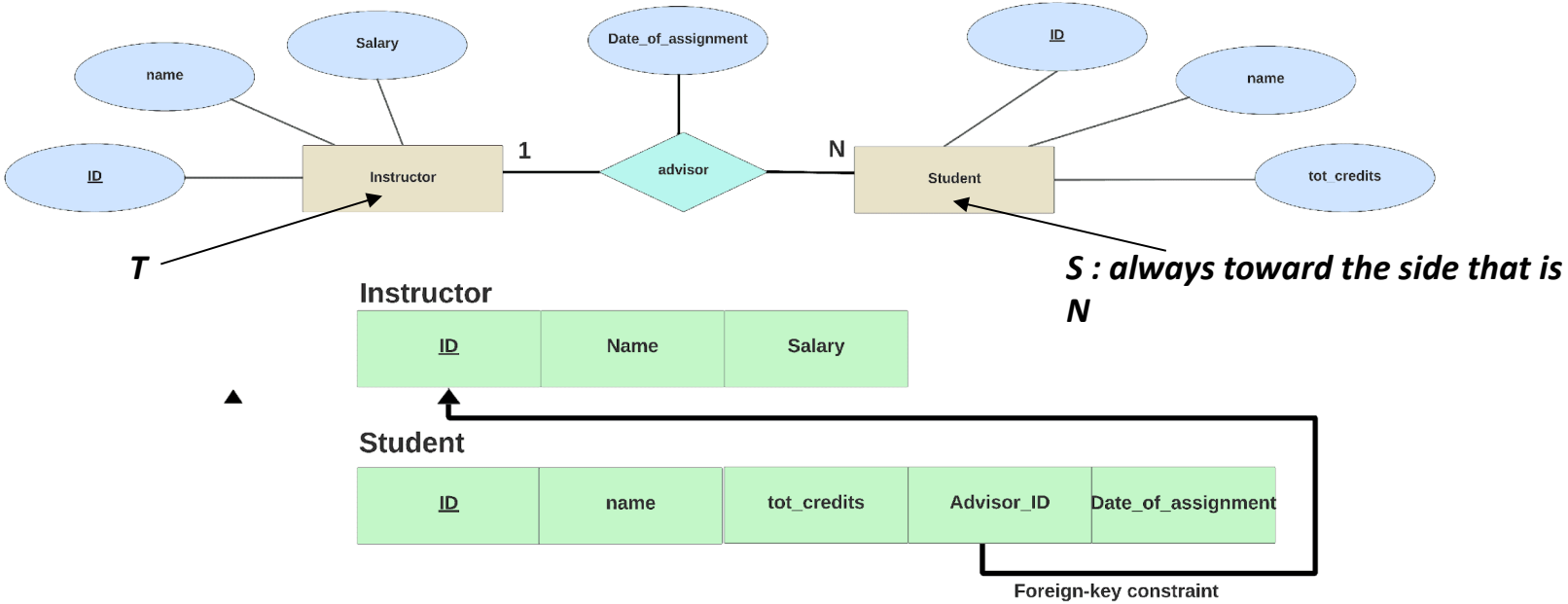
Representation of Relationship Sets

Mapping of Binary 1:N Relation Types

- For each regular binary 1:N relationship type R, identify the relation S that represent the participating entity type at the N-side of the relationship type.
- Include as the foreign key in S the primary key of the relation T that represents the other entity type participating in R.
- Include any simple attributes of the 1:N relation type as attributes of S.

Representation of Relationship Sets

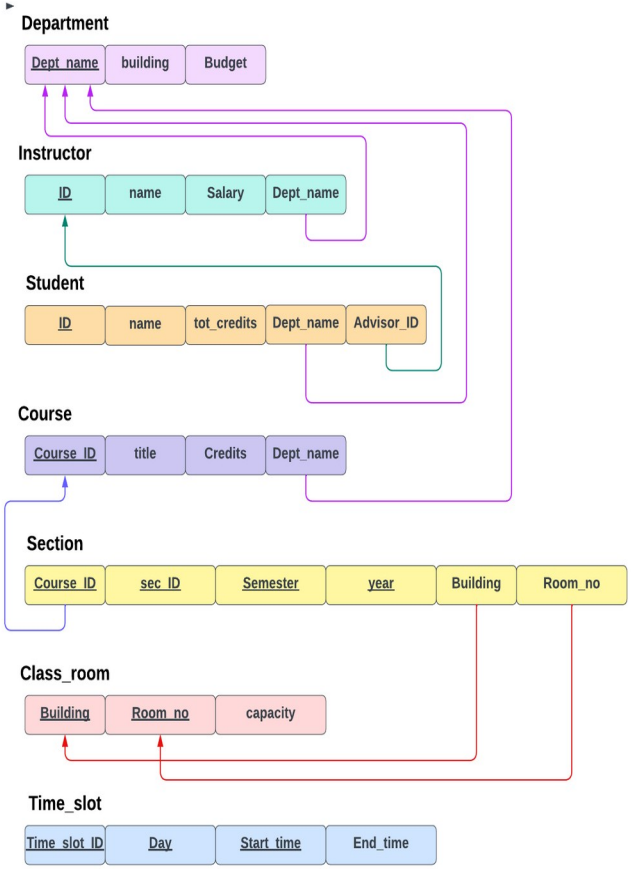
- Consider for this situation, the advisor relation to be one-to-many or 1:N



Database Management Systems

Representation of Relationship Sets

- After incorporating the 1:1 and 1:N relations in the university database would look as shown



Representation of Relationship Sets

PES UNIVERSITY
CELEBRATING 50 YEARS

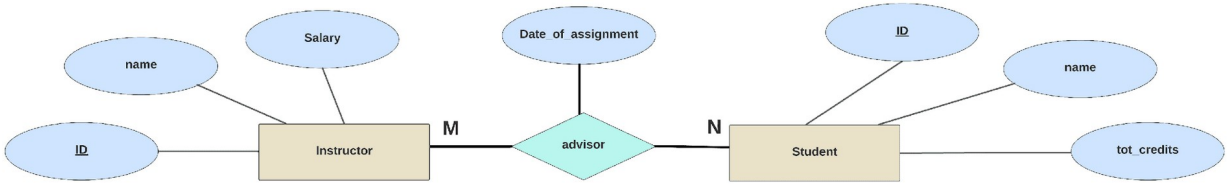
Mapping of Binary M:N Relation Types

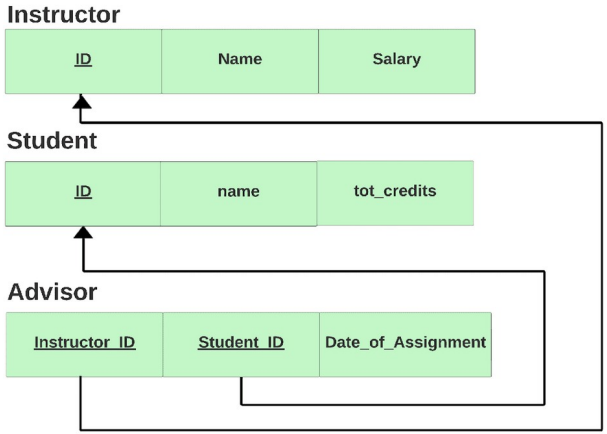
- For each regular binary M:N relationship type R, create a new relation S to represent R. This is a relationship relation.
- Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types; their combination will form the primary key of S.
- Also include any simple attributes of the M:N relationship type (or simple components of composite attributes) as attributes of S.

Database Management Systems

Representation of Relationship Sets

Consider for this situation, the advisor relation to be many-to-many or M:N

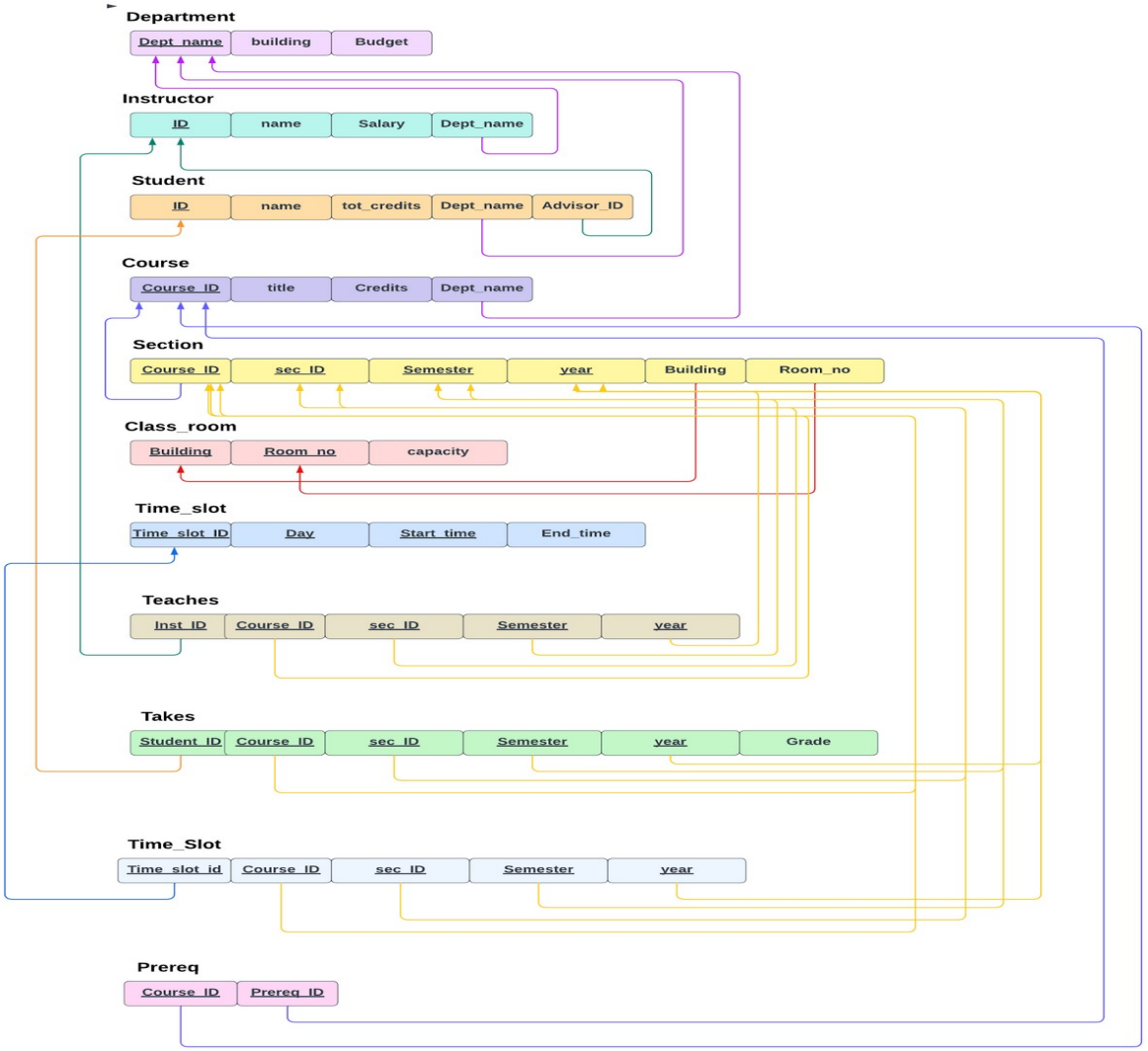




Database Management Systems

Representation of Relationship Sets

- After incorporating the M:N relations in the university database would look as shown



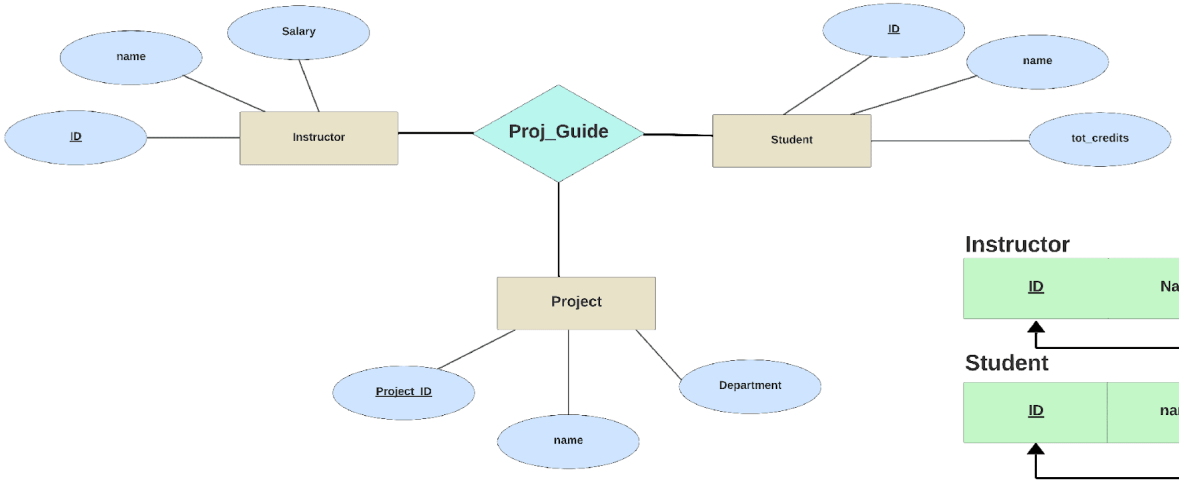
Representation of Relationship Sets

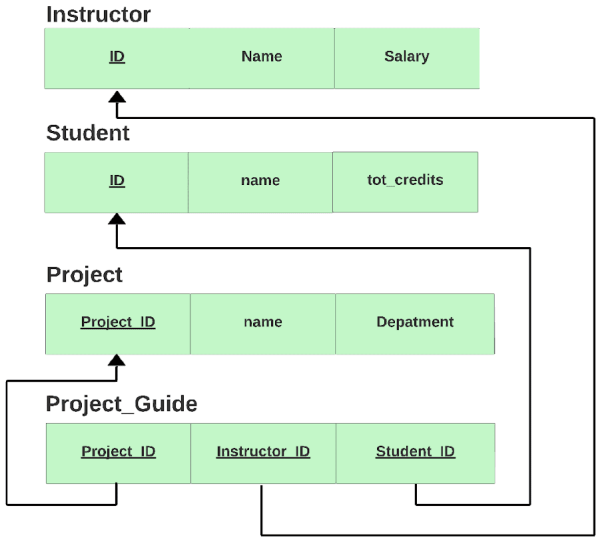
Mapping n-ary relationship sets

- For each n-ary relationship type R, where n>2, create a new relationship S to represent R.
- Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types.
- Also include any simple attributes of the n-ary relationship type (or simple components of composite attributes) as attributes of S.
- Considering the project_guide ternary relationship discussed earlier, the ER and the corresponding Schema is given in the following slide

Database Management Systems

Representation of Relationship Sets





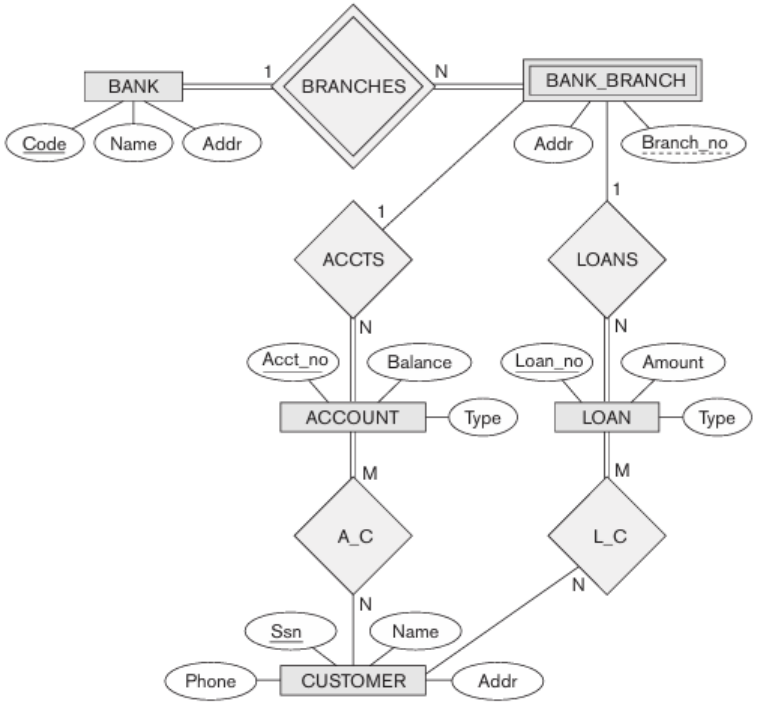
Question

Consider the ER diagram shown in the figure for part of a BANK database.

Each bank can have multiple branches, and each branch can have multiple accounts and loans.

- List the strong entity types.
- Is there a weak entity type? If so, give its name,partial key, and identifying relationship.
- What constraints do the partial key and the identifying relationship of the weak entity type specify in this diagram?

.



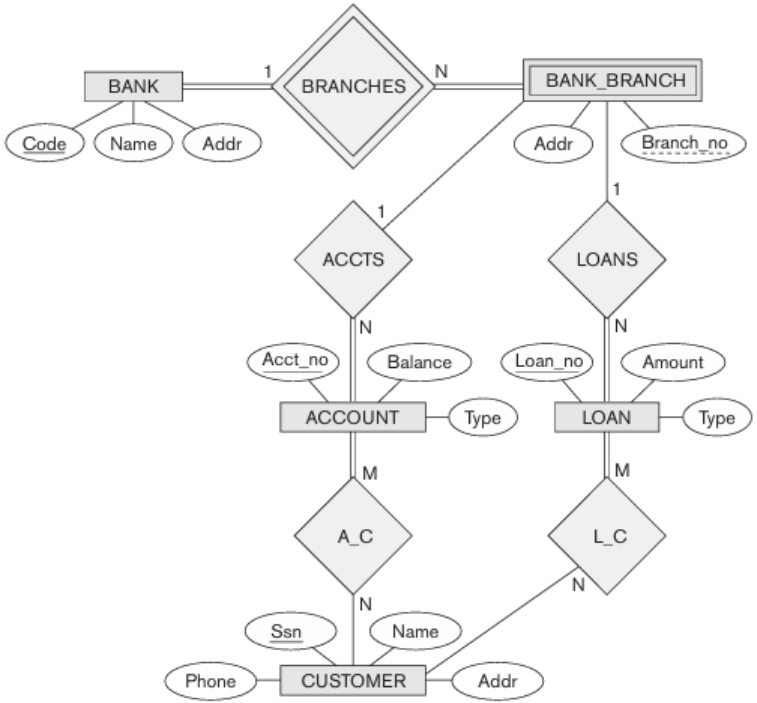
Answers

- List the strong entity types.

  LOAN, BANK, ACCOUNT and CUSTOMER

- Is there a weak entity type? If so, give its name,partial key, and identifying relationship.

  Yes, there is a weak entity BANK_BRANCH and its identifying relationship is BRANCHES.

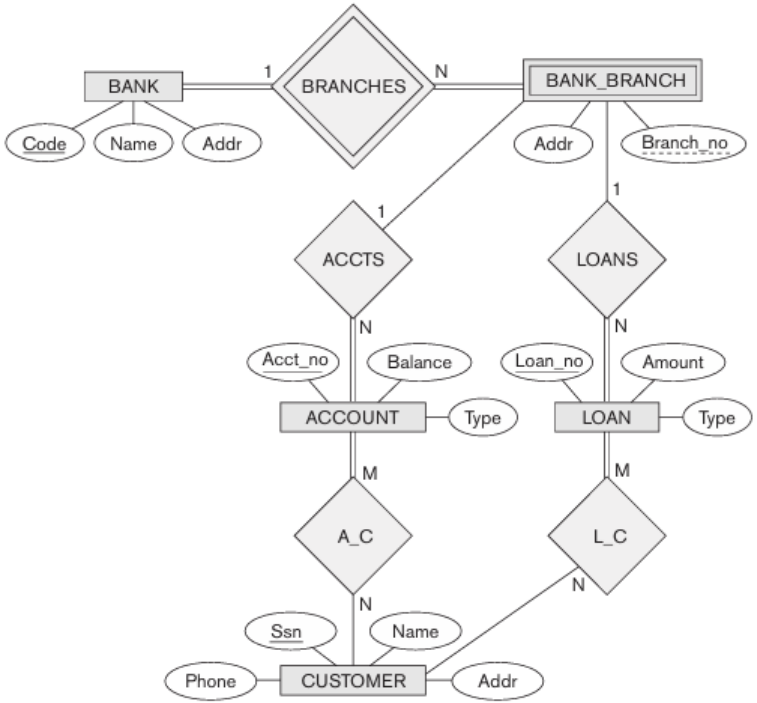


Answers

- What constraints do the partial key and the identifying relationship of the weak entity type specify in this diagram?

The constraints are:

1. **No two branches have the same number**
2. **A bank can have any number of branches but a branch only belongs to one bank.**



1. Which of the following is the first step in converting an ER diagram into a relational schema?

A) Mapping weak entity sets
B) Mapping strong entity sets
C) Mapping relationships
D) Mapping composite attributes

2. What is the primary key in the relational schema for a weak entity set?

A) The discriminator of the weak entity set
B) The primary key of the identifying entity set
C) The combination of the weak entity's discriminator and the identifying entity's primary key
D) A randomly generated primary key

3. In the relational schema for a multivalued attribute, which constraint is added?

A) Primary key constraint

B) Foreign key constraint

C) Unique constraint

D) No constraint is added

4. How is a derived attribute handled in a relational schema?

A) It is stored as a regular attribute

B) It is ignored in the schema

C) It is represented using a stored procedure or function

D) It is merged with other attributes

Database Management Systems

Q & A

Answers:

1. B
2. C
3. B
4. C

THANK YOU

Mahitha G
Department of Computer Science and Engineering
mahithag@pes.edu

Database Management Systems

Relational Operations

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgments: Dr. Nagasundari S (Dept of CSE), Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 1: Introduction to Database Management and SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G.

Department of Computer Science and Engineering

Relational Algebra

A procedural language that consists of a set of operations, where each operation takes one or two relations as input and produces a new relation as the result.

- The **result** of an operation is a **new relation**, *which is formed from one or more input relations.*
- This property makes **the algebra "closed"** (all objects in relational algebra are relations)
- These relations can be further manipulated using operations of the same algebra.
- A **sequence of relational algebra** operations forms a **relational algebra expression**.
- **The result of a relational algebra expression** ➔ **relation** that represents the result of a database query.

Relational Algebra

Operators:

Unary Operators:

Select: σ

Project: ∏

Rename: ρ

Binary Operators:

Union: U

Intersection :∩

Set difference: –

Cartesian product: X

Join: ⋈

Relational Algebra

Use this table for query examples:

University Database sample table/relation

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Database Management Systems

Example-2 Company Database State

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

Unary Operator: Select (σ)

- The **SELECT** operation is used to choose a subset of the tuples from a relation that satisfies a **selection condition.**
- We can consider the SELECT operation to be a filter that keeps only those tuples that satisfy a qualifying condition.

  Notation: $\sigma_p(r)$

- *p* is called the **selection predicate**
- The SELECT operation can also be visualized as a horizontal partition of the relation into two sets of tuples—those tuples that satisfy the condition and are selected, and those tuples that do not satisfy the condition and are filtered out.

Database Management Systems

Unary Operator: Select (σ)

- Example 1: Select those tuples of the *instructor* relation where the instructor is in the "Physics" department.
- Query: σ(Dept_Name = "Physics")(*Instructor*)

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

- Example 2: Select the tuples for all employees who either work in department 4 and make over $25,000 per year, or work in department 5 and make over $30,000.
- Query: *σ(Dno=4 AND Salary>25000) OR (Dno=5 AND Salary>30000)(EMPLOYEE)*

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5

Unary Operator: Select (σ)

PES UNIVERSITY

SELECT Operation Properties

- The SELECT operation σ (<condition>) produces a relation S that has the same schema (same attributes) as R
- SELECT is commutative: $\sigma_{<condition1>}(\sigma_{<condition2>}(R)) = \sigma_{<condition2>}(\sigma_{<condition1>}(R))$

- Because of commutative property, A cascade (sequence) of SELECT operations may be applied in any order:

$\sigma_{<cond1>}(\sigma_{<cond2>}(\sigma_{<cond3>}(R)) = \sigma_{<cond2>}(\sigma_{<cond3>}(\sigma_{<cond1>}(R)))$

or

$\sigma_{<cond1>}(\sigma_{<cond2>}(\sigma_{<cond3>}(R)) = \sigma_{<cond1> \text{ AND } <cond2> \text{ AND } <cond3>}(R))$

- We allow comparisons to be used in the selection predicate.
  $=, \neq, >, \geq. <. \leq$

- We can combine several predicates into a larger predicate by using the connectives:
  $\wedge$ (**and**), $\vee$ (**or**), $\neg$ (**not**)

Example: Find the instructors in Physics with a salary greater $90,000, we write:

$$\sigma_{dept_name=\text{“Physics”} \wedge salary > 90,000}(instructor)$$

- The select predicate may include comparisons between two attributes.

Example: Fetch all the details of the ‘Research’ department.

$$\sigma_{dept_name=\text{“Research”}}(department)$$

NOTE: *The number of tuples in the result of a SELECT is less than (or equal to) the number of tuples in the input relation R.*

Unary Operator: PROJECT(∏)

PES UNIVERSITY

PROJECT(∏)

- A **unary operation** that returns its argument relation, with certain attributes left out.

  Notation: $\prod_{A1,A2,A3 \ldots Ak} (r)$

  where A_1, A_2, …, A_k are attribute names and *r* is a relation name.

- The **PROJECT operation**, *selects certain columns from the table and discards the other columns.*
- **Duplicate rows removed from result**, since relations are sets
- This operation keeps certain columns (attributes) from a relation and discards other columns.
- The PROJECT operation can be visualized as a **vertical partition** of the relation
- The list of specified columns (attributes) is kept in each tuple. The other attributes in each tuple are discarded.

Unary Operator: PROJECT(∏)

- **Example 1:**

Eliminate the *dept_name* attribute of *instructor*

Query*:*

$$\Pi_{ID,\ name,\ salary}\ (instructor)$$

Result:

ID	*name*	*salary*
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

Unary Operator: PROJECT(∏)

- **Example 2: Fetch the Lname, Fname, Salary information about the Employees.**

Query:

$$\prod_{Lname,\ Fname,\ Salary} (EMPLOYEE)$$

Lname	Fname	Salary
Smith	John	30000
Wong	Franklin	40000
Zelaya	Alicia	25000
Wallace	Jennifer	43000
Narayan	Ramesh	38000
English	Joyce	25000
Jabbar	Ahmad	25000
Borg	James	55000

output for example 2

- **Example 3: Fetch the Sex and Salary information about the Employees.**

Query:

$$\prod_{Sex,\ Salary} (EMPLOYEE)$$

Sex	Salary
M	30000
M	40000
F	25000
F	43000
M	38000
M	25000
M	55000

output for example 3

Unary Operator: PROJECT(∏)

Properties of PROJECT operator:

- The project **operation removes any duplicate tuples.**
- This is because the result of the project operation must be a set of tuples.
- Mathematical sets do not allow duplicate elements.
- The number of tuples in the result of projection ∏(R) is always less or equal to the number of tuples in R.
- If the list of attributes includes a key of R, then the number of tuples in the result of PROJECT is equal to the number of tuples in R.

PROJECT is *not* commutative

- $\pi_{<list1>}(\pi_{<list2>}(R)) = \pi_{<list1>}(R)$ as long as <list2>

  contains the attributes in <list1>

Composition of Relational Operations

- The result of **a relational-algebra operation is a relation** and therefore of relational-algebra operations can be composed together into a **relational-algebra expression**.

Example: Find the names of all instructors in the Physics department.

$$\Pi_{name}(\sigma_{dept_name = \text{“Physics”}} (instructor))$$

Instead of giving the name of a relation as the argument of the PROJECT operation, we give an expression that evaluates to a relation.

- We may want to apply several relational algebra operations one after the other.
- Either we can write the operations as a single relational algebra expression by nesting the operations, or we can apply one operation at a time and create intermediate result relations.

Relational Algebra Expressions

- To retrieve the first name, last name and salary of all employees who work in the department 5, we must apply a select and a project operation.
- We can write a single relational algebra expression as follows:

$$\prod \text{Fname,Lname,Salary}(\sigma\ \text{DNo} = 5(\text{Employee}))$$

- Or we can explicitly show the sequence of operations, giving a name to each immediate relation:

$$\text{DEP5_EMPS} \leftarrow \sigma_{DNO=5}\ (Employee)$$

$$\text{RESULT} \leftarrow \prod_{Fname,Lname,Salary}\ (DEP5_EMPS)$$

Unary Operator: RENAME (ρ)

- The general RENAME operation r can be expressed by any of the following forms:

  $\mathbf{r_{S\,(B1,\,B2,\,\ldots,\,Bn\,)}(R)}$ **changes both**:

  - the relation name to S, *and*
  - the column (attribute) names to B1, B2, …..Bn

  $r_S(R)$ changes:

  - the *relation name* only to S

  $r_{(B1,\,B2,\,\ldots,\,Bn\,)}(R)$ changes:

  - the *column (attribute) names* only to B1, B2, …..Bn

Unary Operator: RENAME (ρ)

- **The results of relational-algebra expressions do not have a name that we can use to refer to them. The rename operator, *ρ*, is provided for that purpose.**
- The RENAME operator is denoted by ρ(rho)
- The expression:

$$\rho_x(E)$$

returns the result **of expression *E* under the name *x***

- Another form of the rename operation:

$$\rho_{x(A1,A2,..An)}(E)$$

$S(B1, B2, \ldots, Bn)(\rho)$ or $S(\rho)$ or $(B1, B2, \ldots, Bn)(\rho)$

- In some cases, we may want to rename the attributes of a relation or the relation's name or both.
- Useful when a query requires multiple operations

Unary Operator: RENAME (ρ)

- This is **a unary operator which changes attribute names for a relation without changing any values.**
- Renaming removes the limitations associated with set operators.
- **Notation:** $\rho_{OldName->NewName}(r)$
- For Example, $\rho_{Father->Parent}(Paternity)$. For the given operation, the attribute "Father" will be renamed as "Parent" for the relation Paternity.
- If there are two or more attributes involved in renaming operation, then ordering is more meaningful.

$$\rho_{Brach,Salary->Location,Pay}(Employee)$$

Unary Operator: RENAME (ρ)

- RESULT will have the *same attribute names* as DEP5_EMPS (same attributes as EMPLOYEE

$$RESULT \leftarrow \pi_{FNAME, LNAME, SALARY}(DEP5_EMPS)$$

- The 10 attributes of DEP5_EMPS are *renamed* to F, M, L, S, B, A, SX, SAL, SU, DNO, respectively
  Note: ⟵ the symbol is an assignment operator

$$RESULT(F, M, L, S, B, A, SX, SAL, SU, DNO) \leftarrow \rho_{RESULT(F.M.L.S.B,A,SX,SAL,SU,DNO)}(DEP5_EMPS)$$

The Assignment Operator (←)

- It is convenient at times to write a relational-algebra expression by assigning parts of it to temporary relation variables.
- The assignment operation is denoted by ← and works like assignment in a programming language.
- Example: Fetch the FirstName, LastName and Salary of all employees who work in department 5.

$$DEP5_EMPS \leftarrow \sigma_{DNO=5}(EMPLOYEE)$$

$$RESULT \leftarrow \pi_{FNAME, LNAME, SALARY}(DEP5_EMPS)$$

- With the assignment operation, a query can be written as a sequential program consisting of a series of assignments followed by an expression whose value is displayed as the result of the query.

The Assignment Operator (←)

Example:

(a)

Fname	Lname	Salary
John	Smith	30000
Franklin	Wong	40000
Ramesh	Narayan	38000
Joyce	English	25000

$$TEMP \leftarrow \sigma_{Dno=5}(EMPLOYEE)$$
$$R(First_name, Last_name, Salary) \leftarrow \pi_{Fname, Lname, Salary}(TEMP)$$

(b)

TEMP

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston,TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston,TX	M	40000	888665555	5
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble,TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5

R

First_name	Last_name	Salary
John	Smith	30000
Franklin	Wong	40000
Ramesh	Narayan	38000
Joyce	English	25000

Figure 8.2
Results of a sequence of operations. (a) $\pi_{Fname, Lname, Salary}$ ($\sigma_{Dno=5}$(EMPLOYEE)).
(b) Using intermediate relations and renaming of attributes.

Questions

- **Scenario:** You have a table Employees with columns EmployeeID, Name, Department, Salary, and JoiningYear (eg:2016).

Question: How would you create a query to find the names and departments of employees who joined before 2020, but without showing the EmployeeID and Salary columns?

- **Scenario:** A database contains a Customers table with fields CustomerID, FirstName, LastName, City, and PurchaseAmount.

Question: If you need to rename the City column to Location, write the relational query for this scenario.

- **Scenario:** You are working with a table Products with columns ProductID, ProductName, Category, Price, and StockQuantity.

Question: How would you select the ProductName and Price of all products that are in the Electronics category and have more than 10 items in stock?

Questions

- **Scenario:** You have a table Employees with columns EmployeeID, Name, Department, Salary, and JoiningYear (eg:2016).

Question: How would you create a query to find the names and departments of employees who joined before 2020, but without showing the EmployeeID and Salary columns?

Answer: ∏Name,Department(σ JoiningYear<2020(Employees))

- **Scenario:** A database contains a Customers table with fields CustomerID, FirstName, LastName, City, and PurchaseAmount.

Question: If you need to rename the City column to Location, write the relational query for this scenario.

Answer: $\rho_{City->Location}$ (Customers)

Questions

- **Scenario:** You are working with a table Products with columns ProductID, ProductName, Category, Price, and StockQuantity.

Question: How would you select the ProductName and Price of all products that are in the Electronics category and have more than 10 items in stock?

Answer: $\prod$ProductName,Price(σ Category='Electronics' $\wedge$ StockQuantity>10(Products))

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Set Operations,Equivalent Queries

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgments: Dr. Nagasundari S (Dept of CSE), Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 1: Introduction to Database Management and SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G.

Department of Computer Science and Engineering

What are set operations ?

- Union Operation U
- Set-Intersection Operation ∩
- Set Difference Operation –

Set Operations : UNION Operation

- Set Operations are Binary.
- These operations can be applied on the relations which are union compatible.
- Two relations are said to be union compatible if
    1. Both the relations have the *same* **arity** (number of attributes) and
    2. The corresponding attribute domains must be **compatible** (example: 2nd column of *r* should have the same type of values as does the 2nd column of *s*).

Union operation:

- The result of R U S, is a relation that includes all tuples that are either in R or in S or in both R and S. (R and S should be union compatible)
- Duplicate tuples are eliminated.
- The union operation allows us to combine two relations. Notation: $r \cup s$

Database Management Systems

University Database

PES UNIVERSITY

Instructor Table:

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Course Table:

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

Prerequisites Table:

course_id	*prereq_id*
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-101
CS-319	CS-101
CS-347	CS-101
EE-181	PHY-101

Department Table:

dept_name	*building*	*budget*
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

Figure 2.5 The *department* relation.

Section Table:

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2017	Painter	514	B
BIO-301	1	Summer	2018	Painter	514	A
CS-101	1	Fall	2017	Packard	101	H
CS-101	1	Spring	2018	Packard	101	F
CS-190	1	Spring	2017	Taylor	3128	E
CS-190	2	Spring	2017	Taylor	3128	A
CS-315	1	Spring	2018	Watson	120	D
CS-319	1	Spring	2018	Watson	100	B
CS-319	2	Spring	2018	Taylor	3128	C
CS-347	1	Fall	2017	Taylor	3128	A
EE-181	1	Spring	2017	Taylor	3128	C
FIN-201	1	Spring	2018	Packard	101	B
HIS-351	1	Spring	2018	Painter	514	C
MU-199	1	Spring	2018	Packard	101	D
PHY-101	1	Fall	2017	Watson	100	A

Teaches Table:

ID	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2017
10101	CS-315	1	Spring	2018
10101	CS-347	1	Fall	2017
12121	FIN-201	1	Spring	2018
15151	MU-199	1	Spring	2018
22222	PHY-101	1	Fall	2017
32343	HIS-351	1	Spring	2018
45565	CS-101	1	Spring	2018
45565	CS-319	1	Spring	2018
76766	BIO-101	1	Summer	2017
76766	BIO-301	1	Summer	2018
83821	CS-190	1	Spring	2017
83821	CS-190	2	Spring	2017
83821	CS-319	2	Spring	2018
98345	EE-181	1	Spring	2017

Figure 2.7 The *teaches* relation.

Set Operations : UNION Operation

Schema:

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*

Example: Find all courses (course_id) taught in the Fall 2017 semester, or in the Spring 2018 semester, or in both

Database Management Systems

Set Operations : UNION Operation

Query:

$$\prod_{course_id} (\sigma_{semester=\text{“Fall”} \wedge year=2017} (section)) \cup \prod_{course_id} (\sigma_{semester=\text{“Spring”} \wedge year=2018} (section))$$

Table:

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2017	Painter	514	B
BIO-301	1	Summer	2018	Painter	514	A
CS-101	1	Fall	2017	Packard	101	H
CS-101	1	Spring	2018	Packard	101	F
CS-190	1	Spring	2017	Taylor	3128	E
CS-190	2	Spring	2017	Taylor	3128	A
CS-315	1	Spring	2018	Watson	120	D
CS-319	1	Spring	2018	Watson	100	B
CS-319	2	Spring	2018	Taylor	3128	C
CS-347	1	Fall	2017	Taylor	3128	A
EE-181	1	Spring	2017	Taylor	3128	C
FIN-201	1	Spring	2018	Packard	101	B
HIS-351	1	Spring	2018	Painter	514	C
MU-199	1	Spring	2018	Packard	101	D
PHY-101	1	Fall	2017	Watson	100	A

Output:

course_id
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101

Set Operations : UNION Operation

What will be the first Column Name after Union of Paternity and Maternity Relation?

Is It Meaningful?

Table:

Paternity

Father	Child
Adam	Cain
Adam	Abel
Abraham	Isaac
Abraham	Ishmael

Maternity

Mother	Child
Eve	Cain
Eve	Seth
Sarah	Isaac
Hagar	Ishmael

Paternity ∪ Maternity ???

- The problem: **Father** and **Mother** are different names, but both represent a parent.

Output:

Renaming and Union

Paternity

Father	Child
Adam	Cain
Adam	Abel
Abraham	Isaac
Abraham	Ishmael

Maternity

Mother	Child
Eve	Cain
Eve	Seth
Sarah	Isaac
Hagar	Ishmael

$\rho_{Father \to Parent}(Paternity) \cup \rho_{Mother \to Parent}(Maternity)$

Parent	Child
Adam	Cain
Adam	Abel
Abraham	Isaac
Abraham	Ishmael
Eve	Cain
Eve	Seth
Sarah	Isaac
Hagar	Ishmael

Set Operations : UNION Operation

- To retrieve the social security numbers of all employees who either work in department 5 (RESULT1 below) or directly supervise an employee who works in department 5 (RESULT2 below).
- We can use the UNION operation as follows:

$$DEP5_EMPS \leftarrow \sigma_{DNO=5}(EMPLOYEE)$$

$$RESULT1 \leftarrow \pi_{SSN}(DEP5_EMPS)$$

$$RESULT2(SSN) \leftarrow \pi_{SUPERSSN}(DEP5_EMPS)$$

$$RESULT \leftarrow RESULT1 \cup RESULT2$$

- The union operation produces the tuples that are in either RESULT1 or RESULT2 or both.

Set Operations : UNION Operation

RESULT ← RESULT1 ∪ RESULT2.

RESULT1

Ssn
123456789
333445555
666884444
453453453

RESULT2

Ssn
333445555
888665555

RESULT

Ssn
123456789
333445555
666884444
453453453
888665555

Set Operations : Intersection Operation

- The set-intersection operation allows us to find tuples that are in both the input relations.

  Notation: $r \cap s$

- r, s have the *same* ***arity*** attributes of r and s are **compatible**

Example: Find the set of all courses taught in both the **Fall 2017 and the Spring 2018** semesters.

Query: $\prod_{course_id} (\sigma_{semester="Fall" \wedge year=2017} (section)) \cap \prod_{course_id} (\sigma_{semester="Spring" \wedge year=2018} (section))$

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2017	Painter	514	B
BIO-301	1	Summer	2018	Painter	514	A
CS-101	1	Fall	2017	Packard	101	H
CS-101	1	Spring	2018	Packard	101	F
CS-190	1	Spring	2017	Taylor	3128	E
CS-190	2	Spring	2017	Taylor	3128	A
CS-315	1	Spring	2018	Watson	120	D
CS-319	1	Spring	2018	Watson	100	B
CS-319	2	Spring	2018	Taylor	3128	C
CS-347	1	Fall	2017	Taylor	3128	A
EE-181	1	Spring	2017	Taylor	3128	C
FIN-201	1	Spring	2018	Packard	101	B
HIS-351	1	Spring	2018	Painter	514	C
MU-199	1	Spring	2018	Packard	101	D
PHY-101	1	Fall	2017	Watson	100	A

Output:

course_id
CS-101

Set Operations : Intersection Operation

Example : Graduates ∩ Managers

Graduates

Number	Surname	Age
7274	Robinson	37
7432	O'Malley	39
9824	Darkes	38

Managers

Number	Surname	Age
9297	O'Malley	56
7432	O'Malley	39
9824	Darkes	38

Output:

Graduates ∩ Managers

Number	Surname	Age
7432	O'Malley	39
9824	Darkes	38

Set Operations : SET DIFFERENCE

The set-difference operation allows us to find tuples that are in one relation but are not in another.

Notation: $r - s$

Set differences must be applied on **compatible** relations ie r and s must have the same **arity** and attribute domains of r and s must be **compatible**.

Example: To find all courses taught in the Fall 2017 semester, but not in the Spring 2018 semester.

Query: $\prod_{course_id} (\sigma_{semester="Fall" \wedge year=2017} (section)) - \prod_{course_id} (\sigma_{semester="Spring" \wedge year=2018} (section))$

Output:

course_id
CS-347
PHY-101

Set Operations : Intersection Operation

Example : Graduates - Managers

Graduates

Number	Surname	Age
7274	Robinson	37
7432	O'Malley	39
9824	Darkes	38

Managers

Number	Surname	Age
9297	O'Malley	56
7432	O'Malley	39
9824	Darkes	38

Output:

Graduates - Managers

Number	Surname	Age
7274	Robinson	37

Set Operations :The set operations UNION, INTERSECTION, and MINUS

Example : STUDENT ∪ INSTRUCTOR

STUDENT

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

INSTRUCTOR

Fname	Lname
John	Smith
Ricardo	Browne
Susan	Yao
Francis	Johnson
Ramesh	Shah

Output:

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert
John	Smith
Ricardo	Browne
Francis	Johnson

Set Operations :The set operations UNION, INTERSECTION, and MINUS

Example : STUDENT ∩ INSTRUCTOR.

STUDENT

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

INSTRUCTOR

Fname	Lname
John	Smith
Ricardo	Browne
Susan	Yao
Francis	Johnson
Ramesh	Shah

Output:

Fn	Ln
Susan	Yao
Ramesh	Shah

Set Operations :The set operations UNION, INTERSECTION, and MINUS

Example : STUDENT – INSTRUCTOR.

STUDENT

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

INSTRUCTOR

Fname	Lname
John	Smith
Ricardo	Browne
Susan	Yao
Francis	Johnson
Ramesh	Shah

Output:

Fn	Ln
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

Set Operations :The set operations UNION, INTERSECTION, and MINUS

Example : INSTRUCTOR – STUDENT.

STUDENT

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

INSTRUCTOR

Fname	Lname
John	Smith
Ricardo	Browne
Susan	Yao
Francis	Johnson
Ramesh	Shah

Output:

Fname	Lname
John	Smith
Ricardo	Browne
Francis	Johnson

Some properties of UNION, INTERSECT and DIFFERENCE

Notice that both union and intersection are **commutative** operations; that is:

- $R \cup S = S \cup R$ and $R \cap S = S \cap R$

Both union and intersection can be treated as **n-ary** operations applicable to any number of relations as both are **associative** operations; that is:

- $R \cup (S \cup T) = (R \cup S) \cup T$
- $(R \cap S) \cap T = R \cap (S \cap T)$

The minus operation is not commutative; that is, in general:

- $R - S \neq S - R$

Set Operations : Division Operation

- The Division operation is also a binary operation.

- R(Z) ÷ S(X), where X is a subset of Z.
- Y = Z - X that is let Y is the set of attributes of R that are not attributes of S in the resultant relation (T).

- The result of DIVISION is a relation T(Y) that includes a tuple t if tuples tR appear in R with tR [Y] = t, and with tR [X] = ts for every tuple ts in S.

- For a tuple t to appear in the result T of the DIVISION, the values in t must appear in R in combination with every tuple in S.

Set Operations : Division Operation

Example:

(a) Dividing SSN_PNOS by SMITH_PNOS. (b) T ← R ÷ S

(a)

SSN_PNOS

Essn	Pno
123456789	1
123456789	2
666884444	3
453453453	1
453453453	2
333445555	2
333445555	3
333445555	10
333445555	20
999887777	30
999887777	10
987987987	10
987987987	30
987654321	30
987654321	20
888665555	20

SMITH_PNOS

Pno
1
2

SSNS

Ssn
123456789
453453453

(b)

R

A	B
a1	b1
a2	b1
a3	b1
a4	b1
a1	b2
a3	b2
a2	b3
a3	b3
a4	b3
a1	b4
a2	b4
a3	b4

S

A
a1
a2
a3

T

B
b1
b4

Equivalent Queries

There is more than one way to write a query in relational algebra.

Example:

Find information about courses taught by instructors in the Physics department with salary greater than 90,000.

Query 1: $\sigma_{dept_name=\text{“Physics”} \wedge salary > 90{,}000}(instructor)$

Query 2 : $\sigma_{dept_name=\text{“Physics”}}(\sigma_{salary > 90.000}(instructor))$

The two queries are not identical; they are however equivalent -- they give the same result on the given database.

Binary Operator : The Cartesian-Product Operation

- In Relational Algebra, the Cartesian Product (×) is a binary operation that combines every tuple of one relation with every tuple of another relation.
- Allows to combine information from two relations.
- Denoted as **R x S**
- n - number of attributes in R and m - number of attributes in S
- Result is a relation Q with degree (arity) n + m attributes and Q(A1, A2, . . ., An, B1, B2, . . ., Bm) order of attributes.
- The resulting relation state has one tuple for each combination of tuples—one from R and one from S.
- Hence, if R has nR tuples (denoted as |R| = nR), and S has nS tuples, then R x S will have nR * nS tuples.
- The two operands do NOT have to be "union compatible"

Binary Operator : The Cartesian-Product Operation

PES UNIVERSITY

Sample Employee Table

Fname	Minit	Lname	Ssn	Bdate	Address	Gender	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren,Houston,TX	M	30000.00	888665555	5
Franklin	T	Wong	333445555	1955-12-08	638 voss,Houston,TX	M	40000.00	888665555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice,Houston,TX	F	25000.00	333445555	5
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000.00	333445555	5
James	E	Borg	888665555	1937-11-10	450 Stone, Houston,TX	M	55000.00	NULL	1
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire,Tx	F	43000.00	333445555	4
Ahmed	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston,TX	M	25000.00	987654321	4
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle,Spring,Tx	F	25000.00	333445555	4

Sample Dependent Table

Essn	Dependent_name	Gender	Bdate	Relationship
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse
123456789	Michael	M	1988-01-04	Son
333445555	Alice	F	1986-04-05	Daughter
333445555	Joy	F	1958-05-03	Spouse
333445555	Theodore	M	1983-10-25	Son
987654321	Abner	M	1942-02-28	Spouse

Database Management Systems

Binary Operator : The Cartesian-Product Operation

PES UNIVERSITY

Sample Cartesian Product : Employee X Dependent

Fname	Minit	Lname	Ssn	Bdate	Address	Gender	Salary	Super_ssn	Dno	Essn	Dependent_name	Gender	Bdate	Relati
John	B	Smith	123456789	1965-01-09	731 Fondren,Houston,TX	M	30000.00	888665555	5	987654321	Abner	M	1942-02-28	Spouse
John	B	Smith	123456789	1965-01-09	731 Fondren,Houston,TX	M	30000.00	888665555	5	333445555	Theodore	M	1983-10-25	Son
John	B	Smith	123456789	1965-01-09	731 Fondren,Houston,TX	M	30000.00	888665555	5	333445555	Joy	F	1958-05-03	Spouse
John	B	Smith	123456789	1965-01-09	731 Fondren,Houston,TX	M	30000.00	888665555	5	333445555	Alice	F	1986-04-05	Daught
John	B	Smith	123456789	1965-01-09	731 Fondren,Houston,TX	M	30000.00	888665555	5	123456789	Michael	M	1988-01-04	Son
John	B	Smith	123456789	1965-01-09	731 Fondren,Houston,TX	M	30000.00	888665555	5	123456789	Elizabeth	F	1967-05-05	Spouse
John	B	Smith	123456789	1965-01-09	731 Fondren,Houston,TX	M	30000.00	888665555	5	123456789	Alice	F	1988-12-30	Daught
Franklin	T	Wong	333445555	1955-12-08	638 voss,Houston,TX	M	40000.00	888665555	5	987654321	Abner	M	1942-02-28	Spouse
Franklin	T	Wong	333445555	1955-12-08	638 voss,Houston,TX	M	40000.00	888665555	5	333445555	Theodore	M	1983-10-25	Son
Franklin	T	Wong	333445555	1955-12-08	638 voss,Houston,TX	M	40000.00	888665555	5	333445555	Joy	F	1958-05-03	Spouse
Franklin	T	Wong	333445555	1955-12-08	638 voss,Houston,TX	M	40000.00	888665555	5	333445555	Alice	F	1986-04-05	Daught
Franklin	T	Wong	333445555	1955-12-08	638 voss,Houston,TX	M	40000.00	888665555	5	123456789	Michael	M	1988-01-04	Son
Franklin	T	Wong	333445555	1955-12-08	638 voss,Houston,TX	M	40000.00	888665555	5	123456789	Elizabeth	F	1967-05-05	Spouse
Franklin	T	Wong	333445555	1955-12-08	638 voss,Houston,TX	M	40000.00	888665555	5	123456789	Alice	F	1988-12-30	Daught
Joyce	A	English	453453453	1972-07-31	5631 Rice,Houston,TX	F	25000.00	333445555	5	987654321	Abner	M	1942-02-28	Spouse
Joyce	A	English	453453453	1972-07-31	5631 Rice,Houston,TX	F	25000.00	333445555	5	333445555	Theodore	M	1983-10-25	Son
Joyce	A	English	453453453	1972-07-31	5631 Rice,Houston,TX	F	25000.00	333445555	5	333445555	Joy	F	1958-05-03	Spouse
Joyce	A	English	453453453	1972-07-31	5631 Rice,Houston,TX	F	25000.00	333445555	5	333445555	Alice	F	1986-04-05	Daught
Joyce	A	English	453453453	1972-07-31	5631 Rice,Houston,TX	F	25000.00	333445555	5	123456789	Michael	M	1988-01-04	Son
Joyce	A	English	453453453	1972-07-31	5631 Rice,Houston,TX	F	25000.00	333445555	5	123456789	Elizabeth	F	1967-05-05	Spouse
Joyce	A	English	453453453	1972-07-31	5631 Rice,Houston,TX	F	25000.00	333445555	5	123456789	Alice	F	1988-12-30	Daught
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000.00	333445555	5	987654321	Abner	M	1942-02-28	Spouse
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000.00	333445555	5	333445555	Theodore	M	1983-10-25	Son

The resultant relation will have 56 rows, 15 columns.

Binary Operator : The Cartesian-Product Operation

Generally, CROSS PRODUCT is not a meaningful operation, can become meaningful when followed by other operations

Example (not meaningful):

EMP_DEPENDENTS ← EMPLOYEE x DEPENDENT

EMP_DEPENDENTS will contain every combination of EMPNAMES and DEPENDENT whether or not they are actually related

Database Management Systems

Binary Operator : The Cartesian-Product Operation

PES UNIVERSITY

•To keep only combinations where the DEPENDENT is related to the EMPLOYEE, we add a SELECT operation as follows:

Example (meaningful):

EMP_DEPENDENTS ← EMPLOYEE x DEPENDENT

ACTUAL_DEPS ← σ SSN=ESSN(EMP_DEPENDENTS)

RESULT (ACTUAL_DEPS) will now contains employees and their dependents information

Fname	Minit	Lname	Ssn	Bdate	Address	Gender	Salary	Super_ssn	Dno	Essn	Dependent_name	Gender	Bdate	Relation
John	B	Smith	123456789	1965-01-09	731 Fondren,Houston,TX	M	30000.00	888665555	5	123456789	Alice	F	1988-12-30	Daughter
John	B	Smith	123456789	1965-01-09	731 Fondren,Houston,TX	M	30000.00	888665555	5	123456789	Elizabeth	F	1967-05-05	Spouse
John	B	Smith	123456789	1965-01-09	731 Fondren,Houston,TX	M	30000.00	888665555	5	123456789	Michael	M	1988-01-04	Son
Franklin	T	Wong	333445555	1955-12-08	638 voss,Houston,TX	M	40000.00	888665555	5	333445555	Alice	F	1986-04-05	Daughter
Franklin	T	Wong	333445555	1955-12-08	638 voss,Houston,TX	M	40000.00	888665555	5	333445555	Joy	F	1958-05-03	Spouse
Franklin	T	Wong	333445555	1955-12-08	638 voss,Houston,TX	M	40000.00	888665555	5	333445555	Theodore	M	1983-10-25	Son
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire,Tx	F	43000.00	333445555	4	987654321	Abner	M	1942-02-28	Spouse

Binary Operator : The Cartesian-Product Operation

The issue with Cartesian Product is that it returns **all possible combinations of rows** from both tables.

This results in **redundant and meaningless combinations** unless you filter them using a condition.(as shown in previous slide: σ SSN=ESSN)

A **natural or join**, on the other hand, **performs a Cartesian Product internally and automatically filters rows by matching columns with the same name and compatible data types**, thus ensuring that there are no redundancies in the results.

Binary Operator : The Join Operation (Natural Join)

- The **join** operation allows us to combine a select operation and a Cartesian-Product operation into a single operation.
- Consider relations *r* (*R*) and *s* (*S*)
- Let "theta" be a predicate on attributes in the schema R "union" S. The join operation r $\bowtie_\theta$ s is defined as follows:

$$r \bowtie_\theta s = \sigma_\theta (r \times s)$$

- Thus

$$\sigma_{instructor.id = teaches.id} (instructor \times teaches)$$

- Can equivalently be written as

$$instructor \bowtie_{Instructor.id = teaches.id} teaches.$$

Database Management Systems

Binary Operator : The Join Operation (Natural Join)

Example: Write an relational algebraic query to retrieve the details of Managers(Fname, Name) and their managing department name

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

$$DEPT_MGR \leftarrow DEPARTMENT \bowtie_{Mgr_ssn=Ssn} EMPLOYEE$$

$$RESULT \leftarrow \pi_{Dname,\ Lname,\ Fname}(DEPT_MGR)$$

DEPT_MGR

Dname	Dnumber	Mgr_ssn	...	Fname	Minit	Lname	Ssn	...
Research	5	333445555	...	Franklin	T	Wong	333445555	...
Administration	4	987654321	...	Jennifer	S	Wallace	987654321	...
Headquarters	1	888665555	...	James	E	Borg	888665555	...

Database Management Systems

Binary Operator : The Join Operation

Q: Consider the employee database of Figure 2.17. Give an expression in the relational algebra to express each of the following queries:

a. Find the name of each employee who lives in city "Miami".

b. Find the name of each employee whose salary is greater than $100000.

c. Find the name of each employee who lives in "Miami" and whose salary is greater than $100000.

employee (*person_name*, *street*, *city*)
works (*person_name*, *company_name*, *salary*)
company (*company_name*, *city*)

Figure 2.17 Employee database.

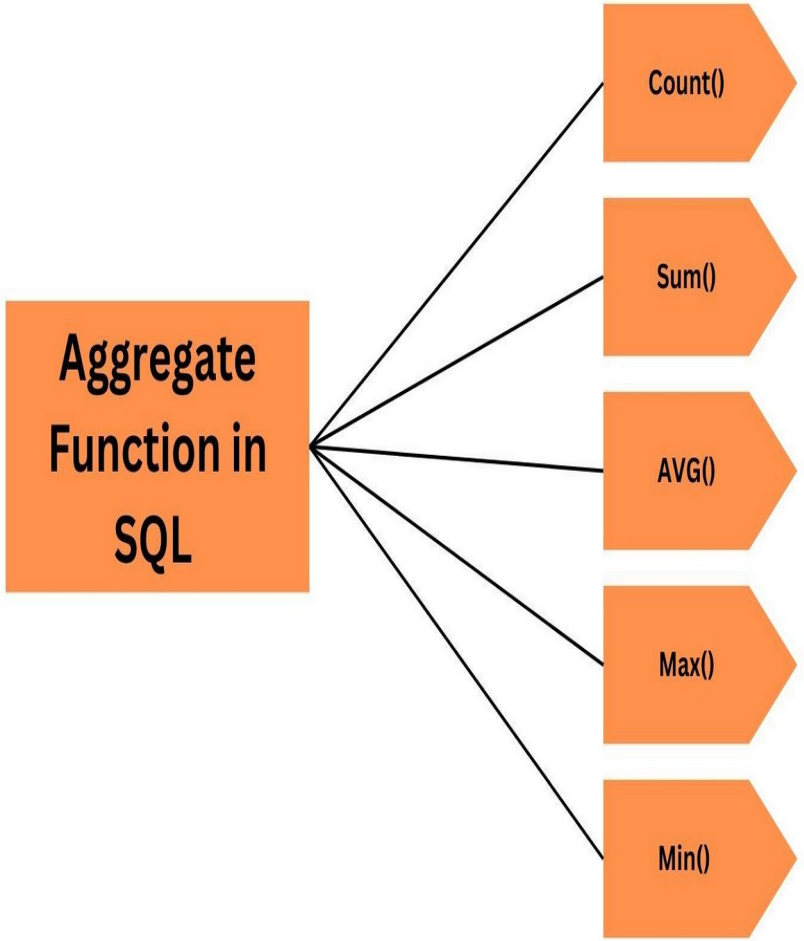
PES UNIVERSITY

Aggregate Functions:

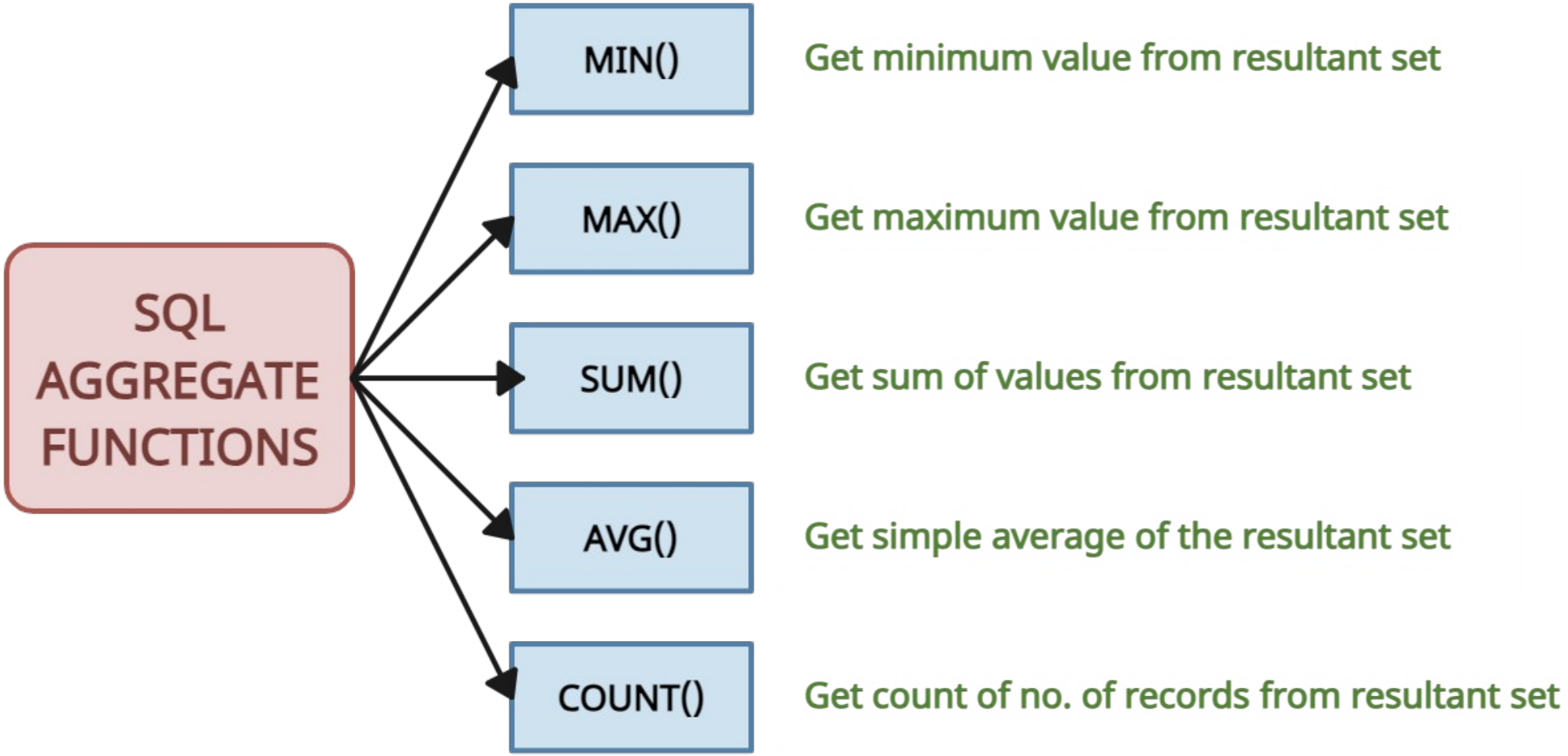
Aggregate functions are functions that take a collection (a set or multiset) of values as input and return a single value.

There are five aggregate functions they are:

1. AVERAGE
2. MINIMUM
3. MAXIMUM
4. SUM
5. COUNT



Types of aggregate functions



Aggregate Functions and Grouping

- A type of request that cannot be expressed in basic relational algebra is to specify mathematical **aggregate functions** on collections of values from the database.
- Examples of such functions include retrieving the average or total salary of all employees or the total number of employee tuples.
- These functions are used in simple statistical queries that summarize information from the database tuples.
- Common functions applied to collections of numeric values include SUM, AVERAGE, MAXIMUM, and MINIMUM.
- The COUNT function is used for counting tuples or values.

Note: count just counts the number of rows, without removing duplicates

PES UNIVERSITY

Use of the Aggregate Functional operation $\mathcal{F}$

- $\mathcal{F}_{\text{MAX Salary}}$ (EMPLOYEE) retrieves the maximum salary value from the EMPLOYEE relation.
- $\mathcal{F}_{\text{MIN Salary}}$ (EMPLOYEE) retrieves the minimum salary value from the EMPLOYEE relation.
- $\mathcal{F}_{\text{SUM Salary}}$ (EMPLOYEE) retrieves the sum of salary values from the EMPLOYEE relation.
- $\mathcal{F}_{\text{COUNT SSN, AVERAGE Salary}}$ (EMPLOYEE) computes the count (number) of employees & their average salary.

Aggregate Functions and Grouping

Grouping can be combined with Aggregate Functions

Example: For each department, retrieve the DNO, COUNT SSN, and AVERAGE SALARY

•A variation of aggregate operation $\mathscr{F}$ allows this:

- Grouping attribute placed to left of symbol
- Aggregate functions to right of symbol
- $_{\text{DNO}}\mathscr{F}_{\text{COUNT SSN, AVERAGE Salary}}$ (EMPLOYEE)

•Above operation groups employees by DNO (department number) and computes the count of employees and average salary per department

Count Function

Count:

Count function is used to count the number of tuples in a relation.

It can work on both numeric and non-numeric data types.

Ex:

A	B	C
α	α	7
α	β	7
β	β	3
β	β	10

Output: $g_{count(*)}(E)=4$

4 is the number of tuples in the above table

Sum Function

Sum-

- Sum function is used to calculate the sum of all selected columns.
- It works on numeric fields only.

Ex:

Relation *r*:

A	B	C
α	α	7
α	β	7
β	β	3
β	β	10

$g_{sum(c)}(r)$

sum-C
27

Average Function

Average-

- The function is used to calculate the average value of the numeric type.
- average function returns the average of all non-Null values.

Ex:

A	B	C
α	α	7
α	β	7
β	β	3
β	β	10

Output: $g_{average(C)}(R)=27/4=6.75$

Minimum Function and Maximum Function

Minimum-

- MIN function is used to find the minimum value of a certain column.
- This function determines the smallest value of all selected values of a column.

Ex-

$g_{min(IC)}(R)=3$

A	B	C
α	α	7
α	β	7
β	β	3
β	β	10

Maximum-

- MAX function is used to find the maximum value of a certain column.
- This function determines the largest value of all selected values of a column.

Ex-

$g_{max(C)}(R)=10$

PES UNIVERSITY

Consider the table:

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Example:

1) Find the sum of the salaries of all employees, the maximum salary, the minimum salary, and the average salary.

Query:

$$\rho_{R(\text{Total_Sal,Highest_Sal, Lowest_sal,Average_Sal})} \Im_{(\text{SUM(Salary), MAX(Salary), MIN(Salary), AVG(Salary))}} (\text{EMPLOYEE})$$

Output:

Total_Sal	Highest_Sal	Lowest_Sal	Average_Sal
345930.00	55000.00	25000.00	38436.666667

Grouping

- In many cases aggregate functions can be applied to subgroups of data.

  Example: Calculate average salary for each department.
- In these cases **data is divided into non-overlapping groups based on specified attributes.**
- **Each group (partition) will consist of the tuples that have the same value of some attribute(s), called the grouping attribute(s).**
- We can then apply the function to **each such group independently to produce summary information** about each group. SQL has a GROUP BY clause for this purpose.
- The **GROUP BY clause specifies the grouping attributes, which should also appear in the SELECT clause,** so that the value resulting from applying each aggregate function to a group of tuples appears along with the value of the grouping attribute(s).

Aggregate Functions and Grouping

Example: a.find the average salary of employees in each department

b. Find the number of employees who work on each project.

Query:

$$\rho_{R(\text{Dno},\text{count}(*),\ \text{Avg}(\text{Salary}))}\left(\ {}_{\text{Dno}}\mathfrak{S}_{(\text{Dno},\text{COUNT}(*),\ \text{AVG}(\text{Salary}))}\ (\text{EMPLOYEE})\right.$$

Output:

Fname	Minit	Lname	Ssn	...	Salary	Super_ssn	Dno
John	B	Smith	123456789		30000	333445555	5
Franklin	T	Wong	333445555		40000	888665555	5
Ramesh	K	Narayan	666884444		38000	333445555	5
Joyce	A	English	453453453	...	25000	333445555	5
Alicia	J	Zelaya	999887777		25000	987654321	4
Jennifer	S	Wallace	987654321		43000	888665555	4
Ahmad	V	Jabbar	987987987		25000	987654321	4
James	E	Bong	888665555		55000	NULL	1

Grouping EMPLOYEE tuples by the value of Dno

Dno	Count (*)	Avg (Salary)
5	4	33250
4	3	31000
1	1	55000

Result of Q24

PES UNIVERSITY

Exercise:

Write Relational Algebra Statements to:

a) Retrieve each department number, the number of employees in the department, and their average salary, while renaming the resulting attributes.
b) Show the department number, the total number of employees, and the average salary of employees in that department, for each department.
c) FInd the total number of employees and the average salary across the entire company.

Answers:

a. $\rho_{R(\text{Dno, No_of_employees, Average_sal})}(_{\text{Dno}}\ \Im\ _{\text{COUNT Ssn, AVERAGE Salary}}\ (\text{EMPLOYEE}))$.

b. $_{\text{Dno}}\ \Im\ _{\text{Count ssn,Average_salary}}(\text{EMPLOYEE})$.

c. $\Im\ _{\text{COUNT ssn, AVERAGE Salary}}(\text{EMPLOYEE})$.

Output:

R

(a)

Dno	No_of_employees	Average_sal
5	4	33250
4	3	31000
1	1	55000

(b)

Dno	Count_ssn	Average_salary
5	4	33250
4	3	31000
1	1	55000

(c)

Count_ssn	Average_salary
8	35125

Fill in the Blanks:

1) The union operation allows us to combine two relations (r and s). The notation used for union is _____.

Ans: r ∪ s

2) For the union operation r ∪ s to be valid, r and s must have the same _____.

Ans: arity (same number of attributes)

3) In the union operation, duplicate tuples are _____.

Ans: eliminated

4) The set-intersection operation allows us to find tuples that are in both input relations. The notation used for intersection is _____.

Ans: r ∩ s

Fill in the Blanks:

5) The set-difference operation allows us to find tuples that are in one relation but not in another. The notation used for set-difference is _____.

Ans: r−s

6) The number of attributes in a relation is referred to as its _____.

Ans: arity

7) The two queries that give the same result on any database but are written differently in relational algebra are called _____queries.

Ans: equivalent

8) In the UNION operation, the two operand relations must be "type _____".

Ans: compatible

PES UNIVERSITY

Can you write the intersection operation using union and set difference operations ?

INTERSECTION can be expressed in terms of union and set difference as follows:

$$R \cap S = ((R \cup S) - (R - S)) - (S - R)$$

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Relational Algebra - Hands on

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgments: Dr. Nagasundari S (Dept of CSE), Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 1: Introduction to Database Management and SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G.

Department of Computer Science and Engineering

Example Queries in Relational Algebra: Procedural Form

- **Q1: Retrieve the name and address of all employees who work for the 'Research' department.**

- **Q2: Retrieve the names of employees who have no dependents.**

Example Queries in Relational Algebra: Procedural Form

- **Q1: Retrieve the name and address of all employees who work for the 'Research' department.**

  $RESEARCH_DEPT \leftarrow \sigma_{DNAME='Research'}(DEPARTMENT)$

  $RESEARCH_EMPS \leftarrow (RESEARCH_DEPT \bowtie_{DNUMBER=DNOEMPLOYEE} EMPLOYEE)$

  $RESULT \leftarrow \pi_{FNAME, LNAME, ADDRESS}(RESEARCH_EMPS)$

- **Q2: Retrieve the names of employees who have no dependents.**

  $ALL_EMPS \leftarrow \pi_{SSN}(EMPLOYEE)$

  $EMPS_WITH_DEPS(SSN) \leftarrow \pi_{ESSN}(DEPENDENT)$

  $EMPS_WITHOUT_DEPS \leftarrow (ALL_EMPS - EMPS_WITH_DEPS)$

  $RESULT \leftarrow \pi_{LNAME, FNAME}(EMPS_WITHOUT_DEPS * EMPLOYEE)$

Example Queries in Relational Algebra: Single Expression

As a single expression, these queries become:

- **Q1: Retrieve the name and address of all employees who work for the 'Research' department.**

  $\pi_{\text{Fname, Lname, Address}} (\sigma_{\text{Dname= 'Research'}} (\text{DEPARTMENT} \bowtie_{\text{Dnumber=Dno}} (\text{EMPLOYEE}))$

- **Q2: Retrieve the names of employees who have no dependents.**

  $\pi_{\text{Lname, Fname}}((\pi_{\text{Ssn}} (\text{EMPLOYEE}) - \rho_{\text{Ssn}} (\pi_{\text{Essn}} (\text{DEPENDENT}))) \bowtie \text{EMPLOYEE})$

Example Queries in Relational Algebra

Consider a database with the following schema:

Sailors(*sid:* integer, *sname:* string, *rating:* integer, *age:* real)
Boats(*bid:* integer, *bname:* string, *color:* string)
Reserves(*sid:* integer, *bid:* integer, *day:* date)

Write relational algebra expressions for the following queries

Example Queries in Relational Algebra

sid	*sname*	*rating*	*age*
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Figure 4.15 An Instance $S3$ of Sailors

sid	*bid*	*day*
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

Figure 4.16 An Instance $R2$ of Reserves

bid	*bname*	*color*
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

Figure 4.17 An Instance $B1$ of Boats

Example Queries in Relational Algebra

1: Retrieve rows corresponding to expert (rating more than 8) sailors

2: Retrieve the rating of each sailor

3: Retrieve the age of sailors

4: Retrieve name and ratings of highly rated sailor

Example Queries in Relational Algebra

Find the names of sailors who have reserved boat 103.

$$\pi_{sname}((\sigma_{bid=103} Reserves) \bowtie Sailors)$$

Example Queries in Relational Algebra

Find the names of sailors who have reserved a red boat.

$$\pi_{sname}((\sigma_{color='red'} Boats) \bowtie Reserves \bowtie Sailors)$$

Find the colors of boats reserved by Lubber.

$$\pi_{color}((\sigma_{sname='Lubber'} Sailors) \bowtie Reserves \bowtie Boats)$$

Example Queries in Relational Algebra

Find the names of sailors who have reserved at least one boat.

$$\pi_{sname}(Sailors \bowtie Reserves)$$

Find the names of sailors who have reserved a red or a green boat.

$$\rho(Tempboats, (\sigma_{color='red'} Boats) \cup (\sigma_{color='green'} Boats))$$
$$\pi_{sname}(Tempboats \bowtie Reserves \bowtie Sailors)$$

$$\rho(Tempboats, (\sigma_{color='red' \vee color='green'} Boats))$$
$$\pi_{sname}(Tempboats \bowtie Reserves \bowtie Sailors)$$

Example Queries in Relational Algebra

(Q6) Find the names of sailors who have reserved a red and a green boat.

$$\rho(Tempboats2, (\sigma_{color='red'} Boats) \cap (\sigma_{color='green'} Boats))$$
$$\pi_{sname}(Tempboats2 \bowtie Reserves \bowtie Sailors)$$

INCORRECT

$$\rho(Tempred, \pi_{sid}((\sigma_{color='red'} Boats) \bowtie Reserves))$$
$$\rho(Tempgreen, \pi_{sid}((\sigma_{color='green'} Boats) \bowtie Reserves))$$
$$\pi_{sname}((Tempred \cap Tempgreen) \bowtie Sailors)$$

Example Queries in Relational Algebra

Find the sids of sailors with age over 20 who have not reserved a red boat.

$$\pi_{sid}(\sigma_{age>20} Sailors) -$$
$$\pi_{sid}((\sigma_{color='red'} Boats) \bowtie Reserves \bowtie Sailors)$$

Example Queries in Relational Algebra

(Q9) Find the names of sailors who have reserved all boats.

$$\rho(Tempsids, (\pi_{sid,bid}Reserves)/(\pi_{bid}Boats))$$
$$\pi_{sname}(Tempsids \bowtie Sailors)$$

Find the names of sailors who have reserved all boats called Interlake.

$$\rho(Tempsids, (\pi_{sid,bid}Reserves)/(\pi_{bid}(\sigma_{bname='Interlake'}Boats)))$$
$$\pi_{sname}(Tempsids \bowtie Sailors)$$

Example Queries in Relational Algebra for practice

Consider a database with the following schema:

Person (name, age, gender)	name is a key
Frequents (name, pizzeria)	(name, pizzeria) is a key
Eats (name, pizza)	(name, pizza) is a key
Serves (pizzeria, pizza, price)	(pizzeria, pizza) is a key

Write relational algebra expressions for the following queries

Example Queries in Relational Algebra

1. Find all pizzerias frequented by at least one person under the age of 18.
2. Find the names of all females who eat either mushroom or pepperoni pizza (or both).
3. Find the names of all females who eat both mushroom and pepperoni pizza.
4. Find all pizzerias that serve at least one pizza that Amy eats for less than $10.00.
5. Find all pizzerias that are frequented by only females or only males.
6. For each person, find all pizzas the person eats that are not served by any pizzeria the person frequents. Return all such person (name) / pizza pairs.
7. Find the names of all people who frequent only pizzerias serving at least one pizza they eat.
8. Find the names of all people who frequent every pizzeria serving at least one pizza they eat.
9. Find the pizzeria serving the cheapest pepperoni pizza. In the case of ties, return all of the cheapest-pepperoni pizzerias.

Example Queries in Relational Algebra

In general there are many correct expressions for each query.
Sample solutions

a. $\pi_{\text{pizzeria}}\left(\sigma_{\text{age}<18}(\text{Person}) \bowtie \text{Frequents}\right)$

b. $\pi_{\text{name}}\left(\sigma_{\text{gender}=\text{'female'} \wedge (\text{pizza}=\text{'mushroom'} \vee \text{pizza}=\text{'pepperoni'})}(\text{Person} \bowtie \text{Eats})\right)$

c. $\pi_{\text{name}}\left(\sigma_{\text{gender}=\text{'female'} \wedge \text{pizza}=\text{'mushroom'}}(\text{Person} \bowtie \text{Eats})\right) \cap$
$\pi_{\text{name}}\left(\sigma_{\text{gender}=\text{'female'} \wedge \text{pizza}=\text{'pepperoni'}}(\text{Person} \bowtie \text{Eats})\right)$

Example Queries in Relational Algebra

d. $\pi_{\text{pizzeria}}\left(\sigma_{\text{name}='\text{Amy}'}(\text{Eats}) \bowtie \sigma_{\text{price}<10}(\text{Serves})\right)$

e. $\left(\begin{array}{l}\pi_{\text{pizzeria}}(\sigma_{\text{gender}='\text{female}'}(\text{Person}) \bowtie \text{Frequents}) - \\ \pi_{\text{pizzeria}}(\sigma_{\text{gender}='\text{male}'}(\text{Person}) \bowtie \text{Frequents})\end{array}\right) \cup$
$\left(\begin{array}{l}\pi_{\text{pizzeria}}(\sigma_{\text{gender}='\text{male}'}(\text{Person}) \bowtie \text{Frequents}) - \\ \pi_{\text{pizzeria}}(\sigma_{\text{gender}='\text{female}'}(\text{Person}) \bowtie \text{Frequents})\end{array}\right)$

Example Queries in Relational Algebra

f. $\text{Eats} - \pi_{\text{name, pizza}}(\text{Frequents} \bowtie \text{Serves})$

g. $\pi_{\text{name}}(\text{Person}) - \pi_{\text{name}}(\text{Frequents} - \pi_{\text{name, pizzeria}}(\text{Eats} \bowtie \text{Serves}))$

h. $\pi_{\text{name}}(\text{Person}) - \pi_{\text{name}}(\pi_{\text{name, pizzeria}}(\text{Eats} \bowtie \text{Serves}) - \text{Frequents})$

i. $\pi_{\text{pizzeria}}(\sigma_{\text{pizza}='\text{pepperoni}'}\text{Serves}) -$

$$\pi_{\text{pizzeria}}\left(\sigma_{\text{price}>\text{price2}}\left(\begin{array}{l}\pi_{\text{pizzeria, price}}(\sigma_{\text{pizza}='\text{pepperoni}'}\text{Serves}) \\ \quad \times \\ \rho_{\text{pizzeria2, price2}}(\pi_{\text{pizzeria, price}}(\sigma_{\text{pizza}='\text{pepperoni}'}\text{Serves}))\end{array}\right)\right)$$

Summary

PES UNIVERSITY
CELEBRATING 50 YEARS

Relational Algebra

- Unary Relational Operations
- Relational Algebra Operations From Set Theory
- Binary Relational Operations
- Additional Relational Operations
- Examples of Queries in Relational Algebra

Q & A

1. **The relational algebra expression for finding the names of instructors in the Physics department is:**

a) Instructor - Physics

b) σ Name (∏ Dept_Name = "Physics" (Instructor))

c) ∏ Name (Instructor ⋈ Physics)

d) ∏ Name (σ Dept_Name = "Physics" (Instructor))

2. **The expression ρ City -> Location (Customers) is used to:**

a) Rename the attribute "City" to "Location" in the "Customers" relation

b) Select only the "City" column from "Customers"

c) Find the intersection of "City" and "Location" columns

d) Perform a join operation on "Customers"

Q & A

3. The result of the operation ∏ Name, Department (σ JoiningYear < 2020 (Employees)) will return:

a) EmployeeIDs of employees who joined before 2020

b) Names and departments of employees who joined before 2020

c) Employee salaries who joined before 2020

d) All attributes of employees who joined before 2020

4. The SIDs of students who are not enrolled in any course is given by:

A. ∏ SID(Student) - ∏ SID(Enrolled)

B. σ SID NOT IN (∏ SID(Enrolled)) (Student)

C. ∏ SID(Student ⋈ Enrolled)

D. ∏ SID(Enrolled) - ∏ SID(Student)

Q & A

PES UNIVERSITY
CELEBRATING 50 YEARS

Answers

1. D
2. A
3. B
4. A

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

SQL overview, Data definition, Structure of SQL queries

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgments: Dr. Nagasundari S (Dept of CSE), Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 1: Introduction to Database Management and SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G.

Department of Computer Science and Engineering

Introduction to SQL Commands

PES UNIVERSITY

SQL language

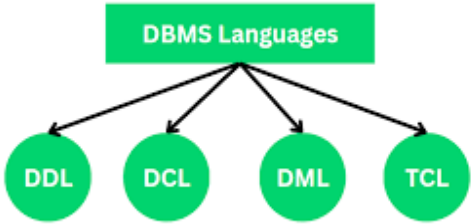
- Considered as one of the major reasons for the commercial success of relational databases.

SQL

- SQL actually comes from the word “SEQUEL” which was the original term used in the paper: “SEQUEL TO SQUARE” by Chamberlin and Boyce. IBM could not copyright that term, so they abbreviated to SQL and copyrighted the term SQL.
- Now popularly known as “Structured Query language”.
- The language has features for Data definition, Data Manipulation, Transaction control , Indexing ,Security specification (Grant and Revoke), Active databases, Multi-media , Distributed databases etc.

Types of database languages

- **Data Definition Language (DDL)** statements are used to define the database structure or schema.
- **Data Manipulation Language (DML)** statements are used for managing data within schema objects
- **Data Control Language (DCL)** statements are used to control access to the database.
- **Transaction Control Language (TCL)** is used to run the changes made by the DML statement.



Schema Concepts in SQL

PES UNIVERSITY

SQL schema

- Identified by a **schema name**
- Includes an **authorization identifier** and **descriptors** for each element
- Each statement in SQL ends with a **semicolon**
- Schema elements include tables, constraints, views, domains and other constraints

Schema Concepts in SQL

PES UNIVERSITY

CREATE SCHEMA statement:

MySQL Syntax:

CREATE SCHEMA Library;

This is same as **CREATE DATABASE Library;** ie schema and database are same in MySql

Equivalent PostgreSQL/Oracle syntax: (for reference)

CREATE SCHEMA Library AUTHORIZATION ‘Jsmith’**;**

Attribute Data Types in SQL

PES UNIVERSITY

Basic data types:

Numeric data types

- Integer numbers: INTEGER, INT, and SMALLINT
- Floating-point (real) numbers: FLOAT or REAL, and DOUBLE PRECISION

Character-string data types

- Fixed length: CHAR(*n*), CHARACTER(*n*)
- Varying length: VARCHAR(*n*), CHAR VARYING(*n*), CHARACTER VARYING(*n*)

Attribute Data Types in SQL

PES UNIVERSITY

Bit-string data types

- Fixed length: BIT(*n*)
- Varying length: BIT VARYING(*n*)

Boolean data type

- Values of TRUE or FALSE or NULL

DATE data type

- Ten positions
- Components are YEAR, MONTH, and DAY in the form YYYY-MM-DD
- Multiple mapping functions available in RDBMSs to change date formats

Note: Learn more about MYSQL inbuilt Date Functions here

Attribute Data Types in SQL

PES UNIVERSITY

Timestamp data type

- Includes the DATE and TIME fields
- Plus a minimum of six positions for decimal fractions of seconds
- Optional WITH TIME ZONE qualifier

INTERVAL data type

- Specifies a relative value that can be used to increment or decrement an absolute value of a date, time, or timestamp

DATE, TIME, Timestamp, INTERVAL data types can be cast or converted to string formats for comparison.

Attribute Data Types in SQL Explained

PES UNIVERSITY

The SQL standard supports a variety of built-in types, including:

• **char(n):** A fixed-length character string with user-specified length n. The full form, character, can be used instead.

• **varchar(n)**: A variable-length character string with user-specified maximum length n. The full form, character varying is equivalent.

• **int:** An integer (a finite subset of the integers that is machine dependent). The full form, integer, is equivalent.

• **smallint**: A small integer (a machine-dependent subset of the integer type).

Attribute Data Types in SQL Explained

- **Numeric(p, d):** A fixed-point number with user-specified precision. The number consists of p digits (plus a sign), and d of the p digits are to the right of the decimal point. Thus, numeric(3,1) allows 44.5 to be stored exactly, but neither 444.5 nor 0.32 can be stored exactly in a field of this type.
- **Real, double precision:** Floating-point and double-precision floating-point numbers with machine-dependent precision.
- **Float(n):** A floating-point number with a precision of at least n digits.
- **Null:** Each type may include a special value called the null value. A null value indicates an absent value that may exist but be unknown or that may not exist at all.
- **ENUM:** Data type in MySQL is a **string object**. It allows us to limit the value chosen from a list of permitted values in the column specification at the time of table creation. It is short for **enumeration**, which means that each column may have one of the specified possible values. ENUM ('value_1' 'value_2' 'value_3')

Database Management Systems

Char vs Varchar

PES UNIVERSITY

Char	*Varchar*
Majorly use Char when you know that the strings are always going to be of the same size. Example : Pincode	Use Varchar when you have little idea about the size of the string that is going to be entered. Example :Name
Since it is a datatype of fixed length the storage taken for a Char will be equal to the length that it is initialised with.	Since it is a datatype of variable length the storage taken for a Char will be equal to the size of the string/data entered and not the entire length that is initialised with.
Char utilises the entire storage by the use of training spaces.	Varchar does not utilize the entire storage but only uses the storage based on the data input
Char gives better performance compared to Varchar	Varchar gives lower performance compared to Char
Char is incapable of holding a null value hence sql will automatically assign a Varchar for a null value	A Char null column is nothing but in reality a Varchar null column

Char vs Varchar

The char data type stores fixed-length strings.

Consider, for example, an attribute A of type char(10).

- If we stored a string “Avi” in this attribute, seven spaces are appended to the string to make it 10 characters long.
- In contrast, if attribute B were of type varchar(10), and we stored “Avi” in attribute B, no spaces would be added.

Advanced Data Types like CLOB, BLOB

Large-Object Types

- Many current-generation database applications need to store attributes that can be large (of the order of many kilobytes)
- These data types include as a photograph, or very large (of the order of many megabytes or even gigabytes), such as a high-resolution medical image or video clips.
- SQL therefore provides large-object data types for character data (**clob**) and binary data (**blob**). The letters “lob” in these data types stand for “Large OBject.”

Advanced Data Types like CLOB, BLOB

PES UNIVERSITY

Large-Object Types

For example, we may declare attributes in oracle as:

book review **clob**(10KB)

image **blob**(10MB)

movie **blob**(2GB)

- For result tuples containing large objects (multiple megabytes to gigabytes), it is inefficient or impractical to retrieve an entire large object into memory.
- LOBs in the database are stored in a way that optimizes the space and provides efficient access within the database tablespaces.
- Internal LOBs (BLOBs, CLOBs) also provide transactional support (Commit, Rollback, and so on) of the database server.

Advanced Data Types like CLOB, BLOB

PES UNIVERSITY

- **BLOBs (Binary LOBs)** used to store unstructured binary (also called “raw”) data, such as video clips.
- **CLOBs (Character LOBs)** used to store large blocks of character data from the database character set.
- Blob and Clob together are known as LOB(Large Object Type). The following are the major differences between Blob and Clob data types.

Binary Large Object (BLOB)	Stores any kind of data in binary format such as images, audio, and video.
Character Large Object (CLOB)	Stores string data in the database having character set format. Used for large set of characters/strings or documents that use the database character.

Advanced Data Types like CLOB, BLOB

Blob	Clob
• The full form of Blob is a Binary Large Object.	• The full form of Clob is Character Large Object.
• This is used to store large binary data.	• This is used to store large textual data.
• This stores values in the form of binary streams.	• This stores values in the form of character streams.
• Using this you can stores files like videos, images, gifs, and audio files. • MySQL supports this with the following datatypes:TINYBLOB • BLOB • MEDIUMBLOB • LONGBLOB	• Using this you can store files like text files, PDF documents, word documents etc. • MySQL supports this with the following datatypes:TINYTEXT • TEXT • MEDIUMTEXT • LONGTEXT

Single Example covers all data types

```
CREATE TABLE example (
    id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    char_column CHAR(50) NOT NULL,
    varchar_column VARCHAR(255) NOT NULL,
    tinytext_column TINYTEXT NOT NULL,
    text_column TEXT NOT NULL,
    mediumtext_column MEDIUMTEXT NOT NULL,
    longtext_column LONGTEXT NOT NULL,
    binary_column BINARY(20) NOT NULL,
    varbinary_column VARBINARY(20) NOT NULL,
    tinyblob_column TINYBLOB NOT NULL,
    blob_column BLOB NOT NULL,
    mediumblob_column MEDIUMBLOB NOT NULL,
```

Single Example covers all data types

```
    longblob_column LONGBLOB NOT NULL,
    enum_column ENUM('value1', 'value2', 'value3') NOT NULL,
    set_column SET('value1', 'value2', 'value3') NOT NULL,
    bool_column BOOLEAN NOT NULL DEFAULT 0,
    date_column DATE NOT NULL,
    datetime_column DATETIME NOT NULL,
    timestamp_column TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON
    UPDATE CURRENT_TIMESTAMP,
    time_column TIME NOT NULL,
    year_column YEAR(4) NOT NULL,
    float_column FLOAT(10,6) NOT NULL DEFAULT '0.000000',
    double_column DOUBLE(10,6) NOT NULL DEFAULT '0.000000'
);
```

SQL Data Definition, Data Types, Standards

PES UNIVERSITY

Terminology:

- **Table**, **row**, and **column** used for relational model terms relation, tuple, and attribute respectively.

CREATE statement

- Main SQL command for data definition
- The CREATE TABLE command is used to create a new relation by giving it a name and specifying its attributes and initial constraints.

Data Definition Language(DDL)

PES UNIVERSITY

- The SQL DDL provides commands for defining table schemas, deleting tables, and modifying table schemas.
- DDL includes commands such as CREATE, ALTER, and DROP.
- DDL is used to CREATE, ALTER, OR DROP the database objects (Table, Views, Users).
- CREATE - to create objects in the database
- ALTER - alters the structure of the database
- DROP - delete objects from the database
- TRUNCATE - remove all records from a table, including all spaces allocated for the records are removed
- COMMENT - add comments to the data dictionary
- RENAME - rename an object

Types of Database Languages: DDL

CREATE TABLE

Syntax:

CREATE TABLE table_name(
Col_name1 datatype(),
Col_name2 datatype(),…
Col_namen datatype(),
);

```
CREATE TABLE `dept_locations` (
  `Dnumber` int(11) NOT NULL,
  `Dlocation` varchar(15) NOT NULL
)
```

Create table from existing table:

CREATE TABLE new_table_name AS SELECT *
FROM existing_table;

CREATE TABLE new_table_name LIKE existing_table;

```
CREATE TABLE `project` (
  `Pname` varchar(15) NOT NULL,
  `Pnumber` int(11) NOT NULL,
  `Plocation` varchar(15) DEFAULT NULL,
  `Dnum` int(11) NOT NULL
)
```

Types of Database Languages: DDL

Difference between CREATE TABLE new_table_name **AS SELECT** * FROM existing_table; and CREATE TABLE new_table_name **LIKE** existing_table;

Feature	as select	like
Copies structure	Yes	Yes
Copies data	Yes	No
Copies constraints/indexes	No	Yes
Copies AUTO_INCREMENT	No	Yes
Flexibility in copying data	Yes can use where and join	No

Types of Database Languages: DDL

PES UNIVERSITY

ALTER TABLE

1) ADD

Syntax:

ALTER TABLE table_name

ADD Col_name datatype()...;

2) MODIFY

Syntax:

ALTER TABLE table_name

MODIFY (fieldname datatype()...);

Learn more examples

3) RENAME

Syntax:

ALTER TABLE table_name

RENAME COLUMN (Old_fieldname TO

New_fieldname...);

4) DROP

Syntax:

ALTER TABLE table_name **DROP**

column_name;

Types of Database Languages: DDL

C) DESCRIBE TABLE

Syntax:

DESCRIBE table_name;

//lists basic description.

SHOW CREATE TABLE table_name;

// lists complete information of table including constraints,index etc..

D) DROP TABLE

Syntax: DROP TABLE table_name;

// Complete table structure will be dropped

E) RENAME

Syntax:

RENAME TABLE table_name to new table_name

F) TRUNCATE

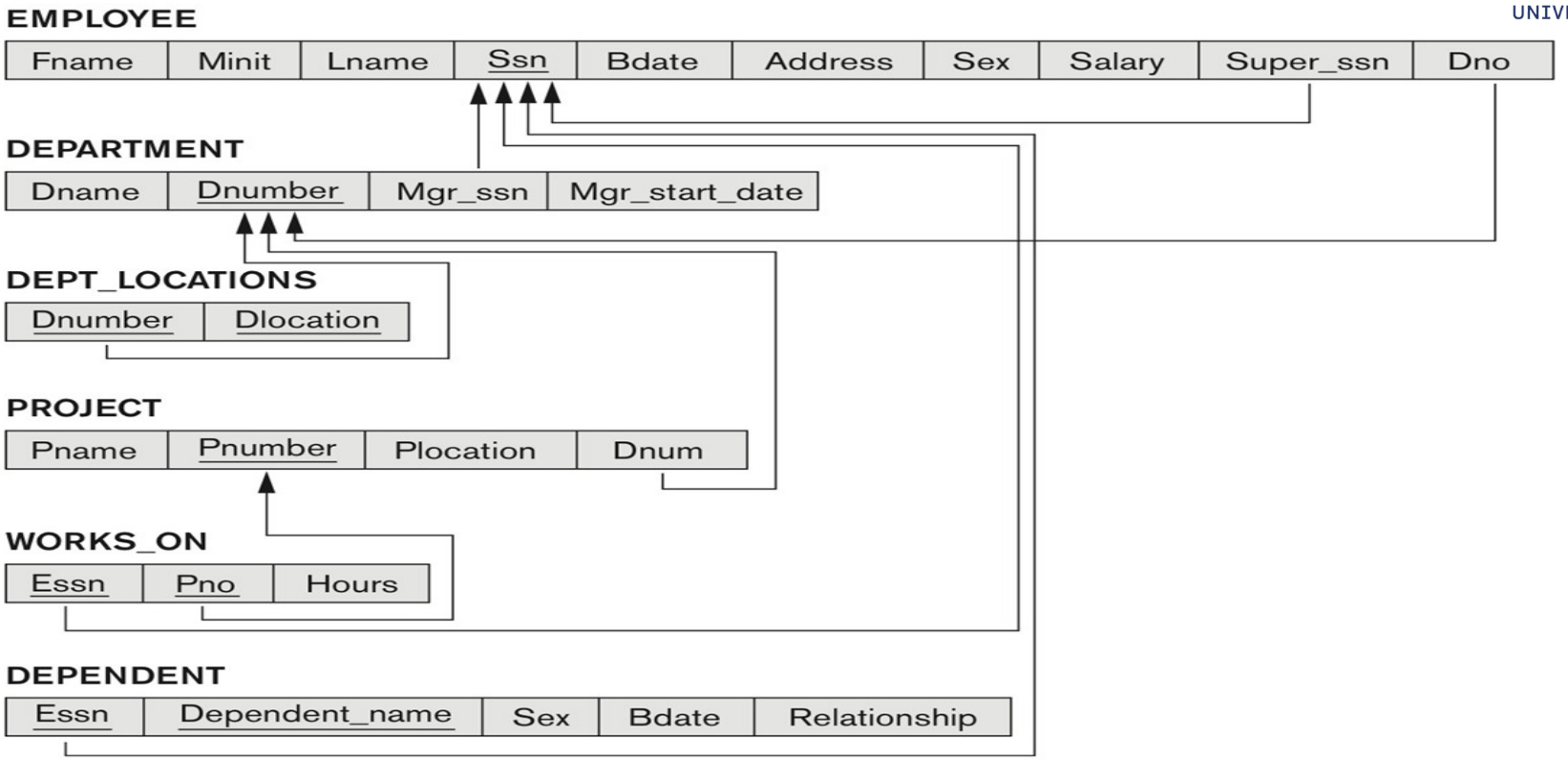
Syntax:

TRUNCATE TABLE table_name; // delete complete data from an existing table. Table Structure remains

Types of Database Languages: DDL

Feature	DROP	TRUNCATE
Removes data	Yes	Yes
Removes table structure	Yes (table is deleted)	No (structure remains)
Auto_Increment reset	N/A (table removed)	Resets to 1 (in MySQL)
Constraints & Indexes	Deleted along with table	Preserved (only rows deleted)
Rollback (MySQL)	Cannot be rolled back	Cannot be rolled back (rollback dealt in further slides)
Use case	When table is no longer needed	When you want to empty the table but keep its structure

COMPANY relational database schema



SQL CREATE TABLE data definition statements for defining the COMPANY schema

PES UNIVERSITY

- CREATE TABLE `department` (`Dname` varchar(15) NOT NULL, `Dnumber` int(11) NOT NULL, `Mgr_ssn` char(9) NOT NULL, `Mgr_start_date` date DEFAULT NULL);

- CREATE TABLE `dependent` (`Essn` char(9) NOT NULL, `Dependent_name` varchar(15) NOT NULL, `Gender` char(1) DEFAULT NULL, `Bdate` date DEFAULT NULL, `Relationship` varchar(8) DEFAULT NULL);

- CREATE TABLE `dept_locations` (`Dnumber` int(11) NOT NULL, `Dlocation` varchar(15) NOT NULL);

SQL CREATE TABLE data definition statements for defining the COMPANY schema

PES UNIVERSITY

- CREATE TABLE `employee` (`Fname` varchar(15) NOT NULL, `Minit` char(1) DEFAULT NULL, `Lname` varchar(15) NOT NULL, `Ssn` char(9) NOT NULL, `Bdate` date DEFAULT NULL, `Address` varchar(30) DEFAULT NULL, `Gender` char(1) DEFAULT NULL, `Salary` decimal(10,2) DEFAULT NULL, `Super_ssn` char(9) DEFAULT NULL, `Dno` int(11) NOT NULL);

- CREATE TABLE `project` (`Pname` varchar(15) NOT NULL, `Pnumber` int(11) NOT NULL, `Plocation` varchar(15) DEFAULT NULL, `Dnum` int(11) NOT NULL);

- CREATE TABLE `works_on` (`Essn` char(9) NOT NULL, `Pno` int(11) NOT NULL, `Hours` decimal(3,1) DEFAULT NULL);

Database Management Systems

One possible database state for the COMPANY relational database schema

PES UNIVERSITY

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

Database Management Systems

One possible database state for the COMPANY relational database schema

PES UNIVERSITY

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

PES UNIVERSITY

1. What is the difference between CHAR(n) and VARCHAR(n)?

A) CHAR(n) stores variable-length strings, VARCHAR(n) stores fixed-length strings

B) CHAR(n) stores fixed-length strings, VARCHAR(n) stores variable-length strings

C) Both store fixed-length strings

D) Both store variable-length strings

2. What is the format of the DATE data type in SQL?

A) DD-MM-YYYY

B) MM-DD-YYYY

C) YYYY-MM-DD

D) Any format can be used

3. What is a key difference between BLOB and CLOB data types?

A) BLOB is used for large binary objects, CLOB is used for large character data

B) Both store large binary objects

C) Both store large character objects

D) CLOB is used for numbers only

4. What is the purpose of the ENUM data type in MySQL?

A) Storing fixed-length character data

B) Storing numeric data

C) Defining a list of possible values for a column

D) Storing date and time values

PES UNIVERSITY

Answers:

1. B
2. C
3. A
4. C

THANK YOU

Mahitha G
Department of Computer Science and Engineering
mahithag@pes.edu

Database Management Systems

SQL overview, Data definition, Structure of SQL queries

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgments: Dr. Nagasundari S (Dept of CSE), Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 1: Introduction to Database Management and SQL

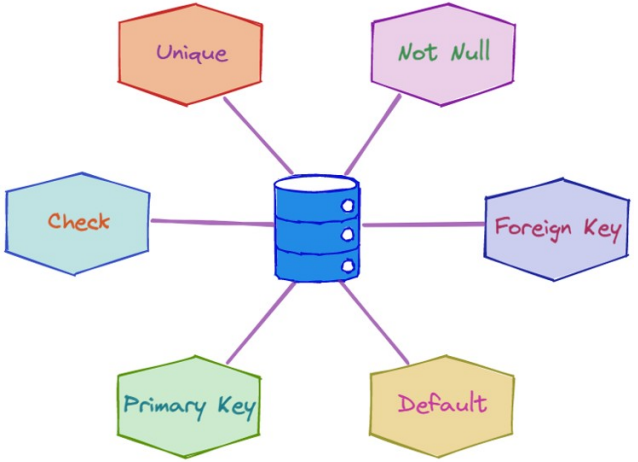
Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G.

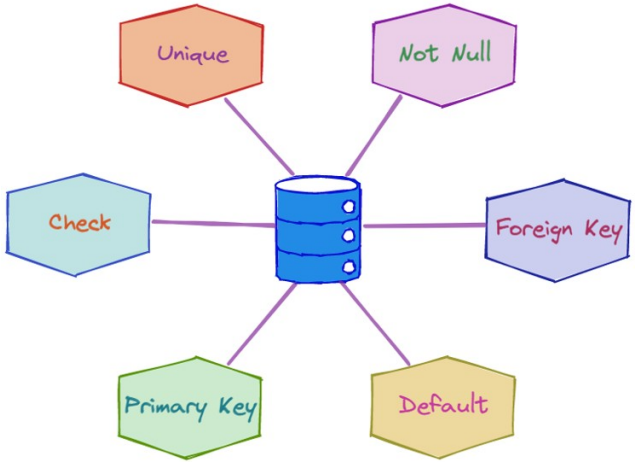
Department of Computer Science and Engineering

Specifying Constraints in SQL

- Constraints in SQL refer to the conditions and restrictions that are applied on the database
- This further means that before inserting data into the database, we are checking for some conditions.
- If the condition we have applied to the database holds true for the data that is to be inserted, then only the data will be inserted into the database tables.
- Constraints can be specified when the table is created with the CREATE TABLE statement, or after the table is created with the ALTER TABLE statement



Specifying Constraints in SQL



Basic constraints:

Relational Model has 3 basic constraint types that are supported in SQL:

- **Key** constraint: A primary key value cannot be duplicated
- **Entity Integrity** Constraint: A primary key value cannot be null
- **Referential integrity** constraints : The “foreign key “ must have a value that is already present as a primary key, or may be null

Specifying Constraints in SQL

PES UNIVERSITY

Other Restrictions on attribute domains:

Default value of an attribute

- **DEFAULT** <value>
- NULL is not permitted for a particular attribute (NOT NULL)

CHECK clause

```
Dnumber INT NOT NULL CHECK (Dnumber > 0 AND Dnumber < 21);
```

Specifying Constraints in SQL

NOT NULL

- Because SQL allows NULLs as attribute values, a constraint NOT NULL may be specified if NULL is not permitted for a particular attribute.
- This is always implicitly specified for the attributes that are part of the primary key of each relation, but it can be specified for any other attributes whose values are required not to be NULL

Example:

```
CREATE TABLE DEPARTMENT
( Dname VARCHAR(15) NOT NULL,
Dnumber INT NOT NULL,
Mgr_ssn CHAR(9) NOT NULL,
Mgr_start_date DATE,
PRIMARY KEY (Dnumber),
UNIQUE (Dname),
FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn) );
```

Specifying Constraints in SQL

DEFAULT

- It is also possible to define a default value for an attribute by appending the clause DEFAULT <value> to an attribute definition.
- The default value is included in any new tuple if an explicit value is not provided for that attribute
- **If no default clause is specified, the default default value is NULL** for attributes that do not have the NOT NULL constraint.

Example:

```
CREATE TABLE DEPARTMENT
(
    Dname VARCHAR(15) DEFAULT 'General',
    Dnumber INT NOT NULL DEFAULT 1,
    Mgr_ssn CHAR(9) NOT NULL,
    Mgr_start_date DATE,
    PRIMARY KEY (Dnumber),
    UNIQUE (Dname),
    FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn)
);
```

Specifying Constraints in SQL

PES UNIVERSITY

PRIMARY KEY clause

- Specifies one or more attributes that make up the primary key of a relation
- Dnumber INT PRIMARY KEY;

UNIQUE clause

- Specifies alternate (secondary) keys (called CANDIDATE keys in the relational model).
- Dname VARCHAR(15) UNIQUE;

Specifying Constraints in SQL

FOREIGN KEY clause

Default operation: reject update on violation

- Attach **referential triggered action** clause
  - Options include SET NULL, CASCADE, and SET DEFAULT
  - Action taken by the DBMS for SET NULL or SET DEFAULT is the same for both ON DELETE and ON UPDATE
  - CASCADE option suitable for “relationship” relations

Specifying Constraints in SQL

Using the Keyword **CONSTRAINT**

- Name a constraint
- Useful for later altering

The general structure of the SQL CONSTRAINT is defined as:

The CONSTRAINT keyword is followed by a constraint name followed by a column or a list of columns.

```
CREATE TABLE EMPLOYEE ( … ,
 Dno INT NOT NULL DEFAULT 1,
 CONSTRAINT EMPPK  PRIMARY KEY (Ssn),
 CONSTRAINT EMPSUPERFK
 FOREIGN KEY (Super_ssn) REFERENCES
EMPLOYEE(Ssn)
 ON DELETE SET NULL ON UPDATE CASCADE,
 CONSTRAINT EMPDEPTFK
 FOREIGN KEY(Dno) REFERENCES
DEPARTMENT(Dnumber)
 ON DELETE SET DEFAULT ON UPDATE CASCADE);
```

Example for Specifying Constraints in SQL

PES UNIVERSITY

Types of SQL CONSTRAINTS

The SQL provides following types of CONSTRAINTS :

Constraint	Description
NOT NULL	This constraint confirms that a column cannot store NULL value.
UNIQUE	This constraint ensures that each row for a column must have a different value.
PRIMARY KEY	This constraint is a combination of a NOT NULL constraint and a UNIQUE constraint. This constraint ensures that the specific column or combination of two or more columns for a table have a unique identity which helps to find a particular record in a table more easily and quickly.
CHECK	A check constraint ensures that the value stored in a column meets a specific condition.
DEFAULT	This constraint provides a default value when specified none for this column.
FOREIGN KEY	A foreign key constraint is used to ensure the referential integrity of the data. in one table to match values in another table.

```
CREATE TABLE mytest(
agent_code char(6) NOT NULL UNIQUE ,
agent_name char(25) NOT NULL UNIQUE ,
working_area char(25) DEFAULT 'Mumbai',
commission decimal(8,2)
CHECK(commission>.1 AND commission<.3));
```

```
CREATE TABLE mytest(
agent_code char(6) NOT NULL UNIQUE ,
agent_name char(25) NOT NULL UNIQUE ,
working_area char(25) CHECK
( working_area IN('London','Brisban','Chennai','Mumbai')) ,
commission decimal CHECK(commission<1));
```

Specifying Constraints in SQL

CREATE TABLE DEPT_LOCATIONS (Dnumber INT NOT NULL,

Dlocation VARCHAR(15) NOT NULL DEFAULT ‘Banglore’,

PRIMARY KEY (Dnumber, Dlocation),

Constraint “fk_dno ” FOREIGN KEY (Dnumber) REFERENCES

DEPARTMENT(Dnumber));

Referential actions

FOREIGN KEY (foreign_key_columns)

REFERENCES parent_table(parent_key_columns)

ON UPDATE action

ON DELETE action;

Referential actions

Delete actions of rows in the parent table

If you delete one or more rows in the parent table, you can set one of the following actions:

- ON DELETE NO ACTION: SQL Server raises an error and rolls back the delete action on the row in the parent table.
- ON DELETE CASCADE: SQL Server deletes the rows in the child table that is corresponding to the row deleted from the parent table.
- ON DELETE SET NULL: SQL Server sets the rows in the child table to NULL if the corresponding rows in the parent table are deleted. To execute this action, the foreign key columns must be nullable.
- ON DELETE SET DEFAULT SQL Server sets the rows in the child table to their default values if the corresponding rows in the parent table are deleted. To execute this action, the foreign key columns must have default definitions. Note that a nullable column has a default value of NULL if no default value specified.

By default, SQL Server applies ON DELETE NO ACTION if you don’t explicitly specify any action.

Referential actions

Update action of rows in the parent table

If you update one or more rows in the parent table, you can set one of the following actions:

- ON UPDATE NO ACTION: SQL Server raises an error and rolls back the update action on the row in the parent table.
- ON UPDATE CASCADE: SQL Server updates the corresponding rows in the child table when the rows in the parent table are updated.
- ON UPDATE SET NULL: SQL Server sets the rows in the child table to NULL when the corresponding row in the parent table is updated. Note that the foreign key columns must be nullable for this action to execute.
- ON UPDATE SET DEFAULT: SQL Server sets the default values for the rows in the child table that have the corresponding rows in the parent table updated.

Specifying Constraints on Tuples Using CHECK

- Additional Constraints on individual tuples within a relation are also possible using CHECK

  CHECK clauses at the end of a CREATE TABLE statement

- Apply to each tuple individually

```
CHECK (Dept_create_date <= Mgr_start_date);
```

Schema Change Statements in SQL

PES UNIVERSITY

Schema evolution commands

- DBA may want to change the schema while the database is operational
- Does not require recompilation of the database schema
- schema evolution commands available in SQL, which can be used to alter a schema by adding or dropping tables, attributes, constraints, and other schema elements.
- This can be done while the database is operational and does not require recompilation of the database schema checks must be done by the DBMS to ensure that the changes do not affect the rest of the database and make it inconsistent

The DROP Command

DROP command

- The DROP command can be used to drop named schema elements, such as tables, domains, types, or constraints.
- One can also drop a whole schema if it is no longer needed by using the DROP SCHEMA command. There are two drop behavior options: CASCADE and RESTRICT.
- For example, to remove the COMPANY database schema and all its tables, domains, and other elements, the CASCADE option is used as follows

Drop behavior options:

CASCADE and RESTRICT

Example:

```
DROP SCHEMA COMPANY CASCADE;
```

This removes the schema and all its elements including tables,views, constraints, etc.

The DROP Command

- If a base relation within a schema is no longer needed, the relation and its definition can be deleted by using the DROP TABLE command.
- For example, if we no longer wish to keep track of dependents of employees in the COMPANY database.
- we can get rid of the DEPENDENT relation by issuing the following command:

  DROP TABLE DEPENDENT CASCADE;

- If the RESTRICT option is chosen instead of CASCADE, a table is dropped only if it is not referenced in any constraints

MySql Commands

To create a new database in MySQL use the **CREATE DATABASE** statement with the below syntax:

Syntax:

CREATE DATABASE [IF NOT EXISTS] database_name

To show the database created in MySQL

SHOW CREATE DATABASE employeedb;

To check the created database using the following query:

SHOW DATABASES;

Describe in sql

The DESCRIBE command in MySQL provides a detailed overview of a table's structure, including its column names, data types used, and any constraints applied to the columns.

```
DESCRIBE table_name;
(or)
SHOW COLUMNS FROM table_name;
(or)
DESC table_name;
```

MySql Commands

SQL command **USE** is used to select a particular database:

Syntax:

USE database_name;

To create table in the database the following query is used:

Create table table_name(…….)

To drop the database

DROP DATABASE [IF EXISTS] database_name;

Example:

Drop database customers;

The ALTER table command

PES UNIVERSITY

Alter table actions include:

- Adding or dropping a column (attribute)
- Changing a column definition
- Adding or dropping table constraints

Example:

```
ALTER TABLE people ADD COLUMN Job VARCHAR(12);
```

The ALTER table command

1) ADD a column in the table

```
ALTER TABLE table_name ADD new_column_name column_definition
[ FIRST | AFTER column_name ];
```

2) Add multiple columns in the table

```
ALTER TABLE table_name   ADD new_column_name column_definition
[ FIRST | AFTER column_name ],
ADD new_column_name column_definition   [ FIRST | AFTER column_name ],
```

3) MODIFY column in the table

```
ALTER TABLE table_name  MODIFY column_name column_definition
[ FIRST | AFTER column_name ];
```

4) DROP column in table

```
ALTER TABLE table_name
DROP COLUMN column_name;
```

The ALTER table command

5) RENAME column in table

ALTER TABLE table_name CHANGE COLUMN old_name new_name column_definition

[FIRST | AFTER column_name]

6) RENAME table

ALTER TABLE table_name
RENAME TO new_table_name;

7) **Altering (Changing) a Column Definition or a Name**

To change a column's definition, use **MODIFY** or **CHANGE** clause along with the ALTER command. For example, to change column **c** from CHAR(1) to CHAR(10), you can use the following command

ALTER TABLE table_name MODIFY c CHAR(10);
ALTER TABLE table_name MODIFY j BIGINT NOT NULL DEFAULT 100;
ALTER TABLE table_name ALTER i SET DEFAULT 1000;

*Note:i,j are columns

Change the column position of MySQL table without losing column data

You can change the column position of MySQL table without losing data with the help of ALTER TABLE command

Syntax:

ALTER TABLE yourTableName MODIFY yourColumnName1 data type AFTER /BEFORE yourColumnName2;

Changing a Column's Default Value

Use SET DEFAULT to specify the default value explicitly, or DROP DEFAULT to remove the current default and allow MySQL to assign the "default default."

To change a default value, use

Syntax:

ALTER *col_name* SET DEFAULT

Ex-

```
ALTER TABLE mytbl ALTER j SET DEFAULT 1000;
```

To drop a default value, use

Syntax:

ALTER *col_name* DROP DEFAULT

Ex-

```
ALTER TABLE mytbl ALTER j DROP DEFAULT;
```

Alter Command for adding a attribute

- We use the alter table command to add attributes to an existing table.
- All tuples in the table are assigned null as the value for the new attribute.
- The form of the alter table command is

  alter table r add A D;

- where r is the name of an existing table, A is the name of the attribute to be added, and D is the type of the added attribute.

Alter Command for dropping a attribute

- We can drop attributes from a table by the command

  alter table r drop A;

- where r is the name of an existing table.
- A is the name of an attribute of the table.
- Many database systems do not support dropping of attributes, although they will allow an entire table to be dropped.

Adding and Dropping Constraints

PES UNIVERSITY

Change constraints specified on a table

Add or drop a named constraint

Adding constraint

ALTER table people add constraint people_gender_fk foreign key (gender)

references gender_tab;

Dropping constraint

ALTER TABLE Persons DROP CONSTRAINT PK_Person;

Dropping Columns, Default Values

PES UNIVERSITY

To drop a column

- Choose either CASCADE or RESTRICT
- CASCADE would drop the column from views etc.
- RESTRICT is possible if no views refer to it.

ALTER TABLE EMPLOYEE **DROP COLUMN** Address **CASCADE**;

Default values can be dropped and altered :

ALTER TABLE DEPARTMENT **ALTER COLUMN** Mgr_ssn **DROP DEFAULT**;

ALTER TABLE DEPARTMENT **ALTER COLUMN** Mgr_ssn **SET DEFAULT** '333445555';

Drop table

PES UNIVERSITY

Drop:

- To remove a table from an SQL database, we use the drop table command.
- The drop table command deletes all information about the dropped table from the database drop table r; // Complete table structure will be dropped

Syntax:

```
DROP TABLE table1;

DROP TABLE IF EXISTS table1;

DROP TABLE IF EXISTS table1, table2, table3;
```

PES UNIVERSITY

Truncate:

A truncate SQL statement is used to remove all rows (complete data) from a table. It is similar to the DELETE statement with no WHERE clause.

Syntax:

TRUNCATE TABLE table_name;

// delete complete data from an existing table. Table Structure remains

Drop vs Truncate

PES UNIVERSITY

Difference between DROP and TRUNCATE in SQL

S.No	DROP	TRUNCATE
1.	It is used to eliminate the whole database from the table.	It is used to eliminate the tuples from the table.
2.	Integrity constraints get removed in the DROP command.	Integrity constraint doesn't get removed in the Truncate command.
3.	The structure of the table does not exist.	The structure of the table exists.
4.	Here the table is free from memory.	Here, the table is not free from memory.
5.	It is slow as compared to the TRUNCATE command.	It is fast as compared to the DROP command.

Rename table

PES UNIVERSITY

RENAME:

RENAME TABLE offers more flexibility. It allows renaming multiple tables in one statement.

This can be useful when replacing a table with a new pre-populated version:

Syntax:

RENAME table table_name to new table_name

RENAME TABLE products TO products_old, products_new TO products;

ALTER - Example

Consider 2 tables - ‘employee’ and ‘department’

```
CREATE TABLE employee (
  emp_id INT PRIMARY KEY,
  name VARCHAR(50)
);


CREATE TABLE department (
  id SMALLINT PRIMARY KEY
);
```

ALTER - Example

1. Adding a New Column

```
ALTER TABLE employee
ADD COLUMN department_id INT;
```

2. Reposition a Column (‘department_id’ after ‘name’)

```
ALTER TABLE employee
MODIFY COLUMN department_id INT AFTER name;
--We use the ‘AFTER’ keyword to reposition col1 after any col2
```

To place a column as the first column in the table, we use the ‘FIRST’ keyword

```
ALTER TABLE employee
MODIFY COLUMN department_id INT FIRST;
```

ALTER - Example

3. Rename a Column (department_id to dept_ref_id)

```
ALTER TABLE employee
RENAME COLUMN department_id TO dept_ref_id;
```

4. Change Datatype of a Column (INT to SMALLINT for dept_ref_id)

```
ALTER TABLE employee
MODIFY COLUMN dept_ref_id SMALLINT;
```

ALTER - Example

5. Adding Constraints

a). NOT NULL

```
ALTER TABLE employee
MODIFY COLUMN dept_ref_id SMALLINT NOT NULL;
--This ensures that dept_ref_id can never have a null value
```

b). DEFAULT

```
ALTER TABLE employee
ALTER COLUMN dept_ref_id SMALLINT NOT NULL SET DEFAULT 100;
--This sets the default dept_id as 100 if no value has been assigned to it
```

ALTER - Example

c). CHECK

ALTER TABLE employee
ADD CONSTRAINT chk_dept CHECK (dept_ref_id > 0);
--The chk_dept is a check constraint which will check if the value inserted is >0 and if not, it returns an error.

d). PRIMARY KEY
Suppose we want to make dept_ref_id as the primary key instead of emp_id. For this, we first have to drop the existing primary key (emp_id) and then add the primary key constraint for defp_ref_id

ALTER TABLE employee
DROP PRIMARY KEY;

ALTER TABLE employee
ADD CONSTRAINT pk_emp PRIMARY KEY (dept_ref_id);

ALTER - Example

e). FOREIGN KEY

- ON DELETE CASCADE, ON UPDATE CASCADE

```
ALTER TABLE employee
ADD CONSTRAINT fk_dept
FOREIGN KEY (dept_ref_id) REFERENCES department(id)
ON DELETE CASCADE
ON UPDATE CASCADE;
```

If the foreign key constraint is being added to an attribute which already has an existing foreign key constraint, we first have to drop the existing constraint and then add a new one. Otherwise, it is not necessary.

```
ALTER TABLE employee
DROP FOREIGN KEY fk_dept; -- In this case, the foreign key is named fk_dept
```

ALTER - Example

- ON DELETE SET NULL

```
ALTER TABLE employee
ADD CONSTRAINT fk_null
FOREIGN KEY (dept_ref_id) REFERENCES department(id)
ON DELETE SET NULL;
```

- ON DELETE SET DEFAULT

```
-- Set column default before applying
ALTER TABLE employee
MODIFY COLUMN dept_ref_id SMALLINT DEFAULT 1;

ALTER TABLE employee
ADD CONSTRAINT fk_default
FOREIGN KEY (dept_ref_id) REFERENCES department(id)
ON DELETE SET DEFAULT;
```

ALTER - Example

e). Dropping Constraints

- DROPPING DEFAULT

```
ALTER TABLE employee
ALTER COLUMN dept_ref_id DROP DEFAULT;
```

- DROPPING CHECK

```
ALTER TABLE employee
DROP CONSTRAINT chk_dept;
```

ALTER - Example

- DROPPING PRIMARY KEY

```
ALTER TABLE employee
DROP PRIMARY KEY;
```

- DROPPING FOREIGN KEY

```
ALTER TABLE employee
DROP FOREIGN KEY fk_default;
```

ALTER - Example

f). Dropping a Column

```
ALTER TABLE employee
DROP COLUMN dept_ref_id;
```

1. Which command is used to delete all records from a table without removing its structure?

A) DELETE

B) DROP

C) TRUNCATE

D) ALTER

2. In SQL, what does the DESCRIBE command do?

A) Shows the details of a table structure

B) Deletes a table

C) Modifies a table

D) Creates a new schema

3. Which of the following statements is used to remove an entire table from the database?

A) DELETE

B) DROP

C) TRUNCATE

D) REMOVE

4. The ALTER TABLE statement is used to?

A) Remove a table

B) Modify an existing table structure

C) Retrieve table data

D) Define a new schema

PES UNIVERSITY

Answers:

1. C
2. A
3. B
4. B

THANK YOU

Mahitha G
Department of Computer Science and Engineering
mahithag@pes.edu

Database Management Systems

Database Modification

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgments: Dr. Nagasundari S (Dept of CSE), Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 1: Introduction to Database Management and SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G.

Department of Computer Science and Engineering

Database Modification

DML commands in Structured Query Language change the data present in the SQL database. We can easily access, store, modify, update and delete the existing records from the database using DML commands.

DML commands:

- SELECT Command
- INSERT Command
- UPDATE Command
- DELETE Command

INSERT COMMAND

- In its simplest form, INSERT is used to add a single tuple (row) to a relation (table).
- We must specify the relation name and a list of values for the tuple.
- There are two forms of the INSERT statement.

Form 1:

The values should be listed in the same order in which the corresponding attributes were specified in the CREATE TABLE command.

Syntax:

INSERT INTO table_name VALUES (value1,value2,…..);

Example:

INSERT INTO EMPLOYEE VALUES (‘Richard’, ‘K’, ‘Marini’, ‘653298653’, ‘1962-12-30’, ‘98 Oak Forest, Katy, TX’, ‘M’, 37000, ‘653298653’, 4);

Database Management Systems

INSERT COMMAND

PES UNIVERSITY

Before Insert:

Fname	Minit	Lname	Ssn	Bdate	Address	Gender	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren,Houston,TX	M	30000.00	888665555	5
Franklin	T	Wong	333445555	1955-12-08	638 voss,Houston,TX	M	40000.00	888665555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice,Houston,TX	F	25000.00	333445555	5
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000.00	333445555	5
James	E	Borg	888665555	1937-11-10	450 Stone, Houston,TX	M	55000.00	NULL	1
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire,Tx	F	43000.00	333445555	4
Ahmed	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston,TX	M	25000.00	987654321	4
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle,Spring,Tx	F	25000.00	333445555	4
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Example: INSERT INTO EMPLOYEE VALUES (‘Richard’, ‘K’, ‘Marini’, ‘653298653’, ‘1962-12-30’, ‘98 Oak Forest, Katy, TX’, ‘M’, 37000, ‘653298653’, 4);

Output:

	5	16:20:58	INSERT INTO EMPLOYEE VALUES ('Richard', 'K', 'Marini', '653298653', '1962-12-30', '98...	1 row(s) affected

Fname	Minit	Lname	Ssn	Bdate	Address	Gender	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren,Houston,TX	M	30000.00	888665555	5
Franklin	T	Wong	333445555	1955-12-08	638 voss,Houston,TX	M	40000.00	888665555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice,Houston,TX	F	25000.00	333445555	5
Richard	K	Marini	653298653	1962-12-30	98 Oak Forest, Katy, TX	M	37000.00	653298653	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000.00	333445555	5
James	E	Borg	888665555	1937-11-10	450 Stone, Houston,TX	M	55000.00	NULL	1
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire,Tx	F	43000.00	333445555	4
Ahmed	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston,TX	M	25000.00	987654321	4
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle,Spring,Tx	F	25000.00	333445555	4
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

INSERT COMMAND

- Form 2 allows the user to specify explicit attribute names that correspond to the values provided in the INSERT command.
- This is useful if a relation has many attributes but only a few of those attributes are assigned values in the new tuple.
- However, the values must include all attributes with NOT NULL specification and no default value.
- Attributes with NULL allowed or DEFAULT values are the ones that can be left out.

Syntax:

INSERT INTO table_name(attribute1,attribute2,….) VALUES (value1,value2,…..);

Example: Enter a tuple for a new EMPLOYEE for whom we know only the Fname, Lname, Dno, and Ssn attributes

Query: INSERT INTO EMPLOYEE (Fname, Lname, Dno, Ssn) VALUES (‘Richard’, ‘Marini’, 4, ‘653298653’);

INSERT COMMAND

PES UNIVERSITY

Another variation of the INSERT command:

A variation of the INSERT command inserts multiple tuples into a relation in conjunction with creating the relation and loading it with the result of a query.

- create a temporary table that has the employee last name, project name, and hours per week for each employee working on a project

Command :

CREATE TABLE WORKS_ON_INFO (Emp_name VARCHAR(15), Proj_name VARCHAR(15), Hours_per_week DECIMAL(3,1));

INSERT COMMAND

PES UNIVERSITY

Example:

```
INSERT INTO WORKS_ON_INFO ( Emp_name, Proj_name, Hours_per_week )
SELECT E.Lname, P.Pname, W.Hours
FROM PROJECT P, WORKS_ON W, EMPLOYEE E
WHERE P.Pnumber = W.Pno AND W.Essn = E.Ssn;
```

Another variation: We can use the command given below to backup the data present in a table. The condition is that the schema of both the backup table and the original should must be the same.

Example:

```
INSERT INTO WORKS_ON_INFO_Backup (select * from WORKS_ON_INFO ) ;
```

Note: WORKS_ON_INFO_Backup table need to be created before.

Database Management Systems

SELECT COMMAND

- The **SELECT** clause lists the attributes desired in the result of a query.
- It corresponds to the projection operation of relational algebra.
- **FROM** clause is used to specify the table name or the relation name.

Example: Find the names of all instructors:

select *fname,Lname*

from EMPLOYEE*;*

Output:

Fname	Lname
John	Smith
John	Smith
Franklin	Wong
Joyce	English
Ramesh	Narayan
James	Borg

Note: Select will be discussed in detail in the next lecture

UPDATE COMMAND

- The UPDATE command is used to modify attribute values of one or more selected tuples.
- As in the DELETE command, a WHERE clause in the UPDATE command selects the tuples to be modified from a single relation.
- However, updating a primary key value may propagate to the foreign key values of tuples in other relations if such a referential triggered action is specified in the referential integrity Additional constraints of the DDL.
- An additional SET clause in the UPDATE command specifies the attributes to be modified and their new values.
- Several tuples can be modified with a single UPDATE command.

Syntax:

```
UPDATE table_name SET column_name = value WHERE condition;
```

Database Management Systems

UPDATE COMMAND

PES UNIVERSITY

Example1: To change the location and controlling department number of project number 10 to ‘Bellaire’ and 5, respectively.

Query: UPDATE PROJECT SET Plocation = ‘Bellaire’, Dnum = 5 WHERE Pnumber = 10;

Before:

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4
NULL	NULL	NULL	NULL

After:

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Bellaire	5
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4
NULL	NULL	NULL	NULL

✔	8	16:35:34	UPDATE PROJECT SET Plocation = 'Bellaire', Dnum = 5 WHERE Pnumber = 10	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0

Database Management Systems

UPDATE COMMAND

Example 1: Give all employees in the ‘Research’ department a 10% raise in salary.

Query: UPDATE EMPLOYEE SET Salary = Salary * 1.1 WHERE Dno = 5;

Before :

fname	salary	dno
John	33000.00	5
Franklin	44000.00	5
Joyce	27500.00	5
Richard	37000.00	4
Ramesh	41800.00	5
James	55000.00	1
Jennifer	43000.00	4
Ahmed	25000.00	4
Alicia	25000.00	4

After :

fname	salary	dno
John	36300.00	5
Franklin	48400.00	5
Joyce	30250.00	5
Richard	37000.00	4
Ramesh	45980.00	5
James	55000.00	1
Jennifer	43000.00	4
Ahmed	25000.00	4
Alicia	25000.00	4

Example 2: Give a 5 percent salary raise to employees whose salary is less than average.

Query: UPDATE employee SET Salary = Salary * 1.05 WHERE salary < (SELECT avg_sal FROM (SELECT AVG(salary) AS avg_sal FROM employee) AS temp);

UPDATE COMMAND

Example 2: Give a 5 percent salary raise to employees whose salary is less than average.

Query: UPDATE employee SET Salary = Salary * 1.05 WHERE salary < (SELECT avg_sal FROM (SELECT AVG(salary) AS avg_sal FROM employee) AS temp);

In PostgreSQL / Oracle / SQL Server:

UPDATE employee SET Salary = Salary * 1.05 WHERE salary < (SELECT AVG(salary) FROM employee);

This does not work in MySQL, reason is it does not allow selecting from the same table you are updating (in the same query) without wrapping it in a derived table.

Database Management Systems

DELETE COMMAND

PES UNIVERSITY

- The DELETE command removes tuples from a relation.
- It includes a WHERE clause, similar to that used in an SQL query, to select the tuples to be deleted.
- Tuples are explicitly deleted from only one table at a time.
- However, the deletion may propagate to tuples in other relations if referential triggered actions are specified in the referential integrity constraints of the DDL.
- Depending on the number of tuples selected by the condition in the WHERE clause, zero, one, or several tuples can be deleted by a single DELETE command.
- A missing WHERE clause specifies that all tuples in the relation are to be deleted. However, the table remains in the database as an empty table.
- We must use the DROP TABLE command to remove the table definition.

Syntax:

DELETE FROM *table_name* WHERE *condition*

Database Management Systems

DELETE COMMAND

Example1: Delete the employee record whose last name is Zelaya .

Query: DELETE FROM EMPLOYEE WHERE Lname = ‘Zelaya’;

Before:

fname	lname	salary	dno
John	Smith	36300.00	5
Franklin	Wong	48400.00	5
Joyce	English	30250.00	5
Richard	Marini	37000.00	4
Ramesh	Narayan	45980.00	5
James	Borg	55000.00	1
Jennifer	Wallace	43000.00	4
Ahmed	Jabbar	25000.00	4
Alicia	Zelaya	25000.00	4

fname	lname	salary	dno
John	Smith	36300.00	5
Franklin	Wong	48400.00	5
Joyce	English	30250.00	5
Richard	Marini	37000.00	4
Ramesh	Narayan	45980.00	5
James	Borg	55000.00	1
Jennifer	Wallace	43000.00	4
Ahmed	Jabbar	25000.00	4

Example2: Delete all employee records [It is similar to truncating employee table]

Query: DELETE FROM EMPLOYEE;

Database Management Systems

Q & A

PES UNIVERSITY

1. Which SQL clause is used to specify the attributes that should be displayed in the output?

A) FROM

B) WHERE

C) SELECT

D) ORDER BY

2. Which DML operation is used to remove data from a database?

A) SELECT

B) INSERT

C) DELETE

D) UPDATE

PES UNIVERSITY

3. In the SQL DML example, "SELECT name FROM instructor WHERE dept_name = 'History'", what does 'name' refer to?

A) A column in the 'instructor' table

B) A table in the database

C) A value in the 'dept_name' column

D) A query condition

4. What is the purpose of the Data Manipulation Language (DML)?

A) Defining the structure of a database

B) Querying the database

C) Managing database access

D) Storing the data in the database

Database Management Systems

Q & A

PES UNIVERSITY

Answers:

1. C
2. C
3. A
4. B

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Additional Basic Operations

Dr. Nagasundari S

Department of Computer Science and Engineering

PES UNIVERSITY

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

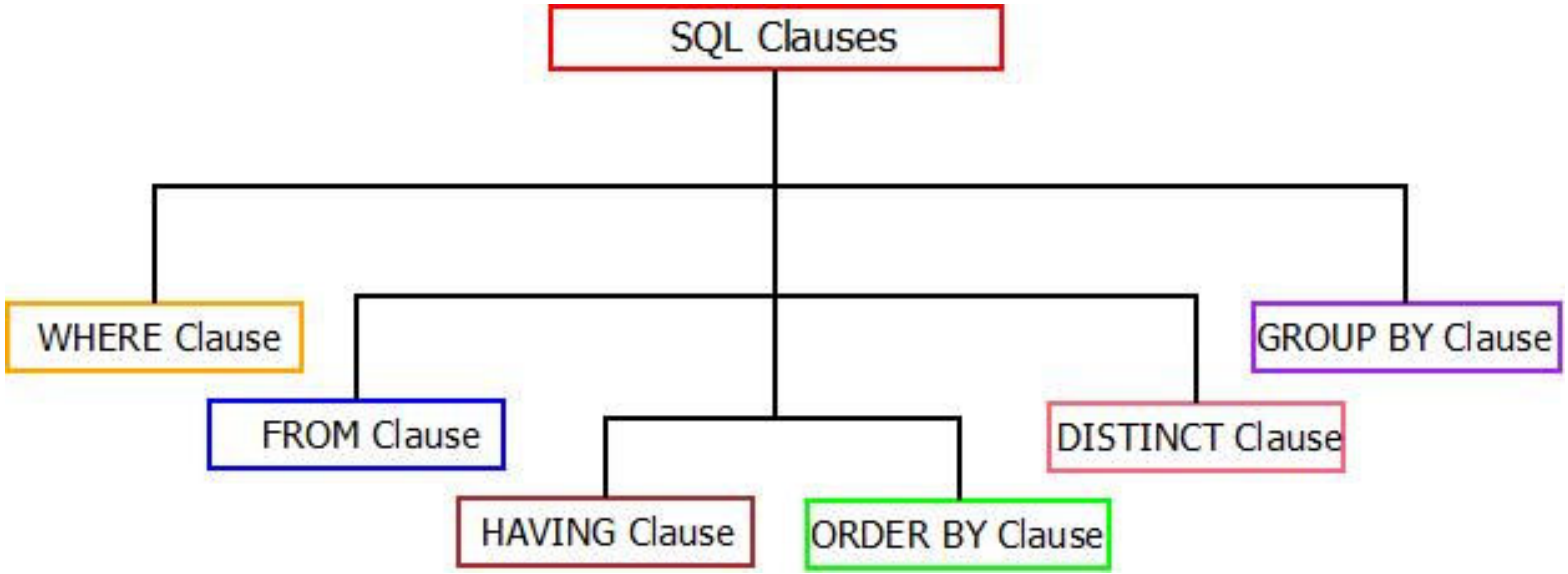
Dr. Nagasundari S

Department of Computer Science and Engineering

PES UNIVERSITY

Additional Basic Operations

- Select clause
- Distinct clause
- Where clause
- From clause
- Order clause
- Having clause
- Group by clause
- String operations



SELECT-FROM-WHERE Structure

SELECT <attribute and function list>
FROM <table list>
[**WHERE** <condition>]
[**GROUP BY** <grouping attribute(s)>]
[**HAVING** <group condition>]
[**ORDER BY** <attribute list>];

Database Management Systems

Select & From clause

- The **SELECT** clause lists the attributes desired in the result of a query.
- It corresponds to the projection operation of relational algebra.
- The **FROM** clause is used to specify the table name or the relation name.

Example: Find the names of all employees.

Query: select Fname, Lname from EMPLOYEE;

Output:

Fname	Lname
John	Smith
John	Smith
Franklin	Wong
Joyce	English
Ramesh	Narayan
James	Borg
Jennifer	Wallace
Ahmed	Jabbar
Alicia	Zelaya

NOTE: SQL names are case insensitive (i.e., you may use upper- or lower-case letters.)

E.g., Name ≡ NAME ≡ name

Some people use upper case wherever we use bold font.

Database Management Systems

Select & distinct clause

- SQL allows duplicates in relations as well as in query results.
- To force the elimination of duplicates, insert the keyword **DISTINCT** after SELECT.

Example: **Retrieve the name and address of all employees (display unique names).**

Query: SELECT **distinct** Fname, Lname, Address

FROM EMPLOYEE;

Output:

Fname	Lname	Address
John	Smith	731 Fondren,Houston,TX
Franklin	Wong	638 voss,Houston,TX
Joyce	English	5631 Rice,Houston,TX
Ramesh	Narayan	975 Fire Oak, Humble, TX
James	Borg	450 Stone, Houston,TX
Jennifer	Wallace	291 Berry, Bellaire,Tx
Ahmed	Jabbar	980 Dallas, Houston,TX
Alicia	Zelaya	3321 Castle,Spring,Tx

Select clause

- **The SELECT clause can contain arithmetic expressions involving the operations, +, –, *, / and operating on constants or attributes of tuples.**

Query: select fname, salary/2 from EMPLOYEE;

Table:

fname	salary
John	30000.00
John	30000.00
Franklin	40000.00
Joyce	25000.00
Ramesh	38000.00

Output:

fname	salary/2
John	15000.000000
John	15000.000000
Franklin	20000.000000
Joyce	12500.000000
Ramesh	19000.000000

- Query returns a relation that is same as the EMPLOYEE relation, except that the value of the attribute *salary* is divided by 2.
- Can rename "s*alary/2"* using the **as** clause:

  select *fname, salary/2* as *monthly_salary*

Database Management Systems

Where clause

- The **WHERE** clause specifies conditions that the result must satisfy (row filter)
- It corresponds to the selection predicate of relational algebra.

Example: Find all employees in department no 5.

Query: select fname ,dno from EMPLOYEE where dno = 5;

Output:

fname	dno
John	5
John	5
Franklin	5
Joyce	5
Ramesh	5

- SQL allows the use of logical connectives **AND, OR,** and **NOT.**
- The operands of the logical connectives can be expressions involving the comparison operators <, <=, >, >=, =, and <>.
- Comparisons can be applied to results of arithmetic expressions

Output:

Example: Find all employees in department no 5 with salary lesser than $70,000.

Query: select fname, salary from employee where dno = 5 and salary < 70000;

Examples :

Example: For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birth date.

Query:
```
SELECT Pnumber, Dnum, Lname, Address, Bdate
    FROM PROJECT, DEPARTMENT, EMPLOYEE
    WHERE Dnum = Dnumber AND Mgr_ssn = Ssn AND Plocation = 'Stafford';
```

Output:

Pnumber	Dnum	Lname	Address	Bdate
10	4	Wallace	291Berry, Bellaire, TX	1941-06-20
30	4	Wallace	291Berry, Bellaire, TX	1941-06-20

Database Management Systems

Order By clause

PES UNIVERSITY

- ORDER BY clause is a simple keyword that can be used with SQL aggregate functions, HAVING clause, or GROUP BY clause.
- The main function of the ORDER BY clause is to sort the result in either ascending or descending order.
- It means, sorting the result requires an ORDER BY clause.
- This can be done using either of the two keywords.

  ASC – For sorting results in ascending order.

  DESC – For sorting results in descending order.
- If we do not place either “ASC or DESC” at the end of the query, the data is sorted in ascending order by default.

Database Management Systems

Order By clause

Example1: List the employees based on descending order of their salary.

Query: select fname, lname, salary from EMPLOYEE order by salary desc;

Example2: List the employees based on ascending order of their salary

Query: select fname, lname, salary from EMPLOYEE order by salary asc;

Output1:

fname	lname	salary
James	Borg	55000.00
Jennifer	Wallace	43000.00
Franklin	Wong	40000.00
Ramesh	Narayan	38000.00
John	Smith	30000.00
Joyce	English	25000.00
Ahmed	Jabbar	25000.00
Alicia	Zelaya	25000.00

fname	lname	salary
Joyce	English	25000.00
Ahmed	Jabbar	25000.00
Alicia	Zelaya	25000.00
John	Smith	30000.00
Ramesh	Narayan	38000.00
Franklin	Wong	40000.00
Jennifer	Wallace	43000.00
James	Borg	55000.00

Rename

- SQL allows renaming relations and attributes using the **as** clause:

  old-name **as** *new-name*

Example: Find the annual salary of employees.

Query: select fname,lname, salary*12 as Annual_Income from EMPLOYEE;

Output:

fname	lname	Annual_Income
John	Smith	360000.00
Franklin	Wong	480000.00
Joyce	English	300000.00
Ramesh	Narayan	456000.00
James	Borg	660000.00
Jennifer	Wallace	516000.00
Ahmed	Jabbar	300000.00
Alicia	Zelaya	300000.00

*NOTE: Keyword **as** is optional and may be omitted instructor **as** T ≡ instructor T*

Database Management Systems

String comparison

- SQL includes a string-matching operator for comparisons on character strings.
- The operator **LIKE** uses patterns that are described using two special characters:

  percent (%): The % character matches any substring.

  underscore (_): The _ character matches any character.

Example: Retrieve all employees whose address is in Houston, Texas.

Query: SELECT Fname, Lname FROM EMPLOYEE WHERE Address LIKE '%Houston,TX%';

Output:

Fname	Lname	Address
John	Smith	731 Fondren,Houston,TX
Franklin	Wong	638 voss,Houston,TX
Joyce	English	5631 Rice,Houston,TX
James	Borg	450 Stone, Houston,TX
Ahmed	Jabbar	980 Dallas, Houston,TX

*Note: Match the string "100%" **like** '100 \%' **escape** '\' in that above we use backslash (\) as an escape character*

Database Management Systems

String comparison

Examples: **Find all employees who were born during the 1960s.**

Query: SELECT Fname, Lname, bdate FROM EMPLOYEE WHERE Bdate LIKE '196_ _ _ _ _ _ _ _';

Output:

Fname	Lname	bdate
John	Smith	1965-01-09
Ramesh	Narayan	1962-09-15
Ahmed	Jabbar	1969-03-29
Alicia	Zelaya	1968-01-19

THANK YOU

Dr. Nagasundari S
Department of Computer Science and Engineering

nagasundaris@pes.edu

Database Management Systems

Set Operations and Null Values

Dr. Nagasundari S

Department of Computer Science and Engineering

PES UNIVERSITY

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

SET Operators in SQL

SQL Set operations are used to combine two or more SQL SELECT statements.

Types of Set Operations:

1. Union
2. Union All
3. Intersect
4. Intersect All
5. Except
6. Except All

Union

1. Union

In the UNION operation, the **number of columns and datatypes** must be the same in both the tables on which the UNION operation is being applied.

It **eliminates duplicate rows** from its result set.

Syntax:

SELECT *column_name* FROM *table1*

UNION

SELECT *column_name* FROM *table2*;

Example

Consider the relation: *student_course(name, course, grade)* consisting of records of grades obtained by students in various courses.

The first image displays students and the grades obtained by them in the course ‘Physics’

The second image displays students and the grades obtained by them in ‘Mathematics’

Further on, these tables will be used to demonstrate the usage of set operations

name	course	grade
Alex	Physics	S
Bert	Physics	S
Charles	Physics	A
Dennis	Physics	B
Evans	Physics	B

name	course	grade
Alex	Mathematics	S
Bert	Mathematics	S
Charles	Mathematics	A
Dennis	Mathematics	A
Evans	Mathematics	A

Union

Query :

```
SELECT grade FROM student_course
WHERE course = 'Physics'
UNION
SELECT grade FROM student_course
WHERE course = 'Mathematics';
```

grade
S
A
B

Output :

name	course	grade
Alex	Physics	S
Bert	Physics	S
Charles	Physics	A
Dennis	Physics	B
Evans	Physics	B

name	course	grade
Alex	Mathematics	S
Bert	Mathematics	S
Charles	Mathematics	A
Dennis	Mathematics	A
Evans	Mathematics	A

Union All

2. Union All

The UNION ALL operation is similar to the UNION operation. It returns the set **without removing duplicates** and sorting the data. If a particular value is present c1 times in the first set(or relation) and c2 times in the second set(or relation), the number of duplicates or copies present in the result would be **c1 + c2**.

Syntax:

SELECT *column_name* FROM *table1*

UNION ALL

SELECT *column_name* FROM *table2;*

Database Management Systems

Union All

Query:

Output :

```
SELECT grade FROM student_course
WHERE course = 'Physics'
UNION ALL
SELECT grade FROM student_course
WHERE course = 'Mathematics';
```

grade
S
S
A
B
B
S
S
A
A
A

name	course	grade
Alex	Physics	S
Bert	Physics	S
Charles	Physics	A
Dennis	Physics	B
Evans	Physics	B

name	course	grade
Alex	Mathematics	S
Bert	Mathematics	S
Charles	Mathematics	A
Dennis	Mathematics	A
Evans	Mathematics	A

Union All

Query:

```
SELECT distinct grade FROM student_course
WHERE course = 'Physics'
UNION ALL
SELECT distinct grade FROM student_course
WHERE course = 'Mathematics';
```

grade
S
A
B
S
A

name	course	grade
Alex	Physics	S
Bert	Physics	S
Charles	Physics	A
Dennis	Physics	B
Evans	Physics	B

name	course	grade
Alex	Mathematics	S
Bert	Mathematics	S
Charles	Mathematics	A
Dennis	Mathematics	A
Evans	Mathematics	A

Notice the difference between the results of the two UNION ALL operations. This is due to the inclusion of the DISTINCT keyword. DISTINCT eliminates any duplicates and hence, the output of each SELECT statement has no duplicates. However, due to the usage of UNION ALL, duplicates are not eliminated in the result of the UNION ALL operation.

Intersect

3. Intersect

It is used to combine two SELECT statements. The INTERSECT operation returns common rows from both the SELECT statements.

In the Intersect operation, the **number of columns and corresponding data types** must be the same.

It has **no duplicates.**

Syntax:

SELECT *column_name* FROM *table1*

INTERSECT

SELECT *column_name* FROM *table2*;

Intersect

Query :

```
SELECT grade FROM student_course
WHERE course = 'Physics'
INTERSECT
SELECT grade FROM student_course
WHERE course = 'Mathematics';
```

Output :

grade
S
A

name	course	grade
Alex	Physics	S
Bert	Physics	S
Charles	Physics	A
Dennis	Physics	B
Evans	Physics	B

name	course	grade
Alex	Mathematics	S
Bert	Mathematics	S
Charles	Mathematics	A
Dennis	Mathematics	A
Evans	Mathematics	A

Intersect All

4. Intersect All

INTERSECT ALL is similar to the INTERSECT operation. It helps **retain duplicates**.

If a particular value is present c1 times in the first set(or relation) and c2 times in the second set(or relation), the number of duplicates or copies present in the result would be **min(c1,c2)**.

Syntax:

SELECT *column_name* FROM *table1*

INTERSECT ALL

SELECT *column_name* FROM *table2*;

Intersect All

Query:

```
SELECT grade FROM student_course
WHERE course = 'Physics'
INTERSECT ALL
SELECT grade FROM student_course
WHERE course = 'Mathematics';
```

Output :

grade
S
S
A

name	course	grade
Alex	Physics	S
Bert	Physics	S
Charles	Physics	A
Dennis	Physics	B
Evans	Physics	B

name	course	grade
Alex	Mathematics	S
Bert	Mathematics	S
Charles	Mathematics	A
Dennis	Mathematics	A
Evans	Mathematics	A

Except

5. Except

It combines the result of two SELECT statements. EXCEPT operator is used to display the rows that are present in the first query but absent in the second query.

It has **no duplicates**

Syntax:

SELECT *column_name* FROM *table1*

EXCEPT

SELECT *column_name* FROM *table2*;

Except

Query :

```
SELECT grade FROM student_course
WHERE course = 'Physics'
EXCEPT
SELECT grade FROM student_course
WHERE course = 'Mathematics';
```

grade
B

Output :

name	course	grade
Alex	Physics	S
Bert	Physics	S
Charles	Physics	A
Dennis	Physics	B
Evans	Physics	B

name	course	grade
Alex	Mathematics	S
Bert	Mathematics	S
Charles	Mathematics	A
Dennis	Mathematics	A
Evans	Mathematics	A

```
SELECT grade FROM student_course
WHERE course = 'Mathematics'
EXCEPT
SELECT grade FROM student_course
```

grade

(Empty Set)

Except All

6. Except All

EXCEPT ALL is similar to the EXCEPT operation. It helps **retain duplicates**. If a particular value is present c1 times in the first set(or relation) and c2 times in the second set(or relation), the number of duplicates or copies present in the result would be **max(c1-c2,0)** i.e. c1 – c2 if the difference is positive else 0 (the value itself will not be present)

Syntax:

SELECT *column_name* FROM *table1*

EXCEPT ALL

SELECT *column_name* FROM *table2*;

Except All

Query :

```
SELECT grade FROM student_course
WHERE course = 'Physics'
EXCEPT ALL
SELECT grade FROM student_course
WHERE course = 'Mathematics';
```

grade
B
B

Output :

name	course	grade
Alex	Physics	S
Bert	Physics	S
Charles	Physics	A
Dennis	Physics	B
Evans	Physics	B

name	course	grade
Alex	Mathematics	S
Bert	Mathematics	S
Charles	Mathematics	A
Dennis	Mathematics	A
Evans	Mathematics	A

```
SELECT grade FROM student_course
WHERE course = 'Mathematics'
EXCEPT ALL
SELECT grade FROM student_course
```

grade
A
A

Null values - Introduction

SQL has various rules for dealing with NULL values.

Generally, NULL has one of the three representations:

1. **Unknown value**: value exists but is not known, or it is not known whether or not the value exists. For example, when a person's date of birth is not known, it is represented as NULL in the database.
2. **Unavailable or withheld value**: value exists but is purposely withheld. For example, a person might not want to disclose his/her date of birth.
3. **Not applicable attributes**: the attribute does not apply to this tuple or is undefined for this tuple. For example, an attribute LastCollegeDegree would be NULL for a person who has no college degrees.

Null values - Introduction

- It is often not possible to determine the intended meaning of NULL. Hence, **SQL does not distinguish among the different meanings of NULL.**
- In general, each individual NULL value is considered to be **different** from every other NULL value in the various database records.
- Null values present special problems in relational operations, including arithmetic operations, comparison operations, and set operations.
- Result of an arithmetic operation (+,-,* or /) is **NULL** if **any of the input values are NULL**.

Null values in Arithmetic operations

Example:

SELECT Hours FROM WORKS_ON;

Hours
32.5
7.5
10.0
10.0
10.0
10.0
20.0
20.0
40.0
NULL
15.0
20.0

SELECT Hours+10 from WORKS_ON;

Hours+10
42.5
17.5
20.0
20.0
20.0
20.0
30.0
30.0
50.0
NULL
25.0
30.0

The first image shows a part of the *Hours* column of the *WORKS_ON* relation. The second image shows the result of adding 10 to each value in the *Hours* column. Notice how the **result of adding 10 to NULL results in NULL.**

Null values in comparison operations

PES UNIVERSITY

- Comparisons involving NULL values pose a greater difficulty.
- To handle comparisons involving NULL values in SQL, a third logical value **UNKNOWN**, in addition to **TRUE** and **FALSE**, is used.
- For example, the expression **(1 < NULL)** would be evaluated as **UNKNOWN**. Similarly, **NOT(1< NULL)** would be evaluated as **UNKNOWN** as well.
- In other words, any comparison operation involving NULL values would result in an UNKNOWN.

Three-valued logic truth table

Truth table for NOT, AND, OR operators:

(a)

AND	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN

(b)

OR	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN

(c)

NOT	
TRUE	FALSE
FALSE	TRUE
UNKNOWN	UNKNOWN

Is null and is not null

- **If the WHERE predicate evaluates to FALSE or UNKNOWN for a tuple, the tuple is not added to the result.**
- When a JOIN condition is specified, tuples with NULL values for the join attributes are not included in the result (unless it is an OUTER JOIN)
- SQL uses comparison operators **IS** and **IS NOT** to check if an attribute value is NULL or not, respectively

Example: **Retrieve the names of all employees who do not have supervisors.**

Query: SELECT Fname, Lname FROM EMPLOYEE WHERE Super_ssn IS NULL;

Output:

Fname	Lname
James	Borg

Is unknown and is not unknown

- SQL allows us to test whether the result of a comparison is UNKNOWN , rather than TRUE or FALSE, by using the clauses **IS UNKNOWN** and **IS NOT UNKNOWN**.

Example: Retrieve the ssn of employees and their corresponding project numbers which satisfy the condition “Hours > 10 is unknown”.

Query: SELECT Essn, Pno FROM WORKS_ON WHERE Hours > 10 IS UNKNOWN;

Output:

Essn	Pno
888665555	20

Only those tuples whose *Hours* attribute value is NULL will return UNKNOWN on performing the comparison.

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
453453453	1	20.0
453453453	2	20.0
666884444	3	40.0
888665555	20	NULL
987654321	20	15.0
987654321	30	20.0

Null values in distinct clause

- When a query uses the **SELECT DISTINCT** clause, duplicate tuples are eliminated.
- For this purpose, when comparing values of corresponding attributes from two tuples, the values are treated as **identical** if either **both are non NULL and equal in value**, or **both are NULL.**
- **For example, the tuples ('A', NULL) and ('A', NULL) are treated as identical. Hence, DISTINCT retains only one copy of such distinct tuples.**
- NOTE: Treatment of NULL above is different from the way NULL values are treated in predicates, where a comparison "NULL=NULL" would return UNKNOWN, rather than TRUE.
- The approach of treating tuples as identical if they have the same values for all attributes, even if some of the values are NULL, is also used for the set operations UNION, INTERSECTION, and EXCEPT.

THANK YOU

Dr. Nagasundari S
Department of Computer Science and Engineering

nagasundaris@pes.edu

Database Management Systems

Aggregate Functions

Dr. Nagasundari S
Department of Computer Science and Engineering

PES UNIVERSITY

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

Aggregate functions - Introduction

- Aggregate functions are used to summarize information from multiple tuples into a single-tuple summary.
- Grouping is used to create subgroups of tuples before summarization.

Built-in aggregate functions:

- **COUNT** - returns the number of tuples or values specified in a query
- **SUM** – returns the sum of a set (or multiset) of numeric values
- **MAX** – returns the maximum value from a set(or multiset) of numeric values
- **MIN** - returns the minimum value from a set(or multiset) of numeric values
- **AVG** – returns the average of a set(or multiset) of numeric values

NOTE: Multiset is a collection similar to a set wherein the multiplicity of every value is greater than or equal to 1. In other words, it is a set that can contain duplicate values.

Aggregate functions - Example

- The functions MAX and MIN can also be used with attributes that have non-numeric domains if the domain values have a total ordering among one another. DATE, TIME, TIMESTAMP, and alphanumeric strings are examples of such non-numeric domains.
- **Total order** - For any two values in the domain, it can be determined that one appears before the other in the defined order.

Example: **Find the sum of salaries of all employees, the maximum salary, the minimum salary, and the average salary**

Query: SELECT SUM(Salary), MAX(Salary), MIN(Salary), AVG(Salary) FROM EMPLOYEE;

Output:

SUM(Salary)	MAX(Salary)	MIN(Salary)	AVG(Salary)
281000.00	55000.00	25000.00	35125.000000

Database Management Systems

As clause

- The AS clause can be used to rename column names to more meaningful ones.

Query: SELECT SUM (Salary) AS Total_Sal, MAX (Salary) AS Highest_Sal, MIN (Salary) AS Lowest_Sal, AVG (Salary) AS Average_Sal FROM EMPLOYEE;

Output:

Total_Sal	Highest_Sal	Lowest_Sal	Average_Sal
281000.00	55000.00	25000.00	35125.000000

Count - Example

Example: **Retrieve the total number of employees in the company.**

Query: SELECT COUNT(*) FROM EMPLOYEE;

Output:

COUNT(*)
8

Example: **Find the number of employees in the 'Research' department.**

Query: SELECT COUNT(*) FROM EMPLOYEE, DEPARTMENT

WHERE DNO = DNUMBER AND DNAME = 'Research';

Output:

COUNT(*)
4

- In general, COUNT(*) can be used to find the number of rows present in the result of the query.

Aggregate functions - Example

Example:

Find the sum of salaries of all employees from the 'Research' department, as well as the maximum salary, the minimum salary, and the average salary in this department.

Query: SELECT SUM(Salary), MAX(Salary), MIN(Salary), AVG(Salary)

FROM (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber)

WHERE Dname = 'Research'; //Tables are specified in FROM clause – so join operation

Output:

SUM(Salary)	MAX(Salary)	MIN(Salary)	AVG(Salary)
133000.00	40000.00	25000.00	33250.000000

Use of distinct

- The COUNT function can be used to count the number of values in a column as well

Example: **Count the number of distinct salary values in the database.**

Query: SELECT COUNT(DISTINCT Salary) FROM EMPLOYEE;

Output:

COUNT(DISTINCT Salary)
6

- Note the usage of the DISTINCT in the previous query. To understand its significance, we shall run the same query without using DISTINCT.

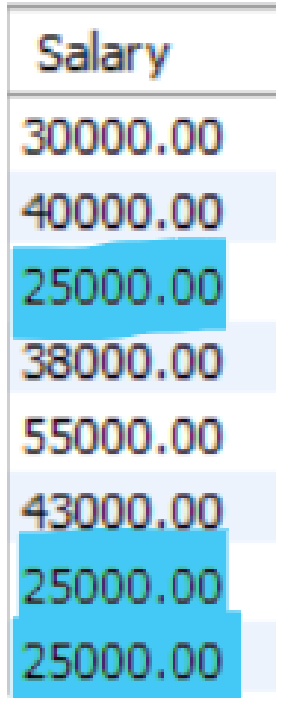
Query: **SELECT COUNT(Salary) FROM EMPLOYEE;**

Output:

COUNT(Salary)
8

Use of distinct

- SQL, by default, does not eliminate duplicates in the result of a query. Duplicate retention can be specified using the ALL keyword
- If duplicate elimination is required, it must be explicitly specified by using the DISTINCT keyword.
- Notice how there are three copies of the same value in the *Salary* column.
- The total number of values in the *Salary* column equals 8, and hence COUNT (Salary) returns 8 (no duplicate elimination).
- By using the DISTINCT keyword, we are specifying duplicate elimination. Hence, out of the three copies, two are discarded and one is retained. Therefore, COUNT(DISTINCT Salary) returns 6 (8 – 2 (discarded copies))



Salary
30000.00
40000.00
25000.00
38000.00
55000.00
43000.00
25000.00
25000.00

Aggregate functions – Handling of Null values

- In general, when an aggregate function is applied to a collection of values, NULL values are discarded before the calculation.
- **If the collection becomes empty because all the values are NULL, COUNT returns zero and other aggregate functions return NULL.**

Example: **Find the average number of hours that employees work on each project.**

Query: SELECT Pno, AVG(Hours) as avg_hours FROM WORKS_ON GROUP BY Pno;

Output:

Pno	avg_hours
1	26.25000
2	12.50000
3	25.00000
10	18.33333
20	12.50000
30	18.33333

Aggregate functions – Handling of Null values

- Let's have a look at *WORKS_ON* tuples that correspond to *Pno* = 20:

Essn	Pno	Hours
333445555	20	10.0
888665555	20	NULL
987654321	20	15.0

- Notice how one of the entries in the *Hours* column is NULL.
- When AVG is applied to the *Hours* column for *Pno* = 20, it discards the NULL value and considers the other two values. Therefore, the average computation for *Pno* = 20 is as follows:

  (10 + 15)/ 2 = 12.5

Aggregate functions in nested queries

- Aggregate functions can also be used in selection conditions involving nested queries.
- They can be used in correlated nested queries, which are then placed in the WHERE clause of an outer query.

Example: **Retrieve the names of all employees who have two or more dependents.**

Query:
```
SELECT Lname, Fname FROM EMPLOYEE
    WHERE ( SELECT COUNT(*) FROM DEPENDENT WHERE Ssn = Essn ) >= 2;
```

Output:

Lname	Fname
Smith	John
Wong	Franklin

Aggregate functions in nested queries

The correlated query returns the count of dependents for every employee, which is then checked if it is greater than or equal to 2. All tuples from the *EMPLOYEE* table that satisfy the said condition are then selected.

- SQL has two functions that are applied to Boolean values – **SOME** and **ALL**.
- The SOME function **returns True if at least one element in the collection is True.**
- The ALL function **returns True if all the elements in the collection are True.**

Grouping

- In many cases, we would want to apply the aggregate functions to subgroups of tuples in a relation where the subgroups are based on some attribute values.
- For example, we may want to find the average salary of employees in each department or the number of employees who work on each project.
- In such cases, we need to partition the relation into non overlapping subsets(or groups)of tuples.
- The attributes used to partition a relation into groups are called **grouping attributes**. All tuples that have the same values for all grouping attributes are placed in the same group.
- **If NULLs exist in the grouping attribute, then a separate group is created for all tuples with a NULL value in the grouping attribute.**
- Aggregate functions can be applied independently to each group to produce summary information about each group.

Group By clause

- Grouping can be done in SQL using the GROUP BY clause.
- The GROUP BY clause specifies the grouping attributes. They should be present in the SELECT clause as well so that the value of the grouping attributes will appear in the result relation along with the aggregated values.

Example: **For each department, retrieve the department number, the number of employees in the department, and their average salary.**

Query: SELECT Dno, COUNT(*), AVG (Salary) FROM EMPLOYEE GROUP BY Dno;

Output:

Dno	COUNT(*)	AVG (Salary)
1	1	55000.000000
4	3	31000.000000
5	4	33250.000000

Group By clause - Example

Query: SELECT Dno, COUNT(*), AVG (Salary) FROM EMPLOYEE GROUP BY Dno;

Output:

Dno	COUNT(*)	AVG (Salary)
1	1	55000.000000
4	3	31000.000000
5	4	33250.000000

- In the above query, *Dno* is the grouping attribute. The *EMPLOYEE* relation is partitioned into groups in such a way that each group has tuples with the same value of *Dno*.
- Hence, each group consists of employees who work for the same department.
- The COUNT and AVG functions are applied to each group independently and are displayed in the result relation.
- The SELECT clause must contain only the grouping attributes and aggregate functions applied on each group of tuples.

Database Management Systems

Group By clause – How it works

An illustration of how grouping works is provided for the previous example

Fname	Minit	Lname	Ssn	...	Salary	Super_ssn	Dno
John	B	Smith	123456789		30000	333445555	5
Franklin	T	Wong	333445555		40000	888665555	5
Ramesh	K	Narayan	666884444		38000	333445555	5
Joyce	A	English	453453453	...	25000	333445555	5
Alicia	J	Zelaya	999887777		25000	987654321	4
Jennifer	S	Wallace	987654321		43000	888665555	4
Ahmad	V	Jabbar	987987987		25000	987654321	4
James	E	Bong	888665555		55000	NULL	1

Grouping EMPLOYEE tuples by the value of Dno

Dno	Count (*)	Avg (Salary)
5	4	33250
4	3	31000
1	1	55000

Result of Q24

Database Management Systems

Group By clause along with Join

Example: **For each project, retrieve the project number, the project name, and the number of employees who work on that project.**

Query: SELECT Pnumber, Pname, COUNT(*) FROM PROJECT, WORKS_ON

WHERE Pnumber = Pno GROUP BY Pnumber, Pname;

Output:

Pnumber	Pname	COUNT(*)
10	Computerization	3
30	Newbenefits	3
1	ProductX	2
2	ProductY	3
3	ProductZ	2
20	Reorganization	3

- The above query is an example of using GROUP BY along with JOIN.
- In such statements, relations are first joined using the condition specified in the WHERE clause. Grouping and aggregation are then performed on the result of this joining

Having clause

- There are some scenarios wherein we would want to retrieve the values of aggregate functions for groups that satisfy certain conditions.
- SQL provides the HAVING clause, which can appear in conjunction with a GROUP BY clause, for this purpose.
- **The HAVING clause is used to specify conditions for groups of tuples, whereas the WHERE clause is used to specify conditions for individual tuples.**
- HAVING provides a condition on the summary information regarding the group of tuples associated with each value of the grouping attributes.
- **Only the groups that satisfy the condition are retrieved in the result of the query.**

Having clause - Example

Example:

For each project on which more than two employees work, retrieve the project number, the project name, and the number of employees who work on the project.

Query:

```
SELECT Pnumber, Pname, COUNT(*) FROM PROJECT, WORKS_ON
WHERE Pnumber = Pno GROUP BY Pnumber, Pname HAVING COUNT(*) > 2;
```

Output:

Pnumber	Pname	COUNT(*)
10	Computerization	3
30	Newbenefits	3
2	ProductY	3
20	Reorganization	3

Database Management Systems

Having clause – How it works

An illustration of how the HAVING clause works is provided for the previous example

Pname	Pnumber	...	Essn	Pno	Hours
ProductX	1		123456789	1	32.5
ProductX	1		453453453	1	20.0
ProductY	2		123456789	2	7.5
ProductY	2		453453453	2	20.0
ProductY	2		333445555	2	10.0
ProductZ	3		666884444	3	40.0
ProductZ	3		333445555	3	10.0
Computerization	10	...	333445555	10	10.0
Computerization	10		999887777	10	10.0
Computerization	10		987987987	10	35.0
Reorganization	20		333445555	20	10.0
Reorganization	20		987654321	20	15.0
Reorganization	20		888665555	20	NULL
Newbenefits	30		987987987	30	5.0
Newbenefits	30		987654321	30	20.0
Newbenefits	30		999887777	30	30.0

These groups are not selected by the HAVING condition of Q26.

After applying the WHERE clause but before applying HAVING

Pname	Pnumber	...	Essn	Pno	Hours
ProductY	2		123456789	2	7.5
ProductY	2		453453453	2	20.0
ProductY	2		333445555	2	10.0
Computerization	10		333445555	10	10.0
Computerization	10	...	999887777	10	10.0
Computerization	10		987987987	10	35.0
Reorganization	20		333445555	20	10.0
Reorganization	20		987654321	20	15.0
Reorganization	20		888665555	20	NULL
Newbenefits	30		987987987	30	5.0
Newbenefits	30		987654321	30	20.0
Newbenefits	30		999887777	30	30.0

After applying the HAVING clause condition

Pname	Count (*)
ProductY	3
Computerization	3
Reorganization	3
Newbenefits	3

Result of Q26
(Pnumber not shown)

Group By clause - Example

Example: **For each project, retrieve the project number, the project name, and the number of employees from department 5 who work on that project.**

Query: SELECT Pnumber, Pname, COUNT (*)

FROM PROJECT, WORKS_ON, EMPLOYEE

WHERE Pnumber = Pno AND Dno = 5 AND Ssn=Essn

GROUP BY Pnumber, Pname;

Output:

Pnumber	Pname	COUNT(*)
1	ProductX	2
2	ProductY	3
3	ProductZ	2
10	Computerization	1
20	Reorganization	1

Where predicate and Having predicate

- We must be extra careful when two different conditions apply – one specified in the WHERE clause and the other specified in the HAVING clause.
- **The rule is that the condition specified in the WHERE clause is executed first, to select the individual tuples, and then the HAVING clause is executed, to select groups of tuples.**
- **This means that the HAVING clause groups only those tuples that are selected by the WHERE clause.**
- Consider the following example:

Retrieve the count of number of employees whose salary exceeds $40,000 in each department, but only for those departments where more than two employees work.

Where predicate and Having predicate

Query: SELECT Dno, COUNT(*) FROM EMPLOYEE

WHERE Salary>40000 GROUP BY Dno HAVING COUNT(*) > 2;

- At the outlook, this might seem like the correct query for the given question. However, closer inspection reveals that this might not be the case.
- As stated earlier, the WHERE clause is executed first and its result is what the HAVING clause works on.
- In other words, the WHERE clause first returns only those employees whose salary exceeds $40,000. This is followed by the grouping of the filtered relation and the selection of groups by the HAVING clause condition.
- Hence, only departments with more than two employees who each earn more than $40,000 are selected, which is not the expected result.

Where predicate and Having predicate

The correct query is given below

Query:
```
SELECT Dno, COUNT(*) FROM EMPLOYEE
    WHERE Salary>40000 AND Dno IN
    ( SELECT Dno FROM EMPLOYEE GROUP BY Dno HAVING COUNT(*) > 2)
    GROUP BY Dno;
```

Output:

Dno	COUNT(*)
4	1

This shows that there is only one employee with a salary greater than $40,000 belonging to a department(here department 4) with more than two employees. All other departments either have two or fewer employees or don't have employees whose salary exceed $40,000.

Where predicate and Having predicate

Let's try executing the incorrect query

Query: SELECT Dno, COUNT(*) FROM EMPLOYEE

WHERE Salary>40000 GROUP BY Dno HAVING COUNT(*) > 2;

Output:

Dno	COUNT(*)

- The output of the query is an empty relation. This is because the query returns only those departments with more than two employees whose salaries exceed $40,000.
- From the output of the correct query, we can see that there are no departments that satisfy this condition. Hence, an empty relation is returned.

Database Management Systems

Difference between Where and Having predicate

PES UNIVERSITY

Aspect	WHERE	HAVING
Purpose	Filters **rows** before grouping/aggregation	Filters **groups/aggregated results** after GROUP BY
When is it Applied	Applied **before** grouping	Applied **after** grouping
Works On	Works on **individual row values** (columns)	Works on **aggregated values** (like sum(), count() etc)
Use of Aggregates	Cannot use aggregate functions.	Can use aggregate functions.
Order of execution	First **WHERE** is executed then subsequent statements	HAVING is executed **after** WHERE has been applied

THANK YOU

Dr. Nagasundari S
Department of Computer Science and Engineering

nagasundaris@pes.edu

Database Management Systems

Join expressions

Dr. Nagasundari S

Department of Computer Science and Engineering

PES UNIVERSITY

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

Join expressions

- Join operations take two relations and return as a result another relation.
- A join operation is a Cartesian product that requires tuples in the two relations to match (under some conditions).
- It also specifies the attributes that are present in the result of the join
- The join operations are typically used as subquery expressions in the **FROM** clause

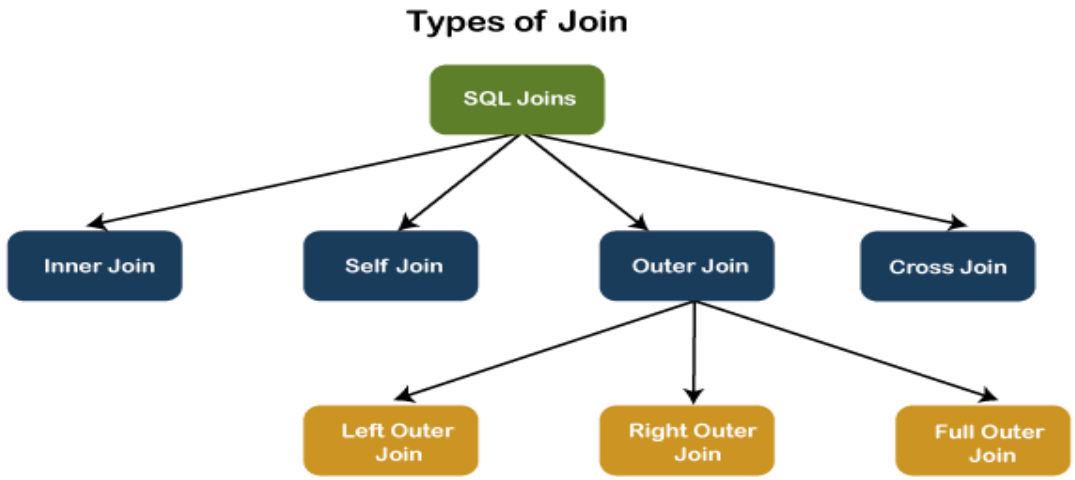
Three types of joins:

Natural join

Inner join

Outer join

Cross join



Natural Join

- **A natural join is a type of join operation that creates an implicit join by combining tables based on columns with the same name and data type.**
- In a NATURAL JOIN on two relations R and S, no join condition is specified.
- An implicit EQUIJOIN condition for each pair of attributes with the same name from R and S is created.
- Each such pair of attributes is included only once in the resulting relation.
- If the names of the join attributes are not the same in the base relations, it is possible to rename the attributes so that they match, and then to apply NATURAL JOIN.

Natural Join

Points to remember:

- There is no need to specify the column names for natural join.
- The resultant table always contains unique columns.
- It is possible to perform a natural join on more than two tables.
- Do not use the ON clause.

The following is a basic syntax to illustrate the natural join:

```
SELECT [column_names | *]
FROM table_name1
NATURAL JOIN table_name2;
```

Database Management Systems

Natural Join

PES UNIVERSITY

Employee Table:

Employee_ID	employee_name	employee_salary
1	Arun Tiwari	50000
2	Sachin Rathi	64000
3	Harshal Pathak	48000
4	Arjun Kuwar	46000
5	Sarthak Gada	62000
6	Saurabh Sheik	53000
7	Shubham Singh	29000
8	Shivam Dixit	54000
9	Vicky Gujral	39000
10	Vijay Bose	28000
NULL	NULL	NULL

Department	Department_name	Employee_ID
1	Production	1
2	Sales	3
3	Marketing	4
4	Accounts	5
5	Development	7
6	HR	9
7	Sales	10
NULL	NULL	NULL

Database Management Systems

Natural Join

Loan:

loan_id	branch	amount
1	B1	15000
2	B2	10000
3	B3	20000
4	B4	100000
5	B5	150000
6	B6	50000
7	B7	35000
8	B8	85000
NULL	NULL	NULL

CustID	CustName	loan_id
1	Sonakshi Dixit	1
2	Shital Garg	4
3	Swara Joshi	5
4	Isha Deshmukh	2
5	Swati Bose	7
6	Asha Kapoor	10
7	Nandini Shah	9
NULL	NULL	NULL

Natural Join

Example: List the employees along with their department names.

Query: SELECT employee_name, employee_salary,Department_name

FROM employee NATURAL JOIN Department;

Output:

employee_name	employee_salary	Department_name
Arun Tiwari	50000	Production
Harshal Pathak	48000	Sales
Arjun Kuwar	46000	Marketing
Sarthak Gada	62000	Accounts
Shubham Singh	29000	Development
Vicky Gujral	39000	HR
Vijay Bose	28000	Sales

Database Management Systems

Natural Join

Example: List the customers along with their loan amount and address.

Query: SELECT CustName, CustID, amount, mobile ,address

FROM Borrower NATURAL JOIN loan NATURAL JOIN cust_info ;

Output:

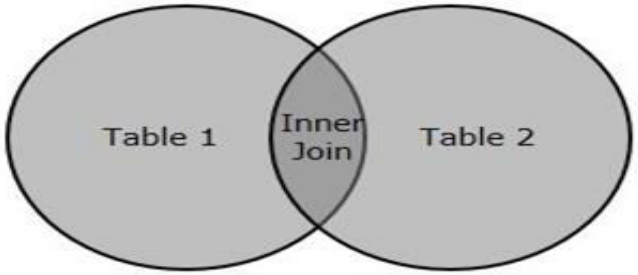
CustName	CustID	amount	mobile	address
Sonakshi Dixit	1	15000	598675498654	777 Brockton Avenue, Abington MA 251
Shital Garg	2	100000	698853747888	337 Russell St, Hadley MA 103
Swara Joshi	3	150000	234456977555	20 Soojian Dr, Leicester MA 154
Isha Deshmukh	4	10000	987656789666	780 Lynnway, Lynn MA 19
Swati Bose	5	35000	756489372222	700 Oak Street, Brockton MA 23

Customer info table:

CustID	mobile	address
1	598675498654	777 Brockton Avenue, Abington MA 251
2	698853747888	337 Russell St, Hadley MA 103
3	234456977555	20 Soojian Dr, Leicester MA 154
4	987656789666	780 Lynnway, Lynn MA 19
5	756489372222	700 Oak Street, Brockton MA 23

Inner Join

- SQL Inner Join is a type of join that is used to combine records from two related tables, based and select the rows depending on the condition (join predicate)
- Inner Join query compares each row of the first table with each row of the second table to find all pairs of rows that satisfy the join-predicate.
- When the join-predicate is satisfied, column values for each matched pair of rows of both tables are combined into a result row.



- The Inner Join is a default join; i.e., even if the “Join” keyword is used instead of “Inner Join”, tables are joined using matching records of common columns, by default.
- The most important and frequently used join in SQL is the Inner Join. It is also referred as Equijoin.

Inner Join

Example : Write a query to perform inner join considering employee table and department table.

Query:

```
SELECT e.Employee_ID, e.Employee_Name, e.Employee_Salary, d.DepartmentID, d.Department_Name FROM employee e INNER JOIN department d ON e.Employee_ID = d.Employee_ID;
```

Output:

Employee_ID	Employee_Name	Employee_Salary	DepartmentID	Department_Name
1	Arun Tiwari	50000	1	Production
3	Harshal Pathak	48000	2	Sales
4	Arjun Kuwar	46000	3	Marketing
5	Sarthak Gada	62000	4	Accounts
7	Shubham Singh	29000	5	Development
9	Vicky Gujral	39000	6	HR
10	Vijay Bose	28000	7	Sales

Inner Join

Example Write a query to perform inner join considering loan table and borrower table.

Query:

```
SELECT l.Loan_ID, l.Branch, l.Amount, b.CustID, b.CustName FROM Loan l JOIN
Borrower b ON l.Loan_ID = b.Loan_ID;
```

Output:

Loan_ID	Branch	Amount	CustID	CustName
1	B1	15000	1	Sonakshi Dixit
4	B4	100000	2	Shital Garg
5	B5	150000	3	Swara Joshi
2	B2	10000	4	Isha Deshmukh
7	B7	35000	5	Swati Bose

THANK YOU

Dr. Nagasundari S
Department of Computer Science and Engineering

nagasundaris@pes.edu

Database Management Systems

Join expressions

Dr. Nagasundari S

Department of Computer Science and Engineering

PES UNIVERSITY

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

Outer Join

- An extension of the join operation that avoids loss of information in few cases.
- Computes the join and then adds tuples from one relation that does not match tuples in the other relation to the result of the join.
- In the SQL outer JOIN, all the content from both the tables is integrated together.
- Even though the records from both the tables are matched or not, the matching and non-matching records from both the tables will be considered an output of the outer join in SQL.

There are three different types of outer join in SQL:

Left Outer Join

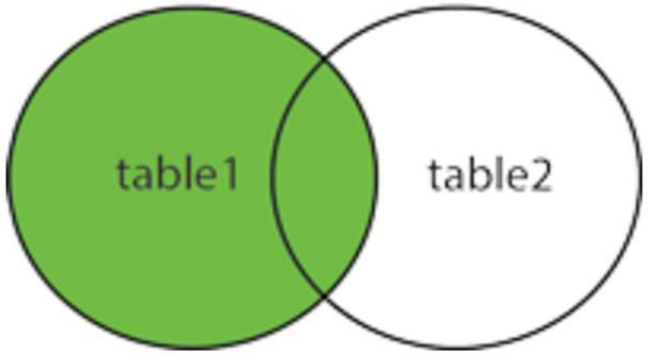
Right Outer Join

Full Outer Join

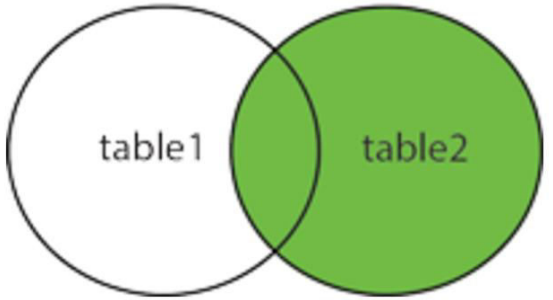
Database Management Systems

Outer Join

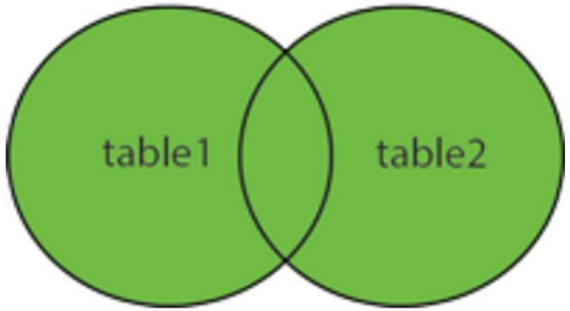
LEFT JOIN



RIGHT JOIN



FULL OUTER JOIN

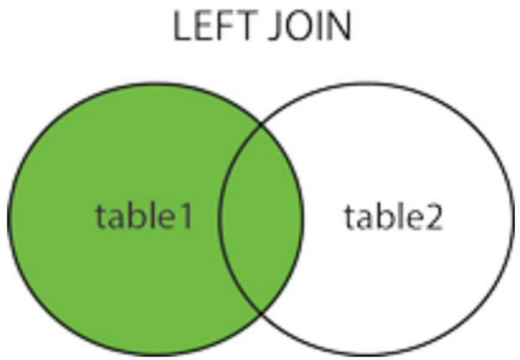


LEFT Outer Join

- In LEFT OUTER JOIN every tuple in the left table must appear in the result. If it does not have a matching tuple, it is padded with NULL values for the attributes of the right table.
- If we use the left outer join to combine two different tables, then we will get all the records from the left table.
- But we will get only those records from the right table, which have the corresponding key in the left table ie the matching tuple

Syntax:

```
SELECT TableName1.columnName1, TableName2.columnName2
FROM TableName1 LEFT OUTER JOIN TableName2 ON
TableName1.ColumnName = TableName2.ColumnName;
```



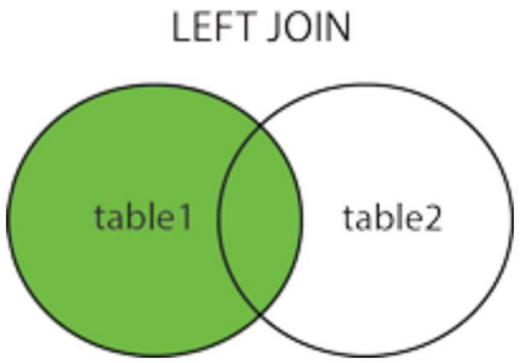
LEFT Outer Join

Example : Write a query to perform left outer join considering employee table as the left table and department table as the right table.

Query:

```
SELECT e.Employee_ID, e.Employee_Name, e.Employee_Salary, d.DepartmentID, d.Department_Name FROM employee e LEFT OUTER JOIN department d ON e.Employee_ID = d.Employee_ID;
```

Output:



Employee_ID	Employee_Name	Employee_Salary	DepartmentID	Department_Name
1	Arun Tiwari	50000	1	Production
2	Sachin Rathi	64000	NULL	NULL
3	Harshal Pathak	48000	2	Sales
4	Arjun Kuwar	46000	3	Marketing
5	Sarthak Gada	62000	4	Accounts
6	Saurabh Sheik	53000	NULL	NULL
7	Shubham Singh	29000	5	Development
8	Shivam Dixit	54000	NULL	NULL
9	Vicky Gujral	39000	6	HR
10	Vijay Bose	28000	7	Sales

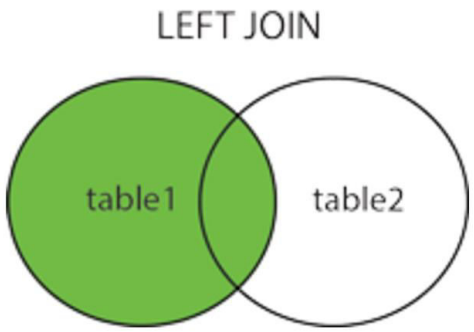
LEFT Outer Join

Example Write a query to perform left outer join considering loan table as the left table and borrower table as the right table.

Query:

```
SELECT L.Loan_ID, L.Branch, L.Amount, B.CustID, B.CustName FROM Loan L LEFT OUTER JOIN Borrower B ON L.Loan_ID = B.Loan_ID;
```

Output:



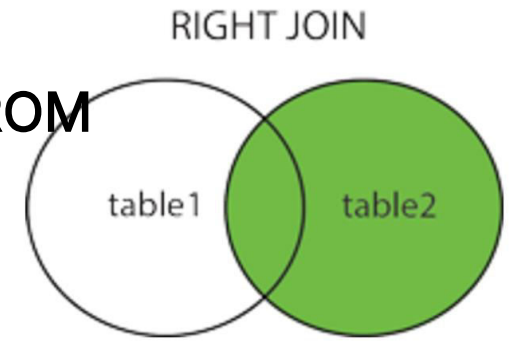
Loan_ID	Branch	Amount	CustID	CustName
1	B1	15000	1	Sonakshi Dixit
2	B2	10000	4	Isha Deshmukh
3	B3	20000	NULL	NULL
4	B4	100000	2	Shital Garg
5	B5	150000	3	Swara Joshi
6	B6	50000	NULL	NULL
7	B7	35000	5	Swati Bose
8	B8	85000	NULL	NULL

Right Outer Join

- In RIGHT OUTER JOIN every tuple in the right table must appear in the result. If it does not have a matching tuple, it is padded with NULL values for the attributes of the left table. Right outer join is the reverse of the left outer join.
- If we use the right outer join to combine two different tables, then we will get all the records from the right table.
- But we will get only those records from the left table, which have the corresponding key in the right table.

Syntax :

SELECT TableName1.columnName1, TableName2.columnName2 **FROM** TableName1 RIGHT OUTER JOIN TableName2 **ON** TableName1.ColumnName = TableName2.ColumnName;



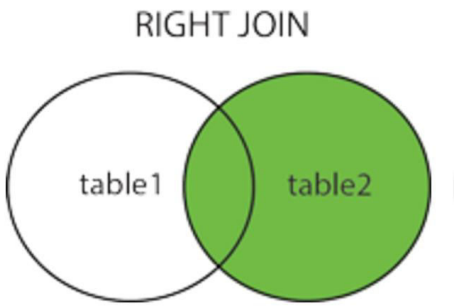
Right Outer Join

Example:

Write a query to perform right outer join considering employee table as the left table and department table as the right table.

Query:

```
SELECT e.Employee_ID, e.Employee_Name, e.Employee_Salary, d.DepartmentID, d.Department_Name FROM employee e RIGHT OUTER JOIN department d ON e.Employee_ID = d.Employee_ID;
```



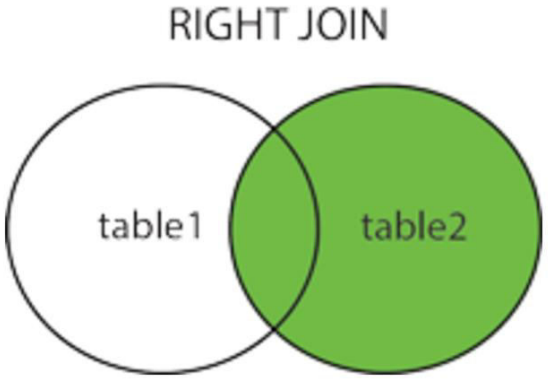
Output:

Employee_ID	Employee_Name	Employee_Salary	DepartmentID	Department_Name
1	Arun Tiwari	50000	1	Production
3	Harshal Pathak	48000	2	Sales
4	Arjun Kuwar	46000	3	Marketing
5	Sarthak Gada	62000	4	Accounts
7	Shubham Singh	29000	5	Development
9	Vicky Gujral	39000	6	HR
10	Vijay Bose	28000	7	Sales

Right Outer Join

Example:

Write a query to perform right outer join considering the loan table as the left table and the borrower table as the right table.



Query:

```
SELECT l.Loan_ID, l.Branch, l.Amount, b.CustID, b.CustName
FROM Loan l RIGHT OUTER JOIN Borrower b ON l.Loan_ID = b.Loan_ID;
```

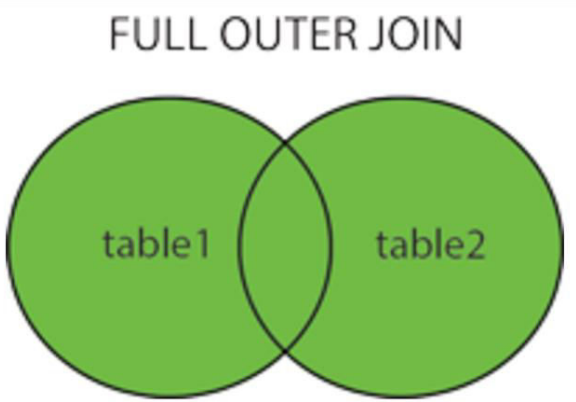
Output:

Loan_ID	Branch	Amount	CustID	CustName
1	B1	15000	1	Sonakshi Dixit
4	B4	100000	2	Shital Garg
5	B5	150000	3	Swara Joshi
2	B2	10000	4	Isha Deshmukh
7	B7	35000	5	Swati Bose
NULL	NULL	NULL	6	Asha Kapoor
NULL	NULL	NULL	7	Nandini Shah

Database Management Systems

Full Outer Join

- If we use a full outer join to combine two different tables, then we will get all the records from both tables.
- We will get all the records from the left table as well as the right table.
- **MySQL doesn't support FULL OUTER JOIN directly**.
- So to implement full outer join in MySQL, we will execute two queries in a single query.
- The first query will be of LEFT OUTER JOIN, and the second query will be of RIGHT OUTER JOIN.
- We will combine the first and the second query with the UNION operator to see the results of FULL OUTER JOIN.



Database Management Systems

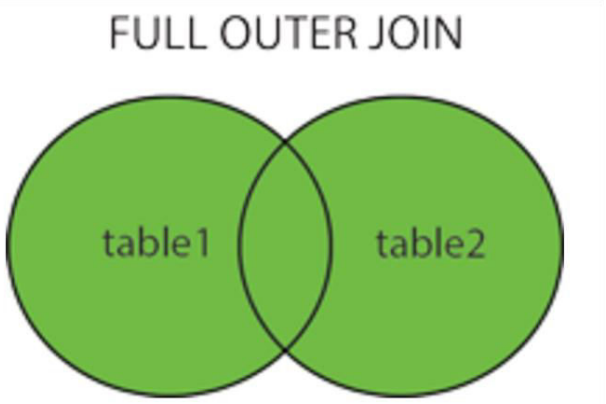
Full Outer Join

PES UNIVERSITY

Example 1:

Write a query to perform full outer join considering the employee table as the left table and department table as the right table.

Query: SELECT e.Employee_ID, e.Employee_Name, e.Employee_Salary, d.DepartmentID, d.Department_Name FROM department d LEFT OUTER JOIN employee e ON e.Employee_ID = d.Employee_ID UNION SELECT e.Employee_ID, e.Employee_Name, e.Employee_Salary, d.DepartmentID, d.Department_Name FROM department d RIGHT OUTER JOIN employee e ON e.Employee_ID = d.Employee_ID;



Output:

Employee_ID	Employee_Name	Employee_Salary	DepartmentID	Department_Name
1	Arun Tiwari	50000	1	Production
3	Harshal Pathak	48000	2	Sales
4	Arjun Kuwar	46000	3	Marketing
5	Sarthak Gada	62000	4	Accounts
7	Shubham Singh	29000	5	Development
9	Vicky Gujral	39000	6	HR
10	Vijay Bose	28000	7	Sales
2	Sachin Rathi	64000	NULL	NULL
6	Saurabh Sheik	53000	NULL	NULL
8	Shivam Dixit	54000	NULL	NULL

Full Outer Join

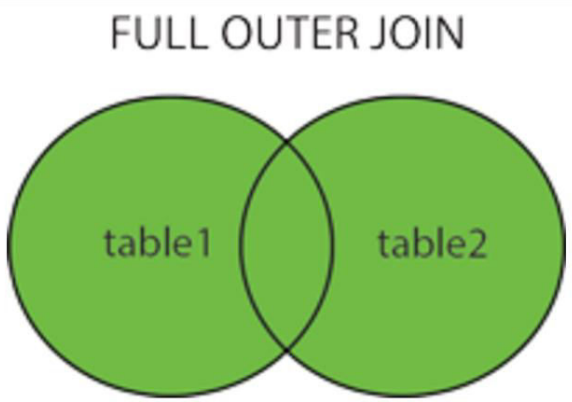
Example 1:

Write a query to perform full outer join considering loan table as the left table and borrower table as the right table.

Query: SELECT l.Loan_ID, l.Branch, l.Amount, b.CustID, b.CustName FROM Loan l LEFT OUTER JOIN Borrower b ON l.Loan_ID = b.Loan_ID UNION SELECT l.Loan_ID, l.Branch, l.Amount, b.CustID, b.CustName FROM Loan l RIGHT OUTER JOIN Borrower b ON l.Loan_ID = b.Loan_ID;

Output:

Loan_ID	Branch	Amount	CustID	CustName
1	B1	15000	1	Sonakshi Dixit
2	B2	10000	4	Isha Deshmukh
3	B3	20000	NULL	NULL
4	B4	100000	2	Shital Garg
5	B5	150000	3	Swara Joshi
6	B6	50000	NULL	NULL
7	B7	35000	5	Swati Bose
8	B8	85000	NULL	NULL
NULL	NULL	NULL	6	Asha Kapoor
NULL	NULL	NULL	7	Nandini Shah



Cross Join

- Cross Join returns the Cartesian Product
- It does not use a join condition (ON ...), so it doesn't care about matching columns.
- **For example,** consider the tables loan (having 8 rows) and borrower (having 7 rows)
- Rows Returned: (Rows in Table A) * (Rows in Table B) = 8*7 = 56
- **Query: select * from loan cross join borrower;**

```
mysql> select *
    -> from loan
    -> cross join borrower;
+---------+--------+--------+--------+---------------+---------+
| loan_id | branch | amount | CustID | CustName      | loan_id |
+---------+--------+--------+--------+---------------+---------+
|       1 | B1     |  15000 |      7 | Nandini Shah  |       9 |
|       1 | B1     |  15000 |      6 | Asha Kapoor   |      10 |
|       8 | B8     |  85000 |      4 | Isha Deshmukh |       2 |
|       8 | B8     |  85000 |      3 | Swara Joshi   |       5 |
|       8 | B8     |  85000 |      2 | Shital Garg   |       4 |
|       8 | B8     |  85000 |      1 | Sonakshi Dixit|       1 |
+---------+--------+--------+--------+---------------+---------+
56 rows in set (0.00 sec)
```

THANK YOU

Dr. Nagasundari S
Department of Computer Science and Engineering

nagasundaris@pes.edu

Database Management Systems

Nested Queries - 1

Dr. Nagasundari S
Department of Computer Science and Engineering

PES UNIVERSITY

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

Nested queries - Introduction

PES UNIVERSITY

- A **nested query** can be defined as a **complete SELECT-FROM-WHERE block** within another SQL query. The other query is called the **outer query**.
- Nested queries are used to fetch existing values in the database that can be used by the outer query for comparison.
- **They can appear in the WHERE clause, FROM clause, SELECT clause, and other SQL clauses as needed.**
- SQL provides a comparison operator **IN**, which checks if a value is present in a set(or multiset) of values. **It returns True if the value is present in the set and False** otherwise. It essentially checks for Set membership.
- The **NOT IN** operator returns True if the value is not present in the set and False otherwise.

Nested queries - Example

Example: **Retrieve the list of project numbers of projects that have an employee with the last name ‘Smith’ involved either as a manager or as a worker.**

Query:
```
SELECT DISTINCT Pnumber
    FROM PROJECT WHERE Pnumber IN
    ( SELECT Pnumber FROM PROJECT, DEPARTMENT, EMPLOYEE
    WHERE Dnum = Dnumber AND Mgr_ssn = Ssn AND Lname = 'Smith’ )
    OR Pnumber IN
    ( SELECT Pno FROM WORKS_ON, EMPLOYEE
    WHERE Essn = Ssn AND Lname = 'Smith’ );
```

Output:

Pnumber
1
2

Nested queries – How it works

- Let's understand how this query works.
- Consider the first sub-query:

  (SELECT Pnumber FROM PROJECT, DEPARTMENT, EMPLOYEE

  WHERE Dnum = Dnumber AND Mgr_ssn = Ssn AND Lname = 'Smith')

- This query selects the project number of all projects handled by departments whose manager's last name is 'Smith'.
- On executing this subquery, we obtain an empty set. The output is shown below:

Pnumber

(Empty Set)

- This is because there is no manager whose last name is 'Smith'.

Manager_last_name
Wong
Borg
Wallace

Nested queries - How it works

Consider the second subquery:

(SELECT Pno FROM WORKS_ON, EMPLOYEE

WHERE Essn = Ssn AND Lname = 'Smith');

- This subquery selects the project number of all projects that employees with the last name 'Smith' work on.
- The output of this subquery is shown below:

Pno
1
2

Nested queries – How it works

(SELECT Pnumber FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE Dnum = Dnumber AND Mgr_ssn = Ssn AND Lname = 'Smith') ⇒ **()**

(SELECT Pno FROM WORKS_ON, EMPLOYEE
WHERE Essn = Ssn AND Lname = 'Smith'); ⇒ **(1, 2)**

By plugging in the results of the nested queries in the outer query, we obtain the equivalent query:

```
SELECT DISTINCT Pnumber
FROM PROJECT WHERE Pnumber IN
()
OR Pnumber IN
(1, 2)
```

Nested queries

- In general, a nested query **returns a table(relation)** which is a set(or multiset) of tuples.
- However, there are certain scenarios wherein a nested query returns a table with **a single attribute and a single tuple**.
- In such cases, we can treat the result as a **scalar** i.e. a single value, and apply operations applicable to singular values.
- For example, the equal operator (=) is not applicable when one of the operands is a set of values(which is usually what a nested query returns).
- **However, when a nested query returns a single value, we can use the = operator instead of the IN operator.**
- SQL allows the use of tuples of values in comparisons by placing them within parentheses.

Nested queries - Example

Example: Retrieve the ssn of all employees who work on any of the same (project, hours) combination that employee ‘John Smith’ (ssn = ‘123456789’) works on.

Query:

```
SELECT DISTINCT Essn FROM WORKS_ON
    WHERE (Pno, Hours) IN
    ( SELECT Pno, Hours FROM WORKS_ON
    WHERE Essn = '123456789’ );
```

Output:

Essn
123456789

- The IN operator compares the subtuple of values in parentheses (Pno, Hours) within each tuple in *WORKS_ON* with the set of type-compatible tuples produced by the nested query.

Additional set comparison operators

- In addition to the IN and NOT IN operators, a number of other comparison operators can be used to compare a single value to a set or multiset of values.
- These operators are of the form – ***comp_op [ANY | ALL | SOME]***
- ***comp_op*** denotes comparison operators such as =, <> (not equals), >, >=, <, <= which are used to compare two values.
- ***comp_op ALL*** returns True if the result of applying ***comp_op*** between the single value and an element of the set is True **for every element of the set.** For example, the expression 1 < ALL (2,3) evaluates as True because 1(the single value here) is lesser than(comp_op) both 2 and 3(the elements of the set operand).
- ***comp_op SOME(or ANY)*** returns True if the result of applying ***comp_op*** between the single value and an element of the set is True **for at least one element in the set.**

Additional set comp. operators - Example

Example: **Retrieve the last and first names of all employees whose salary is greater than the salary of all the employees in department 5**

Query: SELECT Lname, Fname FROM EMPLOYEE

WHERE Salary > ALL (SELECT Salary FROM EMPLOYEE WHERE Dno = 5);

Output:

Lname	Fname
Borg	James
Wallace	Jennifer

- First, the nested query gets executed. It returns the salary of all employees who work in department 5. The output is shown below:

Salary
30000.00
40000.00
25000.00
38000.00

Additional set comp. operators – How it works

- The outer query essentially becomes:

  SELECT Lname, Fname FROM EMPLOYEE

  WHERE Salary > ALL **(30000.00, 40000.00, 25000.00, 38000.00)**

- For every tuple in the *EMPLOYEE* relation, the value in the *salary* attribute is compared with the output of the nested query using > ALL. The WHERE clause evaluates to True if the value in the *salary* column for that particular tuple is greater than all the values in the nested query output. A table with the last name, first name, and salary of employees is shown to verify the result.
- **The same can be achieved using the MAX function.**

Lname	Fname	Salary
Smith	John	30000.00
Wong	Franklin	40000.00
English	Joyce	25000.00
Narayan	Ramesh	38000.00
Borg	James	55000.00
Wallace	Jennifer	43000.00
Jabbar	Ahmed	25000.00
Zelaya	Alicia	25000.00

Additional set comp. operators - Example

Example: **Retrieve the last and first names of all employees whose salary is greater than the salary of at least one employee in department 5**

Query:
```
SELECT Lname, Fname FROM EMPLOYEE
    WHERE Salary > SOME ( SELECT Salary FROM EMPLOYEE WHERE Dno = 5 );
```

Output:

Lname	Fname
Smith	John
Wong	Franklin
Narayan	Ramesh
Borg	James
Wallace	Jennifer

Notice how there are more employees who satisfy the given condition as compared to the previous example. This is because > SOME(and > ANY) evaluates as True if the employee salary in the outer query is greater than even one value in the nested query output.

Additional set comp. operators

- The previous example could be solved using the MIN function as well.
- = SOME or = ANY is equivalent to the IN operator.
- <> ALL is equivalent to the NOT IN operator.

Levels of nesting queries

- In general, we can have several levels of nested queries.
- Ambiguity among attribute names arises if attributes belonging to the relation in the FROM clause of the outer query and the relation in the FROM clause of the nested query have the same name.
- The rule is that a reference to an unqualified attribute refers to the relation declared in the innermost query
- For example, consider the query:

```
SELECT   DISTINCT Pnumber
FROM     PROJECT
WHERE    Pnumber IN
         ( SELECT    Pnumber
           FROM      PROJECT, DEPARTMENT, EMPLOYEE
           WHERE     Dnum = Dnumber AND
                     Mgr_ssn = Ssn AND Lname = 'Smith' )
         OR
         Pnumber IN
         ( SELECT    Pno
           FROM      WORKS_ON, EMPLOYEE
           WHERE     Essn = Ssn AND Lname = 'Smith' );
```

The relation PROJECT has been referenced both in the outer query and the inner query. The attributes Pnumber and Dnum in the nested query correspond to the PROJECT relation in the nested query and not the one in the outer query.

Levels of nesting queries

- **To refer to an attribute of a relation specified in the outer query, we specify and refer to an alias for that relation.**

Consider the following query

```
SELECT    E.Fname, E.Lname
FROM      EMPLOYEE AS E
WHERE     E.Ssn IN    ( SELECT    D.Essn
                        FROM      DEPENDENT AS D
                        WHERE     E.Fname = D.Dependent_name
                                  AND E.Sex = D.Sex );
```

- Here E is the alias for the relation *EMPLOYEE*. There are two attributes named *Sex* – one in the *EMPLOYEE* table and another in the *DEPENDENT* table.
- We must qualify E.Sex because it refers to the *Sex* attribute of the *EMPLOYEE* relation from the outer query, and not to the *Sex* attribute of the *DEPENDENT* relation.

Levels of nesting queries

- **If there were any unqualified references to *Sex* in the nested query, they would refer to the *Sex* attribute of the *DEPENDENT* relation.**
- However, we would not have to qualify attributes *such as Ssn* (of the *EMPLOYEE* relation) if they appeared in the nested query because the *DEPENDENT* relation does not have an attribute called *Ssn*, so there is no ambiguity.
- It is generally advisable to create tuple variables (aliases) for all the tables referenced in an SQL query to avoid potential errors and ambiguities

THANK YOU

Dr. Nagasundari S
Department of Computer Science and Engineering

nagasundaris@pes.edu

Database Management Systems

Nested Queries - 2

Dr Nagasundari and Prof Mahitha G
Department of Computer Science and Engineering

PES UNIVERSITY

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr Nagasundari and Prof Mahitha G
Department of Computer Science and Engineering

Database Management Systems

Subqueries in the From clause

PES UNIVERSITY

- SQL allows a subquery expression to be used in the FROM clause.
- Since a subquery returns a relation as a result, it can be inserted in a SELECT-FROM-WHERE expression at any place where a relation can appear.

Example: **Find the average employee salary of those departments where the average salary is greater than $32,000.**

- One way to frame the query would be to use the GROUP BY and HAVING clauses.
- We can achieve the same by using nested subqueries. First, extract department numbers and their corresponding average salaries from the *EMPLOYEE* table using the following query:

  SELECT Dno, AVG(Salary) FROM EMPLOYEE GROUP BY Dno

- The resulting relation can be then be filtered using the WHERE clause.

Using Group By & Having Clauses

Query:
```
SELECT Dno, ROUND(AVG(Salary),2) as avg_salary
    FROM EMPLOYEE
    GROUP BY Dno
    HAVING AVG(Salary) > 32000;
```

Output :

Dno	avg_salary
1	55000.00
5	33250.00

NOTE: The **ROUND**() function in MySQL is used to round a number to a specified number of decimal places. Its syntax is given by ROUND(X, D) where X is the number to be rounded and D (optional) is the number of decimal places up to which X is to be rounded. If D is not provided, X is rounded to the nearest integer and returned.

Database Management Systems

Using Nested Queries

Query: SELECT Dno, avg_salary

FROM (SELECT Dno, ROUND(AVG(Salary),2) FROM EMPLOYEE GROUP BY Dno)

AS dept_avg_salary(Dno, avg_salary)

WHERE avg_salary > 32000;

Output:

Dno	avg_salary
1	55000.00
5	33250.00

Subqueries in the From clause - How it works

SELECT Dno, ROUND(AVG(Salary),2) FROM EMPLOYEE GROUP BY Dno

- This subquery returns department numbers and corresponding average employee salaries(rounded to 2 decimal places). The output of this subquery is shown below:

Dno	avg_salary
1	55000.00
4	31000.00
5	33250.00

AS dept_avg_salary(Dno, avg_salary)

- This part of the query is used to specify the names of the result relation and its attributes. Hence, the subquery result relation is named *dept_avg_salary*, with the attributes *Dno* and *avg_salary*. Note that failing to the name this result will throw an error:

Error Code: 1248. Every derived table must have its own alias

Subqueries in the From clause - How it works

WHERE avg_salary > 32000

- The WHERE clause selects those tuples from *dept_avg_salary* whose *avg_salary* exceed 32000.

SELECT Dno, avg_salary

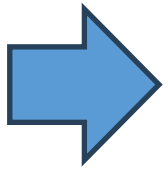
- The SELECT clause selects the specified attributes *Dno, avg_salary* from the tuples selected by the WHERE clause.

Subqueries in the From clause

- **Nested subqueries in the FROM clause are supported by most but not all SQL implementations.**
- Some SQL implementations, notably **MySQL** and **PostgreSQL**, require that each subquery relation in the FROM clause must be given a name.
- Oracle allows a subquery result relation to be given a name but does not allow renaming the attributes of the relation.
- Modifications to the written MySQL query can overcome this restriction.

MySQL

```
SELECT Dno, avg_salary
FROM
(SELECT Dno, ROUND(AVG (salary),2)
FROM EMPLOYEE
GROUP BY Dno)
AS dept_avg_salary(Dno, avg_salary)
WHERE avg_salary > 32000;
```



Oracle

```
SELECT Dno, ROUND(AVG(salary),2) AS avg_salary
FROM
(SELECT Dno, ROUND(AVG (salary),2)
FROM EMPLOYEE
GROUP BY Dno)
AS dept_avg_salary
WHERE avg_salary > 32000;
```

Subqueries in the From clause - Example

Example: Find the maximum of total employee salaries of each department across all departments.

Query:
```
SELECT MAX(tot_salary)
    FROM (SELECT Dno, SUM(Salary) FROM EMPLOYEE GROUP BY Dno)
    AS dept_total(Dno, tot_salary);
```

Output:

MAX(tot_salary)
133000.00

Database Management Systems

Common Table Expressions (CTE) - Introduction

PES UNIVERSITY

- A **Common Table Expression (CTE)** in SQL is a temporary result set that is defined and used within the execution scope of a **SELECT**, **INSERT**, **UPDATE**, or **DELETE** statement .
- CTEs are defined using the WITH clause and can be referenced multiple times within the main SQL query.

They can be useful for the following:

- Simplifying complex queries
- Breaking down subqueries
- Recursive querying

Common Table Expressions (CTE) - Introduction

PES UNIVERSITY

Basic Syntax of a CTE (with clause–recap)

WITH **cte_name** AS (SELECT **query1……**)
SELECT **query2**...FROM cte_name;

- **cte_name** is the name of the CTE
- **query1** is a valid SQL query that returns a result set, which will be treated as a virtual table within the main query.
- **query2** is an outer query which uses CTE like a table

Can use **multiple CTEs** separated by commas

```
WITH cte1 AS (...), cte2 AS (...)
SELECT ...
FROM cte1 JOIN cte2;
```

Common Table Expressions (CTE) Example 1

Simple example of CTE considering Company Database

```
WITH cte_emp AS (
    SELECT ssn, fname, dno, salary
    FROM Employee
    WHERE salary > 50000    -- filter applied here
)
SELECT fname FROM cte_emp; –use of cte
```

Common Table Expressions (CTE) Example 2

Retrieve name, dno, salary of all Employees earning above average salary

```
WITH avg_salary AS (
    SELECT AVG(salary) AS avg_sal
    FROM Employee
)
SELECT fname, dno, salary
FROM Employee, avg_salary
WHERE Employee.salary > avg_salary.avg_sal;
```

Query without using "with"

```
select fname,dno,salary from employee where salary>(select
avg(salary) from employee);
```

Common Table Expressions (CTE) Example 3

Get employees with salary > 40,000 along with their department name (without CTE)

```
select fname,salary,dname from employee join department on dno=dnumber where
salary>40000;
```

Get employees with salary > 40,000 along with their department name using CTE

```
WITH cte_emp AS (
    SELECT ssn, fname, dno, salary
    FROM Employee
    WHERE salary > 40000
),
cte_dept AS (
    SELECT dnumber, dname
    FROM department
)
SELECT e.fname, e.salary, d.dname
FROM cte_emp e JOIN cte_dept d ON e.dno = d.dnumber;
```

With clause

- The WITH clause provides a way of defining a temporary relation whose definition is available only to the query in which the WITH clause occurs.

Example: **Find the ssn of the employee who works for the highest number of hours on a particular project. Display the project number and the number of hours the employee has worked as well**

Query:
```
WITH max_work(max_hours) AS
        (SELECT MAX(Hours) FROM WORKS_ON)
        SELECT Essn, Pno, Hours
        FROM WORKS_ON, max_work
        WHERE Hours = max_hours;
```

Output:

Essn	Pno	Hours
666884444	3	40.0

With clause – How it works

- The WITH clause in the query defines the temporary relation *max_work*, which is a single tuple-single attribute relation holding the maximum hours that an employee works. The contents of *max_work* are as shown :

max_hours
40.0

- The relation *max_work* is used in the main query. A join is performed between the *WORKS_ON* and *max_work* relations on the condition that the *Hours* value matches *max_hours*. All tuples in the *WORKS_ON* relation with the maximum value of *Hours* get selected(here there is only one such tuple).
- The *Essn, Pno,* and *Hours* values of the selected tuples are retrieved.

With clause

- Note that the same result could be achieved using nested queries.

Query: SELECT Essn, Pno, Hours

FROM WORKS_ON

WHERE Hours = (SELECT MAX(Hours) FROM WORKS_ON);

Output:

Essn	Pno	Hours
666884444	3	40.0

- However, using nested subqueries would have made the query harder to read and understand.
- The WITH clause makes the query logic clearer. It also permits this temporary relation to be used in multiple places within a query.

With clause - Example

Example: **Find all departments where the total salary is greater than the average of the total salary at all departments.**

Query:
```
WITH dept_total(Dno,tot_salary)
        AS(SELECT Dno, SUM(Salary) FROM EMPLOYEE GROUP BY Dno),
dept_total_avg(avg_tot_salary) AS (SELECT AVG(tot_salary) FROM dept_total)
        SELECT Dname, Dnumber FROM dept_total, dept_total_avg, DEPARTMENT
        WHERE tot_salary > avg_tot_salary AND Dnumber = Dno;
```

Output:

Dname	Dnumber
Research	5

With clause – How it works

dept_total(Dno,tot_salary) AS(SELECT Dno, sum(Salary)

FROM EMPLOYEE GROUP BY Dno)

- This part of the query retrieves department number and total salary for each department and the result is named *dept_total.* It would look like this:

Dno	tot_salary
1	55000.00
4	93000.00
5	133000.00

dept_total_avg(avg_tot_salary) AS (SELECT AVG(tot_salary)FROM dept_total)

- This part of the query retrieves the average of total salaries in all departments by making use of *dept_total* which was defined previously. The result is named *dept_total_avg*.

With clause – How it works

- *dept_total* and *dept_total_avg* are then joined on the condition that *tot_salary* is greater than *avg_tot_salary*.
- The result of the previous join is joined with *DEPARTMENT* to obtain the department name, which is not available in the result of the join between *dept_total* and *dept_total_avg.*

THANK YOU

Dr Nagasundari and Prof Mahitha G

nagasundaris@pes.edu and mahithag@pes.edu

Database Management Systems

Common Table Expressions - CTE

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgment: Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof Mahitha G
Department of Computer Science and Engineering

PES UNIVERSITY

Recursive CTEs

- A recursive CTE is one that references itself within that CTE.
- The recursive **CTE** is useful when working with hierarchical data as the CTE continues to execute until the query returns the entire hierarchy.
- A typical example of hierarchical data is a table that includes a list of **employees** like in the company database employee has recursive relationship, which represents the employee–supervisor relationship.
- In SQL, recursive queries are typically implemented using Common Table Expressions (CTEs) with the WITH RECURSIVE clause.

Database Management Systems

Common Table Expressions (CTE) - Recursive CTE

Here's the general syntax for creating a recursive query

```
WITH RECURSIVE cte_name (column_list) AS (
  -- Anchor member: The initial query that forms the base case
  SELECT ...
  FROM ...
  WHERE …
UNION ALL
  -- Recursive member: The query that refers to the CTE itself
  SELECT ...
  FROM cte_name  -- Refer to the CTE within itself
  WHERE ...
)-- The final SELECT statement outside the CTE to retrieve the result
```

Common Table Expressions (CTE) - Recursive CTE

Here's an explanation of each part of the syntax:

- **cte_name:** This is the name you give to the Common Table Expression (CTE), which you can use to refer to it within the CTE itself and in the final SELECT statement.
- **column_list:** Specify the list of columns you want to include in the CTE's result set.
- **Anchor member:** This is the initial query that forms the base case of the recursion. It selects the starting point or points for the recursive process. It's important to ensure that this query doesn't reference the CTE itself.

Common Table Expressions (CTE) - Recursive CTE

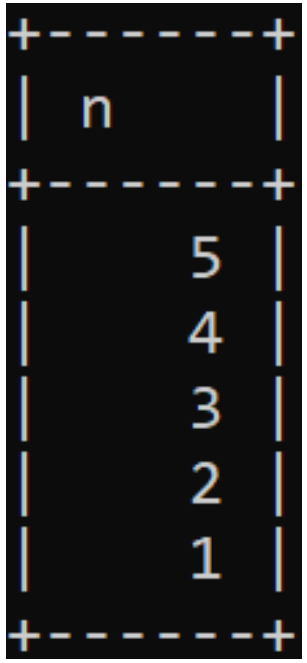
Here's an explanation of each part of the syntax:

- **UNION ALL:** This is used to combine the results of the anchor member with the results of the recursive member. UNION ALL is used because it includes duplicate rows, which is often necessary in recursive queries.
- **Recursive member:** This is the query that refers to the CTE itself, creating the recursion. It selects records based on relationships established in previous iterations. This query must reference the CTE by its name and should include a termination condition to stop the recursion.
- **Final SELECT statement:** This query retrieves the final result set from the CTE. It typically follows the recursive part of the CTE definition.

Common Table Expressions (CTE) - Recursive CTE

Example: Given an example of a recursive query that counts down from 5 to 1:

```
WITH RECURSIVE Countdown (n) AS (
  -- Anchor member
  SELECT 5
  UNION ALL
  -- Recursive member
  SELECT n - 1
  FROM Countdown
  WHERE n > 1
)
-- Final SELECT statement
SELECT n FROM Countdown;
```

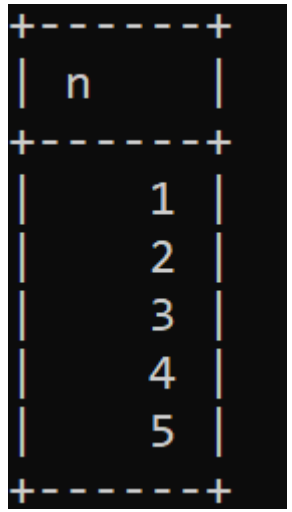


n
5
4
3
2
1

Common Table Expressions (CTE) - Recursive CTE

Example: Given an example of a recursive query that counts down from 1 to 5:

```
WITH RECURSIVE cte(n) AS (
    -- Anchor part of the query
    SELECT 1
    UNION ALL
    -- Recursive part of the query
    SELECT n + 1 FROM cte WHERE n < 5
)
SELECT * FROM cte;
```



n
1
2
3
4
5

Common Table Expressions (CTE) - Recursive CTE Example

PES UNIVERSITY

Execution Flow:

A **recursive CTE** has two parts:

1. **Anchor member** → the initial result set
2. **Recursive member** → repeatedly applied until it stops producing rows

The database engine runs it in an iterative fashion.

Run the anchor query first: SELECT 1

This produces the first row: 1.

Then, Apply the recursive part - SQL substitutes the current contents of cte into:

SELECT n + 1 FROM cte WHERE n < 5

From (1), since 1 < 5, it generates: 2 which is added to the result.

Common Table Expressions (CTE) - Example for recursive

Give a query to extract the organizational hierarchy beginning with a specific employee, typically the CEO, and extending downward through the reporting structure.

```
with recursive orghierarchy as(
 select ssn,fname,lname,super_ssn, 1 as level
 from employee
 where super_ssn is null
 union all
 select e.ssn,e.fname,e.lname,e.super_ssn, OH.level+1
 from employee e
 join orghierarchy OH on e.super_ssn=OH.ssn
 )
 select * from orghierarchy;
```

Common Table Expressions (CTE) - Example for recursive

PES UNIVERSITY

Output:

```
+-----------+----------+---------+-----------+-------+
| ssn       | fname    | lname   | super_ssn | level |
+-----------+----------+---------+-----------+-------+
| 888665555 | James    | Borg    | NULL      |     1 |
| 123456789 | John     | Smith   | 888665555 |     2 |
| 333445555 | Franklin | Wong    | 888665555 |     2 |
| 453453453 | Joyce    | English | 333445555 |     3 |
| 666884444 | Ramesh   | Narayan | 333445555 |     3 |
| 987654321 | Jennifer | Wallace | 333445555 |     3 |
| 999887777 | Alicia   | Zelaya  | 333445555 |     3 |
| 987987987 | Ahmed    | Jabbar  | 987654321 |     4 |
+-----------+----------+---------+-----------+-------+
8 rows in set (0.04 sec)
```

Database Management Systems

Case Statement in SQL

The **CASE** statement in **SQL** is a versatile conditional expression that enables us to incorporate **conditional logic** directly within our queries. It allows you to return specific results based on certain conditions, enabling dynamic query outputs.

- It is commonly used to create new columns based on conditional logic, provide custom values, or control query outputs based on certain conditions.
- If no condition is true then the ELSE part will be executed. If there is no ELSE part then it returns NULL.

Syntax:

```
CASE case_value
WHEN condition THEN result1
WHEN condition THEN result2
...
Else result
END CASE;
```

Database Management Systems

Switch Case in SQL

Write an SQL query to display the first name and salary of each employee, along with a derived column that categorizes employees into 'High', 'Medium', or 'Low' salary levels based on the following conditions:

- **Salary >= 55000 THEN 'High'**
- **Salary >= 40,000 THEN 'Medium' else low**

Query:SELECT Fname, Salary,

```
CASE
    WHEN Salary >= 55000 THEN 'High'
    WHEN Salary >= 40000 THEN 'Medium'
    ELSE 'Low'
END AS Salary_Level
```

```
+----------+----------+--------------+
| Fname    | Salary   | Salary_Level |
+----------+----------+--------------+
| John     | 30000.00 | Low          |
| Franklin | 40000.00 | Medium       |
| Joyce    | 25000.00 | Low          |
| Ramesh   | 38000.00 | Low          |
| James    | 55000.00 | High         |
| Jennifer | 43000.00 | Medium       |
| Ahmed    | 25000.00 | Low          |
| Alicia   | 25000.00 | Low          |
+----------+----------+--------------+
8 rows in set (0.00 sec)
```

Database Management Systems

Switch Case in SQL

PES UNIVERSITY

Write an SQL query to display the first name and salary of each employee, along with a new column called Bonus_Status that categorizes employees based on their salary as follows:

- **Salary ≥ 50,000 → 'Eligible for 20% Bonus'**
- **Salary between 40,000 and 49,999 → 'Eligible for 10% Bonus'**
- **Salary < 40,000 → 'No Bonus'**

—TRY this

Expected Output:

Fname	Salary	Bonus_Status
John	30000.00	No Bonus
Franklin	40000.00	Eligible for 10% Bonus
Joyce	25000.00	No Bonus
Ramesh	38000.00	No Bonus
James	55000.00	Eligible for 20% Bonus
Jennifer	43000.00	Eligible for 10% Bonus
Ahmed	25000.00	No Bonus
Alicia	25000.00	No Bonus

Switch Case in SQL

Solution for the previous query

```
SELECT
    Fname,  Salary,
    CASE
        WHEN Salary >= 50000 THEN 'Eligible for 20% Bonus'
        WHEN Salary >= 40000 THEN 'Eligible for 10% Bonus'
        ELSE 'No Bonus'
    END AS Bonus_Status
FROM employee;
```

Database Management Systems

Switch Case in SQL

PES UNIVERSITY

Write a query to display each employee's first name, number of projects they are working on, and a new column Workload_Status as follows:

- 3 or more projects → Overloaded
- 2 projects → Normal Load
- 1 or 0 projects → Light Load

Expected Output:

Fname	Project_Count	Workload_Status
John	2	Normal Load
Franklin	4	Overloaded
Joyce	2	Normal Load
Ramesh	1	Light Load
James	1	Light Load
Jennifer	2	Normal Load
Ahmed	2	Normal Load
Alicia	2	Normal Load

Switch Case in SQL

Solution for the previous query

```
SELECT
    e.Fname,    COUNT(w.Pno) AS Project_Count,
    CASE
        WHEN COUNT(w.Pno) >= 3 THEN 'Overloaded'
        WHEN COUNT(w.Pno) = 2 THEN 'Normal Load'
        ELSE 'Light Load'
    END AS Workload_Status
FROM employee e
LEFT JOIN works_on w ON e.Ssn = w.Essn
GROUP BY e.Fname;
```

Switch Case in SQL

The same query using a CTE to show how CASE can be combined with CTE for better readability

```
WITH Project_Counts AS (
    SELECT e.Fname, COUNT(w.Pno) AS Proj_Count
    FROM employee e LEFT JOIN works_on w ON e.Ssn = w.Essn
    GROUP BY e.Fname
)
SELECT Fname,Proj_Count,
    CASE
        WHEN Proj_Count >= 3 THEN 'Overloaded'
        WHEN Proj_Count = 2 THEN 'Normal Load'
        ELSE 'Light Load'
    END AS Workload_Status
FROM Project_Counts;
```

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Corelated Nested Query

Dr. Nagasundari S

Department of Computer Science and Engineering

PES UNIVERSITY

Database Management Systems

Unit 2: Relational Model and Database Design

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

Correlated Nested queries

- A **correlated nested query** can be defined as a nested query in which the WHERE clause references **attributes of a relation declared in the outer query**.
- A nested query is **evaluated once for each tuple** (or combination of tuples) of the relation in the outer query.

Example: Retrieve the name of each employee who has a dependent who is of the same sex as the employee.

Query:
```
SELECT E.Fname, E.Lname FROM EMPLOYEE AS E WHERE E.Ssn IN
        ( SELECT D.Essn FROM DEPENDENT AS D  WHERE E.Gender = D.Gender );
```

Output:

Fname	Lname
Franklin	Wong

Correlated Nested queries – How it works?

- For each *EMPLOYEE* tuple, the nested query is evaluated, and the *Essn* values of all *DEPENDENT* tuples with the same *Gender* as that of the *EMPLOYEE* tuple are retrieved.
- If the *Ssn* value of the *EMPLOYEE* tuple is in the result of the nested query, the *EMPLOYEE* tuple is selected
- In general, a query written with nested SELECT-FROM-WHERE blocks and using the = or IN comparison operators can always be expressed as a single block query.
- For the current example, the equivalent query without a nested query is given below:

```
SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E, DEPENDENT AS D
WHERE E.Ssn = D.Essn AND E.Gender = D.Gender;
```

Exists and Unique

PES UNIVERSITY

- **EXISTS** and **UNIQUE** are Boolean functions that can be used in the WHERE clause
- The EXISTS function in SQL is used to check whether the **result of a nested query is empty** (contains no tuples) or not.
- **The result of EXISTS is True if the nested query result contains at least one tuple, and False if the nested query result contains no tuples.**
- On the other hand, NOT EXISTS returns True if there are no tuples in the result of the nested query, and False if the nested query result contains at least one tuple.
- EXISTS and NOT EXISTS are typically used in conjunction with a correlated nested query.

Exists - Example

Example: Retrieve the name of each employee who has a dependent who is of the same sex as the employee.

Query:
```
SELECT E.Fname, E.Lname FROM EMPLOYEE AS E
    WHERE EXISTS ( SELECT * FROM DEPENDENT AS D
    WHERE E.Ssn = D.Essn AND E.Gender = D.Gender);
```

Output:

Fname	Lname
Franklin	Wong

- For each *EMPLOYEE* tuple, the nested query is evaluated and it returns all *DEPENDENT* tuples with the same *Essn* and *Gender* as the *EMPLOYEE* tuple.
- If at least one tuple exists in the result of the nested query, then the *EMPLOYEE* tuple under consideration is selected.

Not Exists - Example

Example: Retrieve the names of employees who have no dependents

Query: SELECT Fname, Lname FROM EMPLOYEE

WHERE NOT EXISTS (SELECT * FROM DEPENDENT WHERE Ssn = Essn);

Output:

Fname	Lname
Joyce	English
Ramesh	Narayan
James	Borg
Ahmed	Jabbar
Alicia	Zelaya

- For each *EMPLOYEE* tuple, the correlated nested query selects all *DEPENDENT* tuples whose *Essn* value matches the *Ssn* value in the *EMPLOYEE* tuple
- If the result is empty (i.e. no dependents), the *EMPLOYEE* tuple is selected as the condition in the WHERE clause evaluates to True.

Exists - Example

Example: List the names of managers who have at least one dependent.

Query:
```
SELECT Fname, Lname FROM EMPLOYEE
    WHERE EXISTS ( SELECT * FROM DEPENDENT WHERE Ssn = Essn )
    AND EXISTS ( SELECT * FROM DEPARTMENT WHERE Ssn = Mgr_ssn );
```

Output:

Fname	Lname
Franklin	Wong
Jennifer	Wallace

- For each *EMPLOYEE* tuple, the first nested query selects all *DEPENDENT* tuples related to the *EMPLOYEE* tuple.
- The second nested query selects all *DEPARTMENT* tuples managed by *EMPLOYEE tuple.*
- The *EMPLOYEE* tuple is selected if both the nested queries return at least one tuple each.

Not Exists - Example

Example: Retrieve the name of each employee who works on all the projects controlled by department number 4

There are two ways to write this query

Query: SELECT Fname, Lname FROM EMPLOYEE

```
WHERE NOT EXISTS ( ( SELECT Pnumber FROM PROJECT WHERE Dnum = 4)
EXCEPT ( SELECT Pno FROM WORKS_ON WHERE Ssn = Essn) );
```

Output:

Fname	Lname
Ahmed	Jabbar
Alicia	Zelaya

Not Exists – How it works

(SELECT Pnumber FROM PROJECT WHERE Dnum = 4)

- This nested query retrieves the project number of all projects managed by department 4. It is not correlated with the outer query.

(SELECT Pno FROM WORKS_ON WHERE Ssn = Essn)

- This nested query is correlated with the outer query. For each *EMPLOYEE* tuple, it retrieves the project number of all projects that the employee works on.

Not Exists – How it works

```
( ( SELECT Pnumber FROM PROJECT WHERE Dnum = 4)
EXCEPT ( SELECT Pno FROM WORKS_ON WHERE Ssn = Essn) );
```

- Set difference operation is performed between the results of the nested queries i.e. set of project numbers under department 4 MINUS set of project numbers that a particular employee works on.
- The result is the set of project numbers controlled by department 4 that the employee under consideration does not work on.
- If that particular employee works on all projects controlled by department 4, the result of the above query would be an empty relation.
- The condition in the WHERE clause would be satisfied and the *EMPLOYEE* tuple under consideration would be selected

Not Exists - Example

The second way to write this query

Query: SELECT Fname, Lname FROM EMPLOYEE

```
WHERE NOT EXISTS ( SELECT * FROM WORKS_ON B
WHERE ( B.Pno IN ( SELECT Pnumber FROM PROJECT WHERE Dnum = 4 )
AND NOT EXISTS ( SELECT * FROM WORKS_ON C
WHERE C.Essn = Ssn AND C.Pno = B.Pno )));
```

Output:

Fname	Lname
Ahmed	Jabbar
Alicia	Zelaya

Not Exists – How it works

```
SELECT Fname, Lname FROM EMPLOYEE
WHERE NOT EXISTS ( SELECT * FROM WORKS_ON B
                   WHERE ( B.Pno IN ( SELECT Pnumber FROM PROJECT WHERE Dnum = 4 )
                   AND NOT EXISTS ( SELECT * FROM WORKS_ON C
                                    WHERE C.Essn = Ssn AND C.Pno = B.Pno )));
```

Each color represents a level of nesting

Let us try to understand how this query works:

- There is an outer query with 2 levels of nested subqueries. The outermost nested query is correlated with the outer query. There are two inner subqueries nested in the outermost nested subqueries – one which is correlated with the outermost nested query and the other which is not.

Not Exists – How it works

- The outermost nested query is evaluated for each *EMPLOYEE* tuple.
- As mentioned, one of the inner nested queries is not correlated with the outermost nested query:

  (SELECT Pnumber FROM PROJECT WHERE Dnum = 4)
- This query returns the project number of all projects controlled by department 4.
- The other inner nested query is correlated and is therefore evaluated for every *WORKS_ON* tuple in the outermost nested query:

  (SELECT * FROM WORKS_ON C WHERE C.Essn = Ssn AND C.Pno = B.Pno)
- For each *EMPLOYEE* tuple and *WORKS_ON* tuple, this query selects all tuples from the *WORKS_ON* relation whose *Essn* value matches the *Ssn* of the EMPLOYEE tuple and project number matches the *Pno* of the outer *WORKS_ON* tuple.

Not Exists – How it works

- Now, let's look at the outermost nested query (evaluated for every *EMPLOYEE* tuple) :

```
( SELECT * FROM WORKS_ON B WHERE ( B.Pno IN
( SELECT Pnumber FROM PROJECT WHERE Dnum = 4 )
AND NOT EXISTS ( SELECT * FROM WORKS_ON C
WHERE C.Essn = Ssn AND C.Pno = B.Pno )));
```

- This query selects only those *WORKS_ON* tuples that are concerned with projects controlled by department 4 and which the employee under consideration does not work on.
- If the employee under consideration works on all projects under department 4, then there would be no tuples selected by the outermost nested query. This would evaluate the WHERE predicate to True (as we are using NOT EXISTS) and hence, the *EMPLOYEE* tuple under consideration would be selected

THANK YOU

Dr. Nagasundari S
Department of Computer Science and Engineering

nagasundaris@pes.edu

Database Management Systems

Views,Users

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgment: Dr. Nagasundari S (Dept of CSE), Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof Mahitha G
Department of Computer Science and Engineering

View

- A view in MySQL is a virtual table.
- It doesn’t store actual data by itself but instead stores a query that pulls data from one or more tables (or even other views)
- Primarily used to hide complex queries and simplify into a convenient table like storage
- Since this is stored in memory, we can reuse these views and avoid writing SQL queries multiple times.
- Generally used when we want a table for specific use case or want to assign roles (Example given in further slides)

View

- To create a view, we can select fields from one or more tables present in the database.
- A view can either have specific rows based on a certain condition or all the rows of a table.
- We define a view in SQL by using the CREATE VIEW command.

  CREATE VIEW view1 AS <query expression>;

  where <query expression> is any legal query expression.

The view name is represented by view1.

View

- We can create a view from one table or using joins with multiple tables as well
- If you create a multi join view then only **read** operations are allowed

NOTE: A view is **updatable** (CRUD operations) only if these criterias are met:

- The view is created on a single table only
- Does not contain aggregate functions like sum, max, count etc
- Does not contain distinct, group by, having or union
- Does not have joins or subqueires (query within query like select for example)
- Query is not recursive in nature

Database Management Systems

Views Demo

PES UNIVERSITY

Table Schema

Field	Type	Null	Key	Default	Extra
Author_ID	varchar(50)	NO	PRI	NULL	
Name	varchar(50)	NO		NULL	
Age	int	YES		NULL	
Email	varchar(50)	YES	UNI	NULL	
Bio	text	YES		NULL	
Password	varchar(50)	NO		NULL	

Assume that there is a table called **Authors** with the following Schema and values.
We see how views work using this table alone (crud operations)

Table Values

Author_ID	Name	Age	Email	Bio	Password
A001	Alice Johnson	35	alice.johnson@example.com	Expert in AI and ML	password123
A002	Bob Smith	40	bob.smith@example.com	Data Scientist	password456
A003	Carol White	28	carol.white@example.com	Researcher in NLP	password789
A004	David Brown	50	david.brown@example.com	Professor in Quantum Computing	password321
A005	Eve Davis	32	eve.davis@example.com	Cloud Computing Specialist	password654
A006	Frank Martin	45	frank.martin@example.com	Expert in Computer Vision	password987
A007	Grace Lee	30	grace.lee@example.com	Cryptography Enthusiast	password147
A008	Henry Wilson	27	henry.wilson@example.com	Cybersecurity Analyst	password258
A009	Ivy Scott	34	ivy.scott@example.com	Data Engineering Expert	password369

Database Management Systems

Views Demo

Query

```
create view experienced_authors as
select * from authors
where age>=30 and Bio like '%Expert%';

select * from experienced_authors;
```

Let's create a view to only check experienced authors who are experts (Age>=30 and have the word Expert in their bio)

Result

```
13 •    select * from experienced_authors;
14
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Author_ID	Name	Age	Email	Bio	Password
A001	Alice Johnson	35	alice.johnson@example.com	Expert in AI and ML	password123
A006	Frank Martin	45	frank.martin@example.com	Expert in Computer Vision	password987
A009	Ivy Scott	34	ivy.scott@example.com	Data Engineering Expert	password369

Database Management Systems

Views Demo

PES UNIVERSITY

Query

```
-- Inserting a new experienced author
insert into experienced_authors (Author_ID, Name, Age, Email, Bio, Password)
values ("A010", "John Wick", 52, "john.wick@example.com", "Expert in Pencils", "password567");

-- Updating Alice's Age to 40 from 35
update experienced_authors set Age=40 where Author_ID='A001';
```

Performing insert and update operations on the view (Alice age change and john wick added)

Result

```
22      -- Updated Table with changes
23 •    select * from experienced_authors;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Author_ID	Name	Age	Email	Bio	Password
A001	Alice Johnson	40	alice.johnson@example.com	Expert in AI and ML	password123
A006	Frank Martin	45	frank.martin@example.com	Expert in Computer Vision	password987
A009	Ivy Scott	34	ivy.scott@example.com	Data Engineering Expert	password369
A010	John Wick	52	john.wick@example.com	Expert in Pencils	password567

Database Management Systems

Views Demo

Query

```
46      -- Deleting Ivy Scott (A009) from the view
47 •    delete from experienced_authors where Author_ID="A009" and Name="Ivy Scott";
48 •    select * from experienced authors;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Author_ID	Name	Age	Email	Bio	Password
A001	Alice Johnson	40	alice.johnson@example.com	Expert in AI and ML	password123
A006	Frank Martin	45	frank.martin@example.com	Expert in Computer Vision	password987
A010	John Wick	52	john.wick@example.com	Expert in Pencils	password567

Deleting Ivy Scott from the view

Authors Table

```
50      -- All the crud operations are reflected on the original table as well
51 •    select * from authors;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Author_ID	Name	Age	Email	Bio	Password
A001	Alice Johnson	40	alice.johnson@example.com	Expert in AI and ML	password123
A002	Bob Smith	40	bob.smith@example.com	Data Scientist	password456
A003	Carol White	28	carol.white@example.com	Researcher in NLP	password789
A004	David Brown	50	david.brown@example.com	Professor in Quantum Computing	password321
A005	Eve Davis	32	eve.davis@example.com	Cloud Computing Specialist	password654
A006	Frank Martin	45	frank.martin@example.com	Expert in Computer Vision	password987
A007	Grace Lee	30	grace.lee@example.com	Cryptography Enthusiast	password147
A008	Henry Wilson	27	henry.wilson@example.com	Cybersecurity Analyst	password258
A010	John Wick	52	john.wick@example.com	Expert in Pencils	password567

NOTE: Please note all the changes made to the view is also reflected on the original tables as well
Alice's age has been changed to 40, John wick was added and Ivy Scott was removed from authors table

Use Cases of View

PES UNIVERSITY

Example:

Create a view that lists all course sections offered by the Physics department in the Fall 2017 semester with the building and room number of each section.

Query:

```
create view physics_fall_2017 as
select course.course_id, sec_id, building, room_number
from course, section
where course.course_id = section.course_id
and course.dept_name = 'Physics'
and section.semester = 'Fall';and section.year = 2017;
```

tput:

course_id	sec_id	building	room_number
PHY-101	1	Watson	100

PES UNIVERSITY

Example:

Retrieve all Physics courses offered in the Fall 2017 semester in the Watson building.

Query:

```
select course id
from physics_fall_2017
where building = 'Watson';
```

Output:

course_id
PHY-101

Using View within a view

Example:

Retrieve the course ID and room number of all Physics courses offered in the Fall 2017 semester in the Watson building.

Query:

```
create view physics_fall_2017_watson as

select course_id, room_number

from physics_fall_2017

where building = 'Watson';
```

Output:

course_id	room_number
PHY-101	100

Advantages of View

- **Complexity:** Views help to reduce complexity. Different views can be created on the same base table for different users.
- **Security:** It increases security by excluding sensitive information from the view.
- **Query Simplicity:** It helps to simplify commands from the user. A view can draw data from several different tables and present it as a single table.
- **Consistency:** A view can present a consistent, unchanged image of the structure of the database. Views can be used to rename columns without affecting the base table.
- **Data Integrity:** If data is accessed and entered through a view, the DBMS can automatically check the data to ensure that it meets the specified integrity constraints.
- **Storage Capacity:** Views take very little space to store the data.
- **Logical Data Independence:** Views can make the application and database tables to a certain extent independent.

Disadvantages of View

- You cannot INSERT if the base table has any not null columns that do not appear in the view.
- You cannot INSERT or UPDATE if any of the columns referenced in the INSERT or UPDATE contains group functions or columns defined by an expression.
- You can't execute INSERT, UPDATE, DELETE statements on a view if with READ ONLY option is enabled.
- You can't create a view using temporary tables.
- You cannot INSERT, UPDATE, or DELETE if the view contains group functions GROUP BY, DISTINCT, or a reference to a pseudo column row_num.
- You can't pass parameters to the SQL server views.
- You can't associate rules and defaults with views.

Database Management Systems

Difference between Views and CTE

PES UNIVERSITY

Feature	Views	CTEs (Common Table Expressions)
Definition	A virtual table stored in the database, defined by a query.	A temporary result set defined within a query using WITH.
Persistence	Persisted in the database schema until explicitly dropped.	Exists only for the duration of the query execution.
Reusability	Can be reused across multiple queries once created.	Can only be used inside the single query where it is defined.
Scope	Global (available to all users with permissions).	Local (visible only within that query).
Complexity Handling	Good for hiding complex joins, filters, and calculations for repeated use.	Good for breaking down complex queries into readable parts (step-by-step).
Updatability	Some views are updatable (if based on a single table without aggregations).	CTEs are not updatable – they're just temporary results.
Syntax	CREATE VIEW view_name AS <query>; SELECT * FROM view_name;	WITH cte_name AS (<query>) SELECT * FROM cte_name;

Roles and Users

In SQL, "role" and "user" are two distinct concepts used for managing access and permissions within a database management system.

User:

- A user is an individual or entity that interacts with the database.
- Users are typically associated with specific individuals or applications that need to perform operations on the database.
- Each user has a unique username and, in some database systems, a password for authentication. Users are granted permissions to perform specific actions (e.g., SELECT, INSERT, UPDATE, DELETE) on database objects (e.g., tables, views).

Roles vs Users

PES UNIVERSITY

Role:

- A role is a named group of users or other roles.
- Roles are used to simplify and manage permissions and access control in a database.Instead of granting permissions to individual users, you can assign permissions to roles.
- Users are then added to roles, and they inherit the permissions associated with those roles. This makes it easier to manage and change permissions for multiple users at once.
- Roles can also be nested, meaning you can have roles within roles, creating a hierarchical permission structure.

Using roles and users together

Here's how roles and users are typically used together:

- You create roles and assign specific permissions to these roles. For example, you might create a "Readers" role with only SELECT permissions and a "Writers" role with INSERT, UPDATE, and DELETE permissions on certain tables.
- You add individual users to the roles they need. For instance, you might add UserA to the "Readers" role and UserB to the "Writers" role.
- Users then inherit the permissions associated with the roles they belong to. So, UserA will have SELECT permissions because they are in the "Readers" role, and UserB will have INSERT, UPDATE, and DELETE permissions because they are in the "Writers" role.

Creating new users

PES UNIVERSITY

Syntax for Create User:

CREATE USER 'username'@'host' IDENTIFIED BY 'password';

- 'username' : the account name.
- 'host' : where the user can connect from:
  - 'localhost' : only local machine.
  - '%' : any host.
  - Specific IP or hostname possible.
- 'password' : the login password for authentication.

Example: CREATE USER 'student'@'localhost' IDENTIFIED BY 'stud123';

To delete the created user:

DROP USER 'username'@'host';

Granting privileges

PES UNIVERSITY

- MySQL has a feature that provides many control options to the administrators and users on the database.
- We have already learned how to create a new user using the CREATE USER statement in the MySQL.
- Now, we are going to learn about granting privileges to a user account. MySQL provides GRANT statements to give access rights to a user account.
- The SQL standard includes the privileges SELECT, INSERT, UPDATE, and DELETE.

Granting privileges

Syntax:
```
GRANT privileges
ON database.table
TO 'username'@'host';
```

Privileges : what the user can do (SELECT, INSERT, UPDATE, etc.).
Database.table : the scope:

- ***.*** : all databases, all tables.
- **mydb.*** : all tables in mydb.
- **mydb.mytable** : specific table.

username@host : user account.

Granting privileges

PES UNIVERSITY

Common Privileges

- **Data privileges**:
  - SELECT, INSERT, UPDATE, DELETE
- **Structure privileges**:
  - CREATE, ALTER, DROP, INDEX
- **Administrative privileges**:
  - GRANT OPTION allows the user to grant privileges to others
  - SUPER is for superuser permissions
  - ALL PRIVILEGES is a shorthand for all available privileges.

Granting privileges

Examples:

a) **Give read-only access on one table**

```
GRANT SELECT ON school.students TO 'alice'@'localhost';
```

a) **Give full access on a database**

```
GRANT ALL PRIVILEGES ON school.* TO 'bob'@'localhost';
```

To check the privileges assigned to a specific user, use the following command

```
SHOW GRANTS FOR user_name;
```

If we want to **assign all privileges** to all databases in the current server

```
GRANT ALL ON school.* TO user_name;
```

Revoking privileges

To revoke an authorization, we use the REVOKE statement.

It takes a form almost identical to that of GRANT.

Syntax:

```
REVOKE privileges
ON database.table
FROM ‘username’@’localhost’;
```

Example:

```
REVOKE select
ON school.*
FROM ‘alice’@’localhost’;
```

Roles

Any authorization that can be granted to a user can be granted to a role.

Roles are granted to users just as authorizations are.

Roles can be created in SQL as follows:

```
create role Assistant_employee;
```

Roles can then be granted privileges just as the users :

```
grant select
on courses
to Assistant_employee;
```

Roles

Roles can be granted to users, as well as to other roles:

```
create role MD;

grant MD to user_name;
```

Revoking privileges from roles:

```
REVOKE INSERT, UPDATE, DELETE ON database_name FROM role_name;
```

Removing roles:

To delete one or more roles, you use the DROP ROLE statement as follows:

```
DROP ROLE role_name[, role_name, ...];

DROP ROLE role1, role2;
```

References

- https://www.devart.com/dbforge/mysql/studio/create-view-in-mysql.html
- https://thanusan25thanu.medium.com/mysql-views-are-not-only-query-able-but-also-updatable-91ac4730bc11
- https://dev.mysql.com/doc/refman/8.4/en/views.html
- https://www.red-gate.com/simple-talk/databases/mysql/working-with-mysql-views/
- https://sqlpad.io/tutorial/ctes-vs-views-in-sql-essential-distinctions-for-tech-interviews
- https://learnsql.com/blog/difference-between-sql-cte-and-view
- https://minhngocda.medium.com/view-vs-cte-in-sql-difference-and-use-cases-227307d0d672

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Triggers

Dr. Nagasundari S
Department of Computer Science and Engineering

PES UNIVERSITY

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

Triggers - Introduction

PES UNIVERSITY

- A trigger is a statement that the system executes automatically as a side effect of a modification to the database.

Defining a trigger involves:

- Specifying when a trigger needs to be executed. This consists of two components:

  Event – which causes the trigger to be checked

  Condition – which must be satisfied for trigger execution to proceed

- Specifying the **action** to be taken when the trigger executes

- Once we enter a trigger into the database, the system executes it whenever the specified event occurs and the corresponding condition is satisfied.

Need for Triggers

- **Triggers can be used to implement certain integrity constraints that cannot be specified using the constraint mechanism of SQL.**
- Triggers are also useful mechanisms for alerting humans or for starting certain tasks automatically when certain conditions are met.
- Consider a scenario wherein a student has taken up a new course and a new tuple needs to be inserted in the *TAKES* relation. We could design a trigger that updates the tuple in the *STUDENT* relation for the student taking the course by adding the number of credits for the course to the student's total credits, whenever a new tuple is inserted in the *TAKES* relation.
- Triggers usually cannot perform updates outside the database. However, some database systems provide built-in support for sending email from triggers.
- Triggers can be used in various applications, such as maintaining database consistency, monitoring database updates, and updating derived data automatically.

Triggers - Syntax

Let's have a look at the syntax of the CREATE TRIGGER statement.

Syntax:

```
CREATE TRIGGER trigger_name
(AFTER | BEFORE) (INSERT | UPDATE | DELETE) ON table_name
FOR EACH ROW
BEGIN
   --variable declarations
   --trigger code
END;
```

- The CREATE TRIGGER statement is used to create a trigger with the name *trigger_name* on the relation *table_name*.

Triggers - Syntax

Let's try to understand the syntax of the CREATE TRIGGER statement:

CREATE TRIGGER trigger_name

- The first line of the syntax (shown above) is used to **specify the name of the trigger.**

(AFTER | BEFORE) (INSERT | UPDATE | DELETE) ON table_name

- The second line of the syntax is used to specify the **event which causes the trigger to be checked**.

➢ table_name - specifies the **relation** which the trigger must check for modification

➢ INSERT | UPDATE | DELETE - specifies the **type of modification to be checked** i.e. whether the trigger must be checked on insertion, updation or deletion of tuples.

➢ AFTER | BEFORE - specifies **when the trigger must be checked** i.e. before or after the specified relation undergoes modification.

Triggers - Syntax

FOR EACH ROW

- Multiple tuples could be modified by a single SQL statement(for eg, multiple tuples could be inserted using one INSERT statement). The third line specifies that the trigger code would explicitly iterate over each modified row.
- The part of the query between the **BEGIN and END keywords corresponds to the action** to be taken on the modified tuples. This involves declaring any variables that would be used and specifying the SQL statements to be executed. Actions could be the execution of stored procedures as well.

Sample database description

Consider a sample database that stores information about students and the marks obtained by them in various courses. It has three tables:

- *Student_sample(SRN, name)* – stores information about students which includes their name and SRN (a unique identifier for each student).
- *Marks_sample(SRN, Course, Marks)* - stores information about marks obtained by students(identified by their SRN) in different courses.
- *Marks_history(SRN, Course, Marks)* – a backup table that stores information that has been deleted from the *Marks_sample* table.

Sample database description

The entries in the tables are shown below:

Student_sample:

SRN	Name
1	Harry
2	Ron
3	Hermione
4	Draco

Marks_sample:

SRN	Course	Marks
1	C01	98
1	C02	89
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98

Note that the Marks_history table is initially empty.

Database Management Systems

Triggers - Example

Example: Create a trigger that checks if the entered marks are valid or not whenever an entry is made in the Marks_sample table.

Code:

```
DELIMITER //
CREATE TRIGGER CheckMarks
BEFORE INSERT ON Marks_sample
FOR EACH ROW
BEGIN
        IF NEW.marks < 0 OR NEW.marks > 100 THEN
                SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'Invalid marks: Marks must be between 0 and 100';
        END IF;
END;//
DELIMITER ;
```

Triggers - How it works

- The given SQL statement creates a trigger *CheckMarks.* It gets executed **whenever there is an insertion** in the *Marks_sample* relation. The keyword BEFORE specifies that the **trigger execution occurs before the triggering operation** (in this case, insertion of tuples in the *Marks_sample* relation) is executed.
- The trigger action gets executed **for every row that is inserted**. This is specified by the FOR EACH ROW clause.
- The trigger checks if the *Marks* value of the inserted tuple is **lesser than 0 or greater than 100** (i.e. it checks if the ***Marks* value is invalid**). If the specified condition is satisfied, the trigger raises an **error**, an appropriate **error message** is displayed and the **tuple is not inserted** (trigger execution occurs before insertion). If the condition is not satisfied, the tuple is inserted.

Database Management Systems

Triggers - How it works

Let's try inserting into the *Marks_sample* relation using the statement:

INSERT INTO Marks_sample VALUES (1,'C03',101);

BEFORE:

SRN	Course	Marks
1	C01	98
1	C02	89
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98

AFTER:

SRN	Course	Marks
1	C01	98
1	C02	89
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98

Output:

13 12:19:02 INSERT INTO Marks_sample VALUES (1,'C03',101) Error Code: 1644. Invalid marks: Marks must be between 0 and 100 0.000 sec

The tuple does not get inserted as the *Marks* value is greater than 100.

Triggers - How it works

Note that if the *Marks* value is NULL, then the tuple gets inserted.

```
INSERT INTO Marks_sample VALUES (1,'C03',NULL);
```

BEFORE:

SRN	Course	Marks
1	C01	98
1	C02	89
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98

AFTER:

SRN	Course	Marks
1	C01	98
1	C02	89
1	C03	NULL
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98

Database Management Systems

Triggers - Example

Example: Implement a trigger that mimics the behaviour of ON UPDATE CASCADE.

Code:

```
DELIMITER //
CREATE TRIGGER update_marks_on_cascade
BEFORE UPDATE ON Student_sample
FOR EACH ROW
BEGIN
        SET FOREIGN_KEY_CHECKS = 0;
        UPDATE Marks_sample    SET SRN = NEW.SRN    WHERE SRN = OLD.SRN;
        SET FOREIGN_KEY_CHECKS = 1;
END;//
DELIMITER ;
```

NOTE : SET FOREIGN_KEY_CHECKS = 0 disables all foreign key checks(i.e. even if a foreign key constraint violation occurs, an error will not be raised). This is done temporarily to update the *Marks_sample* relation, after which the foreign key checks is enabled by specifying the SET FOREIGN_KEY_CHECKS = 1

Triggers - How it works

- The trigger *update_marks_on_cascade* updates the *SRN* values of tuples in the *Marks_sample* relation which are related to the tuples(in the *Student_sample* relation) being modified.
- To see its effect, let's execute the given query:

```
UPDATE Student_sample SET SRN = 5 WHERE name = 'Harry';
```

BEFORE:

Student_sample

SRN	Name
1	Harry
2	Ron
3	Hermione
4	Draco

Marks_sample

SRN	Course	Marks
1	C01	98
1	C02	89
1	C03	NULL
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98

AFTER:

Student_sample

SRN	Name
2	Ron
3	Hermione
4	Draco
5	Harry

Marks_sample

SRN	Course	Marks
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98
5	C01	98
5	C02	89
5	C03	NULL

Triggers - How it works

- The trigger automatically **updates the *SRN* values of all tuples** in the *Marks_sample* relation that are **related** to the tuple with *name* 'Harry'.

If this trigger would not have been created (and if ON UPDATE CASCADE was not specified in the foreign key constraint in the *Marks_sample* table), then:

- updating the SRN of any tuple in the *Student_sample* relation that have **related tuples** in the *Marks_sample* relation would have **thrown an error** as this updation will violate the foreign key constraint.
- However, updating the SRN of tuples that do not have any related tuples in the Marks_sample relation would not thrown an error.

Triggers - Example

Example: Implement a trigger that stores all tuples deleted from the *Marks_sample* table in the *Marks*_history table

Code:

```
DELIMITER //
CREATE TRIGGER backup_marks_info
BEFORE DELETE
ON Marks_sample
FOR EACH ROW
BEGIN
INSERT INTO Marks_history SELECT * FROM Marks_sample WHERE SRN = OLD.SRN AND COURSE =OLD.COURSE;
END //
DELIMITER ;
```

Database Management Systems

Triggers - How it works

Let's see the effect of this trigger by executing the given query:

```
DELETE FROM Marks_sample WHERE SRN = 2;
```

BEFORE:

Marks_sample

SRN	Course	Marks
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98
5	C01	98
5	C02	89
5	C03	NULL

Marks_history

SRN	Course	Marks

(empty set)

AFTER:

Marks_sample

SRN	Course	Marks
3	C01	95
3	C02	98
5	C01	98
5	C02	89
5	C03	NULL

Marks_history

SRN	Course	Marks
2	C01	100
2	C02	98
2	C03	80

THANK YOU

Dr. Nagasundari S
Department of Computer Science and Engineering

nagasundaris@pes.edu

Database Management Systems

Functions, Procedures

Dr. Nagasundari S

Department of Computer Science and Engineering

PES UNIVERSITY

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

Functions

- We have already seen several functions that are built into the SQL language.
- In this section, we show how developers can write their own functions and procedures, store them in the database, and then invoke them from SQL statements.
- Functions and procedures allow "business logic" to be stored in the database and executed from SQL statements.
- These can be defined either by the procedural component of SQL or by an external programming language such as Java, C, or C++.
- Functions that are created by the user in the system database or a user-defined database are known as user-defined functions.
- The UDF functions accept parameters, perform actions, and returns the result.

Functions

- These functions help us to simplify our development by encapsulating complex business logic and making it available for reuse anywhere based on the needs.
- The user-defined functions make the code needed to query data a lot easier to write.
- They also improve query readability and functionality, as well as allow other users to replicate the same procedure.

Functions

- function in SQL Server always accepts parameters, either single or multiple and returns a single value or table.
- The functions are useful in the simplification of our code.
- Suppose we might have a complex computation that appears in a number of queries.
- In such a case, we can build a function that encapsulates the formula and uses it in each query instead of in each query.

Syntax:

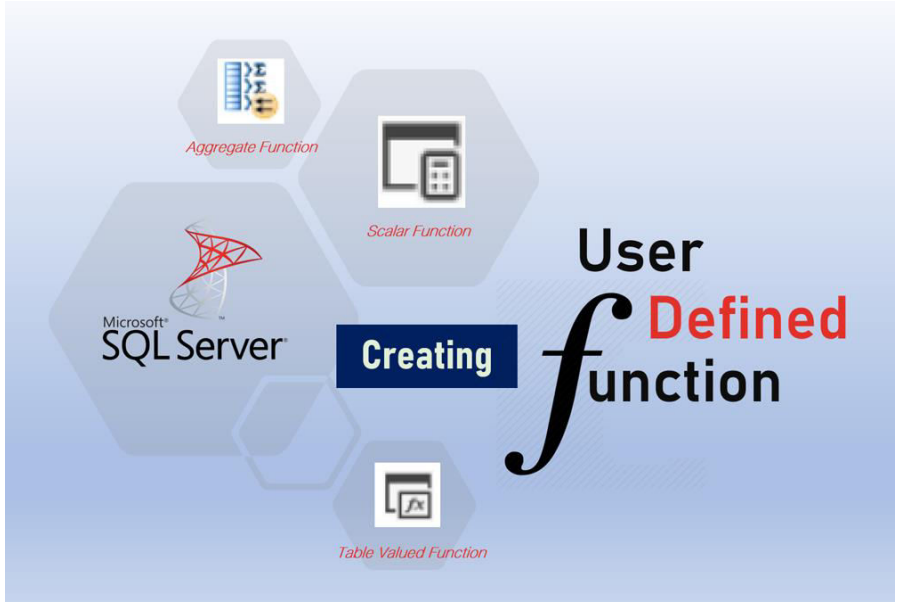
```
CREATE FUNCTION schema_name.function_name (parameter_list)
RETURNS data_type AS
BEGIN
   statements
   RETURN value
END
```

Functions

- **function_name :** It is the name of the function that needs to be created in MySQL.
- **parameter1, parameter2,… :** We can pass the optional parameters to the functions that need to be declared while creating it in the () brackets.
- **A function can contain none, one or more than one parameter.**
- **BEGIN and END –** BEGIN keyword marks the beginning of the function while END marks the completion of function in MYSQL.

How to Delete function?

```
DROP FUNCTION IF EXISTS function_name;
```



Functions

- **RETURN Datatype** – We can return any type of value from the execution of the function.
- The type of value that will be returned needs to be specified after the RETURN clause.
- Once, MySQL finds the RETURN statement while execution of the function, execution of the function is terminated and the value is returned.
- **DETERMINISTIC** – The function can be either deterministic or non-deterministic which needs to be specified here.
- When the function returns the same value for the same values of parameter then it is called deterministic. However, if the function returns a different value for the same values of functions then we can call that function to be nondeterministic.
- Functions will be NON-DETERMINISTIC by default.

Functions

PES UNIVERSITY

Example:

```
CREATE FUNCTION Dept_size(deptno INT)
        RETURNS VARCHAR(7)
        BEGIN
                DECLARE No_of_emps INT;
                SELECT COUNT(*) INTO No_of_emps FROM EMPLOYEE WHERE Dno = deptno;
                IF No_of_emps > 3 THEN    RETURN 'HUGE';
                ELSEIF No_of_emps > 2 THEN    RETURN 'LARGE';
                ELSEIF No_of_emps > 1 THEN    RETURN 'MEDIUM';
                ELSE    RETURN 'SMALL';
                END IF;
END
```

Query: select Dname,Dnumber,Dept_size(Dnumber)from department;

Output:

Dname	Dnumber	Dept_size(Dnumber)
Administration	4	HUGE
Headquarters	1	SMALL
Research	5	HUGE

Functions

Example: Write a function to find the age of employes

```
CREATE FUNCTION no_of_years(date1 date)
RETURNS INTEGER deterministic
        BEGIN
                DECLARE date2 DATE;
                Select current_date()into date2;
                RETURN year(date2)-year(date1);
        END
```

Query: select Fname,no_of_years(Bdate) from employee;

Output:

Fname	no_of_years(Bdate)
John	58
Franklin	68
Joyce	51
Richard	61
Ramesh	61
James	86
Jennifer	82
Ahmed	54
Alicia	55

THANK YOU

Dr. Nagasundari S
Department of Computer Science and Engineering

nagasundaris@pes.edu

Database Management Systems

Functions, Procedures

Dr. Nagasundari S

Department of Computer Science and Engineering

PES UNIVERSITY

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

Stored Procedures

- A procedure (often called a stored procedure) is a collection of pre-compiled SQL statements stored inside the database.
- It is a subroutine or a subprogram in the regular computing language.
- A procedure always contains a name, parameter lists, and SQL statements. We can invoke the procedures by using triggers, other procedures and applications such as Java , Python , PHP , etc.
- The following syntax is used for creating a stored procedure in MySQL.

```
DELIMITER &&
CREATE PROCEDURE procedure_name [[IN | OUT | INOUT] parameter_name datatype
        [, parameter        datatype]) ]
        BEGIN
                Declaration_section
                Executable_section
        END &&
DELIMITER ;
```

Stored Procedures

Stored procedures are useful in the following circumstances:

- If a database program is needed by several applications, it can be stored at the server and invoked by any of the application programs.
- This reduces duplication of effort and improves software modularity.
- Executing a program at the server can reduce data transfer and communication cost between the client and server in certain situations.
- These procedures can enhance the modeling power provided by views by allowing more complex types of derived data to be made available to the database users via the stored procedures.
- Additionally, they can be used to check for complex constraints that are beyond the specification power of assertions and triggers.

Stored Procedures

Modes of Procedure Parameters:

- IN: It is the default mode. It takes a parameter as input, such as an attribute. When we define it, the calling program has to pass an argument to the stored procedure. This parameter's value is always protected.
- OUT: It is used to pass a parameter as output. Its value can be changed inside the stored procedure, and the changed (new) value is passed back to the calling program. It is noted that a procedure cannot access the OUT parameter's initial value when it starts.
- INOUT: It is a combination of IN and OUT parameters. It means the calling program can pass the argument, and the procedure can modify the INOUT parameter, and then passes the new value back to the calling program.
- How to call a stored procedure?
- Syntax: CALL procedure_name (parameter(s))

PES UNIVERSITY

Consider this table

student_id	student_code	student_name	subject	marks	phone
1	101	Mark	English	68	3454569353
2	102	Joseph	Physics	70	9876543565
3	103	John	maths	70	9765326975
4	104	Barack	maths	90	8769875325
5	105	Rinky	maths	85	6753157975
6	106	Adam	Science	92	7964225686
7	107	Andrew	Science	83	5674243757
8	108	Brayan	Science	85	7523416567
10	110	Alexandar	Biology	67	2347346438

Stored Procedures

PES UNIVERSITY

Procedure without Parameter:

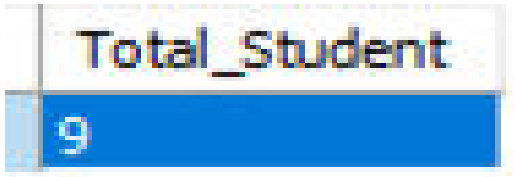
Example: Suppose we want to display all records of this table whose marks are greater than 70 and count all the table rows.

The following code creates a procedure named get_merit_students:

```
DELIMITER $$
CREATE PROCEDURE`get_merit_student`()
BEGIN
        SELECT * FROM student WHERE marks > 70;
        SELECT COUNT(student_code) AS Total_Student FROM student;
END$$
DELIMITER ;
```

Query: CALL get_merit_student();

Output:



Total_Student
9

Procedures with IN Parameter

In this procedure, we have used the IN parameter as **'var1'** of integer type to accept a number from users. Its body part fetches the records from the table using a **SELECT** statement and returns only those rows that will be supplied by the user. It also returns the total number of rows of the specified table. See the procedure code:

```
DELIMITER &&
CREATE PROCEDURE get_student (IN var1 INT)
BEGIN
    SELECT * FROM student_info LIMIT var1;
    SELECT COUNT(stud_code) AS Total_Student FROM student_info;
END &&
DELIMITER ;
```

Output:

student_id	student_code	student_name	subject	marks	phone
1	101	Mark	English	68	3454569353
2	102	Joseph	Physics	70	9876543565
3	103	John	maths	70	9765326975

Stored Procedures

Query:

```
Call get_student(3);
```

Output:

student_id	student_code	student_name	subject	marks	phone
1	101	Mark	English	68	3454569353
2	102	Joseph	Physics	70	9876543565
3	103	John	maths	70	9765326975

Query:

```
Call get_student(5);
```

Output:

student_id	student_code	student_name	subject	marks	phone
1	101	Mark	English	68	3454569353
2	102	Joseph	Physics	70	9876543565
3	103	John	maths	70	9765326975
4	104	Barack	maths	90	8769875325
5	105	Rinky	maths	85	6753157975

Procedures with OUT Parameter

In this procedure, we have used the OUT parameter as the **'highestmark'** of integer type. Its body part fetches the maximum marks from the table using a **MAX() function**. See the procedure code:

```
DELIMITER &&
CREATE PROCEDURE `display_max_mark` (OUT highestmark INTEGER)
BEGIN
    SELECT max(marks) INTO highestmark FROM student;
END&&
DELIMITER ;
```

Query:

```
call display_max_mark(@output);
select @output;
```

Output:

@output
92

Procedures with INOUT Parameter:

In this procedure, we have used the INOUT parameter as **'var1'** of integer type. Its body part first fetches the marks from the table with the specified **id** and then stores it into the same variable var1. The var1 first acts as the IN parameter and then OUT parameter. Therefore, we can call it the INOUT parameter mode. See the procedure code:

```
DELIMITER &&
CREATE PROCEDURE `display_marks` (INOUT var1 INTEGER)
BEGIN
    SELECT marks INTO var1 FROM student WHERE stud_id = var1;
END&&
DELIMITER ;
```

Stored Procedures

Query:

```
SET @M = '3';
CALL display_marks(@M);
SELECT @M;
```

Output:

@M
70

Query:

```
SET @M = '4';
CALL display_marks(@M);
SELECT @M;
```

Output:

@M
90

PES UNIVERSITY

How to delete/drop stored procedures in MySQL?

MySQL also allows a command to drop the procedure. When the procedure is dropped, it is removed from the database server also. The following statement is used to drop a stored procedure in MySQL:

DROP PROCEDURE [IF EXISTS] procedure_name;

Ex: DROP PROCEDURE display_marks;

We can verify it by listing the procedure in the specified database using the SHOW PROCEDURE STATUS command.

SHOW PROCEDURE STATUS [LIKE 'pattern' | WHERE search_condition]

Ex: SHOW PROCEDURE STATUS WHERE db = 'mystudentdb';

STORED PROCEDURE with While and if-else

```
CREATE PROCEDURE StudentMarksWithLoop()
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE marks INT;
    DECLARE student_name VARCHAR(25);

    WHILE done = 0 DO
        SELECT marks, student_name INTO marks,
        student_name
        FROM student
        WHERE done = 0
        LIMIT 1;

        IF marks >= 50 THEN
            SELECT CONCAT(student_name, ' passed')
AS Result;
        ELSE
            SELECT CONCAT(student_name, ' failed')
AS Result;
        END IF;

        SET done = 1;
    END WHILE;
END //

DELIMITER ;
```

Note: This WHILE loop that continues to process students until the done flag is set to 1. Inside the loop, we retrieve the marks and student_name for one student at a time, check if they passed or failed, and then set the done flag to 1 to exit the loop after processing one student.

STORED PROCEDURE Vs FUNCTION

Stored Procedure	Function
Supports in, out and in-out parameters,i.e., input and output parameters	Supports only input parameters, no output parameters.
Stored procedures can call functions as needed	The function cannot call a stored procedure
There is no provision to call procedures from select/having and where statements	You can call functions from a select statement
Transactions can be used in stored procedures	No transactions are allowed
Can do exception handling by inserting try/catch blocks	No provision for explicit exception handling
Need not return any value	Must return a result or value to the caller
All the database operations like insert, update, delete can be performed	Only select is allowed

THANK YOU

Dr. Nagasundari S
Department of Computer Science and Engineering

nagasundaris@pes.edu

Database Management Systems

Window Functions

Prof. Mahitha G, Prof. Revathi G P

Department of Computer Science and Engineering

Acknowledgment: Dr. Nagasundari S (Dept of CSE), Teaching Assistants

Database Management Systems

Windows Functions in SQL

PES UNIVERSITY

What are window functions?

Window functions let you do calculations across rows, but instead of grouping them like SUM() or AVG() normally does, they let you keep every row and just add extra calculated values.

Think of it like like having a “window” (a set of rows related to the current row), and you calculate something (sum, average, rank, running total, etc.) inside that window.

How are they different from aggregate functions?

While aggregate functions collapse rows into one result, window functions add new values without collapsing rows.

Consider the following examples:

Database Management Systems

Windows Functions in SQL - Example

Sample schema - Departments and Employees table

```
mysql> select * from departments;
+---------+-------------+
| dept_id | dept_name   |
+---------+-------------+
|       1 | HR          |
|       2 | Sales       |
|       3 | Engineering |
+---------+-------------+
3 rows in set (0.01 sec)
```

```
mysql> select * from employees;
+--------+----------+-------------+--------+------------+
| emp_id | emp_name | department  | salary | hire_date  |
+--------+----------+-------------+--------+------------+
|    101 | Alice    | Sales       |   5000 | 2020-01-15 |
|    102 | Bob      | Sales       |   7000 | 2019-03-12 |
|    103 | Charlie  | Engineering |   9000 | 2018-06-01 |
|    104 | David    | Engineering |   8000 | 2021-07-19 |
|    105 | Eva      | HR          |   6000 | 2020-09-30 |
|    106 | Frank    | Sales       |   7500 | 2022-02-11 |
+--------+----------+-------------+--------+------------+
6 rows in set (0.00 sec)
```

Database Management Systems

Windows Functions in SQL - Example

```
mysql> SELECT department, AVG(salary)
    -> FROM employees
    -> GROUP BY department;
+-------------+-------------+
| department  | AVG(salary) |
+-------------+-------------+
| Sales       |   6500.0000 |
| Engineering |   8500.0000 |
| HR          |   6000.0000 |
+-------------+-------------+
3 rows in set (0.00 sec)
```

Executing the query shown using aggregate function AVG we get a single row for each department.

```
mysql> SELECT
    ->   emp_name, department, salary,
    ->   AVG(salary) OVER (PARTITION BY department) AS dept_avg_salary
    -> FROM employees;
+----------+-------------+--------+-----------------+
| emp_name | department  | salary | dept_avg_salary |
+----------+-------------+--------+-----------------+
| Charlie  | Engineering |   9000 |       8500.0000 |
| David    | Engineering |   8000 |       8500.0000 |
| Eva      | HR          |   6000 |       6000.0000 |
| Alice    | Sales       |   5000 |       6500.0000 |
| Bob      | Sales       |   7000 |       6500.0000 |
| Frank    | Sales       |   7500 |       6500.0000 |
+----------+-------------+--------+-----------------+
6 rows in set (0.00 sec)
```

Using window functions, all rows in the employee table are maintained along with average salary of the department they belong to.

Windows Functions in DBMS - General Syntax

Syntax

```
SELECT column_name1,
window_function(column_name2)
OVER([PARTITION BY column_name1]) AS new_column
FROM table_name;
```

Key Terms

window_function= any aggregate or ranking function
column_name1= column to be selected
column_name2= column on which window function is to be applied
new_column= Name of new column
table_name= Name of table

Windows Functions in SQL - Example

Let's now try to understand what this query does in detail:

```
SELECT name, department, salary,AVG(salary)
OVER (PARTITION BY department)
AS dept_avg_salary
FROM employees;
```

1. OVER clause tells SQL to not collapse rows like GROUP BY and instead, calculate an aggregate in a 'window' while keeping all rows visible.
2. PARTITION BY department creates separate "mini-groups" (windows) for each department i.e one window for Sales,one window for Engineering,one window for HR. Within each window, calculations are done.
3. AVG(salary)
   Inside each department, SQL calculates the average salary.

Windows Functions in DBMS

PES UNIVERSITY

Why are window functions required?

Window functions let you analyze data across rows without losing individual rows. They're required when you want both:

1. Detailed row-level data (every employee, every order, etc.)
2. Aggregates or comparisons across groups (totals, averages, rankings, etc.)

Traditional GROUP BY only gives you summary rows, but window functions give you summary + detail in the same query.

Common use cases of window functions include Ranking & Row Numbers,Running Totals & Cumulative Sums,Moving Averages (Trends),Comparisons to Group Values,Percentiles & Distribution,Lag & Lead (Previous/Next Row Access) etc.

Windows Functions in SQL - Types

PES UNIVERSITY

Types of Window Functions in SQL

SQL window functions can be categorized into two primary types:

- **Aggregate window functions**
- **Ranking window functions**.

These two types serve different purposes but share a common ability to perform calculations over a defined set of rows while retaining the original data.

Aggregate Windows Functions in SQL

1. Aggregate Window Function

Aggregate window functions calculate aggregates over a window of rows while retaining individual rows.

Common aggregate functions include:

SUM(): Sums values within a window.
AVG(): Calculates the average value within a window.
COUNT(): Counts the rows within a window.
MAX(): Returns the maximum value in the window.
MIN(): Returns the minimum value in the window.

Aggregate Windows Functions in SQL

Some other examples:

1. **To find Maximum salary in each department:**

```
SELECT emp_ID, emp_name, Department, Salary, MAX(Salary)
OVER (PARTITION BY Department) AS MaxDeptSalary
FROM Employees;
```

```
mysql> SELECT Emp_ID, emp_name, Department, Salary, MAX(Salary)
    -> OVER (PARTITION BY Department) AS MaxDeptSalary FROM Employees;
+--------+----------+-------------+--------+---------------+
| Emp_ID | emp_name | Department  | Salary | MaxDeptSalary |
+--------+----------+-------------+--------+---------------+
|    103 | Charlie  | Engineering |   9000 |          9000 |
|    104 | David    | Engineering |   8000 |          9000 |
|    105 | Eva      | HR          |   6000 |          6000 |
|    101 | Alice    | Sales       |   5000 |          7500 |
|    102 | Bob      | Sales       |   7000 |          7500 |
|    106 | Frank    | Sales       |   7500 |          7500 |
+--------+----------+-------------+--------+---------------+
6 rows in set (0.04 sec)
```

PES UNIVERSITY

2. Count of employees in each department:

SELECT Emp_ID, emp_name, Department, Salary, COUNT(*)
OVER (PARTITION BY Department) AS DeptEmployeeCount
FROM Employees;

```
mysql> SELECT Emp_ID, emp_name, Department, Salary,
    ->        COUNT(*) OVER (PARTITION BY Department) AS DeptEmployeeCount
    -> FROM Employees;
+--------+----------+-------------+--------+-------------------+
| Emp_ID | emp_name | Department  | Salary | DeptEmployeeCount |
+--------+----------+-------------+--------+-------------------+
|    103 | Charlie  | Engineering |   9000 |                 2 |
|    104 | David    | Engineering |   8000 |                 2 |
|    105 | Eva      | HR          |   6000 |                 1 |
|    101 | Alice    | Sales       |   5000 |                 3 |
|    102 | Bob      | Sales       |   7000 |                 3 |
|    106 | Frank    | Sales       |   7500 |                 3 |
+--------+----------+-------------+--------+-------------------+
6 rows in set (0.03 sec)
```

PES UNIVERSITY

2. Ranking Window Function

Ranking window functions assign a position (rank/number) to each row within a partition, usually based on an ORDER BY clause.

Common Ranking Functions:

1) **RANK()** : Assigns ranks to rows with the same value receiving the same rank.If there are ties, the next rank is skipped.
2) **DENSE_RANK()** : Similar to RANK(), but ranks are not skipped.If there are ties, the next rank is consecutive.
3) **ROW_NUMBER()** : Assigns a unique sequential number to each row (1, 2, 3 …).No ties, every row has a different number.

Ranking Window Functions in SQL

1. RANK() Function

- RANK() is a window function that assigns a rank (1st, 2nd, 3rd, …) to each row within a partition.
- The ranking is based on the ORDER BY clause inside the window.
- Ties (equal values) get the same rank, and the next rank is skipped.For example: if two rows are tied at rank 1, the next row will be rank 3 (not 2) as shown below.
- Example:

```
mysql> SELECT emp_name, department, salary,        RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank_in_dept FROM Employees;
+----------+-------------+--------+--------------+
| emp_name | department  | salary | rank_in_dept |
+----------+-------------+--------+--------------+
| Charlie  | Engineering |   9000 |            1 |
| David    | Engineering |   8000 |            2 |
| Eva      | HR          |   6000 |            1 |
| Bob      | Sales       |   7000 |            1 |
| Frank    | Sales       |   7000 |            1 |
| Alice    | Sales       |   5000 |            3 |
+----------+-------------+--------+--------------+
6 rows in set (0.01 sec)
```

PES UNIVERSITY

2. DENSE_RANK() Function

- DENSE_RANK() is a window function that also assigns ranks (1st, 2nd, 3rd, …) to rows within a partition.
- The ranking is based on the ORDER BY clause inside the window.
- Ties (equal values) get the same rank.
- No ranks are skipped (this is the main difference from RANK()).
- Example: Alice is now numbered 2 instead of 3 when rank was used.

```
mysql> SELECT emp_name, department, salary,
    ->        DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dense_rank_in_dept
    -> FROM Employees;
+----------+-------------+--------+--------------------+
| emp_name | department  | salary | dense_rank_in_dept |
+----------+-------------+--------+--------------------+
| Charlie  | Engineering |   9000 |                  1 |
| David    | Engineering |   8000 |                  2 |
| Eva      | HR          |   6000 |                  1 |
| Bob      | Sales       |   7000 |                  1 |
| Frank    | Sales       |   7000 |                  1 |
| Alice    | Sales       |   5000 |                  2 |
+----------+-------------+--------+--------------------+
6 rows in set (0.01 sec)
```

PES UNIVERSITY

3. ROW_NUMBER() Function

- The ROW_NUMBER() function assigns a unique sequential number to each row within a partition.
- The numbering starts at 1 for each partition.
- Unlike RANK() or DENSE_RANK(), there are no ties → every row gets its own distinct number, even if values are equal.
- Example: No ties, all rows are unique.

```
mysql> SELECT emp_name, department, salary,
    ->        ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS row_num_in_dept
    -> FROM Employees;
+----------+-------------+--------+-----------------+
| emp_name | department  | salary | row_num_in_dept |
+----------+-------------+--------+-----------------+
| Charlie  | Engineering |   9000 |               1 |
| David    | Engineering |   8000 |               2 |
| Eva      | HR          |   6000 |               1 |
| Bob      | Sales       |   7000 |               1 |
| Frank    | Sales       |   7000 |               2 |
| Alice    | Sales       |   5000 |               3 |
+----------+-------------+--------+-----------------+
6 rows in set (0.04 sec)
```

Lag Function in SQL

LAG() Function

- The LAG() function lets you look at the value of a column from a previous row within the same partition.
- It is very useful for comparing current row values with earlier ones (like salary growth, stock price change, etc.).
- You can specify how many rows back you want to look (default = 1 row).

General syntax:

```
LAG(column_name, offset, default_value)
OVER (PARTITION BY partition_column
ORDER BY order_column)
```

column_name is the column you want to peek ahead into.
offset is how many rows forward to look (default = 1).
default_value is optional, what to return if there's no row that far ahead (default = NULL).

Database Management Systems

Lag Function in SQL

Example:

```
mysql> SELECT emp_name, department, salary,
    ->        LAG(salary, 1) OVER (PARTITION BY department ORDER BY salary DESC) AS prev_salary
    -> FROM Employees;
+----------+-------------+--------+-------------+
| emp_name | department  | salary | prev_salary |
+----------+-------------+--------+-------------+
| Charlie  | Engineering |   9000 |        NULL |
| David    | Engineering |   8000 |        9000 |
| Eva      | HR          |   6000 |        NULL |
| Bob      | Sales       |   7000 |        NULL |
| Frank    | Sales       |   7000 |        7000 |
| Alice    | Sales       |   5000 |        7000 |
+----------+-------------+--------+-------------+
6 rows in set (0.05 sec)
```

Use Cases of LAG():

- Compare current salary with previous salary in the same department.
- Track daily stock price changes by comparing today vs yesterday.
- Calculate differences, growth %, or trends row by row.

Lead Function in SQL

LEAD() Function

- The LEAD() function lets you look at the value of a column from a future row within the same partition.
- It is the opposite of LAG().
- Useful for comparing the current row with the *next one* (e.g., predicting trends, seeing who comes after).

General syntax:

```
LEAD(column_name, offset, default_value)
OVER (PARTITION BY partition_column
ORDER BY order_column)
```

column_name is the column you want to peek ahead into.
offset is how many rows forward to look (default = 1).
default_value is optional, what to return if there's no row that far ahead (default = NULL).

Database Management Systems

Lead Function in SQL

Example:

```
mysql> SELECT emp_name, department, salary,
    ->        LEAD(salary, 1) OVER (PARTITION BY department ORDER BY salary DESC) AS next_salary
    -> FROM Employees;
+----------+-------------+--------+-------------+
| emp_name | department  | salary | next_salary |
+----------+-------------+--------+-------------+
| Charlie  | Engineering |   9000 |        8000 |
| David    | Engineering |   8000 |        NULL |
| Eva      | HR          |   6000 |        NULL |
| Bob      | Sales       |   7000 |        7000 |
| Frank    | Sales       |   7000 |        5000 |
| Alice    | Sales       |   5000 |        NULL |
+----------+-------------+--------+-------------+
6 rows in set (0.04 sec)
```

Use Cases of LEAD():

- See what the next employee's salary is in the same department.
- Calculate salary differences ahead of time (LEAD(salary) - salary).
- Compare today's stock price with tomorrow's price (useful in analytics & predictions).
- Identify transitions (e.g., who replaces whom in a sequence).

Windows Functions in SQL - Practical Use Cases

Practical Use Cases for Window Functions

Window functions are extremely versatile and can be used in a variety of practical scenarios. Below are some examples of how these functions can be applied in real-world data analysis.

SaleID	EmpID	EmpName	Department	CustomerID	SaleDate	Month	Amount
1	101	Alice	HR	C001	2023-01-05	Jan	1000
2	101	Alice	HR	C002	2023-02-10	Feb	2000
3	101	Alice	HR	C001	2023-03-15	Mar	1500
4	102	Bob	HR	C003	2023-01-20	Jan	1200
5	102	Bob	HR	C001	2023-02-25	Feb	1800
6	103	Charlie	IT	C004	2023-01-10	Jan	7000
7	103	Charlie	IT	C005	2023-02-18	Feb	7500
8	104	David	IT	C006	2023-01-28	Jan	7200

Windows Functions in SQL - Practical Use Cases

PES UNIVERSITY

1. Top 2 Sales by Each Employee (RANK)

SELECT * FROM (SELECT *, RANK() OVER (PARTITION BY EmpID ORDER BY Amount DESC) AS SaleRank FROM Sales) AS ranked WHERE SaleRank <= 2;

```
mysql> SELECT * FROM ( SELECT *, RANK() OVER (PARTITION BY EmpID ORDER BY Amount DESC) AS SaleRank FROM Sales ) AS ranked WHERE SaleRank <= 2;
+--------+-------+---------+------------+------------+------------+-------+--------+----------+
| SaleID | EmpID | EmpName | Department | CustomerID | SaleDate   | Month | Amount | SaleRank |
+--------+-------+---------+------------+------------+------------+-------+--------+----------+
|      2 |   101 | Alice   | HR         | C002       | 2023-02-10 | Feb   |   2000 |        1 |
|      3 |   101 | Alice   | HR         | C001       | 2023-03-15 | Mar   |   1500 |        2 |
|      5 |   102 | Bob     | HR         | C001       | 2023-02-25 | Feb   |   1800 |        1 |
|      4 |   102 | Bob     | HR         | C003       | 2023-01-20 | Jan   |   1200 |        2 |
|      7 |   103 | Charlie | IT         | C005       | 2023-02-18 | Feb   |   7500 |        1 |
|      6 |   103 | Charlie | IT         | C004       | 2023-01-10 | Jan   |   7000 |        2 |
|      8 |   104 | David   | IT         | C006       | 2023-01-28 | Jan   |   7200 |        1 |
+--------+-------+---------+------------+------------+------------+-------+--------+----------+
7 rows in set (0.01 sec)
```

2. **Running Total of Sales per Employee**

SELECT EmpID, EmpName, Month, Amount, SUM(Amount) OVER (PARTITION BY EmpID ORDER BY SaleDate) AS RunningTotal FROM Sales;

```
mysql> SELECT EmpID, EmpName, Month, Amount, SUM(Amount) OVER (PARTITION BY EmpID ORDER BY SaleDate) AS RunningTotal FROM Sales;
+-------+---------+-------+--------+--------------+
| EmpID | EmpName | Month | Amount | RunningTotal |
+-------+---------+-------+--------+--------------+
|   101 | Alice   | Jan   |   1000 |         1000 |
|   101 | Alice   | Feb   |   2000 |         3000 |
|   101 | Alice   | Mar   |   1500 |         4500 |
|   102 | Bob     | Jan   |   1200 |         1200 |
|   102 | Bob     | Feb   |   1800 |         3000 |
|   103 | Charlie | Jan   |   7000 |         7000 |
|   103 | Charlie | Feb   |   7500 |        14500 |
|   104 | David   | Jan   |   7200 |         7200 |
+-------+---------+-------+--------+--------------+
8 rows in set (0.00 sec)
```

PES UNIVERSITY

3. First Sale of Each Customer

SELECT * FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY SaleDate) AS rn FROM Sales) AS first_sales WHERE rn = 1;

```
mysql> SELECT * FROM ( SELECT *, ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY SaleDate) AS rn FROM Sales ) AS first_sales WHERE rn = 1;
+--------+-------+---------+------------+------------+------------+-------+--------+----+
| SaleID | EmpID | EmpName | Department | CustomerID | SaleDate   | Month | Amount | rn |
+--------+-------+---------+------------+------------+------------+-------+--------+----+
|      1 |   101 | Alice   | HR         | C001       | 2023-01-05 | Jan   |   1000 |  1 |
|      2 |   101 | Alice   | HR         | C002       | 2023-02-10 | Feb   |   2000 |  1 |
|      4 |   102 | Bob     | HR         | C003       | 2023-01-20 | Jan   |   1200 |  1 |
|      6 |   103 | Charlie | IT         | C004       | 2023-01-10 | Jan   |   7000 |  1 |
|      7 |   103 | Charlie | IT         | C005       | 2023-02-18 | Feb   |   7500 |  1 |
|      8 |   104 | David   | IT         | C006       | 2023-01-28 | Jan   |   7200 |  1 |
+--------+-------+---------+------------+------------+------------+-------+--------+----+
6 rows in set (0.01 sec)
```

PES UNIVERSITY

4. **Compare Current Sale with Previous Sale (LAG)**

SELECT EmpID, EmpName, Month, Amount, LAG(Amount) OVER (PARTITION BY EmpID ORDER BY SaleDate) AS PrevAmount FROM Sales;

```
mysql> SELECT EmpID, EmpName, Month, Amount, LAG(Amount) OVER (PARTITION BY EmpID ORDER BY SaleDate) AS PrevAmount FROM Sales;
+-------+---------+-------+--------+------------+
| EmpID | EmpName | Month | Amount | PrevAmount |
+-------+---------+-------+--------+------------+
|   101 | Alice   | Jan   |   1000 |       NULL |
|   101 | Alice   | Feb   |   2000 |       1000 |
|   101 | Alice   | Mar   |   1500 |       2000 |
|   102 | Bob     | Jan   |   1200 |       NULL |
|   102 | Bob     | Feb   |   1800 |       1200 |
|   103 | Charlie | Jan   |   7000 |       NULL |
|   103 | Charlie | Feb   |   7500 |       7000 |
|   104 | David   | Jan   |   7200 |       NULL |
+-------+---------+-------+--------+------------+
8 rows in set (0.00 sec)
```

PES UNIVERSITY

5. **Rank Employees by Sale Amount within Department**

SELECT EmpID, EmpName, Department, Amount, RANK() OVER (PARTITION BY Department ORDER BY Amount DESC) AS RankInDept FROM Sales;

```
mysql> SELECT EmpID, EmpName, Department, Amount, RANK() OVER (PARTITION BY Department ORDER BY Amount DESC) AS RankInDept FROM Sales;
+-------+---------+------------+--------+------------+
| EmpID | EmpName | Department | Amount | RankInDept |
+-------+---------+------------+--------+------------+
|   101 | Alice   | HR         |   2000 |          1 |
|   102 | Bob     | HR         |   1800 |          2 |
|   101 | Alice   | HR         |   1500 |          3 |
|   102 | Bob     | HR         |   1200 |          4 |
|   101 | Alice   | HR         |   1000 |          5 |
|   103 | Charlie | IT         |   7500 |          1 |
|   104 | David   | IT         |   7200 |          2 |
|   103 | Charlie | IT         |   7000 |          3 |
+-------+---------+------------+--------+------------+
8 rows in set (0.00 sec)
```

References

1) MySQL Window Functions Reference
2) SQL Server Window Functions (Microsoft Docs)
3) Geeks for Geeks

THANK YOU

Mahitha G,Revathi G P
Department of Computer Science and Engineering

mahithag@pes.edu,revathigp@pes.edu

Database Management Systems

Full Text Search in MySQL

Prof. Mahitha G,Prof. Revathi G P

Department of Computer Science and Engineering

Acknowledgment: Dr. Nagasundari S (Dept of CSE), Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G.

Department of Computer Science and Engineering

Database Management Systems

Full Text Search in MySQL

PES UNIVERSITY

Full-Text Search is a feature in MySQL where you can search for words or phrases in large text-based columns (CHAR, VARCHAR, TEXT) using natural language processing (NLP). Instead of just checking if a string is present, full-text search **looks for relevant matches** and **ranks results based on how well they match the search terms**.

Full-Text Search in MySQL allows you to perform advanced text-based searches on columns with textual data. It is particularly useful for searching large text fields like articles, descriptions, or comments. It is more powerful than **LIKE/REGEXP**, since it ranks results by relevance.

Features -

- **Full-Text Indexes** → much faster than **LIKE**
- **Search Modes** → Natural Language, Boolean, Query Expansion
- **Relevance Ranking** → Scores based on closeness of match
- Handles stopwords, synonyms, and word proximity.

Database Management Systems

Full Text Search in MySQL

MySQL Full-Text search provides a way to implement various advanced search technique such as **Natural Language Search**, **Boolean Text Search** and **Query Expansion**.

The key feature of Full Text Search is that it uses the **MATCH(column) AGAINST('search terms')** syntax to search.

Suppose you have a table called ‘Product’ with different columns such as product_id, product_name, description and price. Suppose you want to search for the products that have "smartphone" in their name or description. The SQL query for that would be :

```
SELECT * FROM products
WHERE MATCH(product_name, description)
AGAINST('smartphone' IN BOOLEAN MODE);
```

Here,
MATCH function is used to perform full text search on product_name and description column.
AGAINST specifies the search term "smartphone" and in boolean mode.

Database Management Systems

Full Text Search vs Normal String Search

PES UNIVERSITY

Feature	LIKE/REGEXP	FULL TEXT SEARCH
Index Usage	Does not use indexes effectively → Slow for large data	Uses special indexes → Faster searches
Relevance Ranking	No relevance ranking, just returns matches	Results ranked by relevance
Natural Language Support	Not supported	Supported (Natural Language Mode)
Word Boundary Detection	Not accurate (matches substrings like “cat” in “concatenate”)	Accurate word boundary detection
Boolean Logic	Not supported	Supported (AND, OR, NOT operators)
Phrase Searching	Possible but imprecise	Supported and more accurate

Features of Full Text Search

Some more features of MySQL Full-Text Search include :

1. **Natural Language Search**: This means that it can understand user query in more human-like ways, taking into account the word proximity , synonyms, and stop words.

1. **Moderate Index size**: The size of a FULL-TEXT index is relatively small.

1. **Boolean Search**: MySQL full-text search also supports boolean operators like "AND", "OR" and "NOT" which allows users to create more complex search queries.

1. **Native SQL like Interface**: MySQL provides the SQL- like statement to perform full-text search.

1. **Rich-Querying Capabilities**

Working of Full Text Search

By default or with the **IN NATURAL LANGUAGE MODE** modifier, the **MATCH()** function performs a natural language search for a string against a text collection.

A collection is a set of one or more columns included in a FULLTEXT index. The search string is given as the argument to **AGAINST()**.

For each row in the table, MATCH() returns a relevance value; that is, a similarity measure between the search string and the text in that row in the columns named in the MATCH() list.

Working :

1. Use columns of type VARCHAR or TEXT
2. Create a **FULLTEXT INDEX** on the column(s)
3. Use MATCH(column1, column2) with AGAINST('word')

Database Management Systems

Working of Full Text Search

Example :

```
mysql> CREATE TABLE posts (
    ->         id INT UNSIGNED AUTO_INCREMENT NOT NULL PRIMARY KEY,
    ->         title VARCHAR(200),
    ->         body TEXT
    ->       ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.204 sec)
```

```
mysql> INSERT INTO posts (title,body) VALUES
    ->         ('MySQL Tutorial','DBMS stands for DataBase ...'),
    ->         ('How To Use MySQL Well','After you went through a ...'),
    ->         ('Optimizing MySQL','In this tutorial, we show ...'),
    ->         ('1001 MySQL Tricks','1. Never run mysqld as root. 2. ...'),
    ->         ('MySQL vs. YourSQL','In the following database comparison ...'),
    ->         ('MySQL Security','When configured properly, MySQL ...');
Query OK, 6 rows affected (0.029 sec)
Records: 6  Duplicates: 0  Warnings: 0
```

Database Management Systems

Working of Full Text Search

PES UNIVERSITY

```
mysql> SELECT * FROM posts
    ->       WHERE MATCH (title,body)
    ->       AGAINST ('database' IN NATURAL LANGUAGE MODE);
+----+--------------------+----------------------------------------+
| id | title              | body                                   |
+----+--------------------+----------------------------------------+
|  1 | MySQL Tutorial     | DBMS stands for DataBase ...           |
|  5 | MySQL vs. YourSQL  | In the following database comparison ... |
+----+--------------------+----------------------------------------+
2 rows in set (0.022 sec)
```

Working of Full Text Search

Explanation :

- We perform a **full-text search** in **natural language mode** for the word "database".
- It searches the title and body fields for relevance to the term "database".
- Returns rows where the relevance score is **non-zero** (i.e., the word "database" appears and is significant).

Natural Language Mode **:**

1. Uses natural language processing to rank results based on relevance.
2. Ignores **stopwords** (like "a", "the", etc.) and very short words (less than 4 characters, by default).
3. **Does not require special syntax** (unlike BOOLEAN mode).

Modes of Full Text Search

Mode	Description
Natural Language Mode	Default mode; ranks relevance
Boolean Mode	Allows operators (+, -, *) for complex queries
Query Expansion Mode	Expands query terms based on top matches

Relevance

In MySQL Full-Text Search, **Relevance** is a numeric score that tells **how closely a row matches the search terms**.

It's a **floating-point number** (0.0, 0.25, 1.2, etc.)

Higher Score = Better Match

Relevance is calculated using:

1. How often the search term appears in the column
2. How rare the term is across all rows.
3. Length of the document (shorter docs = higher relevance if term is present)

Relevance

Example :

```
SELECT id, title,
    MATCH(title, body) AGAINST('database') AS relevance
FROM posts
WHERE MATCH(title, body) AGAINST('database')
ORDER BY relevance DESC;
```

What it does:

1. Selects matching rows which match against the word ‘database’
2. Shows the relevance score
3. Orders results by relevance descending (highest match first)

Relevance

Now: What If You Don’t Mention Relevance?

SELECT id, title FROM posts WHERE MATCH(title, body) AGAINST('database');

What happens:
Results are still matched using full-text search.
MySQL will still use the **FULLTEXT index**.

By default, **MySQL will sort the results by relevance**, **if and only if**:

1. You **don’t use any ORDER BY**.
2. You’re **using MATCH(...) AGAINST(...) in the WHERE clause**.
3. The **FULLTEXT index** is being used (not bypassed).
4. It’s a **simple query (no joins)**.

So yes — even **without ORDER BY or showing relevance**, the **rows will still be sorted by highest relevance**, **automatically**, under those conditions.

Important Note

If you **add any other ORDER BY clause**, MySQL will stop auto-sorting by relevance.

SELECT id, title FROM posts WHERE MATCH(title, body) AGAINST('database') ORDER BY id; -- Will sort by id, not relevance

Boolean Mode

What is BOOLEAN MODE?

BOOLEAN MODE allows you to:

1. Use **operators** (+, -, ", *, etc.) to include, exclude, or group terms.
2. Search for **partial words** using wildcards (*).
3. Write **complex search logic** (AND, OR, NOT-like behavior).
4. Search even if the word is a **stopword** or appears in **every row**.

You must **explicitly write**:
AGAINST ('your_query' IN BOOLEAN MODE)

Syntax Example

SELECT * FROM posts WHERE MATCH(title, body) AGAINST('+mysql -database' IN BOOLEAN MODE);
-- Here, we are including ‘mysql’ and excluding ‘database;

Database Management Systems

Boolean Mode

PES UNIVERSITY

Common Boolean Operators

✅ Common Boolean Operators

Operator	Meaning	Example
`+`	Word **must** be present	`+mysql` (must contain "mysql")
`-`	Word **must not** be present	`-database` (must not contain "database")
No operator	Word is **optional**, but affects relevance	`mysql database`
`"`	Search **exact phrase**	`"mysql tutorial"`
`*`	**Wildcard** (suffix only)	`data*` (matches "data", "database")
`()`	**Group terms**	`+(mysql database)`
`< >`	Boost/penalize relevance	`>mysql <oracle`

Boolean Mode

Example :

Suppose we have a table ‘Posts’ and we insert these values into it

```
INSERT INTO posts (title, body) VALUES
('MySQL Tutorial', 'Learn the basics of database systems'),
('Advanced Database', 'We explore indexing and performance'),
('Cooking Tips', 'Always use fresh ingredients'),
('MySQL vs PostgreSQL', 'Comparison of two database systems'),
('Oracle Tips', 'Learn how to use Oracle effectively');
```

Boolean Mode

PES UNIVERSITY

Example Queries in Boolean Mode

1. Include one word, exclude another

SELECT * FROM posts WHERE MATCH(title, body) AGAINST('+mysql -oracle' IN BOOLEAN MODE);

Must contain "mysql", must **not** contain "oracle"

2. Exact Phrase Match

SELECT * FROM posts WHERE MATCH(title, body) AGAINST('"database systems"' IN BOOLEAN MODE);

Must contain the **exact phrase** 'database systems'

PES UNIVERSITY

3. Prefix Search with Wildcard

SELECT * FROM posts WHERE MATCH(title, body) AGAINST('data*' IN BOOLEAN MODE);

Matches: "data", "database", "databases", etc.

Only suffix wildcards are supported (e.g., data*), not *data.

4. Grouping Words

SELECT * FROM posts WHERE MATCH(title, body) AGAINST('+(mysql database) -oracle' IN BOOLEAN MODE);

Database Management Systems

Boolean Mode - Practical Exercise

Here's a practical exercise on **Full-Text Search in MySQL** using Boolean and Natural Language Modes. It has :

- A sample table 'articles'
- Inserted data
- Practice questions ranging from basic to advanced

CREATE TABLE articles (article_id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(255), content TEXT, FULLTEXT (title, content));

INSERT INTO articles (title, content) VALUES
('Introduction to Databases', 'Databases are organized collections of data. MySQL is a popular open-source database.'),
('MySQL Optimization Techniques', 'Learn indexing, query plans, and caching to speed up MySQL queries.'),
('Oracle vs MySQL', 'Comparison of Oracle and MySQL databases based on performance and cost.'),
('PostgreSQL Guide', 'PostgreSQL is an advanced open-source RDBMS.'),
('Data Science and SQL', 'SQL plays a key role in data science workflows.'),
('MySQL Tutorial for Beginners', 'This MySQL tutorial covers basic to intermediate topics for new developers.'),
('Cooking for Beginners', 'This guide is about basic cooking techniques and not related to databases.');

Database Management Systems

Boolean Mode - Practical Exercise

PES UNIVERSITY

1. Retrieve all articles that mention the word **“MySQL”** using natural language mode.

1. Get the relevance score for the term **"MySQL"**.

1. Find articles that **must include "MySQL"** but **must not include "Oracle"**.

1. Find articles that contain **either "tutorial" or "guide"**.

1. Find articles that **must contain “data”** and any word that starts with **"scien"** (like "science", "scientific").

(Solutions are mentioned in the notes!)

THANK YOU

Mahitha G,Revathi G P
Department of Computer Science and Engineering

mahithag@pes.edu,revathigp@pes.edu

Database Management Systems

Informal Design Guidelines for Schemas

Prof. Mahitha G

Dept of Computer Science and Engineering

Acknowledgments: Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 3: Design Concepts and Implementation

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G

Dept of Computer Science and Engineering

PES UNIVERSITY

What is relational database design?

- The grouping of attributes to form "good" relation schemas

Two levels of relation schemas

- The logical or conceptual level
- The storage or implementation level

Two types of Design Methodology

- Bottom up Design (Design by Synthesis)
- Top down Design (Design by Analysis)

Database Management Systems

Design Methodologies

if you start from bottom you willl have to collect binary relationship between all the leaf nodes in a pair of 2 which becomes cumbersome over the period of time

- A **bottom-up design methodology** considers the basic relationships among individual attributes as the starting point and uses those to construct relation schemas. *This approach is not very popular because it suffers from the problem of having to collect a large number of binary relationships among attributes as the starting point.* For practical situations, it is next to impossible to capture binary relationships among all such pairs of attributes.
- In contrast, a **top-down design methodology starts with a number of groupings of attributes into relations** that exist together naturally, **for example**, on an invoice, a form, or a report. The relations are then analyzed individually and collectively, leading to further decomposition until all desirable properties are met.

in this one you just analyse stuff from top to bottom and naturally find the necessaary relationship and groupings

Informal Measures of Quality

PES UNIVERSITY

- Design is concerned mainly with base relations
- What are the criteria for "good" base relations?

Four Informal measures of quality for relation schema design:

- Making sure that the semantics of the attributes are clear in the schema
- Reducing the redundant information in tuples
- Reducing the NULL values in tuples
- Disallowing the possibility of generating spurious tuples

These measures are not always independent of one another

Guideline 1 - Clear Semantic Attributes in the Schema

semantics usually mean meaning

PES UNIVERSITY

Guideline-01:

Informally, each **tuple in a relation should represent one entity or relationship instance**. (Applies to individual relations and their attributes).

- **Semantics of a relation** refers to the interpretation of attributes values in a tuple.
- The ease with which the **meaning of a relation's attributes can be explained** is an **informal measure** of how well the relation is designed.
- Attributes of different entities (EMPLOYEEs, DEPARTMENTs, PROJECTs) should not be mixed in the same relation
- Only foreign keys should be used to refer to other entities
- **Entity and relationship attributes should be kept apart** as much as possible.

Guideline 1 - Clear Semantic Attributes in the Schema

PES UNIVERSITY

- Design a relation schema so that it is **easy to explain its meaning.**
- Intuitively, if a relation schema corresponds to one entity type or one relationship type, it is straightforward to explain its meaning.
- Otherwise, if the **relation corresponds to a mixture of multiple entities** and relationships, **semantic ambiguities will result and the relation cannot be easily explained.**

Database Management Systems

Guideline 1 - Clear Semantic Attributes in the schema

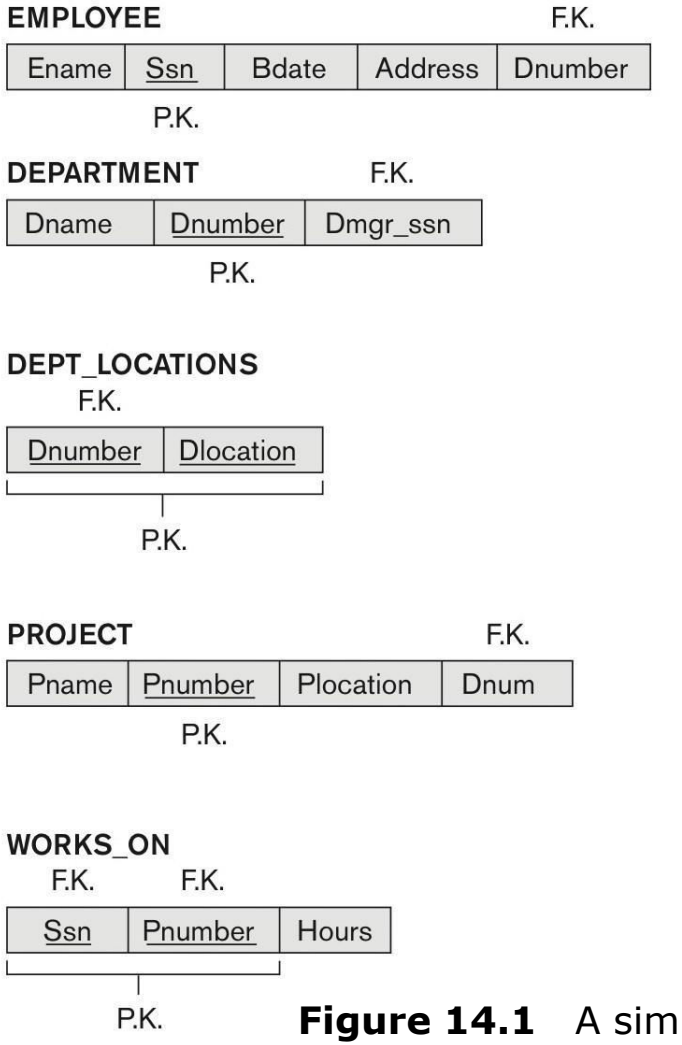


Figure 14.1 A simplified COMPANY relational database schema.

entity: real word object, eg. project ,employee , department

now they are saying we should be able to know clear meaning of attributes meaning
-differnt object(entity) 's attributes shouldn't be mixed together
-for referring other entities use foreing keys should be used
-entity and relationship attributes should be kept away
-

see in the diagram ,this shcema follows all the rules

Database Management Systems

Guideline 2 - Redundant Information in Tuples and Update Anomalies

PES UNIVERSITY

What is the problem with Information stored redundantly?

- Wastes storage
- Causes problems with:
- Update anomalies: **Storing natural joins of base relations leads to an additional problem referred to as update anomalies.** These can be classified into

➢ insertion anomalies,

➢ deletion anomalies, and

➢ modification anomalies

Update anomalies

Occurs when data is **duplicated** in multiple places, and an update in one place requires updates in all others. If not updated everywhere, data becomes inconsistent.

StudentID	StudentName	Course	Instructor
101	Alice	DBMS	Prof. Mehta
102	Bob	DBMS	Prof. Mehta
103	Charlie	OS	Prof. Rao

Now, if **Prof. Mehta** changes to **Prof. Sharma**,
we must update **all rows** where the instructor is "Prof. Mehta."
If we forget one row → inconsistent data.
→ **Problem:** Redundant data causes inconsistency during updates.

Insertion anomalies

Suppose a **new course "AI"** is introduced but no student has enrolled yet.
We **cannot insert the course "AI"** because *StudentID* and *StudentName* fields cannot be null.

StudentID	StudentName	Course	Instructor
✗	✗	AI	Prof. Gupta

Problem: You can't insert valid info unless you have unrelated data.

Deletion anomalies

Occurs when **deleting data unintentionally removes other valuable information.**

Example:
If we delete the record of student **Charlie (ID 103)** who was the only one taking **OS**,we also lose the information that **Prof. Rao** teaches the **OS** course.

StudentID	StudentName	Course	Instructor
101	Alice	DBMS	Prof. Sharma
102	Bob	DBMS	Prof. Sharma
✗ Delete 103	Charlie	OS	Prof. Rao

→ **Problem:** Loss of essential data due to deletion of a single record.

Guideline 2 - Redundant Information in Tuples and Update Anomalies

PES UNIVERSITY

GUIDELINE 2:

- Design a **schema that does not suffer from the insertion, deletion and update anomalies.**
- If there are any anomalies present, then note them so that applications can be made to take them into account.
- Guidelines may sometimes have to be violated in order to improve the performance of certain queries.
- **If emp_dept is used as view – resolve anomalies using triggers/functions/procedures**

Database Management Systems

Sample states for EMP_DEPT and EMP_PROJ discussed in following Examples

Redundancy

EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

Redundancy Redundancy

EMP_PROJ

Ssn	Pnumber	Hours	Ename	Pname	Plocation
123456789	1	32.5	Smith, John B.	ProductX	Bellaire
123456789	2	7.5	Smith, John B.	ProductY	Sugarland
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford
333445555	20	10.0	Wong, Franklin T.	Reorganization	Houston

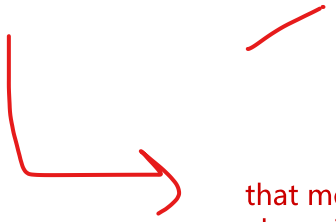
Update Anomaly

Consider the relation:

EMP_PROJ(Ssn, PNumber, Ename, Pname, Location)

Update Anomaly:

Changing the name of project number 1 from “ProductX” to “ProductA” may cause this update to be made for all 100 employees working on project P1.



that means all 100 rows need to be updated, if one also misses then inconsistent data

Update Anomaly

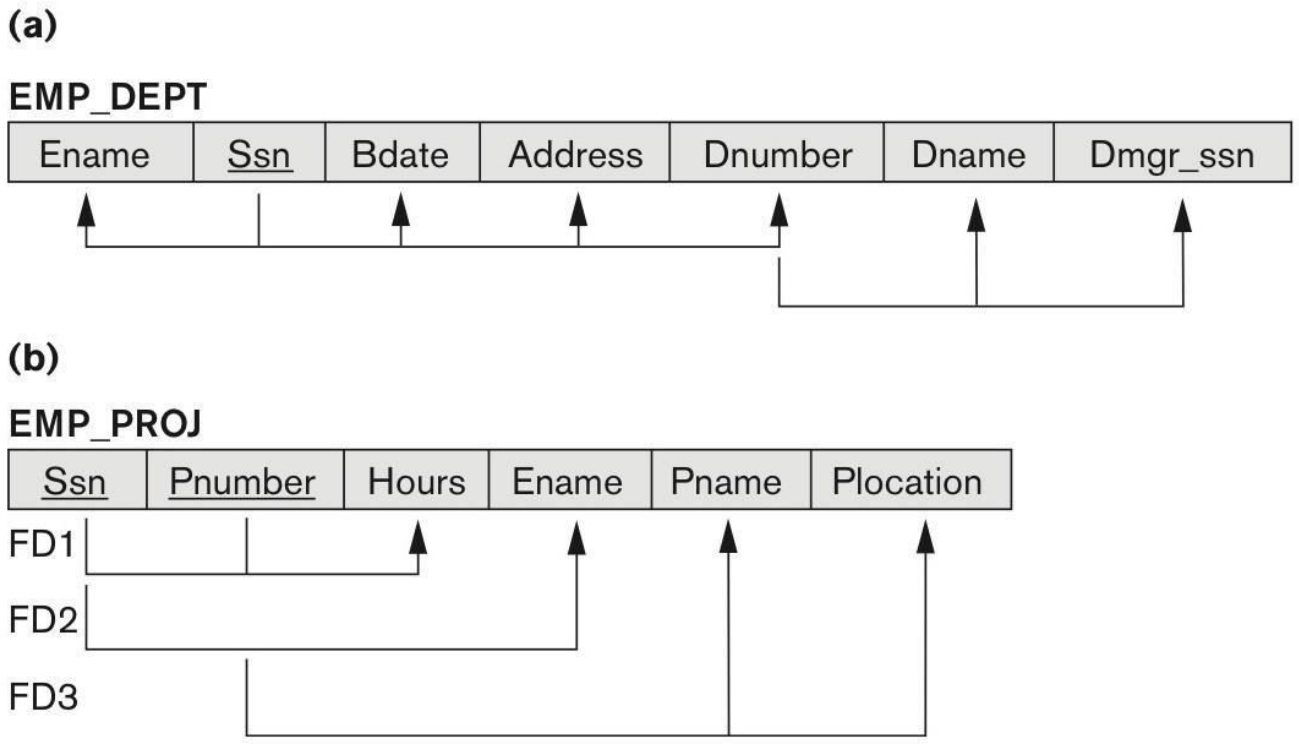


Figure 14.3
Two relation schemas suffering from update anomalies. (a) EMP_DEPT and (b) EMP_PROJ.

Insert Anomaly

Consider the relation:

EMP_PROJ(Ssn, PNumber, Ename, Pname, Location)

Insert Anomaly:

Cannot insert a project unless an employee is assigned to it.

Conversely,

Cannot insert an employee unless an he/she is assigned to a project.

can't keep primary key ssn and pnumber null right

Delete Anomaly

Consider the relation:

EMP_PROJ(Ssn, PNumber, Ename, Pname, Location)

Delete Anomaly:

When a project is deleted, it will result in deleting all the employees who work on that project.

Alternately, if an employee is the sole employee on a project, deleting that employee would result in deleting the corresponding project.

Guideline 3 - Null Values in Tuples

PES UNIVERSITY

GUIDELINE 3:

Relations should be designed such that **their tuples will have as few NULL values as possible.** If NULLs are unavoidable, make sure that they apply in exceptional cases only and do not apply to a majority of tuples in the relation.

- Attributes that are NULL frequently could be placed in separate relations (with the primary key)

Reasons for nulls:

- Attribute not applicable or invalid
- Attribute value unknown (may exist)
- Value known to exist, but unavailable

Guideline 3 - Null Values in Tuples

- For example, if only 15% of employees have individual offices, **there is little justification for including an attribute Office_number in the EMPLOYEE relation;** rather, a relation EMP_OFFICES(Essn, Office_number) can be created to include tuples for only the employees with individual offices } that reduces null values right

Guideline 4 - Generation of Spurious Tuples – avoid at any cost

- Bad designs for a relational database may result in erroneous results for certain JOIN operations
- The **"lossless join" property** is used to guarantee meaningful results for join operations

GUIDELINE 4:

Design relation schemas so that they can be joined with equality conditions on **attributes that are appropriately related (primary key, foreign key) pairs in a way that guarantees that no spurious tuples are generated.**

Avoid relations that contain matching attributes that are not (foreign key, primary key) combinations because joining on such attributes may produce spurious tuples

FOR CLEAR UNDERSTANDING OF THIS EXAMPLE REFER THIS:
file:///E:/CollegeWork/PESU_SEM_5/DBMS/Unit3/pdf/Reference/lossless_join.pdf

Guideline 4 - Generation of Spurious Tuples – avoid at any cost

NJB (Nonadditive Join Test for Binary Decompositions). A decomposition $D = \{R1, R2\}$ of R has the **lossless (nonadditive) join property** with respect to a set of functional dependencies F on R *if and only if* either

- The FD $((R1 \cap R2) \rightarrow (R1 - R2))$ is in F^{+15}, or
- The FD $((R1 \cap R2) \rightarrow (R2 - R1))$ is in F^{+}

TEACH

Student	Course	Instructor
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe
Narayan	Operating Systems	Ammar

FD1: {Student, Course} → Instructor
FD2:14 Instructor → Course

Relation is decomposed into one of the three following possible pairs:

1. R1 (Student, Instructor) and R2(Student, Course)
2. R1 (Course, Instructor) and R2(Course, Student)
3. R1 (Instructor, Course) and R2(Instructor, Student)

Imagine you have one big table, and you split it into two smaller tables.
Later you join them back.
✔ If you get the same original table,
👉 Lossless join (GOOD)
❌ If you get extra wrong rows,
👉 Spurious tuples → Lossy join (BAD)

👉 Lossless join = no spurious tuples
👉 Lossy join = spurious tuples appear → BAD

Guideline 4 - Generation of Spurious Tuples – avoid at any cost

(a)

EMP_LOCS

Ename	Plocation

P.K.

EMP_PROJ1

Ssn	Pnumber	Hours	Pname	Plocation

P.K.

basically while joining join should happen on primary-foreing key relation ensures lossless join

If you perform a natural join on the common attribute **Plocation**, the join condition is not based on a key–foreign key relationship(**Plocation is not in the PK in EMP_PROJ1**) and hence will produce spurious tuples.

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Functional Dependencies

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgments: Prof. Nivedita Kasturi(Dept of CSE), Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 3: Design Concepts and Implementation

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G

Dept of Computer Science and Engineering

Database Management Systems

Functional Dependencies

- Each relation schema consists of a number of attributes, and the relational database schema consists of a number of relation schemas.
- We have assumed that **attributes are grouped to form a relation schema by using the common sense of the database designer** or by mapping a database schema design from a conceptual data model such as the ER data model.
- we did not develop **any measure of appropriateness or goodness to measure the quality of the design**, other than the intuition of the designer.
- There are **two levels at which we can discuss the goodness of relation schemas**. The first is the **logical (or conceptual) level**—how users interpret the relation schemas and the meaning of their attributes. The second is the **implementation (or physical storage) level**—how the tuples in a base relation are stored and updated.

Database Management Systems

Functional Dependencies

WE SAW IN PREVIOU SLIDES top down USES analysis

- The theory is applicable primarily to **the top-down design approach**, and as such is more appropriate when performing design of databases by analysis and decomposition of sets of attributes that appear together in files, in reports, and on forms in real-life situations.
- **Relational database design ultimately produces a set of relations**. The implicit goals of the design activity are information preservation and minimum redundancy.
- **A functional dependency is a constraint between two sets of attributes from the database.**
- Suppose that our relational database schema has n attributes A1, A2, … , An; let us think of the whole database as being described by a single universal relation schema R = {A1, A2, … , An}.
- We **do not imply** that we will actually store the database as a single universal table;
- **We use this concept only in developing the formal theory of data dependencies**

they are using this to just make us understand

Database Management Systems

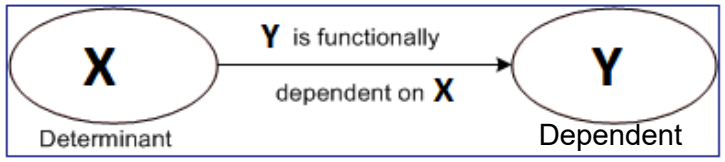
Functional Dependencies

- Some of the most commonly **used types of real-world constraints** can be represented formally as keys (super keys, candidate keys, and primary keys), or as functional dependencies

Functional dependencies (FDs)

- **Are used to specify *formal measures* of the "goodness" of relational designs**
- And keys are used to define **normal forms** for relations
- Are **constraints** that are derived from the *meaning* and *interrelationships* of the data attributes

- *The functional dependency (FD) is a relationship that exists between two attributes, Where one attribute can directly or indirectly derived from the other attribute.*
- **A set of attributes X *functionally determines* a set of attributes Y if the value of X determines a unique value for Y**



Functional dependency between X and Y

Defining Functional Dependencies

Candidate keys are all minimal sets of attributes that uniquely identify a tuple. One of them is chosen as the primary key.

- A functional dependency, *denoted by X → Y, between two sets of attributes X and Y that are subsets of R specifies a constraint on the possible tuples that can form a relation state r of R. The constraint is that, for any two tuples t1 and t2 in r that have t1[X] = t2[X], they must also have t1[Y] = t2[Y].*
- **The values of the X component of a tuple uniquely (or functionally) determine the values of the Y component. (Or) there is a functional dependency from X to Y, or that Y is functionally dependent on X.**
- ***X functionally determines Y in a relation schema R if, and only if, whenever two tuples of r(R) agree on their X-value, they must necessarily agree on their Y-value.***

Note the following:

- If a constraint on R states that there cannot be more than one tuple with a given X-value in any relation instance r(R)—that is, **X is a candidate key of R**—this implies that X → Y for any subset of attributes Y of R (because the key constraint implies that no two tuples in any legal state r(R) will have the same value of X). If X is a candidate key of R, then X → R. **If X → Y in R, this does not say whether or not Y → X in R**

Database Management Systems

Use of Functional Dependencies

Use functional dependencies in two ways:

1. To test instances of relations to see whether they satisfy a given set F of functional dependencies.
2. To specify constraints on the set of legal relations.

To Summarize

- X → Y in R specifies a *constraint* on all relation instances r(R)
- Written as X → Y; can be displayed graphically on a relation schema
- FDs are derived from the real-world constraints on the attributes
- A functional dependency is a **property of the semantics or meaning of the attributes.**
- **Relation extensions r(R)** that satisfy the functional dependency constraints are called legal relation states (or legal extensions) of R.
- For example, {State, Driver_license_number} → Ssn should normally hold for any adult in the United States and hence should hold whenever these attributes appear in a relation

Database Management Systems

Notational Conventions

- Set of attributes -> α
- Relate a Schema (R) for relation (r) -> r(R)
- If a set of attributes is a super key -> K
- We use a lowercase name for relations -> “teacher”

For simplicity, we assume that attribute names have only one meaning within the database schema.

A superkey is:
Any set of attributes that can uniquely identify a row in a table.

🧠 Key Points to Understand
✔ Superkey can be one attribute or multiple attributes
As long as they uniquely identify a tuple.
✔ Superkeys may have extra unnecessary attributes
This is very important.
Example:
If Email uniquely identifies a student, then:
- {Email} → superkey
- {Email, Phone} → also a superkey
- {Email, Name, Phone} → still a superkey (but redundant)

✔ Candidate keys are minimal superkeys
Meaning:
- No extra attribute inside
- Pure, smallest unique identifier

Primary key is chosen from candidate keys.

🔥 Quick Example
Consider table:
STUDENT(SID, Email, Phone, Name)
Assuming all these are unique:
Superkeys
- {SID}
- {Email}
- {Phone}
- {SID, Email}
- {Email, Phone}
- {SID, Email, Phone, Name}
- And many more…

Candidate keys
(minimal superkeys — no extra attribute)
- {SID}
- {Email}
- {Phone}

Primary key
(we choose one)
- {SID}

Database Management Systems

Defining Functional Dependencies

Employee_ID	Employee_Name	Department	Salary
101	John	HR	50000
102	Mary	Finance	60000
103	Sarah	HR	55000

In this example, let's say we want to explore the functional dependency "Department → Salary," which means that for any two tuples with the same value for the "Department" attribute, they must have the same value for the "Salary" attribute.

Is the functional dependency satisfied?

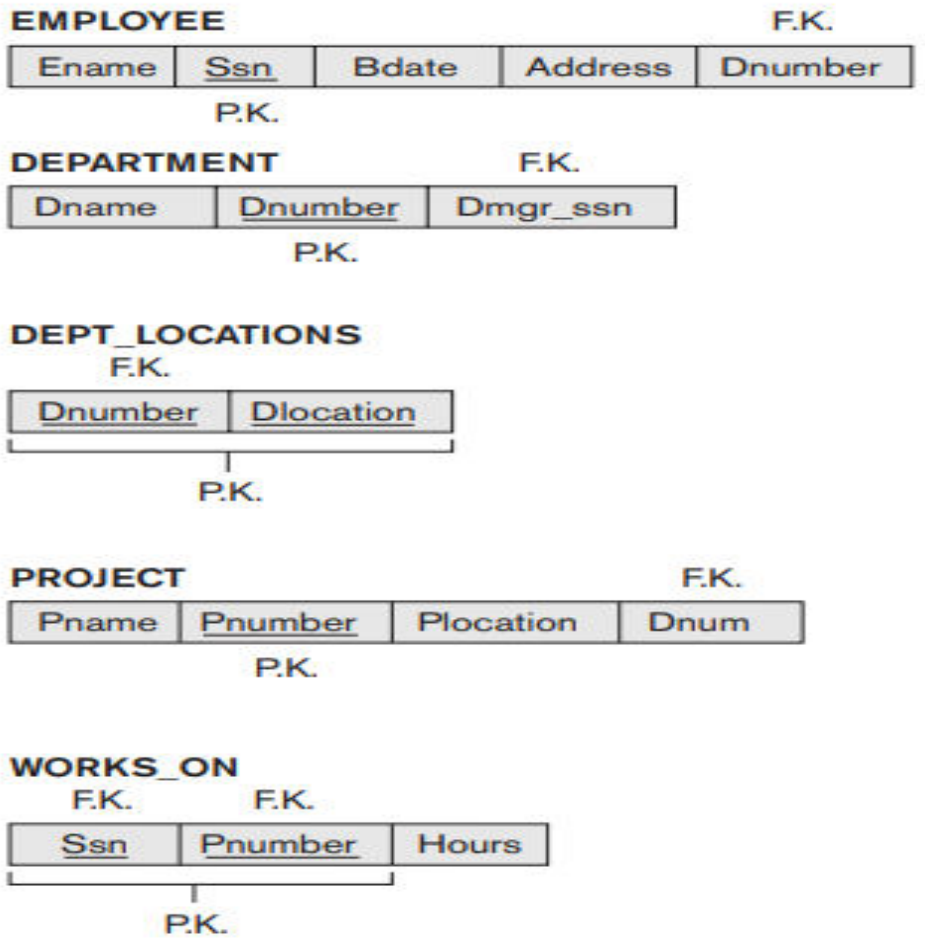
here we see john and sarah being from same dept HR have different salaries hence this FD is not satisfied

wkt if X-->Y
then if t1[X]=t2[X] => t1[Y]=t2[Y]

Database Management Systems

Examples of FD constraints (1)

- Social security number determines employee name
  - SSN → ENAME
- Project number determines project name and location
  - PNUMBER → {PNAME, PLOCATION}
- Employee ssn and project number determines the hours per week that the employee works on the project
  - {SSN, PNUMBER} → HOURS



Database Management Systems

Examples of FD constraints (2)

- **The main use of FD is to describe further a relational schema R by specifying constraints on its attributes that must hold all the times.**

- An FD is a property of the attributes in the schema R

- **The constraint must hold on *every* relation instance r(R)**

- If K is a key of R, then **K functionally determines all attributes in R**
  - (since we never have two distinct tuples with t1[K]=t2[K])

here K is superkey basically meaning keys that uniquely identifies , that means if department is HR then salary should be always 50000,we can't have two different salaries with same key that' s what they are saying if t1[K]=t2[K] then we can never have different results it has to be the same result or row

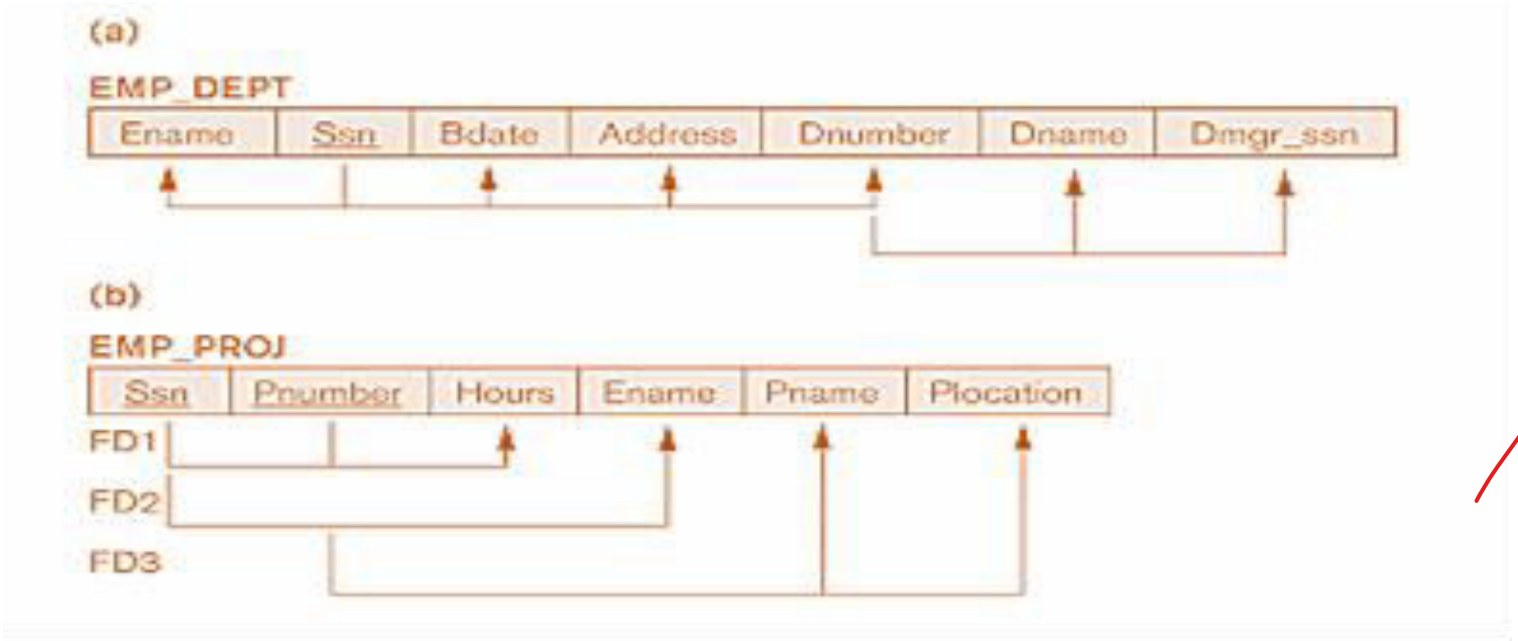
Database Management Systems

Diagrammatic Notation for displaying FDs

- Each FD is displayed as a horizontal line.
- The left hand side attributes are connected by vertical lines to the line representing the FD, whereas the right hand side attributes are connected by the lines with arrows pointing toward the attributes.

finally it should look like

Try and take a guess at what these FDs are?



Database Management Systems

Types of Functional Dependency

- An attribute of relation schema R is called a **prime attribute** of R if it is a member of some candidate key of R.
- An attribute is called **non prime** if it is not a prime attribute—that is, if it is not a member of any candidate key.
- Both Ssn and Pnumber are prime attributes of WORKS_ON, whereas other attributes of WORKS_ON are nonprime

Full Functional Dependency:

A functional dependency X → Y is a full functional dependency if removal of any attribute A from X means that the dependency does not hold anymore; that is, for any attribute A ε X, (X − {A}) does not functionally determine Y.

belongs to symbol

Database Management Systems

Types of Functional Dependency

- A functional dependency is said to be full dependency "if and only if the determinant of the functional dependency if either candidate key or super key, and the dependent can be either prime or non-prime attribute".

left side of arrow wala part

right side of arrow wala part

Example: Consider the following determinant ABC → D

i.e., ABC determines D

BC→D, C→D, A→D

ass mentioned earlier removal of any attribute from left side means it's not fully functional dependent anymore

These subsets of ABC, cannot determine D ✓

So D is Fully Functional Dependent.

- {Ssn, Pnumber} → Hours is a full dependency (neither Ssn → Hours nor Pnumber → Hours holds).

Types of Functional Dependency

Partial Functional Dependency:

- A functional dependency X → Y is a partial dependency "if some attribute A ε X can be removed from X and the dependency still holds; that is, for some A ε X, (X – {A}) → Y".
- However, the dependency If a non-prime attribute of the relation is getting derived by only a part of the candidate key, then such dependency is known as Partial Dependency.

Example: Consider the following

{Ssn, Pnumber} → Ename is partial because Ssn → Ename holds.

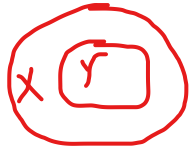
Types of Functional Dependency

- **Transitive Functional Dependency:**

  If a non-prime attribute of a relation is getting derived by either another non-prime attribute or the combination of the part of the candidate key along with non-prime attribute, then such dependency is defined as Transitive dependency. i.e., in a relation, there may be dependency among non-key fields. Such dependency is called Transitive Functional Dependency.

  Example: X→Y, and Y→Z Determine X→Z holds.

Types of Functional Dependency



- **Trivial Functional Dependency:**

  It is basically related to Reflexive rule. i.e., if X is a set of attributes, and Y is subset of X then X→Y holds.

  Example: ABC→BC is a Trivial Dependency,

  Wherein X{ABC} and Y{BC}

- **Multi-Valued Dependency:**

  Consider 3 fields X, Y, and Z in a relation. If for each value of X, there is a well-defined set of values Y and Well-defined set of values of Z and set of values of Y is independent of the set values of Z. This dependency is Multi-valued Dependency. i.e.,

  X → → Y / Z.

  CAR MODEL INDEPENDENTLY DETERMINES MANUF MONTH AS WELL AS CAR MODEL INDEPENDENTLY DETERMINES COLOUR
  THAT MEANS MONTH AND COLOUR ARE INDEPNEDENT OF EACH OTHER

  CAR_MODEL → → MANUF_MONTH CAR_MODEL → → COLOUR

Database Management Systems

Ruling Out FDs

Consider the following TEACH relation.
we can say that the FD: Text → Course may exist.

because for value of text they have exactly one value of course

However, the FDs Teacher → Course, Teacher → Text, Course → Text are ruled out.
(This is because the element on the LHS have similar values in the current column, but are not the same on the RHS column)

here these all have multiple values of Y for one value of X so ruled out

TEACH

Teacher	Course	Text
Smith	Data Structures	Bartram
Smith	Data Management	Martin
Hall	Compilers	Hoffman
Brown	Data Structures	Horowitz

CAR_MODEL	MANUF_MONTH	COLOUR
Swift	Jan	Red
Swift	Jan	Blue
Swift	Feb	Red
Swift	Feb	Blue
i20	Mar	White
i20	Mar	Black
i20	Apr	White
i20	Apr	Black

What FDs may exist?

- A relation *R*(A, B, C, D) with its extension.
- The following FDs may hold because the 4 tuples in the current extension have no violation of these constraints: B -> C; C->B; {A,B} -> C; {A,B} -> D; and {C,D} -> B
- The following do not hold: A-> B, B-> A ; D -> C

A	B	C	D
a1	b1	c1	d1
a1	b2	c2	d2
a2	b2	c2	d3
a3	b3	c4	d3

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Inference Rules and Closure

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgments: Prof. Nivedita Kasturi (Dept of CSE), Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 3: Design Concepts and Implementation

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G

Dept of Computer Science and Engineering

Inference Rules for FDs (1)

- **Definition:** An *FD X → Y is **inferred from** or **implied by** a set of dependencies F specified on R if X → Y holds in every legal relation state r of R; that is, whenever r satisfies all the dependencies in F, X → Y also holds in r.*
- Given a set of FDs F, **we can infer additional FDs** that hold whenever the FDs in F hold.
- Examples : if each department has one manager, so that Dept_no uniquely determines Mgr_ssn (Dept_no → Mgr_ssn), and a manager has a unique phone number called Mgr_phone (Mgr_ssn → Mgr_phone), then these two dependencies together imply that Dept_no → Mgr_phone. This is an inferred or implied FD and need not be explicitly stated in addition to the two given FDs.

Database Management Systems

Inference Rules for FDs (2)

PES UNIVERSITY

Armstrong's inference rules:

- IR1. (**Reflexive**) If Y is a *subset-of* X, then X → Y always holds.
- IR2. (**Augmentation**) If X → Y, then XZ → YZ always holds.
  (Notation: XZ stands for X U Z)
  Example: if {Ssn} → {Ename} then {Ssn, Bdate}→ {Ename, Bdate}
- IR3. (**Transitive**) If X → Y and Y → Z, then X → Z always holds.
  Example: if {Ssn} → {Dnumber} and {Dnumber} → {Dname} then {Ssn}→ {Dname}

- IR1, IR2, IR3 form a **sound** and **complete** set of inference rules
- These are rules hold and all other rules that hold can be deduced from these

NOTE: By applying these rules repeatedly, **we can find all of F+,** given F. **This collection of rules is called Armstrong's axioms (Sound + Complete)**

Database Management Systems

Inference Rules for FDs (3)

- Some additional inference rules that are useful:
- **Decomposition**: If X → YZ, then X → Y and X → Z
- **Union/Additive**: If X → Y and X → Z, then X → YZ

  Example: if {Ssn} → {Ename} and {Ssn} → {Address} then {Ssn}→ {Ename, Address}
- **Pseudo-Transitivity**: If X → Y and WY → Z, then WX → Z
- **The last three inference rules, as well as any other inference rules, can be deduced from IR1, IR2, and IR3 (completeness property)**
- One important cautionary note regarding the use of these rules: **Although X → A and X → B implies X → AB by the union rule stated above, X → A and Y → B does not imply that XY → AB, Also, XY → A does not necessarily imply either X → A or Y → A.**

YOU CAN BREAK THINGS ONLY ON THE RIGHT SIDE OF ARROW
YOU CAN NOT BREAK THINGS ON LEFT SIDE OF ARROW

✔ Only dependencies with the SAME LHS can union

Closure

- Suppose we are given a relation schema r(A, B, C, G, H, I) and the set of functional dependencies: A → B A → C CG → H CG → I B → H
- The functional dependency: **A → H is logically implied.**
- That is, we can show that, whenever a relation instance satisfies our given set of functional dependencies, **A → H must also be satisfied by that relation instance.**
- Suppose that t1 and t2 are tuples such that: **t1[A] = t2[A]** Since we are given that A → B, it follows from the definition of functional dependency that: **t1[B] = t2[B]** Then, since we are given that B → H, it follows from the definition of functional dependency that: **t1[H] = t2[H]**
- Therefore, we have shown that, whenever t1 and t2 are tuples such that **t1[A] = t2[A]**, it must be that **t1[H] = t2[H]**. But that is exactly the definition of A → H.

Algorithm to determine Closure of X under F

- **Determining X+, the Closure of X under F**

Input: A set F of FDs on a relation schema R, and a set of attributes X, which is a subset of R.

X+ := X;

repeat

oldX+ := X+;

for each functional dependency Y → Z in F do

if X+ ⊇ Y then X+ := X+ ∪ Z;

until (X+ = oldX+);

Database Management Systems

Closure of Attribute Sets

For example, consider the following relation schema about classes held at a university in a given academic year.
CLASS (Classid, Course#, Instr_name, Credit_hrs, Text, Publisher, Classroom, Capacity).

Let F, the set of functional dependencies for the above relation include the following f.d.s: FD1: Classid→ Course#, Instr_name, Credit_hrs, Text, Publisher, Classroom, Capacity; FD2: Course# → Credit_hrs;
FD3: {Course#, Instr_name} → Text, Classroom;
FD4: Text → Publisher
FD5: Classroom → Capacity

Note that the above FDs express certain semantics about the data in the relation CLASS.

Database Management Systems

Closure of Attribute Sets

PES UNIVERSITY

For example, FD1 states that each class has a unique Classid.

FD3 states that when a given course is offered by a certain instructor, the text is fixed and the instructor teaches that class in a fixed room.
Using the inference rules about the FDs and applying the definition of closure,

we can define the following closures:
{ Classid } + = { Classid , Course#, Instr_name, Credit_hrs, Text, Publisher, Classroom, Capacity } = CLASS
{ Course#} + = { Course#, Credit_hrs}
{ Course#, Instr_name } + = { Course#, Instr_name,Credit_hrs, Text, Publisher, Classroom, Capacity }

Closure of Attribute Sets

- Note that each closure above has an interpretation that is revealing about the attribute(s) on the left-hand side.
- For example, the closure of Course# has only Credit_hrs besides itself. It does not include Instr_name because different instructors could teach the same course; it does not include Text because different instructors may use different texts for the same course.
- Note also that the **closure of {Course#, Instr_nam} does not include Classid, which implies that it is not a candidate key.**
- This further implies that a course with given Course# could be offered by different instructors, which would make the courses distinct classes.

Database Management Systems

Closure of Functional Dependencies

Let F be a set of functional dependencies. The closure of F, denoted by F+, is the set of all functional dependencies logically implied by F. Given F, we can compute F+ directly from the formal definition of functional dependency.

$F^+ = F$
apply the reflexivity rule /* Generates all trivial dependencies */
repeat
  for each functional dependency f in F^+
    apply the augmentation rule on f
    add the resulting functional dependencies to F^+
  for each pair of functional dependencies f_1 and f_2 in F^+
    if f_1 and f_2 can be combined using transitivity
      add the resulting functional dependency to F^+
until F^+ does not change any further

Figure 7.7 A procedure to compute F^+.

NOTE:

Since a set of size n has 2^n subsets, there are a total of **2^n × 2^n** = 2^(2n) possible functional dependencies, where n is the number of attributes in R

For a relation **R** with **n attributes**:
- The total number of possible subsets of attributes = 2^n
- A functional dependency has:
  - A subset on the left side (determinant)
  - A subset on the right side (dependent)

Therefore:
Total possible functional dependencies= 2^n * 2^n = 2^(2n)

applied all 3 armstrong rules

Database Management Systems

Closure of Functional Dependencies

- Example:

F = {Ssn → {Ename, Bdate, Address, Dnumber}, Dnumber → {Dname, Dmgr_ssn} }

- Some of the additional functional dependencies that we can infer from F are the following:

1. Ssn → {Dname, Dmgr_ssn}
2. Ssn → Ssn
3. Dnumber → Dname

- The closure F+ of F is the set of all functional dependencies that can be inferred from F.

- To determine a systematic way to infer dependencies, we need to use all inference rules learnt in previous lecture.

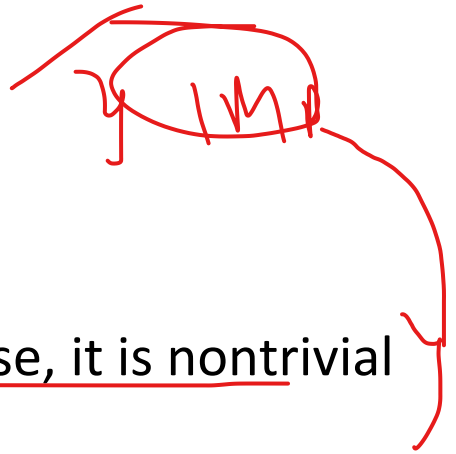
Database Management Systems

Closure of Functional Dependencies

- We can compute F+, the closure of F, by repeatedly applying Armstrong's Axioms:

1. Reflexive rule: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
2. Augmentation rule: if $\alpha \rightarrow \beta$, then $\gamma\ \alpha \rightarrow \gamma\ \beta$
3. Transitivity rule: if $\alpha \rightarrow \beta$, and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$

- These rules are
  Sound -- generate only functional dependencies that actually hold,
  Complete -- generate all functional dependencies that hold.



- Formally, a functional dependency $X \rightarrow Y$ is trivial if $X \supseteq Y$; otherwise, it is nontrivial

Database Management Systems

Example of Closure of Functional Dependencies

- R = (A, B, C, G, H, I)
  F = { A → B
  A → C
  CG → H
  CG → I
  B → H}
- Some members of F^+
  - A → H
    - by transitivity from A → B and B → H
  - AG → I
    - by augmenting A → C with G, to get AG → CG
      and then transitivity with CG → I
  - CG → HI
    - by augmenting G on both sides of CG → I to infer CG → CGI,
      and augmenting I on both sides of CG → H to infer CGI → HI,
      and then transitivity

you might think we could have got this with union of CG-->H, CG-->I , yeah you can do that , but here they are just tring to teach it with basic rules i.e armstrong you can use union as well because actually union is derived from armstrong rules only

Database Management Systems

Example of Closure of Attribute Sets

Consider a relation R (A , B , C , D , E , F , G) with the functional dependencies-

$$A \rightarrow BC$$
$$BC \rightarrow DE$$
$$D \rightarrow F$$
$$CF \rightarrow G$$

Now, let us find the closure of D and BC

Closure of attribute D-

$D^+ = \{ D \}$

$= \{ D , F \}$ (Using $D \rightarrow F$)

We can not determine any other attribute using attributes D and F contained in the result set.

Thus,

$$\mathbf{D^+ = \{ D , F \}}$$

Example of Closure of Attribute Sets

Closure of attribute set {B, C}-

$\{ B , C \}^+ = \{ B , C \}$
$= \{ B , C , D , E \}$ (Using $BC \rightarrow DE$)
$= \{ B , C , D , E , F \}$ (Using $D \rightarrow F$)
$= \{ B , C , D , E , F , G \}$ (Using $CF \rightarrow G$)
Thus,

$$\mathbf{\{ B , C \}^+ = \{ B , C , D , E , F , G \}}$$

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Practice Problems - Inference and Closure

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgments: Prof. Nivedita Kasturi (Dept of CSE), Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 3: Design Concepts and Implementation

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G

Dept of Computer Science and Engineering

Inference Rules - Problem 1

For the below given set F of functional dependencies:

F = { A -> B, BC -> D } **Derive:** AC -> D

Database Management Systems

Inference Rules - Problem 1

F = {

A -> B

BC -> D

}

Derive: AC -> D

Solution:

Step 1:

- Given: A -> B
- Augment both sides with C: AC -> BC

Step 2:

- Derived: AC → BC
- Given: BC → D
- By transitivity: AC → D

Database Management Systems

Inference Rules - Problem 2

For the below given set F of functional dependencies:

F = { A -> BC, B -> D, C -> E }

Derive: A -> DE

anotehr soln

decompose you get
A->B , A->C
now
B->D you already have then A->B, B->D
gives you A->D
now do union of A->D and A->C you get A->CD
now augment C->E with D you get CD->DE
hence you get A->CD->DE

Database Management Systems

Inference Rules - Problem 2

F = {

A -> BC

B -> D

C -> E

}

Derive: A -> DE

Solution:

Step 1:

- Given: A → BC
- By decomposition: A → B and A → C

Step 2:

- Derived: A → B
- Given: B → D
- By transitivity: A → D

Step 3:

- Derived: A → C
- Given: C → E
- By transitivity: A → E

Step 4:

- Derived: A → D
- Derived: A → E
- By union: A → DE

Database Management Systems

Inference Rules - Problem 3

For the below given set F of functional dependencies:

F = { AB -> C, C -> D, D -> E }

Derive: AB -> E

Database Management Systems

Inference Rules - Problem 3

F = {

AB -> C

C -> D

D -> E

}

Derive: AB -> E

Solution:

Step 1:

- Given: AB → C
- Given: C → D
- By transitivity: AB → D

Step 2:

- Derived: AB → D
- Given: D → E
- By transitivity: AB → E

Database Management Systems

Inference Rules - Problem 4

For the below given set F of functional dependencies:

F = { A -> B, A -> C, BC -> D } **Derive:** A → D

Database Management Systems

Inference Rules - Problem 4

F = {

A -> B

A -> C

BC -> D

Derive: A → D

Solution:

Step 1:

- Given: A → B
- Given: A → C
- By union: A → BC

Step 2:

- Derived: A → BC
- Given: BC → D
- By transitivity: A → D

Database Management Systems

Inference Rules - Problem 5

For the below given set F of functional dependencies:

F = { A -> B, C -> D, BD -> E } **Derive:** AC → E

Database Management Systems

Inference Rules - Problem 5

F = {

A -> B

C -> D

BD -> E

}

Derive: AC → E

Solution:

Step 1:

- Given A -> B
- Augment both sides with C: AC → BC

Step 2:

- Given C -> D
- Augment both sides with A: AC → AD

Step 3:

- By decomposition of AC→BC into AC→B and AC→C

Step 4:

- By decomposition on AC -> AD into AC → D and AC→A

Step 5:

- By union on AC → B and AC → D : AC → BD

Step 6:

- Derived AC → BD
- Given BD -> E
- By transitivity on: AC → E

Problem 6

Supposing we are given a relation R{A, B, C, D, E, F} with a set of FDs as shown below:

$A \rightarrow BC$

$B \rightarrow E$

$CD \rightarrow EF$

Let us **show that AD → F holds for R** and is a member of the closure.

(1) $A \rightarrow BC$	{Given, A->B & A->C}
(2) $A \rightarrow C$	{Decomposition of (1)}
(3) $AD \rightarrow CD$	{Augmentation of (2) by adding D}
(4) $CD \rightarrow EF$	{Given}
(5) $AD \rightarrow EF$	{Transitivity of (3) and (4)}
(6) $AD \rightarrow F$	{Decomposition of (5)}

Problem 7

Consider R = (Street, Zip, City) ; and the dependencies

F = { City Street -> Zip, Zip -> City }

Show that: Street Zip ->Street Zip City

1. Zip ->City – Given
2. Street Zip -> Street City – Augmentation of (1) by Street
3. City Street -> Zip – Given
4. City Street -> City Street Zip – Augmentation of (3) by City Street
5. Street Zip -> City Street Zip – Transitivity of (2) and (4)

Database Management Systems

Problem 8

Consider the relation schema <R,F> where R= (ABCDEGHI) and dependencies

F= { AB->E, BE->I, E ->G GI ->H }. **Show that AB ->GH is derived by F.**

1.	AB ->E		Given
2.	E ->G	Given	
3.	BE->I	Given	
4.	GI ->H		Given
5.	AB ->G		Transitivity on (1) and (2)
6.	AB -> BE		Augmentation (1) by B
7.	AB -> I		Transitivity on (6) and (3)
8.	AB -> GI		Union on (5) and (7)
9.	AB -> H		Transitivity on (8) and (4)
10.	AB -> GH		Union on (5) and (9)

Closure - Problem 1

Given the relation R (A, B, C, D, E, F, G) and set of functional dependency F { A -> B, BC -> DE, AEG -> G}.

Find the closure of A, AC and ADE

Closure - Problem 1

R (A, B, C, D, E, F, G)

F = { A -> B

BC -> DE

AEG -> G}

Find the closure of A, AC and ADE

Solution -

A+ = { A, B }

AC+ = { A, C, B, D, E }

ADE+ = { A, D, E, B }

Database Management Systems

Closure - Problem 2

Given the relation R (A, B, C, D, E) and set of functional dependency

F = { A -> BC, CD -> E, B -> D, E -> A}. **Find the closure of B**

Closure - Problem 2

R (A, B, C, D, E)

F = { A -> BC

CD -> E

B -> D

E -> A}

Find the closure of B

Solution -

B+ = { B, D }

Closure - Problem 3

Given R and F, Find the closure of BCD

R (A, B, C, D, E, F, G, H)

F = { A -> BC, CD -> E, D -> AEH, E -> C, ABH -> BD, DH -> BC}

Closure - Problem 3

R (A, B, C, D, E, F, G, H)

F = { A -> BC

CD -> E

D -> AEH

E -> C

ABH -> BD

DH -> BC}

Find the closure of BCD

Solution -

BCD+ = { B, C, D, E, A, H }

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Equivalence

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgments: Prof. Nivedita Kasturi(Dept of CSE), Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 3: Design Concepts and Implementation

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G

Dept of Computer Science and Engineering

Equivalence of Sets of FDs

- *A set of functional dependencies F is said to cover another set of functional dependencies E if every FD in E is also in F+; that is, if every dependency in E can be inferred from F*; alternatively, we can say that E is covered by F.
- **Two sets of functional dependencies E and F are equivalent if E+ = F+.**
- Equivalence means that every FD in E can be inferred from F, and every FD in F can be inferred from E; that is, **E is equivalent to F if both the conditions—E covers F and F covers E—hold.**
- We can determine **whether F covers E by calculating X+ with respect to F for each FD X → Y in E,** and then **checking whether this X+ includes the attributes in Y.**

Equivalence of Sets of FDs

Equivalence of Two Sets of Functional Dependencies-

- Two different sets of functional dependencies for a given relation may or may not be equivalent.
- If F and G are the two sets of functional dependencies, then following 3 cases are possible-

Case-01: F covers G (F ⊇ G)
Case-02: G covers F (G ⊇ F)
Case-03: Both F and G cover each other (F = G)

Equivalence of Sets of FDs

Case-01: Determining Whether F Covers G-

Following steps are followed to determine whether F covers G or not-

Step-01:

- **Take the functional dependencies of set G into consideration.**
- For each functional dependency X → Y, find the closure of X using the functional dependencies of set G.

Step-02:

- Take the functional dependencies of set G into consideration.
- **For each functional dependency X → Y, find the closure of X using the functional dependencies of set F.**

Step-03:

- Compare the results of Step-01 and Step-02.
- **If the functional dependencies of set F has determined all those attributes that were determined by the functional dependencies of set G,** then it means F covers G.
- Thus, we conclude F covers G (F ⊇ G) otherwise not.

Equivalence of Sets of FDs

Case-02: Determining Whether G Covers F-

Following steps are followed to determine whether G covers F or not-

Step-01:

- **Take the functional dependencies of set F into consideration**.
- For each functional dependency X → Y, find the closure of X using the functional dependencies of set F.

Step-02:

- Take the functional dependencies of set F into consideration.
- **For each functional dependency X → Y, find the closure of X using the functional dependencies of set G.**

Step-03:

- Compare the results of Step-01 and Step-02.
- **If the functional dependencies of set G has determined all those attributes that were determined by the functional dependencies of set F**, then it means G covers F.
- Thus, we conclude G covers F (G ⊇ F) otherwise not.

Equivalence of Sets of FDs

Case-03: Determining Whether Both F and G Cover Each Other-

- If F covers G and G covers F, then both F and G cover each other.
- Thus, if both the above cases hold true, we conclude both F and G cover each other (F = G).

Equivalence of Sets of FDs : Example

Show that the following two sets of FDs are equivalent:

F = {A → C, AC → D, E → AD, E → H} and

G = {A → CD, E → AH}

Which of the following holds true?

(A) G ⊇ F

(B) F ⊇ G

(C) F = G

(D) All of the above

Equivalence of Sets of FDs : Solution

Determining whether F covers G-

Consider FD G = {A → CD, E → AH}

Step-01:

- (A)+ = { A , C , D } // closure of left side of A → CD **using set G**
- (E)+ = { A , C , D , E , H } // closure of left side of E → AH **using set G**

Step-02:

- (A)+ = { A , C , D } // closure of left side of A → CD **using set F**
- (E)+ = { A , C , D , E , H } // closure of left side of E → AH **using set F**

Step-03:

Comparing the results of Step-01 and Step-02, we find Functional dependencies of set F can determine all the attributes which have been determined by the functional dependencies of set G.

- Thus, we conclude F covers G i.e. F ⊇ G.

Equivalence of Sets of FDs : Solution

Determining whether G covers F-

Consider FD F = {A → C, AC → D, E → AD, E → H}

Step-01:

- (A)+ = { A , C , D } // closure of left side of A → C using set F
- (AC)+ = { A , C , D } // closure of left side of AC → D using set F
- (E)+ = { A , C , D , E , H } // closure of left side of E → AD and E → H using set F

Step-02:

- (A)+ = { A , C , D } // closure of left side of A → C using set G
- (AC)+ = { A , C , D } // closure of left side of AC → D using set G
- (E)+ = { A , C , D , E , H } // closure of left side of E → AD and E → H using set G

Step-03:

Comparing the results of Step-01 and Step-02, we find-

- **Functional dependencies of set G can determine all the attributes which have been determined by the functional dependencies of set F.**
- Thus, we conclude G covers F i.e. G ⊇ F.

Equivalence of Sets of FDs : Solution

PES UNIVERSITY

Determining whether both F and G cover each other-

- From Step-01, we conclude F covers G.
- From Step-02, we conclude G covers F.
- Thus, we conclude both F and G cover each other i.e. F = G.

Thus, Option (D) is correct.

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Minimal Cover

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgments: Prof. Nivedita Kasturi (Dept of CSE), Teaching Assistants

Database Management Systems

Finding Minimal Cover of FD's

- Just as **we applied inference rules to expand on a set *F* of FDs to arrive at *F*+, its closure**, it is possible to ***think in the opposite direction*** to see if **we could shrink or reduce the set *F* to its *minimal form*** so that the minimal set is still equivalent to the original set *F.*
- Will use the concept of an **extraneous attribute in a FD for defining the minimum cover.**
- **Definition:** An attribute in a functional dependency is considered **extraneous attribute** if we can remove it without changing the closure of the set of dependencies. Formally, **given F**, the set of functional dependencies and a functional dependency $X \rightarrow A$ in F **, attribute *Y* is extraneous in *X* if *Y is a subset of X***, and F logically implies $(F - (X \rightarrow A) \cup \{ (X - Y) \rightarrow A \})$

Database Management Systems

Example Minimal Cover of FD's

Show:
Attribute **Y** is **extraneous** in **X** of a dependency **X → A**
➔if removing **Y** from **X** does not change the meaning of the dependency set.
Formally:
Y is extraneous in X if
$F = (F – \{ X \rightarrow A \}) \cup \{ (X – Y) \rightarrow A \}$

Example
Consider a relation R(A, B, C)Let the set of functional dependencies be :
$F = \{ AB \rightarrow C , A \rightarrow C \}$

Step 1: Check if attribute B is extraneous in $AB \rightarrow C$
- Original dependency: $AB \rightarrow C$
- We suspect B might be unnecessary.

So we remove B:
- If $A \rightarrow C$ already holds, then we do not need B
- And indeed, F already contains $A \rightarrow C$ So,

$$F – \{ AB \rightarrow C \} \cup \{ A \rightarrow C \}$$
$$= \{ A \rightarrow C , A \rightarrow C \}$$
$$= \{ A \rightarrow C \}$$

This gives the **same closure** as original F.
Meaning did not change So **B is extraneous** in **AB → C**

•both **A and B together** determine C.
•But we also have **A alone determines C**
•So why include B?
•Removing B does not change anything.

Database Management Systems

Minimal Sets of FDs

A set of FDs is **minimal** if it satisfies the following conditions:

1. Every dependency in F has a single attribute for its RHS.

2. We cannot replace any dependency X → A in F with a dependency Y → A, where Y is a proper subset of X and still has a set of dependencies that is equivalent to F. (**removing the redundancies via extraneous attributes on the LHS of a dependency**)

   if Y is proper subset of X then it's almost similar to X no and can be extranneous and would lead to redundancy

3. We cannot remove any dependency from F and still have a set of dependencies that is equivalent to F. (**removing the redundancies via having a dependency that can be inferred from the remaining FDs in F**)

Minimal Sets of FDs

PES UNIVERSITY

DEFINITION:

A minimal cover of a set of functional dependencies E is a minimal set of dependencies (in the standard canonical form and without redundancy) that is equivalent to E.

Algorithm: Finding a Minimal Cover F for a Set of Functional Dependencies E

Input: A set of functional dependencies E.

1. Set F:= E.
2. Replace each functional dependency X → {A1, A2, ..., An} in F by the n functional dependencies X →A1, X →A2, ..., X → An**(*Place FDs in a canonical form, Preparatory step*)**
3. For each functional dependency X → A in F for each attribute B that is an element of X
   if { {F – {X → A} } ∪ { (X – {B}) → A} } is equivalent to F then replace X → A with (X – {B}) → A in F.
   (* The above constitutes a removal of the extraneous attribute B from X *)
4. For each remaining functional dependency X → A in F
   if {F – {X → A} } is equivalent to F,
   then remove X → A from F.
   (* The above constitutes a removal of the redundant dependency X → A from F *)

Computing the Minimal Sets of FDs: Example 1

Let the set of FDs be *E*: {*B* → *A*, *D* → *A*, *AB* → *D*}.
Find the minimum cover of *E*.

- **Step 1:** All above dependencies are in **canonical** (they have only one attribute on the RHS) ;

- **In step 2:** we need to determine if *AB* → *D* has any redundant **(EXTRANEOUS) attribute** on the left-hand side; that is, can it be replaced by *B* → *D* or *A* → *D*?

  Since B → A, by augmenting with B on both sides (IR2), we have BB → AB, or B → AB (i). However, AB → D as given (ii).

  Hence by the transitive rule (IR3), we get from (i) and (ii), B → D. Hence AB → D may be replaced by B → D.

  We now have a set equivalent to original E , say E′ : {B → A, D → A, B → D}. No further reduction is possible in step 2 since all FDs have a single attribute on the left-hand side.

Computing the Minimal Sets of FDs: Example 1

- In **step 3** we look for a **redundant FD in E'.** By using the transitive rule on $B \rightarrow D$ and $D \rightarrow A$, we derive $B \rightarrow A$. Hence $B \rightarrow A$ is redundant in E' and can be eliminated.
- Hence the minimum cover of E is F: $\{B \rightarrow D, D \rightarrow A\}$.

The original set F can be inferred from E; In other words, the two sets F and E are equivalent.

according to definition minmal cover of set of FDs E is minimal set of dependencies that is equivalent to E
we found F which is minimal cover of FD E

Minimal Cover: Other Method – Example 1

Find the minimal cover of the set of functional dependencies given: {A → C, AB → C, C → DI, CD → I, EC → AB, EI → C}

Answer: Minimal Cover **Fc = {A → C, C → D, C → I, EC → A, EC → B, EI → C}**.

Solution:

Step 1: Decompose RHS of FD

FD= { A → C
AB → C
C → D
C → I
CD → I
EC → A
EC →B
EI → C}

Other Method - Example 1

Step 2: Remove the redundant FD from above set

a. Check if A → C is redundant.

Find A+ = {ACDI} {Including A→C in FD}

A+={ A} {Excluding A→C from FD}

As we got different attributes for A+ when included it in FD for finding closure and even when we excluded it from FD.

A → C is not redundant

b. Check if AB→C is redundant.

AB+={ABCDI} {Including AB→C in FD}

AB+={ABCDI} {Excluding AB→C from FD}

As we have got same closure for including AB→C and Excluding AB→C. then we can drive AB→C from given FD without AB→C so it's a redundant FD. Remove it from FD

AB → C is redundant

Therefore updated FD is FD1 = {A → C,C → D ,C → I ,CD → I, EC → A, EC →B, EI → C}

Other Method - Example 1

Now for the below calculation use FD1

c. Check if C→D is redundant.
Find C+= {CDI} {Including C→D in FD1}
C+={CI} {Excluding C→D from FD1}
C→D is not redundant

d. Check if C→I is redundant.
Find C+ ={CDI} {Including C→I in FD1}
C+={CDI} { Excluding C→I from FD1}
As we got same set of attributes for C+ even when we excluded the C→I from FD1.
Therefore we can infer the C→I from given set of FD1 so it can be removed from FD1 as its **redundant.**
C→I is redundant.

So updated FD is FD2={ A → C,C → D ,CD → I, EC → A, EC →B, EI → C}

Other Method - Example 1

Now for below calculation use FD2

e. Check if EC→A is redundant.
Find EC+= {ECABDI} {Including EC→A in FD2}
EC+={ECBDI} {Excluding EC→A in FD2}
EC→A is Not redundant

f. Check if EC→B is redundant.
Find EC+= {ECBADI} {Including EC→B in FD2}
EC+={ECADI} {Excluding EC→B in FD2}
EC→B is Not redundant

g. Check if CD→I is redundant.
Find CD+={CDI} {Including CD→I in FD2}
CD+={CD} {Excluding CD→I in FD2}
CD→I is not redundant.

Other Method - Example 1

h. Check if EI→C is redundant.

Find EI+={EICDAB} {Including EI→C in FD2}

EI+= {EI} {Excluding EI→C in FD2}

EI→C is not redundant.

Therefore FD is **FD2={ A → C,C → D ,CD → I, EC → A, EC →B, EI → C}**

Step 3: Check for extraneous attribute

Here consider the FD where we have more than one attribute on LHS

Following FD has more than one attribute on LHS

CD → I, EC → A, EC →B, EI → C

Now we will find extraneous attributes:

Database Management Systems

Other Method - Example 1

a. CD→I

Find closure for CD+, C+ and D+ from FD2. check if any closure has got same attributes, then only that attribute alone can be sufficient for FD

CD+ = {CDI}

C+= {CDI}

D+= {D}

In the above result the **C+ has the same set of attributes as CD+ so C attribute alone can determine the I.** Hence **D is an extraneous attribute.**

Consider FD as C→I.

Therefore, updated FD is FD3= {A → C, C → D, C → I, EC → A, EC →B, EI → C}

b. EC→A

Find closure for EC+, E+ and C+ using FD3

EC+={ECABID}

E+={E}

C+={CDI}

In the above closure, **we didn't get closure of a single attribute similar to EC so no extraneous attribute.**

Database Management Systems

Other Method - Example 1

c. EC→B

Find closure of EC+, E+ and C+ using FD3

EC+={ECBAID}

E+={E}

C+={CDI}

In the above closure we didn't get closure of a single attribute similar to **EC so no extraneous attribute**

Therefore, updated FD is FD3= {A → C, C → D, C → I, EC → A, EC →B, EI → C}

d. EI→C

Find closure of EI+, E+ and I+

EI+={EICDIAB}

E+={E}

I+={I}

In the above closure, **we didn't get closure of a single attribute similar to EI so no extraneous attribute**

Therefore, minimal set of FD is {A → C, C → D, C → I, EC → A, EC →B, EI → C}

Algorithm to determine the key of a relation

Algorithm: Finding a Key K for R, given a set F of Functional Dependencies

Input: A universal relation R and a set of functional dependencies F on the attributes of R.

1. Set K := R;

2. For each attribute A in K

{ Compute $(K - A)^+$ with respect to F;

If $(K - A)^+$ contains all the attributes in R,

then set K := K - {A} };

In algorithm, **we start by setting K to all the attributes of R**; we can say that R itself is always a default super key. We *then remove one attribute at a time and check whether the remaining attributes still form a super key.*

This algorithm, determines only one key out of the possible candidate keys for R; the key returned depends on the order in which attributes are removed from R in step 2.

Database Management Systems

Finding Candidate Keys

We can determine the candidate keys of a given relation using the following steps-

Step-01: Determine all essential attributes of the given relation.

- Essential attributes are **those not present on RHS** of any functional dependency.
- Essential attributes are always a part of every candidate key.
- This is because other attributes can not determine them.

Step-02: The remaining attributes of the relation are non-essential attributes. This is because they can be determined by using essential attributes.

Now, the following two cases are possible-

Database Management Systems

Finding Candidate Keys

We can determine the candidate keys of a given relation using the following steps-

Case-01: If all essential attributes together can determine all remaining non-essential attributes, then-

- The **combination of essential attributes is the candidate key.**
- It is the only possible candidate key.

Case-02: If all *essential attributes together can not determine all remaining non-essential attributes, then-*

- **The set of essential attributes and some non-essential attributes will be the candidate key(s).**
- In this case, multiple candidate keys are possible.
- To find the candidate keys, we check different combinations of essential and non-essential attributes.

Database Management Systems

Practice Problem: Finding Candidate Keys

PES UNIVERSITY

1. Let R = (A, B, C, D, E, F) be a relation scheme with the following dependencies-

C → F
E → A
EC → D
A → B

Which of the following is a key for R?
CD
EC
AE
AC

Also, determine the total number of candidate keys and super keys.

Database Management Systems

Practice Problem: Finding Candidate Keys

PES UNIVERSITY

Solution- We will find candidate keys of the given relation in the following steps-

Step-01: Determine all essential attributes of the given relation.
Essential attributes of the relation are- C and E(as C and E are not present on RHS of FD). So, **attributes C and E will definitely be a part of every candidate key.**

Step-02: Now, We will check if the essential attributes together can determine all remaining non-essential attributes. To check, we find the closure of CE.
So, we have-

$\{ CE \}^+$
$= \{ C , E \}$
$= \{ C , E , F \}$ (Using $C \rightarrow F$)
$= \{ A , C , E , F \}$ (Using $E \rightarrow A$)
$= \{ A , C , D , E , F \}$ (Using $EC \rightarrow D$)
$= \{ A , B , C , D , E , F \}$ (Using $A \rightarrow B$)

Practice Problem: Finding Candidate Keys

We conclude that **CE can determine all the attributes of the given relation.**
So, CE is the only possible candidate key of the relation.

Thus, Option (B) is correct.

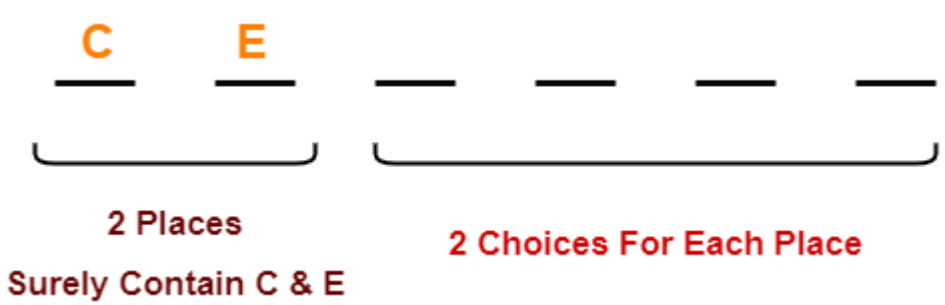
Total Number of Candidate Keys- Only one candidate key CE is possible.

Database Management Systems

Practice Problem: Finding Candidate Keys

Total Number of Super Keys- There are total 6 attributes in the given relation of which-

- There are 2 essential attributes- C and E.
- Remaining 4 attributes are non-essential attributes.
- Essential attributes will be definitely present in every key.
- Non-essential attributes may or may not be taken in every super key.



leftover 4 non essential attributes
you can either chose it or not
so 2 options for each and position doesn't matter
2*2*2*2=16 superkeys where CE is alaways included

genearl formula total numbe of attributes =n
nor of cnadidate keys =x
then nor 0f superkeys=2^(n-x)

So, number of super keys possible = 2 x 2 x 2 x 2 = 16.
Thus, total number of super keys possible = 16.

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Practice Problems - Equivalence, Minimal Cover

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgments: Prof. Nivedita Kasturi(Dept of CSE), Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 3: Design Concepts and Implementation

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G

Dept of Computer Science and Engineering

Examples of Equivalence

1. Let us take an example to show the relationship between two FD sets. A relation R(A,B,C,D) having two FD sets F = {A->B, B->C, AB->D} and G = {A->B, B->C, A->C, A->D}

Step 1: Checking whether all FDs of FD1 are present in FD2

- A->B in set FD1 is present in set FD2.
- B->C in set FD1 is also present in set FD2.
- AB->D is present in set FD1 but not directly in FD2 but we will check whether we can derive it or not. For set FD2, (AB)+ = {A, B, C, D}. It means that AB can functionally determine A, B, C, and D. So AB->D will also hold in set FD2.

As all FDs in set FD1 also hold in set FD2, FD2 ⊃ FD1 is true.

Examples of Equivalence

Step 2: Checking whether all FDs of FD2 are present in FD1

- A->B in set FD2 is present in set FD1.
- B->C in set FD2 is also present in set FD1.
- A->C is present in FD2 but not directly in FD1 but we will check whether we can derive it or not. For set FD1, (A)+ = {A, B, C, D}. It means that A can functionally determine A, B, C, and D. SO A->C will also hold in set FD1.
- A->D is present in FD2 but not directly in FD1 but we will check whether we can derive it or not. For set FD1, (A)+ = {A, B, C, D}. It means that A can functionally determine A, B, C, and D. SO A->D will also hold in set FD1.

As all FDs in set FD2 also hold in set FD1, FD1 ⊃ FD2 is true.

Step 3: As FD2 ⊃ FD1 and FD1 ⊃ FD2 both are true FD2 =FD1 is true. These two FD sets are semantically equivalent.

Examples of Equivalence

2. Let us take another example to show the relationship between two FD sets. A relation R2(A,B,C,D) having two FD sets FD1 = {A->B, B->C,A->C} and FD2 = {A->B, B->C, A->D}

Step 1: Checking whether all FDs of FD1 are present in FD2

- A->B in set FD1 is present in set FD2.
- B->C in set FD1 is also present in set FD2.
- A->C is present in FD1 but not directly in FD2 but we will check whether we can derive it or not. For set FD2, (A)+ = {A, B, C, D}. It means that A can functionally determine A, B, C, and D. SO A->C will also hold in set FD2.

As all FDs in set FD1 also hold in set FD2, FD2 ⊃ FD1 is true.

Examples of Equivalence

Step 2: Checking whether all FDs of FD2 are present in FD1

- A->B in set FD2 is present in set FD1.,

- B->C in set FD2 is also present in set FD1.
- A->D is present in FD2 but not directly in FD1 but we will check whether we can derive it or not. For set FD1, (A)+ = {A,B,C}. It means that A can't functionally determine D.
- So A->D will not hold in FD1.

As all FDs in set FD2 do not hold in set FD1, FD2 $\not\subset$ FD1.

Step 3: In this case, FD2 $\supset$ FD1 and FD2 $\not\subset$ FD1, these two FD sets are not semantically equivalent.

Examples of Equivalence

3. When are two sets of functional dependencies equivalent? How can we determine their equivalence?

Answer : Two sets of functional dependencies
F and G are considered equivalent if they imply the same set of functional dependencies. In other words, the closure of F (F +) must be the same as the closure of G (G
+).

To determine their equivalence:
Compute the closure
F+ and the closure G+for G.
If F+=G+
F + =G + , the two sets are equivalent.

Database Management Systems

Practice Problems on Equivalence

PES UNIVERSITY

1. **Find whether they are equivalent. FD1 = {x->y, xy->z,w->xz,w->vy } and FD2 = {x->yz, w->xv}**

Soln: FD1 ≡ FD2

1. **Find whether they are equivalent. F = {A->B, B->C, A->C} and G = {A->B, B->C, A->D}**

Soln: F not equivalent G

Examples of Minimal Cover

1. Let the given set of FDs be G : {A → BCDE, C*D* → E}.
Find the minimum cover of G.

Examples of Minimal Cover

1. Let the given set of FDs be G : {A → BCDE, *CD* → E}.
Find the minimum cover of G.

Step 1: All above dependencies are not in **canonical** (they do not have one attribute on the RHS) ; so we have to convert them into:

- **Step 1 :** F: { A -> B, A -> C, A -> D, A -> E, CD -> E}

- **Step 2 :** For CD - > E, neither C nor D is extra on the LHS, since we can not show that C -> E OR D -> E from the given FDs. Hence we can not replace it with either.

- **Step 3 :** Check any FD is redundant? Since A -> CD and CD -> E, by transitive rule (IR3), we get A -> E, Thus A -> E is redundant in G

Hence Minimum Cover of G, F : { A -> BCD, CD -> E}

Database Management Systems

Practice Problems on Minimal Cover

1. Let the given set of FDs be G : {v → w, vw→ x, y→ vxz}. Find the minimum cover of G.

Soln: Minimal cover is { v→wx , y→vz}

2.Let the given set of FDs be G : {A → B, C→ B, D→ ABC, AC→ D}. Find the minimum cover of G.

Soln: Minimal cover is {A→B, C→B, D→AC, AC→D}

Database Management Systems

Practice Problems on Candidate Key

1. **Let R = (A, B, C, D, E) be a relation scheme with the following dependencies-**
   AB → C
   C → D
   B → E

ANSWER: CANDIDATE KEY=AB NOR OF SUPER KEYS=8

Determine the total number of candidate keys and super keys.

2. Consider the relation scheme R(E, F, G, H, I, J, K, L, M, N) and the set of functional dependencies-

{ E, F } → { G }
{ F } → { I , J }
{ E, H } → { K, L }
{ K } → { M }
{ L } → { N }

ANSWER: CANDIDATE KEY= EFH NOR OF SUPER KEYS=2^7=128

Determine the total number of candidate keys and super keys.

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Normal Forms based on Primary Keys (1NF, 2NF, 3NF)

Prof. Mahitha G

Dept of Computer Science and Engineering

Acknowledgments: Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 3: Design Concepts and Implementation

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G

Dept of Computer Science and Engineering

Normalization of Relations

PES UNIVERSITY

Normalization of data:

- Is a process of analyzing the given relation schema based on their FDs and keys to achieve the desirable properties of **minimizing redundancy** and **minimizing insertion, deletion and update anomalies**

Normalization:

- The process of **decomposing unsatisfactory "bad" relations by breaking up** their attributes into smaller relations

Normal form:

- The normal form of a relation refers to the **highest normal form** condition that it meets and hence indicates the degree to which it has been normalized

Normalization of Relations

2NF, 3NF, BCNF

- **based on keys and FDs** of a relation schema

4NF

- **based on keys, multi-valued** dependencies : MVDs;

5NF

- **based on keys, join dependencies** : JDs

Additional properties may be needed to ensure a good relational design (**lossless join, dependency preservation**)

a lossless join refers to a property of a decomposition where a relation (table) is broken down into smaller relations, and when these smaller relations are joined back together, the original relation can be perfectly reconstructed without any loss of data or introduction of spurious (incorrect) tuples.

Dependency preservation in a database management system (DBMS) means that a relational decomposition (splitting a table into smaller tables) retains all the original functional dependencies (FDs) from the parent table in the new tables

Practical Use of Normal Forms

- **Normalization** is carried out in practice so that the resulting designs are of **high quality** and meet the desirable properties

- **The practical utility of these normal forms becomes questionable when the constraints on which they are based** are ***hard to understand*** **or to** ***detect***
- The database designers *need not* normalize to the highest possible normal form (usually up to 3NF and BCNF. 4NF rarely used in practice.)

- **Denormalization**:
  The process of **storing the join of higher normal form relations as a base relation**—which is in a lower normal form

Database Management Systems

Denormalization

- **Denormalization** is a *database optimization technique in which we add redundant data to one or more tables.* This can help us **avoid costly joins** in a relational database.
- Note that denormalization does not mean *not doing normalization*. It is an optimization technique that is applied after doing normalization.
- In a traditional normalized database, we store data in separate logical tables and attempt to minimize redundant data. We may strive to have only one copy of each piece of data in database.
- For example, in a normalized database, we might have a **Courses** table and a **Teachers** table. **Each entry in Courses would store the teacherID** for a Course but not the teacherName. *When we need to retrieve a list of all Courses with the Teacher's name, we would do a join between these two tables.*

Denormalization

- In some ways, this is great; **if a teacher changes his or her name**, we only have to update the name in one place.

- The *drawback is that if tables are large, we may spend an unnecessarily long time doing joins on tables.*

- Denormalization**, then, strikes a different compromise. Under denormalization, we decide that we're okay with some redundancy and some extra effort to update the database in order to get the efficiency advantages of fewer joins.**

Database Management Systems

Denormalization

Pros of Denormalization:-

- Retrieving data is faster since we do fewer joins
- Queries to retrieve can be simpler(and therefore less likely to have bugs), since we need to look at fewer tables.

★ Pros of Denormalization
✔ 1. Faster data retrieval
Fewer joins → faster SELECT queries
This is the main reason denormalization is used, especially in databases serving millions of requests.
✔ 2. Simpler queries
Query logic becomes easier:
Less joins
Less complexity
Less chance of mistakes
✔ 3. Better performance for analytical or reporting systems
Data warehouses often denormalize to speed up aggregation queries.

Cons of Denormalization:-

- Updates and inserts are more expensive.
- Denormalization can make update and insert code harder to write.
- Data may be inconsistent . Which is the “correct” value for a piece of data?
- Data redundancy necessitates more storage.

★ Cons of Denormalization
✖ 1. Updates are more expensive
Because the same piece of information is stored in multiple places.
Example: teacherName is stored in several rows.
If teacher changes name, you must update it everywhere
✖ 2. Insert operations are harder
When inserting a new course, you must copy redundant data correctly.
✖ 3. Possibility of inconsistency
Example:
teacherName = “John” in one row
teacherName = “Jon” (mistyped) in another row
Which one is correct?
This risk exists only because we duplicated data.
✖ 4. More storage required
Redundant data takes more space.

Database Management Systems

Definitions of Keys and Attributes Participating in Keys

- A **superkey** of a relation schema R = {A1, A2, ..., An} is a **set of attributes S (*subset-of* R) with the property that no two tuples t1 and t2 in any legal relation state r of R will have t1[S] = t2[S]**

basically no two rows can have same value for same set of attributes

- A **key K is a superkey with the *additional property* that removal of any attribute from K will cause K not to be a superkey any more** (ie minimal superkey).

Database Management Systems

Definitions of Keys and Attributes Participating in Keys

- If a relation schema has more than one key, each is called a **candidate** key.
- One of the candidate keys is *arbitrarily* designated to be the **primary key**, and the others are called **secondary keys**.
- If no candidate key for a relation, the entire relation can be treated as a default super key
- An attribute of relation R is called a **prime attribute** if it is a member of some candidate key of R.
- An attribute is called **nonprime** if its not a prime attribute ie its not a member of *some* candidate key

Database Management Systems

First Normal Form

(atomic = single indivisible value)
In simple terms: Every field (cell) must contain only 1 value, not a group/set/list.

- States that the **domain of an attribute must include only atomic** values (single value from the domain of attribute)

Disallows

- composite attributes
- multivalued attributes
- **nested relations**; attributes whose values for an *individual tuple* are non-atomic
- Considered to be part of the definition of a relation
- Most RDBMSs allow only those relations to be defined that are in First Normal Form

1NF--> don't allow composite ,mutlivalued and nested attributes

Database Management Systems

Normalization into 1NF

dotted arrows show multivalued attributes

(a)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocations

(b)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

Dlocations have multivalued attribute that violates 1NF principles

(c)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocation
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

introduced redundancy but now satisfies 1NF as doesn't have multivalued attribute Dlocations and shown as Dloaction now

Figure 14.9
Normalization into 1NF. (a) A relation schema that is not in 1NF. (b) Sample state of relation DEPARTMENT. (c) 1NF version of the same relation with redundancy.

Database Management Systems

Normalizing nested relations into 1NF

this is a nested relation

(a)

EMP_PROJ

		Projs	
Ssn	Ename	Pnumber	Hours

(b)

EMP_PROJ

Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0
333445555	Wong, Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya, Alicia J.	30	30.0
		10	10.0
987987987	Jabbar, Ahmad V.	10	35.0
		30	5.0
987654321	Wallace, Jennifer S.	30	20.0
		20	15.0
888665555	Borg, James E.	20	NULL

(c)

EMP_PROJ1

Ssn	Ename

EMP_PROJ2

Ssn	Pnumber	Hours

broken into two tables in order not to have nested relations

Figure 14.10
Normalizing nested relations into 1NF. (a) Schema of the EMP_PROJ relation with a nested relation attribute PROJS. (b) Sample extension of the EMP_PROJ relation showing nested relations within each tuple. (c) Decomposition of EMP_PROJ into relations EMP_PROJ1 and EMP_PROJ2 by propagating the primary key.

Database Management Systems

Second Normal Form

separately they don't hold true

- Uses the concepts of **FDs, primary key**

Definitions

- **Prime attribute:** An attribute that is member of the primary key K
- **Full functional dependency:** a FD Y -> Z where removal of any attribute from Y means the FD does not hold any more

Examples:

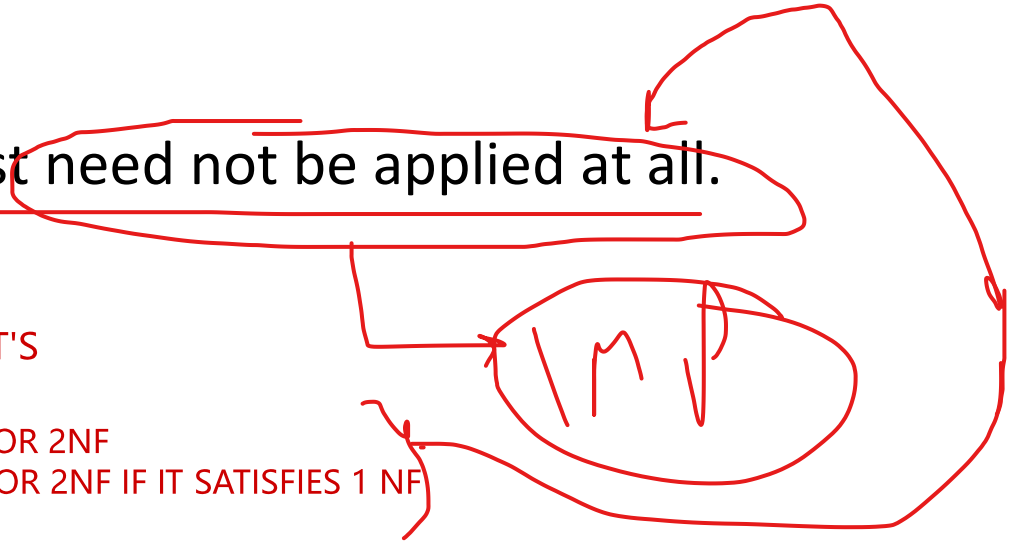
- {SSN, PNUMBER} -> HOURS is a full FD since neither SSN -> HOURS nor PNUMBER -> HOURS hold
- {SSN, PNUMBER} -> ENAME is not a full FD (it is called a partial dependency) since SSN -> ENAME also holds

holds true individually as well
Fully functional dependency can't hold true if removal of any attribute is done from LHS and it still holds

Second Normal Form

2NF: Fully functional dependency+1NF

- A relation schema R is in second normal form (2NF) **if every non-prime attribute A in R is fully functionally dependent on the primary key and R is in 1NF**
- R can be decomposed into 2NF relations via the process of 2NF normalization or "second normalization"
- The test for 2NF involves **testing for FDs whose left hand side attributes are part of primary key.**
- If primary key contains a single attributes the test need not be applied at all.



BASICALLY IF YOU HAVE A CANDIDATE KEY ABC-->D
IT'S FULLY FUNCTIONALLY DEPENDENT ONLY IF ABC TOGETHER DETERMINES D
A OR B OR C SHOULDN'T SEPARATELY DETERMINE SOME OTHER ATTRIBUTE IF THAT'S THE CASE IT'S PARTIALLY DEPENDENT AND VIOLATES 2NF

IF INSTEAD OF ABC-->D AND ABC TOGETHER IS CANDIDATE KEY THEN WE NEED TO CHECK FOR 2NF
IF YOU HAVE DEPENDENCY LIKE A-->C AND A IS A PRIMARY KEY THEN NO NEED TO CHECK FOR 2NF IF IT SATISFIES 1 NF THEN IT SATISFIES 2NF ALSO BECAUSE FOR 2NF IT SHOULD HAVE A CANDIDATE KEY

Third Normal Form

Definition:

Transitive functional dependency: a FD X -> Z that can be derived from two FDs X -> Y and Y -> Z

- Examples:
  - SSN -> DMGRSSN is a **transitive** FD
    - Since SSN -> DNUMBER and DNUMBER -> DMGRSSN hold
  - SSN -> ENAME is **non-transitive**
    - Since there is no set of attributes X where SSN -> X and X -> ENAME

Database Management Systems

Third Normal Form

- A relation schema R is in **third normal form (3NF)** if it is in 2NF *and* no non-prime attribute A in R is transitively dependent on the primary key
- R can be decomposed into 3NF relations via the process of 3NF normalization

NOTE:

- In X -> Y and Y -> Z, with X as the primary key, we consider this a problem only if Y is not a candidate key.
- When Y is a candidate key, there is no problem with the transitive dependency.

  E.g., Consider EMP (SSN, Emp#, Salary).

  Here, SSN -> Emp# -> Salary and Emp# is a candidate key.

WKT THAN TRANSITIVE MEANS A-->B AND B-->C THEN A--C
BUT NOW FOR 3NF SAYS : IT NEEDS FIRST 2NF TO HOLD TRUE+ NO NON PRIME ATTRIBUTE SHOULD BE TRANSITIVELY DEPENDENT ON A PRIIME ATTRIBUTE
THAT MEANS A (PRIME) ---> B (NON PRIME) ---> C(NON PRMEI) THIS CONDITION SHOULD NOT HAPPEN YOU SEE C IS A NON PRIME ATTRIBUTE AND IS TRANSITIVELY DEPENDENT ON ATTRIBUTE A THAT IS PRIME, THIS SHOULD NOT HAPPEN THIS VIOLATE 3NF

ANOTHER IMP POINT :: IF A (PRIME) ---> B (PRIME OR CANDIDATE KEY) ---> C(NON PRMEI) THIS IS FINE (NOT A VIOLATION OF 3NF)

SO BASICALLY SHORT TRICK: NON PRIME (B) DETERMINING A NON PRIME (C) IN A TRANSITIVE DEPENDENCY STARTING WITH PRIME(A) NOT ALLOWED

Normal Forms Defined Informally

1st normal form

- All attributes are atomic

2nd normal form

- All attributes depend on **the whole key**

3rd normal form

- All attributes depend on **nothing but the key**

Database Management Systems

Normalizing into 2NF and 3NF

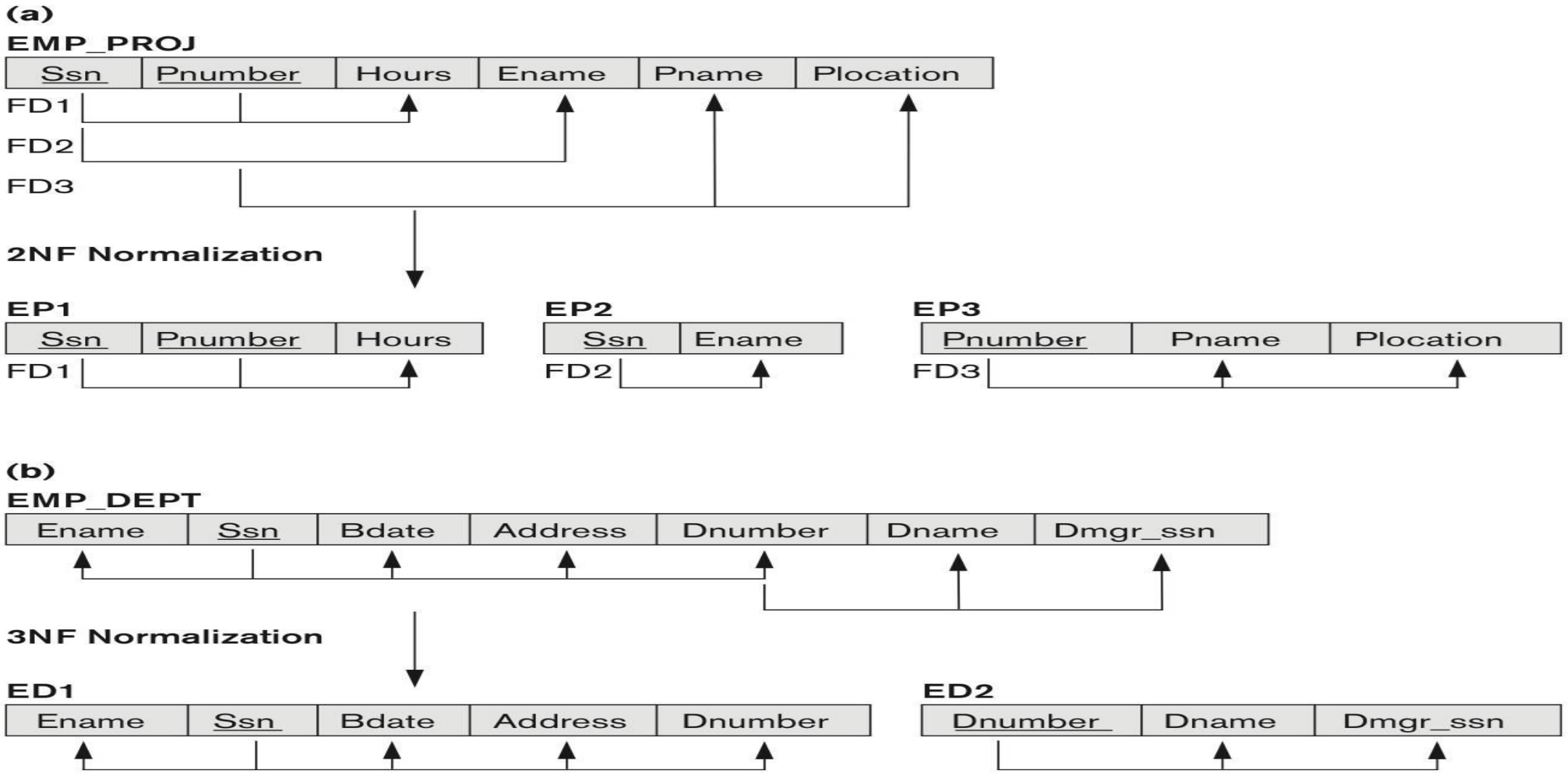


Figure 14.11 Normalizing into 2NF and 3NF. (a) Normalizing EMP_PROJ into 2NF relations. (b) Normalizing EMP_DEPT into 3NF relations.

Database Management Systems

Normalization into 2NF and 3NF

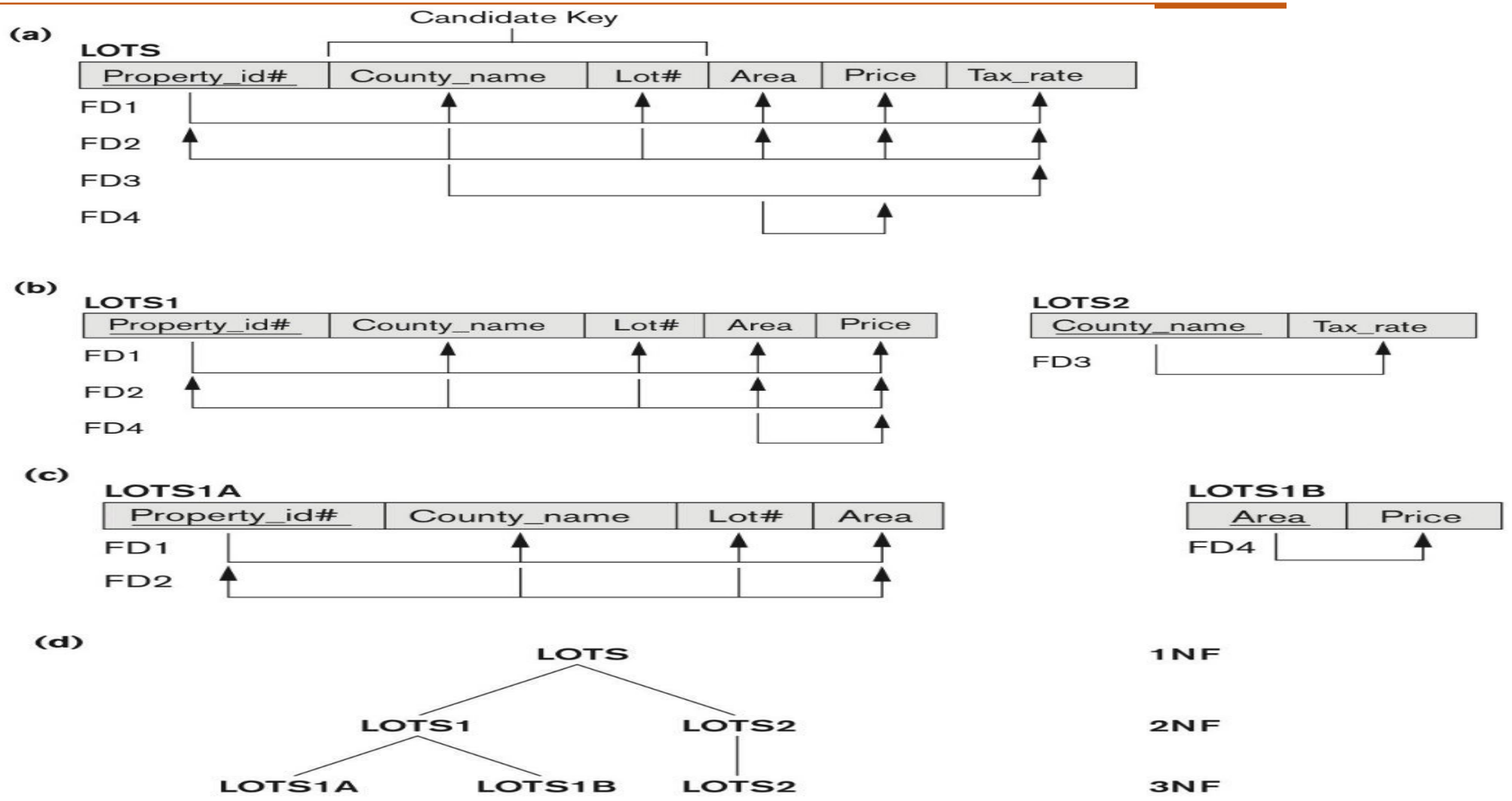


Figure 14.12 Normalization into 2NF and 3NF. (a) The LOTS relation with its functional dependencies FD1 through FD4. (b) Decomposing into the 2NF relations LOTS1 and LOTS2. (c) Decomposing LOTS1 into the 3NF relations LOTS1A and LOTS1B. (d) Progressive normalization of LOTS into a 3NF design.

FOR BETTER UNDERSTANDING PLEASE REFER THIS AS WELL ONLY AFTER YOU HAVE READ ON YOUR OWN TILL HERE AND DIDN'T UNDERSTAND PROPERLY
file:///E:/CollegeWork/PESU_SEM_5/DBMS/Unit3/pdf/Reference/2NF_3NF_slide47.pdf

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

General Definitions of 2NF and 3NF, BCNF

Prof. Mahitha G

Dept of Computer Science and Engineering

Acknowledgments: Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 3: Design Concepts and Implementation

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G

Dept of Computer Science and Engineering

General Normal Form Definitions (For Multiple Keys)

- The above definitions considers the primary key only
- The following more general definitions take into account relations with multiple candidate keys
- Any attribute involved in a candidate key is a *prime attribute*
- All other attributes are called *non-prime attributes.*

General Definition of 2NF (For Multiple Candidate Keys)

- A relation schema R is in **second normal form (2NF)** if every non-prime attribute A in R is fully functionally dependent on *every* key of R
- In Figure 14.12 the FD

  County_name → Tax_rate violates 2NF.

BASICALLY WE HAD (county_name, LOt#) which was a candidate key and county_name was alone determining tax rate separately and creating a partial dependency which is not allowed in 2NF as it needs Fully functional dependency

So second normalization converts LOTS into

LOTS1 (Property_id#, County_name, Lot#, Area, Price)

LOTS2 (County_name, Tax_rate)

Database Management Systems

Figure 14.12 Normalization into 2NF and 3NF

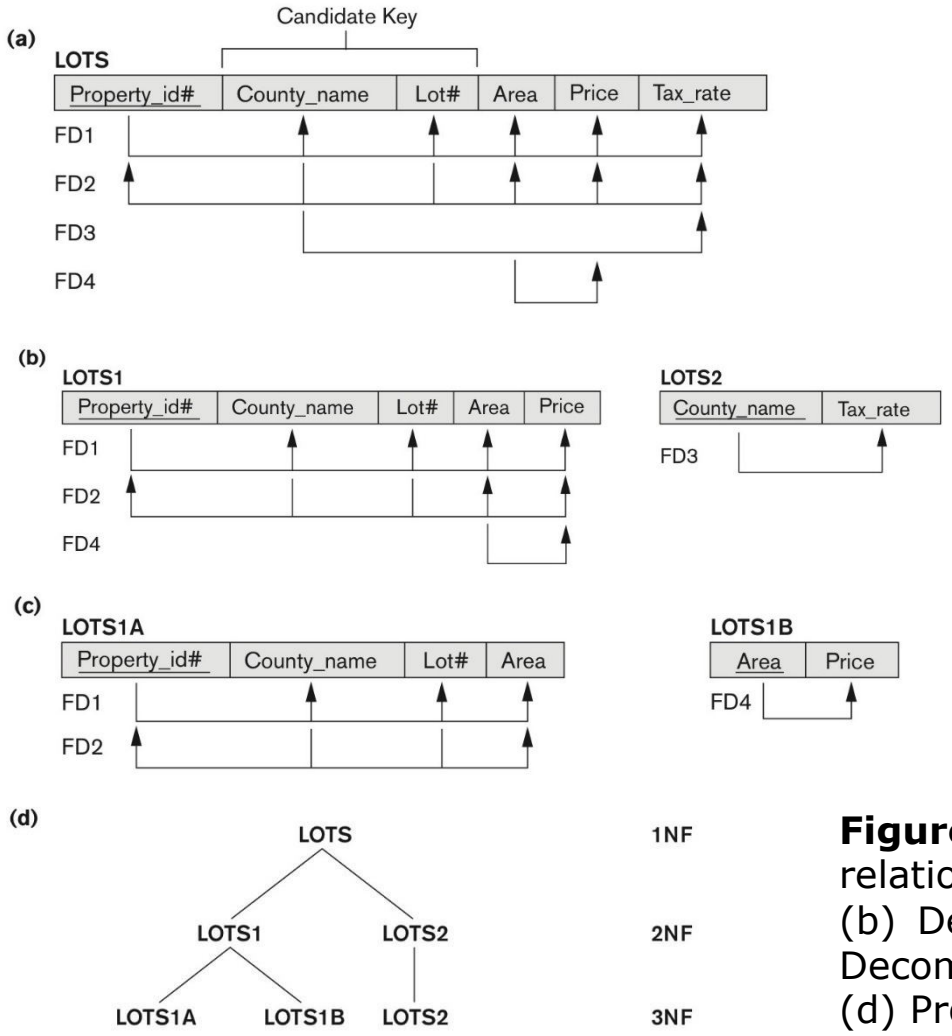


Figure 14.12 Normalization into 2NF and 3NF. (a) The LOTS relation with its functional dependencies FD1 through FD4. (b) Decomposing into the 2NF relations LOTS1 and LOTS2. (c) Decomposing LOTS1 into the 3NF relations LOTS1A and LOTS1B. (d) Progressive normalization of LOTS into a 3NF design.

whenever you see key written anywhere in slides it usually means minimal superkey but you can consider it to be candidate key for easier understnading

Database Management Systems

General Definition of Third Normal Form

PES UNIVERSITY

Definition:

- **Superkey** of relation schema R - a set of attributes S of R that contains a key of R
  - A relation schema R is in **third normal form (3NF)** if whenever a FD X → A holds in R, then either:
  - (a) X is a superkey of R, or
  - (b) A is a prime attribute of R

- LOTS1 relation violates 3NF because

  Area → Price ;

  and Area is not a superkey in LOTS1. (see Figure 14.12).

SUPERKEY CONTAINS CANDIDATE KEY , CANDIDATE KEY WILL ACT AS A PRIMARY KEY
NOW CHILL THIS IS THE SAME THING AS PREVIOUS JUST A NEW CONDITION
I SAID PREVIOSLY property_id(prime) ---> Area(non prime) ---> price(non prime) if this happens then 3NF is violated right now you see CONSIDER THIS Area(X) and price(R)
so let anything happen for violation of 3NF ,property_id(prime) ---> Area(non prime) ---> price(non prime) this shouldn't happen right
now see previosly i said if A(prime) --> B(prime) --> C(non prime) thi

is fine
basically TRICK was non prime ---> non prime shouldn't happen righ
what if A(prime) --> B(non prime)-->C(prime) still it doesn't violate th 3NF condition i told you eariler neither does it violate the trick
non prime --> non prime
This is what they are telling in complicated way here

property_id(prime) ---> Area(X)(superkey containg candidate or prim
price(non prime) [ALLOWED]
OR
property_id(prime) ---> Area(X)(non superkey) --> price(prime) [ALLOWED]
OR
property_id(prime) ---> Area(X)(superkey) --> price(prime) [ALLOWEI

none of these above condition violates 3NF

Interpreting the General Definition of Third Normal Form

- Consider the 2 conditions in the Definition of 3NF:
- A relation schema R is in third normal form (3NF) if whenever a FD X → A holds in R, then either:

  (a) X is a superkey of R, or

  OR

  (b) A is a prime attribute of R

- Condition (a) catches two types of violations :

  - one where a prime attribute functionally determines a non-prime attribute. This catches 2NF violations due to non-full functional dependencies.

  - second, where a non-prime attribute functionally determines a non-prime attribute. This catches 3NF violations due to a transitive dependency.

Interpreting the General Definition of Third Normal Form

ALTERNATIVE DEFINITION of 3NF:

We can restate the definition as:

A relation schema R is in **third normal form (3NF)** if every non-prime attribute in R meets both of these conditions:

- It is fully functionally dependent on every key of R
- It is non-transitively dependent on every key of R

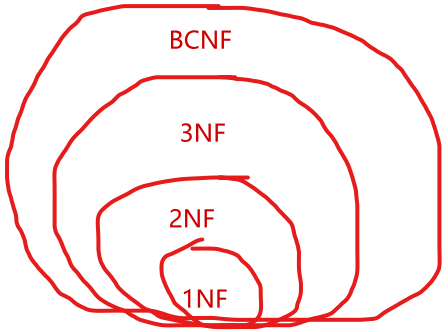
Note that stated this way, a relation in 3NF also meets the requirements for 2NF.

- The condition (b) from the last slide takes care of the dependencies that "slip through" (are allowable to) 3NF but are "caught by" BCNF which we discuss next.

Database Management Systems

BCNF (Boyce-Codd Normal Form)

- A relation schema R is in Boyce-Codd Normal Form (BCNF) if whenever an FD X → A holds in R, then X is a superkey of R
- Each normal form is strictly stronger than the previous one
- Every 2NF relation is in 1NF
- Every 3NF relation is in 2NF
- Every BCNF relation is in 3NF
- There exist relations that are in 3NF but not in BCNF
- Hence BCNF is considered a stronger form of 3NF
- The goal is to have each relation in BCNF (or 3NF)



Achieving the BCNF by Decomposition

- Along with 1NF, 2Nf, 3NF, BCNF a relation should also have additional 2 properties.
- **Process of normalization through decomposition must also conform the existence of additional properties**

- i) **Non additive join or lossless join property:** states that spurious tuples should not be generated upon joining the decomposed relations.

- ii) **Dependency Preservation Property:** ensures that each FD is represented in some individual relation resulting after decomposition.

- **Property i) is extremely critical and must** be achieved at any cost
- **Property ii) is desirable**, sometimes sacrificed

Database Management Systems

BCNF (Boyce-Codd Normal Form)

- A relation schema R is in 3NF if whenever X -> A holds in R either
  a) X is a superkey of R or
  b) A is a prime attribute.

- BCNF: A relation schema R is in BCNF if whenever a FD X -> A holds in R , the X is a superkey of R

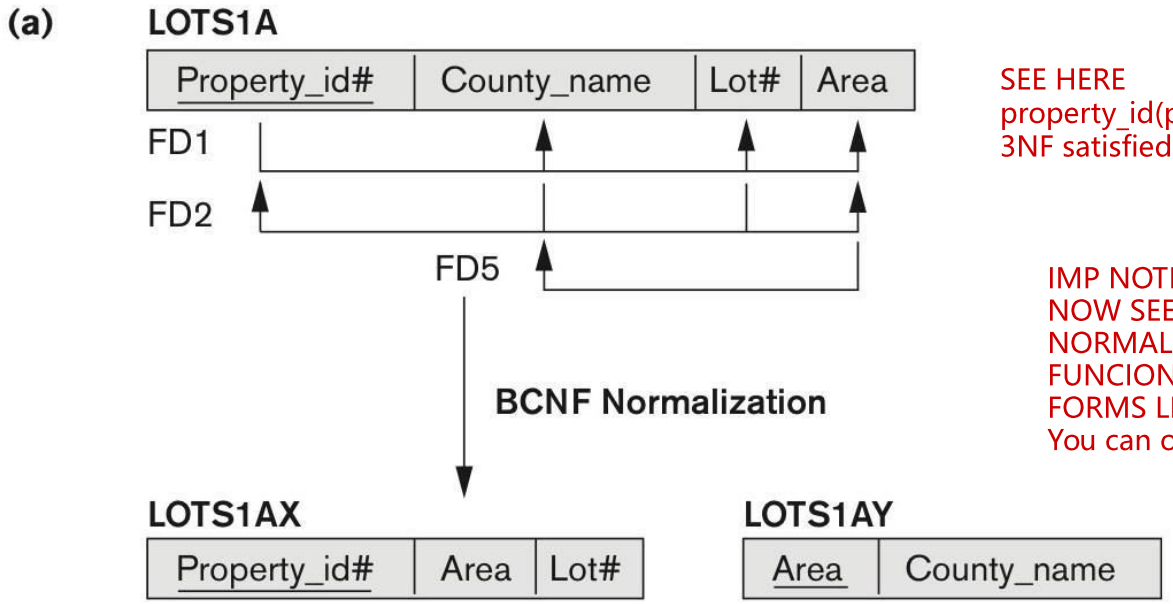
NO OR AND ALL NOW IT'S STRICT DOESNT' GIVE FREEDOM X HAS TO BE SUPERKEY

- In practice, most relation schemas that are in 3NF are in BCNF, only if there exists some X->A in R with X not being superkey and A being prime attribute, will R be in 3NF but not in BCNF.

- Figure b (next slide) C is not a superkey and B is a prime attribute.

Database Management Systems

Fig. 14.13 BCNF (Boyce-Codd Normal Form)



SEE HERE
property_id(primary key) --> Area(non prime or not superkey) --> county_name(candidate key)
3NF satisfied but BCNF violated as Area is not Superkey

IMP NOTICE:
NOW SEE FUNCTIONAL DEPENDENCY COULD NOT BE PRESERVED AS SOON AS WE DID BCNF NORMALIZATION , SO BASICALLY
FUNCIONAL DEPENDENCY may or may not BE PRSERVED FOR BCNF AND ABOVE NORMAL FORMS LIKE 4NF 5NF etc.
You can only preserve them until 3NF

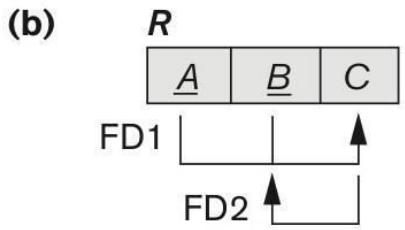


Figure 14.13
Boyce-Codd normal form. (a) BCNF normalization of LOTS1A with the functional dependency FD2 being lost in the decomposition. (b) A schematic relation with FDs; it is in 3NF, but not in BCNF due to the f.d. C → B.

A(prime) -->C(non prime) --> B(prime)

↳ 3NF but not BCNF

Figure 14.14 A relation TEACH that is in 3NF but not in BCNF

PES UNIVERSITY

TEACH

Student	Course	Instructor
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe
Narayan	Operating Systems	Ammar

Figure 14.14
A relation TEACH that is in 3NF but not BCNF.

Achieving the BCNF by Decomposition (1)

GMESA

PES UNIVERSITY

- Two FDs exist in the relation TEACH:
  fd1: { student, course} -> instructor
  fd2: instructor -> course

- {student, course} is a candidate key for this relation and that the dependencies shown follow the pattern in Figure 14.13 (b).
  So this relation is in 3NF *but not in* BCNF
  {student,course}(candidate)-->(instructor)(non prime or non superkey) --> course(prime or partt of candidate key)

- A relation **NOT** in BCNF should be decomposed so as to meet this property, while possibly forgoing the preservation of all functional dependencies in the decomposed relations.

Database Management Systems

Achieving the BCNF by Decomposition (2)

- Three possible decompositions for relation TEACH
  - D1: {student, instructor} and {student, course}
  - D2: {course, instructor } and {course, student}
  - D3: {instructor, course } and {instructor, student} -> good decomposition

- All three decompositions will lose fd1. We have to settle for sacrificing the functional dependency preservation. But we cannot sacrifice the non-additivity property after decomposition. basically lossless join property
- Out of the above three, only the 3rd decomposition will not generate spurious tuples after join.(and hence has the non-additivity property).

- A test to determine whether a binary decomposition (decomposition into two relations) is non-additive (lossless) is discussed under Property NJB on the next slide. We then show how the third decomposition above meets the property.

Test for checking non-additivity of Binary Relational Decompositions

PES UNIVERSITY

- Testing Binary Decompositions for Lossless Join (Non-additive Join) Property

- Binary Decomposition: Decomposition of a relation R into two relations.

PROPERTY NJB (non-additive join test for binary decompositions):

A decomposition D = {R1, R2} of R has the lossless join property with respect to a set of functional dependencies F on R *if and only if* either

- The f.d. ((R1 ∩ R2) → (R1- R2)) is in F^+, or
- The f.d. ((R1 ∩ R2) → (R2 - R1)) is in F^+.

either of these should hold

Test for checking non-additivity of Binary Relational Decompositions

If you apply the NJB test to the 3 decompositions of the TEACH relation:

- D1 gives **Student** → Instructor or **Student** → Course, none of which is true.
- D2 gives **Course** → Instructor or **Course** → Student, none of which is true.
- However, in D3 we get **Instructor** → Course or **Instructor** → Student.

Since **Instructor** → Course is indeed true, the NJB property is satisfied and D3 is determined as a non-additive (good) decomposition.

done in notebook

General Procedure for achieving BCNF when a relation fails BCNF

Here we make use the following algorithm:

- Let R be the relation not in BCNF, let X be a subset-of R, and let $X \rightarrow A$ be the FD that causes a violation of BCNF. Then R may be decomposed into two relations:
  - (i) $R - A$ and (ii) $X \cup A$.
  - If either $R - A$ or $X \cup A$. is not in BCNF, repeat the process.

Note that the f.d. that violated BCNF in TEACH was Instructor →Course. Hence its BCNF decomposition would be :

(**TEACH** – COURSE) and (Instructor ∪ Course), which gives

the relations: (Instructor, Student) and (Instructor, Course) that we obtained before in decomposition D3.

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Higher Normal Forms

Prof. Mahitha G

Dept of Computer Science and Engineering

Acknowledgments: Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 3: Design Concepts and Implementation

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G

Dept of Computer Science and Engineering

Multivalued Dependencies and Fourth Normal Form (1)

Definition:

- A **multivalued dependency** (**MVD**) X —>> Y specified on relation schema R, **where X and Y are both subsets of R**, specifies the following constraint on any relation state r of R: *If two tuples t_1 and t_2 exist in r such that $t_1[X] = t_2[X]$, then two tuples t_3 and t_4 should also exist in r with the following properties, where we use Z to denote (R - (X ∪ Y))*:

  $t_3[X] = t_4[X] = t_1[X] = t_2[X]$.

  $t_3[Y] = t_1[Y]$ and $t_4[Y] = t_2[Y]$.

  $t_3[Z] = t_2[Z]$ and $t_4[Z] = t_1[Z]$.

An MVDX→→Yexistsinrelation R when for a given value of X, the values of Y are independent of the remaining attributes (Z = R– (XUY)).

- An MVD X —>> Y in R is called a **trivial MVD** if (a) **Y is a subset of X**, or (b) X ∪ Y = R. Its called trivial bcoz it does not specify any significant constraint on R
- An MVD that satisfies neither (a) nor (b) is called **nontrivial MVD.**

Database Management Systems

Multivalued Dependencies and Fourth Normal Form (1)

SID	SNAME	CID	CNAME
S1	A	C1	C
S1	A	C2	D
S2	B	C1	C
S2	B	C2	D

MVD'S ARE

- SID->->CID;
- SID->->CNAME;
- SNAME->->CNAME

Multivalued Dependencies and Fourth Normal Form (3)

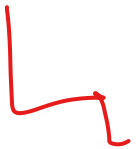
Definition:

- A *relation schema R is in **4NF** with respect to a set of dependencies F (that includes functional dependencies and multivalued dependencies) if, for every nontrivial multivalued dependency X —>> Y in F^+, X is a superkey for R.*

basically check for X-->>Y MVD and there should be X as superkye

Note: ***F^+* is the (complete) set of all dependencies** (functional or multivalued) that will hold in every relation state *r* of *R* that satisfies *F*. It is also called the **closure** of *F*.

MVD are consequences of 1NF which disallows attributes in a tuple to have a set of values. MVD are introduced bcoz of 2nd option (expansion of key) of normalizing in1NF.



if you remove MVD with all those condition above then you satisfy 4NF conditions

Join Dependencies and Fifth Normal Form (1)

Definition:

- A **join dependency** (**JD**), denoted by JD(R_1, R_2, ..., R_n), specified on relation schema R, specifies a constraint on the states r of R.
- The constraint states that every legal state r of R should have a **non-additive join decomposition into R_1, R_2, ..., R_n; that is, for every such r we have**

$$\bowtie(\pi_{R_1}(r), \pi_{R_2}(r), ..., \pi_{R_n}(r)) = r$$

Note: an MVD is a special case of a JD where n = 2.

- **A join dependency JD(R_1, R_2, ..., R_n), specified on relation schema R, is a trivial JD if one of the relation schemas R_i in JD(R_1, R_2, ..., R_n) is equal to R.**

if while joining something that has been decomposed already has the whole earlier table R then it will give R only no so trivial

A Join Dependency (JD) says:
A relation R can be losslessly reconstructed by joining n projections of R.

Join Dependency (JD):
A relation **R** has a join dependency if **R** can be reconstructed (losslessly) by joining **n** different projections of R.
Formally:
JD(R_1, R_2, …, R_n) means:
$R = R_1 \bowtie R_2 \bowtie \dots \bowtie R_n$

An MVD **X** →→ **Y** exists in relation R when for a given value of X, the values of Y are independent of the remaining attributes (Z = R – XY).

R(Student, Course, Hobby)
MVD:
Student →→ Course
means:
•For each Student, list of Courses is independent of list of Hobbies.
We can decompose into:
R_1(Student, Course)
R_2(Student, Hobby)
And join them:
$R = R_1 \bowtie R_2$
— This is a **JD with n = 2** → therefore also an MVD.

Why an MVD is a special case of a JD
For **MVD X** →→ **Y**, the relation **R** can be decomposed into **2** projections:
•R_1 = (X, Y)
•R_2 = (X, Z)
So in MVD, we always join **two** projections to reconstruct original relation:
R = (X,Y) ⋈ (X,Z)
This matches the definition of a **join dependency with n = 2**:
℗**JD(R_1, R_2)** where R_1 = (X,Y) and R_2 = (X,Z)
Thus,
℗**Every MVD is a special case of a JD with 2 components (n = 2)**
But **not every JD with n = 2 is necessarily an MVD.**

Join Dependencies and Fifth Normal Form (2)

PES UNIVERSITY

Definition:

- A relation schema *R* is in **fifth normal form** (**5NF**) (or **Project-Join Normal Form** (**PJNF**)) *with respect to a set F of functional, multivalued, and join dependencies if,*
  - **for every nontrivial join dependency JD(R_1, R_2, ..., R_n) in F^+** (that is, implied by *F*),
  - **every R_i is a superkey of *R*.**

every Ri is a superkey mean all R1, R2, R3 will be superkey hence it will uniquely determine stuff if everything is unique chances of redundant data willl decrease and there will be no loss so youwill get lossless join no

- Discovering join dependencies in practical databases with hundreds of relations is next to impossible. Therefore, 5NF is rarely used in practice.

in JD every JD is a superkey hence can be seen as indcation of 5NF and decomposing them into R1 R2 R3 when joined again gives whole table back losslessly

Database Management Systems

Figure 14.15 Fourth and fifth normal forms.

4 time it appears MVD esists t3[X] = t4[X] = t1[X] = t2[X]

also satisfies other properties given in first slide

(a) EMP

Ename	Pname	Dname
Smith	X	John
Smith	Y	Anna
Smith	X	Anna
Smith	Y	John

(b) EMP_PROJECTS

Ename	Pname
Smith	X
Smith	Y

EMP_DEPENDENTS

Ename	Dname
Smith	John
Smith	Anna

(c) SUPPLY

Sname	Part_name	Proj_name
Smith	Bolt	ProjX
Smith	Nut	ProjY
Adamsky	Bolt	ProjY
Walton	Nut	ProjZ
Adamsky	Nail	ProjX
Adamsky	Bolt	ProjX
Smith	Bolt	ProjY

(d)

R_1

Sname	Part_name
Smith	Bolt
Smith	Nut
Adamsky	Bolt
Walton	Nut
Adamsky	Nail

R_2

Sname	Proj_name
Smith	ProjX
Smith	ProjY
Adamsky	ProjY
Walton	ProjZ
Adamsky	ProjX

R_3

Part_name	Proj_name
Bolt	ProjX
Nut	ProjY
Bolt	ProjY
Nut	ProjZ
Nail	ProjX

Figure 14.15 Fourth and fifth normal forms. (a) The EMP relation with two MVDs: Ename –>> Pname and Ename –>> Dname. (b) Decomposing the EMP relation into two 4NF relations EMP_PROJECTS and EMP_DEPENDENTS. (c) The relation SUPPLY with no MVDs is in 4NF but not in 5NF if it has the JD(R1, R2, R3). (d) Decomposing the relation SUPPLY into the 5NF relations R1, R2, R3.

Numerical Examples on Normalization:2NF

PES UNIVERSITY

1. Given a relation R(P, Q, R, S, T) and Functional Dependency set FD = { PQ → R, S → T }, determine whether the given R is in 2NF? If not convert it into 2 NF.

Numerical Examples on Normalization:2NF

1. Given a relation R(P, Q, R, S, T) and Functional Dependency set FD = { PQ → R, S → T }, determine whether the given R is in 2NF? If not convert it into 2 NF.

Solution:

Step 1 : Find out candidate key using closure approach.

PQS + = PQSRT

Since the closure of PQS contains all the attributes of R, hence **PQS is Candidate Key**

Step 2: Find prime and non prime attribute

Since R has 5 attributes: - P, Q, R, S, T and **Candidate Key is PQS, Therefore, prime attributes (part of candidate key) are P, Q, and S** while a non-prime attribute is R and T

1. Given a relation R(P, Q, R, S, T) and Functional Dependency set FD = { PQ → R, S → T }, determine whether the given R is in 2NF? If not convert it into 2 NF.

Step 3: Check the existing FDs.

a) FD: PQ → R does not satisfy the definition of 2NF, that non-prime attribute(R) is partially dependent on part of candidate key PQS.

b) FD: S → T does not satisfy the definition of 2NF, as a non-prime attribute(T) is partially dependent on candidate key PQS (i.e., key should not be broken at any cost).

Hence, FD PQ → R and S → T, the above table R(P, Q, R, S, T) is not in 2NF

Numerical Examples on Normalization:2NF

1. Given a relation R(P, Q, R, S, T) and Functional Dependency set FD = { PQ → R, S → T }, determine whether the given R is in 2NF? If not convert it into 2 NF.

Step 4: Decompose the table (Convert the table R(P,Q,R,S,T) in 2NF)

Since due to FD: PQ → R and S → T, our table was not in 2NF, let's decompose the table

R1(P, Q, R) (Now in table R1 FD: PQ → R is Full F D, hence R1 is in 2NF)

R2(S, T) (Now in table R2 FD: S → T is Full F D, hence R2 is in 2NF) And create one table for the key, since the key is PQS. **R3(P, Q, S)**

Finally, the decomposed tables which is in 2NF are:

a) R1(P, Q, R)

b) R2(S, T)

c) R3(P, Q, S)

Numerical Examples on Normalization:3NF

PES UNIVERSITY

1. Given a relation R(X, Y, Z) and Functional Dependency set FD = { X → Y and Y → Z }, determine whether the given R is in 3NF? If not convert it into 3 NF.

Numerical Examples on Normalization:3NF

1. Given a relation R(X, Y, Z) and Functional Dependency set FD = { X → Y and Y → Z }, determine whether the given R is in 3NF? If not convert it into 3 NF.

Solution:

Step 1: Find out candidate key:

X + = XYZ

Since the closure of X contains all the attributes of R, hence X is Candidate Key

Step 2: (First check for 2NF) find prime and non prime attributes:

X is a prime and Y, Z are non prime

Step 3: Check the existing FDs

FD: X → Y is in 2NF (as Key is not breaking and its Fully functional dependent)

FD: Y → Z is also in 2NF(as it does not violate the definition of 2NF)

Hence it is in 2NF

PES UNIVERSITY

1. Given a relation R(X, Y, Z) and Functional Dependency set FD = { X → Y and Y → Z }, determine whether the given R is in 3NF? If not convert it into 3 NF.

Step 5: Check For 3 NF (A relational schema R is said to be in 3NF, First, it should be in 2NF and, no non-prime attribute should be transitively dependent on the Key of the table.)

FD: X → Y is in 3NF (as X is a super Key)
FD: Y → Z is not in 3NF (as neither Y is Key nor Z is a prime attribute)
Hence R is not in 3NF.

Step 6: Convert the table R(X, Y, Z) into 3NF:
Decompose the table
FD: Y → Z was creating issue, hence one table R1(Y, Z)
Create one Table for key X, R2(X, Y), since X → Y

Hence decomposed tables which are in 3NF are:
R1(X, Y)
R2(Y, Z)

Database Management Systems

Numerical Examples on Normalization:BCNF

PES UNIVERSITY

1. Consider a Relation R with five attributes A,B,C,D,E. The functional dependencies are
FD = {A -> BC, C -> DE) Find if it is in BCNF or not?

Database Management Systems

Numerical Examples on Normalization:3NF

1. Consider a Relation R with five attributes A,B,C,D,E. The functional dependencies are
FD = {A -> BC, C -> DE) Find if it is in BCNF or not?

Step 1: Candidate key is A.

Step 2: check for each FDs:
A -> BC (determinant is a CK, No violation of BCNF)
C->DE (C is not a CK, hence violation of BCNF)

Step 3: Decomposition of relations
R1(A,BC)
R2(C,D,E)
Hence, R1 and R2 is in BCNF

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Practice Problems

Prof. Mahitha G

Dept of Computer Science and Engineering

Acknowledgments: Teaching Assistants

PES UNIVERSITY

Database Management Systems

Unit 3: Design Concepts and Implementation

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G

Dept of Computer Science and Engineering

Database Management Systems

Numerical Examples on Closure

PES UNIVERSITY

1. Given relational schema R(P Q R S T U V) having following attribute P Q R S T U and V, also there is a set of functional dependency denoted by FD = { P->Q, QR->ST, PTU->V }.
   Determine Closure of $(QR)^+$ and $(PR)^+$

1. Given relational schema R(P Q R S T) having following attributes P Q R S and T, also there is a set of functional dependency denoted by FD = { P->QR, RS->T, Q->S, T-> P }.
   Determine Closure of $(T)^+$

Database Management Systems

Numerical Examples on Equivalence of Two Sets of Functional Dependencies

PES UNIVERSITY

In DBMS, Two different sets of functional dependencies for a given relation may or may not be equivalent. If F and G are the two sets of functional dependencies, then following 3 cases are possible-

Case-01: F covers G ($F \supseteq G$)
Case-02: G covers F ($G \supseteq F$)
Case-03: Both F and G cover each other ($F = G$)

Database Management Systems

Numerical Examples on Equivalence of Two Sets of Functional Dependencies

PES UNIVERSITY

1. **A relation R (A , C , D , E , H) is having two functional dependencies sets F and G as shown-**

Set F-

A → C
AC → D
E → AD
E → H

Set G-

A → CD
E → AH

Which of the following holds true?
(A) G ⊇ F
(B) F ⊇ G
(C) F = G
(D) All of the above

Numerical Examples on Equivalence of Two Sets of Functional Dependencies

Solution-

CASE 1: Determining whether F covers G-

Step-01: $(A)^+ = \{ A , C , D \}$ // closure of left side of A → CD using set G
$(E)^+ = \{ A , C , D , E , H \}$ // closure of left side of E → AH using set G

Step-02: $(A)^+ = \{ A , C , D \}$ // closure of left side of A → CD using set F
$(E)^+ = \{ A , C , D , E , H \}$ // closure of left side of E → AH using set F

Step-03: Comparing the results of Step-01 and Step-02, we find-
Functional dependencies of set F can determine all the attributes which have been determined by the functional dependencies of set G.
Thus, we conclude F covers G i.e. $F \supseteq G$.

Numerical Examples on Equivalence of Two Sets of Functional Dependencies

Solution-

CASE 2: Determining whether G covers F-

Step-01: $(A)^+ = \{ A , C , D \}$ // closure of left side of A → C using set F
$(AC)^+ = \{ A , C , D \}$ // closure of left side of AC → D using set F
$(E)^+ = \{ A , C , D , E , H \}$ // closure of left side of E → AD and E → H using set F

Step-02: $(A)^+ = \{ A , C , D \}$ // closure of left side of A → C using set G
$(AC)^+ = \{ A , C , D \}$ // closure of left side of AC → D using set G
$(E)^+ = \{ A , C , D , E , H \}$ // closure of left side of E → AD and E → H using set G

Step-03: Comparing the results of Step-01 and Step-02, we find-
Functional dependencies of set G can determine all the attributes which have been determined by the functional dependencies of set F.
Thus, we conclude G covers F i.e. $G \supseteq F$.

Numerical Examples on Equivalence of Two Sets of Functional Dependencies

Solution-

CASE 3: Determining whether both F and G cover each other-

From Step-01, we conclude F covers G.
From Step-02, we conclude G covers F.
Thus, we conclude both F and G cover each other i.e. F = G.

Thus, Option (D) is correct.

Numerical Examples on Equivalence of Two Sets of Functional Dependencies

PES UNIVERSITY

Practice examples:

Q 1. Suppose, a relational schema R (A, B, C) and set of functional dependencies F and G are as follow:

F : { A → B,
B → C,
C → A }

G : { A → BC,
B → A,
C → A }

Check the equivalency of functional dependencies F and G.

Q 2. Suppose, a relational schema R (v w x y z) and set of functional dependencies F and G are as follow:

F : { w → x,
wx → y,
z → wy,
z → v }

G :{ w → xy,
z → wx}

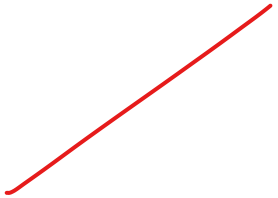
Check the equivalency of functional dependencies F and G.

Database Management Systems

Numerical Examples on Minimal Covers

PES UNIVERSITY

1. **Find the minimal cover of the set of functional dependencies given; {A → C, AB → C, C → DI, CD → I, EC → AB, EI → C}**



Database Management Systems

Numerical Examples on Minimal Covers

2. The following functional dependencies hold true for the relational scheme

R (W , X , Y , Z) –

X → W

WZ → XY

Y → WXZ

Write the irreducible equivalent (Minimal cover) for this set of functional dependencies.

ANSWER: x-->w, y-->xz

Database Management Systems

Numerical Examples on Normalization:2NF

PES UNIVERSITY

1. Given a relation R(P, Q, R, S, T, U, V, W, X, Y) and Functional Dependency set FD = { PQ → R, PS → VW, QS → TU, P → X, W → Y }, determine whether the given R is in 2NF? If not convert it into 2 NF.

Numerical Examples on Normalization:2NF

1. Given a relation R(P, Q, R, S, T, U, V, W, X, Y) and Functional Dependency set FD = { PQ → R, PS → VW, QS → TU, P → X, W → Y }, determine whether the given R is in 2NF? If not convert it into 2 NF.

Solution:

Step 1: Find out candidate key from the above diagram using closure approach.

PQS + = P Q S R T U V W X Y

Since the closure of PQS contains all the attributes of R, hence **PQS is Candidate Key**

Step 2: find prime and non prime attribute

Since R has 10 attributes: - P, Q, R, S, T, U, V, W, X, Y, prime attribute(part of candidate key) are P, Q, and S while non-prime attribute are R, T, U, V, W, X and Y

Database Management Systems

Numerical Examples on Normalization:2NF

1. Given a relation R(P, Q, R, S, T, U, V, W, X, Y) and Functional Dependency set FD = { PQ → R, PS → VW, QS → TU, P → X, W → Y }, determine whether the given R is in 2NF? If not convert it into 2 NF.

Step 2: Check the existing FDs.

FD: **PQ → R** does not satisfy the definition of 2NF, that non-prime attribute(R) is partially dependent on part of candidate key PQS

FD: **PS → VW** does not satisfy the definition of 2NF, that non-prime attribute(VW) is partially dependent on part of candidate key PQS

FD: **QS → TU** does not satisfy the definition of 2NF, that non-prime attribute(TU) is partially dependent on part of candidate key PQS

FD: **P → X** does not satisfy the definition of 2NF, that non-prime attribute(X) are partially dependent on part of candidate key PQS

FD: **W → Y** does **not violate** the definition of 2NF, as the non-prime attribute(Y) is dependent on the non-prime attribute(W), which is not related to the definition of 2NF.

Database Management Systems

Numerical Examples on Normalization:2NF

1. Given a relation R(P, Q, R, S, T, U, V, W, X, Y) and Functional Dependency set FD = { PQ → R, PS → VW, QS → TU, P → X, W → Y }, determine whether the given R is in 2NF? If not convert it into 2 NF.

Step 5: Decompose the table

R1(P, Q, R) (Now in table R1 FD: PQ → R is Full F D, hence R1 is in 2NF)

R2(P, S, V, W) (Now in table R2 FD: PS → VW is Full F D, hence R2 is in 2NF)

R3(Q, S, T, U) (Now in table R3 FD: QS → TU is Full F D, hence R3 is in 2NF)

R4(P, X) (Now in table R4 FD : P → X is Full F D, hence R4 is in 2NF)

R5(W, Y) (Now in table R5 FD: W → Y is Full F D, hence R2 is in 2NF)

And create one table for the key, since the key is PQS.

Finally, the decomposed tables which is in 2NF are:

R1(P, Q, R)
R2(P, S, V, W)
R3(Q, S, T, U)
R4(P, X)
R5(W, Y)
R6(P, Q, S)

Numerical Examples on Normalization:2NF

2. Practice Example

Given a relation R(A, B, C, D, E) and Functional Dependency set FD = { A → B, B → E, C → D}, determine whether the given R is in 2NF? If not convert it into 2 NF.

4 tables
a,b
c,d
b,e
a,c (for keys)

Database Management Systems

Numerical Examples on Normalization:3NF

PES UNIVERSITY

1. Given a relation R(X, Y, Z, W, P) and Functional Dependency set FD = { X → Y, Y → P, and Z → W}, determine whether the given R is in 3NF? If not convert it into 3 NF.

Database Management Systems

Numerical Examples on Normalization:3NF

1. Given a relation R(X, Y, Z, W, P) and Functional Dependency set FD = { X → Y, Y → P, and Z → W}, determine whether the given R is in 3NF? If not convert it into 3 NF.

Solution:

Step 1: find out candidate key:

XZ + = XZYPW

Since the closure of XZ contains all the attributes of R, hence **XZ is Candidate Key**

Step 2: (First check for 2NF) find prime and non prime attributes:

R has 5 attributes: - X, Y, Z, W, P and Candidate Key is XZ, Therefore, prime attribute (part of candidate key) are X and Z while a non-prime attribute are Y, W, and P

Step 2: Check the existing FDs

FD: **X → Y** does not satisfy the definition of 2NF, (as key is breaking)

FD: **Z → W** does not satisfies the definition of 2NF, (as key is breaking)

Numerical Examples on Normalization:3NF

1. Given a relation R(X, Y, Z, W, P) and Functional Dependency set FD = { X → Y, Y → P, and Z → W}, determine whether the given R is in 3NF? If not convert it into 3 NF.

Step 4: Decompose the relation R for 2NF

R1(X,Y) , R2(Z,W), R3(X, Z), R4(X, Y,P)

Step 5: Now Check For 3 NF (for R4)

FD: X → Y is ~~in NOT~~ in 3NF (~~as neither X is a super Key nor~~ Y ~~is a prime attrib~~ute)

X is a superkey and Y is not a prime attribute so follows 3NF

FD: Y → P is not in 3NF (as neither Y is Key nor is a prime attribute)

Hence R4 is not in 3NF.

Step 6: Convert the table R(X, Y, Z,W, P) into 3NF:

Decompose the table (R3)

Hence decomposed tables which are in 3NF are:

R1(X, Y) {Using FD X → Y}

R2(Z, W) {Using FD Z → W}

R3(X, Z) { Using Candidate Key XZ }

R4(Y, P) {Using FD Y → P}

Numerical Examples on Normalization:3NF

2. Given a relation R(P, Q, R, S, T, U, V, W, X, Y) and Functional Dependency set FD = { PQ → R, P → ST, Q → U, U → VW, and S → XY}, determine whether the given R is in 3NF? If not convert it into 3 NF.

2. Given a relation R(P, Q, R, S, T, U, V, W, X, Y) and Functional Dependency set FD = { PQ → R, P → ST, Q → U, U → VW, and S → XY}, determine whether the given R is in 3NF? If not convert it into 3 NF.

Solution:

Step 1: find out candidate key:

PQ + = P Q R S T U X Y V W

Since the closure of XZ contains all the attributes of R, hence **PQ is Candidate Key**

Step 2: (First check for 2NF) find prime and non prime attributes:

R has 10 attributes prime attribute (part of candidate key) are P and Q while a non-prime attribute are R S T U V W X Y V W

Database Management Systems

Numerical Examples on Normalization:3NF

2. Given a relation R(P, Q, R, S, T, U, V, W, X, Y) and Functional Dependency set FD = { PQ → R, P → ST, Q → U, U → VW, and S → XY}, determine whether the given R is in 3NF? If not convert it into 3 NF.

Step 4: Check the existing FDs

FD: **PQ → R** satisfy the definition of 3NF, as PQ Super Key

FD: **P → ST** does not satisfy the definition of 3NF, that neither P is Super Key nor ST is the prime attribute

P is not a superkey alone as it doesn'tdetermine all the attributes of the relation
that means closure of P+_ should contain all the attributes of relation

FD: **Q → U** does not satisfy the definition of 3NF, that neither Q is Super Key nor U is a prime attribute

FD: **U → VW** does not satisfy the definition of 3NF, that neither U is Super Key nor VW is a prime attribute

FD: **S → XY** does not satisfy the definition of 3NF, that neither S is Super Key nor XY is a prime attribute

★ STEP 3 — 3NF DECOMPOSITION
We create a separate table for every violating FD, and also keep one key table.
✔ R1(P, Q, R)(from PQ → R)
✔ R2(P, S, T)(from P → ST)
✔ R3(Q, U)(from Q → U)
✔ R4(U, V, W)(from U → VW)
✔ R5(S, X, Y)(from S → XY)
✔ R6(P, Q)(to preserve the candidate key)

Database Management Systems

Numerical Examples on Normalization:BCNF

PES UNIVERSITY

1. Consider a Relation R with four attributes W,X,Y,Z
The functional dependencies are F = {WX -> Y, X -> Z; Y -> W}
Find if it is in BCNF or not?

Database Management Systems

Numerical Examples on Normalization:BCNF

1. Consider a Relation R with four attributes W,X,Y,Z
The functional dependencies are F = {WX -> Y, X -> Z; Y -> W}
Find if it is in BCNF or not?

you have to try with all as XZ+ doesn't determine all of them

Step 1: Candidate key is WX and XY.

candidate key

Step 2: check for each FDs:
WX -> Y(determinant is a CK, No violation of BCNF)
X->Z (X is not a KEY, hence violation of BCNF)
Y -> W (Y is not a KEY, hence violation of BCNF)

Step 3: Decomposition of relations
R1(W,X,Y)
R2(X,Z)
R3(Y,W)
Hence, R1,R2 and R3 are in BCNF

Database Management Systems

Numerical Examples on Normalization:BCNF

PES UNIVERSITY

2. Consider the below mentioned relation and Find if it is in BCNF or not?

Author	Nationality	Book title	Genre	Number of pages
William Shakespeare	English	The Comedy of Errors	Comedy	100
Markus Winand	Austrian	SQL Performance Explained	Textbook	200
Jeffrey Ullman	American	A First Course in Database Systems	Textbook	500
Jennifer Widom	American	A First Course in Database Systems	Textbook	500

to find candidate key first find all the attributes that have unique values
we see only author and booktitle has unqique value

now you so they are essential
now find closure and check

Database Management Systems

Numerical Examples on Normalization:BCNF

author and booktitle separately are not superekys and can't determine all others attributes of whole table
so they don't satisfy bcnf rule of lhs being superkey

PES UNIVERSITY

2. Consider the below mentioned relation and Find if it is in BCNF or not?

Step 1: Candidate key is {author, book title}.

Step 2: These FDs are existed in this relation:
author → nationality
book title → genre, number of pages
And both of the FDs violating BCNF rules

Step 3: Decomposition of relations
R1(author, nationality)
R2(booktitle, genre, number of pages)
R3(Author, booktitle)

Hence, R1,R2 and R3 are in BCNF

Author	Nationality	Book title	Genre	Number of pages
William Shakespeare	English	The Comedy of Errors	Comedy	100
Markus Winand	Austrian	SQL Performance Explained	Textbook	200
Jeffrey Ullman	American	A First Course in Database Systems	Textbook	500
Jennifer Widom	American	A First Course in Database Systems	Textbook	500

Database Management Systems

Numerical Examples on Minimal Covers

PES UNIVERSITY

References:

https://www.javatpoint.com/dbms-questions-on-normalization

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

PES UNIVERSITY

Database Management Systems

Unit 3: Design Concepts and Implementation

Slides adapted from Author Slides of "Database System Concepts", Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems", Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Prof. Mahitha G

Dept of Computer Science and Engineering

Introduction

Query Processing *includes translations of **high-level** queries **into low-level** expressions that can be used at the physical level of the file system, query optimization, and actual execution of the query to get the actual result.*

- High-level SQL is translated into low-level operations/expressions.
- This translation is systematic (methodical) across the query pipeline.
- It's applied at the physical storage level and during query optimization.
- The final low-level plan is executed to produce the result efficiently.

Introduction

1. A query expressed in a high-level query language such as SQL must **first be scanned, parsed, and validated.**
2. The **scanner i**dentifies the **query tokens such as SQL keywords, attribute names, and relation names** that appear in the text of the query,
3. The **parser** checks the **query syntax to determine whether it is formulated according to the syntax rules of the query language**. The query must also be validated by checking that all attribute and relation names are valid and semantically meaningful names in the schema of the particular database being queried.

Introduction

- The **query optimizer module** has the **task of producing a good execution plan**, and
- the **code generator** **generates the code to execute that plan**.
- The **runtime database processor** has the **task of executing the query code to produce the query result.**
- An **internal representation** of the query is then created, usually as a **tree data structure called a query tree.** It is also possible to *represent the query using a graph data structure called a query graph, which is generally a directed acyclic graph (DAG).*

Introduction

- The **DBMS** *must then devise an execution strategy or query plan for retrieving the results of the query from the database files.*
- A query has **many possible execution strategies**, and *the process of choosing a suitable one for processing a query is known as query optimization*

PES UNIVERSITY

DBMS techniques to process a query

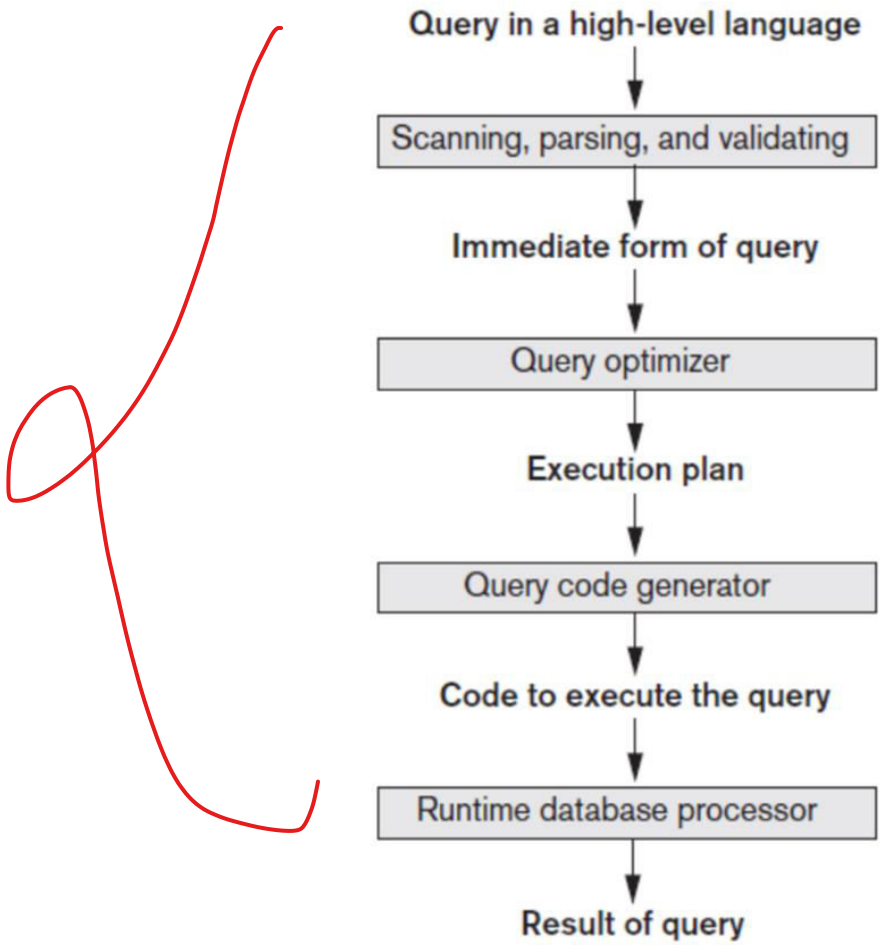
- Scanner identifies query tokens
- Parser checks the query syntax
- Validation checks all attribute and relation names
- Query tree (or DAG) created
- Execution strategy or query plan devised

Query optimization

- Planning a good execution strategy

Database Management Systems

Query Processing



Code can be:

Executed directly (interpreted mode)

Stored and executed later whenever needed (compiled mode)

Figure 18.1 Typical steps when processing a high-level query

Translating SQL Queries into Relational Algebra and Other Operators

In practice, SQL is the query language that is used in most commercial RDBMSs. An SQL **query is first translated into an equivalent extended relational algebra expression—represented as a query tree data structure—that is then optimized.**

SQL queries are **decomposed into query blocks**, which form the **basic units** that can be **translated into the algebraic operators** and **optimized**

- **Query decomposed into query blocks**
  - Basic units that can be translated into the algebraic operators
  - Contains single SELECT-FROM-WHERE expression
  - May contain GROUP BY and HAVING clauses

Translating SQL Queries into Relational Algebra and Other Operators

Example :

```
SELECT Lname, Fname
FROM EMPLOYEE
WHERE Salary > ( SELECT MAX (Salary)
                 FROM   EMPLOYEE
                 WHERE  Dno=5 );
```

Inner Block :

```
( SELECT MAX (Salary)
  FROM   EMPLOYEE
  WHERE  Dno=5 )
```

Outer Block :

```
SELECT  Lname, Fname
FROM    EMPLOYEE
WHERE Salary > c
```

Note: c represents the result returned from the inner block

Translating SQL Queries into Relational Algebra and Other Operators

Inner Block translated into :

$$\Im_{\text{MAX Salary}}(\sigma_{\text{Dno}=5}(\text{EMPLOYEE}))$$

Outer Block translated into :

$$\pi_{\text{Lname,Fname}}(\sigma_{\text{Salary}>c}(\text{EMPLOYEE}))$$

Query optimizer chooses execution plan for each query block

Database Management Systems

Additional Operators : Semi-Join and Anti-Join

Most modern SQL engines (MySQL, PostgreSQL, etc.) convert EXISTS / NOT EXISTS subqueries into semi joins and anti joins internally for performance.

This helps avoid redundant joins or large intermediate result sets.

Semi-Join (S=) generally used for **unnesting EXISTS, IN, and ANY** subqueries

Syntax: **T1.X S = T2.Y**

T1 is the left table and T2 is the right table of the semi-join

A row of T1 is returned as soon as T1.X finds a match with any value of T2.Y without searching for further matches. This is in contrast to finding all possible matches in inner join.

Database Management Systems

Additional Operators : Semi-Join and Anti-Join

Anti-join (A=) is also used for unnesting NOT EXISTS, NOT IN, and ALL subqueries.

Syntax: T1.x A = T2.y

where T1 is the left table and T2 is the right table of the anti-join.
A row of T1 is rejected as soon as T1.x finds a match with any value of T2.y.
A row of T1 is returned, only if T1.x does not match with any value of T2.y.

```
SELECT sname
FROM supplier
WHERE sname IN (
    SELECT sname
    FROM supply
    WHERE part_name = 'Bolt'
);

SELECT sname
FROM supplier S
WHERE EXISTS (
    SELECT *
    FROM supply SP
    WHERE SP.sname =
S.sname
);
```

```
SELECT DISTINCT
supplier.sname
FROM supplier
JOIN supply ON supplier.sname
= supply.sname
WHERE supply.part_name =
'Bolt';

SELECT DISTINCT S.sname
FROM supplier S
JOIN supply SP ON SP.sname
= S.sname;
```

Database Management Systems

Algorithms for SELECT Operation

There are many algorithms for executing a SELECT operation, which is basically a search operation to locate the records in a disk file that satisfy a certain condition.

- Search operation to locate records in a disk file that satisfy a certain condition
- File scan or index scan (if search involves an index)

Search Methods for Simple Selection

- A number of **search algorithms** are possible for selecting records from a file.
- *These are also known as **file scans**, because they scan the records of a file to search for and retrieve records that satisfy a selection condition.*
- *If the search algorithm involves the use of an index, the index search is called an **index scan**.*

Algorithms for SELECT Operation

Consider the following operations to understand the algorithms

OP1: $\sigma_{Ssn = '123456789'}$ (EMPLOYEE)

OP2: $\sigma_{Dnumber > 5}$ (DEPARTMENT)

OP3: $\sigma_{Dno = 5}$ (EMPLOYEE)

OP4: $\sigma_{Dno = 5 \text{ AND } Salary > 30000 \text{ AND } Sex = 'F'}$ (EMPLOYEE)

OP5: $\sigma_{Essn='123456789' \text{ AND } Pno = 10}$(WORKS_ON)

Search Methods for Simple Selection

The below search methods are **examples of some of the search algorithms** that can be used to implement a select operation:

S1. Linear Search (Brute Force Algorithm)

- *Retrieve **every record** in the file, and test whether its attribute values satisfy the selection condition*.
- Since the records are grouped into disk blocks, each disk block is read into a main memory buffer, and then a search through the records within the disk block is conducted in main memory.

S2. Binary Search

- If the **selection condition involves an equality comparison on a key attribute** on which the file is **ordered**, binary search can be used.
- It is more efficient than linear search
- An example is OP1 if Ssn is the ordering attribute for the EMPLOYEE file.

Search Methods for Simple Selection

PES UNIVERSITY

S3a. Using a Primary Index

- If the *selection condition involves an equality comparison on a **key attribute** with a primary index, we use the primary index to retrieve the record.*
- Note that this condition retrieves a single record (at most).
- For example, Ssn = '123456789' in OP1

yeah makes sense no
if you are using a primary key that value will be unique for that attribute and will come once only so only one record with SSN=123456789 will be retrieved

S3b. Using a Hash Key

- If the *selection condition involves an equality comparison on a **key attribute** with a hash key*, we use the hash key to retrieve the record.
- Note that this condition retrieves a single record (at most).
- For example, Ssn = '123456789' in OP1

hash also produces one unqiue value for one comparison no

PES UNIVERSITY

S4. Using a Primary Index to Retrieve Multiple Records

- If *the comparison condition is >, >=, <, or <= on a key field with a primary index, we use the index to find the record satisfying the corresponding equality condition , then retrieve all subsequent records in the (ordered) file.*
- For example, Dnumber > 5 in OP2
- For the condition Dnumber < 5, retrieve all the preceding records.

There are many more methods for simple selection, conjunctive selection, disjunctive selection.

PES UNIVERSITY

JOIN Operation

- The JOIN operation is one of the **most time-consuming operations** in query processing.
- Many of the join operations encountered in queries are of the EQUIJOIN and NATURAL JOIN varieties
- There are many possible ways to implement a **two-way join**, which is a join on two files. Joins involving more than two files are called **multiway joins**.

PES UNIVERSITY

Techniques for implementing two-way joins:

J1. Nested-Loop Join (Nested-Block Join)

- This is the **default (brute force) algorithm**, as it does not require any special access paths on either file in the join.
- For each **record *t* in the outer loop *R***, retrieve every **record *s* from the inner loop *S*** and test whether the two records satisfy the join condition ***t*[*A*] = *s*[*B*]**.

Employee(EID, Name, DeptID)

EID	Name	DeptID
1	John	10
2	Mary	20
3	Alex	10

Department(DeptID, DeptName)

DeptID	DeptName
10	HR
20	Finance

Join Condition: Employee.DeptID = Department.DeptID

For each employee, it checks all the departments until it finds a match in the DeptID

An index is a lookup structure (like B-tree/hash) that lets the database quickly find rows matching a value — so the join does not need to scan the entire inner table.

PES UNIVERSITY

J2. Index-Based Nested-Loop Join

- Improves on nested-loop join by using an index on the join attribute of the inner relation.
- Instead of scanning the inner table completely, use the index to find matching tuples quickly.

Employee(EID, Name, DeptID)

EID	Name	DeptID
1	John	10
2	Mary	20
3	Alex	10

Department(DeptID, DeptName)

DeptID	DeptName
10	HR
20	Finance

Join Condition: Employee.DeptID = Department.DeptID

Suppose an index exists on Department.DeptID
For each Employee.DeptID, directly **look up** matching DeptID using the index. Thus, the whole process is much faster

J3. Sort-Merge Join

- *If the records of R and S are physically sorted (ordered) by value of the join attributes A and B, respectively,* we can implement the join in the most efficient way possible.
- Both files are scanned concurrently in order of the join attributes, matching the records that have the same values for *A* and *B*.
- Both relations are **sorted** on the join attribute then merged in a single pass (similar to merge step in merge sort).

we use 2 pointers consider i and j , i for LHS table join attribute and j for RHS table join attribute and start joining and whenever something doesnt' join move front this saves time and doesn't nested for loops making complexity O(m+n) and everything happens in single pass

Student(SID, Name, DeptID)

SID	Name	DeptID
1	Adam	10
2	Bella	20
3	Chris	30

Department(DeptID, DeptName)

DeptID	DeptName
10	CS
20	IT
30	ECE

Here,

- Both tables are sorted on DeptID.
- Merge by scanning both once - Compare DeptIDs sequentially and output matching pairs.

Database Management Systems

Methods for Implementing Joins

★ Partition-Hash Join (J4) — Short Explanation
1⃣ Phase 1 — Partitioning
- Use a hash function h() on the join attribute (example: CID).
- Split both tables R and S into smaller partitions (buckets).
- All rows with the same hash value go to the same bucket.
- Smaller table's buckets are kept in main memory.

Example: h(CID) = CID % 3
- CID C1 → bucket 1
- CID C2 → bucket 2
- CID C3 → bucket 3

2⃣ Phase 2 — Probing (Joining)
- Read the second table (S).
- For each row, compute the same hash h().
- Go directly to the matching bucket and join only those rows, not the whole table.

➡ This makes the join much faster than searching whole tables.

J4. Partition-Hash Join

- The records of **files *R* and *S* are partitioned into smaller files.**
- The **partitioning** of each file is done using the same **hashing function *h*** on the join attribute *A* of *R* (for partitioning file R) and *B* of *S* (for partitioning file S).

➔First, a single pass through the file with fewer records (say, *R*) **hashes its records to the various partitions of R;** this is called the **partitioning phase**, since the records of *R* are partitioned into the hash buckets.

- In the simplest case, we assume that the smaller file can fit entirely in main memory after it is partitioned, so that the *partitioned subfiles of R are all kept in main memory.*

PES UNIVERSITY

J4. Partition-Hash Join

- The **collection of records with the same value of $h(A)$ are placed in the same partition,** which is a **hash bucket** in a hash table in main memory.

➔

In the **second phase, called the probing phase,** a single pass through the other file (S) then hashes each of its records using the same hash function $h(B)$ to *probe* the appropriate bucket, and that record is combined with all matching records from R in that bucket.

- This simplified description of partition-hash join assumes that the smaller of the two files *fits entirely into memory buckets* after the first phase.

Methods for Implementing Joins

Example :

Orders(OID, CID, Amount)

OID	CID	Amount
1	C1	200
2	C2	300
3	C3	150

Customers(CID, Name)

CID	Name
C1	John
C2	Mary
C3	Alex

Here,

- We first hash both tables on CID (using (CID)%3 for example)
  Partition 1: O1(C1), C1
  Partition 2: O2(C2), C2
  Partition 3: O3(C3), C3
- Then, we join each partition pair in memory

Output

OID	CID	Amount	Name
1	C1	200	John
2	C2	300	Mary
3	C3	150	Alex

Summary

- An overview of the techniques used by DBMSs in processing high-level queries.
- SQL queries are translated into relational algebra.
- The operations of semi-join and anti-join, to which nested queries are mapped to avoid doing the regular inner join.
- External sorting, is commonly needed during query processing to order the tuples of a relation while dealing with aggregation, duplicate elimination.
- Cases of selection and the algorithms employed for simple selection based on one attribute and complex selections using conjunctive and disjunctive clauses.

References:

- Selection Algorithms
- Join Algorithms

Query optimization

PES UNIVERSITY

What is Query Optimization?

- The process of improving the efficiency of SQL queries.
- Ensures faster data retrieval and reduced resource usage.
- Goal: *Minimize execution time and maximize performance.*

Why Optimization is Important

Large databases can slow down poorly written queries.
Efficient queries save CPU, memory, and I/O.
Improves user experience and scalability.

Query optimization

PES UNIVERSITY

Common Causes of Slow Queries

- **Missing or unused indexes:**
  Queries take longer as MySQL scans the entire table instead of using indexes.
- *SELECT using **
  Fetches unnecessary columns, increasing data retrieval time.
- **Complex joins and nested subqueries:**
  Increase computation and memory usage due to multiple table scans.
- **Functions in WHERE clauses:**
  Prevent index usage, forcing full table scans.
- **Large unfiltered data sets:**
  Processing massive unfiltered rows slows query execution.

Query optimization

PES UNIVERSITY

Key Optimization Techniques

1. **Use Indexes Wisely**
   → Speed up searches and joins
2. **Retrieve Only Needed Columns**
   → Reduce data transfer and processing time
3. **Optimize Joins**
   → Join only required tables with proper indexes
4. **Filter Early Using WHERE**
   → Limit rows before further operations
5. **Use LIMIT for Large Results**
   → Fetch only the needed records quickly

Database Management Systems

Using EXPLAIN for Query Analysis

PES UNIVERSITY

- The MySQL EXPLAIN is a powerful tool for analyzing and optimizing SQL queries.
- It provides insight into **how MySQL executes a query including the details about the chosen execution plan** the order in which tables are accessed and the use of the indexes.
- Understanding the output of the **EXPLAIN** can help database developers and administrators identify performance bottlenecks and optimize query performance.

- Syntax :

```
EXPLAIN SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

- When you execute an **SQL** query prefixed with the **EXPLAIN** MySQL returns information about how it plans to execute the query. This information is presented in the form of a table with the various columns each providing insights into the different aspects of the query execution.

```
mysql> EXPLAIN SELECT E.Fname, E.Lname
    -> FROM EMPLOYEE E
    -> WHERE NOT EXISTS (
    ->     SELECT Pno
    ->     FROM WORKS_ON W
    ->     JOIN EMPLOYEE S ON S.Ssn = W.Essn
    ->     WHERE S.Lname = 'Smith'
    ->       AND S.Fname = 'John'
    ->       AND W.Pno NOT IN (
    ->             SELECT W2.Pno
    ->             FROM WORKS_ON W2
    ->             WHERE W2.Essn = E.Ssn
    ->       )
    -> );
+----+--------------------+------+------------+--------+---------------------------+---------+---------+------------
--------------------+------+----------+---------------------------------------+
| id | select_type        | table | partitions | type   | possible_keys             | key     | key_len | ref
                   | rows | filtered | Extra                                 |
+----+--------------------+------+------------+--------+---------------------------+---------+---------+------------
--------------------+------+----------+---------------------------------------+
|  1 | PRIMARY            | E     | NULL       | ALL    | NULL                      | NULL    | NULL    | NULL
                   |   40 |   100.00 | Using where                           |
|  2 | DEPENDENT SUBQUERY | S     | NULL       | ALL    | PRIMARY                   | NULL    | NULL    | NULL
                   |   40 |     2.50 | Using where                           |
|  2 | DEPENDENT SUBQUERY | W     | NULL       | ref    | PRIMARY                   | PRIMARY | 36      | companydb.S
.ssn               |    1 |   100.00 | Using index                           |
|  2 | DEPENDENT SUBQUERY | W2    | NULL       | eq_ref | PRIMARY,fk_workson_project | PRIMARY | 40      | companydb.E
.ssn,companydb.W.pno |    1 |   100.00 | Using where; Not exists; Using index |
+----+--------------------+------+------------+--------+---------------------------+---------+---------+------------
--------------------+------+----------+---------------------------------------+
4 rows in set, 2 warnings (0.013 sec)
```

Database Management Systems

MySQL EXPLAIN Example 1

- Consider a scenario where we are using the database airportdb
- Before running a query, you can use EXPLAIN to determine if the query is offloaded to MySQL HeatWave for processing.

```
mysql> EXPLAIN SELECT airline_id,
          COUNT(*) AS flight_id FROM flight
          WHERE flight_id <= 66810
          GROUP BY airline_id
          ORDER BY airline_id;

*************************** 1. row ***************************
           id: 1
  select_type: NONE
        table: NULL
   partitions: NULL
         type: NULL
possible_keys: NULL
          key: NULL
      key_len: NULL
          ref: NULL
         rows: NULL
     filtered: NULL
        Extra: Using secondary engine RAPID. Use EXPLAIN FORMAT=TREE to show the plan.
1 row in set, 1 warning (0.2521 sec)
```

HeatWave performing analytics **inside** MySQL using an in-memory parallel engine.
HeatWave is a **high-performance, in-memory, massively parallel query engine** that makes MySQL run:

1.Analytics (OLAP)
2. **Machine learning**
3.**Complex joins, aggregations, subqueries**
4.**Mixed OLTP + OLAP workloads**

Database Management Systems

MySQL EXPLAIN Example 1

- After using EXPLAIN to verify that the query can be offloaded, run the query and note the execution time.

```
mysql> SELECT airline_id, COUNT(*) AS flight_id
          FROM flight
          WHERE flight_id <= 66810
          GROUP BY airline_id
          ORDER BY airline_id;
+------------+-----------+
| airline_id | flight_id |
+------------+-----------+
|          1 |       436 |
|          2 |       430 |
|...         |           |
+------------+-----------+
113 rows in set (0.2953 sec)
```

- To **compare MySQL HeatWave query execution time with DB System execution time**, disable the use_secondary_engine variable and run the query again to see how long it takes to run on the DB System.

```
mysql> SET SESSION use_secondary_engine=OFF;

mysql> SELECT airline_id, COUNT(*) AS flight_id
          FROM flight
          WHERE flight_id <= 66810
          GROUP BY airline_id
          ORDER BY airline_id;
+------------+-----------+
| airline_id | flight_id |
+------------+-----------+
|          1 |       436 |
|          2 |       430 |
|...         |           |
+------------+-----------+
113 rows in set (0.4476 sec)
```

Database Management Systems

INDEXING - With and Without

The best way to **improve the performance of SELECT operations is to create indexes on one or more of the columns that are tested in the query.** The **index entries act like pointers to the table rows**, allowing the query to quickly determine which rows match a condition in the WHERE clause, and retrieve the other column values for those rows.

1. Without Indexing

SELECT * FROM Employee WHERE salary > 50000;

- MySQL performs a **full table scan** — every record checked.
- Cost grows linearly with table size (O(n)).

PES UNIVERSITY

2. With Indexing

```
CREATE INDEX idx_salary ON Employee(salary);
EXPLAIN SELECT * FROM Employee WHERE salary > 50000;
```

- MySQL uses the B-tree index to jump directly to qualifying records.
- Only relevant blocks read.
- Cost drastically reduced (O(log n)).

Database Management Systems

INDEXING - With and Without Example

Suppose we have a table populated with a million records and we want to select all the fields for a customer where customer_id = 12345

Without Indexing

```
| 849554 |       12345 | 3296.33 |
| 860317 |       12345 |  185.02 |
| 891240 |       12345 | 4281.51 |
| 915502 |       12345 | 3490.56 |
+--------+-------------+---------+
20 rows in set (0.15 sec)
```

With Indexing

```
| 860317 |       12345 |  185.02 |
| 891240 |       12345 | 4281.51 |
| 915502 |       12345 | 3490.56 |
+--------+-------------+---------+
20 rows in set (0.00 sec)
```

The time taken to retrieve the rows is much faster when indexing is used

References

1. **MySQL Official Documentation – Optimization Overview**
   https://dev.mysql.com/doc/refman/8.0/en/optimization.html

2. **MySQL :: Using EXPLAIN to Optimize Queries**
   https://dev.mysql.com/doc/refman/8.0/en/explain.html

THANK YOU

Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Next-Gen Data Management

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgment: Teaching Assistants

Database Management Systems

Database Application

- Database connectivity refers to integrating applications with a DBMS.
- Python + MySQL is a popular combination for lightweight and scalable solutions.
- **Use cases:** web apps, desktop apps, data analysis, automation.

Requirements

- Python (version 3+ recommended)
- MySQL Server (installed and running)
- Python
- package: mysql-connector-python
- Basic SQL knowledge

Environment Setup

Install the following tools on your system:

- **Python 3.x** – Download from https://www.python.org
- **MySQL Server** – Download from https://dev.mysql.com/downloads/mysql/
- **MySQL Workbench** *(optional but useful for GUI database management)*

Database Application

Scenario:

A university department wants to build a **lightweight desktop application** to manage and analyze student performance across various subjects. Faculty members should be able to:

- Insert new student records
- Update marks after re-evaluation
- Read all students details
- Generate subject-wise reports.
- Delete data if not required

PES UNIVERSITY
CELEBRATING 50 YEARS

Step 1. Install MySQL Python Connector:

Open Command Prompt or Terminal and run

pip install mysql-connector-python

```
E:\D DRIVE\PES\DBMS2025\FDP_Materials>pip install mysql-connector-python
Collecting mysql-connector-python
  Downloading mysql_connector_python-9.3.0-cp313-cp313-win_amd64.whl.metadata (7.7 kB)
Downloading mysql_connector_python-9.3.0-cp313-cp313-win_amd64.whl (16.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 16.4/16.4 MB 4.0 MB/s eta 0:00:00
Installing collected packages: mysql-connector-python
Successfully installed mysql-connector-python-9.3.0
```

PES UNIVERSITY
CELEBRATING 50 YEARS

Step 2. Create a Database using MySQL

```
mysql> create database university_db;
Query OK, 1 row affected (0.00 sec)

mysql> use university_db;
Database changed
mysql>
```

Note : use name as lightweightdesktopapplication

PES UNIVERSITY
CELEBRATING 50 YEARS

Step 3. Create tables in university_db database and populate the data

```
CREATE TABLE students (
    student_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    department VARCHAR(50)
);


CREATE TABLE subjects (
    subject_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100)
);


CREATE TABLE marks (
    mark_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT,
    subject_id INT,
    marks INT,
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (subject_id) REFERENCES subjects(subject_id)
);
```

```
mysql> CREATE TABLE students (
    ->     student_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     name VARCHAR(100),
    ->     department VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE subjects (
    ->     subject_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     name VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE marks (
    ->     mark_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     student_id INT,
    ->     subject_id INT,
    ->     marks INT,
    ->     FOREIGN KEY (student_id) REFERENCES students(student_id),
    ->     FOREIGN KEY (subject_id) REFERENCES subjects(subject_id)
    -> );
Query OK, 0 rows affected (0.02 sec)
```

Insert statements:

```
-- Step 1
INSERT INTO students (name, department) VALUES
('Alice Johnson', 'CSE'),
('Bob Smith', 'ECE'),
('Charlie Lee', 'ME'),
('Diana Patel', 'CSE');
```

```
-- Step 2
INSERT INTO subjects (name) VALUES
('Mathematics'),
('Data Structures'),
('Digital Electronics'),
('Thermodynamics');
```

```
-- Step 3
-- Assume student IDs 1–4, subject IDs 1–4
INSERT INTO marks (student_id, subject_id, marks) VALUES
(1, 1, 85),
(1, 2, 90),
(2, 1, 78),
(2, 3, 88),
(3, 1, 65),
(3, 4, 70),
 (4, 2, 95),
(4, 3, 89);
```

Database Management Systems

Database Application: Table creation and insertion of Data

```
mysql> INSERT INTO students (name, department) VALUES
    -> ('Alice Johnson', 'CSE'),
    -> ('Bob Smith', 'ECE'),
    -> ('Charlie Lee', 'ME'),
    -> ('Diana Patel', 'CSE');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO subjects (name) VALUES
    -> ('Mathematics'),
    -> ('Data Structures'),
    -> ('Digital Electronics'),
    -> ('Thermodynamics');
Query OK, 4 rows affected (0.00 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO marks (student_id, subject_id, marks) VALUES
    -> (1, 1, 85), -- Alice - Math
    -> (1, 2, 90), -- Alice - DS
    -> (2, 1, 78), -- Bob - Math
    -> (2, 3, 88), -- Bob - Digital
    -> (3, 1, 65), -- Charlie - Math
    -> (3, 4, 70), -- Charlie - Thermo
    -> (4, 2, 95), -- Diana - DS
    -> (4, 3, 89); -- Diana - Digital
Query OK, 8 rows affected (0.00 sec)
Records: 8  Duplicates: 0  Warnings: 0
```

Step 4: Python code for connecting to database and performing CRUD operations

- **Check for prerequisites :**

Installation of

1. Python (version 3.10 or higher) :Needed for match-case statement.
2. Python MySQL Connector:
   pip install mysql-connector-python

- **Create a python file and add code given**

- **To Run:**

1.Save this as **.py**

2.Open terminal → navigate to the file directory.

3.Run: python **filename.py** or py **filename.py**

PES UNIVERSITY
CELEBRATING 50 YEARS

How the Connection Works

You're using the mysql.connector package to connect to a MySQL server.

connect() creates a connection to the MySQL server with the given:

- **host: "localhost"** -- The server is on your local machine.
- **user: "root"** -- MySQL username.
- **password:** “yourpassword” – Password for the user.
- **database:** "lightweightdesktopapplication" -- The database you're working on.

conn is your **connection object** — it lets you communicate with the DB.

cursor = conn.cursor() creates a **cursor object** used to execute SQL statements

Database Management Systems

Database Application

import mysql.connector

```
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="yourpassword",
    database="university_db"
)
cursor = conn.cursor()
```

PES UNIVERSITY
CELEBRATING 50 YEARS

What is a cursor?

- A **cursor** is an object used to interact with the database.
- It is used to **Execute SQL queries** (SELECT, INSERT, UPDATE, DELETE).
- It **Fetches results** using:
  - fetchone() → one row
  - fetchmany(n) → n rows
  - fetchall() → all rows
- It Works **within a connection** and requires conn.commit() for changes.

PES UNIVERSITY
CELEBRATING 50 YEARS

1. Inserting New Data

We have a function to add a **new student** to the student table.

%s is a placeholder — values are passed as a tuple (name, dept) to prevent SQL injection.

conn.commit() saves the changes to the database.

Similar logic applies to:

- insert_subject(name)
- insert_marks(student_id, subject_id, marks)

```
def insert_student(name, dept):
    cursor.execute("INSERT INTO students (name, department) VALUES (%s, %s)", (name, dept))
    conn.commit()
    print(f"\nStudent '{name}' added.")
```

```
insert_student("Emma", "CSE")
insert_student("John", "ECE")
```

Database Application

PES UNIVERSITY
CELEBRATING 50 YEARS

Output:

```
Student 'Emma' added.

Student 'John' added.

Subject 'Databases' added.

Subject 'Algorithms' added.

Marks 85 inserted for Student 5, Subject 5.

Marks 90 inserted for Student 5, Subject 6.

Marks 78 inserted for Student 6, Subject 5.
```

2. Reading Data from db

cursor.execute("SELECT * FROM students") – Runs an SQL SELECT query.

cursor.fetchall() retrieves all results.

The function list_tables() displays:

- All students
- All subjects
- All marks, joined with names for readability

Database Management Systems

Database Application

```python
def list_tables():
    print("\nStudents:")
    cursor.execute("SELECT * FROM students")
    for row in cursor.fetchall():
        print(row)

    print("\nSubjects:")
    cursor.execute("SELECT * FROM subjects")
    for row in cursor.fetchall():
        print(row)

    print("\nMarks:")
    cursor.execute("""
        SELECT m.mark_id, s.name, sub.name, m.marks
        FROM marks m
        JOIN students s ON m.student_id = s.student_id
        JOIN subjects sub ON m.subject_id = sub.subject_id
    """)
    for row in cursor.fetchall():
        print(f"Mark ID: {row[0]}, Student: {row[1]}, Subject: {row[2]}, Marks: {row[3]}")
```

```python
# READ
list_tables()
```

Database Application

PES UNIVERSITY
CELEBRATING 50 YEARS

Output:

```
Students:
(1, 'Alice Johnson', 'CSE')
(2, 'Bob Smith', 'ECE')
(3, 'Charlie Lee', 'ME')
(4, 'Diana Patel', 'CSE')
(5, 'Emma', 'CSE')
(6, 'John', 'ECE')

Subjects:
(1, 'Mathematics')
(2, 'Data Structures')
(3, 'Digital Electronics')
(4, 'Thermodynamics')
(5, 'Databases')
(6, 'Algorithms')

Marks:
Mark ID: 1, Student: Alice Johnson, Subject: Mathematics, Marks: 85
Mark ID: 2, Student: Alice Johnson, Subject: Data Structures, Marks: 90
Mark ID: 3, Student: Bob Smith, Subject: Mathematics, Marks: 78
Mark ID: 4, Student: Bob Smith, Subject: Digital Electronics, Marks: 88
Mark ID: 5, Student: Charlie Lee, Subject: Mathematics, Marks: 65
Mark ID: 6, Student: Charlie Lee, Subject: Thermodynamics, Marks: 70
Mark ID: 7, Student: Diana Patel, Subject: Data Structures, Marks: 95
Mark ID: 8, Student: Diana Patel, Subject: Digital Electronics, Marks: 89
Mark ID: 9, Student: Emma, Subject: Databases, Marks: 85
Mark ID: 10, Student: Emma, Subject: Algorithms, Marks: 90
Mark ID: 11, Student: John, Subject: Databases, Marks: 78
```

PES UNIVERSITY
CELEBRATING 50 YEARS

Reading Data from db–to generate a report

SELECT m.mark_id, s.name, sub.name, m.marks
FROM marks m
JOIN students s ON m.student_id = s.student_id
JOIN subjects sub ON m.subject_id = sub.subject_id

On running this command, we can get an overall report for each student based on their ID

```
def generate_subject_report(subject_id):
    cursor.execute("""
        SELECT s.name, m.marks
        FROM marks m
        JOIN students s ON m.student_id = s.student_id
        WHERE m.subject_id = %s
    """, (subject_id,))
    print(f"\nReport for Subject ID {subject_id}:")
    for row in cursor.fetchall():
        print(f"Student: {row[0]} → Marks: {row[1]}")
```

PES UNIVERSITY
CELEBRATING 50 YEARS

Output:

```
generate_subject_report(1)   # Report for Maths
```

```
Report for Subject ID 1:
Student: Alice Johnson → Marks: 95
Student: Charlie Lee → Marks: 65
```

PES UNIVERSITY
CELEBRATING 50 YEARS

3. Update data in db

The Update Function :

Updates the marks table for the given student.

conn.commit() is needed to apply the change.

%s placeholders are replaced securely with values.

```
def update_marks(student_id, subject_id, new_marks):
    cursor.execute("""
        UPDATE marks SET marks = %s WHERE student_id = %s AND subject_id = %s
    """, (new_marks, student_id, subject_id))
    conn.commit()
    print(f"\nMarks updated - Student {student_id}, Subject {subject_id} = {new_marks}")
```

```
# UPDATE
update_marks(1, 1, 95)   # Update Alice's Maths marks
```

Database Management Systems

Database Application

PES UNIVERSITY
CELEBRATING 50 YEARS

Output:

```
Marks updated → Student 1, Subject 1 = 95

Students:
(1, 'Alice Johnson', 'CSE')
(2, 'Bob Smith', 'ECE')
(3, 'Charlie Lee', 'ME')
(4, 'Diana Patel', 'CSE')
(5, 'Emma', 'CSE')
(6, 'John', 'ECE')

Subjects:
(1, 'Mathematics')
(2, 'Data Structures')
(3, 'Digital Electronics')
(4, 'Thermodynamics')
(5, 'Databases')
(6, 'Algorithms')

Marks:
Mark ID: 1, Student: Alice Johnson, Subject: Mathematics, Marks: 95
Mark ID: 2, Student: Alice Johnson, Subject: Data Structures, Marks: 90
Mark ID: 3, Student: Bob Smith, Subject: Mathematics, Marks: 78
Mark ID: 4, Student: Bob Smith, Subject: Digital Electronics, Marks: 88
Mark ID: 5, Student: Charlie Lee, Subject: Mathematics, Marks: 65
Mark ID: 6, Student: Charlie Lee, Subject: Thermodynamics, Marks: 70
Mark ID: 7, Student: Diana Patel, Subject: Data Structures, Marks: 95
Mark ID: 8, Student: Diana Patel, Subject: Digital Electronics, Marks: 89
Mark ID: 9, Student: Emma, Subject: Databases, Marks: 85
Mark ID: 10, Student: Emma, Subject: Algorithms, Marks: 90
Mark ID: 11, Student: John, Subject: Databases, Marks: 78
```

PES UNIVERSITY
CELEBRATING 50 YEARS

4. Delete data in db

The Delete Function to delete student information

First deletes related data in marks table for that student (to avoid foreign key conflicts).

Then deletes the student from students.

Both deletes are executed, then committed.

```
def delete_student(student_id):
    cursor.execute("DELETE FROM marks WHERE student_id = %s", (student_id,))
    cursor.execute("DELETE FROM students WHERE student_id = %s", (student_id,))
    conn.commit()
    print(f"\nStudent {student_id} and their marks deleted.")
```

```
delete_student(2)          # Delete Bob
```

Database Management Systems

Database Application

PES UNIVERSITY
CELEBRATING 50 YEARS

Output:

```
Student 2 and their marks deleted.

Students:
(1, 'Alice Johnson', 'CSE')
(3, 'Charlie Lee', 'ME')
(4, 'Diana Patel', 'CSE')
(5, 'Emma', 'CSE')
(6, 'John', 'ECE')

Subjects:
(1, 'Mathematics')
(2, 'Data Structures')
(3, 'Digital Electronics')
(4, 'Thermodynamics')
(5, 'Databases')
(6, 'Algorithms')

Marks:
Mark ID: 1, Student: Alice Johnson, Subject: Mathematics, Marks: 95
Mark ID: 2, Student: Alice Johnson, Subject: Data Structures, Marks: 90
Mark ID: 5, Student: Charlie Lee, Subject: Mathematics, Marks: 65
Mark ID: 6, Student: Charlie Lee, Subject: Thermodynamics, Marks: 70
Mark ID: 7, Student: Diana Patel, Subject: Data Structures, Marks: 95
Mark ID: 8, Student: Diana Patel, Subject: Digital Electronics, Marks: 89
Mark ID: 9, Student: Emma, Subject: Databases, Marks: 85
Mark ID: 10, Student: Emma, Subject: Algorithms, Marks: 90
Mark ID: 11, Student: John, Subject: Databases, Marks: 78
```

Database Management Systems

Q&A

What is the primary purpose of integrating Python with MySQL in database applications?

A. To create GUI-based games
B. To perform advanced image processing
C. To enable scalable and lightweight database operations
D. To convert SQL to NoSQL automatically

Which of the following commands is used to install the MySQL connector package in Python?

A. pip install python-mysql-connector
B. install mysql-connector
C. pip install mysql-connector-python
D. python install mysql

Database Management Systems

Q&A

In the sample university desktop application scenario, which table stores the relationship between students and their performance in subjects?

A. students

B. subjects

C. performance

D. marks

Q&A

What is the primary purpose of integrating Python with MySQL in database applications?

A. To create GUI-based games

B. To perform advanced image processing

C. To enable scalable and lightweight database operations

D. To convert SQL to NoSQL automatically

Which of the following commands is used to install the MySQL connector package in Python?

A. pip install python-mysql-connector

B. install mysql-connector

C. pip install mysql-connector-python

D. python install mysql

Q&A

In the sample university desktop application scenario, which table stores the relationship between students and their performance in subjects?

A. students

B. subjects

C. performance

D. marks

References

https://www.python.org

https://dev.mysql.com/downloads/mysql/

THANK YOU

Prof. Mahitha G

Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Database Transactions, ACID Properties

Prof. Mahitha G

Department of Computer Science and Engineering

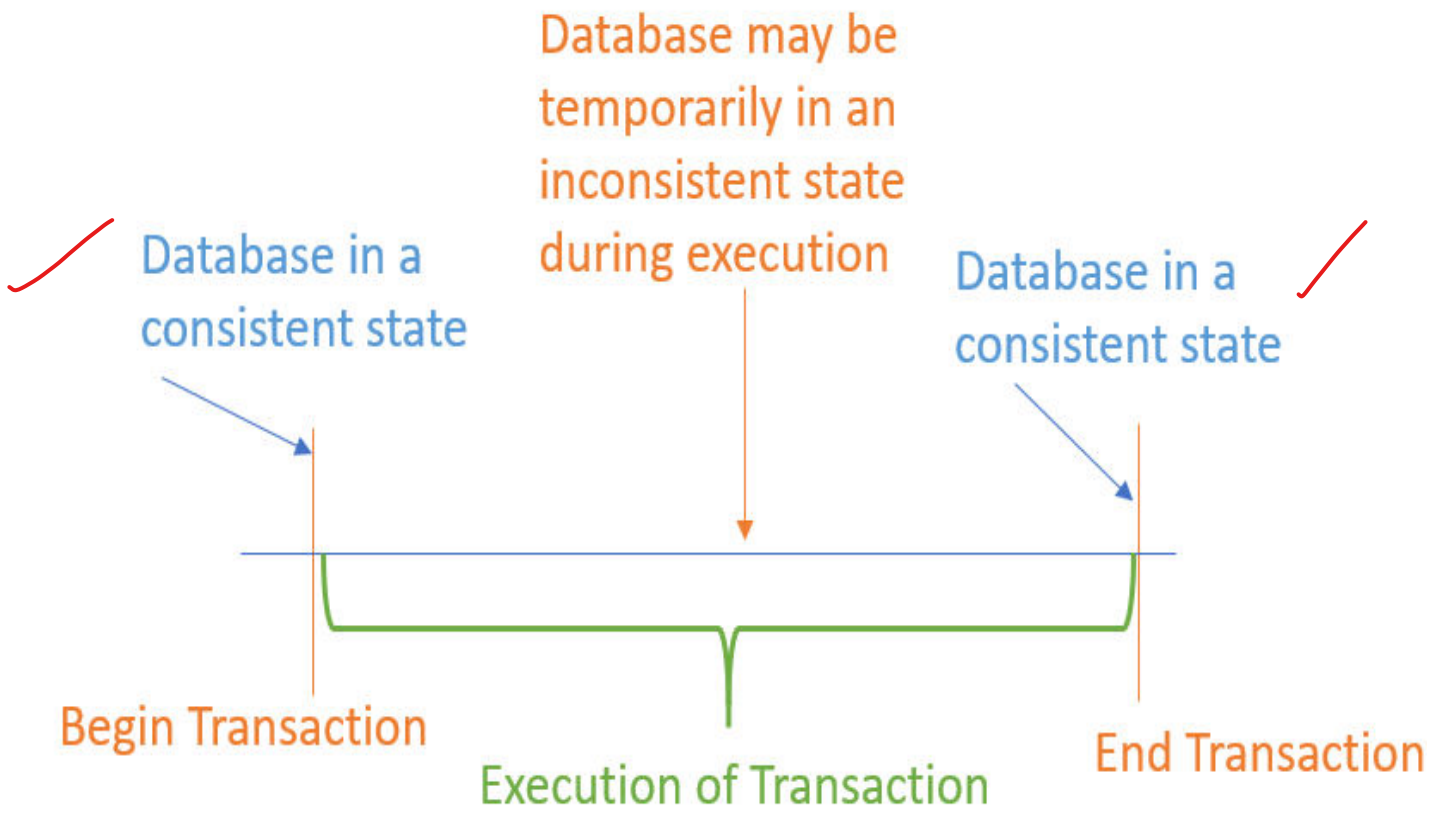
Acknowledgment: Dr. Nagasundari S (Dept of CSE), Teaching Assistants

Database Management Systems

BEFORE STARTING READ THIS
file:///E:/CollegeWork/PESU_SEM_5/DBMS/Unit4/pdf/Reference/Transactionsintro.pdf

Outline

- ❑Transaction Concept
- ❑Simple Transaction Model
- ❑ACID Properties



Database Management Systems

Single-User versus Multi User Systems

- One criterion for classifying a database system is according to the **number of users who can use the system concurrently.**
- A DBMS is a single-user system if, at most, one user at a time can use it.
- A DBMS is multi-user if many users can use the system and access the database **concurrently.**

AT A TIME

Introduction to Transactions

A transaction is a **unit of program execution in a database system** that:

- Accesses (**reads**) one or more data items.
- Possibly updates (**writes**/changes) one or more data items.

It is **the smallest unit of work** that is either fully completed or fully rolled back (**no partial completion is allowed**).

Transaction may be initiated by a user program written in a high-level data-manipulation language (typically **SQL**), or *programming language (e.g., C++ or Java), with embedded database accesses in JDBC or ODBC*.

- E.g., transaction to transfer $50 from account A to account B:

1. **read**(A)
2. $A := A - 50$
3. **write**(A)
4. **read**(B)
5. $B := B + 50$
6. **write**(B)

Introduction to Transactions

A collection of several operations on the database appears to be a **single unit** from the ***point of view of the database user***. However, the **database system consists of several operations.**

For Example: A transfer of funds from a checking account to a savings account is a single operation from the customer's standpoint; however, within the database system, it consists of several operations.

Before: X : 500	Y: 200
Transaction T	
T1	T2
Read (X) X: = X – 100 Write (X)	Read (Y) Y: = Y + 100 Write (Y)
After: X : 400	Y : 300

It's essential that these operations occur, or that, in case of ~~failure, none occur.~~

ATOMICITY

Collections of operations that form a single logical unit of work are called transactions.

Transaction Concept

Two main issues to deal with:

- **Failures of various kinds, such as hardware failures and system crashes**
- **Concurrent execution of multiple transactions**

A transaction is delimited by statements (or function calls) of the form begin transaction and end transaction*. The transaction consists of all operations executed between the begin transaction and the end transaction.*

Begin and end transaction statements

Specify transaction boundaries, all operations between a part of a single transaction.

- **Read-only transaction**: If the database operations in a transaction do not update the database.
- **Read-write transaction:** If the database operations in a transaction update the database as well as retrieve.

A Simple Transactional Model

- We shall illustrate the transaction concept using a **simple bank application** consisting of several accounts and a set of transactions that access and update those accounts. **Transactions access data using two operations:**
- **read(X):** which transfers the **data item X from the database to a variable**, also called X, in a **buffer in main memory** belonging to the transaction that executed the read operation.
- **write(X):** which transfers the **value in the variable X in the main-memory buffer** of the transaction that executed the write to the data item X in the database.
- It is important to know *if a change to a data item appears only in main memory or if it has been written to the database on disk.*
- In a real database system**, the write operation does not necessarily result in the immediate update of the data on the disk**; the write operation may be temporarily stored elsewhere and executed on the disk later. For now, however, we shall assume that the write operation updates the database immediately.

A Simple Transactional Model

- **Executing a read_item(X) command includes the following steps**
  1. Find the **address of the disk block** that contains item X.
  2. Copy that disk block into a **buffer in main memory** (if that disk block is not already in some main memory buffer). The size of the buffer is the same as the disk block size.
  3. Copy *item X from the buffer to the program variable named X*.
- **Executing a write_item(X) command includes the following steps**
  1. Find the **address of the disk block** that contains item X.
  2. Copy that **disk block into a buffer in main memory** (if that disk block is not already in some main memory buffer).
  3. Copy *item X from the program variable named X into its correct location in the buffer*.
  4. Store the **updated disk block from the buffer back to disk** (either immediately or at some later point in time).

Data Buffers:

- The DBMS maintains data buffers in main memory, each holding one database disk block.
- When all buffers are full, a **buffer replacement policy** decides which buffer to replace.
- LRU (**Least Recently Used**) is a common replacement policy used to choose the buffer to replace.
- If a **buffer is modified, it is written back to disk before being reused**.
- Some buffer replacement policies are specific to DBMS characteristics.

Transaction Concurrency

Transaction-processing systems usually allow multiple transactions to run concurrently. Allowing multiple transactions to update data concurrently causes several complications with consistency of the data, **Ensuring consistency despite concurrent execution of transactions requires extra work; it is far easier to insist that transactions run serially— that is, one at a time, each starting only after the previous one has completed**.

However, there are two good reasons for **allowing concurrency**:

- **Improved throughput and resource utilization**
- **Reduced waiting time**

TOTAL BEFORE TRANSACTION AND AFTER TRANSACTION MUST REMAIN SAME

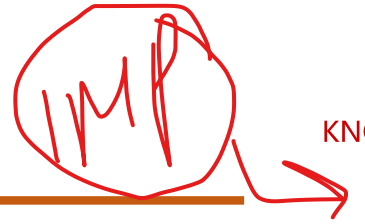
Consistency means:
A transaction must take the database from one valid state to another valid state — without breaking any rules, constraints, or integrity conditions.

In simple words:
👉 The database should never end up in an illegal, corrupted, or impossible state

Step	Operation	Account A	Account B	Total	Status
1	Initial balances	500	300	800	Consistent
2	UPDATE A = A - 200	300	300	600	Inconsistent (money lost)
3	UPDATE B = B + 200	300	500	800	Consistent (if executed)

Database Management Systems

A Simple Transaction Execution



KNOW THIS IN EXAM MIGHT ASK

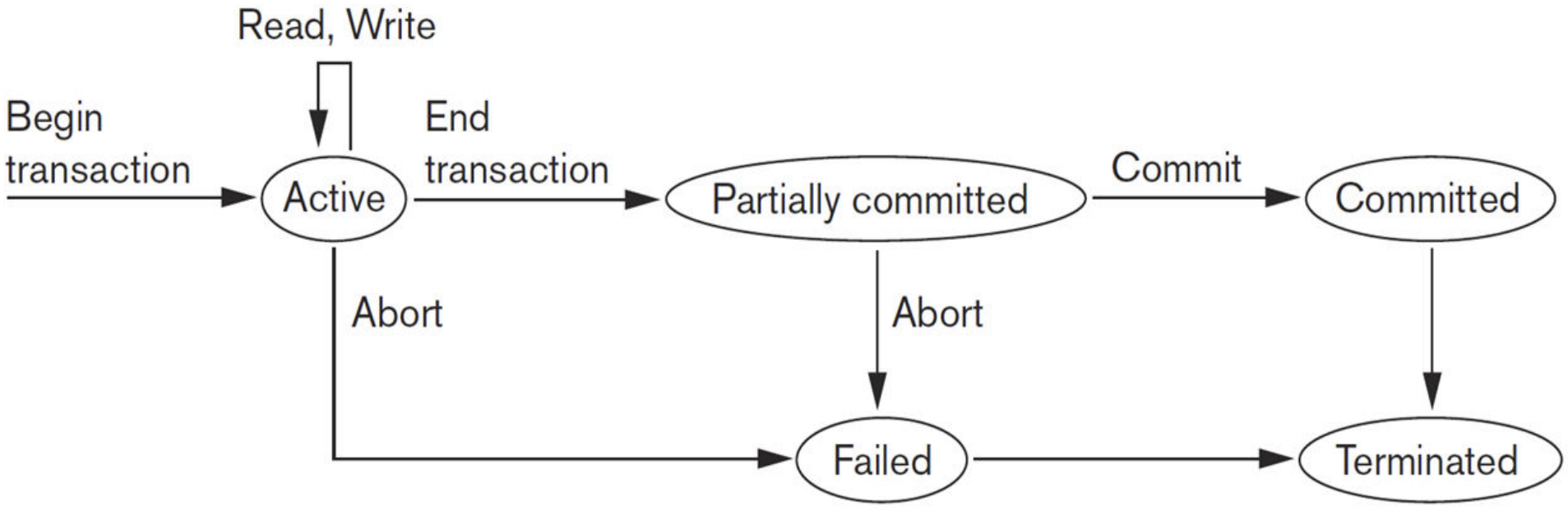


Figure 20.4
State transition diagram illustrating the states for transaction execution.

Transaction States

State transition diagram illustrates how a transaction moves through its execution states.

A transaction goes into an **active state** immediately after it starts execution, where it can execute its **READ and WRITE operations**.

When the transaction ends, it moves to the **partially committed state**. At this point, some types of concurrency control protocols may do additional checks to see if the transaction can be committed or not. Also, some *recovery protocols need to ensure that a system failure will not result in an inability to record the changes* of the transaction permanently.

If these **checks are successful**, the transaction is said to have **reached its commit point** and enters the **committed state.** When a transaction is committed, it has concluded its execution successfully and all its changes must be recorded permanently in the database, even if a system failure occurs.

Transaction States

A transaction can go to the **failed state** if one of the checks fails or if the transaction is aborted during its active state.

The **terminated state** corresponds to the transaction leaving the system.

Failed or aborted transactions may be restarted later—either **automatically** or after being **resubmitted by the user**—as brand new transactions.

Transaction States

When the **transaction is aborted** there are two options:

- It can **restart** the transaction, *but only if the transaction was aborted as a result of some hardware or software error* that was not created through the internal logic of the transaction. A restarted transaction is considered to be a new transaction.
- It can **kill** the transaction. It usually does so because of **some internal logical error that can be corrected only by rewriting the application program**, or because the input was bad, or because the desired data were not found in the database

Transaction cont…

■ BEGIN_TRANSACTION. This marks the beginning of transaction execution.

■ READ or WRITE. These specify read or write operations on the database items that are executed as part of a transaction.

■ END_TRANSACTION. This specifies that READ and WRITE transaction operations have ended and marks the end of transaction execution.

■ COMMIT_TRANSACTION. This signals a successful end of the transaction so that any changes (updates) executed by the transaction can be safely committed to the database and will not be undone.

■ ROLLBACK (or ABORT). This signals that the transaction has ended unsuccessfully, so that any changes or effects that the transaction may have applied to the database must be

Transaction States Examples

```
-- Start the transaction
START TRANSACTION;

-- Read current salary
SELECT Salary FROM Employee WHERE ssn = 111111100 ;

-- Update salary
UPDATE Employee SET Salary = Salary * 1.10 WHERE ssn = 111111100 ;

-- If all is good, commit changes
COMMIT;

-- If there was an error, rollback
-- ROLLBACK;
```

👉 What this slide is teaching

A transaction groups multiple SQL operations.
If all succeed → COMMIT saves the changes.
If any fails → ROLLBACK undoes everything.
Transactions ensure safety and correctness.

Transaction Support in SQL

There is a lot of support from SQL for the development of transaction-based applications

- No explicit Begin Transaction statement is needed by SQL.
- Every transaction must have an explicit end statement
  - COMMIT
  - ROLLBACK
- Access mode is READ ONLY or READ WRITE

Access Mode

Access mode in SQL transactions **defines the type of operations that a transaction can perform.**

There are two primary access modes:

- **READ WRITE:**
  - Default mode (unless using READ UNCOMMITTED).
  - Allows all operations: SELECT, UPDATE, INSERT, DELETE, and CREATE commands.
- **READ ONLY:**
  - Assumed when using the READ UNCOMMITTED isolation level.
  - Restricts transactions to data retrieval only (i.e., SELECT statements).

Transaction Access Mode Examples → READ UNCOMMITTED

Transaction 1 (T1): Updates salary but does **not commit** yet

```
START TRANSACTION;

UPDATE Employee
SET Salary = Salary + 5000
WHERE ssn = 123456789;

-- T1 has not committed yet
```

◇ Transaction 1 (T1)
Updates salary but does not commit.

✔ Change exists in memory
❌ But not committed yet
❌ Not visible under normal isolation levels

Transaction 2 (T2) with READ UNCOMMITTED: Reads the same data

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;

SELECT Salary
FROM Employee
WHERE ssn = 123456789;
```

Because T2 is using READ UNCOMMITTED:
👉 T2 can see T1's uncommitted changes
This is called a dirty read.

Important Points

1.Dirty reads are possible. → Because T2 sees uncommitted data

2.Can improve **performance** in high-concurrency systems. → Because no locks / blocking

3.Not safe for financial or critical transactions.
→ You could read wrong data
→ If T1 rolls back, T2 read something that never really existed

☑ Dirty Read (Definition)
A dirty read happens when:
👉 Transaction T2 reads data that Transaction T1 has modified but NOT yet committed.

Since T1 has not committed:

The data might be rolled back
Meaning T2 read something that could disappear
So the data T2 saw was never valid

That's why it's called "dirty" — the data is unclean, temporary, and unreliable.

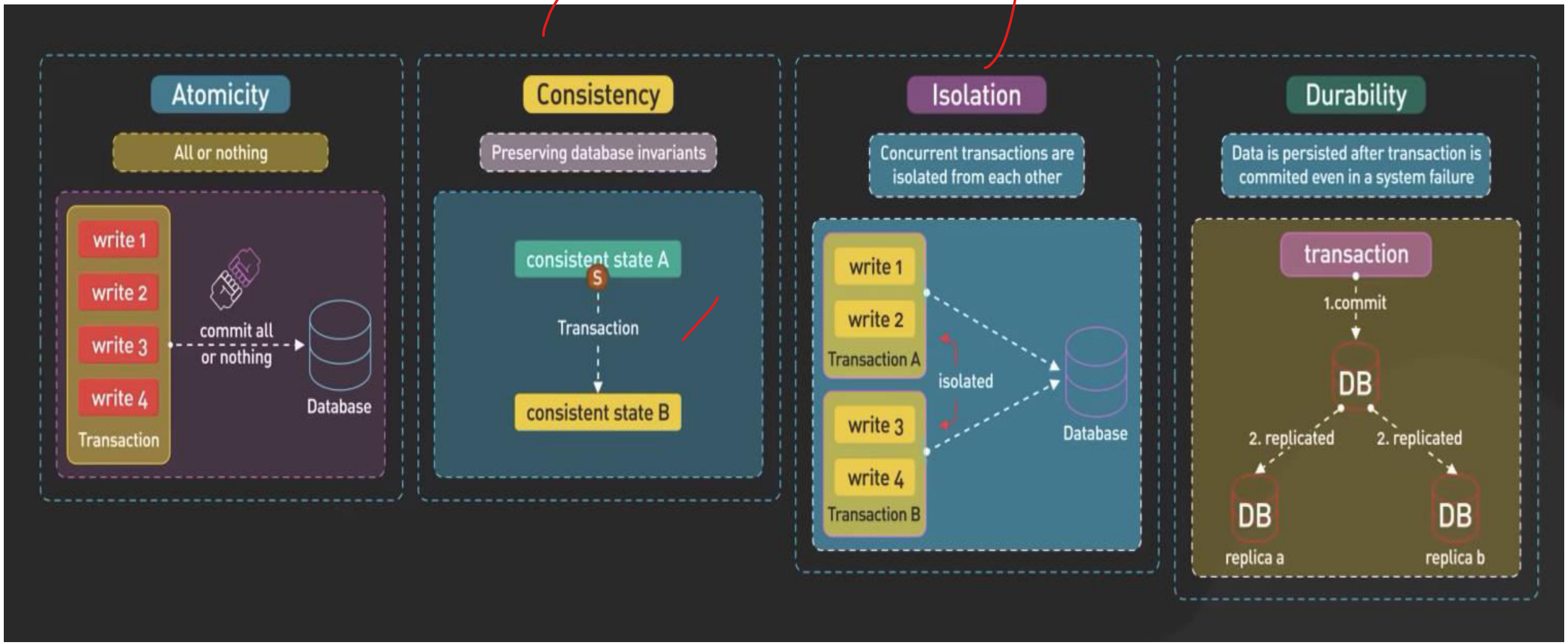
ACID Properties

To preserve the integrity of data the database system must ensure:

- **Atomicity.** Either all operations of the transaction are properly reflected in the database or none are. This "all-or-none" property is referred to as atomicity.
- **Consistency.** Execution of a transaction in isolation preserves the consistency of the database.
- **Isolation.** Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions. **Intermediate transaction results must be hidden from other concurrently executed transactions.**
- **Durability.** After a transaction completes successfully, the changes it has made to the database must persist, even if there are system failures.

Database Management Systems

ACID Properties



Example of Fund Transfer

Transaction to transfer $50 from account A to account B: Transaction (Ti)

1. **read**(*A*)
2. *A* := *A* – 50
3. **write**(*A*)
4. **read**(*B*)
5. *B* := *B* + 50
6. **write**(*B*)

Example of Fund Transfer

Atomicity requirement

- If the transaction fails after step 3 and before step 6, **money will be "lost" leading to an inconsistent database state**
- Failure could be due to software or hardware
- The system should ensure that *updates of a partially executed transaction are not reflected in the database.*
- If the atomicity property is present, **all actions of the transaction are reflected in the database, or none are**
- The basic idea behind ensuring atomicity is this: The database system keeps track (on disk) of the old values of any data on which a transaction performs a write. This information is written to a file called the log.
- **If the transaction does not complete its execution, the database system restores the old values from the log to make it appear as though the transaction never executed. Ensuring atomicity is the responsibility of the database system; specifically, it is handled by a component of the database called the recovery system.**

Example of Fund Transfer

- **Durability:** Once the user has been notified that the transaction has completed (i.e., the transfer of the $50 has taken place), **the updates to the database by the transaction must persist even if there are software or hardware failures.** The recovery system of the database, is responsible for ensuring durability, in addition to ensuring atomicity.

- **Consistency**:
  - The sum of A and B is unchanged by the execution of the transaction

  In general, consistency requirements include
  - Explicitly specified **integrity constraints** such as primary keys and foreign keys
  - Implicit integrity constraints
  - A transaction must see a consistent database.
  - *During transaction execution the database may be temporarily inconsistent.*
  - *When the transaction completes successfully the database must be consistent*

Consistency means:
A transaction must take the database from one valid state to another valid state — without breaking any rule constraints, or integrity conditions.

In simple words:
☞ The database should never end up an illegal, corrupted, or impossible stat

Example of Fund Transfer

- **Isolation** — if between steps 3 and 6, another transaction T2 is allowed to access the partially updated database, **it will see an inconsistent database (the sum *A + B* will be less than it should be).**

T1	T2
1. **read**(*A*)	
2. *A* := *A* – 50	
3. **write**(*A*)	
	read(A), read(B), print(A+B)
4. **read**(*B*)	
5. *B* := *B* + 50	
6. **write**(*B*)	

serially helps because no other transaction will be allowed to happen when one is happening and so it there won't be any chance only of incosistency creeping in as only on transaction is runing

Isolation can be ensured trivially by running transactions serially. That is, one after the other. However, executing multiple transactions concurrently has significant benefits.

Example of Fund Transfer

- Concurrent execution of transactions provides significant performance benefits.
- **The isolation property of a transaction ensures that the concurrent execution of transactions results in a system state that is equivalent to a state that could have been obtained had these transactions executed one at a time in some order**.
- Ensuring the isolation property is the responsibility of a component of the database system called the **concurrency-control system**.

Database Management Systems

Q&A

1. Which of the following ACID properties ensures that once a transaction completes, the changes made will persist even if the system crashes afterward?

(a) Atomicity.
(b) Consistency.
(c) Isolation.
(d) Durability.

2. In a transaction that transfers $50 from account A to account B, what happens if the system fails after updating account A but before updating account B?

(a) The system remains consistent.
(b) The transaction is completed successfully.
(c) The system enters an inconsistent state.
(d) The transaction automatically retries.

28

Q&A

3. Which ACID property ensures that a transaction, when executed in isolation, appears as if no other transactions are executing simultaneously?

(a) Atomicity
(b) Consistency
(c) Isolation
(d) Durability

4. Which of the following best describes a transaction in a database system?

(a) A transaction is a set of operations that only reads data from the database.
(b) A transaction is a unit of program execution that consists of a single operation.
(c) A transaction is a collection of operations that either all occur or none occur, ensuring data integrity.
(d) A transaction always updates the database immediately after an operation is executed.

Q&A

1. Which of the following ACID properties ensures that once a transaction completes, the changes made will persist even if the system crashes afterward?

(a) Atomicity.
(b) Consistency.
(c) Isolation.
(d) Durability.

2. In a transaction that transfers $50 from account A to account B, what happens if the system fails after updating account A but before updating account B?

(a) The system remains consistent.
(b) The transaction is completed successfully.
(c) The system enters an inconsistent state.
(d) The transaction automatically retries.

Q&A

3. Which ACID property ensures that a transaction, when executed in isolation, appears as if no other transactions are executing simultaneously?

(a) Atomicity
(b) Consistency
(c) Isolation
(d) Durability

4. Which of the following best describes a transaction in a database system?

(a) A transaction is a set of operations that only reads data from the database.
(b) A transaction is a unit of program execution that consists of a single operation.
(c) A transaction is a collection of operations that either all occur or none occur, ensuring data integrity.
(d) A transaction always updates the database immediately after an operation is executed.

THANK YOU

Prof. Mahitha G

Department of Computer Science and Engineering

Database Management Systems
Concurrency control – Lock-based Protocols

Prof. Shilpa S

Department of Computer Science and Engineering

Database Management Systems

Concurrency control, Lock-based protocols

PES UNIVERSITY
CELEBRATING 50 YEARS

- ❑ What is Concurrency Control
- ❑ Need for Concurrency Control
- ❑ Isolation Levels
- ❑ Q&A

What is Concurrency Control

Serializability means concurrent transactions behave as if they were executed one after the other (serially), ensuring correct and consistent results.

- Concurrency Control is **a procedure of managing simultaneous transactions**, ensuring their atomicity, isolation, consistency, and serializability.
- Concurrency control restricts **concurrent accesses** in a way that ensures there is no **erroneous update to the database.**
- To ensure the preservation of the isolation property, the system must control the interaction among the concurrent transactions; this control is achieved through one of a variety of mechanisms called **concurrency-control schemes**.
- The most frequently used schemes are **two-phase locking and snapshot isolation**.

PES UNIVERSITY

Why Concurrency Control Is Needed?

- **Prevention of Data Conflicts:** When multiple transactions execute concurrently without control, they can access and modify the same data items, leading to conflicts such as incorrect or inconsistent data updates.

- **Ensuring Data Integrity:** Concurrency control is essential to maintain the accuracy and integrity of the database, preventing issues like race conditions, where the outcome of transactions depends on the timing of their execution.

A Simple Transactional Model

- **Read the set of a transaction**
  - Set of all items read
- **Write a set of transactions**
  - A set of all items written

(a)

T_1

read_item(X);
$X := X - N$;
write_item(X);
read_item(Y);
$Y := Y + N$;
write_item(Y);

(b)

T_2

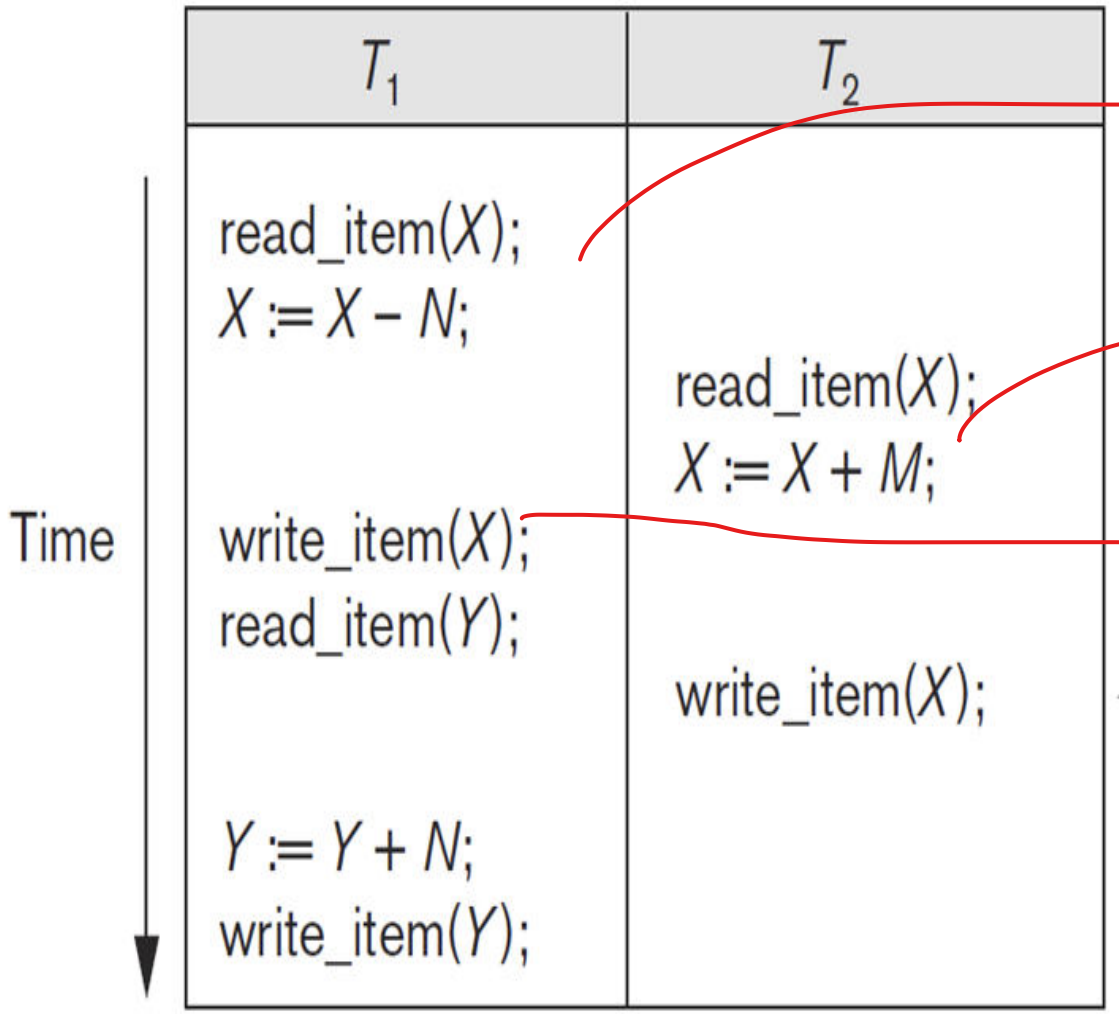
read_item(X);
$X := X + M$;
write_item(X);

Two sample transactions.
(a) Transaction T_1.
(b) Transaction T_2.

Figure 20.3 Concurrency Control Problems

Some problems that occur when concurrent execution is uncontrolled. (a) The lost update problem. (b) The temporary update problem. (c) The incorrect summary problem.

(a)



initially X=5,N=2
so X=X-N=5-2=3

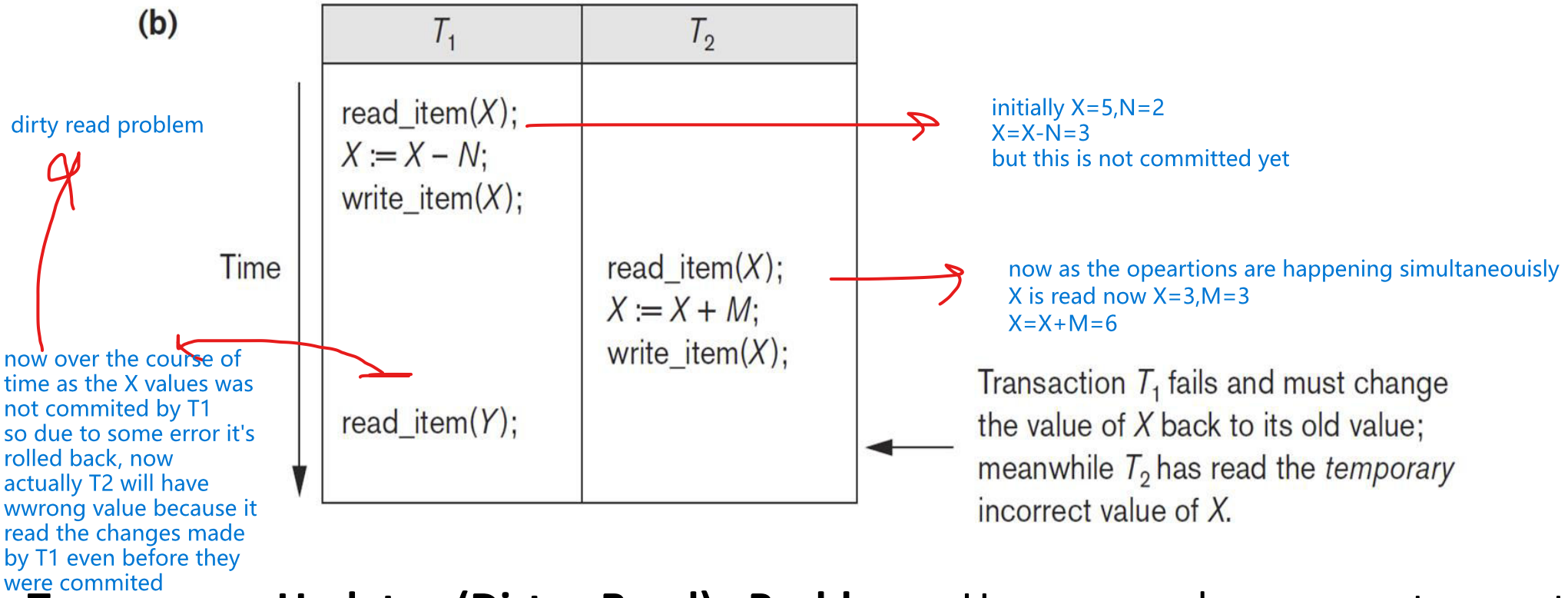
as it is not commited or written back
original X is read here and X=5,M=3
X=X+M=5+3=8

Now X recent value is 8 which is written but original update which resulted in earlier X=3 is lost or overwritten

Item *X* has an incorrect value because its update by T_1 is *lost* (overwritten).

Database Management Systems

Concurrency Control Problems



dirty read problem

initially X=5,N=2
X=X-N=3
but this is not committed yet

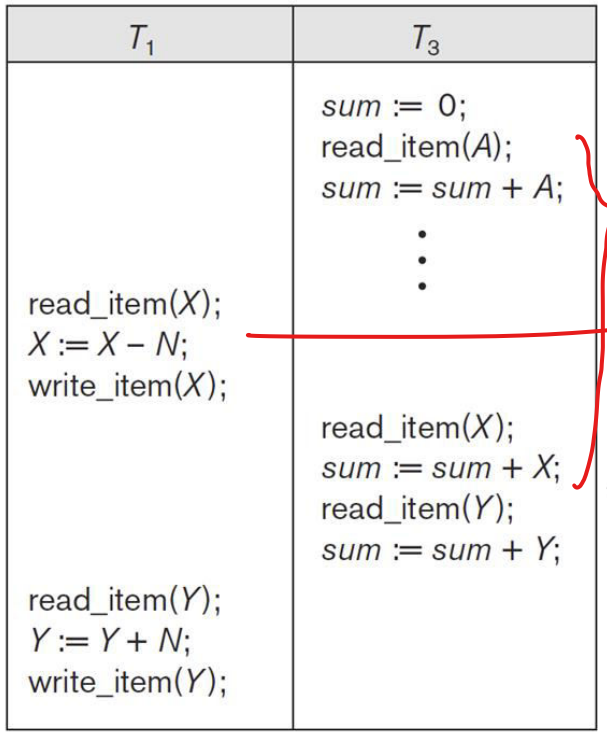
now as the opeartions are happening simultaneouisly
X is read now X=3,M=3
X=X+M=6

now over the course of time as the X values was not commited by T1 so due to some error it's rolled back, now actually T2 will have wwrong value because it read the changes made by T1 even before they were commited

Temporary Update (Dirty Read) Problem: Happens when one transaction reads an uncommitted update made by another transaction, which later fails, causing the read to be based on invalid data.

Database Management Systems
Concurrency Control Problems

(c)



T_1	T_3

	sum := 0;
	read_item(*A*);
	sum := *sum* + *A*;
	. . .

read_item(*X*);
X := *X* − *N*;
write_item(*X*);	

	read_item(*X*);
	sum := *sum* + *X*;
	read_item(*Y*);
	sum := *sum* + *Y*;

read_item(*Y*);
Y := *Y* + *N*;
write_item(*Y*);	

T2 is running an aggregate function i.e doing Sum=Sum+A,B,C + + + ...X,Y,Z

but X values is modified before the aggregate function even reached X
so the new values of sum will be off by N because before it could reach X , N was subtracted from X by T1 leading to wrong final summary by T2

T_3 reads *X* after *N* is subtracted and reads *Y* before *N* is added; a wrong summary is the result (off by *N*).

Incorrect Summary Problem: Arises when a transaction calculates an aggregate function while other transactions are updating related data, leading to an inaccurate result due to partial updates.

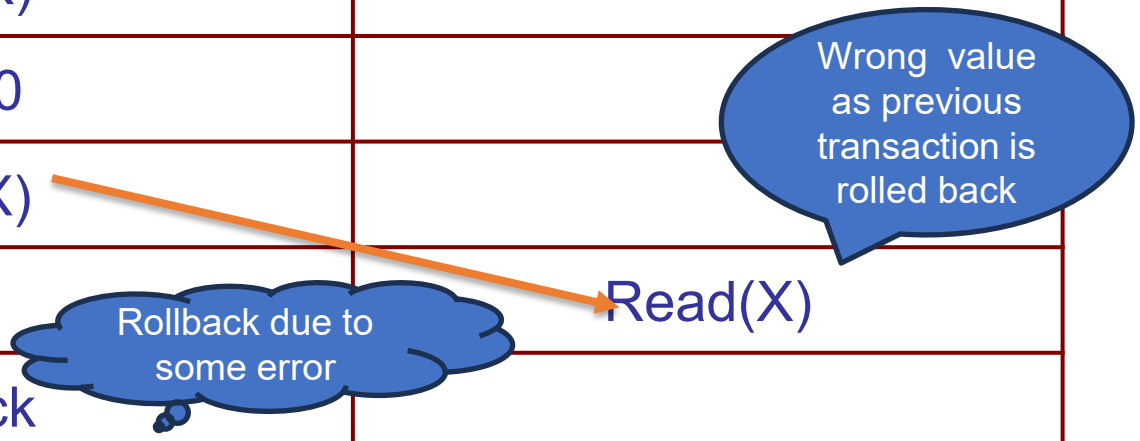
The Unrepeatable Read Problem:

A transaction T reads the same item twice and the item is changed by another transaction T’ between the two reads. Hence, T receives different values for its two reads of the same item.

Need for Concurrency Control

Example :

Time/Transactions	T1	T2
1	Read(X)	
2	X=X-10	
3	Write(X)	
4		Read(X)
5	Rollback	



Inconsistent value of X in Transactions T1 and T2.

Database Management Systems

Isolation Levels

basically allows dirty reads, reading when data is not commited by another transaction also

solves dirtry read but still has nor repeatable or unrepeatalbe read problem

- Isolation levels control how transactions in a database are isolated from one another, ensuring data consistency and preventing conflicts between concurrent transactions.
- The isolation level option is specified using the statement

  ISOLATION LEVEL <isolation>, where the value for **<isolation>** can be:

  - READ UNCOMMITTED: Lowest isolation, allows reading uncommitted data
  - READ COMMITTED: Only reads data that has been committed.
  - REPEATABLE READ: Ensures that data read once will not change during the transaction.
  - SERIALIZABLE: Highest isolation, guarantees complete isolation from other transactions.

solves unrepeatable read problem but still has phantrom read problem

best no problem

Isolation Levels

- The default isolation level is SERIALIZABLE, although some systems use READ COMMITTED as their default.

- If a transaction executes at a lower isolation level than SERIALIZABLE, then one or more of the following three violations may occur:

  - **Dirty read:** A transaction T1 may read the update of a transaction T2, which has not yet committed. If T2 fails and is aborted, then T1 would have read a value that does not exist and is incorrect.

  - **Non-repeatable read:** A transaction T1 may read a given value from a table. If another transaction T2 later updates that value and T1 reads that value again, T1 will see a different value.

Isolation Levels

Phantoms:

- A transaction T1 may read a set of rows from a table, perhaps based on some condition specified in the SQL WHERE-clause.
- Suppose that a transaction T2 inserts a new row r that also satisfies the WHERE-clause condition used in T1, into the table used by T1.
- The record r is called a **phantom record** because it was not there when T1 starts but is there when T1 ends. T1 may or may not see the phantom, a row that previously did not exist.
- If the equivalent serial order is T1 followed by T2, then the record r should not be seen; but if it is T2 followed by T1,then the phantom record should be in the result given to T1. If the system cannot ensure the correct behavior, then it does not deal with the phantom record problem.

Isolation Levels

Table 20.1 Possible Violations Based on Isolation Levels as Defined in SQL

	Type of Violation		
Isolation Level	**Dirty Read**	**Nonrepeatable Read**	**Phantom**
READ UNCOMMITTED	Yes	Yes	Yes
READ COMMITTED	No	Yes	Yes
REPEATABLE READ	No	No	Yes
SERIALIZABLE	No	No	No

This table summarizes the possible violations for the different isolation levels. An entry of Yes indicates that a violation is possible and an entry of No indicates that it is not possible. READ UNCOMMITTED is the most forgiving, and SERIALIZABLE is the most restrictive in that it avoids all three of the problems mentioned above.

Isolation Levels

A sample SQL transaction might look like the following:

```
EXEC SQL WHENEVER SQLERROR GOTO UNDO;
EXEC SQL SET TRANSACTION
    READ WRITE
    DIAGNOSTIC SIZE 5
    ISOLATION LEVEL SERIALIZABLE;
EXEC SQL INSERT INTO EMPLOYEE (Fname, Lname, Ssn, Dno, Salary)
    VALUES ('Robert', 'Smith', '991004321', 2, 35000);
EXEC SQL UPDATE EMPLOYEE
    SET Salary = Salary * 1.1 WHERE Dno = 2;
EXEC SQL COMMIT;
GOTO THE_END;
UNDO: EXEC SQL ROLLBACK;
THE_END: ... ;
```

- This SQL code sets the SERIALIZABLE isolation level, ensuring the highest level of isolation for the transaction.
- It performs an INSERT operation for a new employee and an UPDATE on employee salaries, with a rollback mechanism in case of failure.
- The SERIALIZABLE level ensures no dirty reads, non-repeatable reads, or phantom records, maintaining full data consistency.

```
PES0UG00CS000>SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
Query OK, 0 rows affected (0.00 sec)

PES0UG00CS000>
PES0UG00CS000>-- Start the transaction
PES0UG00CS000>START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

PES0UG00CS000>
PES0UG00CS000>-- Insert a new employee 'Aubrey Graham'
PES0UG00CS000>INSERT INTO EMPLOYEE (Fname, Lname, Ssn, Dno, Salary)
    -> VALUES ('Aubrey', 'Graham', '991004321', 2, 35000);
Query OK, 1 row affected (0.00 sec)

PES0UG00CS000>
PES0UG00CS000>-- Update salary for employees in department 2
PES0UG00CS000>UPDATE EMPLOYEE
    -> SET Salary = Salary * 1.1
    -> WHERE Dno = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

PES0UG00CS000>
PES0UG00CS000>-- Commit the transaction (no errors)
PES0UG00CS000>COMMIT;
Query OK, 0 rows affected (0.00 sec)

PES0UG00CS000>
PES0UG00CS000>-- Confirm the transaction was committed successfully
PES0UG00CS000>SELECT 'Transaction committed successfully' AS Status;
+-----------------------------------+
| Status                            |
+-----------------------------------+
| Transaction committed successfully |
+-----------------------------------+
1 row in set (0.00 sec)

PES0UG00CS000>
```

Database Management Systems

Isolation Levels - Sample MYSQL code

```
PES0UG00CS000>SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
Query OK, 0 rows affected (0.00 sec)

PES0UG00CS000>-- Start the transaction
PES0UG00CS000>START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

PES0UG00CS000>
PES0UG00CS000>-- Insert a new employee 'Duckworth Lamar'
PES0UG00CS000>INSERT INTO EMPLOYEE (Fname, Lname, Ssn, Dno, Salary)
    -> VALUES ('Duckworth', 'Lamar', '99000000', 2, 35000);
Query OK, 1 row affected (0.00 sec)

PES0UG00CS000>-- Introduce an error (Invalid Dno)
PES0UG00CS000>-- This will cause the update to fail, triggering a rollback
PES0UG00CS000>UPDATE EMPLOYEE
    -> SET Salary = Salary * 1.1
    -> WHERE Dno = 9999;  -- Invalid department number (causes error)
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0

PES0UG00CS000>-- Rollback the transaction due to the error
PES0UG00CS000>ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

PES0UG00CS000>-- Confirm the transaction was rolled back (display purposes only)
PES0UG00CS000>SELECT 'Transaction rolled back due to an error' AS Status;
+----------------------------------------+
| Status                                 |
+----------------------------------------+
| Transaction rolled back due to an error |
+----------------------------------------+
1 row in set (0.00 sec)

PES0UG00CS000>-- Verify Isolation Level
PES0UG00CS000>SELECT @@transaction_isolation;
+-------------------------+
| @@transaction_isolation |
+-------------------------+
| SERIALIZABLE            |
+-------------------------+
1 row in set (0.00 sec)

PES0UG00CS000>
```

Database Management Systems

Q&A

1. What is the purpose of concurrency control in databases?

(a) To allow multiple transactions to modify the same data simultaneously.
(b) To ensure that transactions are serialized.
(c) To ensure atomicity, isolation, and serializability of transactions.
(d) To prevent the database from being accessed by multiple users.

2. Which of the following isolation levels in a database management system allows a transaction to read only committed data, thus preventing dirty reads but allowing non-repeatable reads and phantom reads?

(a) Read Uncommitted
(b) Read Committed
(c) Repeatable Read
(d) Serializable

Q&A

1. What is the purpose of concurrency control in databases?

(a) To allow multiple transactions to modify the same data simultaneously.
(b) To ensure that transactions are serialized.
(c) To ensure atomicity, isolation, and serializability of transactions.
(d) To prevent the database from being accessed by multiple users.

2. Which of the following isolation levels in a database management system allows a transaction to read only committed data, thus preventing dirty reads but allowing non-repeatable reads and phantom reads?

(a) Read Uncommitted
(b) Read Committed
(c) Repeatable Read
(d) Serializable

THANK YOU

Prof. Shilpa S
Department of Computer Science and Engineering

shilpas@pes.edu

Database Management Systems

Transaction Serializability

Prof. Shilpa S
Department of Computer Science & Engineering

shilpas@pes.edu

BEFORE STARTING THIS READ THIS
file:///E:/CollegeWork/PESU_SEM_5/DBMS/Unit4/pdf/Reference/scheduleunderstanding.pdf

Schedules

- **Schedule** – *a sequences of instructions* **that specify the chronological order in which instructions of concurrent transactions are executed**
  - *A schedule for a set of transactions must consist of all instructions of those transactions*
  - *Must preserve the order in which the instructions appear in each individual transaction.*
- A transaction that **successfully completes its execution** will have a **commit** instructions as the last statement
  - By default transaction is assumed to execute commit instruction as its last step
- A transaction that **fails to successfully complete its execution** will have an **abort** instruction as the last statement

Database Management Systems

Schedules

- A schedule (or history) S of n transactions T1, T2, … , Tn is an ordering of the operations of the transactions.
- Operations from different transactions can be interleaved in the schedule S.
- For each transaction Ti that participates in the schedule S, the operations of Ti in S must appear in the same order in which they occur in Ti.
- The order of operations in S is considered to be a total ordering, meaning that for any two operations in the schedule, one must occur before the other.

Schedules

- **Read-Write Conflict:** Occurs when a read and a write operation on the same data item from different transactions, where changing the order alters the value read.

- **Write-Write Conflict:** Happens when two write operations on the same data item from different transactions, with the final value depending on the order of execution.

Schedules

Conflicting Operations in a Schedule.

- **Two operations in a schedule are said to conflict if they satisfy all three of the following conditions:**

  (1)they belong to different transactions;

  (2)they access the same item X; and

  (3)at least one of the operations is a write_item(X).

- For example, in schedule

  Sa: r1(X); r2(X); w1(X); r1(Y); w2(X); w1(Y)

  Sb: r1(X); w1(X); r2(X); w2(X); r1(Y);

- the operations r1(X) and w2(X) conflict, as do the operations r2(X) and w1(X), and the operations w1(X) and w2(X). However, the operations r1(X) and r2(X) do not conflict, since they are both read, the operations w2(X) and w1(Y) do not conflict because they operate on distinct data items X and Y; and the operations r1(X) and w1(X) do not conflict because they belong to the same transaction.

Schedule 1—a serial schedule in which T1 is followed by T2

T_1	T_2
read(A)	
$A := A - 50$	
write(A)	
read(B)	
$B := B + 50$	
write(B)	
commit	
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
	read(B)
	$B := B + temp$
	write(B)
	commit

Figure 17.2 Schedule 1—a serial schedule in which T_1 is followed by T_2.

Consider again the simplified banking system, which has several accounts and a set of transactions that access and update those accounts.

Let T1 and T2 be two transactions that transfer funds from one account to another. Transaction T1 transfers $50 from account A to account B.

Transaction T2 transfers 10 percent of the balance from account A to account B.

Database Management Systems

Schedule 2

T_1	T_2
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
	read(B)
	$B := B + temp$
	write(B)
	commit
read(A)	
$A := A - 50$	
write(A)	
read(B)	
$B := B + 50$	
write(B)	
commit	

T1 → T2

Figure 17.3 Schedule 2—a serial schedule in which T_2 is followed by T_1.

Suppose the current values of accounts A and B are $1000 and $2000, respectively.

Suppose also that the two transactions are executed one at a time in the order T1 followed by T2. This execution sequence appears in the Figure.

In the figure, the sequence of instruction steps is in chronological order from top to bottom, with instructions of T1 appearing in the left column and instructions of T2 appearing in the right column.

The final values of accounts A and B, after the execution in Figure 17.2 takes place, are $855 and $2145, respectively. Thus, the total amount of money in accounts A and B— that is, the sum A + B—is preserved after the execution of both transactions.

CONSISTENCY

Database Management Systems

Schedule 3

- Let T_1 and T_2 be the transactions defined previously.

The following schedule is not a serial schedule, but it is *equivalent* to Schedule 1

T_1	T_2
read(A)	
$A := A - 50$	
write(A)	
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
read(B)	
$B := B + 50$	
write(B)	
commit	
	read(B)
	$B := B + temp$
	write(B)
	commit

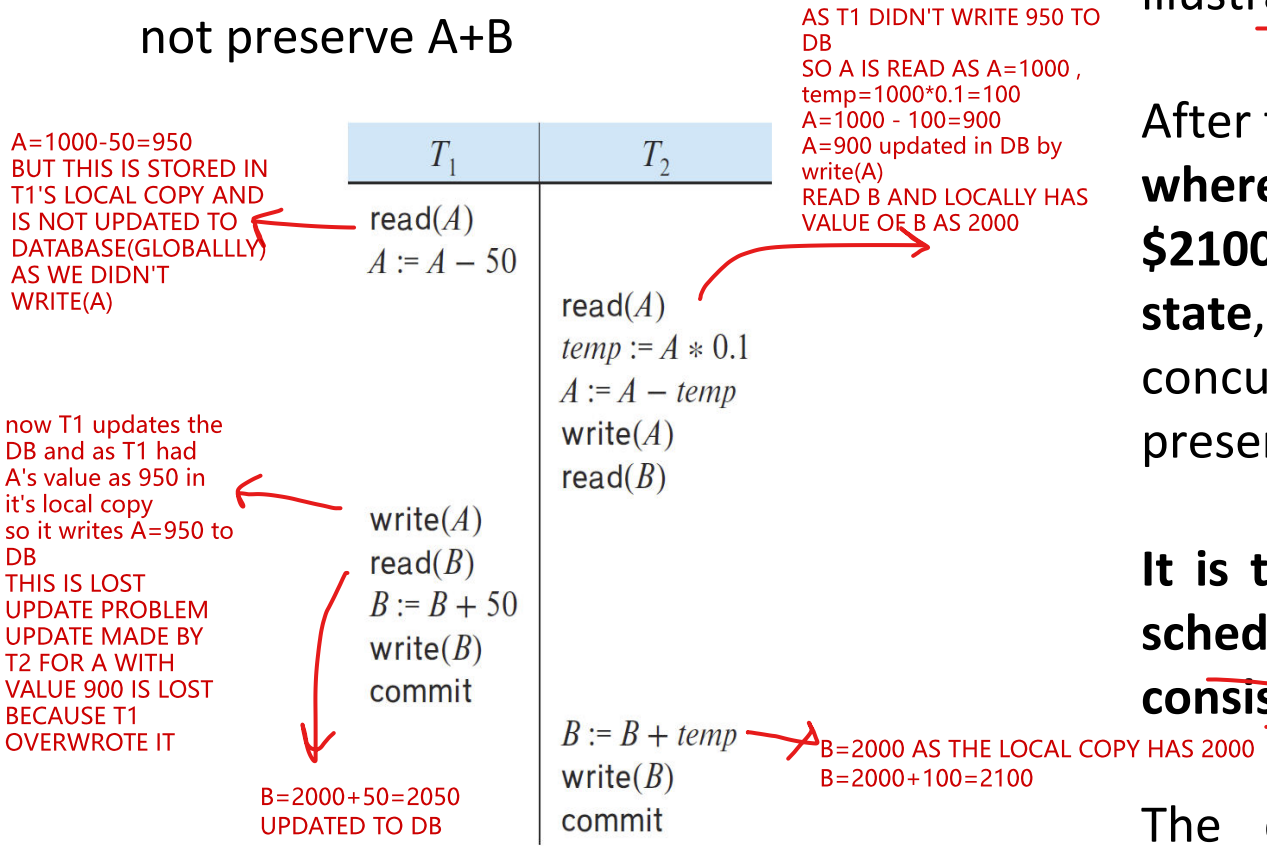
Suppose that the two transactions are executed concurrently. One possible schedule appears in the Figure.

After this execution takes place, we arrive at the same state as the one in which the transactions are executed serially in the order T1 followed by T2. The sum A + B is indeed preserved.

Figure 17.4 Schedule 3—a concurrent schedule equivalent to schedule 1.

Database Management Systems
Schedule 4

- In Schedules 1, 2 and 3, the sum A + B is preserved, whereas this schedule does not preserve A+B



T_1	T_2
read(A)	
$A := A - 50$	
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
	read(B)
write(A)	
read(B)	
$B := B + 50$	
write(B)	
commit	
	$B := B + temp$
	write(B)
	commit

Figure 17.5 Schedule 4—a concurrent schedule resulting in an inconsistent state.

Not all concurrent executions result in a correct state. To illustrate, consider the schedule of Figure.

After the execution of this schedule, **we arrive at a state where the final values of accounts A and B are $950 and $2100, respectively.** This final state is an **inconsistent state**, since we have gained $50 in the process of the concurrent execution. Indeed, the sum A + B is not preserved by the execution of the two transactions.

It is the job of the database system to ensure that any schedule that is executed will leave the database in a consistent state.

The concurrency-control component of the database system carries out this task.

Database Management Systems

Serial and Non serial Schedules

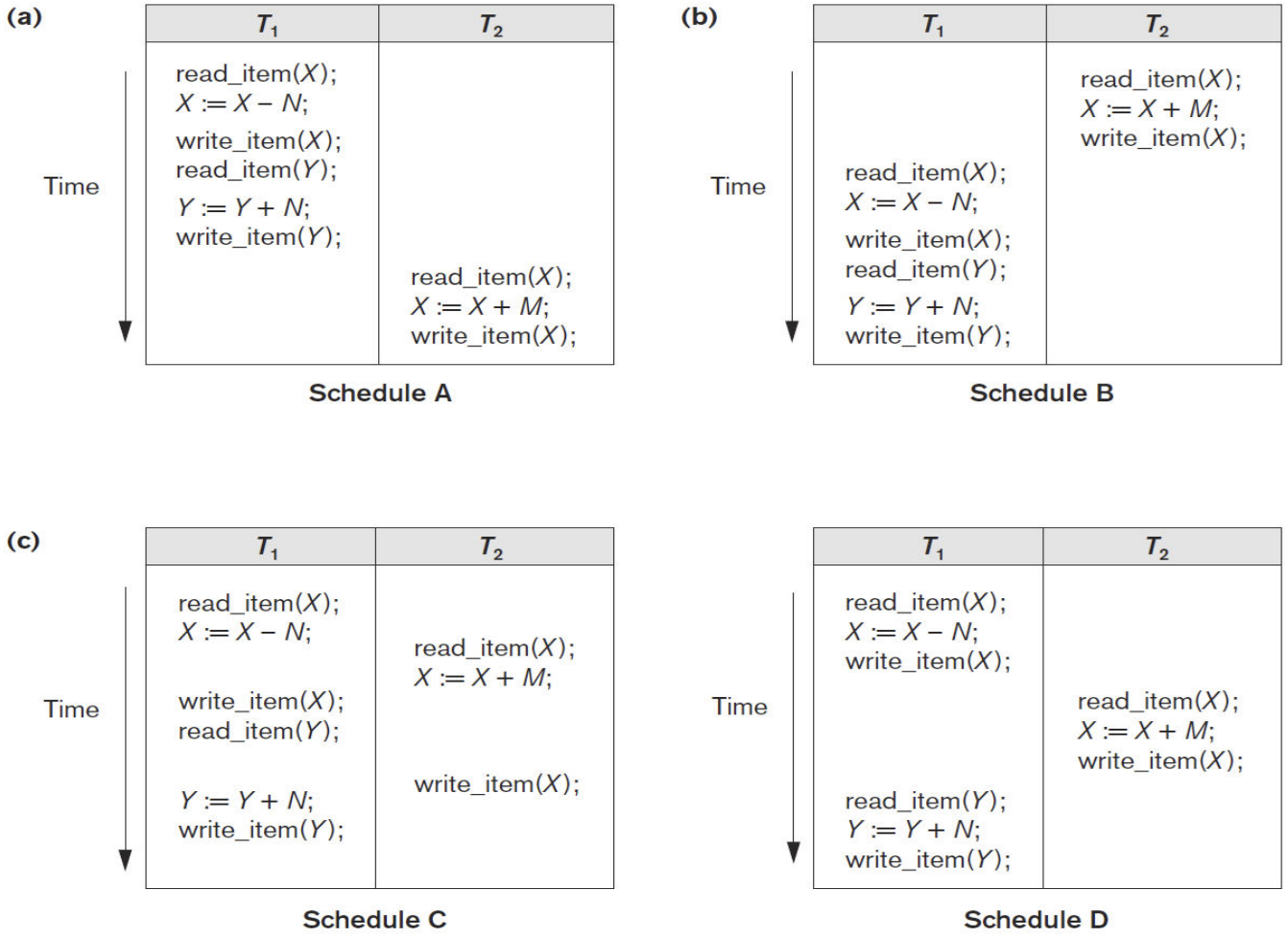


Figure 20.5
Examples of serial and nonserial schedules involving transactions T_1 and T_2. (a) Serial schedule A: T_1 followed by T_2. (b) Serial schedule B: T_2 followed by T_1. (c) Two nonserial schedules C and D with interleaving of operations.

Database Management Systems

Serial Schedules

- **Problem with serial schedules**
  - Limit concurrency by prohibiting interleaving of operations
  - Unacceptable in practice
  - Solution: determine which schedules are equivalent to a serial schedule and allow those to occur
- Serializable schedule of *n* transactions
  - Equivalent to some serial schedule of same *n* transactions

Database Management Systems

Serial Schedules

- Result equivalent schedules
  - Produce the same final state of the database
    - May be accidental
  - Cannot be used alone to define equivalence of schedules

S_1

read_item(X);
$X := X + 10$;
write_item(X);

S_2
read_item(X); $X := X * 1.1$; write_item (X);

Figure 20.6
Two schedules that are result equivalent for the initial value of $X = 100$ but are not result equivalent in general.

Database Management Systems

Conflict Serializable Schedules

- Conflict equivalence of two schedules.
  - Relative order of any two conflicting operations is the same in both schedules.
  - If two conflicting operations are applied in different orders in two schedules, the effect can be different on the database or on the transactions in the schedule, and hence the schedules are not conflict equivalent.
- Serializable schedules
  - Schedule S is serializable if it is conflict equivalent to some serial schedule S'.

Serializability in DBMS

- Some non-serial schedules may lead to inconsistency of the database.
- Serializability is a concept that helps to **identify which non-serial schedules** are correct and will maintain the consistency of the database.

Serializable Schedules

If a given **non-serial schedule** of 'n' transactions is **equivalent to some serial schedule** of 'n' transactions, then it is called as a **serializable schedule**.

Serial Schedules Vs Serializable Schedules

Serializable schedules behave exactly same as serial schedules.
Thus, **serializable schedules are always Consistent**.

Serial Schedules	Serializable Schedules
No concurrency => all the transactions necessarily execute serially one after the other.	**Concurrency =>** multiple transactions can execute concurrently.
Serial schedules lead to **less resource utilization and CPU throughput.**	Serializable schedules **improve both resource utilization and CPU throughput.**
Serial Schedules are **less efficient** as compared to serializable schedules.	Serializable Schedules are always **better** than serial schedules.

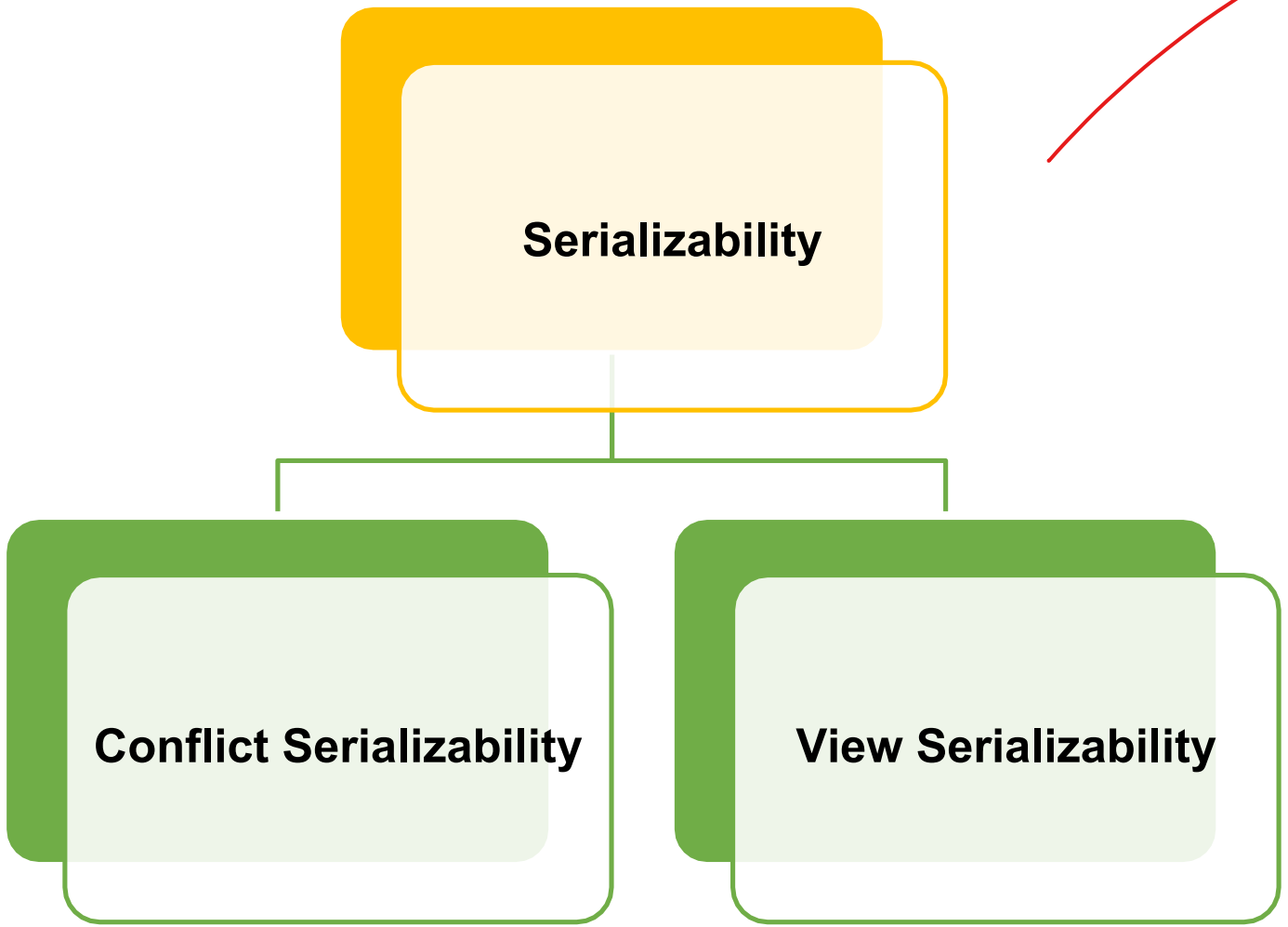
WE ALWAYS WANT TO KEEP CPU RESOURCE UTILIZATIO AS HIGH AS POSSIBLE

Equivalent Schedules

- For two schedules to be equivalent, the operations applied to each data item affected by the schedules should be applied to that item in both schedules in the same order.
- Two definitions of equivalence of schedules are generally used:
  conflict equivalence and view equivalence
- **Conflict Equivalence of Two Schedules. Two schedules are said to be conflict equivalent if the relative order of any two conflicting operations is the same in both schedules.**
- If two conflicting operations are applied in different orders in two schedules, the effect can be different on the database or on the transactions in the schedule, and hence the schedules are not conflict equivalent

BEFOERE STARTING THIS PLEASE READ THIS:
file:///E:/CollegeWork/PESU_SEM_5/DBMS/Unit4/pdf/Reference/conflictequivalenceviewequivalence.pdf

Types of Serializability



Serializability

Before we can consider **how the concurrency-control component of the database system can ensure serializability, we consider how to determine when a schedule is serializable**.

Certainly, *serial schedules are serializable, but if steps of multiple transactions are interleaved, it is harder to determine whether a schedule is serializable.*

Since transactions are programs, it is difficult to determine exactly what operations a transaction performs and how operations of various transactions interact.

For this reason, we shall not consider the various types of operations that a transaction can perform on a data item, but instead consider **only two operations: read and write.**

We assume that, between a **read(Q)** instruction and a **write(Q)** instruction on a data item Q, a transaction may perform an arbitrary sequence of operations on the copy of Q that is residing in the local buffer of the transaction.

this is in transaction's local buffer

Serializability

- **Basic Assumption** – Each transaction preserves database consistency.
- Thus, serial execution of a set of transactions preserves database consistency.
- **A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule**. Different forms of schedule equivalence give rise to the notions of:

  1. Conflict serializability

- Our simplified schedules consist of only **read** and **write** instructions.
- Let us consider a schedule S in which there are two consecutive instructions, I and J, of transactions Ti and Tj , respectively (i ≠ j*). If I and J refer to different data items, then we can swap I and J without affecting the results of any instruction in the schedule*.
- However, *if I and J refer to the same data item Q, then the order of the two steps may matter*. Since we are dealing with only read and write instructions, there are four cases that we need to consider:

Conflicting Instructions

Two operations in a schedule are said to conflict if they satisfy all three of the following conditions:
(1)they belong to different transactions;
(2)they access the same item X; and
(3)at least one of the operations is a write_item(X)

- Instructions I_i and I_j of transactions T_i and T_j respectively, **conflict** if and only if there exists some item Q accessed by both I_i and I_j, and at least one of these instructions wrote Q.
  1. I_i = **read**(Q), I_j = **read**(Q). I_i and I_j don't conflict.
  2. I_i = **read**(Q), I_j = **write**(Q). They conflict.
  3. I_i = **write**(Q), I_j = **read**(Q). They conflict
  4. I_i = **write**(Q), I_j = **write**(Q). They conflict
- Intuitively, a conflict between I_i and I_j forces a (logical) temporal order between them.
- If I_i and I_j are serial in a schedule and they do not conflict, their results would remain the same even if they had been interchanged in the schedule.

Database Management Systems

Conflict Serializability

- If a schedule *S* can be transformed into a schedule *S'* by a series of swaps of non-conflicting instructions, we say that *S* and *S'* are **conflict equivalent**.
- We say that a schedule *S* is **conflict serializable** if it is conflict equivalent to a serial schedule We say that I and J conflict if they

Let's consider two different transactions, T_i and T_j. Then considering the above conditions, the following table follows:

Transaction i	Transaction j	IsConflicting
Readi (X)	Readj (X)	Non-conflicting
Readi (X)	Writej (X)	Conflicting
Writei (X)	Readj (X)	Conflicting
Writei (X)	Writej (X)	Conflicting

Are operations transactions on by different the same data item, and at least one of these instructions is a write operation.

Conflict Serializability

Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1, **by series of swaps of non-conflicting instructions.** Therefore Schedule 3 is conflict serializable.

T_1	T_2
read(A)	
write(A)	
	read(A)
	write(A)
read(B)	
write(B)	
	read(B)
	write(B)

T_1	T_2
read(A)	
write(A)	
	read(A)
read(B)	
	write(A)
write(B)	
	read(B)
	write(B)

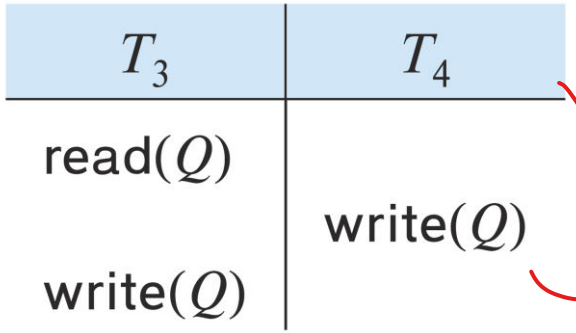
Schedule 5—schedule 3 after swapping of a pair of instructions.

Since the write(A) instruction of T2 in schedule 3 of Figure, does not conflict with the read(B) instruction of T1, we can swap these instructions to generate an equivalent schedule, schedule 5, in Figure. Regardless of the initial system state, schedules 3 and 5 both produce the same final system state.

If a schedule S can be transformed into a schedule S′ by a series of swaps of non-conflicting instructions, we say that S and S′ are conflict equivalent

Conflict Serializability

- Example of a schedule that is not conflict serializable:



here you swap anything and try , all the 3 conditions for a conflic to happe always satisfy so this one can not be conflict equivalent and hence not conflict serializable

read has same item Q as Write , one of them is write and both of them are from different tranaction

similarly

write(Q) and write(Q) of T3 and T4 have same item Q and both are writes adn both belong to different transactions

- We are unable to swap instructions in the above schedule to obtain either the serial schedule
  < T_3, T_4 >, or the serial schedule < T_4, T_3 >.

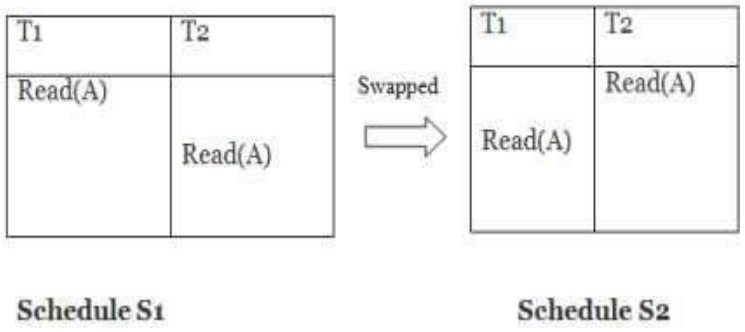
Finally, consider schedule 7 of Figure 17.9; it consists of only the significant operations (that is, the read and write) of transactions T_3 and T_4. This schedule is not conflict serializable, since it is not equivalent to either the serial schedule $<T_3,T_4>$ or the serial schedule $<T_4,T_3>$.

swaps should be made between non cofliciting opeartoins

Conflict Serializability

If a given non-serial schedule can be converted into a serial schedule **by swapping its non-conflicting operations**, then it is called as a **conflict serializable schedule**.

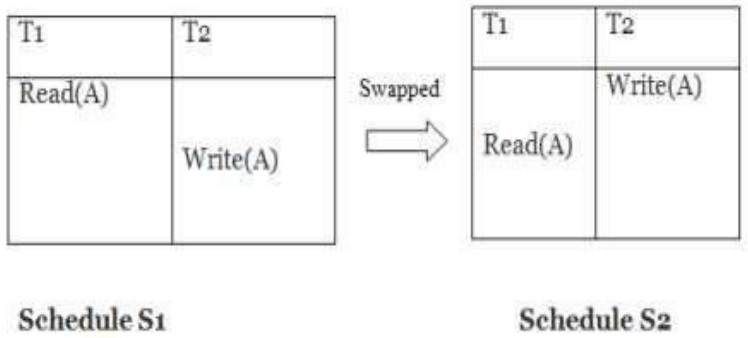
1. T1: Read(A) T2: Read(A)



Schedule S1 Schedule S2

S1=S2
NON CONFLICT

2. T1: Read(A) T2: Write(A)



Schedule S1 Schedule S2

S1 ≠ S2
CONFLICT

Conflict Serializability: Conflicting Operations

all should hold true

Two operations are called as **conflicting operations** if all the following conditions hold true for them-

✓Both the operations belong to **different transactions**

✓Both the operations are on the **same data item**

✓At least one of the two operations is a **write operation**

Conflict Serializability: Conflicting Operations Examples

- **Conflicting** operations pair ($R_1(A)$, $W_2(A)$) because they belong to two different transactions on same data item A and one of them is write operation.

- Similarly, ($W_1(A)$, $W_2(A)$) and ($W_1(A)$, $R_2(A)$) pairs are also **conflicting**.

- On the other hand, ($R_1(A)$, $W_2(B)$) pair is **non-conflicting** because they operate on different data item.

- Similarly, (($W_1(A)$, $W_2(B)$) pair is **non-conflicting.**

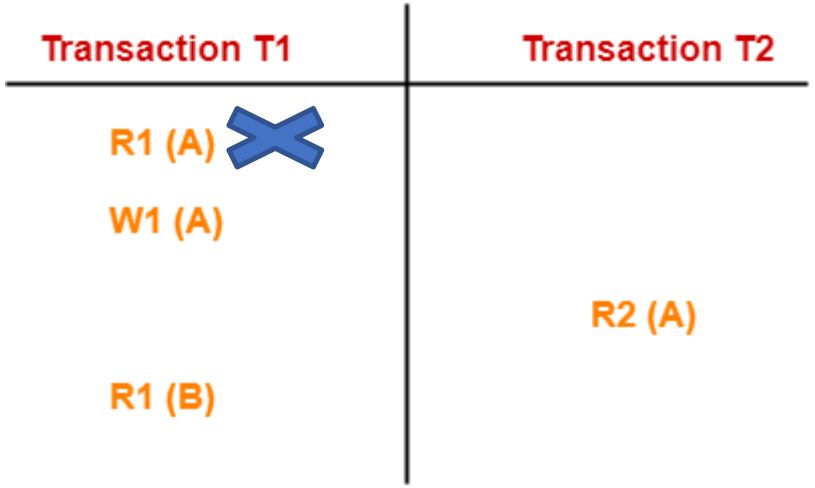
Conflict Serializability: Conflicting Operations

Example- Consider the following schedule-

Transaction T1	Transaction T2
R1 (A)	
W1 (A)	
	R2 (A)
R1 (B)	

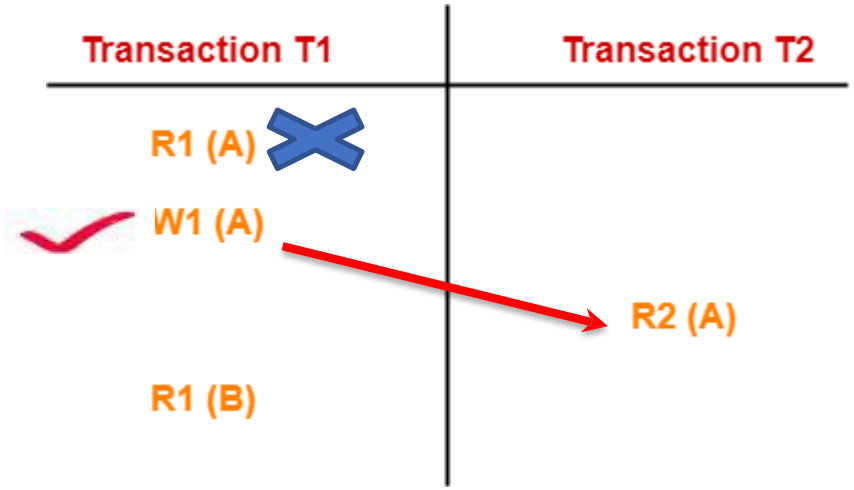
Conflict Serializability: Conflicting Operations

Example- Consider the following schedule-



Conflict Serializability: Conflicting Operations

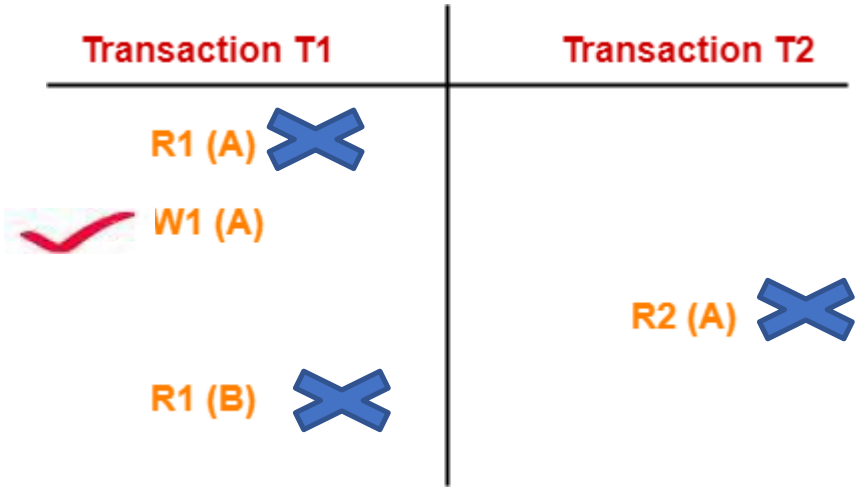
Example- Consider the following schedule-



In this schedule, W1 (A) and R2 (A) are called as conflicting operations.

Conflict Serializability: Conflicting Operations

Example- Consider the following schedule-



In this schedule, W1 (A) and R2 (A) are called as conflicting operations.

Database Management Systems

Testing Conflict Serializability of a schedule

- 1. For each transaction Ti participating in schedule S, create a node labeled Ti in the precedence graph.
- 2. For each case in S where Tj executes a read_item(X) after Ti executes a write_item(X), create an edge (Ti → Tj) in the precedence graph.
- 3. For each case in S where Tj executes a write_item(X) after Ti executes a read_item(X), create an edge (Ti → Tj) in the precedence graph.
- 4. For each case in S where Tj executes a write_item(X) after Ti executes a write_item(X), create an edge (Ti → Tj) in the precedence graph.
- 5. The schedule S is serializable if and only if the precedence graph has no cycles.

basically
r1(x),w2(x)
w1(x),r2(x)
w1(x),w2(x)

Algorithm 20.1 Testing conflict serializability of a schedule S

Checking Whether a Schedule is Conflict Serializable

Follow the following steps to check whether a given non-serial schedule is conflict serializable or not-

Step-01: Find and list all the conflicting operations.

Step-02: Start creating a precedence graph by drawing one node for each transaction.

Step-03: Draw an edge for each conflict pair such that if X_i (V) and Y_j (V) forms a conflict pair then draw an edge from T_i to T_j. This ensures that T_i gets executed before T_j.

Step-04: Check **if there is any cycle** formed in the graph. If there is **no cycle** found, then the **schedule is conflict serializable** otherwise not.

Database Management Systems

Testing for serializability of a schedule:

(a)

Time	T_1	T_2

↓	read_item(X);
	$X := X - N$;
	write_item(X);
	read_item(Y);
	$Y := Y + N$;
	write_item(Y);
		read_item(X);
		$X := X + M$;
		write_item(X);

Schedule A

(b)

Time	T_1	T_2

↓	read_item(X);
	$X := X - N$;
	write_item(X);
	read_item(Y);
	$Y := Y + N$;
	write_item(Y);
		read_item(X);
		$X := X + M$;
		write_item(X);

Schedule B

(c)

Time	T_1	T_2

↓	read_item(X);
	$X := X - N$;
	write_item(X);
	read_item(Y);
	$Y := Y + N$;
	write_item(Y);
		read_item(X);
		$X := X + M$;
		write_item(X);

Schedule C

Time	T_1	T_2

↓	read_item(X);
	$X := X - N$;
	write_item(X);
	read_item(Y);
	$Y := Y + N$;
	write_item(Y);
		read_item(X);
		$X := X + M$;
		write_item(X);

Schedule D

Database Management Systems

Constructing the precedence graphs

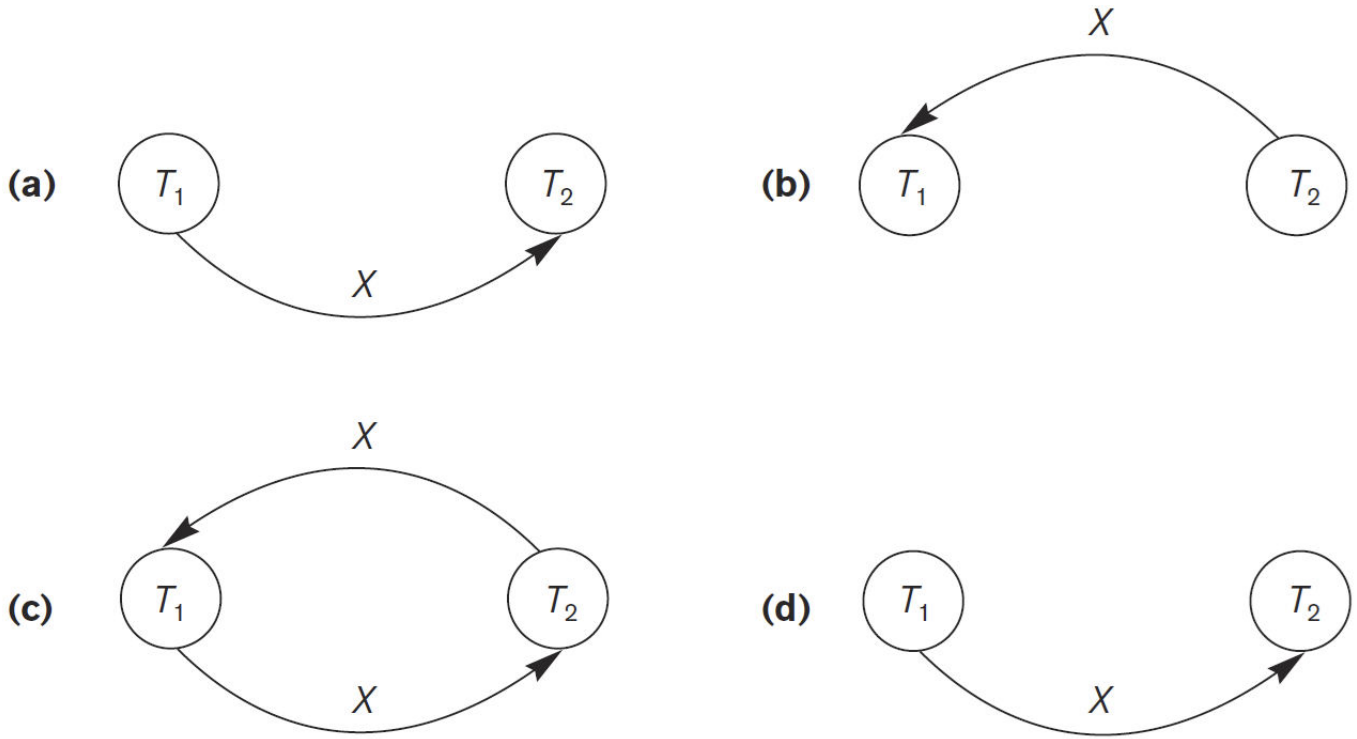


Figure 20.7 Constructing the precedence graphs for schedules A to D from Figure 20.5 to test for conflict serializability (a) Precedence graph for serial schedule A (b) Precedence graph for serial schedule B (c) Precedence graph for schedule C (not serializable) (d) Precedence graph for schedule D (serializable, equivalent to schedule A)

Practice Problems Based On Conflict Serializability

Problem-01:

Check whether the given schedule S is conflict serializable or not-

S : $R_1(A)$, $R_2(A)$, $R_1(B)$, $R_2(B)$, $R_3(B)$, $W_1(A)$, $W_2(B)$

T1	T2	T3
R1(A)		
	R2(A)	
R1(B)		
	R2(B)	
		R3(B)
W1(A)		
	W2(B)	

OR

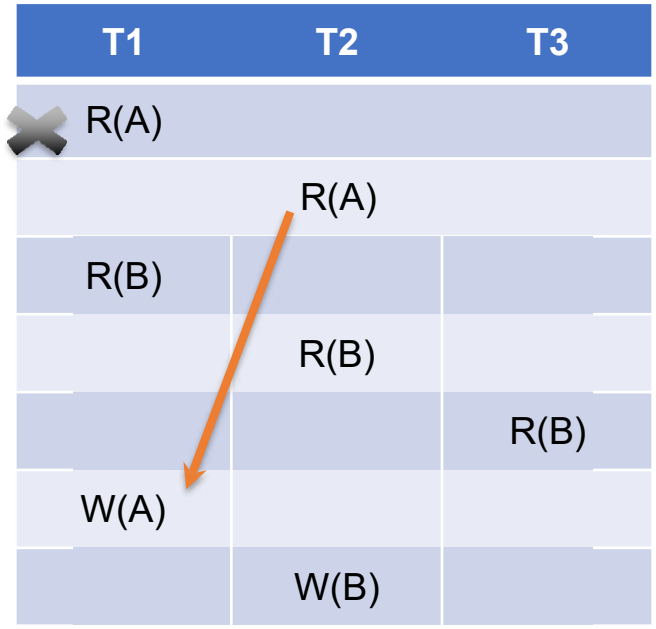
T1	T2	T3
R(A)		
	R(A)	
R(B)		
	R(B)	
		R(B)
W(A)		
	W(B)	

Practice Problems Based On Conflict Serializability

Problem-01:

Check whether the given schedule S is conflict serializable or not-

S : $R_1(A)$, $R_2(A)$, $R_1(B)$, $R_2(B)$, $R_3(B)$, $W_1(A)$, $W_2(B)$

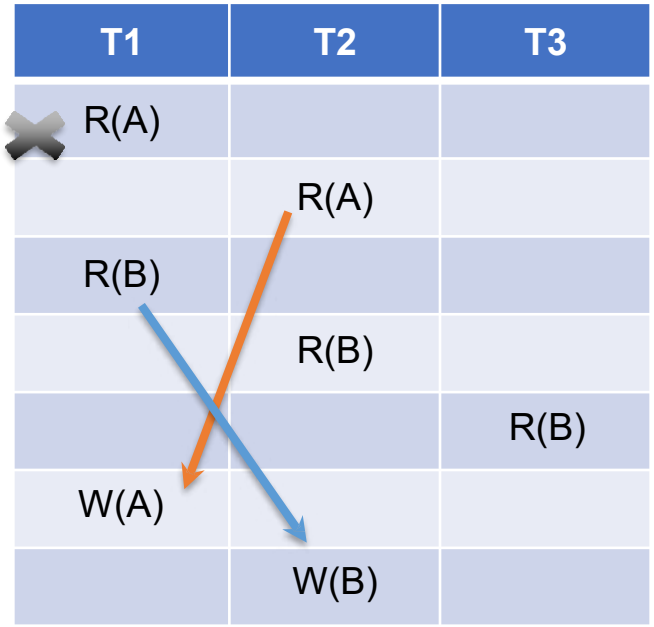


Practice Problems Based On Conflict Serializability

Problem-01:

Check whether the given schedule S is conflict serializable or not-

S : $R_1(A)$, $R_2(A)$, $R_1(B)$, $R_2(B)$, $R_3(B)$, $W_1(A)$, $W_2(B)$

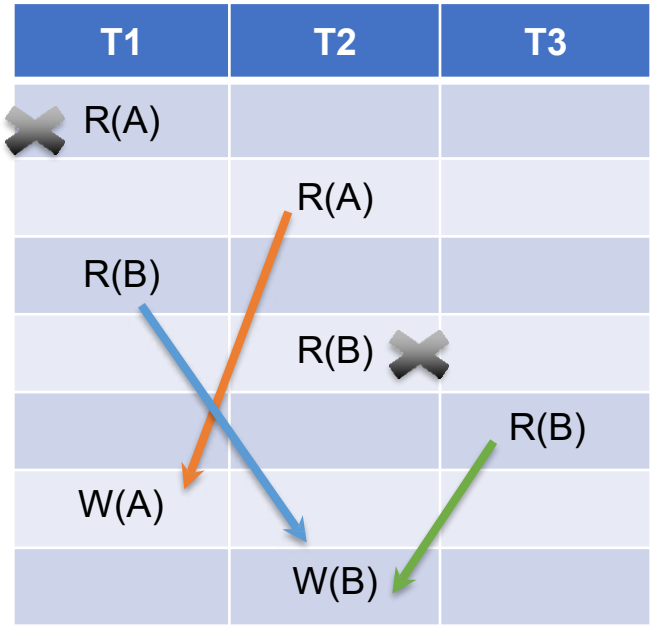


Practice Problems Based On Conflict Serializability

Problem-01:

Check whether the given schedule S is conflict serializable or not-

S : $R_1(A)$, $R_2(A)$, $R_1(B)$, $R_2(B)$, $R_3(B)$, $W_1(A)$, $W_2(B)$



Practice Problems Based On Conflict Serializability

Problem-01:

Check whether the given schedule S is conflict serializable or not-

S : $R_1(A)$, $R_2(A)$, $R_1(B)$, $R_2(B)$, $R_3(B)$, $W_1(A)$, $W_2(B)$

Solution-

Step-01:

List all the conflicting operations and determine the dependency between the transactions-

$R_2(A)$, $W_1(A)$ $(T_2 \rightarrow T_1)$

$R_1(B)$, $W_2(B)$ $(T_1 \rightarrow T_2)$

$R_3(B)$, $W_2(B)$ $(T_3 \rightarrow T_2)$

Practice Problems Based On Conflict Serializability

Problem-01:

Check whether the given schedule S is conflict serializable or not-

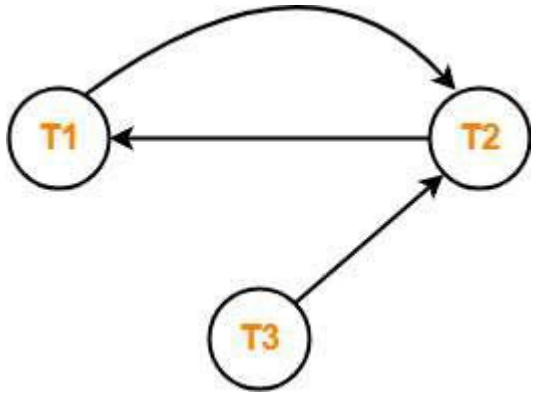
S : $R_1(A)$, $R_2(A)$, $R_1(B)$, $R_2(B)$, $R_3(B)$, $W_1(A)$, $W_2(B)$

Solution-

Step-02:

Draw the precedence graph-

$(T_2 \rightarrow T_1)$
$(T_1 \rightarrow T_2)$
$(T_3 \rightarrow T_2)$



- Clearly, there **exists a cycle** in the precedence graph.
- Therefore, the given schedule **S is not conflict serializable.**

Practice Problems Based On Conflict Serializability

Problem-02:

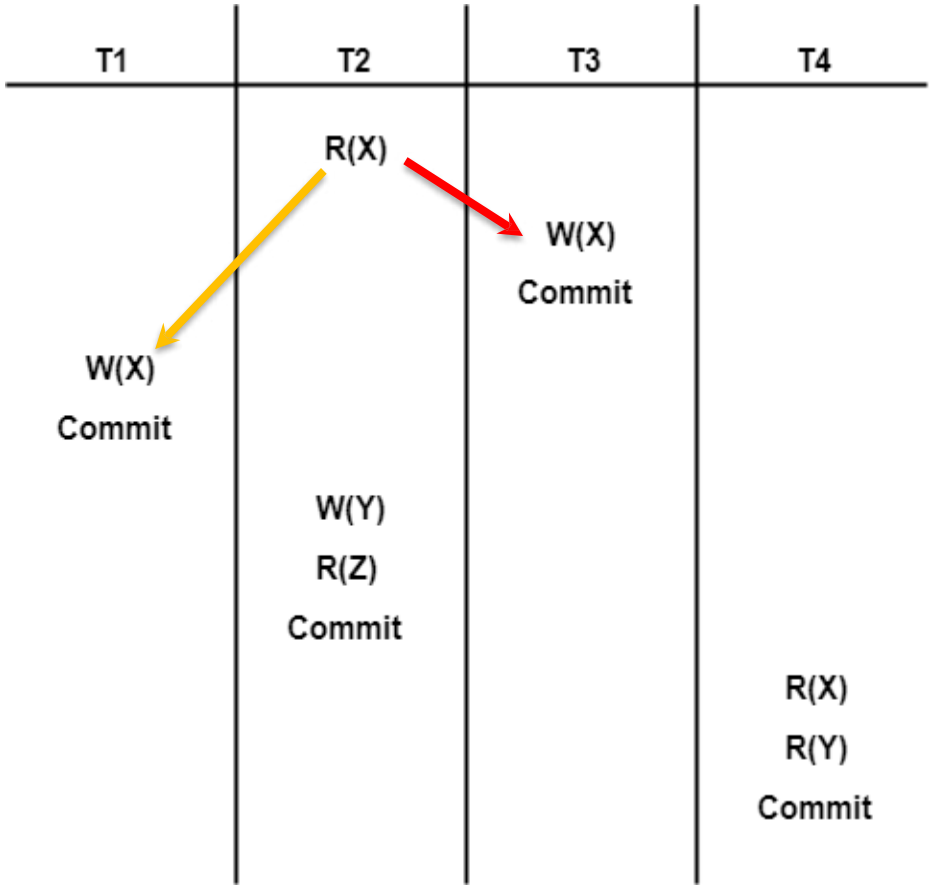
Check whether the given schedule S is conflict serializable or not-

T1	T2	T3	T4
	R(X)		
		W(X)	
		Commit	
W(X)			
Commit			
	W(Y)		
	R(Z)		
	Commit		
			R(X)
			R(Y)
			Commit

Practice Problems Based On Conflict Serializability

Problem-02:

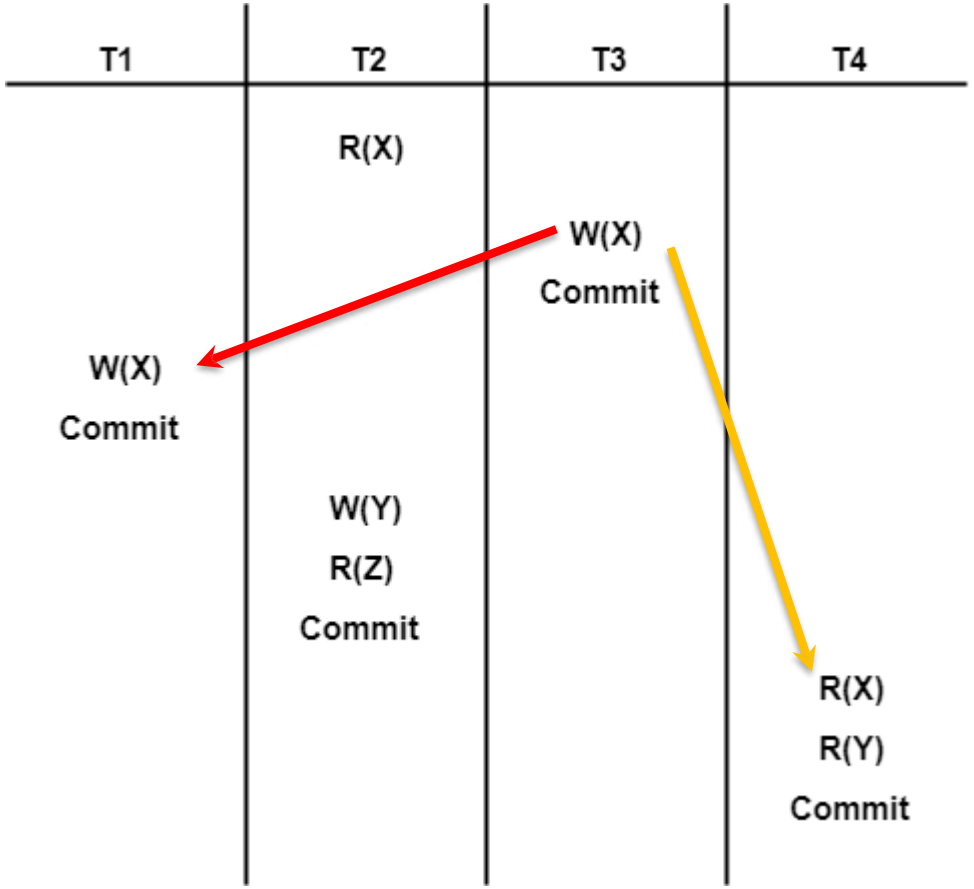
Check whether the given schedule S is conflict serializable or not-



Practice Problems Based On Conflict Serializability

Problem-02:

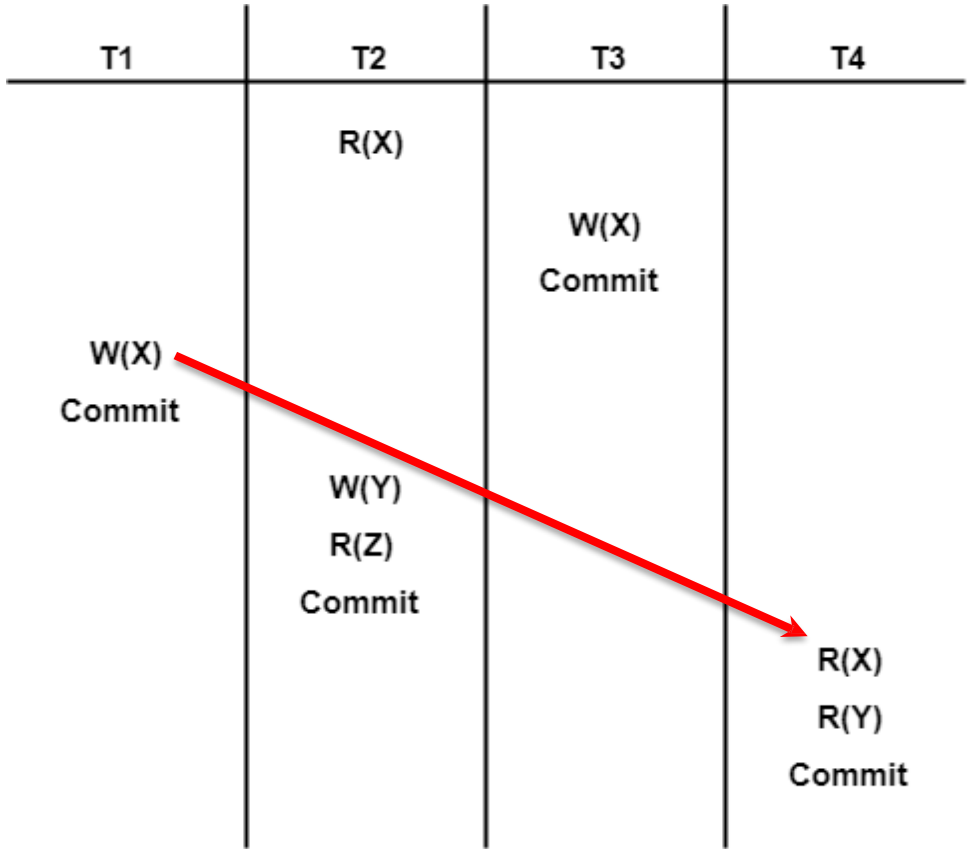
Check whether the given schedule S is conflict serializable or not-



Practice Problems Based On Conflict Serializability

Problem-02:

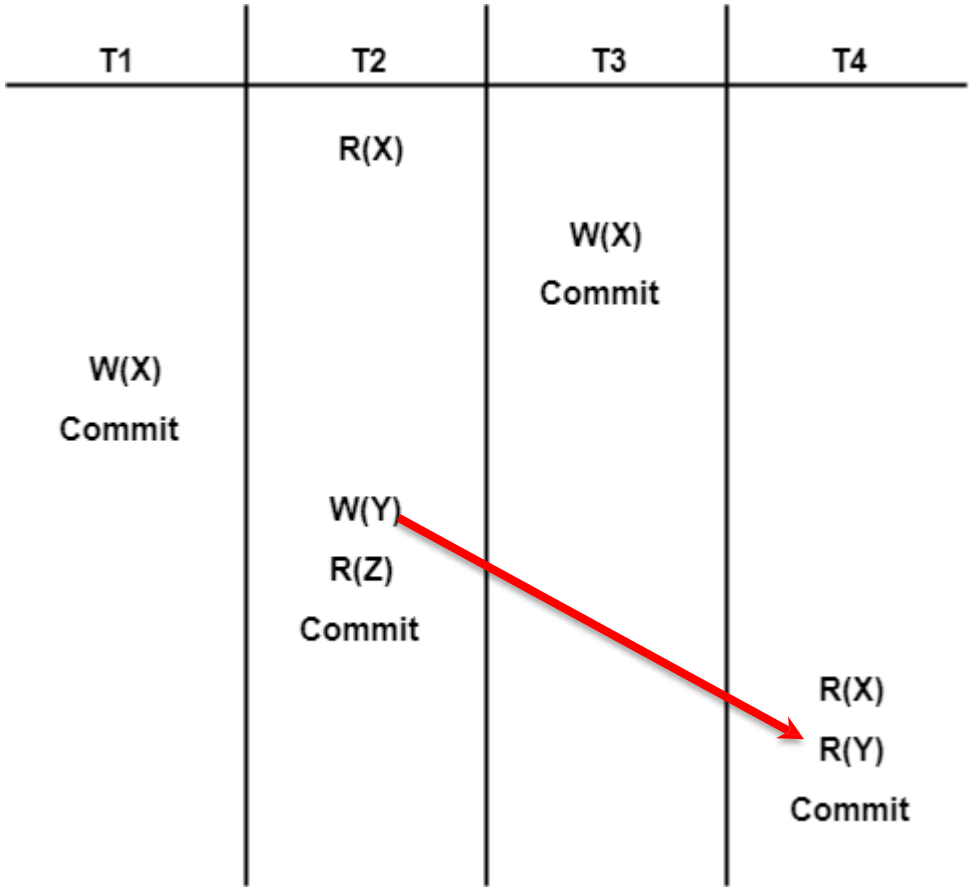
Check whether the given schedule S is conflict serializable or not-



Practice Problems Based On Conflict Serializability

Problem-02:

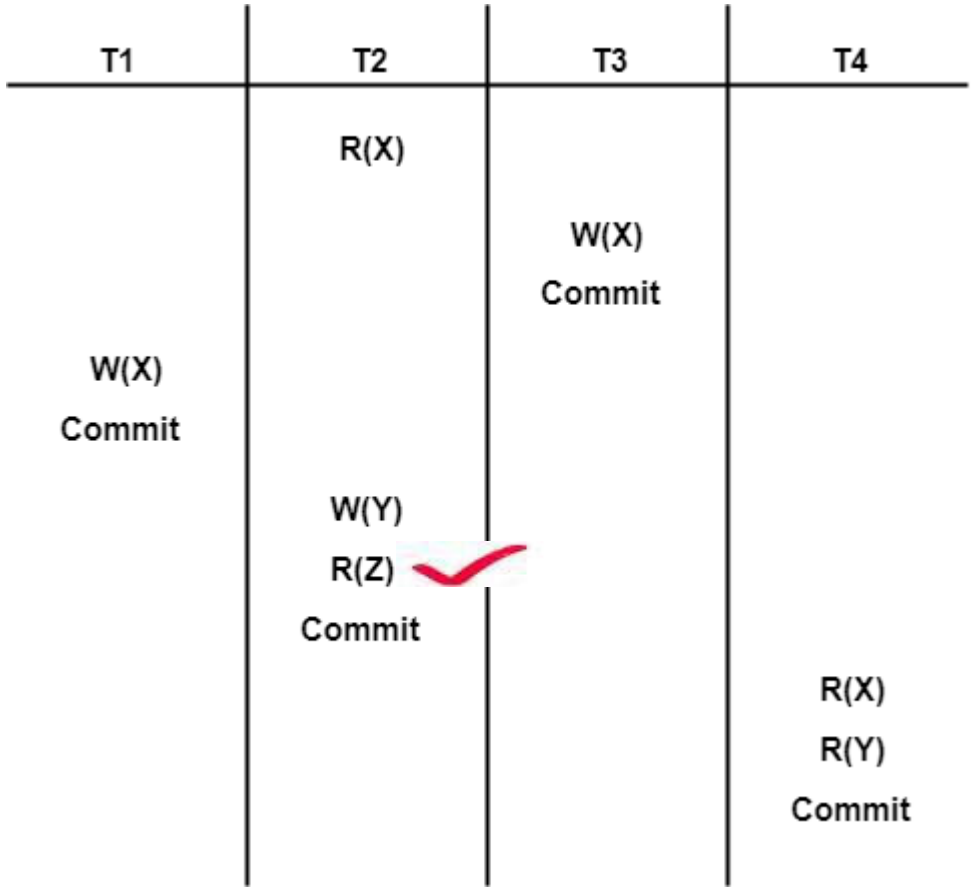
Check whether the given schedule S is conflict serializable or not-



Practice Problems Based On Conflict Serializability

Problem-02:

Check whether the given schedule S is conflict serializable or not-



Practice Problems Based On Conflict Serializability

Solution:

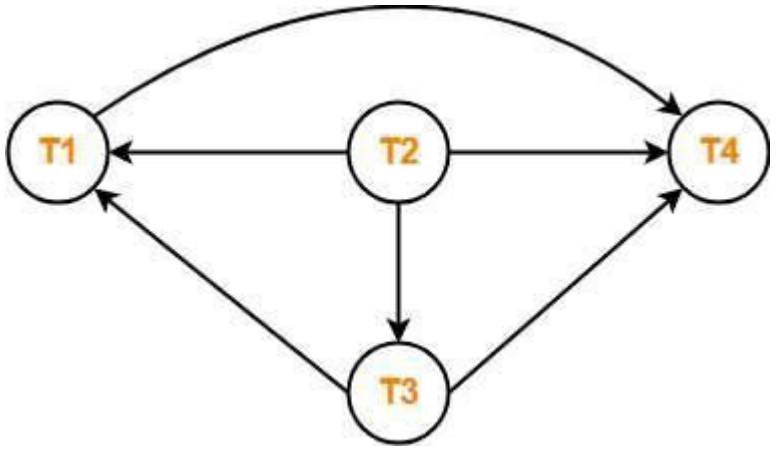
Step-01: List all the conflicting operations and determine the dependency between the transactions-

- $R_2(X)$, $W_3(X)$ $(T_2 \rightarrow T_3)$
- $R_2(X)$, $W_1(X)$ $(T_2 \rightarrow T_1)$
- $W_3(X)$, $W_1(X)$ $(T_3 \rightarrow T_1)$
- $W_3(X)$, $R_4(X)$ $(T_3 \rightarrow T_4)$
- $W_1(X)$, $R_4(X)$ $(T_1 \rightarrow T_4)$
- $W_2(Y)$, $R_4(Y)$ $(T_2 \rightarrow T_4)$

Practice Problems Based On Conflict Serializability

Step-02:

Draw the precedence graph-



$(T_2 \rightarrow T_3)$
$(T_2 \rightarrow T_1)$
$(T_3 \rightarrow T_1)$
$(T_3 \rightarrow T_4)$
$(T_1 \rightarrow T_4)$
$(T_2 \rightarrow T_4)$

- Clearly, there exists **no cycle** in the precedence graph.
- Therefore, the given schedule S **is conflict serializable.**

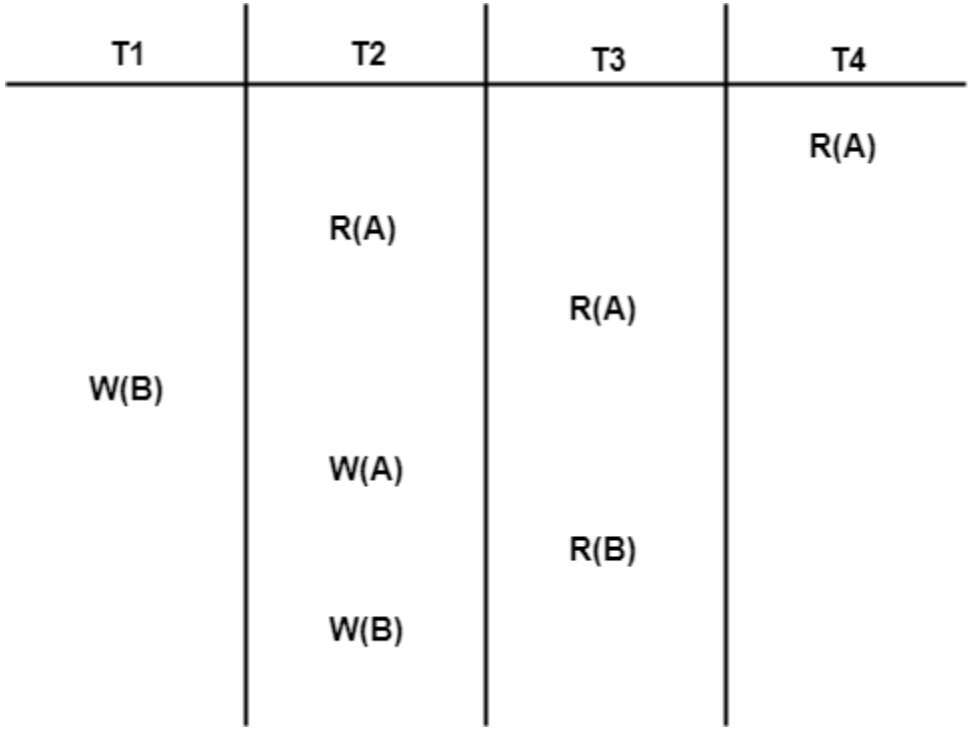
order of seriliazibility will be T2 --->T3---> ---> T1 ---> T4

Trick to know that is first find the node with indegree=0 remove it and keep doing this again and again until nothing is left anymore

Practice Problems Based On Conflict Serializability

Problem-03:

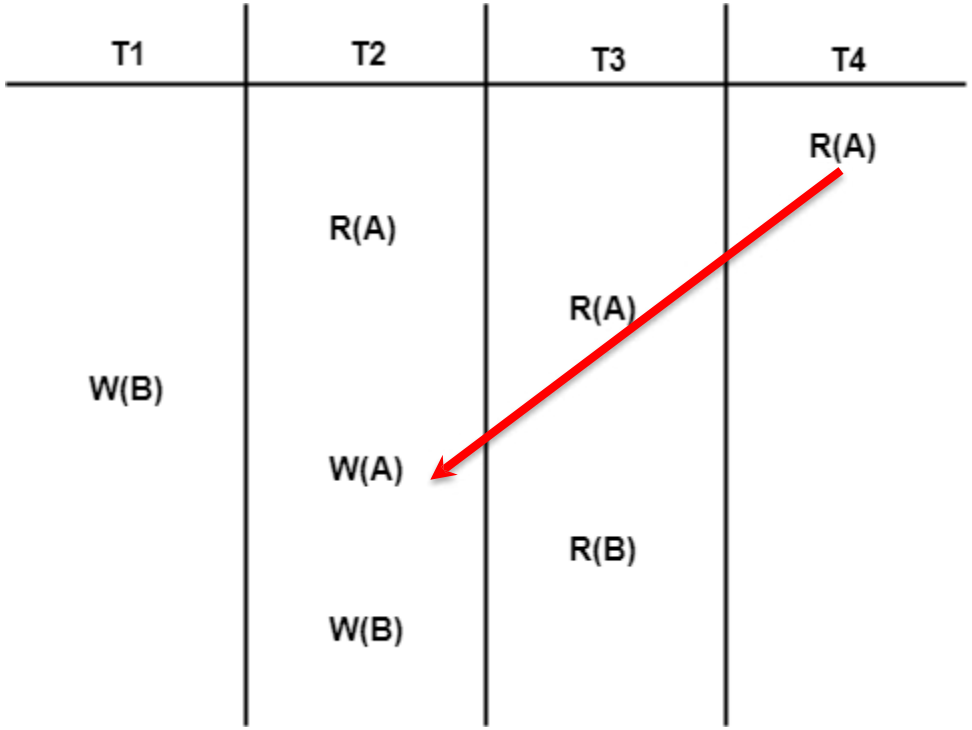
Check whether the given schedule S is conflict serializable or not-



Practice Problems Based On Conflict Serializability

Problem-03:

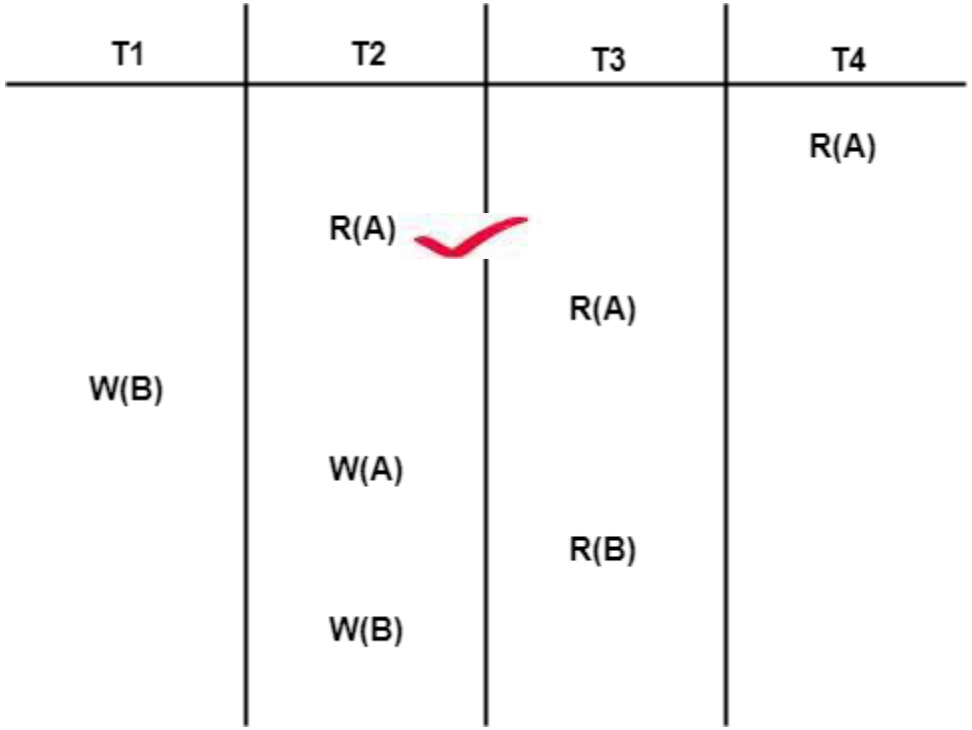
Check whether the given schedule S is conflict serializable or not-



Practice Problems Based On Conflict Serializability

Problem-03:

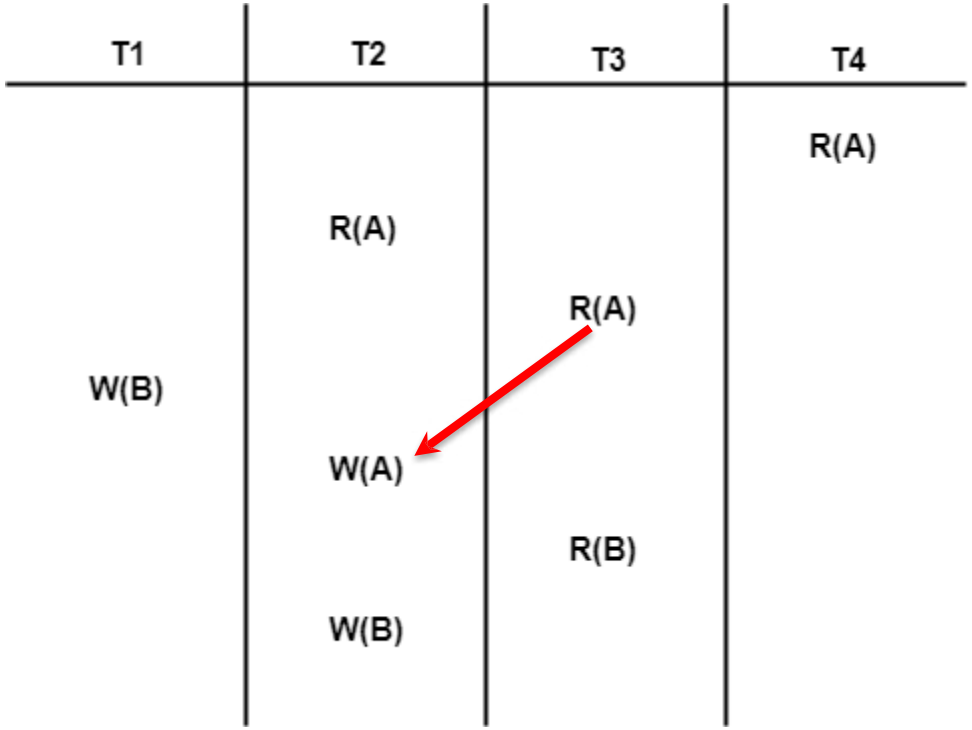
Check whether the given schedule S is conflict serializable or not-



Practice Problems Based On Conflict Serializability

Problem-03:

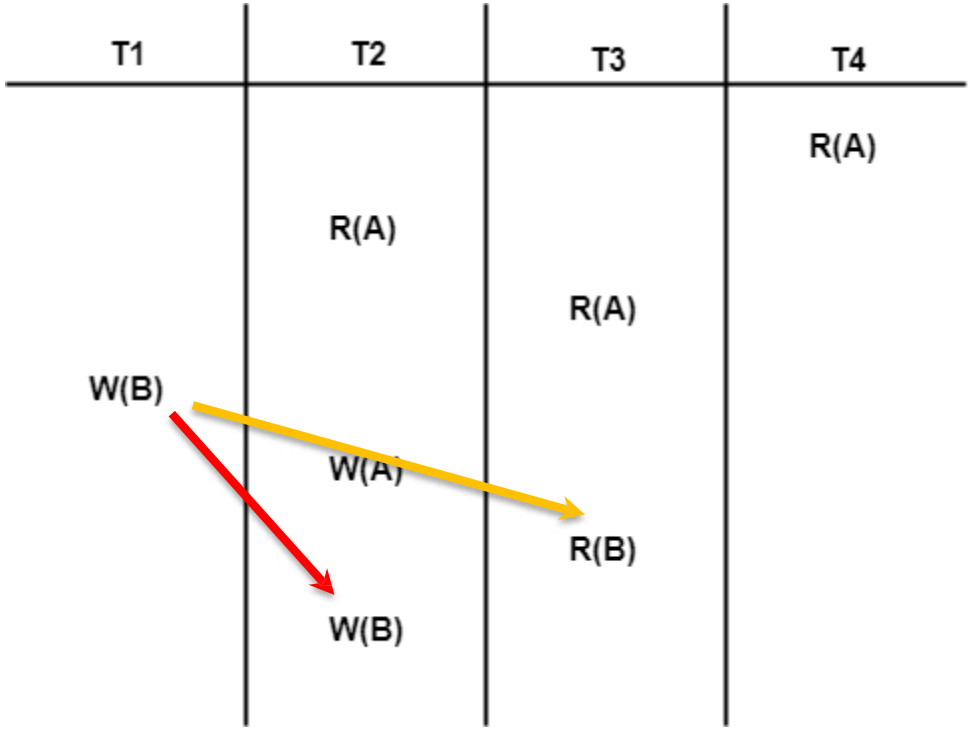
Check whether the given schedule S is conflict serializable or not-



Practice Problems Based On Conflict Serializability

Problem-03:

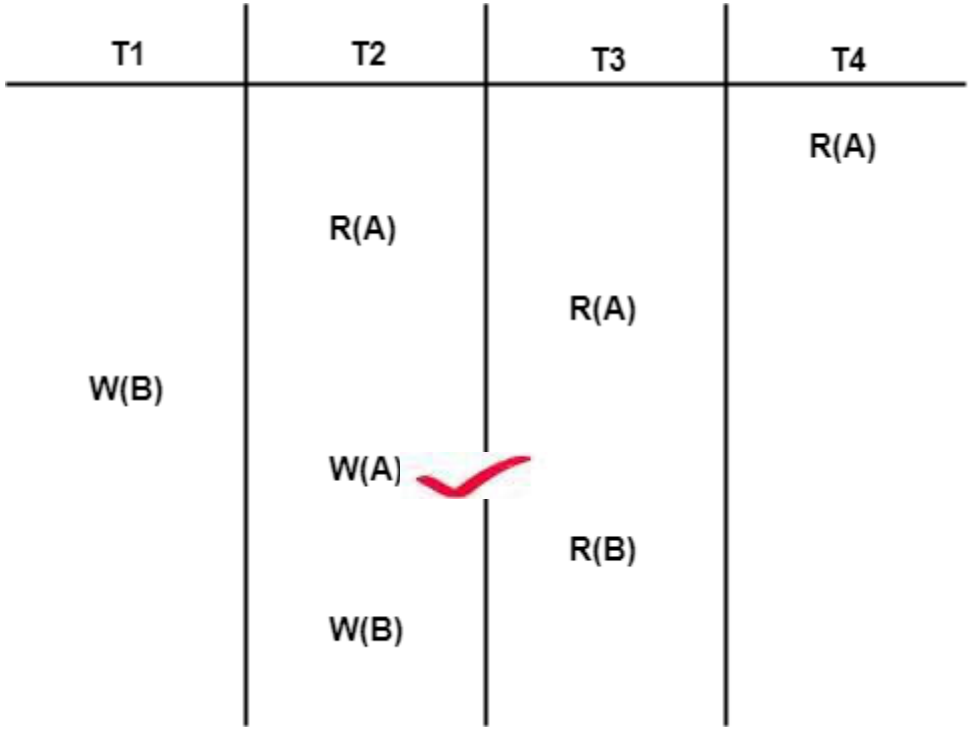
Check whether the given schedule S is conflict serializable or not-



Practice Problems Based On Conflict Serializability

Problem-03:

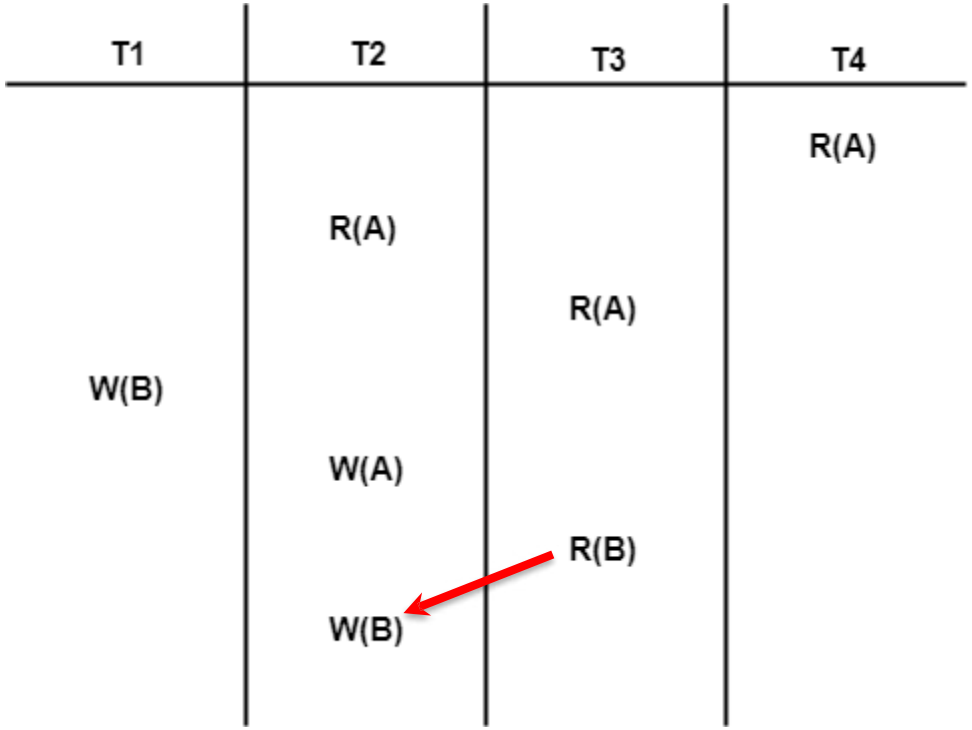
Check whether the given schedule S is conflict serializable or not-



Practice Problems Based On Conflict Serializability

Problem-03:

Check whether the given schedule S is conflict serializable or not-



Practice Problems Based On Conflict Serializability

Solution:

Step-01:

List all the conflicting operations and determine the dependency between the transactions-

- $R_4(A)$, $W_2(A)$ $(T_4 \rightarrow T_2)$
- $R_3(A)$, $W_2(A)$ $(T_3 \rightarrow T_2)$
- $W_1(B)$, $R_3(B)$ $(T_1 \rightarrow T_3)$
- $W_1(B)$, $W_2(B)$ $(T_1 \rightarrow T_2)$
- $R_3(B)$, $W_2(B)$ $(T_3 \rightarrow T_2)$

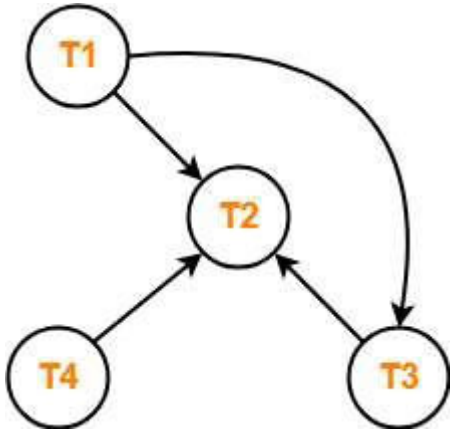
Practice Problems Based On Conflict Serializability

Solution:

$(T_4 \rightarrow T_2)$
$(T_3 \rightarrow T_2)$
$(T_1 \rightarrow T_3)$
$(T_1 \rightarrow T_2)$
$(T_3 \rightarrow T_2)$

Step-02:

Draw the precedence graph-



- Clearly, there exists **no cycle** in the precedence graph.
- Therefore, the given schedule **S is conflict serializable.**

Practice Problems Based On Conflict Serializability

Problem-04:

Check whether the given schedule S is conflict serializable or not-

T1	T2
R(A)	
A = A-10	
	R(A)
	Temp = 0.2 x A
	W(A)
	R(B)
W(A)	
R(B)	
B = B+10	
W(B)	
	B = B+Temp
	W(B)

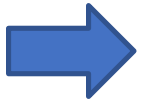
Database Management Systems

Practice Problems Based On Conflict Serializability

Problem-04:

Check whether the given schedule S is conflict serializable or not-

T1	T2
R(A)	
A = A-10	
	R(A)
	Temp = 0.2 x A
	W(A)
	R(B)
W(A)	
R(B)	
B = B+10	
W(B)	
	B = B+Temp
	W(B)

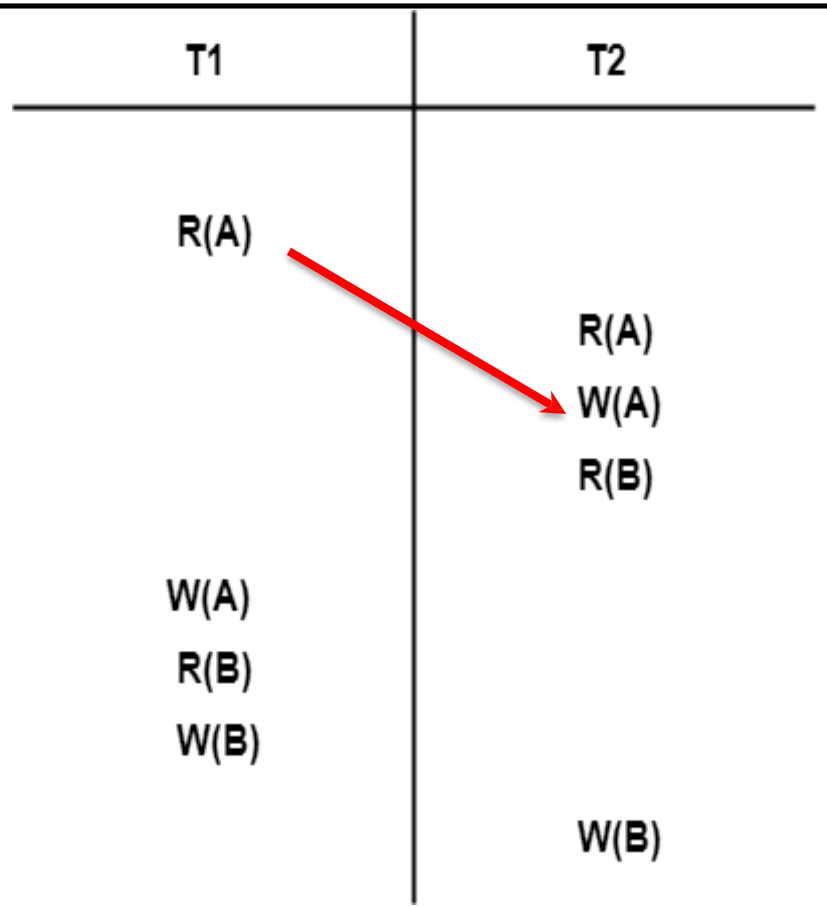


T1	T2
R(A)	
	R(A)
	W(A)
	R(B)
W(A)	
R(B)	
W(B)	
	W(B)

Practice Problems Based On Conflict Serializability

Problem-04:

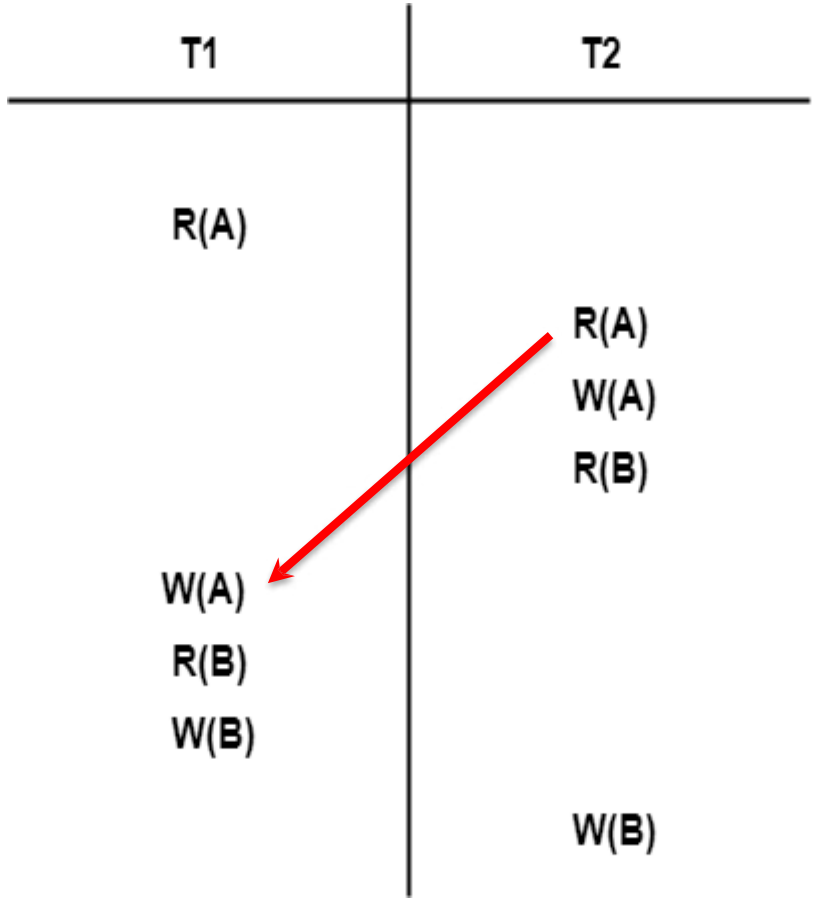
Check whether the given schedule S is conflict serializable or not-



Practice Problems Based On Conflict Serializability

Problem-04:

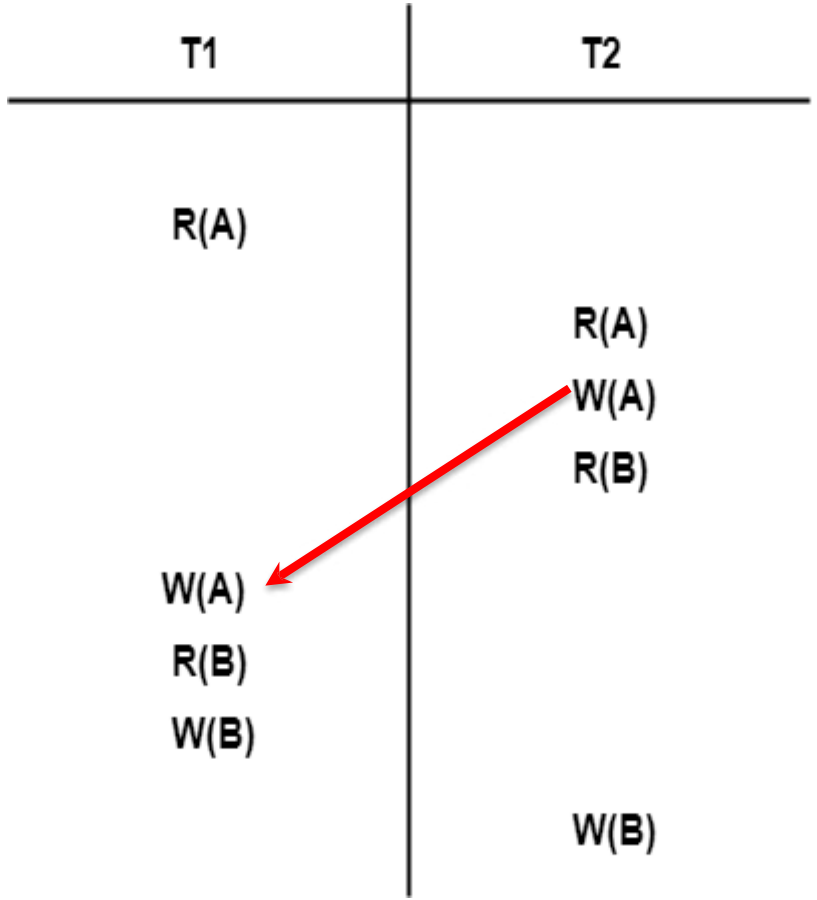
Check whether the given schedule S is conflict serializable or not-



Practice Problems Based On Conflict Serializability

Problem-04:

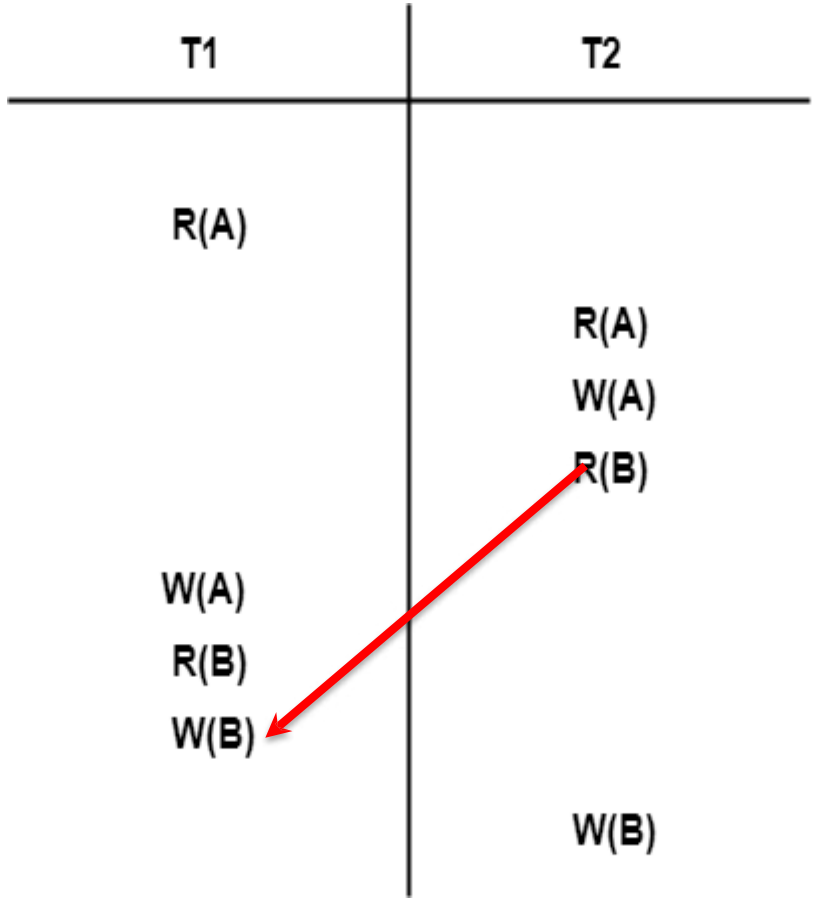
Check whether the given schedule S is conflict serializable or not-



Practice Problems Based On Conflict Serializability

Problem-04:

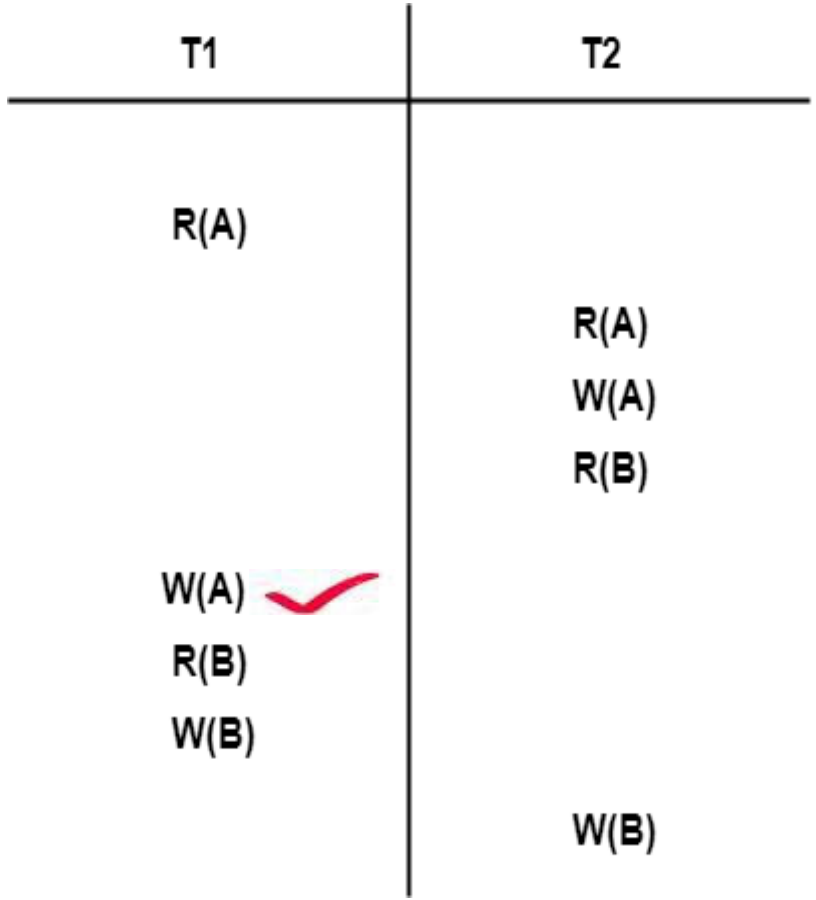
Check whether the given schedule S is conflict serializable or not-



Practice Problems Based On Conflict Serializability

Problem-04:

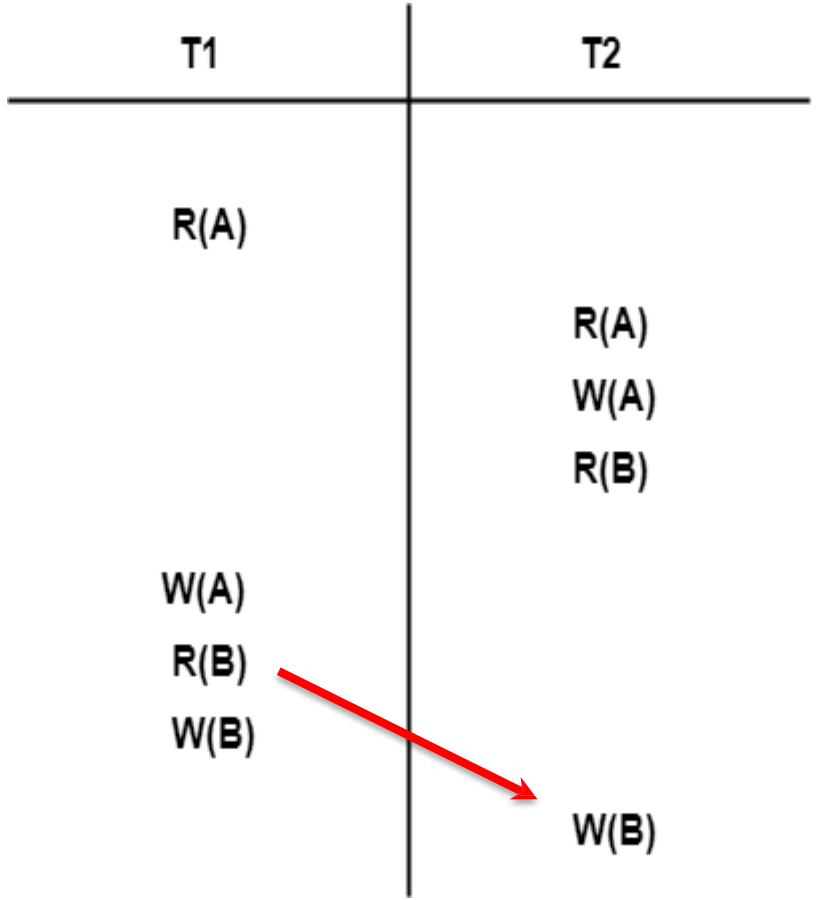
Check whether the given schedule S is conflict serializable or not-



Practice Problems Based On Conflict Serializability

Problem-04:

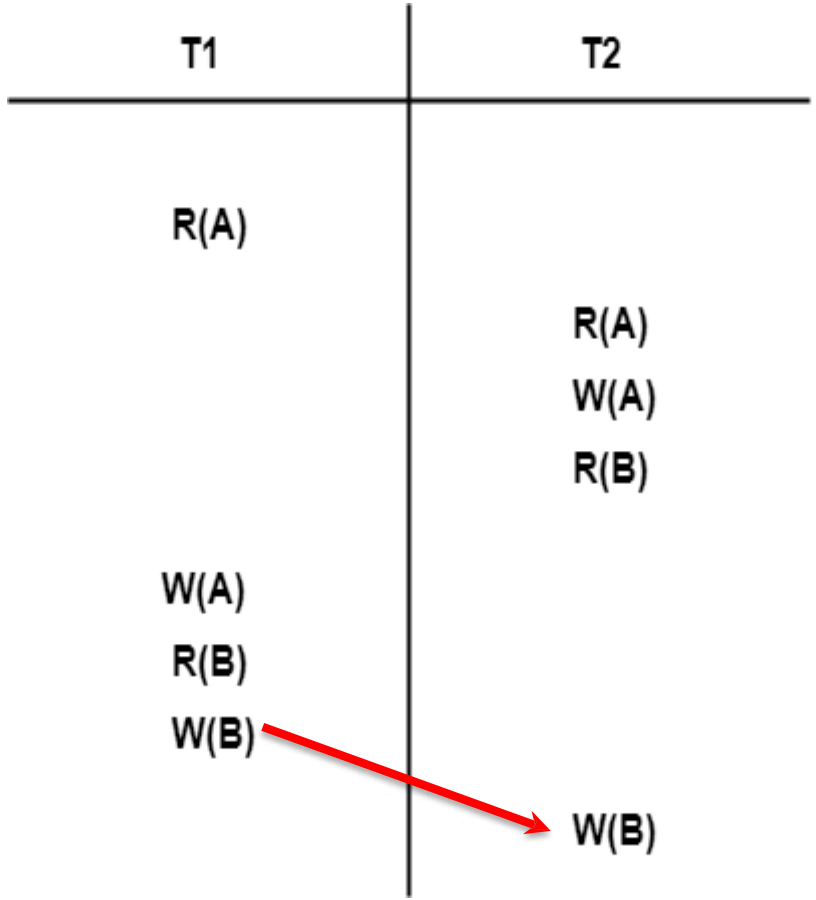
Check whether the given schedule S is conflict serializable or not-



Practice Problems Based On Conflict Serializability

Problem-04:

Check whether the given schedule S is conflict serializable or not-



Practice Problems Based On Conflict Serializability

Step-01:

List all the conflicting operations and dependencies between the transactions-

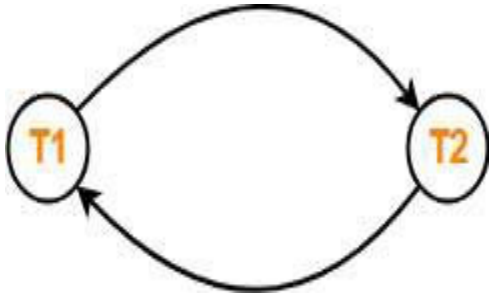
$R_1(A)$, $W_2(A)$	$(T_1 \rightarrow T_2)$
$R_2(A)$, $W_1(A)$	$(T_2 \rightarrow T_1)$
$W_2(A)$, $W_1(A)$	$(T_2 \rightarrow T_1)$
$R_2(B)$, $W_1(B)$	$(T_2 \rightarrow T_1)$
$R_1(B)$, $W_2(B)$	$(T_1 \rightarrow T_2)$
$W_1(B)$, $W_2(B)$	$(T_1 \rightarrow T_2)$

Practice Problems Based On Conflict Serializability

Step-02:

$(T_1 \rightarrow T_2)$
$(T_2 \rightarrow T_1)$

Draw the precedence graph-



- Clearly, there **exists a cycle** in the precedence graph.
- Therefore, the given schedule **S** **is** **not** **conflict serializable.**

Database Management Systems

Serializability

How Serializability is Used for Concurrency Control

- Being serializable is different from being serial
- Serializable schedule gives benefit of concurrent execution
  - Without giving up any correctness
- Difficult to test for serializability in practice
  - Factors such as system load, time of transaction submission, and process priority affect ordering of operations
- DBMS enforces protocols
  - Set of rules to ensure serializability

View Serializability

If a given schedule is found to be view equivalent to some serial schedule, then it is called as a ***view serializable schedule.***

Two schedules are said to be ***view equivalent*** if the order of initial read, final write and update operations is the same in both the schedules.

View equivalent => View Serializable Schedule.

There are other definitions of schedule equivalence based purely on the read and write operations. One such definition is view equivalence, a definition that leads to the concept of view serializability. View serializability is not used in practice due to its high degree of computational complexity

Database Management Systems

View Serializability

☑ 2. VIEW EQUIVALENCE
View equivalence is more general than conflict equivalence.
Two schedules S1 and S2 are view equivalent if:
👉 1. They read the same initial values
👉 2. They read the same values from the same transactions
👉 3. They produce the same final writes (last writer for each item is same)

The idea behind view equivalence is that,

- as long as each read operation of a transaction reads the result of the same write operation in both schedules,
- the write operations of each transaction must produce the same results.
- The read operations are hence said to see the same view in both schedules. the final write operation on each data item is the same in both schedules, so the database state should be the same at the end of both schedules

Let's break those:
Condition 1: Same initial reads
Both schedules must read the original value of a data item from the same place.

Condition 2: Same read-from relationships
If in S1, T2 reads X from T1's write(X), then in S2, T2 must also read X from T1's write(X).
This is key.

Condition 3: Same final writer
The transaction that makes the final write of each data item must be the same in both schedules.

Database Management Systems

View Equivalence and View Serializability

- View equivalence of two schedules
  - As long as each read operation of a transaction reads the result of the same write operation in both schedules, the write operations of each transaction must produce the same results
  - Read operations are hence said to see the same view in both schedules.
- View serializable schedule
  - View equivalent to a serial schedule

Blind Write

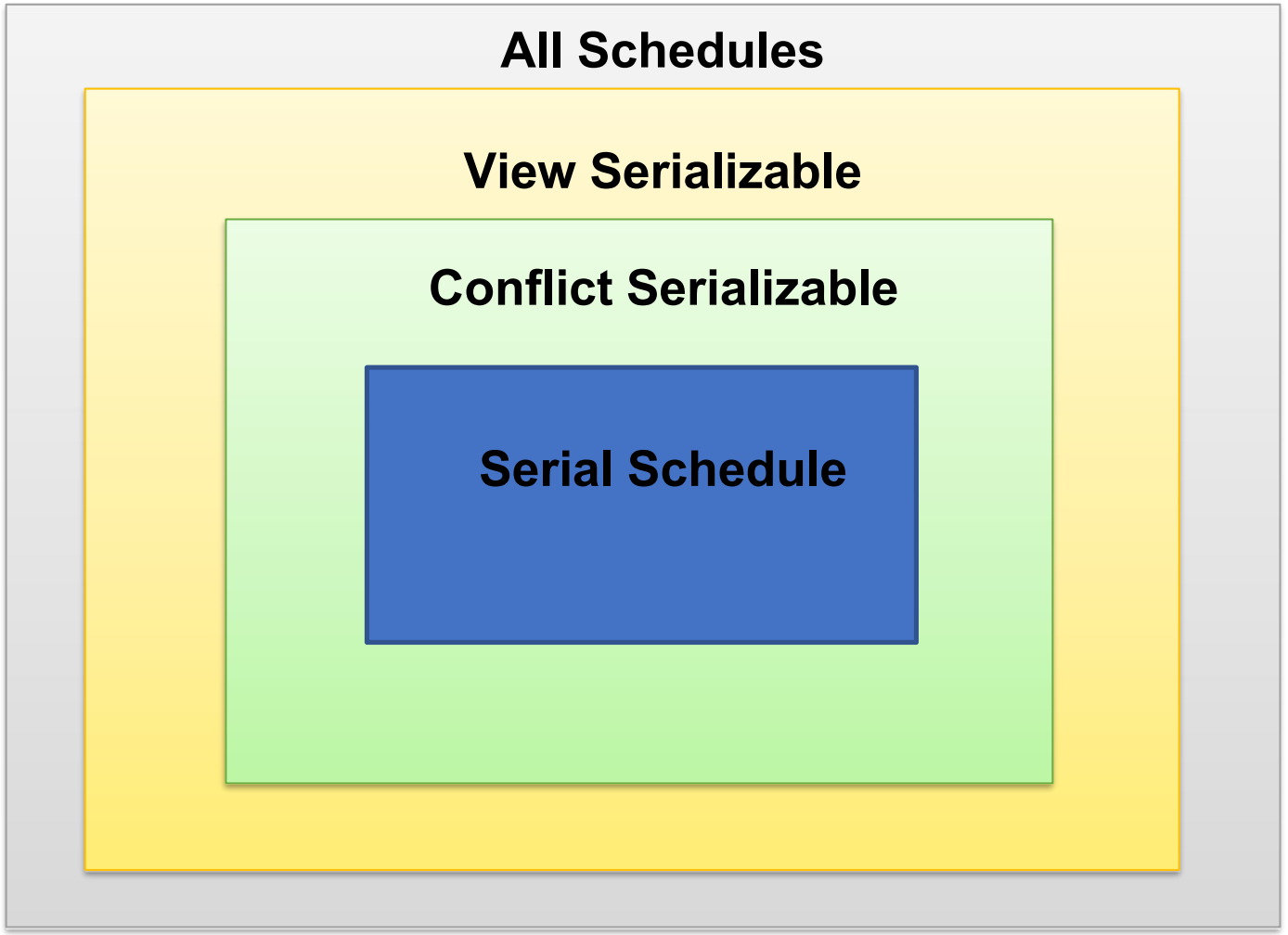
If no read happens before the first write, then it is said to be a *blind write*.

Example. Consider the following schedule:

- **$W_3(X)$** is a blind write, as there is no read before write [$R_3(X)$ before $W_3(X)$]
- **$W_2(X)$** is not a blind write, as a read happens before write [$R_2(X)$ before $W_2(X)$]

T1	T2	T3
	$R_2(X)$	
$R_1(X)$		
		$W_3(X)$
	$W_2(X)$	

Relationship Among Various Schedules



Locks

- A lock is a mechanism to **control concurrent access** to a data item.
- Data items can be locked in two modes :
  - **Exclusive (X)** mode : Data item can be both read as well as written. X-lock is requested using lock-X instruction.
  - **Shared (S)** mode. Data item can only be read. S-lock is requested using lock-S instruction.
- Lock requests are made to concurrency-control manager. Transaction can proceed only after request is granted.

Exclusive. If a transaction Ti has obtained an exclusive-mode lock (denoted by X) on item Q, then Ti can both read and write Q.
Shared. If a transaction Ti has obtained a shared-mode lock (denoted by S) on item Q, then Ti can read, but cannot write, Q

Lock-Based Protocols

- *Allow a transaction to access a data item only if it is currently holding a **lock** on that item.*
- **A transaction acquires a lock on the entire database before it starts and releases the lock after it has committed.**
- While a transaction ***holds a lock, no other transaction is allowed to acquire the lock***, and all must therefore wait for the lock to be released.
- As a result of the locking policy, only one transaction can execute at a time.
- A **concurrency-control policy such as this one leads to poor performance**, since it forces transactions to wait for preceding transactions to finish before they can start. In other words, it provides **a poor degree of concurrency**.

Lock-Based Protocols

Lock-compatibility matrix

- The given figure explains a way to determine the compatibility between different lock modes using a **compatibility function.**

- It is used to define whether a pair of lock modes can peacefully coexist on a shared resource, or if granting a lock of one mode could potentially block another lock of a different mode.

	S	X
S	true	false
X	false	false

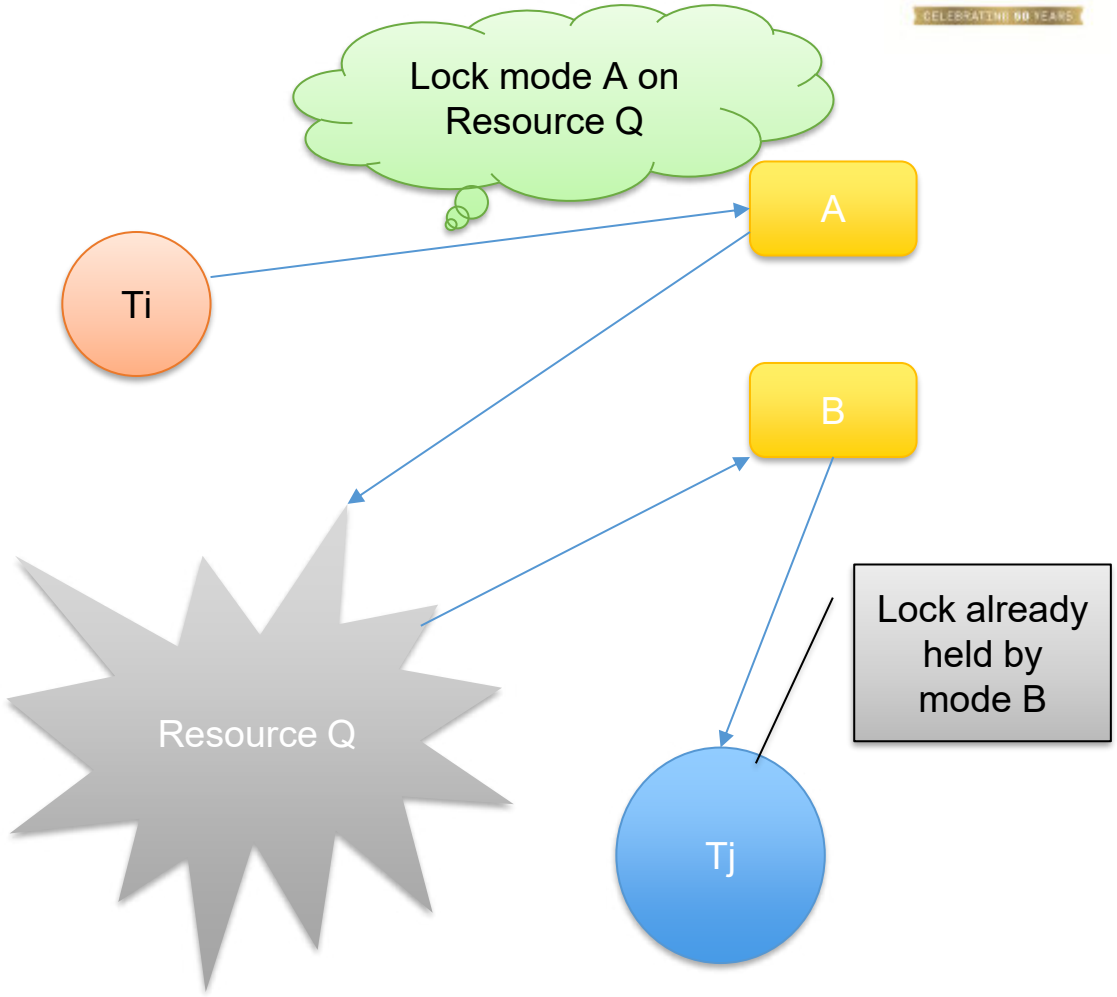
Shared mode is compatible with shared mode, but not with exclusive mode. At any time, several shared-mode locks can be held simultaneously.

Lock-Based Protocols

Lock-compatibility matrix

- Given two lock modes, A and B, compatibility is established based on whether a transaction Ti, requesting a lock of mode A on a resource Q, can be granted that lock despite the presence of a lock of mode B held by another transaction Tj on the same resource Q. **If Ti can acquire the lock without being blocked by the existing B mode lock, then mode A is considered compatible with mode B.**

- An element comp(A, B) of the matrix has the value true if and only if mode A is compatible with mode B



Lock-compatibility matrix

- **The transaction requests the concurrency-control manager. The transaction can proceed with the operation only after the concurrency-control manager grants the lock to the transaction. The use of these two lock modes allows multiple transactions to read a data item, but limits write access to just one transaction at a time.**

- To access a data item, transaction Ti must first lock that item. *If the data item is already locked by another transaction in an incompatible mode, the concurrency-control manager will not grant the* ***lock until all incompatible locks held by other transactions have been released****.* Thus, Ti is made to **wait** until all incompatible locks held by other transactions have been released.

Example

Suppose that the values of accounts A and B are $100 and $200, respectively. If these two transactions are executed serially, either in the order T1, T2 or the order T2,T1, then transaction T2 will display the value $300. **If, however, these transactions are executed concurrently, transaction T2 displays $250, which is incorrect.**

T_1: lock-X(B);
read(B);
$B := B - 50$;
write(B);
unlock(B);
lock-X(A);
read(A);
$A := A + 50$;
write(A);
unlock(A).

T_2: lock-S(A);
read(A);
unlock(A);
lock-S(B);
read(B);
unlock(B);
display($A + B$).

Lock-Based Protocols

EARLIER SLIDE BASICALLY T1 IS DOING B=B-50 AND A=A+50 I.E TRANSFER BU THERE EVEN BEFORE DOING A=A+50 I , WHICH IT HAD TO USE TO DISLPLAY IT LEFT THE LOCK TOO EARLY

Example

The reason is that the T1 unlocked data item B too early, as a result of which T2 saw an inconsistent state.

The image shows the actions executed by the transactions, as well as the points at which the concurrency-control manager grants the locks. **The transaction making a lock request cannot execute its next action until the concurrency-control manager grants the lock.**

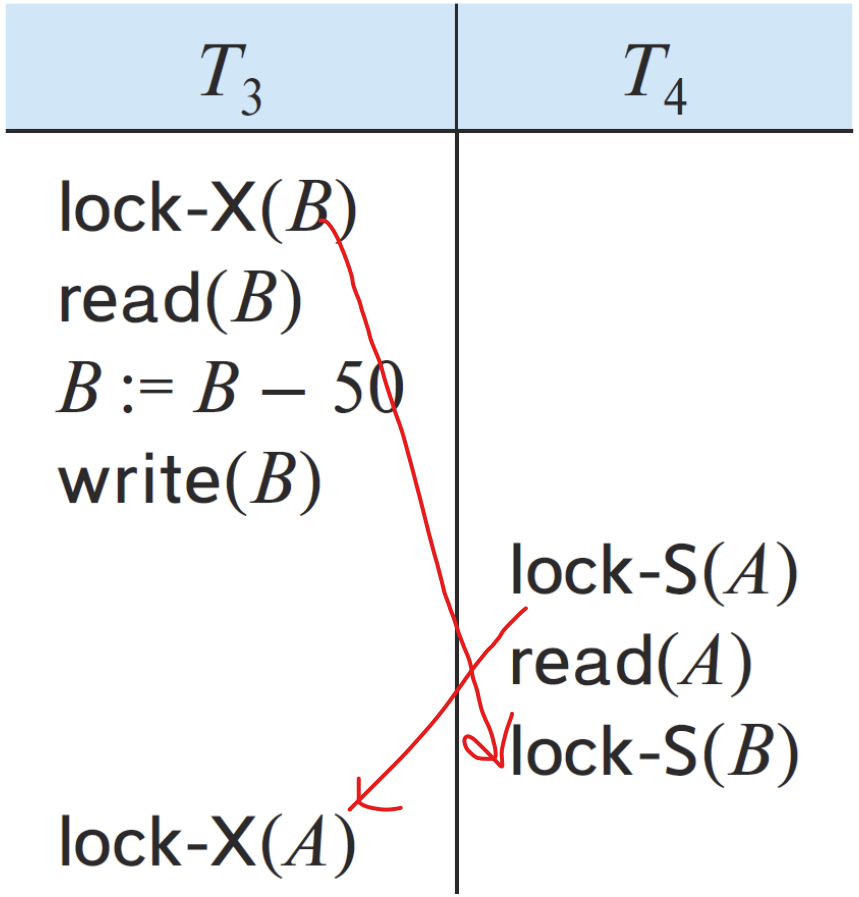
T_1	T_2	concurrency-control manager
lock-X(B)		
		grant-X(B, T_1)
read(B)		
$B := B - 50$		
write(B)		
unlock(B)		
	lock-S(A)	
		grant-S(A, T_2)
	read(A)	
	unlock(A)	
	lock-S(B)	
		grant-S(B, T_2)
	read(B)	
	unlock(B)	
	display($A + B$)	
lock-X(A)		
		grant-X(A, T_1)
read(A)		
$A := A + 50$		
write(A)		
unlock(A)		

if you didn't udner stnad this example refer:
file:///E:/CollegeWork/PESU_SEM_5/DBMS/Unit4/pdf/Reference/lockbasedprotocolexample.pdf

Deadlock

- Consider transactions T3 and T4 where they are with transactions T1 and T2 respectively with the unlocking delayed.

- **Neither T3 or T4 can make progress** — executing lock-S(B) causes T4 to wait for T3 to release its lock on B, while executing lock-X(A) causes T3 to wait for T4 to release its lock on A.
- Such a situation is called a **deadlock**.
- One of the two transactions must rollback.



MySQL 8.0 Command Line Client

```
17 rows in set (0.41 sec)

mysql> use test_deadlock;
ERROR 1049 (42000): Unknown database 'test_deadlock'
mysql> use testdeadlock;
Database changed
mysql> CREATE TABLE `table1` ( `id` INT NOT NULL AUTO_INCREMENT, `name` VARCHAR(255) NOT NULL, `marks` INT NOT NULL, PRI
MARY KEY (`id`) ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.77 sec)

mysql>
mysql> INSERT INTO table1 (id, name, marks) VALUES (1, "abc", 5);
Query OK, 1 row affected (0.36 sec)

mysql>
mysql> INSERT INTO table1 (id, name, marks) VALUES (2, "xyz", 1);
Query OK, 1 row affected (0.10 sec)

mysql> BEGIN;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE table1 SET marks=marks-1 WHERE id=1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE table1 SET marks=marks+1 WHERE id=2;
Query OK, 1 row affected (11.42 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
```

```
First window (continued)
UPDATE table1 SET marks=marks+1 WHERE id=2;
COMMIT;
```

ple.sql

```
AR(255) NOT NULL, `marks` I
```

MySQL 8.0 Command Line Client

```
Enter password: ********
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 12
Server version: 8.0.31 MySQL Community Server - GPL

Copyright (c) 2000, 2022, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database testdeadlock;
Query OK, 1 row affected (0.30 sec)

mysql> use testdeadlock;
Database changed
mysql> BEGIN;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE table1 SET marks=marks+1 WHERE id=2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE table1 SET marks=marks-1 WHERE id=1;
ERROR 1205 (HY000): Lock wait timeout exceeded; try restarting transaction
mysql> UPDATE table1 SET marks=marks-1 WHERE id=1;
ERROR 1213 (40001): Deadlock found when trying to get lock; try restarting transaction
mysql>
```

Deadlock Code Example

Description

- The previous screenshot shows MySQL shell executions demonstrating a **deadlock situation.** Two transactions **attempt to update the same rows** in a different order, leading to lock contention, a deadlock error, and one transaction being rolled back by MySQL.

Inferences

- **Deadlock Detection:** MySQL automatically detects and resolves deadlocks by rolling back one of the conflicting transactions to break the cycle.
- **Lock Contention:** Consistent ordering of row updates in transactions is crucial to avoid deadlocks and prevent lock contention issues.

Locking Protocol

- A **locking protocol** is a set of rules followed by all transactions while requesting and releasing locks.

- Given a locking protocol :
  - A schedule S is **legal** under a locking protocol **if it can be generated by a set of transactions that follow the protocol.**
  - A protocol **ensures** serializability **if all legal schedules under that protocol are serializable**

Granting of Locks

- Transaction T2 has a shared-mode lock on a data item, preventing other transactions from accessing it concurrently for write operation.
- Transaction T1 requests an exclusive-mode lock on the same data item and is put on hold, waiting for T2 to release its lock.
- Transaction T3 requests a shared-mode lock, which is compatible with T2's lock, and T3's request is granted, making T1 wait for T3 to finish.

Lock-Based Protocols

- If transaction T4's request for a shared-mode lock is granted before T3 releases its T1's exclusive-mode request remains delayed.

- This sequence of shared-mode transactions can lead to a situation where T1 remains blocked indefinitely, causing **starvation**, as it's unable to acquire the exclusive-mode lock despite repeated requests.

FOR BETTER UNDERSTANDING REFER THIS FIRST
file:///E:/CollegeWork/PESU_SEM_5/DBMS/Unit4/pdf/Reference/2Phaselocking.pdf

The Two-Phase Locking Protocol

- This protocol ensures conflict-serializable schedules.
  - Phase 1: **Growing Phase** :A transaction may obtain locks, but may not release any lock.
  - Phase 2: **Shrinking Phase** : A transaction may release locks, but may not obtain any new locks
- Initially, a transaction is in the growing phase. The transaction acquires locks as needed. Once the transaction releases a lock, it enters the shrinking phase, and it can issue no more lock requests.
- The protocol ensures serializability. It can be proved that the transactions can be serialized in the order of their **lock points** (i.e., **the point where a transaction acquired its final lock - end of growing phase).**

The Two-Phase Locking Protocol

- Two-phase locking does not ensure freedom from deadlocks.

- Extensions to basic two-phase locking needed to ensure recoverability and freedom from cascading roll-back
  - **Strict two-phase locking**: a transaction must **hold all its exclusive** locks till it commits/aborts. Ensures recoverability and avoids cascading roll-backs
  - **Rigorous two-phase locking**: a transaction must **hold all locks** till commit/abort. Transactions can be serialized in the order in which they commit.

MySQL uses row-level locking for tables to support simultaneous write access by multiple sessions, making them suitable for multi-user, highly concurrent, and OLTP applications.

- Most databases implement rigorous two-phase locking, but refer to it as simply two-phase locking.

The Two-Phase Locking Protocol

	Strict two-phase locking	Rigorous two-phase locking
Lock Release	transaction must **hold all its exclusive** locks till it commits/aborts.	a transaction must hold all locks till commit/abort. Transactions can be serialized in the order in which they commit.
Deadlock Prevention	Can lead to potential deadlocks if not managed properly.	Reduces the chances of deadlocks by allowing lock release and reacquisition.
Resource Utilization	Can lead to underutilization of resources, as locks are held till the end.	Improves resource utilization by allowing other transactions to use released locks.
Transaction Starvation	Can lead to transaction starvation due to lock holding till the end.	Reduces the likelihood of transaction starvation by allowing lock reacquisition during execution.

Lock Conversions

- Two-phase locking protocol with lock conversions:
  - Growing Phase:
    - can acquire a lock-S on an item
    - Can acquire a lock-X on an item
    - can convert a lock-S to a lock-X (Shared to Exclusive) - **upgrade**
  - Shrinking Phase:
    - can release a lock-S
    - can release a lock-X
    - can convert a lock-X to a lock-S (Exclusive to Shared) - **downgrade**
- This protocol ensures serializability

Implementation of Locking

- A lock manager can be **implemented as a process** that receives messages from transactions and sends messages in reply.
- Transactions can **send lock and unlock requests as messages.**
- The lock **manager replies to a lock request by sending a lock grant messages** (or a message asking the transaction to roll back, in case of a deadlock)
- The requesting transaction waits until its request is answered.
- The **lock manager maintains an in-memory data-structure called a lock table** to record granted locks and pending requests. Each **record of the linked list** for a data item notes which transaction made the request, and what lock mode it requested. The record also notes if the request has currently been granted.

Lock Table

- Dark rectangles indicate **granted locks**, light colored ones indicate **waiting requests**
- **Lock table also records the type of lock granted or requested.**
- New request is added to the **end of the queue** of requests for the data item, and granted if it is compatible with all earlier locks
- Unlock requests result in the request being deleted, and later requests are checked to see if they can now be granted
- If transaction aborts, all waiting or granted requests of the transaction are **deleted.**

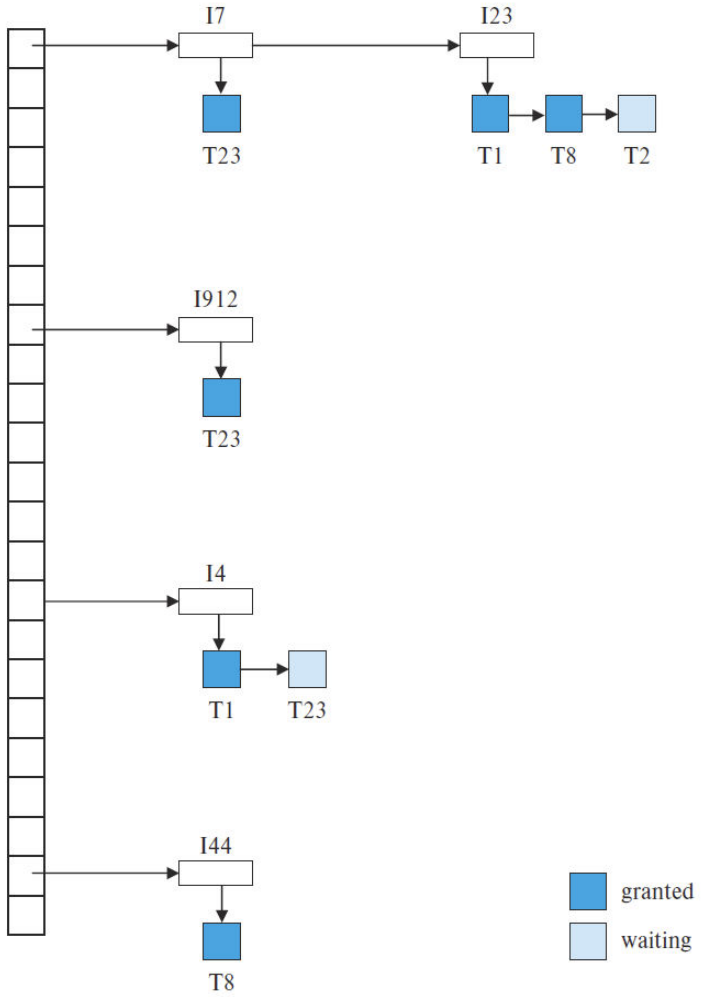


Figure 18.10 Lock table.

Lock Table

- **Lock manager may keep a list of locks held by each transaction**, to implement this efficiently.
- The table contains locks for five different data items, I4, I7, I23, I44, and I912.
- It can be seen in the fig. that T23 has been granted locks on I912 and I7 and is waiting for a lock on I4.
- **This algorithm guarantees freedom from starvation for lock requests, since a request can never be granted while a request received earlier is waiting to be granted**

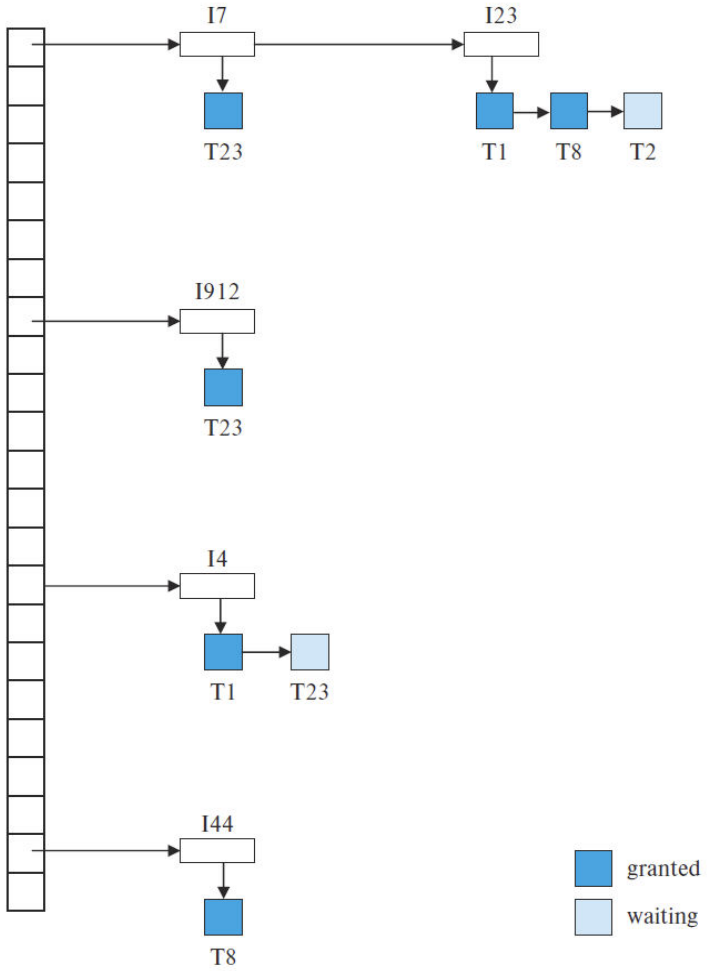


Figure 18.10 Lock table.

Q&A

1. What does it mean for two operations to conflict in a schedule?
(a) They belong to the same transaction.
(b) They access different data items.
(c) At least one of them is a read operation.
(d) They access the same item, and at least one is a write operation.

2. What defines a serializable schedule in database transactions?
(a) A schedule that allows only one transaction at a time.
(b) A non-serial schedule that is equivalent to some serial schedule.
(c) A schedule that does not preserve consistency.
(d) A schedule that leads to data inconsistency.

Q&A

3. In conflict serializability, what does it mean if the precedence graph of a schedule has a cycle?

(a)The schedule is conflict serializable.
(b)The schedule is serial.
(c)The schedule is not conflict serializable.
(d)The schedule is view serializable.

Q&A

1. What does it mean for two operations to conflict in a schedule?

(a) They belong to the same transaction.

(b) They access different data items.

(c) At least one of them is a read operation.

(d) They access the same item, and at least one is a write operation.

2. What defines a serializable schedule in database transactions?

(a) A schedule that allows only one transaction at a time.

(b) A non-serial schedule that is equivalent to some serial schedule.

(c) A schedule that does not preserve consistency.

(d) A schedule that leads to data inconsistency.

Q&A

3. In conflict serializability, what does it mean if the precedence graph of a schedule has a cycle?

(a)The schedule is conflict serializable.

(b)The schedule is serial.

(c)The schedule is not conflict serializable.

(d)The schedule is view serializable.

THANK YOU

Prof. Shilpa S

Department of Computer Science and Engineering

Database Management Systems

Introduction to NoSQL Systems

Prof. Shilpa S

Department of Computer Science and Engineering

NoSQL Databases

PES UNIVERSITY

- NoSQL databases ("**not only SQL**") are **non-tabular databases** and store data differently than relational tables.
- NoSQL databases come in a variety of types based on their data model. The main types are **document, key-value, wide-column, and graph.**
- NoSQL databases *emerged as the cost of storage dramatically decreased*.
- As storage costs rapidly decreased, *the amount of data that applications needed to store and query increased*. This data came in all shapes and sizes — **structured, semi-structured, and polymorphic** — and defining the schema in advance became nearly impossible.

NoSQL Databases

PES UNIVERSITY

- NoSQL databases allow developers to store **huge amounts of unstructured data, giving them a lot of flexibility.**
- Most NOSQL systems are **distributed databases or distributed storage systems, focusing on semi-structured data storage, high performance, availability, data replication, and scalability** instead of emphasizing immediate data consistency, powerful query languages, and structured data storage.

Emergence of NoSQL systems

- Many companies and organizations are faced with applications that store vast amounts of data.
- Consider a **free email application**, such as Google Mail or Yahoo Mail —this application can have millions of users, and each user can have thousands of email messages.
- There is a need for a storage system that can manage all these emails
- A **structured relational SQL system** may not be appropriate because:

  (1) SQL systems offer too many services (**powerful query language, concurrency control, etc**.), which this application may not need

  (2) A structured data model, such as the traditional relational model, may be too restrictive.
- Hence, a **NOSQL system with a dynamic schema and not many restrictions is required.**

Database Management Systems
Emergence of NoSQL systems

- Google developed a NOSQL system known as **BigTable**(column-based or wide column stores), which is used in many of Google's applications that require vast amounts of data storage, such as **Gmail, Google Maps, and Website indexing.**

- Apache HBase is an **open-source NoSQL system** based on similar concepts.

- Amazon developed a **NOSQL system called DynamoDB** (key-value data stores or key-object data stores) that is available through Amazon's cloud services.

- Facebook developed a **NOSQL system called Cassandra**, which is now open source and known as **Apache Cassandra**. This NOSQL system uses concepts from both key-value stores and column-based systems.

Emergence of NoSQL systems

- Other software companies started *developing own solutions and making them available to users who need these capabilities*—for example, MongoDB and CouchDB, which are classified as document-based NOSQL systems or document stores.

- Another category of **NOSQL systems is the graph-based NOSQL systems**, or graph databases; these include Neo4J and GraphBase.

Characteristics of NoSQL Systems

The characteristics of NoSQL systems can be divided into two categories—

- those related to **distributed databases and distributed systems**,
- those related to **data models and query languages**.

NOSQL Characteristics related to distributed databases and distributed systems

1. **Scalability:** In NOSQL systems, horizontal **scalability is employed while the system is operational**, so techniques for distributing the existing data among new nodes without interrupting system operation are necessary.
2. **Availability, Replication and Eventual Consistency:** Many applications that use NOSQL systems require continuous system availability. To accomplish this, **data is replicated over two or more nodes transparently, so that if one node fails, the data is still available on other nodes**. Replication improves data availability and can also improve **read performance**, because read requests can often be serviced from any of the replicated data nodes.

3. **Replication Models**: Two major replication models are used in NOSQL systems: **master-slave and master-master replication.** Master-slave replication requires one copy to be the master copy; all write operations must be applied to the master copy and then propagated to the slave copies, usually using eventual consistency (the slave copies will eventually be the same as the master copy).**The master-master replication allows reads and writes at any of the replicas but may not guarantee that reads at nodes that store different copies see the same values.**

4. **Sharding** : Sharding of the file records is **often employed in NOSQL systems**. This serves to distribute the **load of accessing the file records to multiple nodes**. The combination of sharding the file records and replicating the shards works in tandem to improve load balancing as well as data availability.

5. High-Performance Data Access: In many NOSQL applications, it is necessary to find individual records or objects (data items) from among the millions of data records or objects in a file. To achieve this, most systems use one of two techniques:

hashing or range partitioning on object keys.

- In hashing, a **hash function h(K) is applied to the key K**, and the location of the object with key K is determined by the **value of h(K).**
- In range partitioning, **the location is determined via a range of key values;**

  for example, location i would hold the objects whose key values K are in the range Ki min ≤ K ≤ Ki max.

Characteristics of NoSQL Systems

NOSQL characteristics related to data models and query languages.

1. **Not Requiring a Schema:** The flexibility of **not requiring a schema** is achieved in many NOSQL systems by allowing semi-structured, self-describing data.
2. **Less Powerful Query Languages**: Many applications that **use NOSQL systems may not require a powerful query language such as SQL, because search (read) queries in these systems often locate single objects in a single file based on their object keys**. NOSQL systems typically provide a set of functions and operations as a programming API, so reading and writing the data objects is accomplished by calling the appropriate operations by the programmer.
3. **Versioning**: Some NOSQL systems provide **storage of multiple versions of the data items**, with the timestamps of when the data version was created.

Database Management Systems

Categories of NoSQL systems

1. Document-based NOSQL systems: These systems store data in the form of documents using well-known formats, such as JSON (JavaScript Object Notation).

Examples - CouchDB, MongoDB

2. NOSQL key-value stores: These systems have a simple data model based on fast access by the key to the value associated with the key; the value can be a record or an object or a document or even have a more complex data structure.

Examples - DynamoDB, Redis

3. Column-based / wide column NOSQL systems: These systems partition a table by column into column families (a form of vertical partitioning), where each column family is stored in its own files.

Examples - Cassandra, HBase

4. Graph-based NOSQL systems: Data is represented as graphs, and related nodes can be found by traversing the edges using path expressions. Example: neo4j

Document-based NoSQL Systems

{
"student_id": 101,
"name": "Riya Sharma",
"course": "Computer Science",
"marks": { "DBMS": 90, "AI": 85 }
}

Column-based (Wide Column) NoSQL Systems

RowKey: 101
ColumnFamily: personal_info
name: Riya
email: riya@gmail.com
city: Bangalore

Key–Value Stores

Key: user102
Value: {"name": "Amit", "age": 24,
"city": "Pune"}

Graph-based NoSQL Systems

(A): { name: "Alice" }
(B): { name: "Bob" }
(A)-[FRIENDS_WITH]->(B)

Database Management Systems

Categories of NoSQL systems

Document databases

Store data elements in document-like structures that encode information in formats such as JSON.

+

Common uses include content management and monitoring Web and mobile applications.

+

EXAMPLES:
Couchbase Server, CouchDB, MarkLogic, MongoDB

Graph databases

Emphasize connections between data elements, storing related "nodes" in graphs to accelerate querying.

+

Common uses include recommendation engines and geospatial applications.

+

EXAMPLES:
Allegrograph, IBM Graph, Neo4j

Key-value databases

Use a simple data model that pairs a unique key and its associated value in storing data elements.

+

Common uses include storing clickstream data and application logs.

+

EXAMPLES:
Aerospike, DynamoDB, Redis, Riak

Wide column stores

Also called table-style databases—store data across tables that can have very large numbers of columns.

+

Common uses include Internet search and other large-scale Web applications.

+

EXAMPLES:
Accumulo, Cassandra, HBase, Hypertable, SimpleDB

Database Management Systems

Categories of NoSQL systems

SQL	NOSQL
Relational Database management system	Distributed Database management system
Vertically Scalable	Horizontally Scalable
Fixed or predifined Schema	Dynamic Schema
Not suitable for hierarchical data storage	Best suitable for hierarchical data storage
Can be used for complex queries	Not good for complex queries

CAP Theorem

- The CAP theorem, which was originally introduced as the CAP principle, can be used to explain some of the **competing requirements in a distributed system with replication.**

- The three letters in CAP refer to three desirable properties of distributed systems with replicated data: **consistency** (among replicated copies), **availability** (of the system for read and write operations) and **partition** tolerance (in the face of the nodes in the system being partitioned by a network fault).

CAP Theorem

- **Consistency** means that the **nodes will have the same copies of a replicated data item visible for various transactions.**
- **Availability** means that each **read or write request for a data item will either be processed successfully** or will receive a message that the operation cannot be completed.
- **Partition tolerance** means that the **system can continue operating if the network connecting the nodes has a fault** that results in two or more partitions, where the nodes in each partition can only communicate among each other.

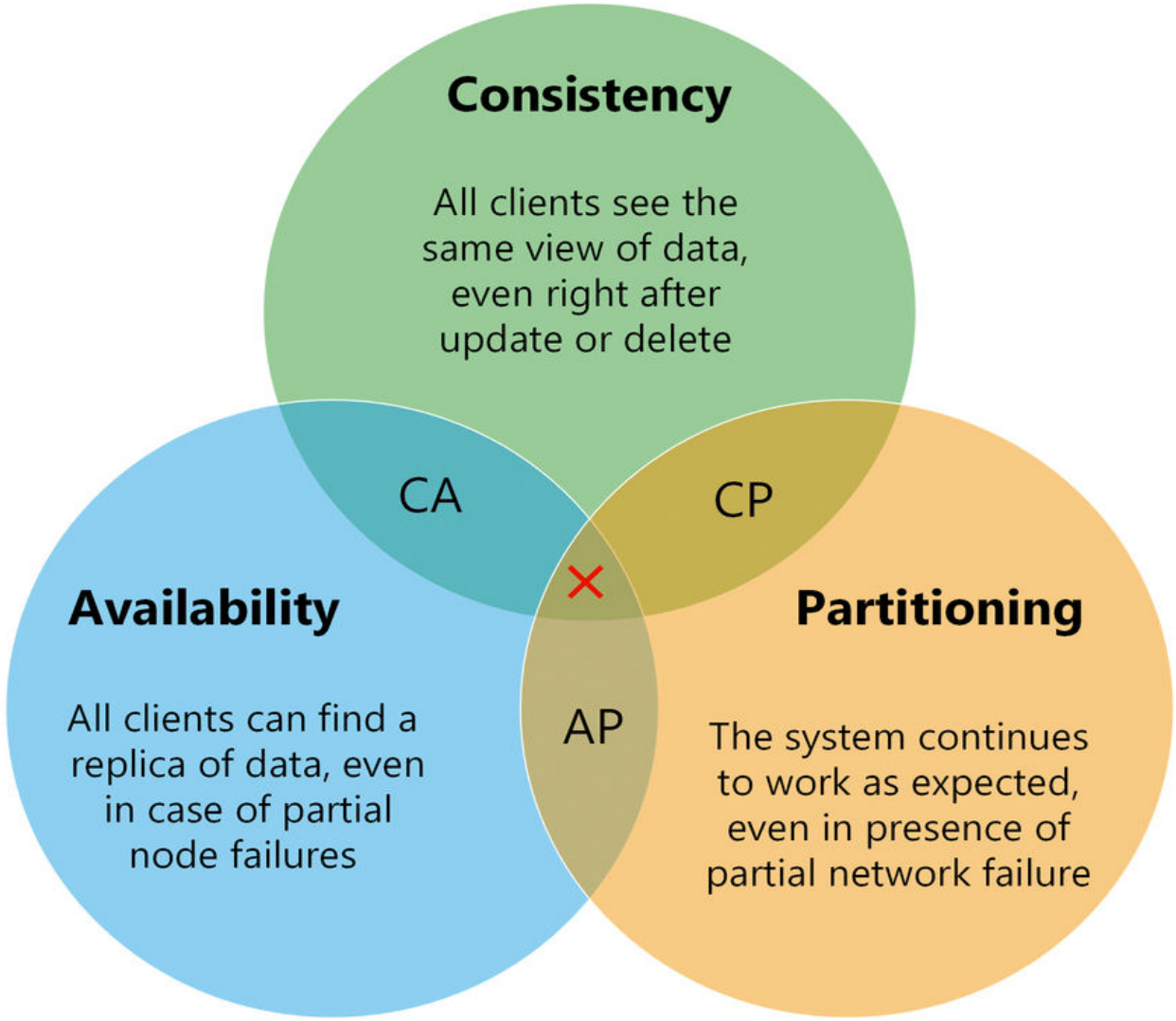
CAP Theorem

The CAP theorem states that:

> **"It is not possible to guarantee all three of the desirable properties—consistency, availability, and partition tolerance—at the same time in a distributed system with data replication".**

Database Management Systems

CAP Theorem



CAP Theorem

- If this is the case, **then the distributed system designer would have to choose two properties out of the three to guarantee.**
- It is generally assumed that in many traditional (SQL) applications, guaranteeing consistency through the ACID properties is important. On the other hand, in a NOSQL distributed data store, **a weaker consistency level is often acceptable**, and **guaranteeing the other two properties (availability, partition tolerance) is important.**
- Hence, **weaker consistency levels are often used in NOSQL system instead of guaranteeing serializability**. In particular, **a form of consistency known as eventual consistency is often adopted in NOSQL systems.**

Database Management Systems

Q&A

1. Which of the following is a key-value database?

(a) CouchDB (b) MongoDB (c) DynamoDB (d) neo4j

2. The A in CAP theorem stands for?

(a) Atomicity (b) Availability (c) Adaptability (d) Active

3. Consider the following statements:

(i) NOSQL systems allow a dynamic schema

(ii) Vertical Scalability is employed in NOSQL systems

(iii) Sharding distributes the load of accessing the file records to multiple nodes.

(iv) Neo4j is a column based NOSQL system

The correct statements are:

(a) (i),(iv) (b) (ii), (iii), (iv) (c) (i),(iii) (d) (ii),(iv)

Database Management Systems

Q&A

1. Which of the following is a key-value database?

(a) CouchDB (b) MongoDB **(c) DynamoDB** (d) neo4j

2. The A in CAP theorem stands for?

(a) Atomicity **(b) Availability** (c) Adaptability (d) Active

3. Consider the following statements:

(i) NOSQL systems allow a dynamic schema

(ii) Vertical Scalability is employed in NOSQL systems

(iii) Sharding distributes the load of accessing the file records to multiple nodes.

(iv) Neo4j is a column based NOSQL system

The correct statements are:

(a) (i),(iv) (b) (ii), (iii), (iv) (c) (ii),(iv) **(d) (i),(iii)**

THANK YOU

Prof. Shilpa S
Department of Computer Science and Engineering

shilpas@pes.edu

Database Management Systems

Key-Value Database(Redis)

Prof Shilpa S and Prof. Mahitha G
Department of Computer Science and Engineering

Acknowledgement: Teaching Assistant

NOSQL Key-Value Stores

PES UNIVERSITY
CELEBRATING 50 YEARS

These are the simplest type of NoSQL databases. Data is stored as a key-value pair.

- **Key:** represents a unique identifier (Like SRN) which is used to locate an item using this key
- **Value:** represents the data (Like Name**) itself which could be of various data formats**
- Works like a giant **dictionary / hash map** that is persisted on disk
- These are usually **schema-less** i.e. , there are no fixed columns or table structure.
- Have **high performance, scalability and flexibility**.
- Common use cases include **caching**, implementing **message queues using pub-sub (**Publisher–Subscriber**)** architecture, fetching preferences or using **session storage**.
- **Examples:** Redis, Amazon DynamoDB, CouchDB, Aerospike

PES UNIVERSITY
CELEBRATING 50 YEARS

Introduction and Features:

- Redis is an **open source, advanced key-value store**
- Used for building high performance, scalable web applications.
- **In memory:** Redis holds its database entirely **in the memory (RAM),** using the disk only for persistence.
- **Versatile Data Structures:** Redis has a relatively rich set of data types like **hashes, lists, sets, strings** etc.
- **Lua scripting support:** Redis has built-in support for **Lua scripting**, which allows developers to run custom code directly within the Redis server
- **Replication and Clustering:** Redis can scale horizontally with Redis Cluster (Clustering) and also can replicate data to any number of slaves (Uses a master slave architecture)

PES UNIVERSITY
CELEBRATING 50 YEARS

Use Cases:

- **Caching:** We can *cache frequently requested information to reduce the load on the main database* and boost an app's performance. **Ex:** Zomato's popular restaurants
- **Message broker:** Redis can act as a middleman, helping applications exchange data through the asynchronous pub/sub architecture. **Ex:** Roblox notification system
- **Streaming data processing and real-time analytics:** Redis can be used for monitoring **website traffic, tracking user activity, analyzing financial transactions**. **Ex:** Xignite
- **Session storage:** Things like login status and shopping cart info can be stored for a user to **prevent re-login on refreshing the page**. **Ex:** Amazon shopping cart
- **Game leaderboard:** Redis can store the player's profile and his score which can help in calculating the leaderboard in real time. **Ex:** HackerRank Leaderboard system

Database Management Systems

```
SET popular:bangalore '[{"id":501, "name":"Empire"}, {"id":823,
"name":"Truffles"}]'
SET popular:mumbai '[{"id":201, "name":"Barbeque Nation"}, {"id":334,
"name":"The Table"}]'
EXPIRE popular:bangalore 300  # expires every 5 minutes
```

User App → API Server → Redis Cache → (if not found) → Main DB (MySQL/PostgreSQL)

Roblox:
•receive instant pop-up notifications like
"Your friend joined the game!"
"New message received!"
"Achievement unlocked!"
These notifications must appear **instantly** — across **millions of active players** worldwide — with **no delays or duplication**.
That's where **Redis** becomes critical.
Redis provides **lightweight, real-time message delivery** using its:
•**Pub/Sub system** for instant broadcasting
•**Streams** (advanced queues) for persistent, ordered event delivery

Xignite is a **financial data provider** that delivers:
•Stock prices
•Market indices
•Forex rates
•Commodities data
•And other market analytics
It serves **banks, fintech platforms, and trading apps** (like Robinhood, Wealthfront, and Nasdaq services).
These applications require **real-time**, **low-latency**, and **high-availability** access to data.

Xignite uses Redis as a high-performance in-memory caching and data delivery layer

Limitations:

- **RAM is expensive:** Redis for large datasets can become **expensive**, especially when scaling up since RAM is significantly more expensive than disk storage.
- **Manual memory management:** Redis does not automatically manage memory like relational databases do. Developers must manually configure eviction policies to decide what happens when memory is full
- **Potential data loss:** Redis keeps everything in RAM, meaning that if the server unexpectedly crashes, all the data may be **lost**.
- **Not suitable for complex queries:** Redis is optimized for key-value lookups and simple operations. It's **not ideal for complex joins, aggregations, or other advanced database queries.**

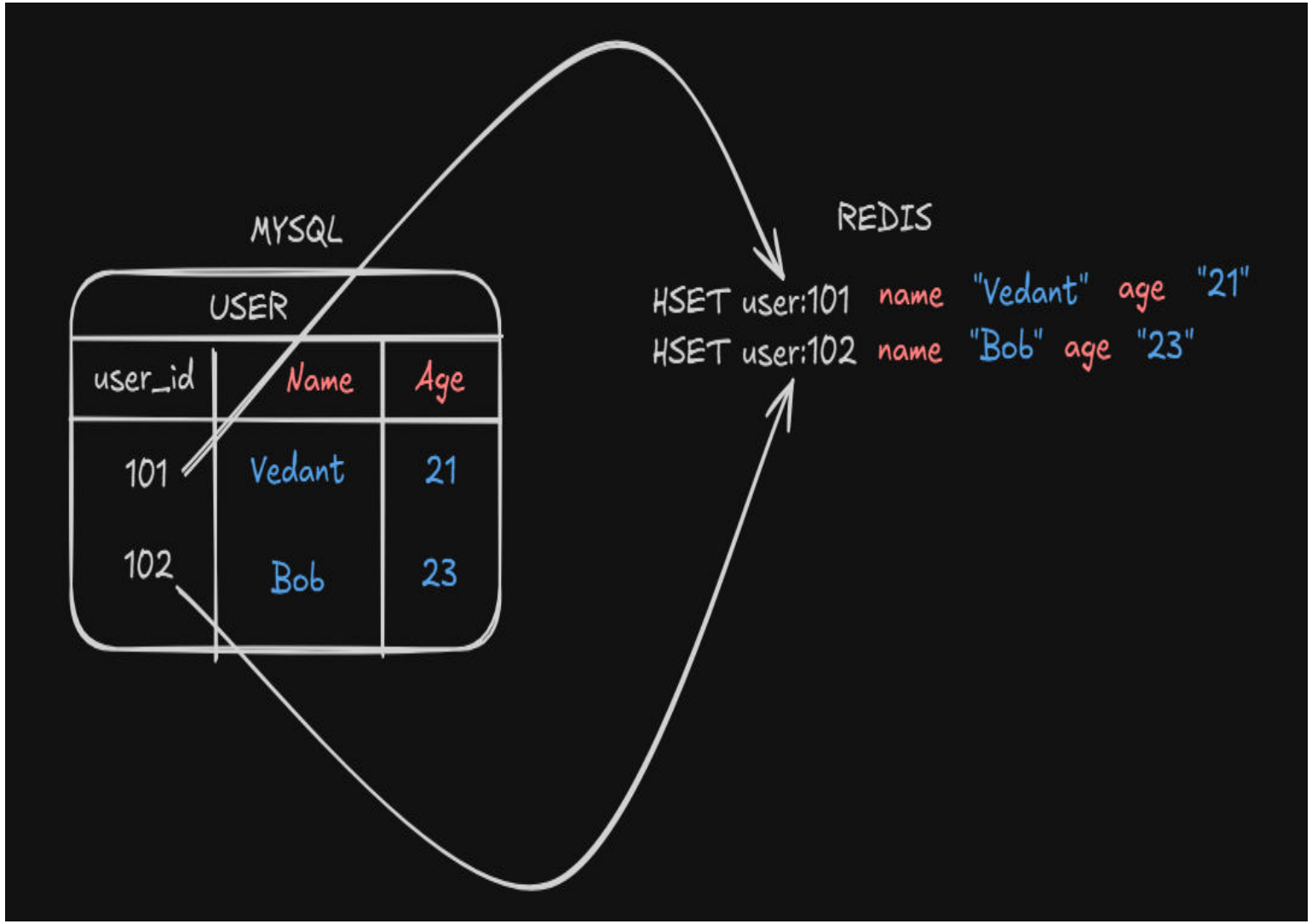
Basics of Redis:

- Redis uses key value pairs. **Each key is a string type** (It's binary safe so it can hold bytes not just text)
- Each value can be of **different data structures** (strings, hashes, sets explained further ahead)
- Each **key should be unique in** Redis and : (colon) is used to show hierarchy (Similar to namespaces concept in OS) A namespace in an Operating System provides isolation by giving each process (or group of processes) its own view of certain system resources — such as process IDs, file systems, users, or networks.
- If we want to store all information of PES in one place then we can create a key called **PES** only
- If we want to store all departments under PES university then we can create a key called **PES:Department where PES:Department is a single key**
- Please note that by default all values in the key-value pair of redis is **strings**

Redis DB

Basics of Redis:

- In the example we have a user table which stores user_id, name and age.
- HSET is a command to store values in Hashes and here key is **user:user_id (user:101, user:102)**
- We need to have unique keys in Redis and to show it belongs to user table, hence we use this format
- *For every user id we can store the name and age fields in Redis* just like MySQL (Note the color coding for the field and value)



PES UNIVERSITY
CELEBRATING 50 YEARS

Key Management Commands in Redis:

- Along with creating namespaces using colon (:), developers need to set a lifecycle of keys
- This means that **each key could have a duration to persist or expire**
- This helps in **organization, session management and caching**
- The table shows a few commands to help you understand better

Command	Description	Illustrative Query Example
EXPIRE key seconds	Sets a timeout on a key in seconds. The key is deleted after the timeout.	EXPIRE user:session:123 3600
TTL key	Returns the remaining time to live of a key in seconds.	TTL user:session:123
PERSIST key	Removes the timeout on a key, making it persistent.	PERSIST user:session:123
DEL key [key...]	Deletes one or more specified keys.	DEL user:session:123 temp:data:abc

Data Structures in Redis:

Data structures are essential to store different formats of data in your keys.

We cover 5 of the most popular data structures in Redis in this course but feel free to explore more!

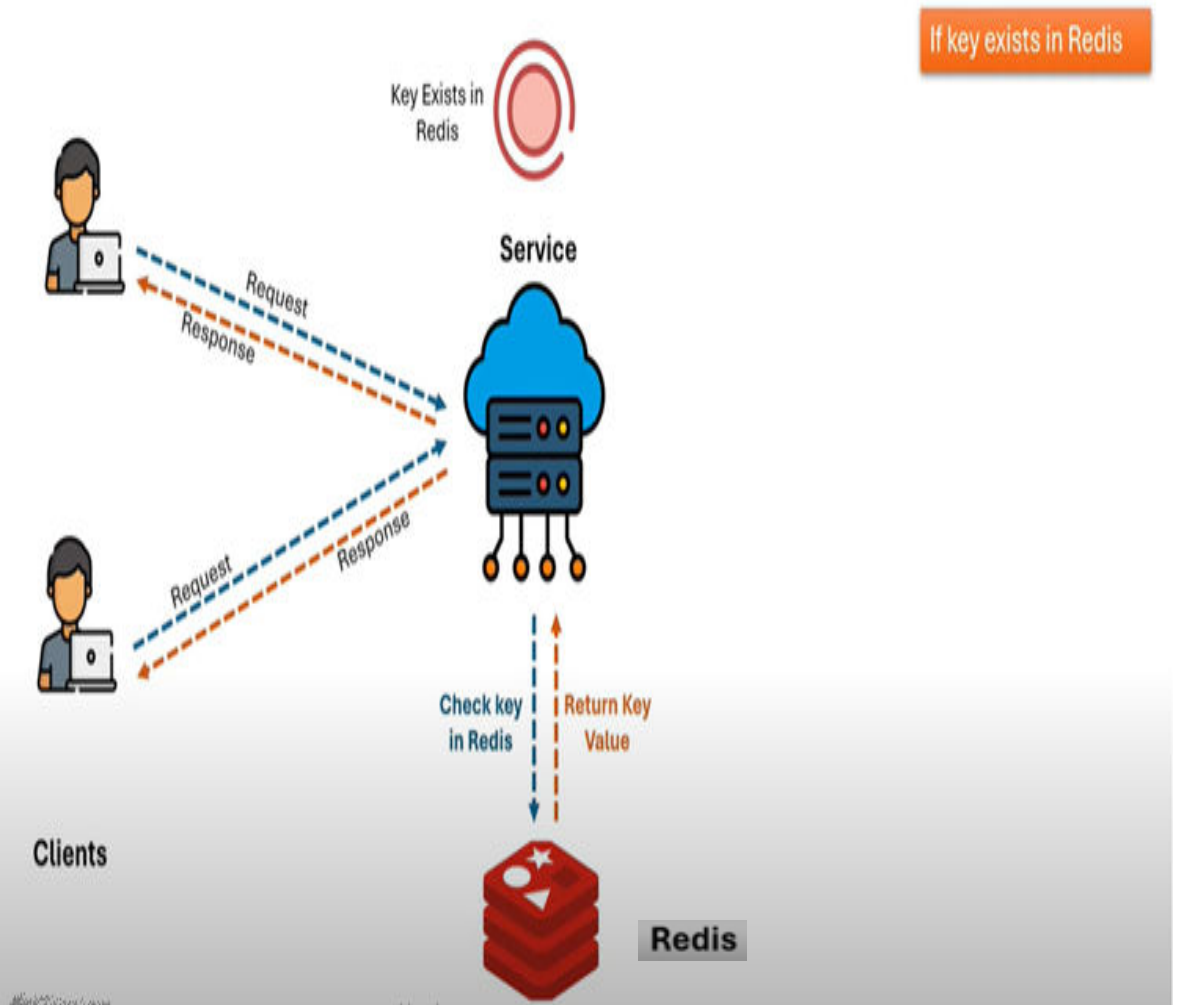
1. **Strings:**
   - It is a binary sequence of bytes (characters). Max size is 512MB
   - **Use Case:** CRUD Operations, Caching, Atomic Counters
2. **Lists:**
   - Lists are sequences of strings that are built upon Linked Lists.
   - **Use Case:** Task Lists (Stacks and queues), Logging
3. **Hashes:**
   - Similar to your dictionaries in Python or JSON format. Map a key-value pair
   - **Use Case:** Store structured data and mimic JSON format

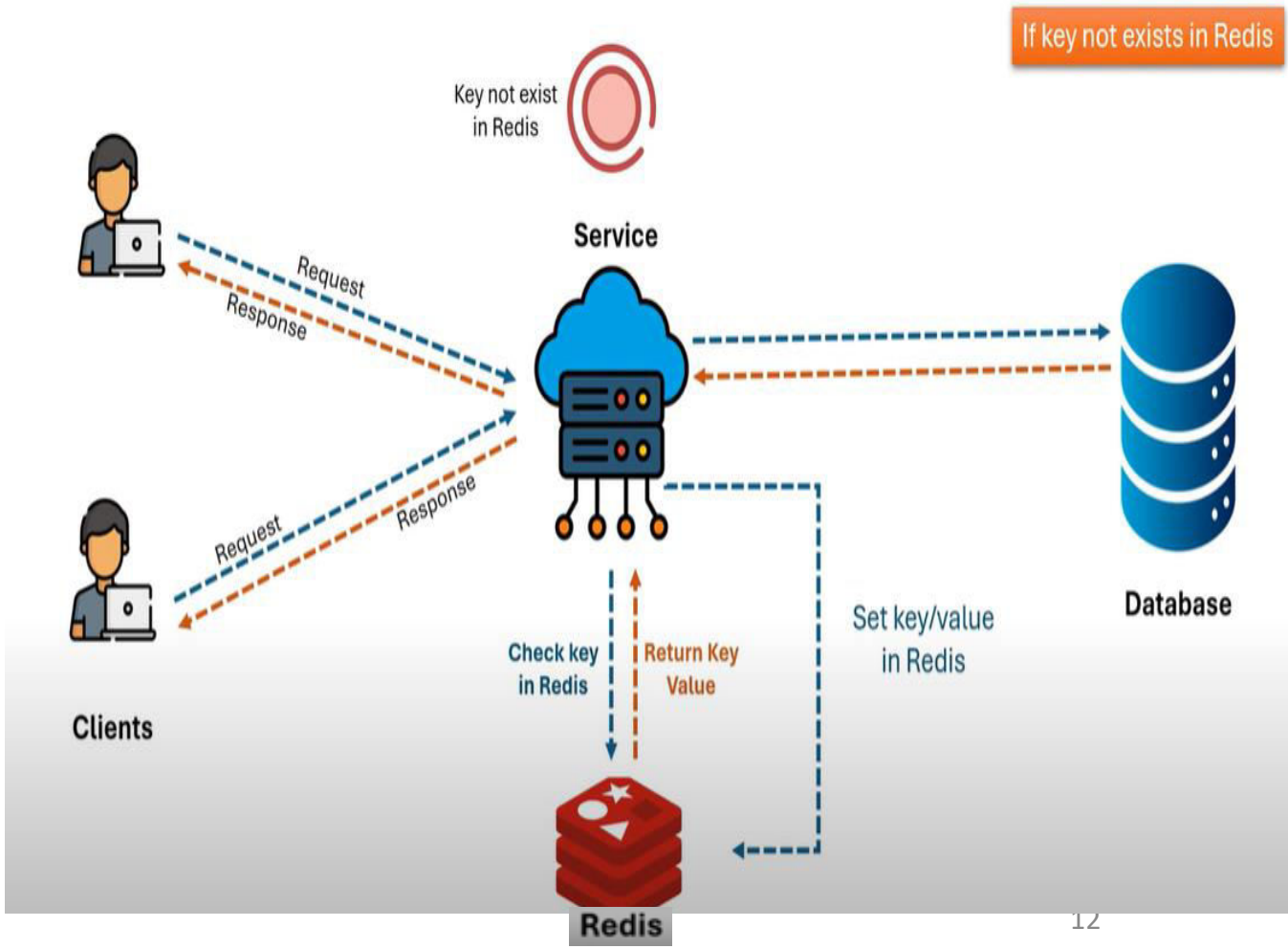
PES UNIVERSITY

Data Structures in Redis:

4. **Sets:**
   - Collection of **unique** strings where order is not preserved. (Similar to Sets in python)
   - **Use Case:** Tracking unique items, Tagging
5. **Sorted Sets (Zsets):**
   - Data structure that combines the properties of a Set and a Scored collection (Similar to priority queues)
   - Every member stored in the Zset has a score associated and members are sorted based on score (in case of a tie of score lexicographic sorting of members is done)
   - **Use Case:** Leaderboard calculation, Ranking mechanism

How Redis Cache Works





PES UNIVERSITY
CELEBRATING 50 YEARS

String Commands in Redis:

Command	Description	Example
SET key value [EX seconds]	Sets the string value of a key, **optionally** with an expiration.	SET user:1:name "Alice" EX 3600
GET key	Gets the value of a key.	GET user:1:name
MSET key1 val1 key2 val2	Sets multiple key-value pairs in one atomic operation.	MSET user:1:name "Alice" user:2:name "Bob"
MGET key1 key2	Gets the values of all specified keys.	MGET user:1:name user:2:name
INCR / DECR key	Increments / Decrements the integer value of a key by one.	INCR page:views
INCRBY / DECRBY key decrement	Increments / Decrements the integer value of a key by a given number.	DECRBY items:in_stock 5
APPEND key value	Appends a value to a key.	APPEND log:file “Logged in"

PES UNIVERSITY
CELEBRATING 50 YEARS

List Commands in Redis:

Command	Description	Example
LPUSH key element [element...]	Prepends one or more elements to the head of a list. (Add to start of list)	LPUSH tasks “Begin task"
RPUSH key element [element...]	Appends one or more elements to the tail of a list. (Add to end of list)	RPUSH logs "User A logged in"
LPOP key	Removes and returns the first element of a list.	LPOP tasks
RPOP key	Removes and returns the last element of a list.	RPOP logs
LRANGE key start stop	Gets a range of elements from a list. (Like splicing in python, index starts at 0)	LRANGE logs 0 99 (Includes both index 0 and 99)
LLEN key	Gets the length of a list.	LLEN tasks
BLPOP key [key...] timeout	Blocking version of LPOP. Waits for an element to become available.	BLPOP tasks_high_priority tasks_low_priority 0
LTRIM key start stop	Trims a list to the specified range.	LTRIM logs 0 4 (Includes both index 0 and 4)

PES UNIVERSITY
CELEBRATING 50 YEARS

Hashes Commands in Redis:

Command	Description	Example
HSET key field value [field value...]	Sets the string value of one or more hash fields.	HSET user:1 name "Alice" age 30
HGET key field	Gets the value of a hash field.	HGET user:1 name
HGETALL key	Gets all the fields and values in a hash.	HGETALL user:1
HDEL key field [field...]	Deletes one or more hash fields.	HDEL user:1 temp_token
HINCRBY key field increment	Increments the integer value of a hash field by a given number.	HINCRBY user:1 login_count 1
HLEN key	Gets the number of fields in a hash.	HLEN user:1
HEXISTS key field	Determines if a hash field exists.	HEXISTS user:1 email (1 if present else 0)

PES UNIVERSITY
CELEBRATING 50 YEARS

Set Commands in Redis:

Command	Description	Example
SADD key member [member...]	Adds one or more members to a set.	SADD article:123 "redis" "database"
SREM key member [member...]	Removes one or more members from a set.	SREM article:123 "old_tag"
SMEMBERS key	Returns all the members of the set.	SMEMBERS article:123
SCARD key	Gets the number of members in a set.	SCARD article:123
SISMEMBER key member	Determines if a given value is a member of a set.	SISMEMBER article:123 "redis“ (1 if present else 0)
SINTER key [key...]	Returns the intersection of multiple sets.	SINTER friends:alice friends:bob
SUNION key [key...]	Returns the union of multiple sets.	SUNION tech_tags devops_tags
SDIFF key [key...]	Returns the difference between the first set and subsequent sets.	SDIFF group:a group:b

PES UNIVERSITY
CELEBRATING 50 YEARS

Sorted Sets (Zsets) Commands in Redis:

Command	Description	Example
ZADD key score member [score member...]	Adds or updates one or more members in a sorted set with their scores.	ZADD leaderboard 1550 "player1"
ZINCRBY key increment member	Increments the score of a member by a given amount.	ZINCRBY leaderboard 50 "player1"
ZRANGE key start stop	Returns a range of members by rank, ordered by score (ascending).	ZRANGE leaderboard 0 9 WITHSCORES (WITHSCORES gives the scores as well)
ZREVRANGE key start stop	Returns a range of members by rank, ordered by score (descending).	ZREVRANGE leaderboard 0 9 (All ranges include both start and stop)
ZRANK key member	Returns the rank of a member (ascending order).	ZRANK leaderboard "player1"
ZREM key member [member...]	Removes one or more members from a sorted set.	ZREM leaderboard "player1"
ZCARD key	Gets the number of members in a sorted set.	ZCARD leaderboard

References

For architecture and theory of redis:

- https://architecturenotes.co/p/redis
- https://blog.bytebytego.com/p/a-crash-course-in-redis
- https://backendless.com/redis-what-it-is-what-it-does-and-why-you-should-care/
- https://www.altexsoft.com/blog/redis-pros-and-cons/

For data structures and hands on:

- https://redis.io/docs/latest/develop/data-types/
- https://www.geeksforgeeks.org/system-design/complete-guide-on-redis-data-types-with-commands-and-storage/
- https://www.tutorialspoint.com/redis/redis_data_types.htm

THANK YOU

Prof. Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Key-Value Database(Redis)

Prof. Mahitha G
Department of Computer Science and Engineering

Acknowledgments: Prof. Shilpa S and Teaching Assistant

PES UNIVERSITY
CELEBRATING 50 YEARS

String Commands Demo in Redis:

Case Study: User Profile & Session Management

- You are building a new feature for a rapidly growing social media application.
- To ensure a **fast and responsive user experience, you've decided to use Redis to cache user profile information and manage temporary user data.**
- Your user data model is simple. Each user has a unique ID (e.g., user:1, user:2) [Hint: These are the names of your keys]
- You need to store their name and occasionally other attributes like height.

Based on the case study, write commands based on these scenarios:

1. At the start of your application deployment, you need to set two user names at once (Vedant and Bob was used for demo you can use anything).
2. Fetch multiple users at once (the two users you created right now).
3. A new user, Alice, has just signed up. You want to store her name and set a 60-second expiration on the key since she is just a temporary user.
4. User Bob (user:2) has decided to add his last name, "Ellison," to his profile. How can you append "Ellison" to his existing name "Bob" to form "BobEllison"?
5. User:1 wants to add his height as “120” com.
6. How would you retrieve the height for user:1 ?
7. During the poll, user:1 realizes they made a mistake and their height is actually 10cm taller. Increment their stored height by 10 (without deleting and recreating the user)

String Commands Demo in Redis:

1. MSET user:1:name "Vedant" user:2:name "Bob" => To **create multiple** user names
2. MGET user:1:name user:2:name => To **fetch multiple** user names
3. SET user:3:name "Alice" EX 60 => **Create** user that **lasts** only for **60 seconds**
4. APPEND user:2:name "Ellison" => **Append** Ellison to user 2's name (BobEllison)
5. SET user:1:height "120" => **Create** user 1's height
6. GET user:1:height => **Fetch** user 1's height
7. INCRBY user:1:height 10 => **Increase** user 1's height by 10

```
 vedant@vedant  ~  redis-cli
127.0.0.1:6379> PING
PONG
127.0.0.1:6379> MSET user:1:name "Vedant" user:2:name "Bob"
OK
127.0.0.1:6379> MGET user:1:name user:2:name
1) "Vedant"
2) "Bob"
127.0.0.1:6379> SET user:3:name "Alice" EX 60
OK
127.0.0.1:6379> MGET user:1:name user:2:name user:3:name
1) "Vedant"
2) "Bob"
3) "Alice"
127.0.0.1:6379> MGET user:1:name user:2:name user:3:name
1) "Vedant"
2) "Bob"
3) (nil)
127.0.0.1:6379> APPEND user:2:name "Ellison"
(integer) 10
127.0.0.1:6379> MGET user:1:name user:2:name user:3:name
1) "Vedant"
2) "BobEllison"
3) (nil)
127.0.0.1:6379> SET user:1:height "120"
OK
127.0.0.1:6379> GET user:1:height
"120"
127.0.0.1:6379> INCRBY user:1:height 10
(integer) 130
127.0.0.1:6379> GET user:1:height
"130"
127.0.0.1:6379>
```

List Commands Demo in Redis:

Case Study: To-Do List Management

- You are **developing a simple and fast to-do list application**.
- To **manage the tasks for each user**, you've chosen to use Redis Lists.
- This allows you to easily add tasks to the beginning or end of the list, view all tasks, and remove them as they are completed.
- For this case study, we will manage a single list with the key **todolist**.
- New tasks with normal priority should be added to the end of the list, while high-priority tasks should be added to the beginning.

Redis DB

Based on the case study, write commands based on these scenarios:

1. Start your day by adding the **task "Do laundry"** to your to-do list
2. Check if this task has been added to the to-do list by viewing the entire todolist
3. Add an urgent task: "Call Dad“ to the todolist at the top of your list.
4. Check how many tasks you have left in your todolist
5. You’ve completed your most urgent task and you want to remove that from the list (task at the top of the list)
6. You’ve completed the least urgent task and you want to remove that from the list (task at the bottom of the list)
7. As you are heading out, you remember you also need to "Buy groceries“. (task at the bottom of the list)
8. Feeling a bit overwhelmed, you decide to focus only on the most important task remaining. How can you trim the list so that it only contains 1 element?

List Commands Demo in Redis:

1. RPUSH todolist "Do laundry" => To **add** event from **right**
2. LRANGE todolist 0 -1 => To **fetch** the todo list from 0 index (start index) to -1 index (last index) [**both inclusive**] [Works like python where -1,-2,-3…]
3. LPUSH todolist "Call Dad" => To **add** event from **left**
4. LLEN todolist => To **fetch length** of list
5. LPOP todolist => To **delete** first element from **left**
6. RPOP todolist => To **delete** first element from **right**
7. RPUSH todolist "Buy groceries" => To **add** event from **right**
8. LTRIM todolist 0 0 => To **trim** the list to contain only elements **within** the **range** (Only 1st task i.e. 0^{th} index)

```
 vedant@vedant  ~  redis-cli
127.0.0.1:6379> PING
PONG
127.0.0.1:6379> RPUSH todolist "Do laundry"
(integer) 1
127.0.0.1:6379> RPUSH todolist "Buy groceries"
(integer) 2
127.0.0.1:6379> LRANGE todolist 0 -1
1) "Do laundry"
2) "Buy groceries"
127.0.0.1:6379> LPUSH todolist "Call Dad"
(integer) 3
127.0.0.1:6379> LRANGE todolist 0 -1
1) "Call Dad"
2) "Do laundry"
3) "Buy groceries"
127.0.0.1:6379> LLEN todolist
(integer) 3
127.0.0.1:6379> LPOP todolist
"Call Dad"
127.0.0.1:6379> LRANGE todolist 0 -1
1) "Do laundry"
2) "Buy groceries"
127.0.0.1:6379> RPOP todolist
"Buy groceries"
127.0.0.1:6379> LRANGE todolist 0 -1
1) "Do laundry"
127.0.0.1:6379> RPUSH todolist "Buy groceries"
(integer) 2
127.0.0.1:6379> LRANGE todolist 0 -1
1) "Do laundry"
2) "Buy groceries"
127.0.0.1:6379> LTRIM todolist 0 0
OK
127.0.0.1:6379> LRANGE todolist 0 -1
1) "Do laundry"
127.0.0.1:6379>
```

Hashes Commands Demo in Redis:

Case Study: Student Management

- Your application **needs to store detailed student profiles with fields like name and age**
- A Redis Hash is the perfect data structure for this, as it allows you to store multiple field-value pairs under a single key
- For this scenario, you will manage user profiles where each user is identified by a key (e.g., user:1)
- Their profile will contain fields like name and age.
- This is similar to your JSON format

Redis DB

Based on the case study, write commands based on these scenarios:

1. Create a new user with his name and age stored (Vedant and 21 are the values used in demo)
2. Fetch the new user’s name. (only name age should not be displayed)
3. You realized name is not enough so you want to fetch all details for this user (fetch field and its values)
4. Create a new user with his name and age stored (Bob and 23 are the values used in demo)
5. It's been two years since Bob signed up, and you need to update his age accordingly by 2 years
6. For analytics, you want to count how many data fields are stored for user:2. What command returns the number of fields in the hash?
7. Bob has requested that his age be removed from his profile due to privacy concerns. How would you delete the age field for user:2?
8. Before displaying the user's age on their profile page, your application needs to verify if the age field actually exists for user:2. What command would you use to check for its existence?

Hashes Commands Demo in Redis:

1. HSET user:1 name "Vedant" age "21" => To **add** user 1 with name and age fields
2. HGET user:1 name => To **fetch** user 1's name
3. HGETALL user:1 => To **fetch** user 1's all details (name and age field)
4. HSET user:2 name "Bob" age "23" => To **add** user 2 with name and age fields
5. HINCRBY user:2 age 2 => To **increase** user 2's age by 2
6. HLEN user:2 => To **fetch** number of fields in user 2 (2 since name and field)
7. HDEL user:2 age => To **delete** user 2's age
8. HEXISTS user:2 age => To **check** if user 2 **has** age **field**

```
 vedant@vedant  ~  redis-cli
127.0.0.1:6379> PING
PONG
127.0.0.1:6379> HSET user:1 name "Vedant" age "21"
(integer) 2
127.0.0.1:6379> HGET user:1 name
"Vedant"
127.0.0.1:6379> HGETALL user:1
1) "name"
2) "Vedant"
3) "age"
4) "21"
127.0.0.1:6379> HSET user:2 name "Bob" age "23"
(integer) 2
127.0.0.1:6379> HGETALL user:2
1) "name"
2) "Bob"
3) "age"
4) "23"
127.0.0.1:6379> HINCRBY user:2 age 2
(integer) 25
127.0.0.1:6379> HGET user:2 age
"25"
127.0.0.1:6379> HLEN user:2
(integer) 2
127.0.0.1:6379> HDEL user:2 age
(integer) 1
127.0.0.1:6379> HEXISTS user:2 age
(integer) 0
127.0.0.1:6379> HGETALL user:2
1) "name"
2) "Bob"
127.0.0.1:6379>
```

Set Commands Demo in Redis:

Case Study: Event RSVP Management

- You are organizing an event and need an efficient way to manage the list of attendees who have RSVP'd
- A Redis Set is the ideal tool for this because it automatically handles uniqueness
- This ensures that a user cannot RSVP more than once.
- You will use a Redis Set with the key rsvp to store the names of all attendees.

Redis DB

Based on the case study, write commands based on these scenarios:

1. Create 3 new users at once (“User A”, “User B” and “User C” used in demo)
2. Fetch all the users that have RSVP’d so far
3. Two more people respond to the invitation: "User A" (who had already RSVP'd) and a new attendee, "User D". How do you add them to the list, ensuring no duplicates are created?
4. Before sending a final confirmation, check if "User D" is officially on the list.
5. For catering purposes, fetch the exact headcount of how many attendees have RSVP’d.
6. Unfortunately, "User D" has to cancel. How do you remove them from the RSVP list?

Set Commands Demo in Redis:

1. SADD rsvp "User A" "User B" "User C" => To **add multiple** user names to the set
2. SMEMBERS rsvp => To **fetch all** users in the set
3. SADD rsvp "User A" "User D" => To **add** user names to the set (removes duplicates automatically)
4. SISMEMBER rsvp "User D" => To **check** if user D is **present** in set (1 if present else 0)
5. SCARD rsvp => To **fetch** length of set
6. SREM rsvp "User D" => To **remove** user D from set

```
 vedant@vedant  ~  redis-cli
127.0.0.1:6379> PING
PONG
127.0.0.1:6379> SADD rsvp "User A" "User B" "User C"
(integer) 3
127.0.0.1:6379> SMEMBERS rsvp
1) "User A"
2) "User B"
3) "User C"
127.0.0.1:6379> SADD rsvp "User A" "User D"
(integer) 1
127.0.0.1:6379> SMEMBERS rsvp
1) "User A"
2) "User B"
3) "User C"
4) "User D"
127.0.0.1:6379> SISMEMBER rsvp "User D"
(integer) 1
127.0.0.1:6379> SISMEMBER rsvp "User E"
(integer) 0
127.0.0.1:6379> SCARD rsvp
(integer) 4
127.0.0.1:6379> SREM rsvp "User D"
(integer) 1
127.0.0.1:6379> SMEMBERS rsvp
1) "User A"
2) "User B"
3) "User C"
127.0.0.1:6379>
```

Sorted Sets (Zset) Commands Demo in Redis:

Case Study: Gaming Leaderboard

- You are building **a real-time leaderboard for a popular online game**
- A Redis Sorted Set is the perfect data structure for this task because it maintains a collection of unique members (players) ordered by an associated score.
- For this case study, you will manage a leaderboard with the key **leaderboard**.

Redis DB

Based on the case study, write commands based on these scenarios:

1. Add 3 players to the leaderboard (Used (player1,100), (player2, 200), (player3,300) for demo)
2. Display the total number of players currently on the leaderboard.
3. Display the leaderboard on the game's website, showing player names ranked from the highest score to the lowest (descending order)
4. For a more detailed view, display the leaderboard with both the player names and their corresponding scores, again, from highest to lowest. (descending order)
5. “Player1” just had a great round and scored 500 more points. Update his score
6. To understand their position relative to the bottom, you need to find the rank of "player1" based on the lowest score (a 0-indexed rank from lowest to highest).
7. More importantly, you want to find the official rank of "player1" from the top of the leaderboard (a 0-indexed rank from highest to lowest).
8. “Player1“ has been found cheating. Remove him from the leaderboard

Sorted Set (Zset) Commands Demo in Redis:

1. ZADD leaderboard 100 "player1" 200 "player2" 300 "player3" => To **add multiple** player names **along with their scores** (scores first then name)
2. ZCARD leaderboard => To **fetch** number of players in the set
3. ZREVRANGE leaderboard 0 -1 => To **fetch** player names in **desc order within range** (start and stop index both inclusive)
4. ZREVRANGE leaderboard 0 -1 WITHSCORES => To **fetch** player names **with scores** in **desc order within range** (start and stop index both inclusive)
5. ZINCRBY leaderboard 500 "player1" => To **increase** player 1's score by 500
6. ZRANK leaderboard "player1" => To **fetch rank** of player 1 (Rank is done from **lowest to highest** so **last** index has **highest** score)
7. ZREVRANK leaderboard "player1" => To **fetch rank** of player 1 (Rank is done from **highest to lowest** so **first** index has **highest** score)
8. ZREM leaderboard "player1" => To **remove** player 1 from the set

```
 vedant@vedant  ~  redis-cli
127.0.0.1:6379> PING
PONG
127.0.0.1:6379> ZADD leaderboard 100 "player1" 200 "player2" 300 "player3"
(integer) 3
127.0.0.1:6379> ZCARD leaderboard
(integer) 3
127.0.0.1:6379> ZREVRANGE leaderboard 0 -1
1) "player3"
2) "player2"
3) "player1"
127.0.0.1:6379> ZREVRANGE leaderboard 0 -1 WITHSCORES
1) "player3"
2) "300"
3) "player2"
4) "200"
5) "player1"
6) "100"
127.0.0.1:6379> ZINCRBY leaderboard 500 "player1"
"600"
127.0.0.1:6379> ZREVRANGE leaderboard 0 -1 WITHSCORES
1) "player1"
2) "600"
3) "player3"
4) "300"
5) "player2"
6) "200"
127.0.0.1:6379> ZRANK leaderboard "player1"
(integer) 2
127.0.0.1:6379> ZREVRANK leaderboard "player1"
(integer) 0
127.0.0.1:6379> ZREM leaderboard "player1"
(integer) 1
127.0.0.1:6379> ZREVRANGE leaderboard 0 -1 WITHSCORES
1) "player3"
2) "300"
3) "player2"
4) "200"
127.0.0.1:6379>
```

THANK YOU

Prof. Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Graph Databases(Neo4j)

Prof. Shilpa S
Department of Computer Science and Engineering

PES UNIVERSITY
CELEBRATING 50 YEARS

What is Neo4j?

- Another category of NoSQL systems is known as **graph databases** (or graph-oriented NoSQL systems).
- Data is represented as a graph → **a collection of nodes (vertices) and edges.**
- Nodes represent entities (e.g., Person, Product).
- Edges represent relationships between nodes (e.g., FRIEND_OF, PURCHASED).
- Many systems fall under graph databases, but our focus is on Neo4j.
- Neo4j is an open-source graph database, implemented in Java.

PES UNIVERSITY
CELEBRATING 50 YEARS

Why is Neo4j Used?

- Traditional relational DBs struggle with complex relationships (too many JOINs).
- Neo4j is optimized for connected data and relationship-heavy queries.
- Supports fast traversals by following direct links instead of rebuilding joins.
- Allows intuitive modeling of real-world networks as graphs.
- Flexible schema → can easily add new nodes, edges, or properties.
- Supports Cypher Query Language (CQL) for simple, readable pattern matching.
- Built-in graph algorithms → shortest path, PageRank, community detection.

PES UNIVERSITY
CELEBRATING 50 YEARS

Real-Life Applications of Neo4j?

- Social Networks (Facebook, LinkedIn): friend suggestions, activity feeds.
- E-commerce/Streaming (Amazon, Netflix): recommendation systems.
- Banking & Finance (Mastercard): fraud detection by analyzing patterns.
- Knowledge Graphs (Google, NASA): link and search across vast datasets.
- IT & Network Management: track dependencies across systems and apps.

Database Management Systems

Neo4j Data Model

- Neo4j organizes data using the concepts of **nodes** and **relationships**.
- Both nodes and relationships can have **properties**, which store the data items associated with nodes and relationships.
- **Nodes** can have **labels** (≥0), and the ones that have the **same label** are grouped into a **collection** that identifies a subset of the nodes in the database graph **for querying purposes**.
- **Relationships** are **directed**; each relationship has a **start node** and **end node** as well as a **relationship type**, which serves a similar role to a node label by identifying similar relationships that have the same relationship type.
- **Properties** can be specified via a **map pattern**, which is made of one or more “name: value” pairs for example {Lname: ‘Smith’, Fname: ‘John’, Minit: ‘B’}.
- To create **nodes and relationships** in Neo4j, we use the Neo4j **CREATE** command, which is part of the high-level declarative query language **Cypher**.

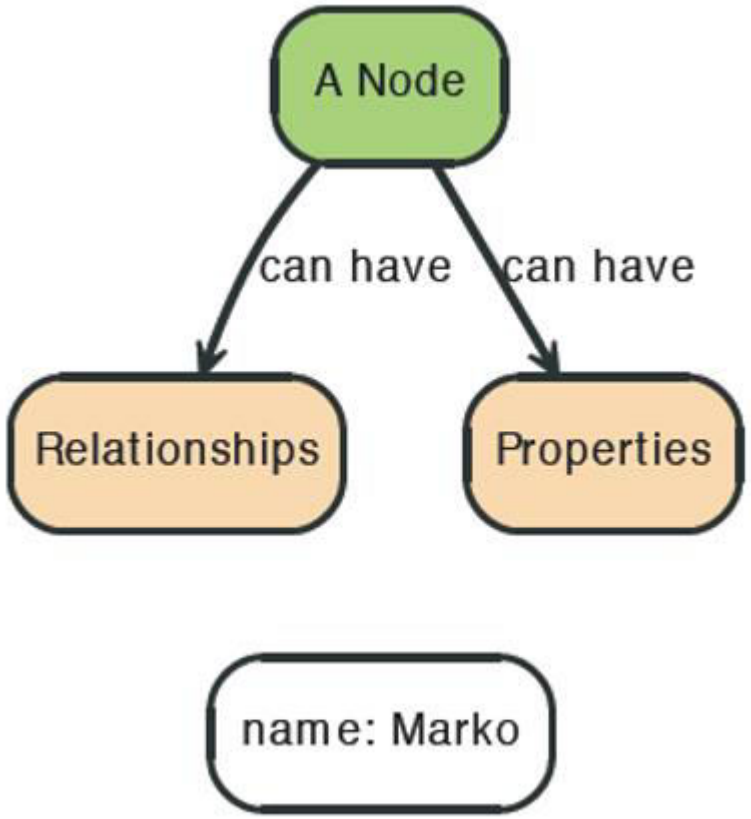
Database Management Systems

Neo4j Data Model

- **Nodes** in Neo4j correspond to **entities**
- **Node labels** correspond to **entity types** and **subclasses**
- **Relationships** correspond to **relationship instances**
- **Properties** correspond to **attributes**

Neo4j	ER Model
Relationship is directed.	Relationship is not directed.
Node may have no label in Neo4j.	Every entity must belong to an entity type, so a node must have a label.
Graph model of Neo4j is used as a basis for an actual high-performance distributed database system	The ER/EER(Enhanced ER) model is mainly used for database design.

Node

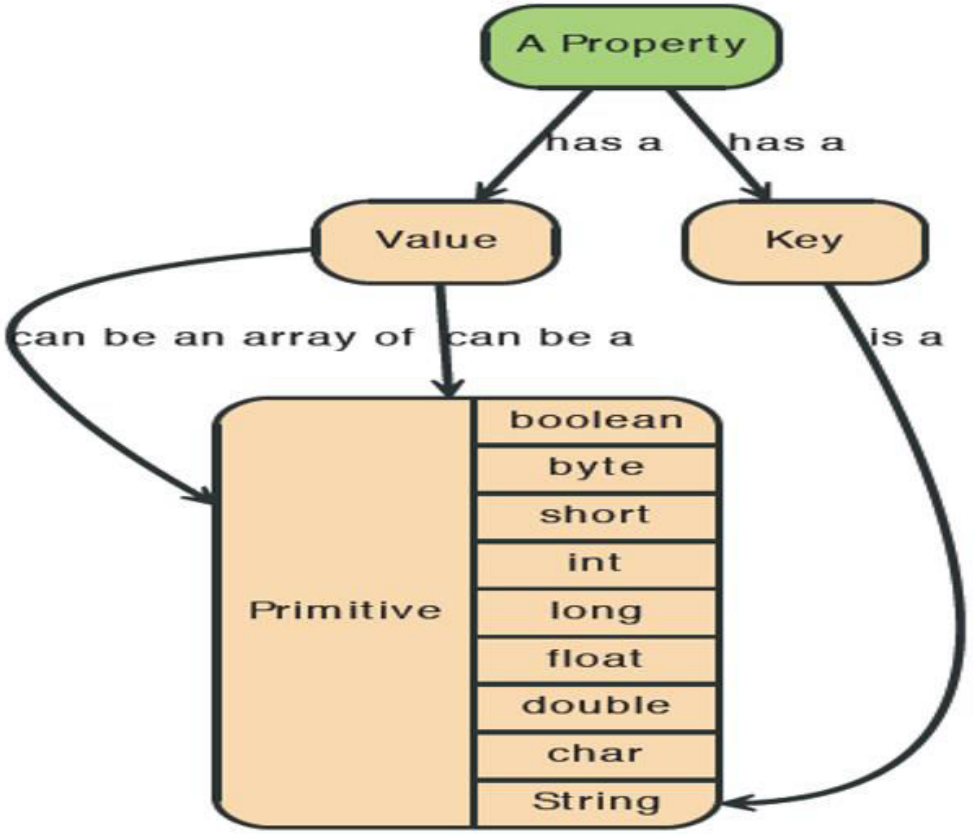

A Node
can have
can have
Relationships
Properties
name: Marko

Database Management Systems

Properties

- Both nodes and relationships can have properties.
- Properties are key-value pairs where the key is a string.
- Property values can be either a primitive or an array of one primitive type.

  For example, String, int and int[] values are valid for properties



Database Management Systems

Labels and Properties

- When a node is created, the node label can be specified. It is also possible to create nodes without any labels.
- In Figure 24.4(a), the node labels are EMPLOYEE, DEPARTMENT, PROJECT, and LOCATION; the created nodes correspond to some of the data from the COMPANY database.
- It is possible that some **nodes** have **multiple labels**.
- For example*, the same node can be labeled as PERSON and EMPLOYEE and MANAGER by listing all the labels separated by the colon symbol as follows: PERSON:EMPLOYEE: MANAGER.*
- Having multiple labels is similar to an entity belonging to an entity type (PERSON) plus some subclasses of PERSON (namely EMPLOYEE and MANAGER).

Figure 24.4
Examples in Neo4j using the Cypher language. (a) Creating some nodes. (b) Creating some relationships.

(a) creating some nodes for the COMPANY data (from Figure 5.6):

```
CREATE (e1: EMPLOYEE, {Empid: '1', Lname: 'Smith', Fname: 'John', Minit: 'B'})
CREATE (e2: EMPLOYEE, {Empid: '2', Lname: 'Wong', Fname: 'Franklin'})
CREATE (e3: EMPLOYEE, {Empid: '3', Lname: 'Zelaya', Fname: 'Alicia'})
CREATE (e4: EMPLOYEE, {Empid: '4', Lname: 'Wallace', Fname: 'Jennifer', Minit: 'S'})
...
CREATE (d1: DEPARTMENT, {Dno: '5', Dname: 'Research'})
CREATE (d2: DEPARTMENT, {Dno: '4', Dname: 'Administration'})
...
CREATE (p1: PROJECT, {Pno: '1', Pname: 'ProductX'})
CREATE (p2: PROJECT, {Pno: '2', Pname: 'ProductY'})
CREATE (p3: PROJECT, {Pno: '10', Pname: 'Computerization'})
CREATE (p4: PROJECT, {Pno: '20', Pname: 'Reorganization'})
...
CREATE (loc1: LOCATION, {Lname: 'Houston'})
CREATE (loc2: LOCATION, {Lname: 'Stafford'})
CREATE (loc3: LOCATION, {Lname: 'Bellaire'})
CREATE (loc4: LOCATION, {Lname: 'Sugarland'})
...
```

Database Management Systems

Relationships and Relationship types

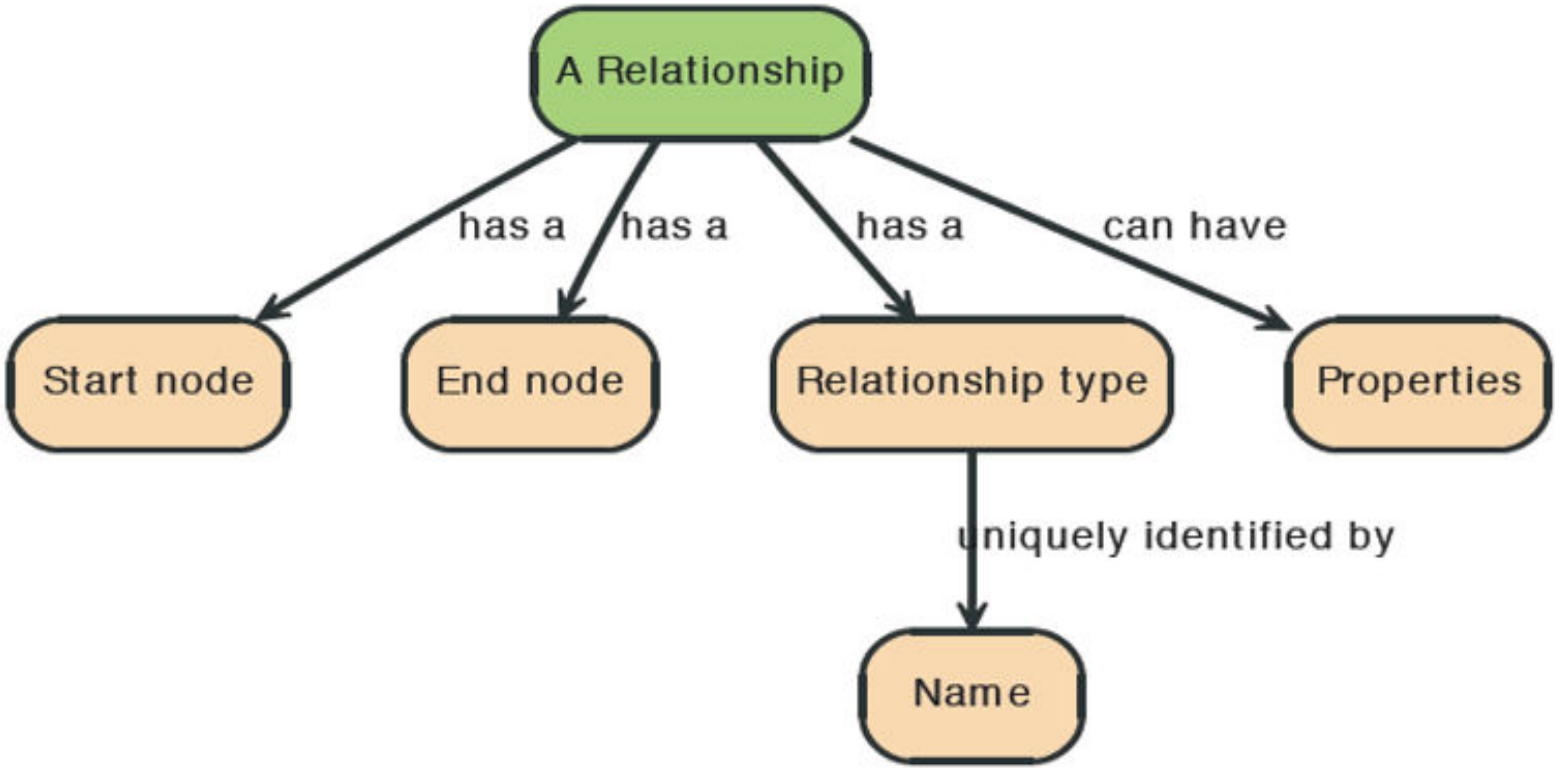
- Figure 24.4(b) shows a few **example relationships** in Neo4j based on the COMPANY database
- The → **specifies the direction of the relationship**, but the relationship can be traversed in either direction.
- The **relationship types (labels)** in Figure 24.4(b) are WorksFor, Manager, LocatedIn, and WorksOn
- Only relationships with the relationship type WorksOn have properties (Hours) in Figure 24.4(b).

(b) creating some relationships for the COMPANY data (from Figure 5.6):

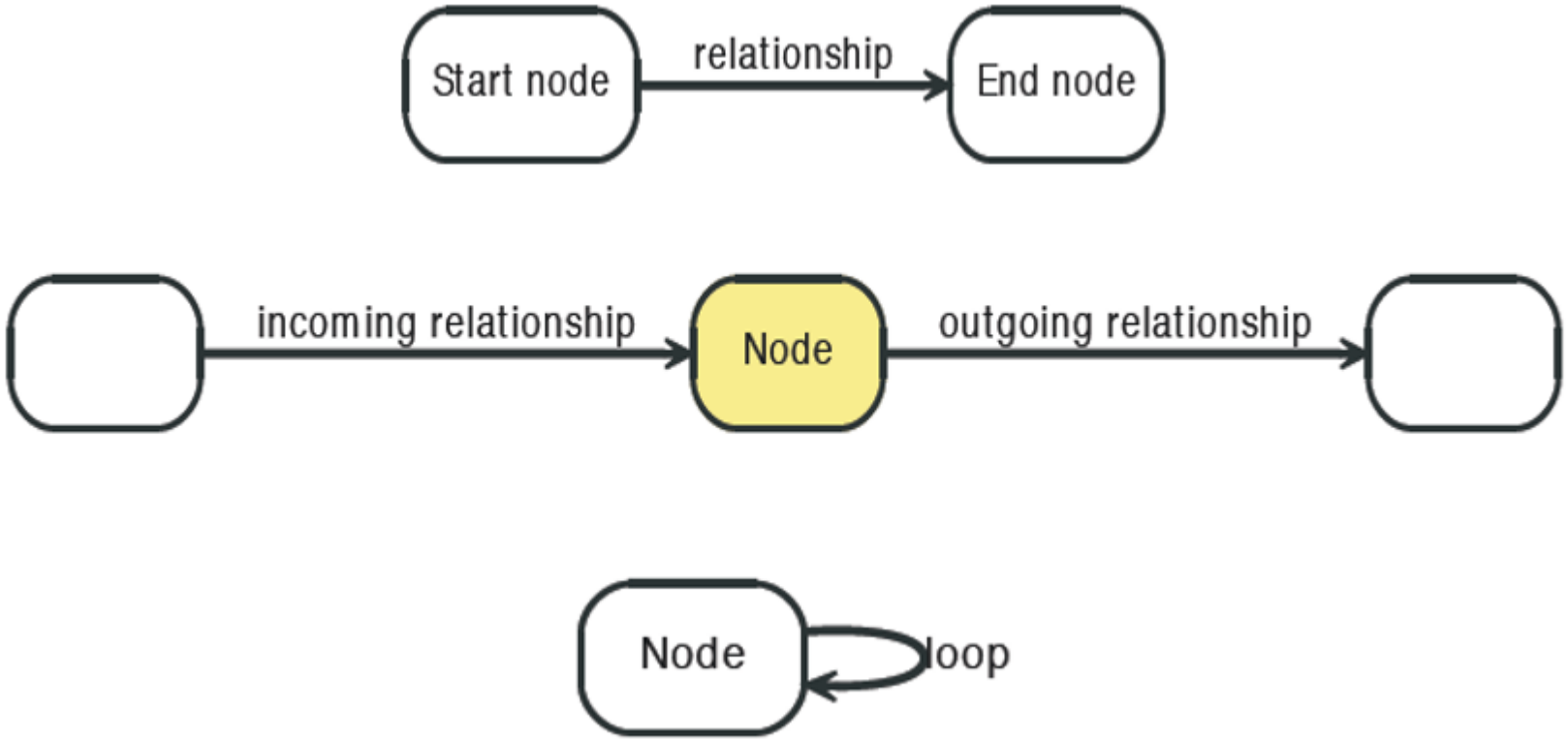
```
CREATE (e1) – [ : WorksFor ] –> (d1)
CREATE (e3) – [ : WorksFor ] –> (d2)
...
CREATE (d1) – [ : Manager ] –> (e2)
CREATE (d2) – [ : Manager ] –> (e4)
...
CREATE (d1) – [ : LocatedIn ] –> (loc1)
CREATE (d1) – [ : LocatedIn ] –> (loc3)
CREATE (d1) – [ : LocatedIn ] –> (loc4)
CREATE (d2) – [ : LocatedIn ] –> (loc2)
...
CREATE (e1) – [ : WorksOn, {Hours: '32.5'} ] –> (p1)
CREATE (e1) – [ : WorksOn, {Hours: '7.5'} ] –> (p2)
CREATE (e2) – [ : WorksOn, {Hours: '10.0'} ] –> (p1)
CREATE (e2) – [ : WorksOn, {Hours: 10.0} ] –> (p2)
CREATE (e2) – [ : WorksOn, {Hours: '10.0'} ] –> (p3)
CREATE (e2) – [ : WorksOn, {Hours: 10.0} ] –> (p4)
...
```

Database Management Systems

Relationship



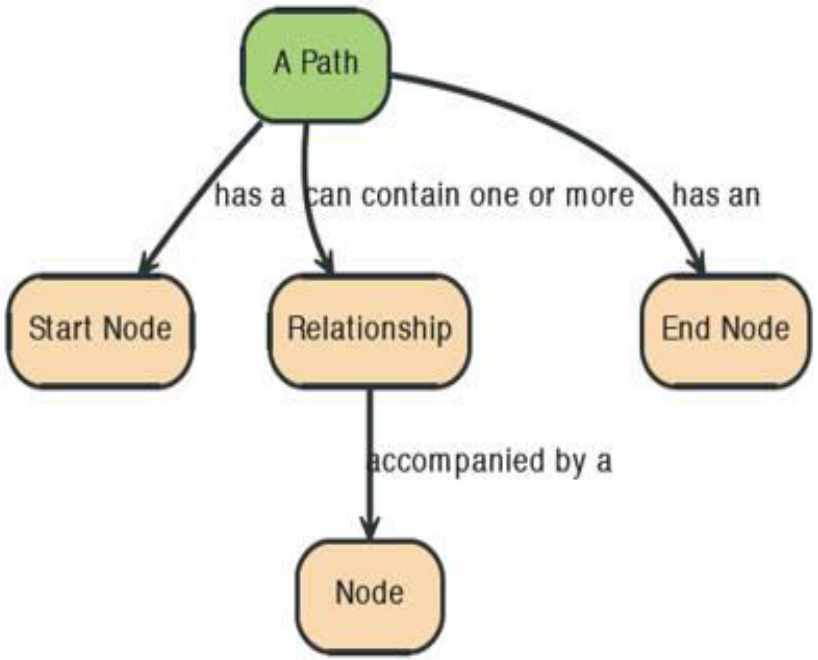
Relationships

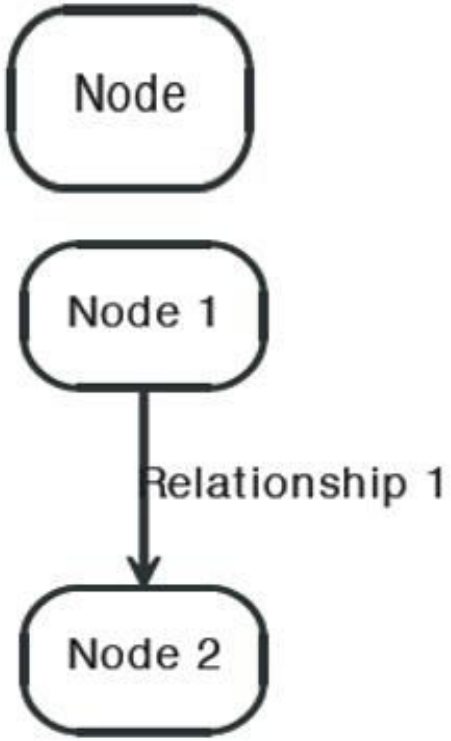


Database Management Systems

Path

A path is one or more nodes with connecting relationships, typically retrieved as a query or traversal result





Path

- A path specifies a **traversal of part of the graph**.
- It is typically used as **part of a query to specify a pattern**, where the query will retrieve from the graph data that matches the pattern.
- A path *is typically specified by a start node, followed by one or more relationships, leading to one or more end nodes that satisfy the pattern.*
- It is somewhat similar to the concepts of path expressions in the context of query languages for **object databases (OQL) and XML (XPat and XQuery).**

Optional Schema

- A schema is **optional** in Neo4j.
- Graphs can be created and used without a schema, but in Neo4j version 2.0, a few schema-related functions were added.
- The main features related to **schema creation involve creating indexes and constraints based on the labels and properties.**
- For example, it is possible to create the equivalent of a **key constraint on a property of a label**, so all nodes in the *collection of nodes associated with the label must have unique values for that property.*

Indexing and node identifiers

- When a node is created, the Neo4j system creates an **internal unique system-defined identifier for each node**.
- To retrieve individual nodes using other properties of the nodes efficiently, the user can **create indexes for the collection of nodes** that have a particular label.
- Typically, **one or more** of the **properties** of the nodes in that collection **can be indexed**.
- **For example**, *Empid can be used to index nodes with the EMPLOYEE label, Dno to index the nodes with the DEPARTMENT label, and Pno to index the nodes with the PROJECT label.*

Database Management Systems

Q&A

1. In Neo4j, what are nodes typically used to represent?
    A. Relationships between entities
    B. Data attributes only
    C. Entities such as people, places, or things
    D. SQL tables

2. What is the storage model of Neo4j based on?
    A. Tables and rows
    B. Documents and collections
    C. Key-value pairs
    D. Nodes and relationships

3. What is a key advantage of Neo4j over relational databases?
    A. Better file storage
    B. Faster disk I/O
    C. Efficient handling of highly connected data
    D. SQL support

Database Management Systems

Q&A

1. In Neo4j, what are nodes typically used to represent?
    A. Relationships between entities
    B. Data attributes only
    C. Entities such as people, places, or things
    D. SQL tables

2. What is the storage model of Neo4j based on?
    A. Tables and rows
    B. Documents and collections
    C. Key-value pairs
    D. Nodes and relationships

3. What is a key advantage of Neo4j over relational databases?
    A. Better file storage
    B. Faster disk I/O
    C. Efficient handling of highly connected data
    D. SQL support

THANK YOU

Prof. Shilpa S
Department of Computer Science and Engineering

shilpas@pes.edu

Database Management Systems

Continuation - Neo4j

Prof. Shilpa S
Department of Computer Science and Engineering

PES UNIVERSITY
CELEBRATING 50 YEARS

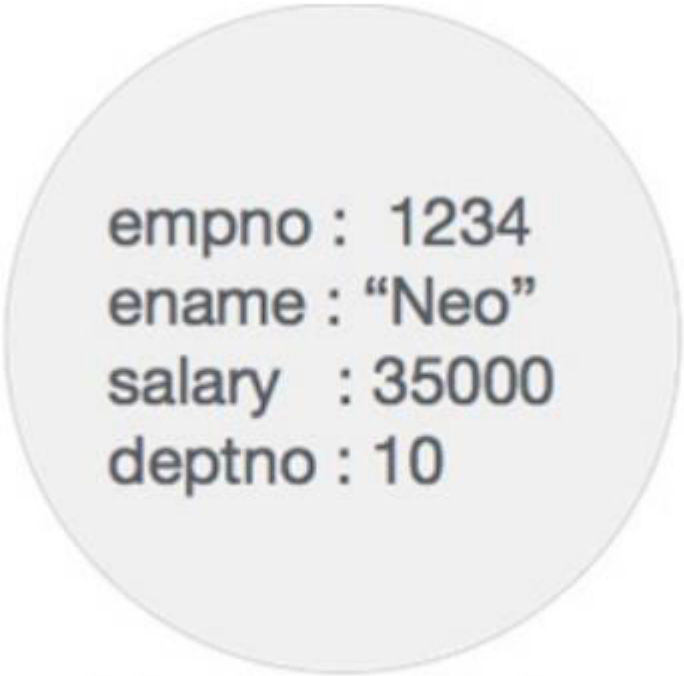
Neo4j - Building Blocks

- Nodes
- Properties
- Relationships
- Labels
- Data Browser

Nodes

- Node is a fundamental unit of a Graph. It contains properties with key-value pairs as shown in the following image.
- Here, Node Name = "Employee"

Contains properties as key-value pairs.



Employee Node

Properties

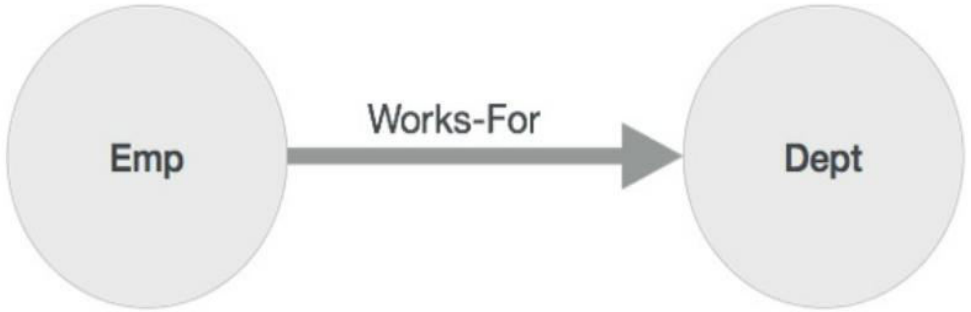
- Property is a key-value pair to describe Graph Nodes and Relationships.

```
Key = Value
```

- Where Key is a String and Value may be represented using any Neo4j Data types.

Relationships

- Relationships are another major building block of a Graph Database. It connects two nodes as depicted in the following figure.
- Here, Emp and Dept are two different nodes. "WORKS_FOR" is a relationship between Emp and Dept nodes.
- As it denotes, the arrow mark from Emp to Dept, this relationship describes −



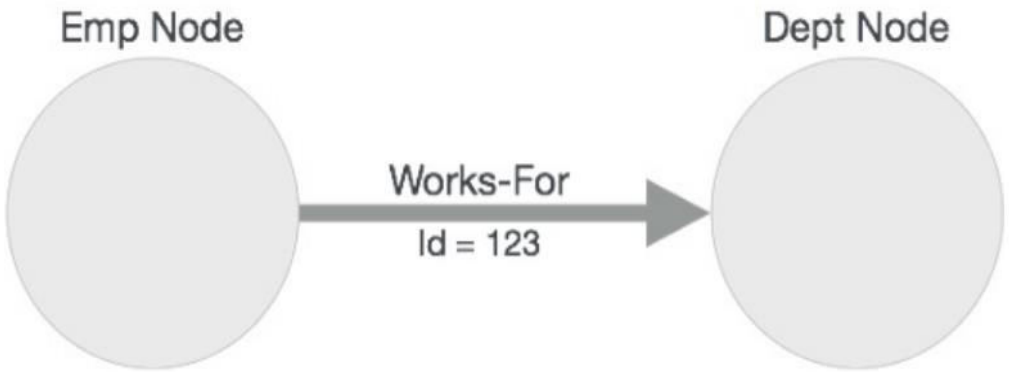

```
Emp WORKS_FOR Dept
```

Relationships

- Each relationship contains one start node and one end node.
- Here, "Emp" is a start node, and "Dept" is an end node.
- As this relationship arrow mark represents a relationship from "Emp" node to "Dept" node, this relationship is known as an "Incoming Relationship" to "Dept" Node and "Outgoing Relationship" to "Emp" node.
- Like nodes, relationships also can contain properties as key-value pairs.
- Here, "WORKS_FOR" relationship has one property as key-value pair.

```
Id = 123
```

- It represents an Id of this relationship.



Labels

- Label associates a common name to a set of nodes or relationships.
- A node or relationship can contain one or more labels.
- We can create new labels to existing nodes or relationships.
- We can remove the existing labels from the existing nodes or relationships.

For example-

- Left side node has a Label: "Emp" and the right side node has a Label: "Dept".
- Relationship between those two nodes also has a Label: "WORKS_FOR".

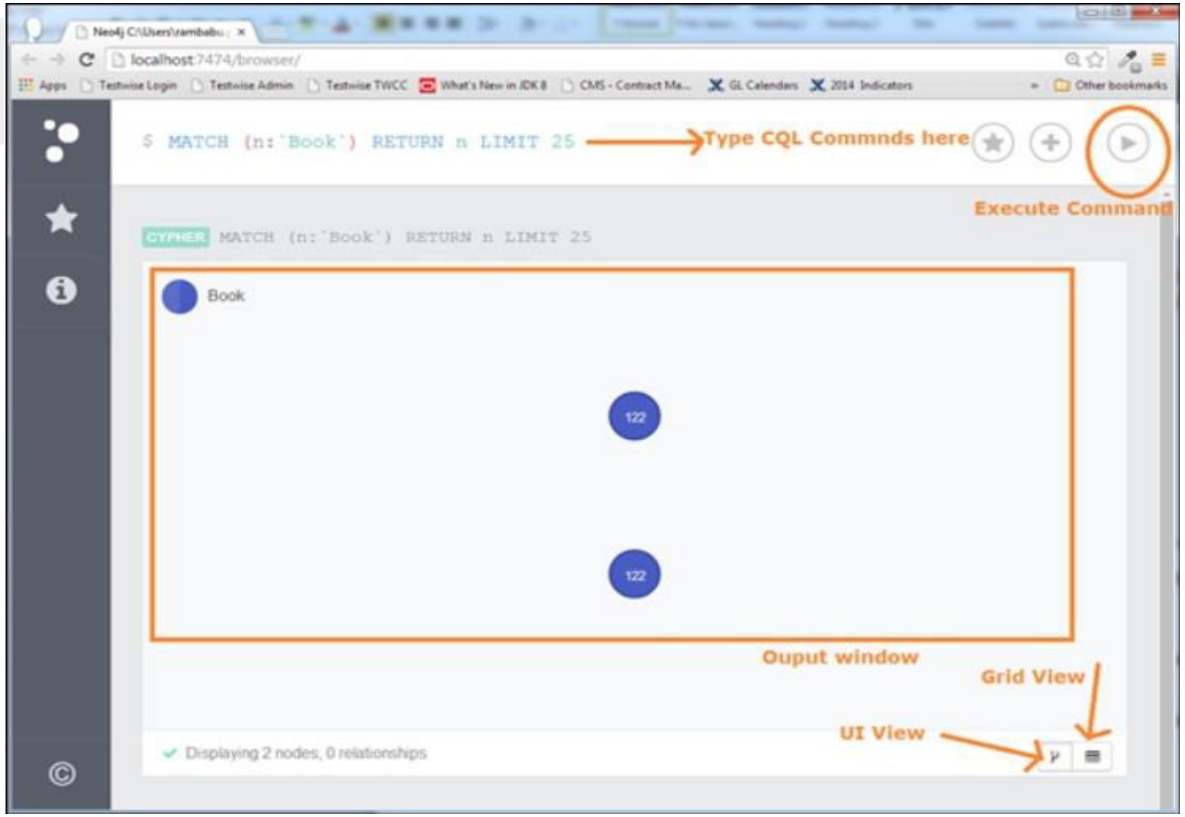
Database Management Systems

Neo4j - Practice Session

Neo4j Data Browser

- Once we install Neo4j, we can access Neo4j Data Browser using the following URL

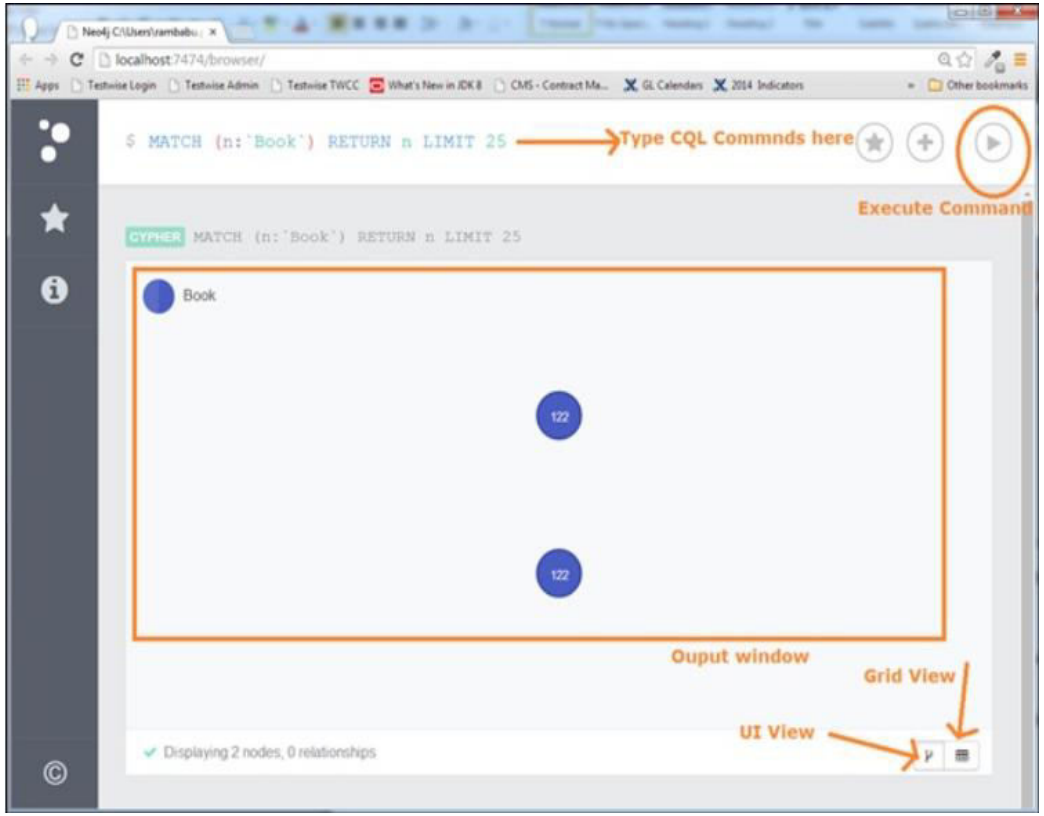
```
http://localhost:7474/browser/
```



- Neo4j Data Browser is used to execute CQL commands and view the output.
- Here, we need to execute all CQL commands at dollar prompt: **"$"**
- Type commands after the dollar symbol and click the "Execute" button to run your commands.

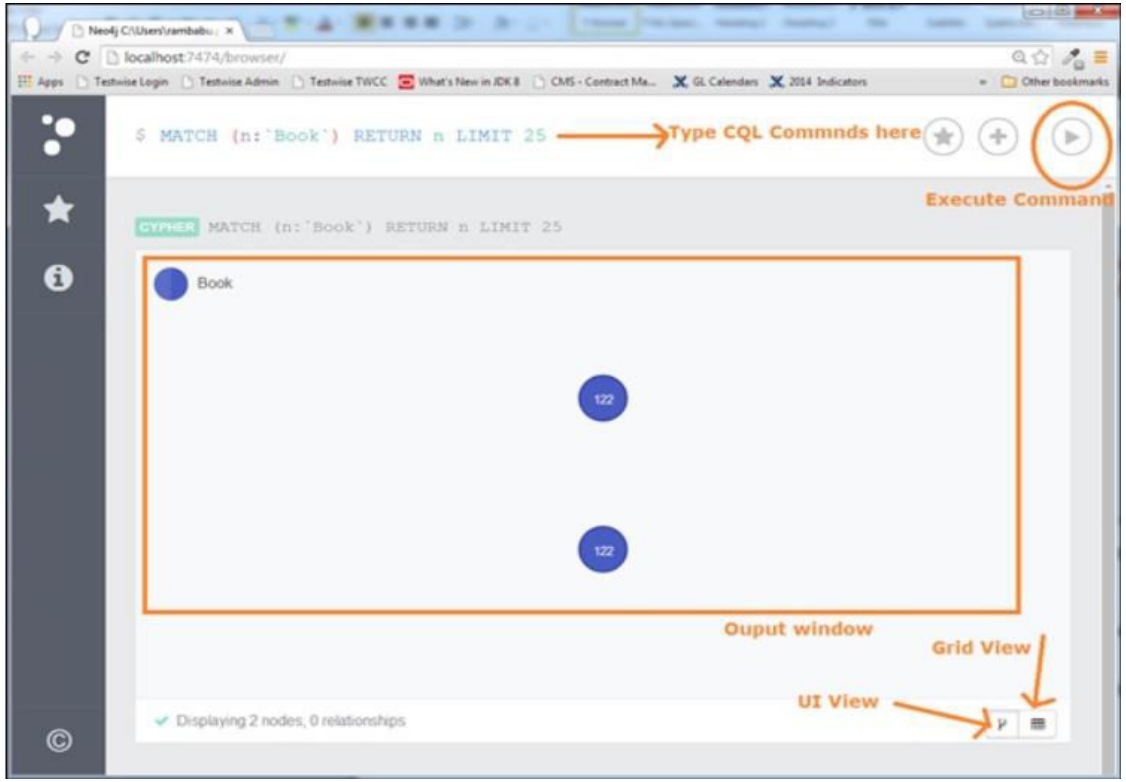
Neo4j Data Browser

- It interacts with Neo4j Database Server, retrieves and displays the results just below the dollar prompt.
- Use "VI View" button to view the results in diagrams format. The above diagram shows results in "UI View" format.
- Use "Grid View" button to view the results in Grid View. The following diagram shows the same results in "Grid View" format.



Neo4j Data Browser

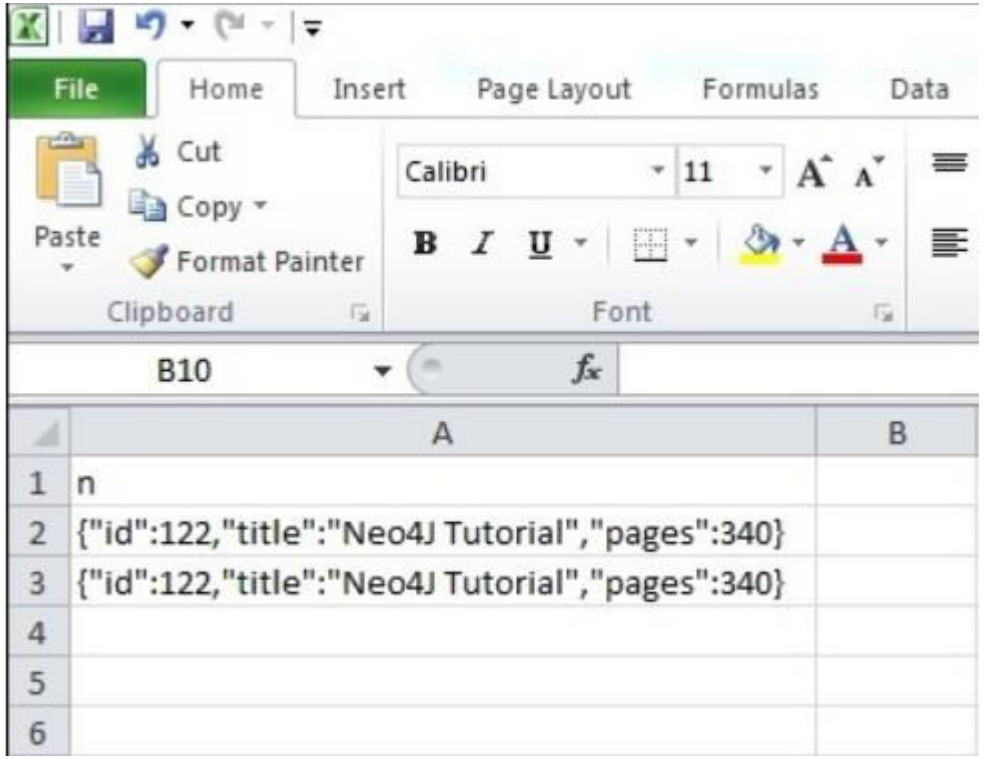
- When we use "Grid View" to view our Query results, we can export them into a file in two different formats.

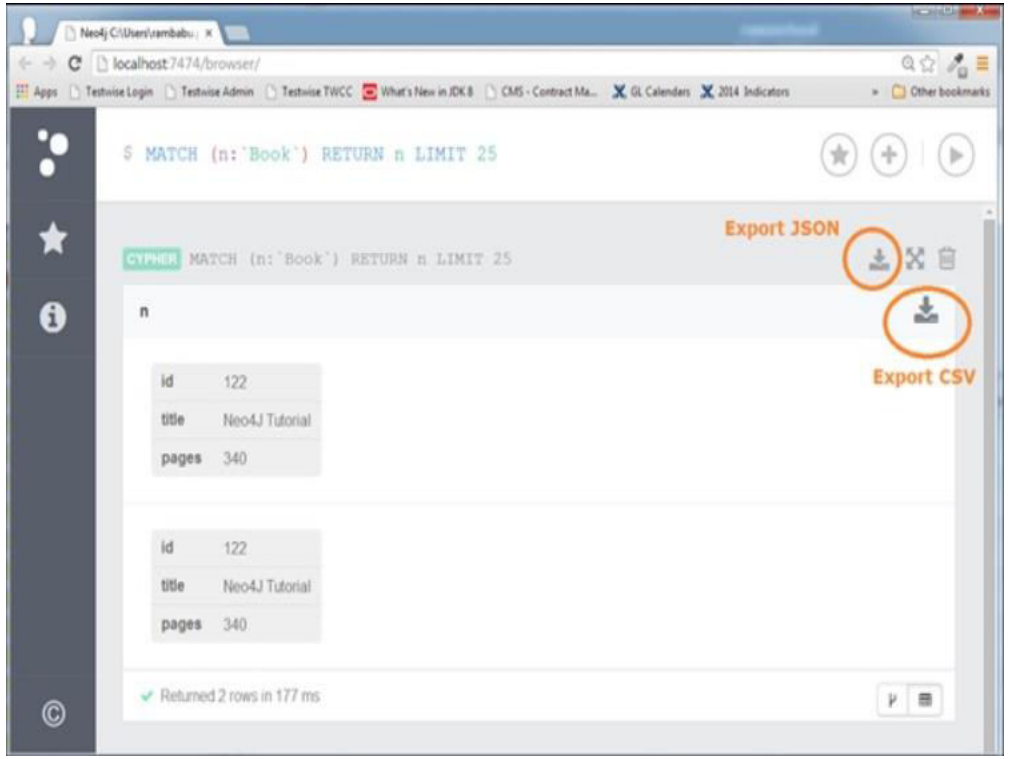


Neo4j Data Browser

- **CSV**

  Click the "Export CSV" button to export the results in csv file format.

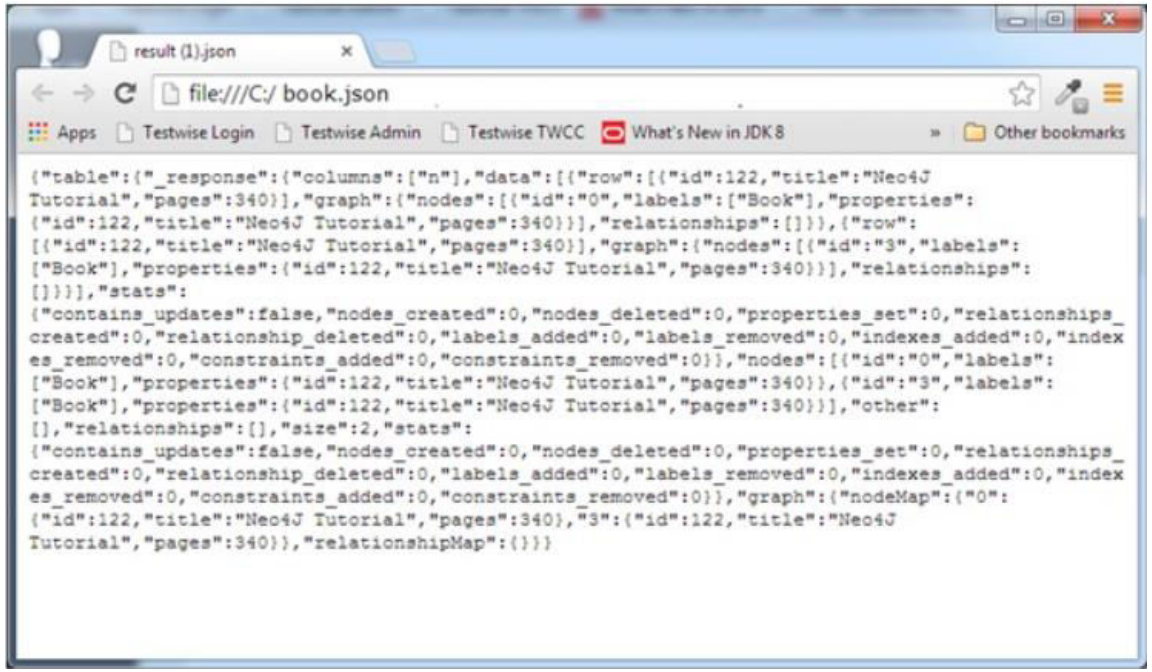


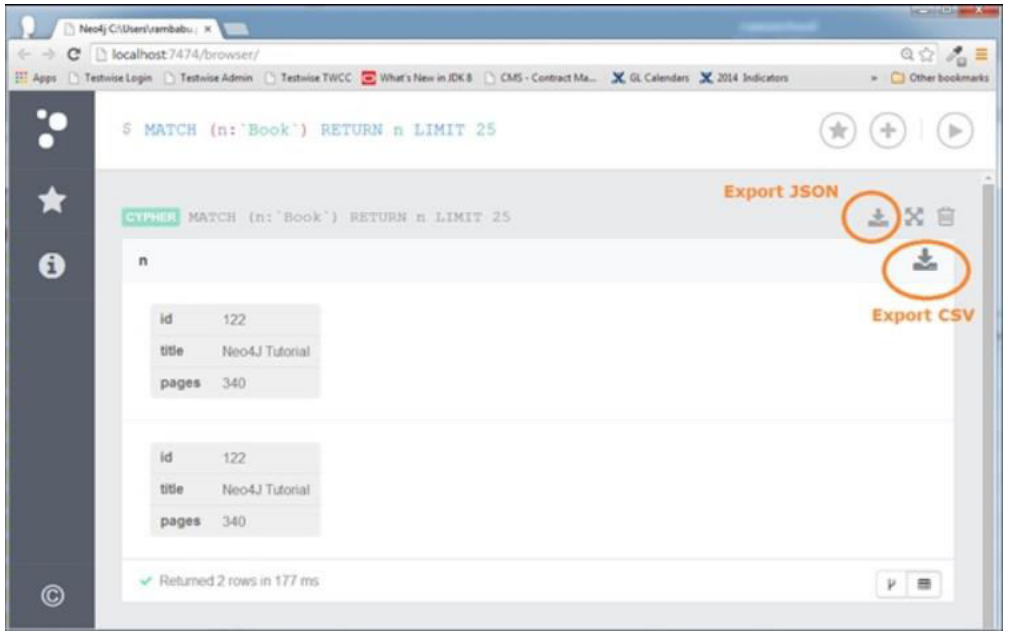


Neo4j Data Browser

- **JSON**

  Click the "Export JSON" button to export the results in JSON file format.





However, if we use "UI View" to see our Query results, we can export them into a file in only one format: JSON

PES UNIVERSITY
CELEBRATING 50 YEARS

Neo4j Read Clauses

Sr.No	Read Clauses	Usage
1	MATCH	This clause is used to search the data with a specified pattern.
2	OPTIONAL MATCH	This is the same as match, the only difference being it can use nulls in case of missing parts of the pattern.
3	WHERE	This clause id is used to add contents to the CQL queries.
4	START	This clause is used to find the starting points through the legacy indexes.
5	LOAD CSV	This clause is used to import data from CSV files.

Database Management Systems

Neo4j - Practice Session

PES UNIVERSITY
CELEBRATING 50 YEARS

Neo4j Write Clauses

Sr.No	Write Clause	Usage
1	CREATE	This clause is used to create nodes, relationships, and properties.
2	MERGE	This clause verifies whether the specified pattern exists in the graph. If not, it creates the pattern.
3	SET	This clause is used to update labels on nodes, properties on nodes and relationships.
4	DELETE	This clause is used to delete nodes and relationships or paths etc. from the graph.
5	REMOVE	This clause is used to remove properties and elements from nodes and relationships.
6	FOREACH	This class is used to update the data within a list.
7	CREATE UNIQUE	Using the clauses CREATE and MATCH, you can get a unique pattern by matching the existing pattern and creating the missing one.
8	Importing CSV files with Cypher	Using Load CSV you can import data from .csv files.

Database Management Systems

Neo4j - Practice Session

PES UNIVERSITY
CELEBRATING 50 YEARS

Neo4j General Clauses

Sr.No	General Clauses	Usage
1	RETURN	This clause is used to define what to include in the query result set.
2	ORDER BY	This clause is used to arrange the output of a query in order. It is used along with the clauses **RETURN** or **WITH**.
3	LIMIT	This clause is used to limit the rows in the result to a specific value.
4	SKIP	This clause is used to define from which row to start including the rows in the output.
5	WITH	This clause is used to chain the query parts together.
6	UNWIND	This clause is used to expand a list into a sequence of rows.
7	UNION	This clause is used to combine the result of multiple queries.
8	CALL	This clause is used to invoke a procedure deployed in the database.

Database Management Systems

Neo4j - Practice Session

PES UNIVERSITY
CELEBRATING 50 YEARS

Neo4j CQL Data Types

Sr.No	CQL Data Type	Usage
1	Boolean	It is used to represent Boolean literals: true, false.
2	byte	It is used to represent 8-bit integers.
3	short	It is used to represent 16-bit integers.
4	int	It is used to represent 32-bit integers.
5	long	It is used to represent 64-bit integers.
6	float	It is used to represent 32-bit floating-point numbers.
7	double	It is used to represent 64-bit floating-point numbers.
8	char	It is used to represent 16-bit characters.
9	String	It is used to represent Strings.

PES UNIVERSITY
CELEBRATING 50 YEARS

Neo4j CQL Operators

Sr.No	Type	Operators
1	Mathematical	+, -, *, /, %, ^
2	Comparison	+, <>, <, >, <=, >=
3	Boolean	AND, OR, XOR, NOT
4	String	+
5	List	+, IN, [X], [X.....Y]
6	Regular Expression	=-
7	String matching	STARTS WITH, ENDS WITH, CONSTRAINTS

PES UNIVERSITY
CELEBRATING 50 YEARS

Neo4j CQL Boolean Operators

Sr.No	Boolean Operators	Description
1	AND	It is a Neo4j CQL keyword to support AND operation. It is like SQL AND operator.
2	OR	It is a Neo4j CQL keyword to support OR operation. It is like SQL AND operator.
3	NOT	It is a Neo4j CQL keyword to support NOT operation. It is like SQL AND operator.
4	XOR	It is a Neo4j CQL keyword to support XOR operation. It is like SQL AND operator.

PES UNIVERSITY
CELEBRATING 50 YEARS

Neo4j CQL Boolean Comparison Operators

Sr.No	Boolean Operators	Description
1	=	It is a Neo4j CQL "Equal To" operator.
2	< >	It is a Neo4j CQL "Not Equal To" operator.
3	<	It is a Neo4j CQL "Less Than" operator.
4	>	It is a Neo4j CQL "Greater Than" operator.
5	<=	It is a Neo4j CQL "Less Than Or Equal To" operator.
6	> =	It is a Neo4j CQL "Greater Than Or Equal To" operator.

Database Management Systems

Q&A

1. What is the output of the following Cypher query?

   MATCH (a:Person)-[:FRIEND]->(b:Person) RETURN a.name, b.name;

A) List of persons only with their names
B) Only persons who have mutual friendships
C) List of directed friendships with names of both persons
D) Throws a syntax error Neo4j,

2. How do you create a node with label Movie and properties title and year?

A) CREATE Movie(title:'Inception', year:2010);
B) CREATE (:Movie {title: 'Inception', year: 2010});
C) INSERT INTO Movie VALUES ('Inception', 2010);
D) NEW Movie = ('Inception', 2010);

Database Management Systems

Q&A

3. What will the following Cypher query return?

```
MATCH (m:Movie)<-[:ACTED_IN]-(a:Actor)
WHERE m.title = 'Matrix’
RETURN a.name;
```

A) All actors in all movies
B) Only actors who acted in "Matrix“
C) All movies and actors in the database
D) Throws an error due to invalid syntax

Database Management Systems

Q&A

1. What is the output of the following Cypher query?

   MATCH (a:Person)-[:FRIEND]->(b:Person) RETURN a.name, b.name;

A) List of persons only with their names
B) Only persons who have mutual friendships
C) List of directed friendships with names of both persons
D) Throws a syntax error Neo4j,

2. How do you create a node with label Movie and properties title and year?

A) CREATE Movie(title:'Inception', year:2010);
B) CREATE (:Movie {title: 'Inception', year: 2010});
C) INSERT INTO Movie VALUES ('Inception', 2010);
D) NEW Movie = ('Inception', 2010);

Q&A

3. What will the following Cypher query return?

```
MATCH (m:Movie)<-[:ACTED_IN]-(a:Actor)
WHERE m.title = 'Matrix’
RETURN a.name;
```

A) All actors in all movies
B) Only actors who acted in "Matrix“
C) All movies and actors in the database
D) Throws an error due to invalid syntax

THANK YOU

Prof. Shilpa S
Department of Computer Science and Engineering

shilpas@pes.edu

Database Management Systems

Neo4j - Syntax

Prof. Shilpa S
Department of Computer Science and Engineering

Database Management Systems

Neo4j - Practice Session

Create a node:

```
Create (n)
```

// creates a single node
// n is an identifier and not the name of the node.

Output: Created one node

View a node:

```
Match(n) return(n);
```

//Fetches a node created in the database
//node is printed with internal property: “id”
// id is used for internal usage, starts from zero, and each node will have only one internal id and unique in the database and is auto-generated by database

Create multiple nodes:

Create (n), (m)

// creates two nodes successfully. n and m are identifiers/variables

Output: Created two nodes

View all nodes created:

Match(n) return n;

//Fetches the nodes created in the database

Output: 3 nodes are displayed including the one created in previous slide

View some nodes:

Match(n) return n limit 2

//Restrict the number of nodes by using limit clause Output: Displays first 2 nodes

Search a node by internal id:

match (n) where id(n) =1 return n

// id builtin function of Neo4j, returns the internal id of the node when used with where clause

Output: Displays the node with internal id=1

Comparison operator to filter out the nodes (<,>,<=,>=,<>):

To fetch the node with internal id <=6

Match(n) where id(n)<=6 return n

Output: Displays all 7 nodes (node with id =0,1,2,3,4,5,6)

Search multiple nodes with internal id's (using in clause)

Match(n) where id(n) in [1,2,0] return n

Output: Displays 3 nodes with id 0, 1, 2

Delete a node by its internal id:

match (n) where id(n) =1 delete n

// deletes the node that matches the where clause, ie deletes a node with id=1

Output: Deleted one node

Delete multiple nodes with id:

Match(n) where id(n) in [2,3] delete n

// two nodes are deleted with internal id 2 and 3

Output: Deleted two nodes

Delete all nodes

Match(n) delete n

//delete all the nodes in the database if there are no relationships in the graph

Output: Deleted “x” nodes

Deleting entire DB:

sudo rm –rf /var/lib/neo4j/data/*

// deletes entire graph directory

//all the data is stored in the graph director, so need to delete the graph directory.

// Before this stop the neo4j server using command **sudo service neo4j stop**

// later restart the server and check in neo4j graphical interface that no data is available in db, can use neo4j from scratch.

Sudo service neo4j start -> to start the server

Sudo service neo4j stop -> to stop the server

Sudo service neo4j status -> to check the status of the server (running/stopped)

Labels are the tags that can be associated to nodes, labels can be used to group the nodes

Labels are optional and each node can have multiple labels

Any valid Unicode string can be used as a label

Creating a node with a label:

Create (n: Person)
Output: Added 1 label, created 1 node

// creates a node with label Person
// label is represented by a colon followed by name of the label and can be applied only to nodes.
// by convention use Camel case

Search node with labels:

Match (n) where n: Person return n
Output: Added 1 label, created 1 node

// returns a node with label Person
// displays both the id and label of the node

Create a node with multiple labels:

create (n : Person : Indian)
Output: Added 2 labels, created 1 node
// a node is created with two labels (person and Indian)
// a node can have multiple labels and a label can have multiple nodes
// a node can have multiple labels as it can represent multiple things at the same time

Search node by multiple labels:

Match (n) where n:Person:Indian return n
Output: Added 1 label, created 1 node,

// returns all the nodes with label Person and Indian
// displays both the id and label of the node

Search a node with Person or Indian label or both:

Match (n) where n :Person or n :Indian return n
//returns nodes with the label Person or with the label Indian or both

Search node by multiple labels:

Match (n) where n:Person:Indian return n

Output: Added 1 label, created 1 node,

// returns all the nodes with label Person and Indian
// displays both the id and label of the node

Search a node with Person or Indian label or both:

Match (n) where n :Person or n :Indian return n

//returns nodes with the label Person or with the label Indian or both

Add label to an existing node using set clause:

Match (n) set n:Employee return n

// add's a label Employee to all the nodes in the database

Add multiple labels to all nodes in database

Match (n) set n:Vacation:Food return n

// add's a label Vacation and Food to all the nodes in the database

Add label to a single selected node by using id:

Match (n) where id (n) =1 set n:Manager return n

//add's a label Manager to the node with the internal id=1

Add a label to multiple selected nodes:

Match (n) where id (n) in [2,3] set n: TeamLeader return n

//adds a label “TeamLeader” to the nodes with internal id 2 and 3.

Remove a label from all nodes in database:

Match (n) remove n:Person return n

//remove's label "Person" from all the nodes

Remove a label from selected nodes:

match (n) where id(n) in [2,3] remove n:Employee return n

//remove's label "Employee" from the nodes with id 2 and 3.

Remove multiple labels from all nodes in database:

Match (n) remove n:Food: Vacation return n

//remove's label Food and vacation from all the nodes which has either of these labels

Update a label of a node:

Match (n) where id(n) = 0 remove n: Manager set n: Director return n

//update's label of the node with internal id 0 from Manager to Director and return the node

List all labels of all nodes:

match (n) return distinct labels(n);

//displays all labels existing in db

// distinct removes duplicate labels, and node labels are case sensitive

Listing all labels for a node:

Match (n) where id(n)=0 return distinct labels(n)

//list all the labels associated with the node with id=0

Database Management Systems

Neo4j - Practice Session

Count the labels available in db:

```
Match (n) return distinct count (labels(n));
//displays number of distinct labels in db
```

List all labels of all nodes:

```
Match (n) return distinct count (labels(n)),labels(n);
// returns label along with the label counts
```

Delete a node with a given label:

```
Match (n) where n:TeamLeader delete n
// deletes all the nodes with the label TeamLeader
```

Create a node with properties (Properties are attributes of a node)

Properties are always associated with a node or relationship

Properties are optional attributes which are represented as key-value pairs as ({})

A node can have zero or more properties

Creating a node with a single property:

create (x: Labelname {Property name: Value}) return x;

create (x: Book {title : “The Bengal Tiger”, price: 2780,publisher:”xyz”}) return x;

// creates a node with label Book and a property named title with its value “The Bengal Tiger”

Creating a node with multiple single properties:

Pair of properties are separated by comma

```
create (x: Labelname {Property_name: Value, Property_name : value}) return x;
create  (x: Book {title : “Database System”, author: “Navathe”, publisher :
“Prearson”,price:876}) return x;
// creates a node with label Book with 3 properties.
//property names are case sensitive and can contain underscore and alpha numeric characters
//must start with a letter
create  (x: Book1 {title : “Database System”, author: “Navathe”, Author : “Elmasri”}) return x;
// creates a node with label Book1 with 3 properties.
```

To define properties of a node

Property is of the form **Key : Value** pair

Key is the name of the property – String

Value can be primitive data type or can be array of primitive datatypes

List of Primitive datatypes:

Boolean, integer, double, long, character, string

Array of primitive datatypes:

Boolean array, int array, double array, long array, char array, string array

// null is not a valid value for the property

To create a node with properties of different datatype:

```
Create (x:Book { title:"the  lives of others", author : "Neel", publisher: ["Chato & Windos","w norton"], price: 285.00, pages : 528, instock : false }) return x
```

// Creates a node with label Book and 6 properties, where title, author property is of type string, publisher of type string array, price is of type float, page is of type integer and instock is of type Boolean.

// creates a node with 6 properties as specified in query

```
Create (x:Book{details:["the nagas",172.00,398]}) return x;
```

// collections containing mixed types cannot be stored in properties, considering the datatype of the first value

```
Create (x:Book{details:["the nagas","172.00","398"]}) return x;
```
//Successfull

Database Management Systems

Neo4j - Practice Session

To filter a node with properties and with specific value:
There are multiple nodes with label book and associated with properties like author, title, pages, price and publisher
Search all nodes where author of the book is “Chetan ”

```
Match (n:Label{property:”value”}) return n;
Match (n:Book{author:”Chetan”}) return n;
// return all the nodes with label book and property author with value “Chetan” , if not available returns zero records

Match (n:Book1{Author:"Elmasri"}) return n;
// returns nodes with label Book1 and Author value Elmasri
```

To filter a node using operators (AND, OR, < , >……)

Search all nodes whose author is a1 or a2 and price of the book is one thousand.

```
Match (n: Book) where n.price<1000 AND (n.author="Neel" or n.author = "Navathe")
return n;
// return all the nodes with label book with price <1000 and author with value Neel or
Navathe
```

Database Management Systems

Neo4j - Practice Session

We created:

Create (x:Book { title:”the lives of others”, author : “Neel”, publisher: [“Chato & Windos”,”w norton”], price: 285.00, pages : 528, instock : false }) return x

To change the title of the Book

Match(n) where n.title=“the lives of others” set n.title=“The lives” return n

//updates the title from “the lives of others” to “the lives” and returns the node

Database Management Systems

Neo4j - Practice Session

Syntax for the MERGE command

MERGE (node: label {properties . . . . . })

Merging a Node with a Label

MERGE (node:label) RETURN node

MERGE (Jadeja:player) RETURN Jadeja

MERGE (CT2013:Tournament{name: "ICC Champions Trophy 2013"})
RETURN CT2013, labels(CT2013)

Creating a Complete Path

In Neo4j, a path is formed using continuous relationships. A path can be created using the create clause.

```
CREATE p = (Node1 {properties})-[:Relationship_Type]-> (Node2
{properties})[:Relationship_Type]->(Node3 {properties}) RETURN p
```

Database Management Systems

Neo4j - Practice Session

To update multiple properties:

```
Match(n:Book {title:“The lives”}) set
n.author=“Gosh”,n.price=324.00,n.stock=true return n
```

//Searches for the nodes with label Book and title =“The lives” and update its author, price and stock properties accordingly.

To delete multiple properties:
The match clause is used to search and remove a clause to delete certain property.

Method 1: Remove property by setting the value to NULL

```
Match(n:Book {author:“Neel”}) set n.publisher=NULL return n
```

//to remove - set publisher to null from the nodes with label Book and author “Neel”
//This removes property publisher.

Method 2: Remove property by setting the value to NULL

```
Match(n:Book) where n.author = “Neel” set n.publisher=NULL return n
```

To delete property from all the nodes
Use Match clause to get all the nodes and remove clause to delete certain property from all.

Method 1: Remove property “pages” from all nodes

```
Match (n) remove n.pages return n
```

//to remove property pages from all the nodes

Method 2: Remove property “title” from the nodes with label bestseller

```
Match(n) where n:Bestseller remove n.title return n;
```

//a property can be used as a filter to delete other properties

Database Management Systems

Neo4j - Practice Session

PES UNIVERSITY
CELEBRATING 50 YEARS

To delete the nodes

Use match clause to select and delete clause to delete a node

Match (n) where n.stock=false delete n

//to delete nodes based on the property stock with value “false”
// based if id the properties can be deleted

Remove a property named price from the node with the internal id = 1

Match(n) where id(n)=1 remove n.price return n

Database Management Systems

Neo4j - Practice Session

We can create a relationship using the CREATE clause. We will specify relationship within the square braces “[ ]” depending on the direction of the relationship it is placed between hyphen “ - ” and arrow “ → ” as shown in the following syntax.

Format : create (node1) – [:RelationshipType]->(node2)

Create two nodes and then relation ship between them

```
Create (chetan:author{name:”Chetan”,dob:1980,age:41})

Create (nagas:Book{title:”Nagas”,price:200,publication:”westland publication”})

//above to queries creates 2 nodes

Create (chetan)-[r:author_of]->(nagas) return Chetan,r,nagas

//above query to create relationship between them
```

THANK YOU

Prof. Shilpa S
Department of Computer Science and Engineering

shilpas@pes.edu

Database Management Systems

Graph Databases(Neo4j) - Hands on Session

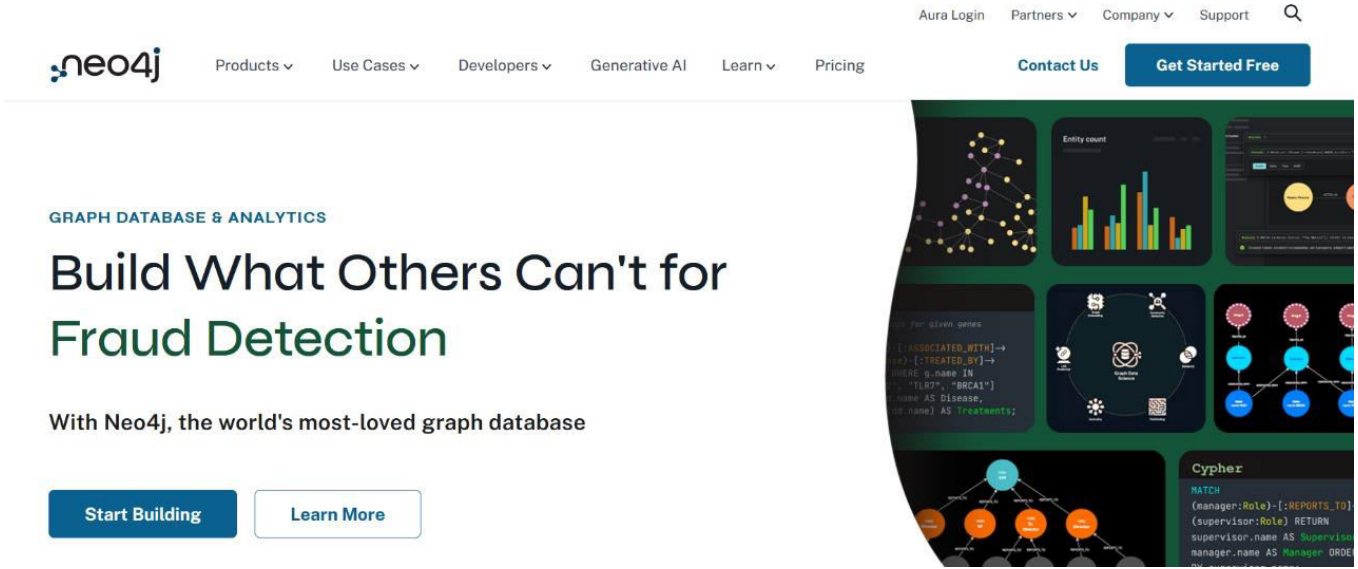
Prof. Mahitha G
Department of Computer Science and Engineering

Database Management Systems

Hands on Session

Step 1: [Login to Neo4J using your Gmail Account]

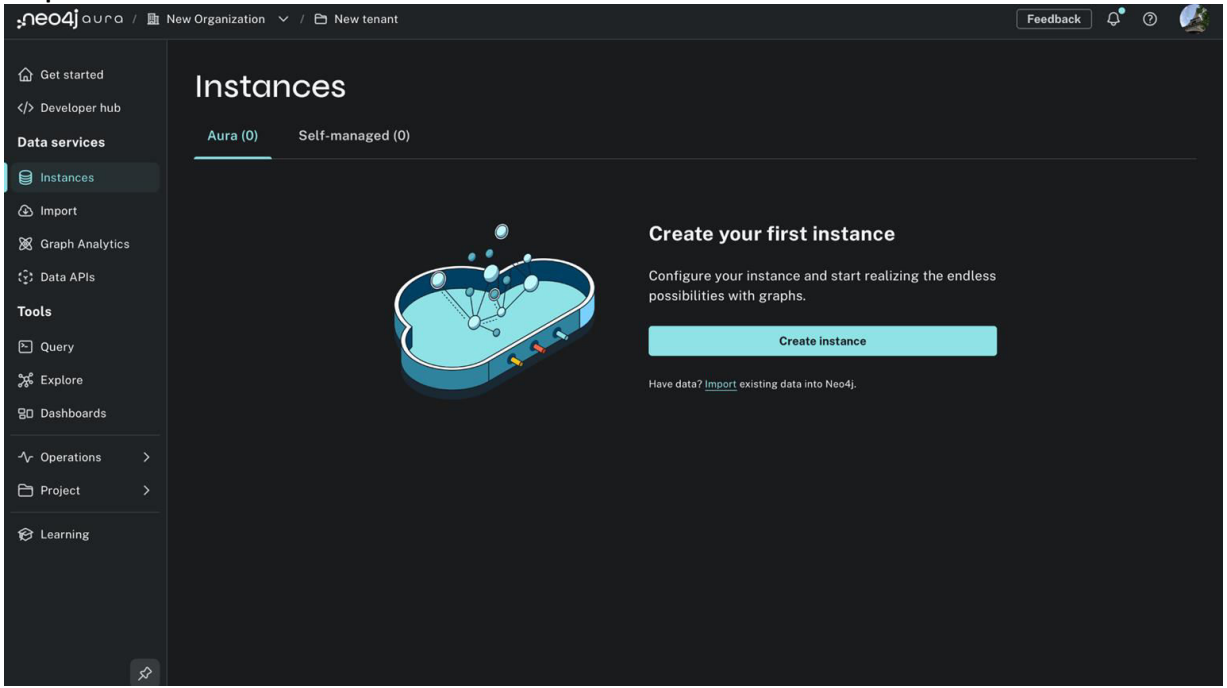
Click on “Get started Free” as shown in the screenshot below.



Database Management Systems

Hands on Session

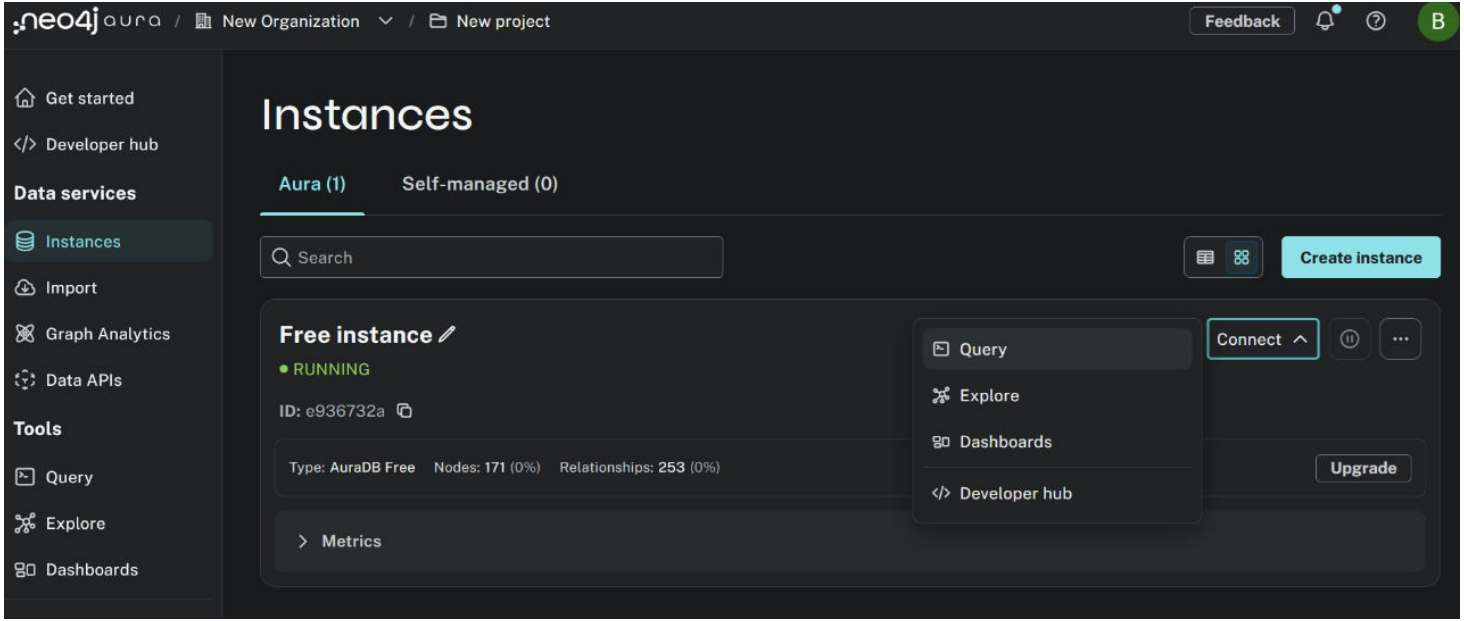
Step 2: Create an instance and download the credentials.



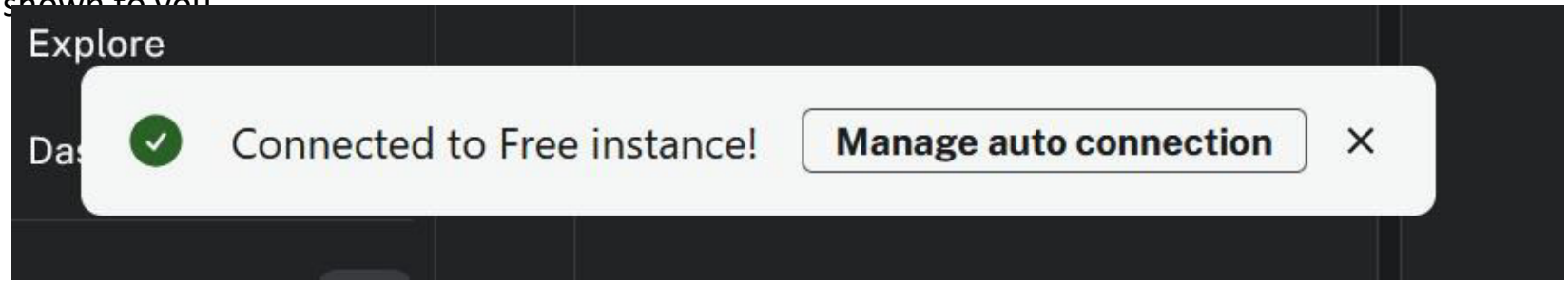
Database Management Systems

Hands on Session

Step 3: Connect to the created instance, by clicking on connect and then pressing on “query”.



Step 4: On successful connection to the instance, the following will be displayed and an editor will be shown to you



Step 5: Create data using the commands in the below txt file.

Create Commands to Insert Data

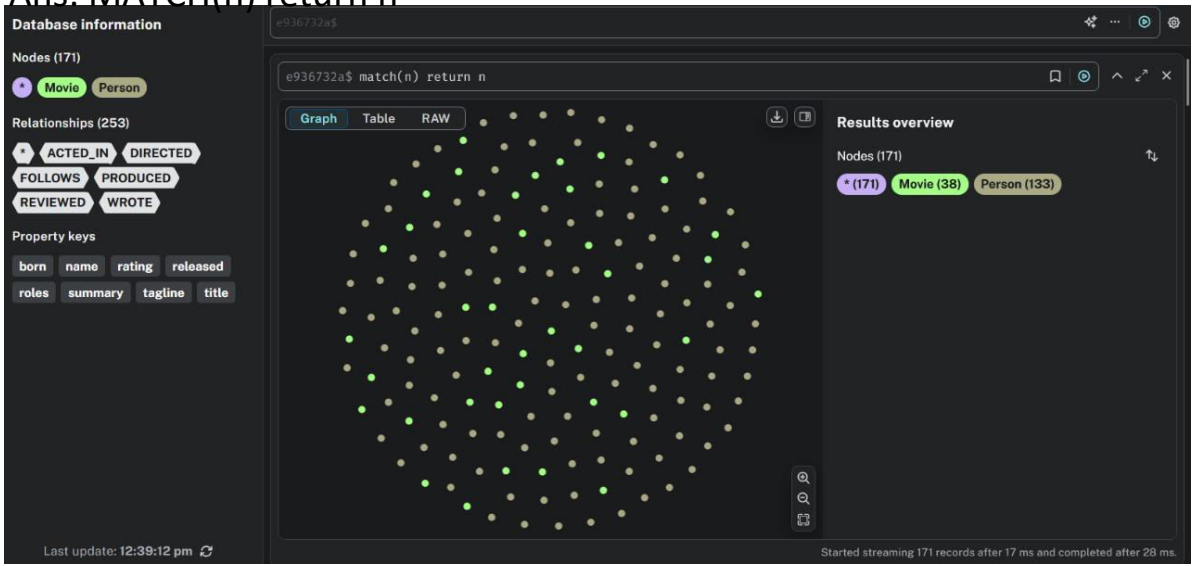
Database Management Systems

Hands on Session

Commands:

1) Write a Cypher query to return all nodes in the database.

Ans: MATCH(n) return n



Database Management Systems

Hands on Session

2) Write a query to return all Movie nodes.

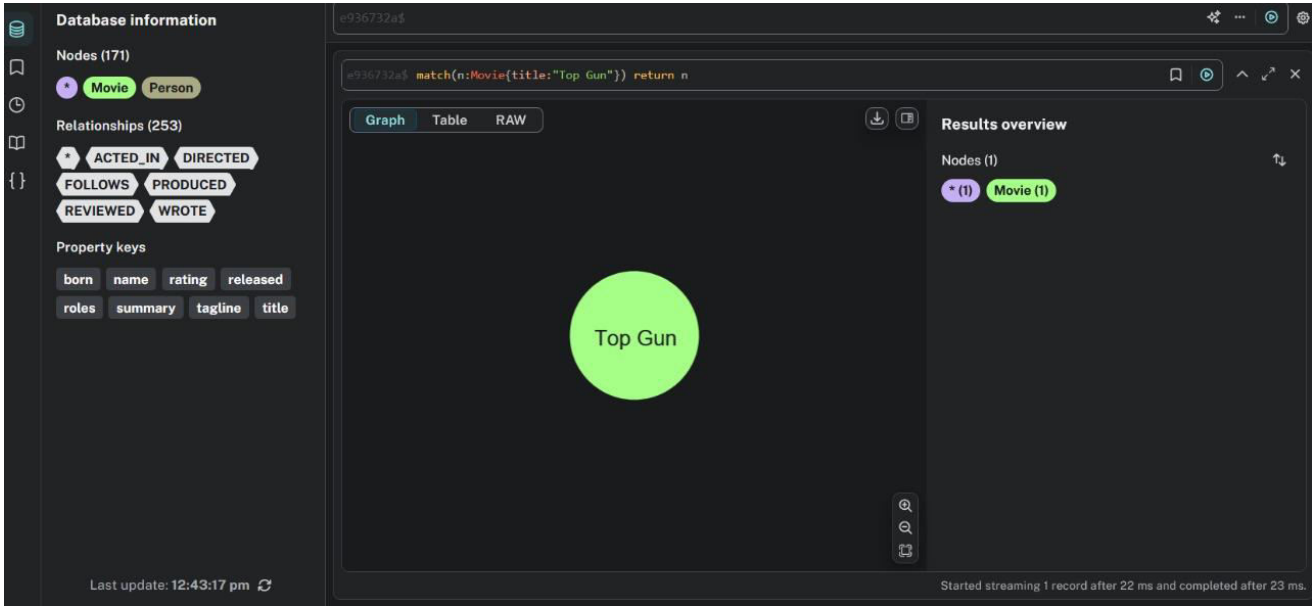
Ans: MATCH (n:Movie) RETURN n

Database Management Systems

Hands on Session

3) Write a query to return the Movie node with the title "Top Gun" using a property match.

Ans: MATCH (n:Movie{title:'Top Gun'}) RETURN n



4) Write a query to return the Movie node with the title "Top Gun" using a WHERE clause.

Ans: MATCH (n:Movie) WHERE n.title = 'Top Gun' RETURN n

5) Write a query to find all movies released after the year 2000, and return their title

Ans:
```
MATCH (m:Movie)
WHERE m.released >= 2000
RETURN m.title
```

6) Write a query to return all people whose name starts with "An" or ends with "on".

Ans:
```
MATCH (person:Person)
WHERE person.name STARTS WITH 'An' OR person.name ENDS WITH 'on'
RETURN person
```

7) Write a query to return all actors who acted in the movie "The Green Mile", along with the movie.

Ans:
```
MATCH (actor:Person)-[r:ACTED_IN]->(movie:Movie)
WHERE movie.title = 'The Green Mile'
RETURN actor,r, movie
```

8) Write a query to return just the actors who acted in the movie "The Green Mile".

Ans:
```
MATCH (actor:Person)-[:ACTED_IN]->(movie:Movie)
WHERE movie.title = 'The Green Mile'
RETURN actor
```

9) Write a query to return the director(s) of "The Green Mile", along with the movie.

Ans:
```
MATCH (actor:Person)-[:DIRECTED]->(movie:Movie)
WHERE movie.title = 'The Green Mile'
RETURN actor, movie
```

10) Write a query to return all directors and the movies they directed.

Ans:MATCH (actor:Person)-[:DIRECTED]->(movie:Movie)
RETURN actor, movie

11) Write a query to return all reviewers who gave a movie a rating greater than 68, along with the movie.

Ans:MATCH (reviewer:Person)-[r:REVIEWED]->(movie:Movie)
WHERE r.rating > 68
RETURN reviewer, movie

12) Write a query to find all movies written by Cameron Crowe, and return the writer, the movie, and all actors in those movies.

Ans:MATCH (writer:Person{name:'Cameron Crowe'})-[:WROTE]->(m1:Movie)
MATCH (actor:Person)-[:ACTED_IN]->(m1:Movie)
RETURN writer, m1, actor

13) Write a query to return each director's name and the number of movies they directed.

Ans:
```
MATCH (director:Person)-[:DIRECTED]->(m:Movie)
RETURN director.name, COUNT(m)
```

14) Write a query to return each reviewer's name and their average rating.

Ans:
```
MATCH (reviewer:Person)-[r:REVIEWED]->(m:Movie)
RETURN reviewer.name, AVG(r.rating)
```

15) Write a query to return each reviewer's name and their average rating, ordered by average rating.

Ans:
```
MATCH (reviewer:Person)-[r:REVIEWED]->(m:Movie)
RETURN reviewer.name, AVG(r.rating)
ORDER BY AVG(r.rating)
```

16) Write a Cypher command to create a Movie node titled "Movie1".

Ans: CREATE (n:MOVIE{title:"Movie1"}) RETURN n

17) Write a command to create a Movie node titled "Movie2" with a release year of 2001.

Ans: CREATE (n:MOVIE{title:"Movie2", released:2001}) RETURN n

18) Write a command to create a Movie node titled "Movie3".

Ans: CREATE (n:MOVIE{title:"Movie3"}) RETURN n

19) Write a command to create a Person node named "Person1".

Ans:CREATE (n:PERSON{name:"Person1"}) RETURN n

20) Write commands to create Person nodes named "Person2".

Ans: CREATE (n:PERSON{name:"Person2"}) RETURN n

21) Write commands to create Person nodes named "Person3".

Ans: CREATE (n:PERSON{name:"Person3"}) RETURN n

22) Write commands to create Person nodes named "Person4".

Ans: CREATE (n:PERSON{name:"Person4"}) RETURN n

23) Write a query to create an ACTED_IN relationship from Person1 to Movie1.

Ans:
```
MATCH (a:PERSON), (b:MOVIE)
WHERE a.name = "Person1" AND b.title = "Movie1"
CREATE (a)-[:ACTED_IN]->(b)
RETURN a,b
```

24) Write a query to create an ACTED_IN relationship from Person3 to Movie2.

Ans:
```
MATCH (a:PERSON), (b:MOVIE)
WHERE a.name = "Person3" AND b.title = "Movie2"
CREATE (a)-[:ACTED_IN]->(b)
RETURN a, b
```

25) Write a query to create an ACTED_IN relationship from Person4 to Movie2.

Ans:
```
MATCH (a:PERSON), (b:MOVIE)
WHERE a.name = "Person4" AND b.title = "Movie2"
CREATE (a)-[:ACTED_IN]->(b)
RETURN a, b
```

26) Write a query to update Person1 by setting their age to 54.

Ans:
```
MATCH (p:PERSON {name: "Person1"})SET p.age = 54 RETURN p
```

27) Write a query to update Person1 by adding the ACTOR label to them.

Ans:
```
MATCH (p:PERSON {name: "Person1"}) SET p:ACTOR
RETURN p
```

28) Write a query to delete the Movie node titled "Movie3".

Ans:
```
MATCH (m:MOVIE {title: "Movie3"})
DELETE m
```

29) Write a query to delete the Movie node titled "Movie2".

Ans:
```
MATCH (m:MOVIE {title: "Movie2"})
DELETE m
```

30) Write a query to detach delete the Movie node titled "Movie2" (delete the node and its relationships).

Ans:
```
MATCH (m:MOVIE {title: "Movie2"})
DETACH DELETE m
```

31) Write a query to return all nodes currently in the database.

Ans:
```
MATCH (n)
RETURN n
```

32) Write a query to detach delete all nodes in the database.

Ans:
```
MATCH (n)
DETACH DELETE n
```

THANK YOU

Prof. Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Introduction to Vector databases

Prof. Nivedita Kasturi

Department of Computer Science and Engineering

Database Management Systems

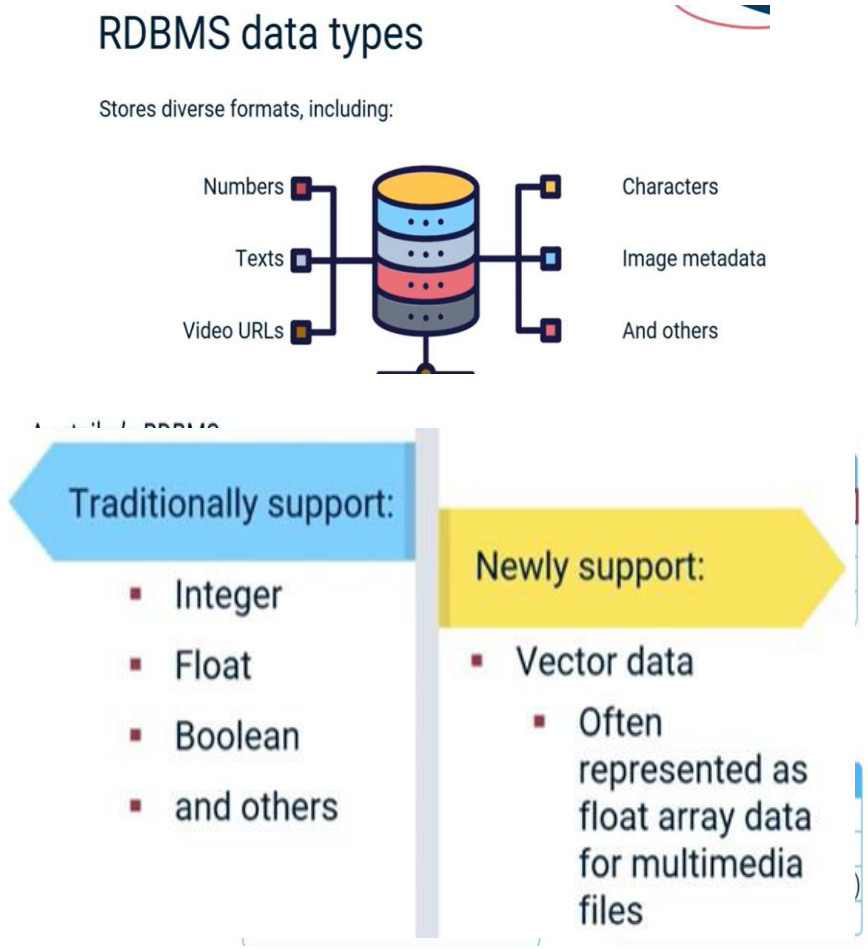
Introduction to Vector databases

- Relational database stores data of type numbers, text, video url, image meta data and BLOB data like images , video. CLOB data like large characters like Pdf or word docs.

- Data is stored as digital form in storage systems and DBMS facilitates the Access, Modification and updates to databases.

- DBMS often manages database as files. SQL databases traditionally support primitive data types like integer, float, Boolean, and other data types. *Many data platforms are evolving to support more complex data types such as vectors to accommodate emerging needs like machine learning applications.*

- Vectors represent float array data from multimedia files, often provided as an extension rather than a default data type



Introduction to Vector databases

- To understand **why we are moving to vector databases or RDBMS with vector search options.** We need to know about full text search :

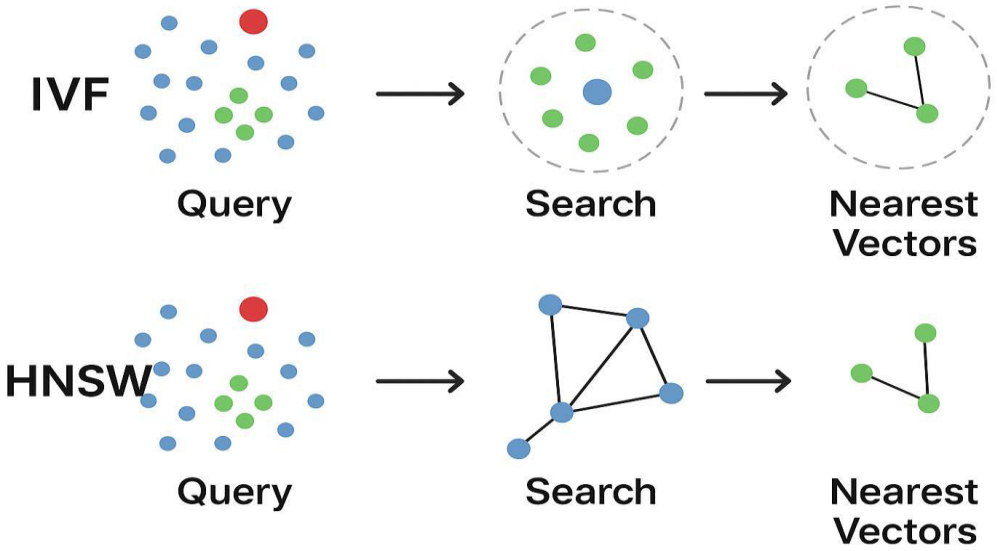
- Full-text search (FTS) **is a feature that allows users to efficiently search for words or phrases within large textual columns (like articles, descriptions, documents) stored in a database**. Unlike simple string pattern matching (using LIKE or regular expressions), **full-text search in a DBMS is designed to handle large volumes** of text data and to provide meaningful, **relevance-ranked results**.

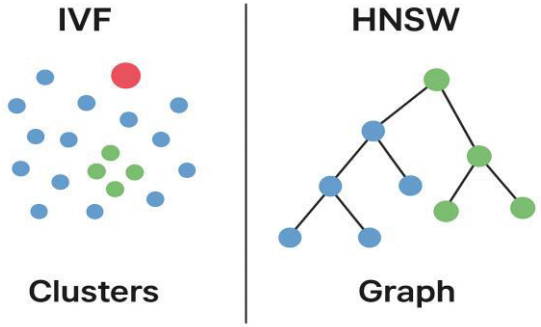
- **Vector search** is better than traditional full-text search in DBMS because it retrieves results based on **semantic meaning**, **not just matching exact words or patterns.** It uses **high-dimensional embeddings** to find conceptually ***similar content, naturally handling synonyms, context, and user intent.*** *This makes it far more effective for modern applications like recommendation systems, question answering, and multilingual search.*

Database Management Systems

Introduction to Vector databases

Full-Text Search (FTS) in DBMS	Vector Databases (Semantic Search)
Finds documents by matching exact words or stems.	Finds documents by matching meaning, using vector similarity.
Works on tokens (lexemes): e.g., "run" = "running".	Works on embeddings (dense vectors) that capture context & meaning.
Uses inverted index: maps words → document IDs.	Uses vector index (like HNSW, IVF): clusters & searches by distance.
Does not understand synonyms or context unless explicitly added.	Naturally handles synonyms & context (e.g., "doctor" ≈ "physician").
Ranks by term frequency (TF-IDF, BM25).	Ranks by vector distance (cosine similarity, Euclidean).
Limited for unstructured data (images, audio, multimodal).	Works for text, images, audio — anything convertible to vectors.





An **inverted index** maps **content (like words or tokens)** to the **documents** (or IDs) that contain them. It's called *"inverted"* because it reverses the normal structure:

- Instead of *document → words*,
- It stores *word → list of documents*.

Word	Document IDs
"apple"	1, 3, 7
"banana"	2, 3
"fruit"	1, 2, 3, 4

Database Management Systems

Introduction to Vector databases

- **Definition:**

Specialized system for handling **high-dimensional vector data**
Efficient at **indexing, querying, and retrieving** such data
Enables advanced analysis & **similarity searches** beyond traditional databases

- **Example (Analogy):**

Like a **super-smart librarian** who knows every book & how they're connected by themes
Makes it easy to find books on similar topics, not just exact matches

- **How It Works:**

Organizes vast data points by **similarity or context**
Finds information based on **semantic relevance** rather than exact criteria
E.g., helps discover articles similar to a specific one

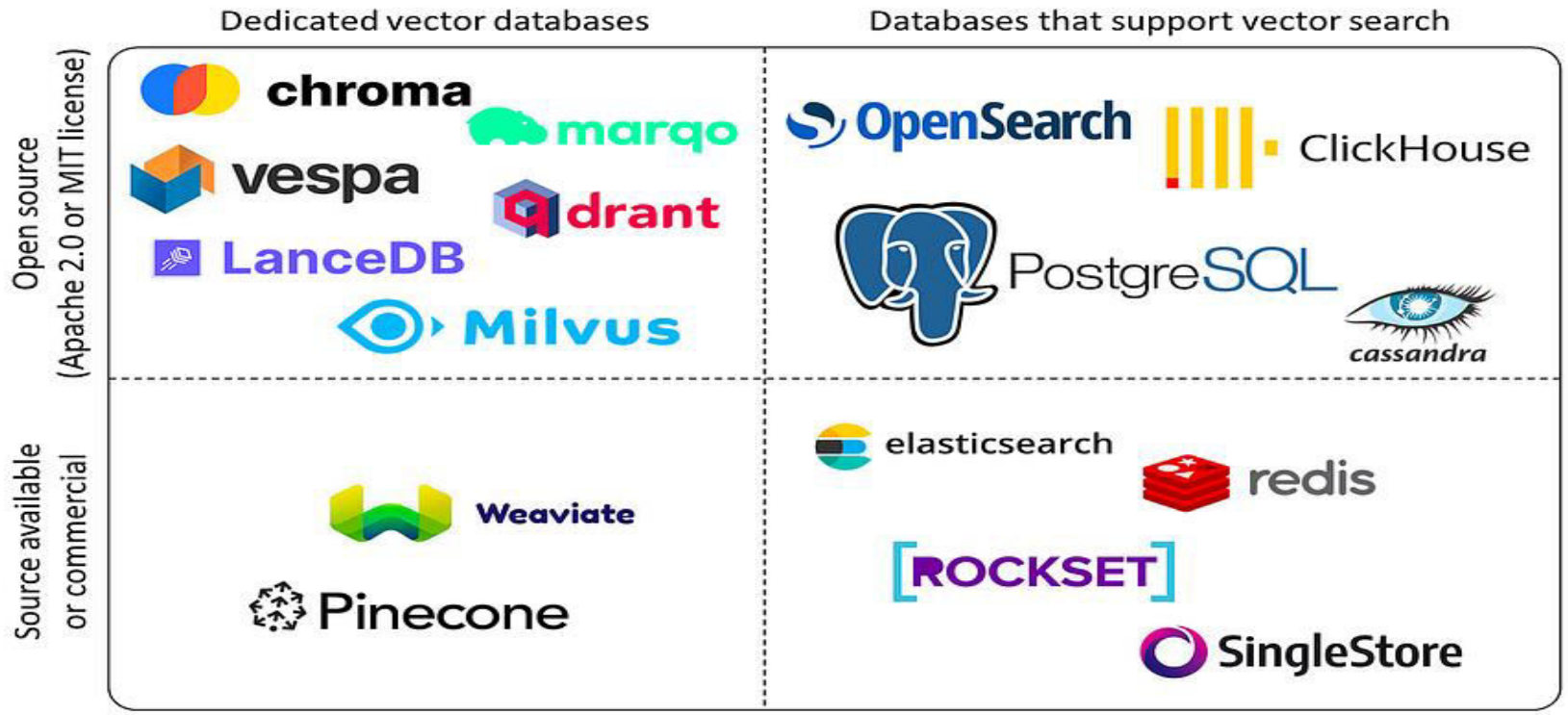
- **Integration with AI:**

Can be combined with **LLMs** to build **Retrieval Augmented Generation (RAG)** systems
Powers advanced search & content generation use cases

Many companies are continually raising money to develop their vector databases or ***to add vector search capabilities to their existing SQL or NoSQL databases.***

Introduction to Vector databases

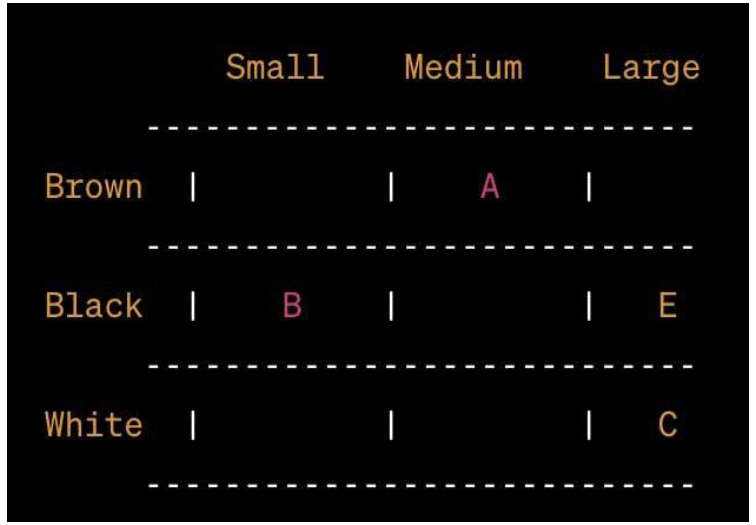
Many companies are continually raising money to develop their vector databases or ***to add vector search capabilities to their existing SQL or NoSQL databases.*** Today, there are a number of different vector databases available like chroma, weaviate, etc.

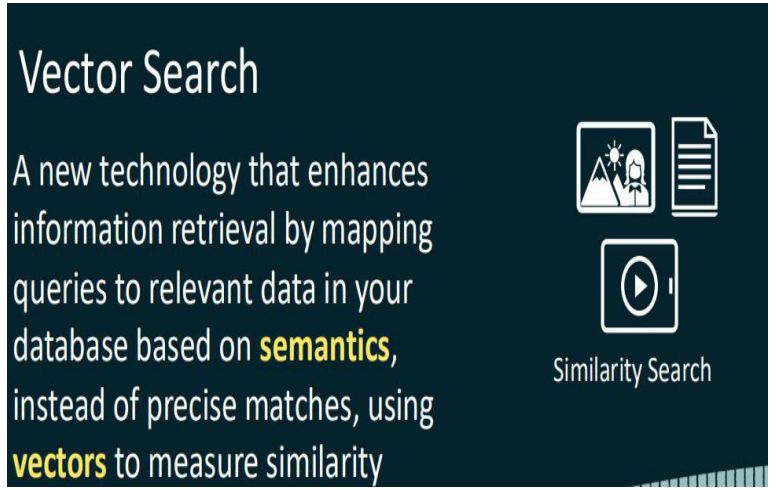


Database Management Systems

Vector Databases

- Vector databases have gained significant importance in various fields *due to their unique ability to efficiently store, index, and search high-dimensional data points,* often referred to as vectors.
- These databases are designed to handle **data** where each entry is **represented** as a **vector in a multi-dimensional space**.
- The *vectors can represent a wide range of information, such as **numerical features**, **embeddings from text or images**, and even complex data like **molecular structures***.
- Let's represent the vector database using a 2D grid where one axis represents the color of the animal (brown, black, white) and the other axis represents the size (small, medium, large).





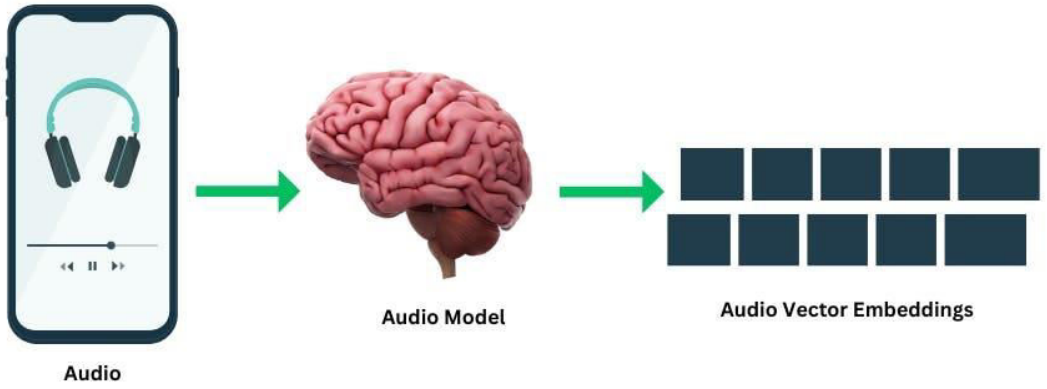
Vector Databases

Example:

- Like a big box of colorful crayons — each crayon is different.
- A vector database acts like a magical sorting machine.
- When you show it your favorite blue crayon, it quickly finds other crayons closest in color.
- Saves time by finding similar items without searching through the whole box!

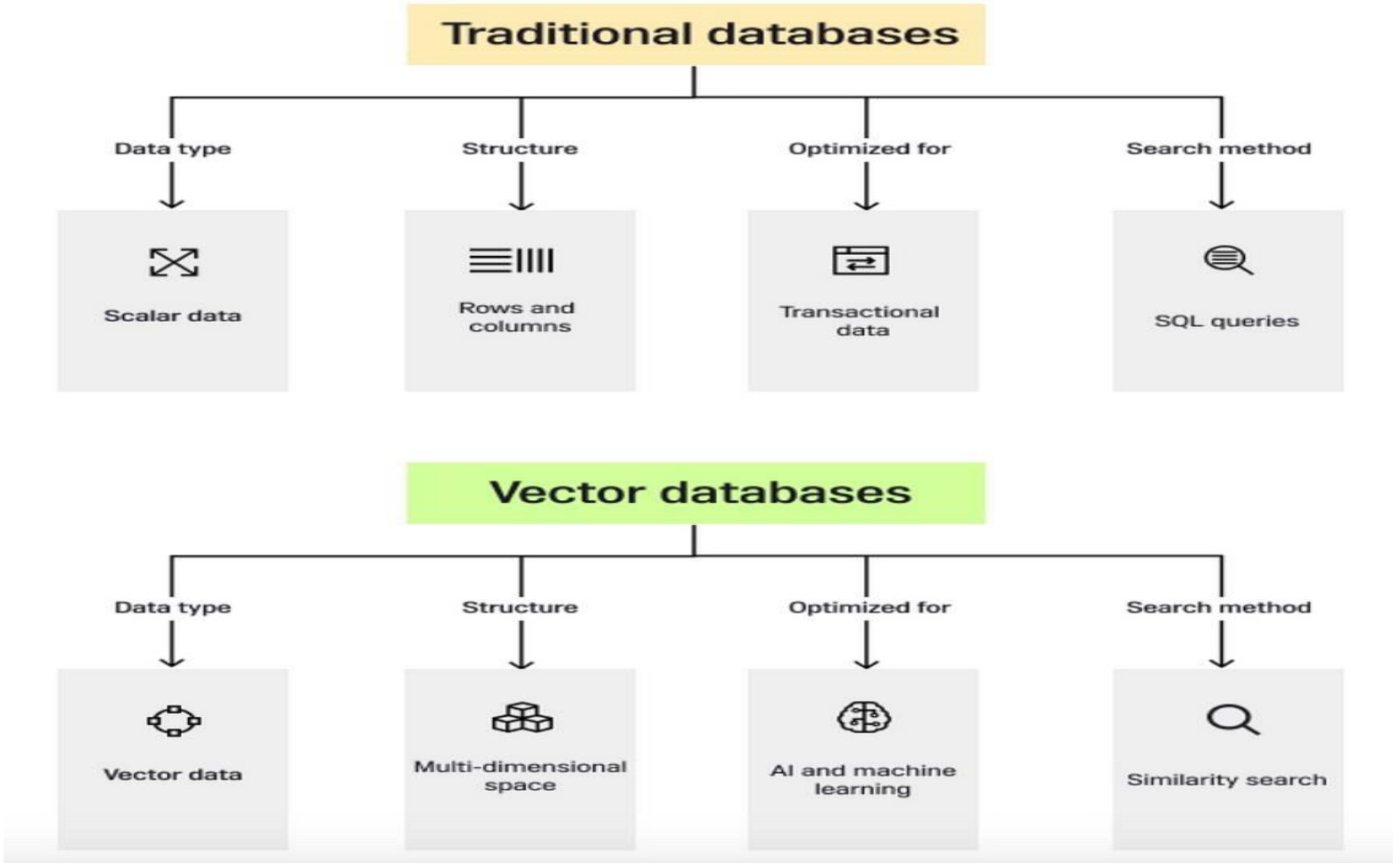
How Do Vector Databases Store Data?

- Store data as **high-dimensional vector embeddings** capturing semantic meaning & relationships
- Use **specialized indexing** (hashing, graph-based) for fast similarity searches
- Ideal for unstructured data like text, images, audio
- Handle semantically similar retrieval tasks efficiently
- **Designed to scale with AI workloads, despite being computationally intensive**
- Critical for modern data management, managing complex data relationships



Database Management Systems

How does a vector database differ from a traditional database?



Vector Database Capabilities

The significance of vector databases lies in their capabilities and applications:

- Efficient Similarity Search:

- Vector databases excel at performing similarity searches, where you can retrieve vectors that **are most similar to a given query vector**.

- High-Dimensional Data:

- Vector databases are designed to **handle high-dimensional data more efficiently**, making them suitable for applications like natural language processing, computer vision, and genomics.

- Machine Learning and AI:

- Vector databases are often **used to store embeddings generated by machine learning models**. These embeddings capture the essential features of the data and can be used for various tasks, such as clustering, classification, and anomaly detection.

Vector Database Capabilities

Real-time Applications:

Many vector databases are optimized for *real-time or near-real-time querying, making them suitable for applications that require quick responses,* such as recommendation systems in e-commerce, fraud detection, and monitoring IoT sensor data.

Personalization and User Profiling:

Vector databases enable personalized experiences by allowing systems to understand and predict user preferences. This is crucial in platforms like streaming services, social media, and online marketplaces.

Spatial and Geographic Data:

Vector databases can *handle geographic data, such as points, lines, and polygons, efficiently*. This is essential in applications like geographical information systems (GIS), location-based services, and navigation applications.

Vector Database Capabilities

Healthcare and Life Sciences:
In *genomics and molecular biology, vector databases are used to store* and analyze genetic sequences, protein structures, and other molecular data.

Data Fusion and Integration:
Vector databases can *integrate data from various sources and types, enabling more comprehensive analysis and insights.* This is valuable in scenarios where data comes from multiple modalities, such as combining text, image, and numerical data.

Multilingual Search:
Vector databases can be used to *create powerful multilingual search engines by representing text documents as vectors in a common space*, enabling cross-lingual similarity searches.

Database Management Systems

Traditional Database's VS Vector Databases

Distinguishing Factor	Traditional Database	Vector Database
Data Model	Structured, relational tables	Unstructured, high-dimensional vectors
Data Storage	Rows and columns	Vectors representing data points
Querying	SQL queries based on relationships	Similarity search based on distance metrics
Relationships	Explicitly defined relationships	Implicit relationships based on similarity
Indexing	Indexes on specific columns	Indexing based on vector space
Scalability	Limited to structured data	Efficient for large-scale, high-dimensional data
Use Cases	CRUD operations, data analysis	Generative AI, image/text retrieval, recommendation systems

Which of the following best describes a primary advantage of vector databases over traditional relational databases?

A) They store structured tabular data more efficiently
B) They allow faster transaction processing for numeric data
C) They enable similarity searches based on semantic meaning
D) They use B-tree indexes for optimizing join operations

Answer: C

Explanation: Vector databases are designed to perform efficient similarity searches based on high-dimensional vector embeddings that capture semantic relationships, unlike traditional relational databases which rely on structured tabular data and exact matches.

In the context of database systems, what does full-text search (FTS) primarily enable?
A) Retrieving documents by matching semantic similarity of content
B) Searching large text columns by tokenizing and indexing words
C) Efficient storage of high-dimensional vectors from machine learning models
D) Generating vector embeddings for text and images
Answer: B

Explanation: Full-text search in traditional DBMS breaks text into tokens (words), builds an index, and allows searching large text fields efficiently based on exact or normalized word matches.

Which of the following is a typical use case for vector databases?
A) Maintaining foreign key constraints across multiple tables
B) Performing multilingual similarity search across text documents
C) Running aggregate queries like SUM and COUNT on numeric fields
D) Implementing ACID transactions for bank account balances
Answer: B

Explanation: Vector databases excel at handling tasks like multilingual search, where text from different languages is represented in a shared vector space for similarity retrieval, a scenario beyond the scope of traditional relational operations.

Database Management Systems

References

https://medium.com/@rfajri912/introduction-to-vector-databases-c0a4a855765d

https://medium.com/data-and-beyond/vector-databases-a-beginners-guide-b050cbbe9ca0

https://qdrant.tech/articles/what-is-a-vector-database/

https://learn.microsoft.com/en-us/data-engineering/playbook/solutions/vector-database/

https://xomnia.com/post/an-introduction-to-vector-databases-for-beginners/

https://www.ibm.com/think/topics/vector-database

https://adasci.org/a-comprehensive-guide-to-vector-databases-and-their-utilities/

THANK YOU

Prof. Nivedita Kasturi

Department of Computer Science and Engineering

niveditak@pes.edu

Database Management Systems

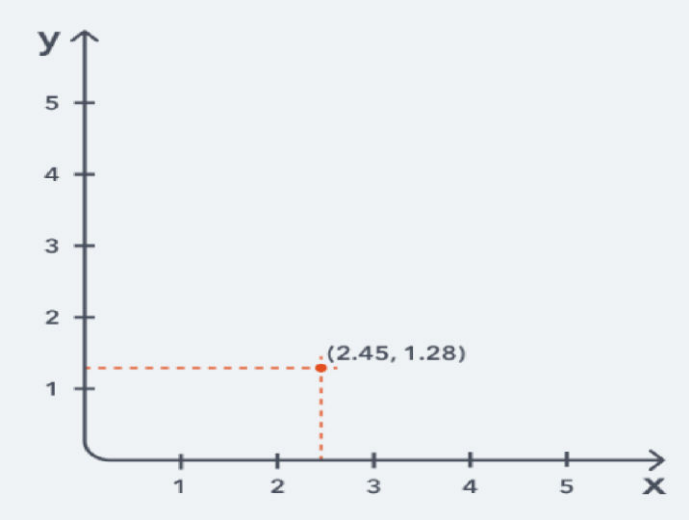
Vector Search with Relational Databases

Prof. Nivedita Kasturi

Department of Computer Science and Engineering

Understanding Vectors

- **A vector is a dynamic array** of numbers, and they are commonly used in math and physics to describe spatial coordinates.
- **For example,** the two-dimensional vector (2.45, 1.28) expresses the location of a point in two-dimensional space:



- **Vectors in Math & Physics:**

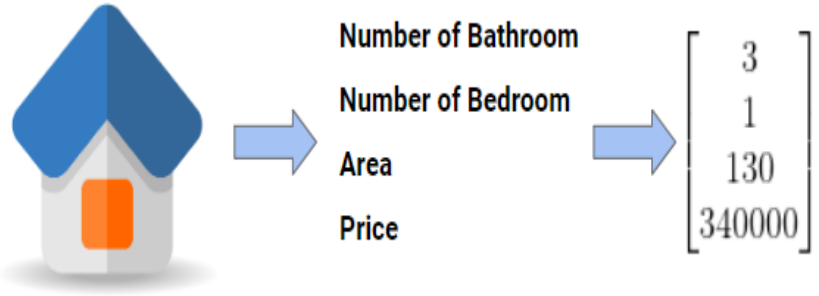
Physics: quantity with **magnitude & direction**

Math: geometric entity showing **magnitude & direction**

- **Simplified view:**

A vector = **list of attributes** of an object

E.g., a house: [bedrooms, bathrooms, area, price]



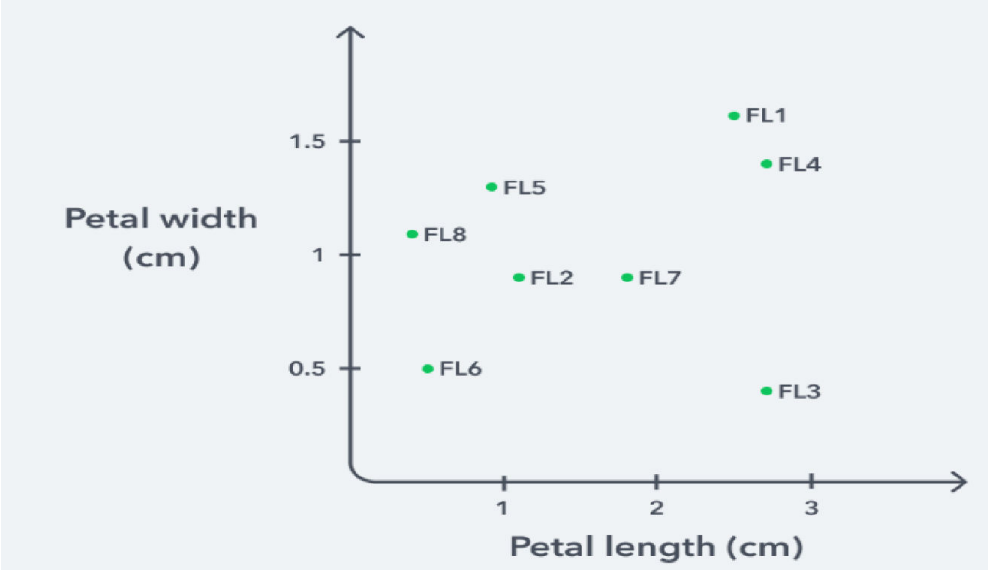
Database Management Systems

Understanding Vectors

- In *machine learning, vectors are still a dynamic array of numbers, but each number describes a feature* of a particular data point.
- For example, if you had a dataset about flowers that contained data about two different features (petal length and petal width), each flower could be represented as a two-dimensional vector:

ID	Petal length (in cm)	Petal width (in cm)	Vector
FL1	2.5	1.6	(2.5, 1.6)
FL2	1.1	0.9	(1.1, 0.9)
FL3	2.7	0.4	(2.7, 0.4)
FL4	2.7	1.4	(2.7, 1.4)
FL5	0.9	1.3	(0.9, 1.3)
FL6	0.5	0.5	(0.5, 0.5)
FL7	1.8	0.9	(1.8, 0.9)
FL8	0.4	1.1	(0.4, 1.1)

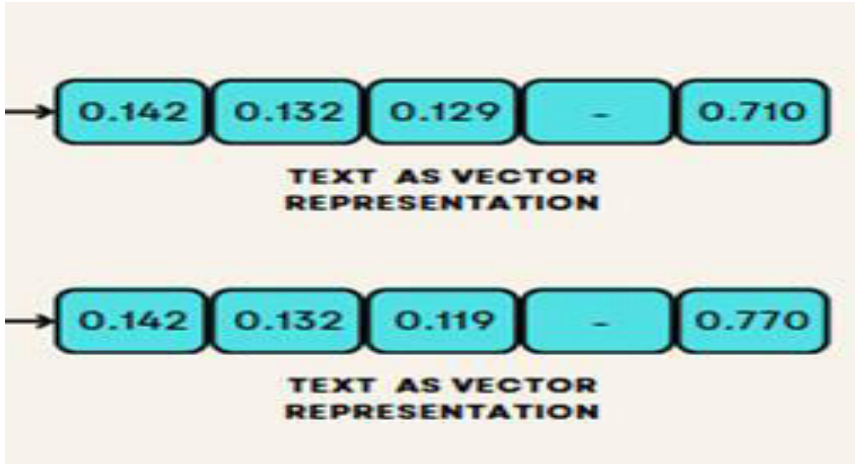
- Plotting these vectors on a graph allows us to see that the *data points for flowers FL1 and FL4 are the closest, which means they're the most similar to each other.*



Understanding Vector Databases

- A *single vector always corresponds to one particular object* — for example, a word, an image, a video, or a document.
- Once *an embedding model has generated some vectors*, ***they can be stored in a vector database, alongside the original content*** that they represent — such as a page URL, link to an image, or a product SKU(Stock Keeping Unit).
- Vectors are sometimes called ***vector embeddings*** — this just means a vector that was produced by an embedding model.

• **Vector Database:**
Stores **vectors** that describe objects or data points.
Defines relationships between these vectors.
• **Examples:**
Text: Vectors for words, paragraphs, documents (chatbots, search)
Images: Pixels as numbers → combined into high-dimensional vectors
Audio: Sound waves → numerical data → vectors (voice recognition)



Understanding Vector Databases

- **Example**, ***in a scenario where an ML application is tasked with recommending products*** based on previous purchases, it would need to determine which products are similar to each other.
- In the **Given example**, there are clusters for each of clothing, phones, and board games, but these clusters are located further apart as they are less similar to each other.
- *Within the clothing category itself, similar items are closer to each other.*
- For example, "black t-shirt" and "black polo shirt" are extremely close to each other (being the same color and shape), whereas "blue jeans" is further away.

Understanding Embedding

- ***Embedding models help us work out which objects are similar by converting very complex data (such as text, images, documents or videos) into vectors, which consist of numerical data.***
- Given that we need to *quantify* which objects are similar, it really helps to have ***numerical*** **data** for solving this problem.
- Converting data into vectors is a way of simplifying data, while still retaining the most important features of it.
- **W**ords are represented as vectors in a high-dimensional space, where each dimension of the space corresponds to a specific aspect or feature of the word's meaning.
- **For example**, dimensions might represent things like context, semantics, or syntax.
- **The procedure to generate the vectors from words is called embeddings**. (Text, documents, sentences, chat messages , Images , Audio, Videos. These data types are converted into vector embeddings)

Understanding Embedding

- Vector embeddings are **numerical interpretations that retain the contextual significance of data** , facilitating the alignment of similar entities within a vector space for similarity searches.
- *Vector embeddings numerically capture the semantic meaning of the objects in relation to other objects*. Thus, **similar objects are grouped together in the vector space**, which means the closer two objects are, the more similar they are.
- For example, **closely related concepts like "Boats" and "Ferries" may be positioned near each other due to their contextual**

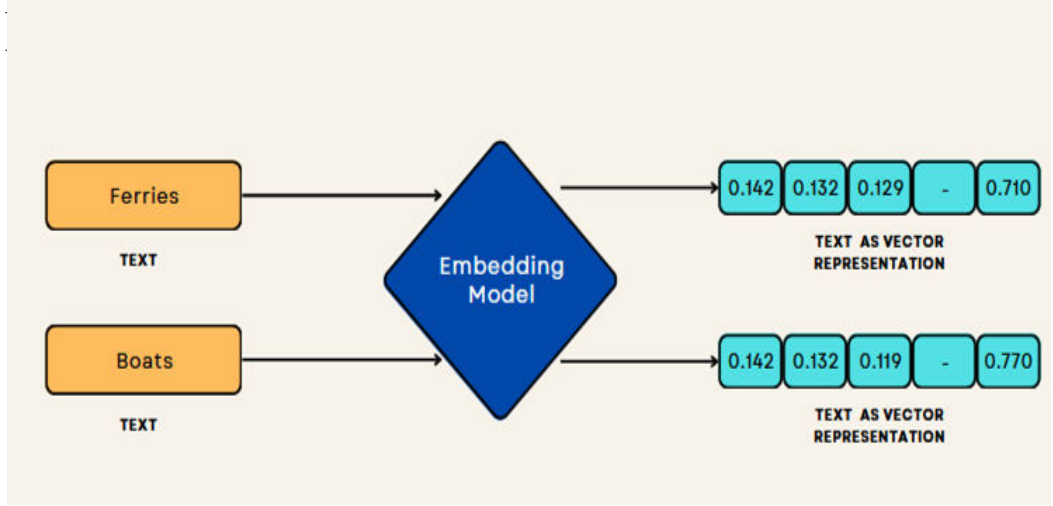


Figure 1. Text-to-Vector Transformation Using an Embedding Model

Understanding Embedding

- There are several **pre-trained embedding models available**, such as those from **OpenAI, Google, MetaAI, or the open-source community.**
- These *models learn from a lot of text to map words into a multi-dimensional vector space.*

OpenAI's Embedding Models	Sentence Transformer Embeddings	Other models and provider
e.g. • text-embedding-ada-002 • text-embedding-davinci-001 • text-embedding-curie-001 • text-embedding-babbage-001 • text-embedding-ada-001	e.g. • all-MiniLM-L6-v2 • all-MiniLM-L12-v1 • all-mpnet-base-v1 • all-roberta-large-v1	• Google Vertex AI • Google PaLM • Aleph Alpha • Elasticsearch • ...

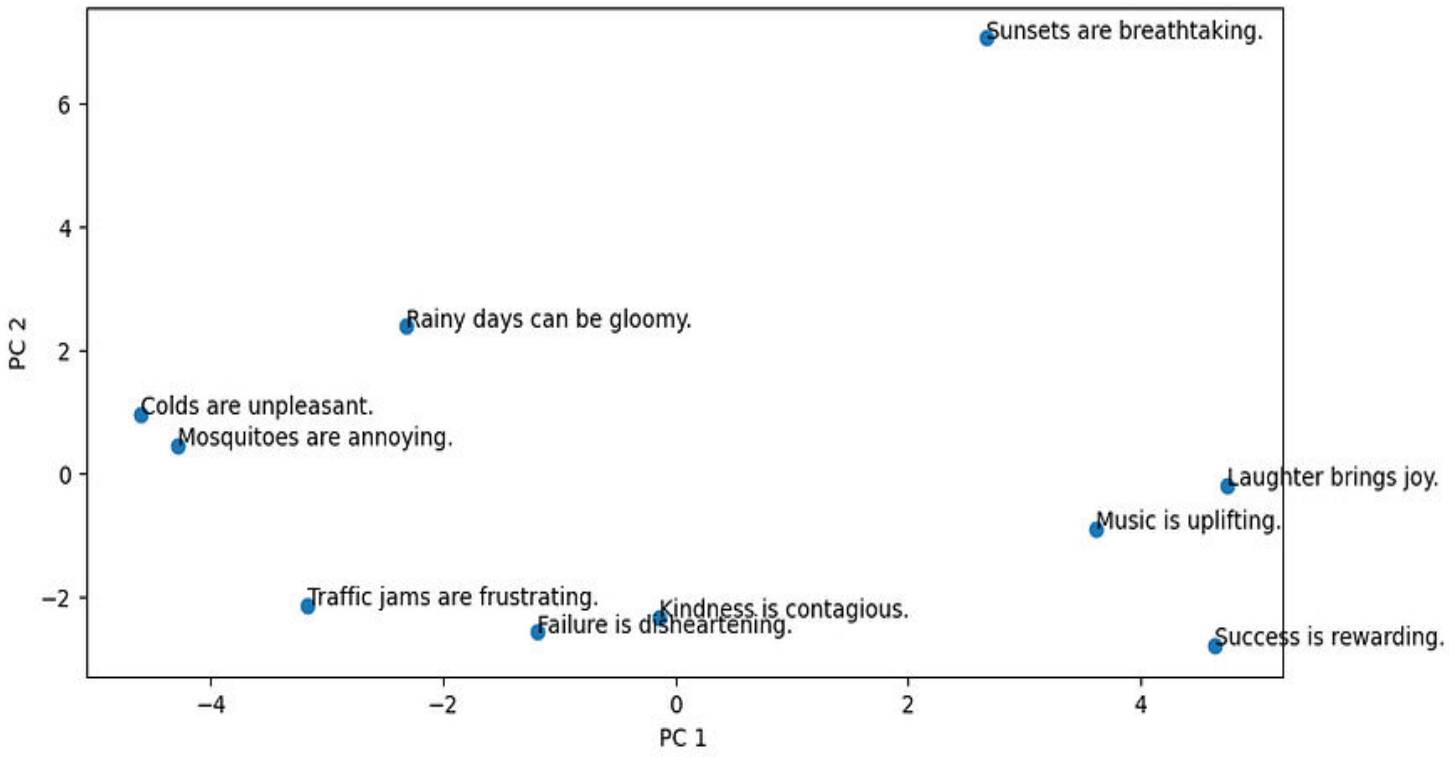
Database Management Systems

Understanding Embedding

we have 10 sample sentences like this:

```
text_chunks = [
"Sunsets are breathtaking.",
"Kindness is contagious.",
"Laughter brings joy.",
"Music is uplifting.",
"Success is rewarding.",
"Traffic jams are frustrating.",
"Rainy days can be gloomy.",
"Failure is disheartening.",
"Mosquitoes are annoying.",
"Colds are unpleasant."
]
```

Using **paraphrase-MiniLM-L6-v2**, a vector is generated and plotted as shown below.

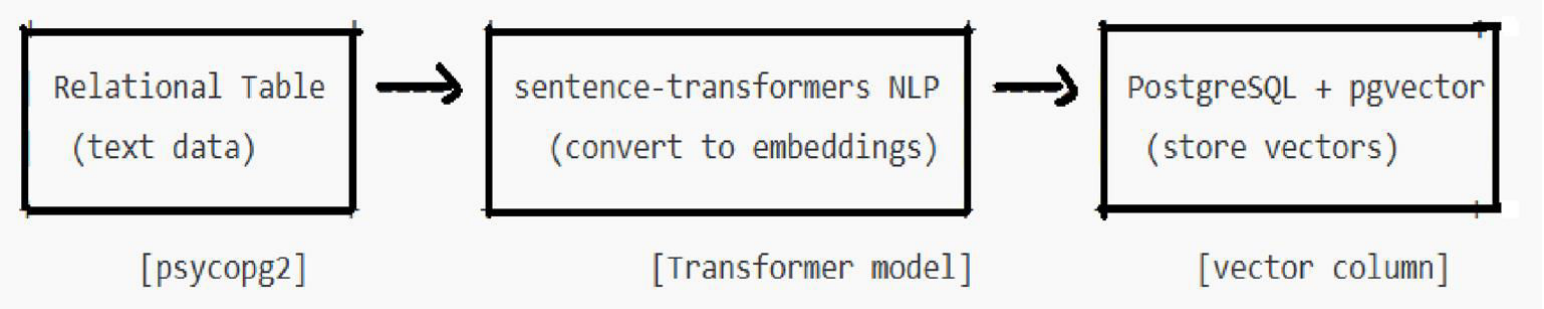


Understanding Embedding

- While the embeddings can be stored as numeric arrays, *searching the arrays stored in SQL databases can be memory-intensive and time-consuming.*
- Every time a search operation needs to be done, **all the vectors are retrieved**, **the cosine similarity is calculated**, and **the matching string is returned based on the value of cosine similarity or distance.**
- The vector length for image, video, and audio data tends to be much bigger than the text data.
- To address this, most RDBMS systems now provide a **wrapper or add-on on top of the existing RDBMS that allows having a vector as a data type.**
- And allows retrieving the data intrinsically based on the similarity between vectors rather than manually calculating the distance or cosine similarity.

Understanding Embedding

- **Embedding models (vectorizers):**Generate vectors from unstructured data (words, sentences, docs)
- **Examples:**
  - OpenAI: text-embedding-ada-002 (used by GPT models)
  - Google: embedding-gecko-001 (used by PaLM 2)
- Each model has its own technique for creating embeddings



Steps for generating embeddings

Database Management Systems

Understanding Embedding

Steps for generating Embedding for products relational DB :

Step 1: Extract Data from Relational Table

Connect to your database (e.g., PostgreSQL)

Select the text field to be embedded (e.g., product description)

Step 2: Generate Embeddings

Use a sentence embedding model (sentence-transformers)

Convert each row's text into a high-dimensional vector (embedding)

Step 3: Store Embeddings into DB

Use pgvector to add a vector column

Update each record with the generated vector

Step 4: Query Using Similarity

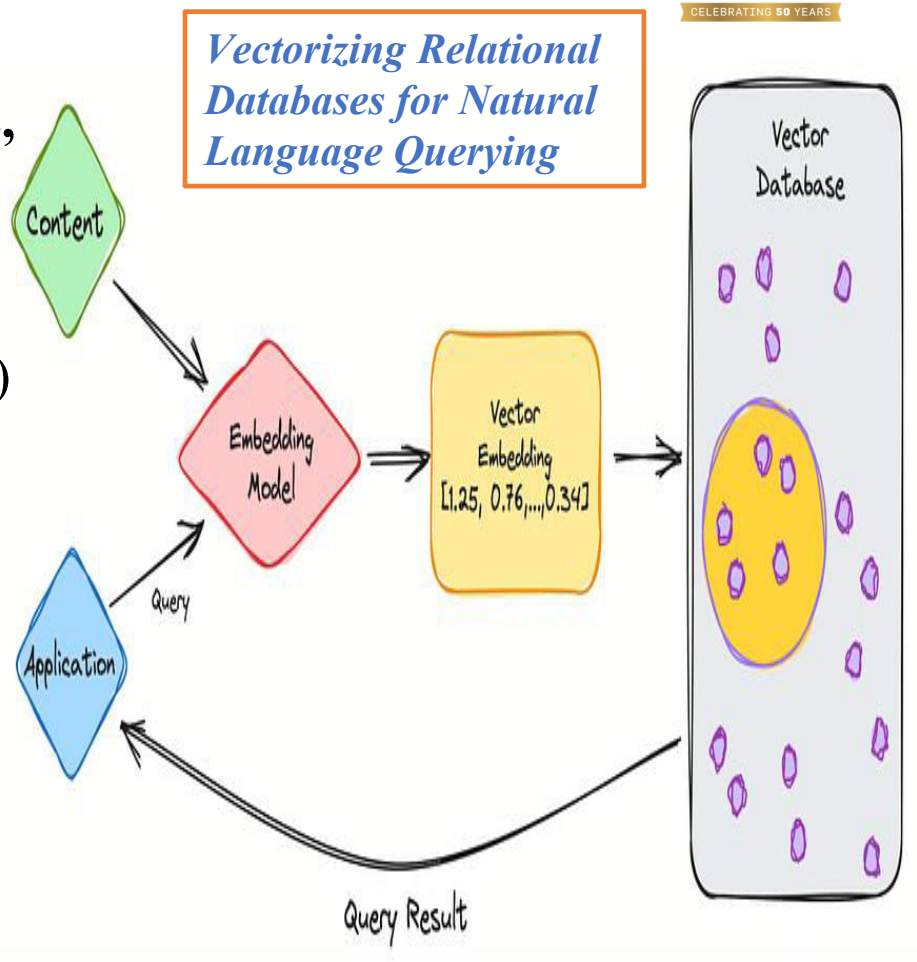
Use vector similarity operators (like <=>)

Find semantically similar rows (nearest neighbors)

Note: Refer to code file "**55_56 code for VectorSearch with RelationalDB**"

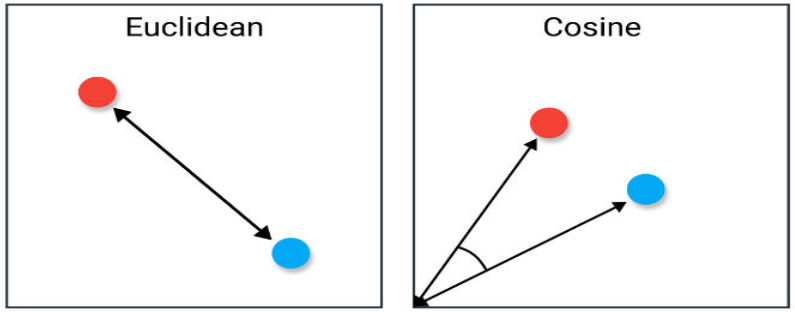
How Vector Databases Work with Embeddings

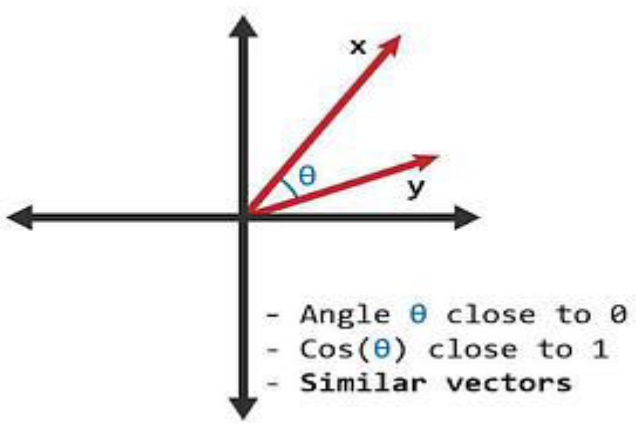
- **Data Processing:**
  - User query may involve raw data: **images, documents, videos, audio**
  - Data can be **structured or unstructured**
- **Embedding Generation:**
  - Processed through an **embedding model** (complex neural network)
  - Converts data into **high-dimensional vectors**
  - Encodes data's characteristics as **vector embeddings**
  - Stored in a **vector database**
- **Retrieval & Similarity Search:**
  - On query, vector DB performs **similarity searches**
  - Finds vectors **most similar** to the query vector
  - Delivers **relevant results quickly**, handling complex searches
- **Benefit:**
  - Enables rapid, accurate management of **vast & varied data types**
  - Ideal for applications needing **high-speed search & retrieval**

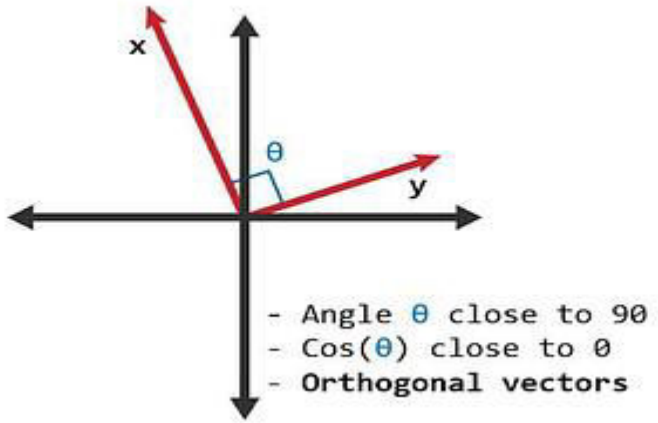


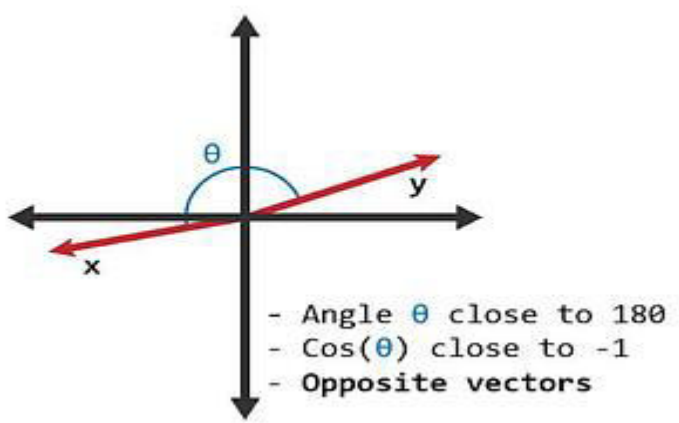
Similarity Score

- There are various ***metrics to measure the distance between vector or data points.***
- Two commonly used metrics are Cosine and Euclidean.
- In the cosine metric, **we determine similarity by calculating the cosine value of the angle (θ) between two vectors.** When the angle between two vectors is close to zero, the cosine value is close to 1. Conversely, if the angle is 90 degrees (referred to as orthogonal vectors in mathematics), the cosine value is zero.









Database Management Systems

Similarity Score

- We can use Cosine metric to calculate similarity between sentence.
  For example,
- let's consider a **new sentence**: "Rainy days make me sad".
- If we want to find a sentence with a similar meaning from our existing list of sentences, calculating the cosine for each sentence provides values like these.

	text_chunk	embeddings	cosine
0	Rainy days can be gloomy.	[0.118068404, 0.00039941736, 0.7828672, 0.7579...	0.764286
1	Colds are unpleasant.	[-0.6519384, 0.47603434, 0.3156743, 1.2116411,...	0.418937
2	Traffic jams are frustrating.	[0.06174645, -0.4527145, 0.46639186, -0.283001...	0.323204
3	Kindness is contagious.	[-0.44167116, -0.006431464, 0.20754915, 0.0754...	0.319614
4	Mosquitoes are annoying.	[0.095289715, 0.061180115, 0.31327882, 0.28330...	0.296069
5	Laughter brings joy.	[-0.10080257, 0.45041445, 0.4479737, -0.647958...	0.255170
6	Failure is disheartening.	[-0.3408494, -0.5745256, 0.75116944, 0.0891811...	0.239249
7	Music is uplifting.	[0.26783174, -0.005341845, 0.38134262, 0.09905...	0.189170
8	Sunsets are breathtaking.	[0.5539868, 0.4602928, 0.067907006, 0.46939665...	0.091070
9	Success is rewarding.	[-0.08308301, 0.31891197, -0.16729859, -0.5068...	0.033636

*'Rainy days can be gloomy' is the most similar sentence to our **new** one. Both convey negative feelings about rain. On the other hand, 'Success is rewarding' yields the smallest cosine value, which makes sense as it expresses a positive sentiment.*

- *That is precisely the objective with the vector database, to rapidly identify similar entries*

Cosine Value	Meaning
1	Sentences are **identical** in direction → very similar
0.8 – 1.0	**Highly similar** meaning
0.5 – 0.8	**Moderately similar**
0.0 – 0.5	**Low similarity**
0	**Completely unrelated**
-1	Opposite meaning (rare for sentence embeddings)

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

sentences = [
    "I love machine learning",
    "I enjoy studying AI"
]

# Convert sentences to TF-IDF vectors
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(sentences)

# Compute cosine similarity
similarity = cosine_similarity(tfidf_matrix[0:1], tfidf_matrix[1:2])
print("Cosine Similarity:", similarity[0][0])
```

Out put:
Cosine Similarity: 0.87

Vector Search in Vector Databases

- Vector search compares **similarity between vectors**
- Each vector = an object or data point
- Retrieves objects **similar to a query vector** by analyzing **semantic meaning**

Distance Metrics:

- Used to **quantify similarity** between vectors

Common metrics:

  - **Euclidean distance**
  - **Cosine similarity**

- Helps find nearest, most relevant results

Vector Indexing:

- Vector database **used several Approximate Nearest Neighbor algorithms to speed up the similarity search process.** Organizes vectors to **speed up retrieval**, critical for large datasets
- Uses clustering & indexing algorithms (e.g., **HNSW**)
- Enables **fast, scalable similarity searches**

Vector Search in Vector Databases

PES UNIVERSITY
CELEBRATING 50 YEARS

Popular Algorithms for clustering and Indexing

- **KNN (K-Nearest Neighbors):**
  - Finds exact nearest neighbors based on distance
  - Good for small to medium datasets
- **ANN (Approximate Nearest Neighbors):**
  - Finds approximate nearest neighbors
  - Improves efficiency for large, high-dimensional data
- **HNSW (Hierarchical Navigable Small World):**
  - Builds hierarchical graph structure during indexing
  - Enables fast neighbor queries, balancing **accuracy & speed**
  - Used in vector DBs like **Azure AI Search** for ANN searches

Q&A

Which of the following is used to measure similarity between vectors in vector search?
a) Primary key indexing
b) Cosine similarity
c) Sorting by timestamps
d) Hashing
Answer: b) Cosine similarity

Which indexing method combines inverted files with vector compression to achieve high-accuracy searches on massive datasets?
a) Flat index
b) IVFPQ
c) Vantage-Point Tree
d) HNSW
Answer: b) IVFPQ

What is a primary benefit of using an HNSW index in vector search?
a) It guarantees exact nearest neighbors with exhaustive search
b) It compresses data using product quantization
c) It builds a hierarchical graph for fast & memory-efficient search
d) It only works for low-dimensional data
Answer: c) It builds a hierarchical graph for fast & memory-efficient search

Database Management Systems

References

https://www.cs.cmu.edu/~15721-f24/slides/17-OracleAIVectorSearch.pdf

https://medium.com/data-and-beyond/vector-databases-a-beginners-guide-b050cbbe9ca0

https://www.youtube.com/watch?v=YPppSOk7yI4&t=1s **(explanation)**

https://www.oreilly.com/library/view/vector-databases/9781098177584/ch01.html

https://learn.microsoft.com/en-us/sql/relational-databases/vectors/vectors-sql-server?view=sql-server-ver17

THANK YOU

Prof. Nivedita Kasturi

Department of Computer Science and Engineering

niveditak@pes.edu

Database Management Systems

Vector Database Hands On (PostgreSQL)

Prof. Mahitha G
Department of Computer Science and Engineering

Acknowledgments: Prof. Shilpa S,
Teaching Assistants

PES UNIVERSITY
CELEBRATING 50 YEARS

Case Study:

Movie Recommendation System Using PostgreSQL and Vector Embeddings

- You are building a PostgreSQL-backed movie recommendation system leveraging semantic vector search.
- Each movie is represented by a title, synopsis, and vector embeddings generated from natural language descriptions.
- We are going to perform full CRUD operations and semantic search to recommend similar movies based on a query description.
- Each CRUD operation has been divided into individual sections

PES UNIVERSITY
CELEBRATING 50 YEARS

STEP 1: Let's create the database

```
-- Create a database dbms_class
CREATE DATABASE dbms_class;

-- Check if the database is created
\list

-- Connect to that database (Enter your postgres password if prompted)
\c dbms_class
```

Database Management Systems

PostgreSQL Hands On

```
vedant@VedWinNAS:/mnt/c/Users/vedan/Downloads$ psql -U postgres -W
Password:
psql (16.10 (Ubuntu 16.10-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# \list
                                                  List of databases
   Name    |  Owner   | Encoding | Locale Provider | Collate |  Ctype  | ICU Locale | ICU Rules |   Access privileges
-----------+----------+----------+-----------------+---------+---------+------------+-----------+-----------------------
 postgres  | postgres | UTF8     | libc            | C.UTF-8 | C.UTF-8 |            |           |
 template0 | postgres | UTF8     | libc            | C.UTF-8 | C.UTF-8 |            |           | =c/postgres          +
           |          |          |                 |         |         |            |           | postgres=CTc/postgres
 template1 | postgres | UTF8     | libc            | C.UTF-8 | C.UTF-8 |            |           | =c/postgres          +
           |          |          |                 |         |         |            |           | postgres=CTc/postgres
(3 rows)

postgres=# CREATE DATABASE dbms_class;
CREATE DATABASE
postgres=# \list
                                                  List of databases
    Name    |  Owner   | Encoding | Locale Provider | Collate |  Ctype  | ICU Locale | ICU Rules |   Access privileges
------------+----------+----------+-----------------+---------+---------+------------+-----------+-----------------------
 dbms_class | postgres | UTF8     | libc            | C.UTF-8 | C.UTF-8 |            |           |
 postgres   | postgres | UTF8     | libc            | C.UTF-8 | C.UTF-8 |            |           |
 template0  | postgres | UTF8     | libc            | C.UTF-8 | C.UTF-8 |            |           | =c/postgres          +
            |          |          |                 |         |         |            |           | postgres=CTc/postgres
 template1  | postgres | UTF8     | libc            | C.UTF-8 | C.UTF-8 |            |           | =c/postgres          +
            |          |          |                 |         |         |            |           | postgres=CTc/postgres
(4 rows)

postgres=# \c dbms_class;
Password:
You are now connected to database "dbms_class" as user "postgres".
dbms_class=#
```

PES UNIVERSITY
CELEBRATING 50 YEARS

STEP 2: Let's a table called movies for our case study

```
-- Create vector extension
CREATE EXTENSION vector;

-- Create movies table
CREATE TABLE movies (
    id SERIAL PRIMARY KEY,
    title TEXT,
    synopsis TEXT,
    embedding VECTOR(8)
);

-- Create index for fast similarity search
CREATE INDEX ON movies USING hnsw (embedding vector_cosine_ops);
```

- We use vectors to store the embeddings.
- We create an index on movies which uses the hnsw (Hierarchical Navigable Small World) algorithm for fast semantic search
- **(embedding vector_cosine_ops):** Here the **column** is called **embedding** on which we are creating the index
- We use cosine similarity to perform the comparison (we can also use Euclidean distance using vector_l2_ops)

To view all the tables in the database we use the command: **\dt**

To view the table schema we use the command: **\d movies or \desc movies**

To view all rows of table we use the command: **SELECT * FROM movies;**

Database Management Systems

PostgreSQL Hands On

```
dbms_class=# create extension vector;
CREATE EXTENSION
dbms_class=# CREATE TABLE movies (
    id SERIAL PRIMARY KEY,
    title TEXT,
    synopsis TEXT,
    embedding VECTOR(8)
);
CREATE TABLE
dbms_class=# CREATE INDEX ON movies USING hnsw (embedding vector_cosine_ops);
CREATE INDEX
dbms_class=# \dt
         List of relations
 Schema |  Name  | Type  |  Owner
--------+--------+-------+----------
 public | movies | table | postgres
(1 row)

dbms_class=# \d movies
                                  Table "public.movies"
  Column   |   Type    | Collation | Nullable |                Default
-----------+-----------+-----------+----------+---------------------------------------
 id        | integer   |           | not null | nextval('movies_id_seq'::regclass)
 title     | text      |           |          |
 synopsis  | text      |           |          |
 embedding | vector(8) |           |          |
Indexes:
    "movies_pkey" PRIMARY KEY, btree (id)
    "movies_embedding_idx" hnsw (embedding vector_cosine_ops)

dbms_class=# select * from movies;
 id | title | synopsis | embedding
----+-------+----------+-----------
(0 rows)

dbms_class=#
```

PES UNIVERSITY
CELEBRATING 50 YEARS

STEP 3: Let's insert values into this table (Embeddings are null for now)

```
INSERT INTO movies (title, synopsis, embedding) VALUES
('Inception', 'A skilled thief leads a team into dreams to steal secrets.', NULL),
('The Matrix', 'A hacker discovers the reality is a simulated world.', NULL),
('Interstellar', 'Explorers travel through a wormhole in search of a new home.', NULL),
('The Social Network', 'The story of Facebook''s founding and rise to success.', NULL),
('Avatar', 'A marine on an alien planet learns to live among the natives.', NULL),
('The Martian', 'An astronaut struggles to survive alone on Mars.', NULL),
('Her', 'A man develops a relationship with an intelligent operating system.', NULL),
('Blade Runner 2049', 'A blade runner uncovers secrets that could change society.', NULL),
('Guardians of the Galaxy', 'A group of intergalactic criminals must save the universe.', NULL),
('Ex Machina', 'A programmer interacts with an AI to test its consciousness.', NULL);
```

Database Management Systems

PostgreSQL Hands On

PES UNIVERSITY
CELEBRATING 50 YEARS

```
dbms_class=# INSERT INTO movies (title, synopsis, embedding) VALUES
('Inception', 'A skilled thief leads a team into dreams to steal secrets.', NULL),
('The Matrix', 'A hacker discovers the reality is a simulated world.', NULL),
('Interstellar', 'Explorers travel through a wormhole in search of a new home.', NULL),
('The Social Network', 'The story of Facebook''s founding and rise to success.', NULL),
('Avatar', 'A marine on an alien planet learns to live among the natives.', NULL),
('The Martian', 'An astronaut struggles to survive alone on Mars.', NULL),
('Her', 'A man develops a relationship with an intelligent operating system.', NULL),
('Blade Runner 2049', 'A blade runner uncovers secrets that could change society.', NULL),
('Guardians of the Galaxy', 'A group of intergalactic criminals must save the universe.', NULL),
('Ex Machina', 'A programmer interacts with an AI to test its consciousness.', NULL);
INSERT 0 10
dbms_class=# select * from movies;
 id |          title          |                                synopsis                                 | embedding
----+-------------------------+-------------------------------------------------------------------------+-----------
  1 | Inception               | A skilled thief leads a team into dreams to steal secrets.              |
  2 | The Matrix              | A hacker discovers the reality is a simulated world.                    |
  3 | Interstellar            | Explorers travel through a wormhole in search of a new home.            |
  4 | The Social Network      | The story of Facebook's founding and rise to success.                   |
  5 | Avatar                  | A marine on an alien planet learns to live among the natives.           |
  6 | The Martian             | An astronaut struggles to survive alone on Mars.                        |
  7 | Her                     | A man develops a relationship with an intelligent operating system.     |
  8 | Blade Runner 2049       | A blade runner uncovers secrets that could change society.              |
  9 | Guardians of the Galaxy | A group of intergalactic criminals must save the universe.              |
 10 | Ex Machina              | A programmer interacts with an AI to test its consciousness.            |
(10 rows)

dbms_class=#
```

PES UNIVERSITY
CELEBRATING 50 YEARS

STEP 6: Let's check if we can connect to our database using python

- Now since we need to create embeddings for the movies, let's use the help of python for this operation.
- Please install the following packages to connect to PostgreSQL and use sentence transformers to generate the embeddings:

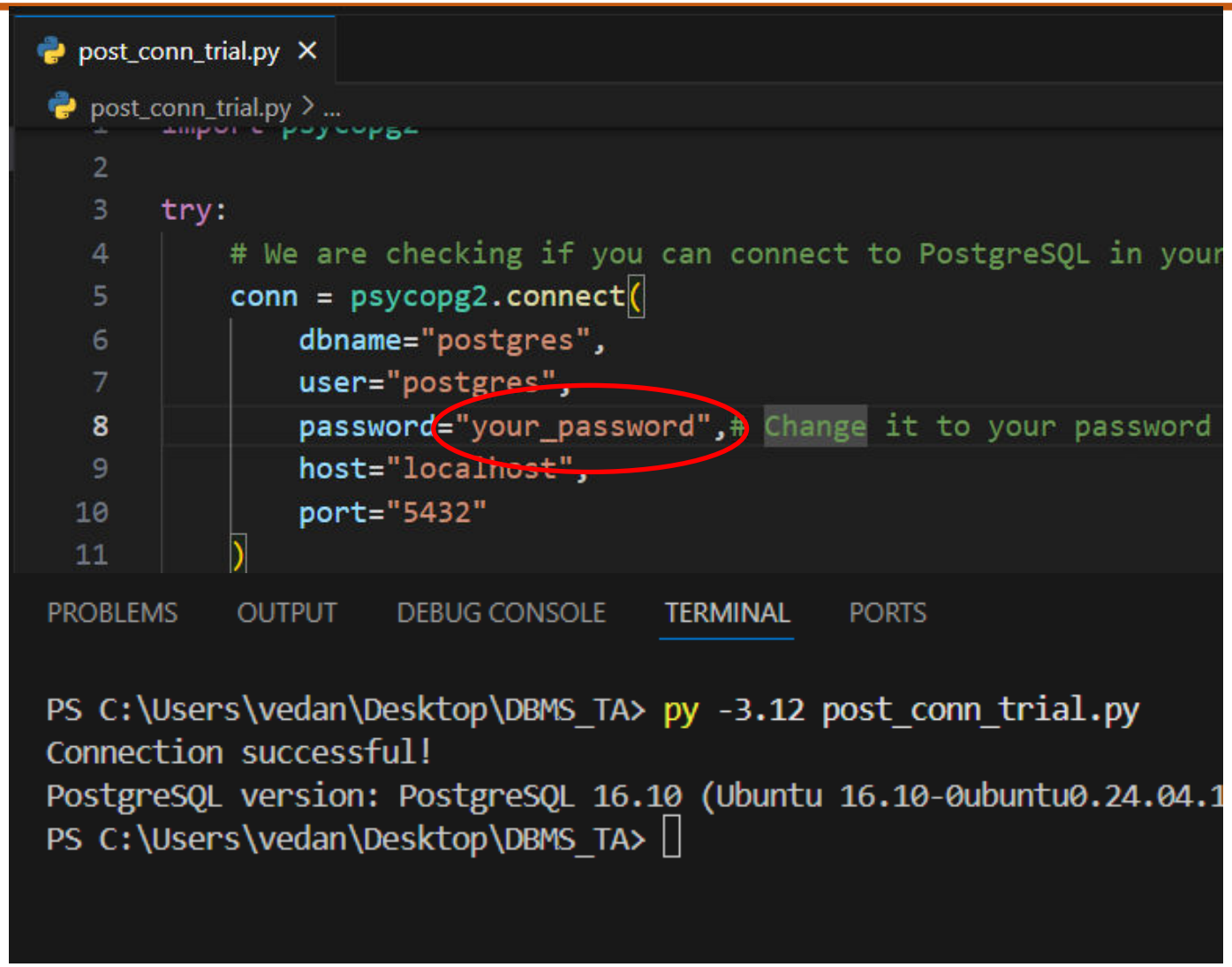
  pip install psycopg2

  pip install sentence-transformers

- Once this is done, let's first perform a test if we can connect to our PostgreSQL database
- For this please download the **post_conn_trial.py** file from the drive: Hands_On
- Make sure you **update the password** to your respective PostgreSQL password accordingly in the **post_conn_trial.py** file and then execute to check if connection is possible.

Database Management Systems

PostgreSQL Hands On



Please change your password in the python file: post_conn_trial.py

If you receive a message like this you can successfully start the demo !

PES UNIVERSITY
CELEBRATING 50 YEARS

STEP 7: Let's create embeddings for each movie and perform a semantic search

- For this please download the **post_hands_on.py** file from the drive: Hands_On
- Make sure you **update the password** to your respective PostgreSQL password accordingly in the **post_hands_on.py** python file
- Once you have updated your password, make sure that your database is running
- Execute the python file on windows and watch the beauty of semantic search unfold!
- The expected outputs are given after the explanation of how this process works

Database Management Systems

PostgreSQL Hands On

post_hands_on.py

```python
import psycopg2
from sentence_transformers import SentenceTransformer

# Connect to PostgreSQL
conn = psycopg2.connect(
    dbname="dbms_class",
    user="postgres",
    password="your_password",  # Replace with your actual password
    host="localhost",
    port="5432"
)
cur = conn.cursor()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\vedan\Desktop\DBMS_TA> python post_hands_on.py
```

Please change your password in the python file: post_hands_on.py

To execute the demo use this command

PES UNIVERSITY
CELEBRATING 50 YEARS

Explanation as to how this works:

- In the python code, we make use of the **all-MiniLM-L6-v2** model to generate our embeddings (you can use any transformer you like but this is what we chose for our demo)
- This model creates embeddings for all the movies. For the demo purposes the dimensions of the vectors are restricted to 8.
- This is done to keep it simple and noticeable but can change this to a much higher number as well (It can be as large as 384 and more as well!)
- Once we generate the embeddings we use the **psycopg2** (PostgreSQL connector) to show all the movies and the embeddings and this is updated in the database as well
- We then perform the similarity search based on our query using vector search (HSNW) and cosine similarity.

Explanation as to how this works:

- For this demo we have written a query to **fetch the top 3 movies** that follow this pattern: "Movies about technology“
- This gives us top 3 movies that are about technology and give us a similarity score.
- Since you created the index with vector_cosine_ops, <=> means cosine distance (not similarity).
- Range: 0.0 (perfect match) to 2.0 (worst)
- So smaller the value the better it is

PES UNIVERSITY
CELEBRATING 50 YEARS

Relevant code section in the post_hands_on.py file to help you understand better:

```
# We perform the similarity search based on our query using vector search (HSNW) and cosine similarity
user_query = "Movies about technology"
query_emb = model.encode(user_query)
emb_small = query_emb[:8]
query_emb_str = "[" + ",".join(map(str, emb_small)) + "]"

cur.execute("""
    SELECT id, title, synopsis, embedding <=> %s AS similarity
    FROM movies
    ORDER BY embedding <=> %s
    LIMIT 3;
""", (query_emb_str, query_emb_str))

results = cur.fetchall()
print(f"\nRecommended movies for: \"{user_query}\"")
for r in results:
    print(f"ID {r[0]} | {r[1]} ({r[3]:.4f}) - {r[2][:]}")
```

Database Management Systems

PostgreSQL Hands On

PES UNIVERSITY
CELEBRATING 50 YEARS

Expected Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\vedan\Desktop\DBMS_TA> py -3.12 post_hands_on.py
2025-10-12 19:19:33.708670: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different nu
t round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-10-12 19:19:39.584800: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different nu
t round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING:tensorflow:From C:\Users\vedan\AppData\Local\Programs\Python\Python312\Lib\site-packages\tf_keras\src\losses.py:2976: The
entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.

Embeddings updated for all movies.

All movies in DB:
ID: 1
        Title: Inception
        Synopsis: A skilled thief leads a team into dreams to steal secrets.
        Embedding: [[-0.04553012,0.060346086,-0.0064700013,0.002296256,-0.033018574,0.042285904,0.0557261,-0.023801899]]
ID: 3
        Title: Interstellar
        Synopsis: Explorers travel through a wormhole in search of a new home.
        Embedding: [[0.05328099,-0.05390857,0.039595485,0.06802813,-0.0169404,-0.08995386,0.030405652,-0.10104687]]
ID: 5
        Title: Avatar
        Synopsis: A marine on an alien planet learns to live among the natives.
        Embedding: [[0.0045827585,-0.014465234,-0.047157586,0.03512791,0.024452623,-0.056165986,0.0134151075,-0.050505262]]
```

Database Management Systems

PostgreSQL Hands On

PES UNIVERSITY
CELEBRATING 50 YEARS

```
ID: 6
        Title: The Martian
        Synopsis: An astronaut struggles to survive alone on Mars.
        Embedding: [[0.04819424,0.06826905,-0.018486457,0.069092065,0.02305683,-0.02402664,0.03772874,0.061583433]]
ID: 7
        Title: Her
        Synopsis: A man develops a relationship with an intelligent operating system.
        Embedding: [[-0.013241268,-0.0002439167,-0.025865622,-0.06283592,-0.077590264,-0.055736955,0.07529632,0.05184985]]
ID: 8
        Title: Blade Runner 2049
        Synopsis: A blade runner uncovers secrets that could change society.
        Embedding: [[-0.032635886,0.0064865253,-0.032821897,0.021678971,0.06779047,0.06279213,-0.06140622,-0.016323814]]
ID: 9
        Title: Guardians of the Galaxy
        Synopsis: A group of intergalactic criminals must save the universe.
        Embedding: [[-0.031359356,0.02772356,-0.004364626,-0.034722563,0.07041033,0.043281883,-0.013645645,-0.021742979]]
ID: 10
        Title: Ex Machina
        Synopsis: A programmer interacts with an AI to test its consciousness.
        Embedding: [[-0.012013591,0.0038372856,-0.060572784,-0.034697838,0.0058271913,-0.042998746,0.14287284,0.026329638]]
ID: 2
        Title: The Matrix
        Synopsis: A computer hacker learns the true nature of reality and fights to free humanity.
        Embedding: [[-0.06289084,0.031028645,-0.08162426,0.0039487635,-0.021426044,-0.020411924,0.037650283,-0.05743101]]
```

PES UNIVERSITY
CELEBRATING 50 YEARS

Expected Output:

We have created the embeddings and now perform similarity search based on our query using vector search (HSNW) and cosine similarity.

For this demo we have written a query to **fetch the top 3 movies** that follow this pattern: "Movies about technology“

```
Recommended movies for: "Movies about technology"
ID 2 | The Matrix (0.2726) - A computer hacker learns the true nature of reality and fights to free humanity.
ID 10 | Ex Machina (0.4468) - A programmer interacts with an AI to test its consciousness.
ID 1 | Inception (0.5127) - A skilled thief leads a team into dreams to steal secrets.

Done.
PS C:\Users\vedan\Desktop\DBMS_TA>
```

PES UNIVERSITY
CELEBRATING 50 YEARS

Expected Output:

To check if it has been reflected in our database let's use select command and check for each movie id what is the corresponding embedding.

```
-- Fetch embeddings for every movie id
SELECT id, embedding FROM movies;
```

```
dbms_class=# SELECT id, embedding FROM movies;
 id |                                                
```

THANK YOU

Prof. Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu

Database Management Systems

Vector Search with NoSQL

Prof. Mahitha G
Department of Computer Science and Engineering

Acknowledgments:Teaching Assistants

Introduction

- We have learnt before that NoSQL Databases are primarily used for horizontal **scalability** and can store various types of data.
- From the slides previously mentioned we also know vectors are used to represent high dimensional data.
- Vector search is an essential technique used to find **similar** items using the concept of vector embeddings.
- Now since we know that these **embedding** operations and search operations are computationally **intensive**, NoSQL databases, especially in-memory ones like **Redis**, are built for the **low-latency** reads/writes needed to serve **real-time** results.

Why use NoSQL for Vector Search ?

PES UNIVERSITY
CELEBRATING 50 YEARS

1. **Unified Data Model:**
   - NoSQL allows you to store the vector embedding alongside the operational metadata that gives it context (e.g., product price, user ID).
   - This **avoids** creating **separate databases** for vectors and business data, **simplifying** your architecture.
2. **Built for AI-Scale:**
   - AI applications generate vast amounts of data. Since NoSQL databases have the concept of a **"flexible schema",** it fits perfectly with storing data in AI applications like recommendation systems.
   - If you are building a RAG system or an LLM chatbot, **semantic caching** can also be used to reduce costs and increase performance
3. **Allow use of Hybrid Search Queries:**
   - Because the vectors are stored with the rest of your data, you can **combine** a semantic **vector search** with **traditional**, precise metadata **filtering** in a single query.

Use Cases and Examples

➢ Real World Use Cases:

- E-commerce Personalization
- Geospatial Data Analysis and Mapping
- Recommendation systems
- Biometrics and Anomaly Detection
- RAG, NLP, Image and Video Recognition and various other AI applications

➢ Database Examples:

- MongoDB Vector Search (via MongoDB Atlas)
- Redis (via Redis Stack)
- Azure Cosmos DB for NoSQL
- Aerospike Database

Using Redis NoSQL for Vector Search

Let's understand how Redis (NoSQL) is used to perform Vector Search:

- Vector search capabilities in Redis are provided by **Redis Stack**, which bundles Redis with powerful **modules** like **RediSearch.**
- **VECTOR,** A native, **binary-safe** data type is used to **perform** vector **comparisons**. This is much faster than storing vectors as text strings.
- Redis also allows you to **build indexes** that **combine VECTOR** fields with **other** data types like text, numbers, and tags. This is the key to enabling powerful hybrid searches.
- It supports different Approximate Nearest Neighbor (**ANN**) **algorithms** to let you choose the right balance between speed, memory usage, and accuracy for your specific needs.

in postgre vector emeddings we had to compare only vector embedding here we can do any datatype because redis store everything as binary safe , and due to its native VECTOR Datatype

Indexing Methods in Redis

Let's discuss the two most common indexing methods used in Redis

1. FLAT (Flat index):

- **How it works:** Brute-force search. Compares the query vector to every other vector.
- **Pros:** 100% accurate results (Exact Nearest Neighbor).
- **Cons:** Slow. O(N) complexity. Only for smaller datasets.

2. HNSW (Hierarchical Navigable Small World):

- **How it works:** Creates a multi-layered graph that allows for very fast traversal to find ANN. It starts at a high, sparse layer and navigates down to denser layers closer to the target.
- **Pros:** Extremely fast (logarithmic complexity), even with billions of vectors.
- **Cons:** It's approximate, but recall is typically over 99%. This is the standard choice for most production systems.

Vector Search in Redis

Vector Search in Redis is a 3-step Process

Step 1: Creating an Index Schema by defining the structure, the vector field and its algorithm.

```
1   FT.CREATE idx:user_prefs                    // index name is idx:user_prefs
2     ON JSON                                   // the type of data to be indexed -> JSON
3       PREFIX 1 user:                          // identifies the keys to be indexed
4     SCHEMA
5       $.descr TEXT                            // Allows full-text search queries
6       $.labels TAG SEPARATOR ","              // Allows exact-match queries, such as primary keys
7       $.vector_embedding VECTOR HNSW          // HNSW vector algorithm is being used
8         6                                     // Six parameters
9         TYPE FLOAT32                          // Each vector element is a 32-bit floating point
10        DIM 3                                 // Each vector has three dimensions
11        DISTANCE_METRIC COSINE                // Uses cosine similarity as metric
```

PES UNIVERSITY
CELEBRATING 50 YEARS

Step 1: Creating an Index Schema by defining the structure, the vector field and its algorithm.

- **FT.index** is used to **create** the vector index (**idx:user_prefs** is the index name)
- **JSON** data type is used and the **index will operate** on this data type (Can also use HSET)
- **PREFIX 1** means that **one prefix is specified**, and user means that **only keys starting with user:** will be indexed. Example: user:1 is indexed but **session:1** is **not** indexed
- **SCHEMA** defines the list of fields that Redis should index (Relate it to your columns in a table)
- Each **field** in the schema has the following things
  - **$.path** which creates the field name by prefixing $. (like your col name)
  - **Type** of the field
- **TAG** indicates that this field is for **exact matching** and does not tokenize words (Used for filtering)
- **VECTOR HNSW** is used to create the **vector** field using the **HSNW** indexing algorithm
- **TYPE FLOAT 32** makes it a 32 bit floating value
- **DIM 3** means the dimensions of the vector is 3
- **DISTANCE_METRIC COSINE** is used to ensure **cosine** similarity is used.

Vector Search in Redis

Vector Search in Redis is a 3-step Process

Step 2: Adding Data and Vectors

NOTE: The vector is passed as a binary blob, not a string.

```
1  JSON.SET user:1 $ '{"user": "samuel", "descr": "Samuel likes mountain and kid\'s bikes.", "labels":
   "mountain, kids", "vector_embedding": [0.9, 0.7, 0.2]}'
2
3  JSON.SET user:2 $ '{"user": "david", "descr": "David likes mountain and kid\'s bikes.", "labels":
   "mountain, kids", "vector_embedding": [0.7, 0.9, 0.1]}'
4
5  JSON.SET user:3 $ '{"user": "pieter", "descr": "Pieter likes kid\'s bikes.", "labels": "kids",
   "vector_embedding": [0.3, 0.9, 0.2]}'
6
7  JSON.SET user:4 $ '{"user": "morti", "descr": "Morti is into kid\'s and road bikes.", "labels":
   "kids, road", "vector_embedding": [0.1, 0.9, 0.7]}'
```

Step 2: Adding Data and Vectors

- Each command follows the pattern: **JSON.SET <key> <path> <JSON data>**
- Let's use the first command as an example:

```
JSON.SET user:1 $ '{"user": "samuel", "descr": "Samuel likes mountain and kid\'s bikes.", "labels":
"mountain, kids", "vector_embedding": [0.9, 0.7, 0.2]}'
```

- **Key:** user:1
- **Fields:**
  - **"user": "samuel"** => The user's name.
  - **"descr": "Samuel likes mountain and kid's bikes."** => Description for text-based search.
  - **"labels": "mountain, kids"** => Tags (to be used for exact matches via the TAG index).
  - **"vector_embedding": [0.9, 0.7, 0.2]** => 3-dimensional embedding vector.

Vector Search in Redis

Vector Search in Redis is a 3-step Process

Step 3: Performing the vector search (semantic search)

NOTE: The command searches for the top-2 most similar vectors to an input vector

```
FT.SEARCH idx:user_prefs "(*)=>[KNN 2 @vector_embedding $input_vector]"
PARAMS 2 input_vector <model_generated_vector> DIALECT 2
```

PES UNIVERSITY
CELEBRATING 50 YEARS

Step 3: Performing the vector search (semantic search)

- **FT.SEARCH** is the command from RediSearch module that is used to **perform** vector search
- **idx:user_prefs** is the name of the index created
- **(*) is called the wildcard search pattern.** This is used to **match all** the documents since we want to filter by similarity and not by precise text matching or tag matching
- **KNN 2** returns the 2 most similar documents to the query
- **@vector_embedding is the field name** in which we had created the index that stores vector embeddings
- **$input_vector** is the parameter **placeholder**. The data comes from the PARAMS command in the 2nd line
- **PARAMS 2** means that there are **2 parameters being passed** (name and value)
- **input_vector** is the **placeholder** name again
- We then pass the model generated embeddings instead of <> part
- **DIALECT 2** lets redis know that we are using **vector search in the RediSearch** module

References

References:

- https://mlops.community/vector-similarity-search-from-basics-to-production/
- https://risingwave.com/blog/redis-vector-database-unlocking-search-potential/
- https://redis.io/blog/vector-databases-101/
- https://redis.io/docs/latest/develop/ai/search-and-query/query/vector-search/
- https://www.weblineindia.com/blog/build-rag-with-langchain-redis-vector-search/
- https://vishnudeva.medium.com/redis-as-a-vector-database-rediscloud-2a444c478f3d

FOR BETTER UNDERSTANDING REFER THIS:
file:///E:/CollegeWork/PESU_SEM_5/DBMS/Unit4/pdf/Reference/redis_vector_search.pdf

THANK YOU

Prof. Mahitha G
Department of Computer Science and Engineering

mahithag@pes.edu